Speech and Language Processing

An Introduction to Natural Language Processing,
Computational Linguistics, and Speech Recognition
with Language Models

Third Edition draft

Daniel Jurafsky
Stanford University

James H. Martin
University of Colorado at Boulder

Draft of January 6, 2026. Comments and typos welcome!

Summary of Contents

Contents

Volume I

LARGE LANGUAGE MODELS

In the first part of the book we introduce the fundamental suite of algorithmic and linguistic tools that make up the modern neural large language model. We begin with tokenization and preprocessing, including Unicode, and then proceed to introduce many basic language modeling ideas using simple n-gram language models, we then introduce the algorithms which are the components of large language models: logistic regression, embeddings, and feedforward networks. Next we are ready to introduce the principles of large language modeling, encoder, decoders and pretraining, then the fundamental transformer architecture, then masked language model and other architectures like RNNs and LSTMs, information retrieval and retrieval-based algorithms like RAG, machine translation and the encoder-decoder model, and finally spoken language modeling including both ASR and TTS.

CHAPTER

1 Introduction

La dernière chose qu'on trouve en faisant un ouvrage est de savoir celle qu'il faut mettre la première.
[The last thing you figure out in writing a book is what to put first.]

Pascal

CHAPTER

2 Words and Tokens

```
User:  I need some help, that much seems certain.
ELIZA: WHAT WOULD IT MEAN TO YOU IF YOU GOT SOME HELP
User:  Perhaps I could learn to get along with my mother.
ELIZA: TELL ME MORE ABOUT YOUR FAMILY
User:  My mother takes care of me.
ELIZA: WHO ELSE IN YOU FAMILY TAKES CARE OF YOU
User:  My father.
ELIZA: YOUR FATHER
User:  You are like my father in some ways.
```

Weizenbaum (1966)

ELIZA

The dialogue above is from **ELIZA**, an early natural language processing system that could carry on a limited conversation with a user by imitating the responses of a Rogerian psychotherapist (Weizenbaum, 1966). ELIZA is a surprisingly simple program that uses pattern matching on words to recognize phrases like "I need X" and change the words into suitable outputs like "What would it mean to you if you got X?". ELIZA's mimicry of human conversation, while very crude by modern standards, was remarkably successful: many people who interacted with ELIZA came to believe that it really *understood* them. As a result, this work led researchers to first think about the impacts of chatbots on their users (Weizenbaum, 1976).

tokenization

Of course modern chatbots don't use the simple pattern-based mimicry that ELIZA pioneered. Yet the pattern-based approach to words instantiated in ELIZA is still relevant today in the context of **tokenization**, the task of separating out or **tokenizing** words and word parts from running text. Tokenization, the first step in modern NLP, includes pattern-based approaches that date back to ELIZA.

To understand tokenization we first need to ask: What is a word? Is *um* a word? What about *New York*? Is the nature of words similar across languages? Some languages, like Vietnamese or Cantonese, have very short words while others, like Turkish, have very long words. We also need to think about how to represent words in terms of **characters**. We'll introduce **Unicode**, the modern system for representing characters, and the **UTF-8** text encoding. And we'll introduce the **morpheme**, the meaningful subpart of words (like the morpheme `-er` in the word `longer`)

BPE

The standard way to tokenize text is to use the input characters to guide us. So once we understand the possible subparts of words, we'll introduce the standard **Byte-Pair Encoding** (**BPE**) algorithm that automatically breaks up input text into tokens. This algorithm uses simple statistics of letter sequences to induce a vocabulary of subword tokens. All tokenization systems also depend on **regular expressions** as a processing step. The regular expression is a language for formally specifying and manipulating text strings, an important tool in all modern NLP systems. We'll introduce regular expressions and show examples of their use

regular expressions

Finally, we'll introduce a metric called **edit distance** that measures how similar two words or strings are based on the number of edits (insertions, deletions, substitutions) it takes to change one string into the other. Edit distance plays a role in NLP whenever we need compare two words or strings, for example in the crucial **word error rate** metric for automatic speech recognition.

2.1 Words

How many words are in the following sentence?

```
They picnicked by the pool, then lay back on the grass and
looked at the stars.
```

This sentence has 16 words if we don't count punctuation as words, 18 if we count punctuation. Whether we treat period (“.”), comma (“,”), and so on as words depends on the task. Punctuation is critical for finding boundaries of things (commas, periods, colons) and for identifying some aspects of meaning (question marks, exclamation marks, quotation marks). Large language models generally count punctuation as separate words.

utterance

Spoken language introduces other complications with regard to defining words. What about this utterance from a spoken conversation? (**Utterance** is the technical linguistic term for the spoken correlate of a sentence).

```
I do uh main- mainly business data processing
```

disfluency **fragment** **filled pause**

This utterance has two kinds of **disfluencies**. The broken-off word *main-* is called a **fragment**. Words like *uh* and *um* are called **fillers** or **filled pauses**. Should we consider these to be words? Again, it depends on the application. If we are building a speech transcription system, we might want to eventually strip out the disfluencies. But we also sometimes keep disfluencies around. Disfluencies like *uh* or *um* are actually helpful in speech recognition in predicting the upcoming word, because they may signal that the speaker is restarting the clause or idea, and so for speech recognition they are treated as regular words. Because different people use different disfluencies they can also be a cue to speaker identification. In fact Clark and Fox Tree (2002) showed that *uh* and *um* have different meanings in English. What do you think they are?

word type **word instance**

Perhaps most important, in thinking about what is a word, we need to distinguish two ways of talking about words that will be useful throughout the book. Word **types** are the number of distinct words in a corpus; if the set of words in the vocabulary is V, the number of types is the **vocabulary size** $|V|$. Word **instances** are the total number N of running words.[1] If we ignore punctuation, the picnic sentence has 14 types and 16 instances:

```
They picnicked by the pool, then lay back on the grass and
looked at the stars.
```

We still have decisions to make! For example, should we consider a capitalized string (like *They*) and one that is uncapitalized (like *they*) to be the same word **type**? The answer is that it depends on the task! *They* and *they* might be lumped together as the same type in some tasks where we care less about the formatting, while for other tasks, capitalization is a useful feature and is retained. Sometimes we keep around two versions of a particular NLP model, one with capitalization and one without capitalization.

So far we have been talking about **orthographic words**: words based on our English writing system. But there are many other possible ways to define words. For example, while orthographically `I'm` is one word, grammatically it functions as two words: the subject pronoun `I` and the verb `'m`, short for `am`.

[1] In earlier tradition, and occasionally still, you might see word instances referred to as word *tokens*, but we now try to reserve the word *token* instead to mean the output of subword tokenization algorithms.

Corpus	Types = $\|V\|$	Instances = N
Shakespeare	31 thousand	884 thousand
Brown corpus	38 thousand	1 million
Switchboard telephone conversations	20 thousand	2.4 million
COCA	2 million	440 million
Google n-grams	13 million	1 trillion

Figure 2.1 Rough numbers of wordform types and instances for some English language corpora. The largest, the Google n-grams corpus, contains 13 million types, but this count only includes types appearing 40 or more times, so the true number would be much larger.

The distinctions get even harder to make once we start to think about other languages. For example the writing systems of languages like Chinese, Japanese, and Thai simply don't have orthographic words at all! That is, they don't use spaces to mark potential word-boundaries. In Chinese, for example, words are composed of characters (called **hanzi** in Chinese). Each character generally represents a single unit of meaning (called a **morpheme**, introduced below) and is pronounceable as a single syllable. Words are about 2.4 characters long on average. But since Chinese has no orthographic words, deciding what counts as a word in Chinese is complex. For example, consider the following sentence:

hanzi

(2.1) 姚明进入总决赛 yáo míng jìn rù zǒng jué sài
"Yao Ming reaches the finals"

As Chen et al. (2017b) point out, this could be treated as 3 words (a definition of words called the 'Chinese Treebank' definition, in which Chinese names (family name followed by personal names) are treated as a single word):

(2.2) 姚明 进入 总决赛
YaoMing reaches finals

But the same sentence could be treated as 5 words ('Peking University' standard), in which names are separated into their own units and some adjectives appear as distinct words:

(2.3) 姚 明 进入 总 决赛
Yao Ming reaches overall finals

Finally, it is possible in Chinese simply to ignore words altogether and use characters as the basic elements, treating the sentence as a series of 7 characters, which works pretty well for Chinese since characters are at a reasonable semantic level for most applications (Li et al., 2019):

(2.4) 姚 明 进 入 总 决 赛
Yao Ming enter enter overall decision game

But that method doesn't work for Japanese and Thai, where the individual character is too small a unit.

These issues with defining words makes it hard to use words as the basis for tokenizing text in NLP across languages.

But there's another problem with words. There are too many of them!!! How many words are there in English? When we speak about the number of words in the language, we are generally referring to word types. Fig. 2.1 shows the rough numbers of types and instances computed from some English corpora.

You will notice that the larger the corpora we look at, the more word types we find! That suggests that there is not a clear answer to how many words there are; the answer keeps growing as we see more data! We can see this fact mathematically

Herdan's Law
Heaps' Law

because the relationship between the number of types $|V|$ and number of instances N is called **Herdan's Law** (Herdan, 1960) or **Heaps' Law** (Heaps, 1978) after its discoverers (in linguistics and information retrieval respectively). It is shown in Eq. 2.5, where k and β are positive constants, and $0 < \beta < 1$.

$$|V| = kN^{\beta} \tag{2.5}$$

The value of β depends on the corpus size and the genre; numbers from 0.44 to 0.56 or even higher have often been reported. Roughly we can say that the vocabulary size for a text goes up a little faster than the square root of its length in words.

function words
content words

There are also variants of the law, which capture the fact that we can distinguish roughly two classes of words. One is **function words**, the grammatical words like English *a* and *of*, that tend not to grow indefinitely (a language tends to have a fixed number of these). The other is **content words**: nouns, adjectives and verbs that tend to have meanings about people and places and events. Nouns, and especially particular nouns like names and technical terms do tend to grow indefinitely. So models that are sensitive to this difference between function words and content words have one value of β for the initial part of the corpus where all words are still appearing, and then a second β afterwords for when only the content words are still appearing. Fig. 2.2 shows an example from Tria et al. (2018) showing two values of β (called γ in their figure) for Heaps law computed on the Gutenberg corpus of books. Note that

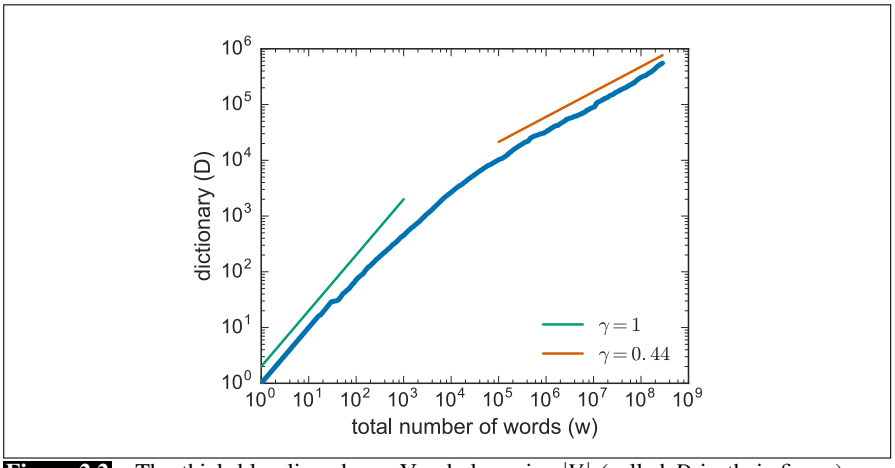


Figure 2.2 The thick blue line shows Vocabulary size $|V|$ (called D in their figure) as a function of text length (their w), computed on the Gutenberg corpus of publicly available books. Note that at the beginning of the corpus, we see both common and function words and the relationship between corpus size and vocabulary is roughly linear (green line, $\gamma = 1$). Later, after the function words have mainly appeared, the number of new words slows down and is closer to the square root of the corpus size. Figure from Tria et al. (2018).

The fact that words grow without end leads to a problem for any computational model. No matter how big our vocabulary, we will never have a vocabulary that captures all the possible words that might occur! That means that our computational model will constantly see **unknown words**: words that it has never seen before. This is a huge problem for machine learning models.

Because of these two problems (first, that many languages don't have orthographic words, and defining them post-hoc is challenging and second, that the number of words grows without bound), language models and other NLP models don't

tend to use words as their unit of processing. Instead, they use smaller units called **subwords** that can be recombined to model new words that our model has never seen before. To think about defining subwords, we first need to talk about units that are smaller than words; **morphemes** and **characters**.

2.2 Morphemes: Parts of Words

Words have parts. At the level of characters, this is obvious. The word *cats* is composed of four characters, 'c', 'a', 't', 's'. But this is also true at a more subtle level: words have components that themselves have coherent meanings. These components are called **morphemes**, and the study of morphemes is called **morphology**. A **morpheme** is a minimal meaning-bearing unit in a language. So, for example, the word *fox* consists of one morpheme (the morpheme *fox*) while the word *cats* consists of two: the morpheme *cat* and the morpheme *-s* that indicates plural.

morphology
morpheme

Here's a sentence in English segmented into morphemes with hyphens:

(2.6) Doc work-ed care-ful-ly wash-ing the glass-es

As we mentioned above, in Chinese, conveniently, the writing system is set up so that each character mainly describes a morpheme. Here's a sentence in Mandarin Chinese with each morpheme character glossed, followed by the translation:

(2.7) 梅 干 菜 用 清 水 泡 软 ， 捞 出 后 ， 沥 干
plum dry vegetable use clear water soak soft , remove out after , drip dry
切 碎
chop fragment
Soak the preserved vegetable in water until soft, remove, drain, and chop

We generally distinguish two broad classes of morphemes: **roots**—the central morpheme of the word, supplying the main meaning—and **affixes**—adding "additional" meanings of various kinds. In the English example above, for the word *worked*, *work* is a root and *-ed* is an affix; similarly for *glasses*, *glass* is a root and *-es* an affix.

root
affix

Affixes themselves fall into two classes, or more correctly a continuum between two poles. At one end, **inflectional morphemes** are grammatical morphemes that tend to play a syntactic role, such as marking agreement. For example, English has the inflectional morpheme *-s* (or *-es*) for marking the **plural** on nouns and the inflectional morpheme *-ed* for marking the past tense on verbs. Inflectional morphemes tend to be productive and often obligatory and their meanings tend to be predictable. **Derivational morphemes** are more idiosyncratic in their application and meaning. Usually they apply only to a specific subclass of words and result in a word of a *different* grammatical class than the root, often with a meaning hard to predict exactly. In the example above, the word *care* (a noun) can be combined with the derivational affix *-full* to produce an adjective (*careful*), and another derivational affix *-ly* to result in an adverb (*carefully*).

inflectional morphemes
derivational morphemes

There is another class of morphemes: **clitics**. A clitic is a morpheme that acts syntactically like a word but is reduced in form and attached (phonologically and sometimes orthographically) to another word. For example the English morpheme *'ve* in the word *I've* is a clitic; it has the grammatical meaning of the word *have*, but in form in cannot appear alone (you can't just say the sentence "'ve"). The English possessive morpheme *'s* in the phrase *the teacher's book* is a clitic. French definite

clitic

article *l'* in the word *l'opera* is a clitic, as are prepositions in Arabic like *b* 'by/with' and conjunctions like *w* 'and'.

morphological typology

The study of how languages vary in their morphology, i.e., how words break up into their parts, is called **morphological typology**. While morphologies of languages can differ along many dimensions, two dimensions are particularly relevant for computational word tokenization.

The first dimension is the **number of morphemes per word**. In some languages, like Vietnamese and Cantonese, each word on average has just over one morpheme. We call languages at this end of the scale **isolating** languages. For example each word in the following Cantonese sentence has one morpheme (and one syllable):

isolating

(2.8) *keoi5 waa6 cyun4 gwok3 zeoi3 daai6 gaan1 uk1 hai6 ni1 gaan1*
he say entire country most big building house is this building
"He said the biggest house in the country was this one"

Alternatively, in languages like Koryak, a Chukotko-Kamchatkan language spoken in the northern part of the Kamchatka peninsula in Russia, a single word may have very many morphemes, corresponding to a whole sentence in English (Arkadiev, 2020; Kurebito, 2017). We call languages toward this end of the scale **synthetic** languages, and the very end of the scale **polysynthetic** languages.

synthetic
polysynthetic

(2.9) *t-ə-nk'e-mejŋ-ə-jetemə-nni-k*
1SG.S-E-midnight-big-E-yurt.cover-E-sew-1SG.S[PFV]
"I sewed a lot of yurt covers in the middle of a night."
(Koryak, Chukotko-Kamchatkan, Russia; Kurebito (2017, 844))

Fig. 2.3 shows an early computation of morphemes per words on a few languages by the linguistic typologist Joseph Greenberg (1960).

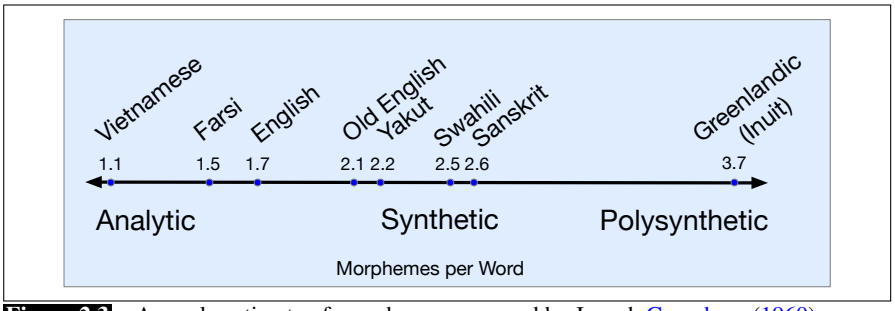


Figure 2.3 An early estimate of morphemes per word by Joseph Greenberg (1960).

agglutinative
fusion

The second dimension is the degree to which morphemes are easily segmentable, ranging from **agglutinative** languages like Turkish, in which morphemes have relatively clean boundaries, to **fusion** languages like Russian, in which a single affix may conflate multiple morphemes, like *-om* in the word *stolom* (table-SG-INSTR-DECL1), which fuses the distinct morphological categories instrumental, singular, and first declension.

The English *-s* suffix in *She read***s** *the article* is an example of fusion, since the suffix means both third person singular but also means present tense, and there's no way to divide up the meaning to different parts of the *-s*.

Although we have loosely talked about these properties (analytic, polysynthetic, fusional, agglutinative) as if they are properties of languages, in fact languages can make use of different morphological systems so it would be more accurate to talk about these as general tendencies.

Nonetheless, the fact morphemes can be hard to define, and that many languages can have complex morphemes that aren't easy to break up into pieces makes it very difficult to use morphemes as a standard for tokenization cross-lingually.

2.3 Unicode

Another option we could consider for tokenization is the level of the individual character. How do we even represent characters across languages and writing system?

Unicode The **Unicode** standard is a method for representing text written using any character in any script of the languages of the world (including dead languages like Sumerian cuneiform, and invented languages like Klingon).

Let's start with a brief historical note about an English-specific subset of Unicode (technically called 'Basic Latin' in Unicode, and commonly referred to as ASCII).

ASCII Starting in the 1960s, the Latin characters used to write English (like the ones used in this sentence), were represented with a code called **ASCII** (American Standard Code for Information Interchange). ASCII represented each character with a single byte. A byte can represent 256 different characters, but ASCII only used 127 of them; the high-order bit of ASCII bytes is always set to 0. (Actually it only used 95 of them and the rest were control codes for an obsolete machine called a teletype). Here's a few ASCII characters with their representation in hex and decimal:

Ch	Hex	Dec	Ch	Hex	Dec		Ch	Hex	Dec	Ch	Hex	Dec
<	3C	60	@	40	64	...	\	5C	92	‘	60	96
=	3D	61	A	41	65	...	[	5D	93	a	61	97
>	3E	62	B	42	66	...	^	5E	94	b	62	98
?	3F	63	C	43	67	...	_	5F	95	c	63	99

Figure 2.4 Some selected ASCII codes for some English letters, with the codes shown both in hexadecimal and decimal.

But ASCII is of course insufficient since there are lots of other characters in the world's writing systems! Even for scripts that use Latin characters, there are many more than the 95 in ASCII. For example, this Spanish phrase (meaning "Sir, replied Sancho") has two non-ASCII characters, ñ and ó:

(2.10) Señor- respondió Sancho-

Devanagari And lots of languages aren't based on Latin characters at all! The **Devanagari** script is used for 120 languages (including Hindi, Marathi, Nepali, Sindhi, and Sanskrit). Here's a Devanagari example from the Hindi text of the Universal Declaration of Human Rights:

अनुच्छेद १(एक): सभी मनुष्य जन्म से स्वतन्त्र तथा मर्यादा और अधिकारों में समान होते हैं। वे तर्क और विवेक से सम्पन्न हैं तथा उन्हें भ्रातृत्व की भावना से परस्पर के प्रति कार्य करना चाहिए।

Chinese has about 100,000 Chinese characters in Unicode (including overlapping and non-overlapping variants used in Chinese, Japanese, Korean, and Vietnamese, collectively referred to as CJKV).

All in all there are more than 150,000 characters and 168 different scripts supported in Unicode 16.0. Even though many scripts from around the world have yet to be added to Unicode, there are so many there, from scripts used by modern languages (Chinese, Arabic, Hindi, Cherokee, Ethiopic, Khmer, N'Ko, Turkish,

Spanish) to scripts of ancient languages (Cuneiform, Ugaritic, Egyptian Hieroglyph, Pahlavi), as well as mathematical symbols, emojis, currency symbols, and more.

2.3.1 Code Points

code point

How does it work? Unicode assigns a unique id, called a **code point**, for each one of these 150,000 characters.

The code point is an abstract representation of the character, and each code point is represented by a number, traditionally written in hexadecimal, from number 0 through 0x10FFFF (which is 1,114,111 decimal). Having over a million code points means there is a lot of room for new characters. It is traditional to represent these code points with the prefix "U+" (which just means "the following is a Unicode hex representation of a code point"). So the code point for the character `a` is `U+0061` which is the same as `0x0061`. (Note that Unicode was designed to be backwards compatible with ASCII, which means that the first 127 code points, including the code for `a`, are identical with ASCII.) Here are some sample code points; some (but not all) come with descriptions:

```
U+0061  a   LATIN SMALL LETTER A
U+0062  b   LATIN SMALL LETTER B
U+0063  c   LATIN SMALL LETTER C
U+00F9  ù   LATIN SMALL LETTER U WITH GRAVE
U+00FA  ú   LATIN SMALL LETTER U WITH ACUTE
U+00FB  û   LATIN SMALL LETTER U WITH CIRCUMFLEX
U+00FC  ü   LATIN SMALL LETTER U WITH DIAERESIS
U+8FDB  进
U+8FDC  远
U+8FDD  违
U+8FDE  连
U+1F600 😀  GRINNING FACE
U+1F00E 🀎  MAHJONG TILE EIGHT OF CHARACTERS
```

glyph

Note that a code point does not specifiy the **glyph**, the visual representation of a character. Glyphs are stored in **fonts**. The code point `U+0061` is an abstract representation of `a`. There can be an indefinite number of visual representations, for example in different fonts like Times Roman (a) or Courier (a), or different font styles like boldface (**a**) or italic (*a*). But all of them are represented by the same code point `U+0061`.

2.3.2 UTF-8 Encoding

While the code point (the unique id) is the abstract Unicode representation of the character, we don't just stick that id in a text file.

encoding

Instead, whenever we need to represent a character in a text string, we write an **encoding** of the character. There are many different possible encoding methods, but the encoding method called UTF-8 is by far the most frequent (for example almost the entire web is encoded in UTF-8).

Let's talk about encodings. The Unicode representation of the word `hello` consists of the following sequence of 5 code points:

```
U+0068  U+0065 U+006C U+006C U+006F
```

We can imagine a very simple encoding method: just write the code point id in a file. Since there are more than 1 million characters, 16 bits (2 bytes) isn't enough, so we'll need to use 4 bytes (32 bit) to capture the 21 bits we need to represent 1.1 million characters. (We could fit it in 3 bytes but it's inconvenient to use multiples of 3 for bytes.)

With this 4-byte representation the word `hello` would be encoded as the following set of bytes:

```
00 00 00 68 00 00 00 65 00 00 00 6C 00 00 00 6C 00 00 00 6F
```

But we don't use this encoding (which is technically called UTF-32) because it makes every file 4 times longer than it would have been in ASCII, making files really big and full of zeros. Also those zeros cause another problem: it turns out that having any byte that is completely zero messes things up for backwards compatibility for ASCII-based systems that historically used a 0 byte as an end-of-string marker.

UTF-8

Instead, the most common encoding standard is **UTF-8** (Unicode Transformation Format 8), which represents characters efficiently (using fewer bytes on average) by writing some characters using fewer bytes and some using more bytes. UTF-8 is thus a **variable-length encoding**.

variable-length encoding

For some characters (the first 127 code points, i.e. the set of ASCII characters), UTF-8 encodes them as a single byte, so the UTF-8 encoding of `hello` is :

```
68 65 6C 6C 6F
```

This conveniently means that files encoded in ASCII are also valid UTF-8 encodings!

But UTF-8 is a variable length encoding, meaning that code points $\geq$128 are encoded as a sequence of two, three, or four bytes. Each of these bytes are between 128 and 255, so they won't be confused with ASCII, and each byte indicates in the first few bits whether it's a 2-byte, 3-byte, or 4-byte encoding.

Code Points		UTF-8 Encoding			
From - To	**Bit Value**	**Byte 1**	**Byte 2**	**Byte 3**	**Byte 4**
U+0000-U+007F	0xxxxxxx	xxxxxxxx			
U+0080-U+07FF	00000yyy yyxxxxxx	110yyyyy	10xxxxxx		
U+0800-U+FFFF	zzzzyyyy yyxxxxxx	1110zzzz	10yyyyyy	10xxxxxx	
U+010000-U+10FFFF	000uuuuu zzzzyyyy yyxxxxxx	11110uuu	10uuzzzz	10yyyyyy	10xxxxxx

Figure 2.5 Mapping from Unicode code point to the variable length UTF-8 encoding. For a given code point in the From-To range, the bit value in column 2 is packed into 1, 2, 3, or 4 bytes. Figure adapted from Unicode 16.0 Core Spec Chapter 3 Table 3-6.

Fig. 2.5 shows how this mapping occurs. For example these rules explain how the character ñ, which has code point U+00F1, or bit sequence `00000000 11110001`, (where blue indicates the sequence yyyyy and red the sequence xxxxxx) is encoded into to the two-byte bit sequence `11000011 10110001` or `0xC3B1`. As a result of these rules, the first 127 characters (ASCII) are mapped to one byte, most remaining characters in European, Middle Eastern, and African scripts map to two bytes, most Chinese, Japanese, and Korean characters map to three bytes, and rarer CJKV characters and emojis and some symbols map to 4 bytes.

UTF-8 has a number of advantages. It's relatively efficient, using fewer bytes for commonly-encountered characters, it doesn't use zero bytes (except when literally representing the NULL character which is U+0000), it's backwards compatible with ASCII, and it's self-synchronizing, meaning that if a file is corrupted, it's always possible to find the start of the next or prior character just by moving up to 3 bytes left or right.

Unicode and Python: Starting with Python 3, all Python strings are stored internally as Unicode, each string a sequence of Unicode code points. Thus string functions and regular expressions all apply natively to code points. For example, functions like `len()` of a string return its length in characters, i.e., code points, not its length in bytes.

When reading or writing from a file, however, the code points need to be encoded and decoding using a method like UTF-8. That is, every file is encoded in some encoding. If it's not UTF-8, it's an older encoding method like ASCII or Latin-1 (iso_8859_1). There is no such thing as a text file without an encoding. The encoding method is specified in Python when opening a file for reading and writing.

2.4 Subword Tokenization: Byte-Pair Encoding

tokenization
tokens

Tokenization, the first stage of natural language processing, is the process of segmenting the running input text into **tokens**.

We've seen three candidates for tokens: words, morphemes and characters. But each has problems as a unit. Words and morphemes seem approximately at the right level for NLP processing, since they tend to have consistent meanings, but they are challenging to define formally. Characters are clearer to define, but seem too small a unit to choose for tokens.

In this section we introduce what we do in practice for NLP: use a data-driven approach to define tokens that will generally result in units about the size of morphemes or words, but occasionally use units as small as characters.

Why tokenize the input? One reason is that converting an input to a deterministic fixed set of units means that different algorithms and systems can agree on simple questions. For example, How long is this text? (How many units are in it?). Or: Is *don't* or *New York* one token or two? Standardizing is thus essential for replicability in NLP experiments, and many algorithms that we introduce in this book (like the **perplexity** metric for language models) assume that all texts have a fixed tokenization.

Tokenization algorithms that include smaller tokens for morphemes and letters also eliminate the problem of **unknown words**. What are these? As we will see in the next chapter, NLP algorithms often learn some facts about language from one corpus (a **training** corpus) and then use these facts to make decisions about a separate **test** corpus and its language. Thus if our training corpus contains, say the words *low*, *new*, and *newer*, but not *lower*, then if the word *lower* appears in our test corpus, our system will not know what to do with it.

subwords

To deal with this unknown word problem, modern tokenizers automatically induce sets of tokens that include tokens smaller than words, called **subwords**. Subwords can be arbitrary substrings, or they can be meaning-bearing units like the morphemes *-est* or *-er*. In modern tokenization schemes, many tokens are words, but other tokens are frequently occurring morphemes or other subwords like *-er*. Every unseen word can thus be represented by some sequence of known subword units. For example, if we had happened not to ever see the word *lower*, when it appears we could segment it successfully into *low* and *er* which we had already seen. In the worst case, a really unusual word (perhaps an acronym like *GRPO*) could be tokenized as a sequence of individual letters if necessary.

Two tokenization algorithms are widely used in modern language models: **byte-pair encoding** (BPE) (Sennrich et al., 2016), and **unigram language modeling**

BPE (ULM) (Kudo, 2018).[2] In this section we introduce the **byte-pair encoding** or **BPE** algorithm (Sennrich et al., 2016; Gage, 1994); see Fig. 2.6.

Like most tokenization schemes, the BPE algorithm has two parts: a **trainer**, and an **encoder**. In general in the token training phase we take a raw training corpus (usually roughly pre-separated into words, for example by whitespace) and induce a vocabulary, a set of tokens. Then a token encoder takes a raw test sentence and encodes it into the tokens in the vocabulary that were learned in training.

2.4.1 BPE training

The **BPE** training algorithm iteratively merges frequent neighboring tokens to create longer and longer tokens. The algorithm begins with a vocabulary that is just the set of all individual characters. It then examines the training corpus, and finds the two characters that are most frequently adjacent. Imagine our original corpus is 10 characters long, using a vocabulary of 5 characters, {A, B, C, D, E}:

```
A B D C A B E C A B
```

The most frequent neighboring pair of characters is "A B" so we merge those, add a new merged token 'AB' to the vocabulary, and replace every adjacent 'A' 'B' in the corpus with the new 'AB':

```
AB D C AB E C AB
```

Now we have a vocabulary of 6 possible tokens {A, B, C, D, E, AB}, and the corpus has length 7. And now the most frequent pair of tokens is "C AB", so we merge those, leading to a vocabulary with 7 tokens {A, B, C, D, E, AB, CAB}, and the corpus has length 5.

```
AB D CAB E CAB
```

The algorithm continues to count and merge, creating new longer and longer character strings, until k merges have been done creating k novel tokens; k is thus a parameter of the algorithm. The resulting vocabulary consists of the original set of characters plus k new symbols. That's the core of the algorithm.

The only additional complication is that in practice, instead of running on the raw sequence of characters, the algorithm is usually run only *inside* words. That is, the algorithm does not merge across word boundaries. To do this, the input corpus is often first separated at white space and punctuation (using the regular expressions that we define later in the chapter). This gives a starting set of strings, each corresponding to the characters of a word, (with the white space usually attached to the start of the word), together with the counts of the words. Then while counts come from a corpus, merges are only allowed within the strings.

Let's see how the full algorithm thus works on this tiny synthetic corpus, where we've explicitly marked the spaces between words:[3]

(2.11) `set␣new␣new␣renew␣reset␣renew`

First, we'll break up the corpus into words, with leading whitespace, together with their counts; no merges will be allowed to go beyond these word boundaries. The result looks like the following list of 4 words and a starting vocabulary of 7 characters:

[2] The **SentencePiece** library includes implementations of both of these (Kudo and Richardson, 2018a), and people sometimes use the name **SentencePiece** to simply mean ULM tokenization.

[3] Yes, we realize this isn't a particularly likely or exciting sentence.

corpus	vocabulary
2 _ n e w	_, e, n, r, s, t, w
2 _ r e n e w	
1 s e t	
1 _ r e s e t	

The BPE training algorithm first counts all pairs of adjacent symbols: the most frequent is the pair `n e` because it occurs in `new` (frequency of 2) and `renew` (frequency of 2) for a total of 4 occurrences. We then merge these symbols, treating `ne` as one symbol, and count again:

corpus	vocabulary
2 _ ne w	_, e, n, r, s, t, w, ne
2 _ r e ne w	
1 s e t	
1 _ r e s e t	

Now the most frequent pair is `ne w` (total count=4), which we merge.

corpus	vocabulary
2 _ new	_, e, n, r, s, t, w, ne, new
2 _ r e new	
1 s e t	
1 _ r e s e t	

Next _ r (total count of 3) get merged to _r, and then _r e (total count 3) gets merged to _re. The system has essentially induced that there is a word-initial prefix `re-`:

corpus	vocabulary
2 _ new	_, e, n, r, s, t, w, ne, new, _r, _re
2 _re new	
1 s e t	
1 _re s e t	

If we continue, the next merges are:

merge	current vocabulary
(_, new)	_, e, n, r, s, t, w, ne, new, _r, _re, _new
(_re, new)	_, e, n, r, s, t, w, ne, new, _r, _re, _new, _renew
(s, e)	_, e, n, r, s, t, w, ne, new, _r, _re, _new, _renew, se
(se, t)	_, e, n, r, s, t, w, ne, new, _r, _re, _new, _renew, se, set

function BYTE-PAIR ENCODING(strings C, number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters
for $i = 1$ **to** k **do** # merge tokens k times
  $t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C
  $t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating
  $V \leftarrow V + t_{NEW}$ # update the vocabulary
  Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus
return V

Figure 2.6 The training part of the BPE algorithm for taking a corpus broken up into individual characters or bytes, and learning a vocabulary by iteratively merging tokens. Figure adapted from Bostrom and Durrett (2020).

2.4.2 BPE encoder

Once we've learned our vocabulary, the BPE **encoder** is used to tokenize a test sentence. The encoder just runs on the test data the merges we have learned from the training data. It runs them greedily, in the order we learned them. (Thus the frequencies in the test data don't play a role, just the frequencies in the training data). So first we segment each test sentence word into characters. Then we apply the first rule: replace every instance of `n e` in the test corpus with `ne`, and then the second rule: replace every instance of `ne w` in the test corpus with `new`, and so on. By the end of course many of the merges simple recreated words in the training set. But the merges also created knowledge of morphemes like the `re-` prefix (that might appear in perhaps unseen combinations like `revisit` or `rearrange`), or the morpheme `new` without an initial space (hence word-internal) that might appear at the start of sentences or in words unseen in training like `anew`.

Of course in real settings BPE is run with tens of thousands of merges on a very large input corpus, to produce vocabulary sizes of 50,000, 100,000, or even 200,000 tokens. The result is that most words can be represented as single tokens, and only the rarer words (and unknown words) will have to be represented by multiple tokens. At least for English. For multilingual systems, the tokens can be dominated by English, leaving fewer tokens for other languages, as we'll discuss below.

2.4.3 BPE in practice

The example above just showed simple BPE learning from sequences of ASCII bytes. How does BPE work with Unicode input? We normally run BPE on the individual bytes of UTF-8-encoded text. That is, we take a Unicode representations of text as a series of code points, encode it in bytes using UTF-8, and we treat each of these individual bytes as the input to BPE. Thus BPE likely begins by rediscovering the 2-byte and common 3-byte sequences that UTF-8 uses to encode various code points. Again, running BPE only inside presegmented words helps avoid problems. Because there are only 256 possible values of a byte, there will be no unknown tokens, although it's possible that BPE will learn some illegal UTF-8 sequences across character boundaries. These will be very rare, and can be eliminated with a filter.

Let's see some examples of the industrial application of the BPE tokenizer used in large systems like OpenAI GPT4o. This tokenizer has 200K tokens, which is a comparatively large number. We can use Tat Dat Duong's Tiktokenizer visualizer (https://tiktokenizer.vercel.app/) to see the number of tokens in a given sentence. For example here's the tokenization of a nonsense sentence we made up; the visualizer uses a center dot to indicate a space:

```
Anyhow,·she's·seen·Jane's·224123·flowers·anyhow!
```

The visualization shows colors to separate out words, but of course the true output of the tokenizer is simply a sequence of unique token ids. (In case you're interested, they were the following 13 tokens: 11865, 8923, 11, 31211, 6177, 23919, 885, 220, 19427, 7633, 18887, 147065, 0)

Notice that most words are their own token, usually including the leading space. Clitics like `'s` are segmented off when they appear on proper nouns like `Jane`, but are counted as part of a word for frequent words like `she's`. Numbers tend to be segmented into chunks of 3 digits. And some words (like *anyhow*) are segmented differently if they appear capitalized sentence-initially (two tokens, `Any` and `how`), then if they appear after a space, lower case (one token `anyhow`).

pretokenization

Some of these are related to preprocessing steps. As we mentioned briefly above, language models usually create their tokens in a **pretokenization** stage that first segments the input using regular expressions, for example breaking the input at spaces and punctuation, stripping off clitics, and breaking numbers into sets of digits. We'll see how to use regular expressions in Section 2.6.

SuperBPE
BoundlessBPE

It's possible to change this pretokenization to allow BPE tokens to span multiple words. For example the **SuperBPE** (Liu et al., 2025) and **BoundlessBPE** (Schmidt et al., 2025) algorithms first induce regular BPE subword tokens by enforcing pretokenization. They then run a second stage of BPE allowing merges across spaces and punctuation. The result is a large set of tokens that can be more efficient (Fig. 2.7).

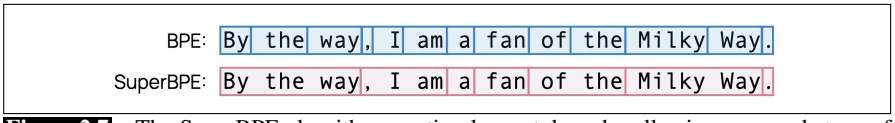


Figure 2.7 The SuperBPE algorithm creating larger tokens by allowing a second stage of merging across spaces. Figure from Liu et al. (2025).

Many of the tokenizers used in practice for large language models are multilingual, trained on many languages. But because the training data for large language models is vastly dominated by English text, these multilingual BPE tokenizers tend to use most of the tokens for English, leaving fewer of them for other languages. The result is that they do a better job of tokenizing English, and the other languages tend to get their words split up into shorter tokens. For example let's look at a Spanish sentence from a recipe for plantains, together with an English translation.

The English has 18 tokens; each of the 14 words is a token (none of the words are split into multiple tokens):

```
In·a·deep·bowl,·mix·the·orange·juice·with·the·sugar,·g
inger,·and·nutmeg.
```

By contrast, the original 16 words in Spanish have been encoded into 33 tokens, a much larger number. Notice that many basic words have been broken into pieces. For example `hondo`, 'deep', has been segmented into `h` and `ondo`. Similarly for `jugo`, 'juice', `nuez`, 'nut' and `jenjibre` 'ginger'):

```
En·un·recipiente·hondo,·mezclar·el·jugo·de·naranja·con
·el·azúcar,·jengibre,·y·nuez·moscada.
```

Spanish is not a particularly low-resource language; this oversegmenting can be even more serious in lower resource languages, often down to individual characters. Oversegmenting into these tiny tokens can cause various problems for the downstream processing of the language. As will become more clear once we introduce transformer models in Chapter 8, such fragmentation can lead to poor representations of meaning, the need for longer contexts, and higher costs to train models (Rust et al., 2021; Ahia et al., 2023).

2.5 Corpora

Words don't appear out of nowhere. Any particular piece of text that we study is produced by one or more specific speakers or writers, in a specific dialect of a

specific language, at a specific time, in a specific place, for a specific function.

Perhaps the most important dimension of variation is the language. NLP algorithms are most useful when they apply across many languages. The world has 7097 languages at the time of this writing, according to the online Ethnologue catalog (Simons and Fennig, 2018). It is important to test algorithms on more than one language, and particularly on languages with different properties; by contrast there is an unfortunate current tendency for NLP algorithms to be developed or tested just on English (Bender, 2019). Even when algorithms are developed beyond English, they tend to be developed for the official languages of large industrialized nations (Chinese, Spanish, Japanese, German etc.), but we don't want to limit tools to just these few languages. Furthermore, most languages also have multiple varieties, often spoken in different regions or by different social groups. Thus, for example, if we're

AAE processing text that uses features of African American English (**AAE**) or African American Vernacular English (AAVE)—the variations of English that can be used by millions of people in African American communities (King 2020)—we must use NLP tools that function with features of those varieties. Twitter posts might use features often used by speakers of African American English, such as constructions like

MAE *iont* (*I don't* in Mainstream American English (**MAE**)), or *talmbout* corresponding to MAE *talking about*, both examples that influence word segmentation (Blodgett et al. 2016, Jones 2015).

It's also quite common for speakers or writers to use multiple languages in a sin-

code switching gle utterance, a phenomenon called **code switching**. Code switching is enormously common across the world; here are examples showing Spanish and (transliterated) Hindi code switching with English (Solorio et al. 2014, Jurgens et al. 2017):

(2.12) Por primera vez veo a @username actually being hateful! it was beautiful:)
[For the first time I get to see @username actually being hateful! it was beautiful:)]

(2.13) dost tha or ra- hega ... dont wory ... but dherya rakhe
["he was and will remain a friend ... don't worry ... but have faith"]

Another dimension of variation is the genre. The text that our algorithms must process might come from newswire, fiction or non-fiction books, scientific articles, Wikipedia, or religious texts. It might come from spoken genres like telephone conversations, business meetings, police body-worn cameras, medical interviews, or transcripts of television shows or movies. It might come from work situations like doctors' notes, legal text, or parliamentary or congressional proceedings.

Text also reflects the demographic characteristics of the writer (or speaker): their age, gender, race, socioeconomic class can all influence the linguistic properties of the text we are processing.

And finally, time matters too. Language changes over time, and for some languages we have good corpora of texts from different historical periods.

Because language is so situated, when developing computational models for language processing from a corpus, it's important to consider who produced the language, in what context, for what purpose. How can a user of a dataset know all these

datasheet details? The best way is for the corpus creator to build a **datasheet** (Gebru et al., 2020) or **data statement** (Bender et al., 2021) for each corpus. A datasheet specifies properties of a dataset like:

Motivation: Why was the corpus collected, by whom, and who funded it?

Situation: When and in what situation was the text written/spoken? For example, was there a task? Was the language originally spoken conversation, edited text, social media communication, monologue vs. dialogue?

Language variety: What language (including dialect/region) was the corpus in?

Speaker demographics: What was, e.g., the age or gender of the text's authors?

Collection process: How big is the data? If it is a subsample how was it sampled? Was the data collected with consent? How was the data pre-processed, and what metadata is available?

Annotation process: What are the annotations, what are the demographics of the annotators, how were they trained, how was the data annotated?

Distribution: Are there copyright or other intellectual property restrictions?

2.6 Regular Expressions

regular expression

One of the most useful tools for text processing in computer science is the **regular expression** (or **regex**), a language for specifying text strings. Regexes are used in every computer language, in text processing tools like Unix `grep`, and in editors like vim or Emacs. And they play an important role in the pre-tokenization step for tokenization algorithms like BPE. Formally, a regular expression is an algebraic notation for characterizing a set of strings. Practically, we can use a regex to search for a string in a text and to specify how to change the string, both of which are key to tokenization.

string

We use regular expressions to search for a **pattern** in a **string** which can be a single line or a longer text. For example, the Python function

```
re.search(pattern,string)
```

scans through the `string` and returns the first match inside it for the `pattern`. In the following examples we generally highlight the exact string that matches the regular expression and show only the first match. We'll use Python syntax, expressing the regex as a raw string delimited by double quotes: `r"regex"`. Raw strings treat backslashes as literal characters, which will be important since many regex patterns we'll introduce use backslashes.

Regular expressions come in different variants, so using an online regex tester can help make sure your regex does what you think it's doing.

2.6.1 Character Disjunction: The Square Bracket

The simplest kind of regular expression is a sequence of simple characters. The pattern `r"Buttercup"` matches the substring `Buttercup` in any string (like the string `I'm called little Buttercup`). But often we need to use special characters. For example, we might want to match *either* some character or another. For example, regular expressions are generally **case sensitive**: `r"s"` matches a lower case `s` but not an upper case `S`. To match both `s` and `S` we can use the **character disjunction** operator, the square braces `[` and `]`. The string of characters inside the braces specifies a disjunction of characters to match. For example, Fig. 2.8 shows that the pattern `r"[mM]"` matches patterns containing either `m` or `M`.

character disjunction

Pattern	Match	String
`r"[mM]ary"`	Mary or mary	"Mary Ann stopped by Mona's"
`r"[abc]"`	'a', 'b', *or* 'c'	"In uomini, in soldati"
`r"[1234567890]"`	any one digit	"plenty of 7 to 5"

Figure 2.8 The use of the brackets `[]` to specify a disjunction of characters.

The regular expression `r"[1234567890]"` specifies any single digit. This can get awkward (imagine typing `r"[ABCDEFGHIJKLMNOPQRSTUVWXYZ]"` to mean an uppercase letter) so the brackets can also be used with a dash (`-`) to specify any one character in a **range**. The pattern `r"[2-5]"` specifies any one of the characters *2*, *3*, *4*, or *5*. The pattern `r"[b-g]"` specifies one of the characters *b*, *c*, *d*, *e*, *f*, or *g*. Some other examples are shown in Fig. 2.9.

range

Regex	Match	Example Patterns Matched
`r"[A-Z]"`	an upper case letter	"we should call it 'Drenched Blossoms' "
`r"[a-z]"`	a lower case letter	"my beans were impatient to be hoed!"
`r"[0-9]"`	a single digit	"Chapter 1: Down the Rabbit Hole"

Figure 2.9 The use of the brackets `[]` plus the dash `-` to specify a range.

The square braces can also be used to specify what a single character *cannot* be, by use of the caret `^`. If the caret `^` is the first symbol after the open square brace `[`, the resulting pattern is negated. For example, the pattern `r"[^a]"` matches any single character (including special characters) except *a*. This is only true when the caret is the first symbol after the open square brace. If it occurs anywhere else, it usually stands for a caret; Fig. 2.10 shows some examples.

Regex	Match (single characters)	Example Patterns Matched
`r"[^A-Z]"`	not an upper case letter	"Oyfn pripetchik"
`r"[^Ss]"`	neither 'S' nor 's'	"I have no exquisite reason for't"
`r"[^.]"`	not a period	"our resident Djinn"
`r"[e^]"`	either 'e' or '^'	"look up ^ now"
`r"a^b"`	the pattern 'a^b'	"look up a^ b now"

Figure 2.10 The caret `^` for negation or just to mean `^`. See below re: the backslash for escaping the period.

2.6.2 Counting, Optionality, and Wildcards

How can we talk about optional elements, like an optional *s* if we want to match both *koala* and *koalas*? We can't use the square brackets, because while they allow us to say "s or S", they don't allow us to say "s or nothing". For this we use the question mark `r"?"`, which means "the preceding character or nothing",. So `r"colou?r"` matches both `color` and `colour`, and `r"koalas?"` matches `koala` or `koalas`.

There's another way to talk about elements that may or may not occur. Consider the language of certain sheep, which consists of strings that look like the following:

```
baa!
baaa!
baaaa!
...
```

This sheep language consists of strings with a `b`, followed by at least two (and arbitrarily more) *a*'s, followed by an exclamation point. To represent this language, we'll use a useful operator that is represented by the asterisk or `*`, called the **Kleene** `*` (generally pronounced "cleany star"). The Kleene star means "zero or more occurrences of the immediately previous character or regular expression". So `r"a*"` means "any string of zero or more `a`s".

Kleene *

Could `r"ba*"` represent the sheep language? It will correctly match `ba` or `baaaaaa`, but there's a problem! It will also match `b`, with no `a`, or `ba` with only one

a. That's because Kleene star means "zero or more occurrences". Instead, for the sheep language we'll want `r"baaa*"`, meaning `b` followed by `aa` followed by zero or more additional `a`s. More complex patterns can also be repeated. So `r"[ab]*"` means "zero or more *a*'s or *b*'s" (not "zero or more right square braces"). This will match strings like *aaaa* or *ababab* or *bbbb*, as well as the empty string. For specifying an integer (a string of digits) we can use `r"[0-9][0-9]*"`. (Why isn't it just `r"[0-9]*"`?)

Kleene +

There is a slightly shorter way to specify "at least one" of some character: the **Kleene +**, which means "one or more occurrences of the immediately preceding character or regular expression". So `r"[0-9]+"` is the normal way to specify "a sequence of digits", and we could also specify the sheep language as `r"baa+!"`.

Besides the Kleene * and Kleene + we can also use explicit numbers as counters, by enclosing them in curly brackets. The operator `r"{3}"` means "exactly 3 occurrences of the previous character or expression". So `r"ax{10}z"` will match `a` followed by exactly 10 `x`'s followed by `z`.

period

An important special character is the **period** (`r"."`), a **wildcard** expression that matches any single character (*except* a newline).

The wildcard is often used together with the Kleene star to mean "any string of characters". For example, suppose we want to find any line in which a particular word, for example, `rose`, appears twice. We can specify this with the regular expression `r"rose.*rose"`, meaning two roses, with a sequence of zero or more characters (of any kind) between them. Fig. 2.11 summarizes.

Regex	Match
*	zero or more occurrences of the previous char or expression
+	one or more occurrences of the previous char or expression
?	zero or one occurrence of the previous char or expression
{n}	exactly *n* occurrences of the previous char or expression
.	any single char
.*	any string of zero or more chars

Figure 2.11 Counting and wildcards.

2.6.3 Anchors and Boundaries

anchors

Anchors are special characters that anchor regular expressions to particular places in a string. The most common anchors are the caret ^ and the dollar sign $. The caret ^ matches the start of a line. The pattern `r"^The"` matches the word `The` only at the start of a line. Thus, the caret ^ has three uses: to match the start of a line, to indicate a negation inside of square brackets, and just to mean a caret. (What are the contexts that allow the system to know which function a given caret is supposed to have?) The dollar sign $ matches the end of a line. So the pattern ␣$ is a useful pattern for matching a space at the end of a line, and `r"^The dog\.$"` matches a line that contains only the phrase *The dog.* with a final period.

Note that we have to use the backslash in the prior example since we want the . to mean "period" and not the wildcard. By contrast, the regular expression `r"^The dog.$"` would match `The dog.` but also `The dog!` and `The dogo`. As we'll discuss below, all the special characters we've defined so far (* + ? . [ ]) need to be backslashed when we mean to use them literally.

There are other anchors: \b matches a word boundary, and \B matches a non word-boundary. Thus, `r"\bthe\b"` matches the word `the` but not the word `other`.

Regex	Match
^	start of line
$	end of line
\b	word boundary
\B	non-word boundary

Figure 2.12 Anchors in regular expressions.

A "word" for the purposes of a regex is defined (based on words in programming languages) as a sequence of digits, underscores, or letters. Thus `r"\b99\b"` will match the string `99` in `There are 99 bottles of beer on the wall` (because 99 follows a space) but not `99` in `There are 299 bottles of beer on the wall` (since 99 follows a number). But it will match `99` in `$99` (since 99 follows a dollar sign ($), which is not a digit, underscore, or letter).

Note that all these anchors and boundary operators technically match the empty string, meaning that they don't eat up any characters of the string. The carat in the pattern `r"^The"` matches the start of `"The"` but doesn't actually advance over the first character `T`. And the pattern `r"the\b the"` matches `the the`; the `\b` is aware of the fact that the space is a boundary, but it matches the empty string right before the space, not the space, so that the space character is available to be matched.

2.6.4 Disjunction, Grouping, and Precedence

Suppose we need to search for texts about pets; perhaps we are particularly interested in cats and dogs. In such a case, we might want to search for either the string *cat* or the string *dog*. Since we can't use the square brackets to search for "cat or dog" (why wouldn't `r"[catdog]"` do the right thing?), we need a new operator, the **disjunction** operator, also called the **pipe** symbol `|`. The pattern `r"cat|dog"` matches either the string `cat` or the string `dog`.

disjunction

Sometimes we need to use this disjunction operator in the midst of a larger sequence. For example, suppose I want to search for mentions of pet fish. How can I specify both *guppy* and *guppies*? We cannot simply say `r"guppy|ies"`, because that would match only the strings `guppy` and `ies`. This is because sequences like `guppy` take **precedence** over the disjunction operator `|`. To make the disjunction operator apply only to a specific pattern, we need to use the parenthesis operators `(` and `)`. Enclosing a pattern in parentheses makes it act like a single character for the purposes of neighboring operators like the pipe `|` and the Kleene*. So the pattern `r"gupp(y|ies)"` would specify that we meant the disjunction only to apply to the suffixes `y` and `ies`.

precedence

The parenthesis operator `(` is also useful when we are using counters like the Kleene*. Unlike the `|` operator, the Kleene* operator applies by default only to a single character, not to a whole sequence. Suppose we want to match repeated instances of a string. Perhaps we have a line that has column labels of the form *Column 1 Column 2 Column 3*. The expression `r"Column␣[0-9]+␣*"` will not match any number of columns; instead, it will match a single column followed by any number of spaces! The star here applies only to the space ␣ that precedes it, not to the whole sequence. With the parentheses, we could write the expression `r"(Column␣[0-9]+␣+)*"` to match the word *Column*, followed by a number and spaces, the whole pattern repeated zero or more times.

This idea that one operator may take precedence over another, requiring us to sometimes use parentheses to specify what we mean, is formalized by the **operator precedence hierarchy** for regular expressions. The following table gives the order

operator precedence

of operator precedence, from highest precedence to lowest precedence.

Parenthesis	`()`
Counters	`* + ? {}`
Sequences and anchors	`the ^my end$`
Disjunction	`|`

Thus, because counters have a higher precedence than sequences, `r"the*"` matches *theeeee* but not *thethe*. Because sequences have a higher precedence than disjunction, `r"the|any"` matches *the* or *any* but not *thany* or *theny*.

Patterns can be ambiguous in another way. Consider the expression `r"[a-z]*"` when matching against the text *once upon a time*. Since `r"[a-z]*"` matches zero or more letters, this expression could match nothing, or just the first letter *o*, *on*, *onc*, or *once*. In these cases regular expressions always match the *largest* string they can; we say that patterns are **greedy**, expanding to cover as much of a string as they can.

greedy

non-greedy

*?

+?

There are, however, ways to enforce **non-greedy** matching, using another meaning of the ? qualifier. The operator ***?** is a Kleene star that matches as little text as possible. The operator **+?** is a Kleene plus that matches as little text as possible.

2.6.5 A Simple Example

Suppose we wanted to write a regex to find cases of the English article *the*. A simple (but incorrect) pattern might be:

```
r"the"
```
(2.14)

One problem is that this pattern will miss the word when it begins a sentence and hence is capitalized (i.e., *The*). This might lead us to the following pattern:

```
r"[tT]he"
```
(2.15)

But we will still overgeneralize, incorrectly return texts with `the` embedded in other words (e.g., *other* or *there*). So we need to specify that we want instances with a word boundary on both sides:

```
r"\b[tT]he\b"
```
(2.16)

The simple process we just went through was based on fixing two kinds of errors: **false positives**, strings that we incorrectly matched like *other* or *there*, and **false negatives**, strings that we incorrectly missed, like *The*. Addressing these two kinds of errors comes up again and again in language processing. Reducing the overall error rate for an application thus involves two antagonistic efforts:

false positives

false negatives

- Increasing **precision** (minimizing false positives)
- Increasing **recall** (minimizing false negatives)

We'll come back to precision and recall with more precise definitions in Chapter 4.

2.6.6 More Operators

Figure 2.13 shows some useful aliases for common ranges:

Finally, certain special characters are referred to by special notation based on the backslash (\) (see Fig. 2.14). The most common of these are the **newline** character `\n` and the **tab** character `\t`.

newline

Regex	Expansion	Match	First Matches
\d	[0-9]	any digit	Party␣of␣5
\D	[^0-9]	any non-digit	Blue␣moon
\w	[a-zA-Z0-9_]	any alphanumeric/underscore	Daiyu
\W	[^\w]	a non-alphanumeric	!!!!
\s	[␣\r\t\n\f]	whitespace (space, tab)	in Concord
\S	[^\s]	Non-whitespace	in␣Concord

Figure 2.13 Aliases for common sets of characters.

How do we refer to characters that are special themselves (like `.`, `*`, `-`, `[`, and `\`) when we mean them literally, not in their special usage? That is, if we are trying to match a period, or a star, or a bracket or paren? To get the literal meaning of a special character, we need to precede them with a backslash, (i.e., `r"\."`, `r"\*"`, `r"\["`, and `r"\\"`).

Regex	Match	First Patterns Matched
*	an asterisk "*"	"K*A*P*L*A*N"
\.	a period "."	"Dr. Livingston, I presume"
\?	a question mark	"Why don't they come and lend a hand?"
\n	a newline	
\t	a tab	

Figure 2.14 Some characters that need to be escaped (via backslash).

2.6.7 Substitutions and Capture Groups

substitution

An important use of regular expressions is in **substitutions**, where we want to replace one string with another. Regular expression can help us specify the string to be replaced as well as the replacement. In Python we use the function `re.sub()` (similar functions exist in other languages and environments).

`re.sub`*(pattern, repl, string)* takes three arguments: a *pattern* to search for, a *replacement* to replace it with, and a *string* in which to do the search and replacing We could for example change every instance of `cherry` to `apricot` in `string`:

```
re.sub(r"cherry", r"apricot", string)
```

Or we could convert to upper case all the instances of a particular name:

```
re.sub(r"janet", r"Janet", string)
```

More often, however, the substitution depends in a more complex way on the string that matched the *pattern*. For example, suppose we have a document in which all the dates are in US format (mm/dd/yyyy) and we want to change them into the format used in the EU and many other regions: (dd-mm-yyyy). The *pattern* `r"\d{2}/\d{2}/\d{4}"` will match a date. But how do we specify in the *replacement* that we want to swap the date and month values?

capture group

The tool in regular expression for this is the **capture group**. A capture group uses parentheses to capture (*store*) the values that we matched in the search, so we can reuse them in the replacement. We put a set of parentheses around the part of the *pattern* we want to capture, and it will get stored in a numbered group (groups are numbered from left to right). Then in the *repl*, we refer back to that group with a number command.

Consider the following expression:

```
re.sub(r"(\d{2})/(\d{2})/(\d{4})", r"\2-\1-\3", string)}
```

We've put parentheses (and) around the two month digits, the two day digits, and the four year digits, thus storing the first 2 digits in group 1, the second 2 digits in group 2, and the final digits in group 3. Then in the *repl* string, we use **number** operators \1, \2, and \3, to refer back to the first, second, and third registers. The result would take a string like

```
The date is 10/15/2011
```

and convert it to

```
The date is 15-10-2011
```

Capture groups can be useful even if we are not doing substitutions. For example we can use them to find repetitions, something we often need in text processing. For example, to find a repeated word in a string, we can use this pattern which searches for a word, captures it in a group, and then refers back to it after whitespace:

```
r"\b([A-Za-z]+)\s+\1\b"
```

Parentheses thus have a double function in regular expressions; they are used to group terms for specifying the order in which operators should apply, and they are used to capture the match. Occasionally we need parentheses for grouping, but don't want to capture the resulting pattern. In that case we use a **non-capturing group**, which is specified by putting the special commands ?: after the open parenthesis, in the form (?: pattern). Non-capture groups are usually used when we are trying to capture only part of a long or complex pattern. Perhaps we are matching a sequence of dates (\d\d/\d\d/\d\d\d\d) separated by spaces and we want to extract only the 15th one. We need to use parenthesis in order to use the counting operator on the first 14, but we don't want to store all the useless information. The following pattern only stores the 15th date in group 1:

non-capturing group

```
r"(?:\d\d/\d\d/\d\d\d\d\s+){14}(\d\d/\d\d/\d\d\d\d)"    (2.17)
```

Substitutions and capture groups are also useful for implementing historically important chatbots like ELIZA (Weizenbaum, 1966). Recall that ELIZA simulates a Rogerian psychologist by carrying on conversations like the following:

$User_2$:	They're always bugging us about something or other.
$ELIZA_2$:	CAN YOU THINK OF A SPECIFIC EXAMPLE
$User_3$:	Well, my boyfriend made me come here.
$ELIZA_3$:	YOUR BOYFRIEND MADE YOU COME HERE
$User_4$:	He says I'm depressed much of the time.
$ELIZA_4$:	I AM SORRY TO HEAR YOU ARE DEPRESSED

ELIZA works by having a series or cascade of regex substitutions each of which matches and changes some part of the input lines. After the input is uppercased, substitutions change all instances of MY to YOUR, and I'M to YOU ARE, and so on. That way when ELIZA repeats back part of the user utterance, it will seem to be referring correctly to the user. The next set of substitutions matches and replaces other patterns in the input, turning the input into a complete response. Here are some examples:

```
re.sub(r".* YOU ARE (DEPRESSED|SAD) .*",r"I AM SORRY TO HEAR YOU ARE \1",input)
re.sub(r".* YOU ARE (DEPRESSED|SAD) .*",r"WHY DO YOU THINK YOU ARE \1",input)
re.sub(r".* ALWAYS .*",r"CAN YOU THINK OF A SPECIFIC EXAMPLE",input)
```

2.6.8 Lookahead Assertions

Finally, there will be times when we need to predict the future: look ahead in the text to see if some pattern matches, but not yet advance the pointer we always keep to where we are in the text, so that we can then deal with the pattern if it occurs, but if it doesn't we can check for something else instead.

lookahead

zero-width

These **lookahead** assertions make use of the `(?` syntax that we saw in the previous section for non-capture groups. The operator `(?= pattern)` is true if `pattern` occurs, but is **zero-width**, i.e. the match pointer doesn't advance, just as we saw with anchors and boundary markers like `\b`. The operator `(?! pattern)` only returns true if a pattern does not match, but again is zero-width and doesn't advance the pointer. Negative lookahead is commonly used when we are parsing some complex pattern but want to rule out a special case. For example suppose we want to capture the first word on the line, but only if it doesn't start with the letter `T`. We can use negative lookahead to do this:

```
r"^(?![tT])(\w+)\b"
```
(2.18)

The first negative lookahead says that the line must not start with a `t` or `T`, but matches the empty string, not moving the match pointer. Then the capture group captures the first word.

2.6.9 Regular Expressions for BPE pre-tokenization

```
>>> import regex as re
>>> pat = re.compile(
... # Contractions: 't and 'm are tokens
...     r"'s|'t|'re|'ve|'m|'ll|'d|"
... # Words:  sequence of Unicode letters (after optional space)
...     r" ?\p{L}+|"
... #Number: sequence of digits (after optional space)
...     r" ?\p{N}+|"
... # Punctuation: sequence of non-alphanumeric/non-space
...                         #(after optional space)
...     r" ?[^\s\p{L}\p{N}]+|"
... # whitespace
...     r"\s+(?!\S)|\s+"
... )
>>> text = "We're 350 dogs! Um, lunch?"
>>> print(pat.findall(text))
['We', "'re", ' 350', ' dogs', '!', ' Um', ',', ' lunch', '?']
>>>
```

Figure 2.15 The GPT-2 pre-tokenizer regular expression, used to split (roughly) on whitespace before running the BPE algorithm.

We described in Section 2.4 how before we run a BPE tokenization algorithm on a corpus we first pre-tokenize, splitting the corpus roughly by whitespace. Then the BPE tokenization algorithm builds up tokens from sequences of characters inside words and doesn't tokenize across word boundaries.

Here's the regular expression used to do this pretokenization that is used for one influential language model, the GPT-2 language model (Radford et al., 2019):

```
r"'s|'t|'re|'ve|'m|'ll|'d| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"
```

This is quite a complex regular expression, and also makes use of some advanced Unicode-related features we haven't described yet. These features are part of a popular external Python 3 library called `regex` (which is more powerful than the internal Python 3 library called `re`).

For example the `regex` library has special `\p` and `\P` operators that can match if the current character has particular Unicode codepoint properties. For example `\p{L}` matches any Unicode letter, `\P{L}` matches any non-letter, `\p{N}` matches any number, and `\P{N}` matches any non-number.

Fig. 2.15 annotates the regular expression more clearly, and also shows the output of running the GPT-2 tokenizer on the sentence `We're 350 dogs! Um, lunch?`.

Note that the tokenizer splits `We're` into `We` and `'re`, that punctuation is split off from `␣dogs` and `␣lunch`, and that some tokens like `␣dogs` start with a space, and others like `We` and `!` do not.

2.7 Simple Unix Tools for Word Tokenization

For English it is possible to do simple naive word tokenization and frequency computation in a single Unix command-line. As Church (1994) points out, this can be useful when we need quick information about a text corpus. We'll make use of some Unix commands: `tr`, used to systematically change particular characters in the input; `sort`, which sorts input lines in alphabetical order; and `uniq`, which collapses and counts adjacent identical lines.

For example let's begin with the 'complete words' of Shakespeare in one file, `sh.txt`. We can use `tr` to tokenize the words by changing every sequence of non-alphabetic characters to a newline ('A-Za-z' means alphabetic and the -c option complements to non-alphabet, so together they mean to change every non-alphabetic character into a newline. The -s ('squeeze') option is used to replace the result of multiple consecutive changes into a single output, so a series of non-alphabetic characters in a row would all be 'squeezed' into a single newline):

```
tr -sc 'A-Za-z' '\n' < sh.txt
```

The output of this command will be:

```
THE
SONNETS
by
William
Shakespeare
From
fairest
creatures
...
```

Now that there is one word per line, we can sort the lines, and pass them to `uniq -c` which will collapse and count them:

```
tr -sc 'A-Za-z' '\n' < sh.txt | sort | uniq -c
```

with the following output:

```
1945 A
72 AARON
19 ABBESS
25 Aaron
6 Abate
1 Abates
...
```

Alternatively, we can collapse all the upper case to lower case:

```
tr -sc 'A-Za-z' '\n' < sh.txt | tr A-Z a-z | sort | uniq -c
```

whose output is

```
14725 a
  97 aaron
   1 abaissiez
  10 abandon
   2 abandoned
   2 abase
   1 abash
  14 abate
   ...
```

Now we can sort again to find the frequent words. The `-n` option to `sort` means to sort numerically rather than alphabetically, and the `-r` option means to sort in reverse order (highest-to-lowest):

```
tr -sc 'A-Za-z' '\n' < sh.txt | tr A-Z a-z | sort | uniq -c | sort -n -r
```

The results show that the most frequent words in Shakespeare, as in any other corpus, are the short **function words** like articles, pronouns, prepositions:

```
27378 the
26084 and
22538 i
19771 to
17481 of
14725 a
13826 you
   ...
```

Unix tools of this sort can be very handy in building quick word count statistics for any corpus in English. For anything more complex, we generally turn to the more sophisticated tokenization algorithms we've discussed above.

2.8 Rule-based tokenization

While data-based tokenization like BPE is the most common way of doing tokenization, there are also situations where we want to constrain our tokens to be words and not subwords. This might be useful if we are running parsing algorithms for English where the parser might need grammatical words as input. Or it can be useful for any linguistic application where we have some a prior definition of the token that we

are interested in studying. Or it can be useful for social science applications where orthographic words are useful domains of study.

In rule-based tokenization, we pre-define a standard and implement rules to implement that kind of tokenization. Let's explore this for English word tokenization.

We have some desiderata for English. We often want to break off punctuation as a separate token; commas are a useful piece of information for parsers, and periods help indicate sentence boundaries. But we'll often want to keep the punctuation that occurs word internally, in examples like *m.p.h.*, *Ph.D.*, *AT&T*, and *cap'n*. Special characters and numbers will need to be kept in prices ($45.55) and dates (`01/02/06`); we don't want to segment that price into separate tokens of "45" and "55". And there are URLs (`https://www.stanford.edu`), Twitter hashtags (`#nlproc`), or email addresses (`someone@cs.colorado.edu`).

Number expressions introduce complications; in addition to appearing at word boundaries, commas appear inside numbers in English, every three digits: `555,500.50`. Tokenization differs by language; languages like Spanish, French, and German, for example, use a comma to mark the decimal point, and spaces (or sometimes periods) where English puts commas, for example, `555 500,50`.

clitic

A rule-based tokenizer can also be used to expand **clitic** contractions that are marked by apostrophes, converting `what're` to the two tokens `what are`, and `we're` to `we are`. A clitic is a part of a word that can't stand on its own, and can only occur when it is attached to another word. Such contractions occur in other alphabetic languages, including French pronouns (`j'ai` and articles `l'homme`).

Depending on the application, tokenization algorithms may also tokenize multiword expressions like `New York` or `rock 'n' roll` as a single token, which requires a multiword expression dictionary of some sort. Rule-based tokenization is thus intimately tied up with **named entity recognition**, the task of detecting names, dates, and organizations (Chapter 17).

Penn Treebank tokenization

One commonly used tokenization standard is known as the **Penn Treebank tokenization** standard, used for the parsed corpora (treebanks) released by the Linguistic Data Consortium (LDC), the source of many useful datasets. This standard separates out clitics (*doesn't* becomes *does* plus *n't*), keeps hyphenated words together, and separates out all punctuation (to save space we're showing visible spaces '␣' between tokens, although newlines is a more common output):

Input: `"The San Francisco-based restaurant," they said, "doesn't charge $10".`

Output: `"␣The␣San␣Francisco-based␣restaurant␣,␣"␣they␣said␣,␣"␣does␣n't␣charge␣$␣10␣"␣.`

In practice, since tokenization is run before any other language processing, it needs to be very fast. For rule-based word tokenization we generally use deterministic algorithms based on regular expressions compiled into efficient finite state automata. For example, Fig. 2.16 shows a basic regular expression that can be used to tokenize English with the `nltk.regexp_tokenize` function of the Python-based Natural Language Toolkit (NLTK) (Bird et al. 2009; `https://www.nltk.org`).

Carefully designed deterministic algorithms can deal with the ambiguities that arise, such as the fact that the apostrophe needs to be tokenized differently when used as a genitive marker (as in *the book's cover*), a quotative as in *'The other class', she said*, or in clitics like *they're*.

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)    # set flag to allow verbose regexps
...     (?:[A-Z]\.)+        # abbreviations, e.g. U.S.A.
...   | \w+(?:-\w+)*        # words with optional internal hyphens
...   | \$?\d+(?:\.\d+)?%?  # currency, percentages, e.g. $12.40, 82%
...   | \.\.\.              # ellipsis
...   | [][.,;"'?():_`-]  # these are separate tokens; includes ], [
... '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

Figure 2.16 A Python trace of regular expression tokenization in the NLTK Python-based natural language processing toolkit (Bird et al., 2009), commented for readability; the `(?x)` verbose flag tells Python to strip comments and whitespace. Figure from Chapter 3 of Bird et al. (2009).

2.8.1 Sentence Segmentation

sentence segmentation

Rule-based segmentation is commonly used for another kind of tokenization process: the sentence. **Sentence segmentation** is a step that is can be optionally applied in text processing. It is especially important when applying NLP algorithms to tasks of detecting structure, like parse structure.

Sentence segmentation depends on the language and the genre. The most useful cues for segmenting a text into sentences in English written text tend to be punctuation, like periods, question marks, and exclamation points. Question marks and exclamation points are relatively unambiguous markers of sentence boundaries, and simple rules can segment sentences when they appear.

The period character ".", on the other hand, is ambiguous between a sentence boundary marker and a marker of abbreviations like *Dr.* or *Inc.* The previous sentence that you just read showed an even more complex case of this ambiguity, in which the final period of *Inc.* marked both an abbreviation and the sentence boundary marker. For this reason, sentence tokenization and word tokenization can be addressed jointly.

Many English sentence tokenization methods work by first deciding (often based on deterministic rules, but sometimes via machine learning) whether a period is part of the word or is a sentence-boundary marker. An abbreviation dictionary can help determine whether the period is part of a commonly used abbreviation; the dictionaries can be hand-built or machine-learned (Kiss and Strunk, 2006), as can the final sentence splitter. In the Stanford CoreNLP toolkit (Manning et al., 2014), for example sentence splitting is rule-based, a deterministic consequence of tokenization; a sentence ends when a sentence-ending punctuation (., !, or ?) is not already grouped with other characters into a token (such as for an abbreviation or number), optionally followed by additional final quotes or brackets.

2.9 Minimum Edit Distance

We often need a way to compare how similar two words or strings are. As we'll see in later chapters, this comes up most commonly in tasks like automatic speech recognition or machine translation, where we want to know how similar the sequence

of words is to some reference sequence of words.

minimum edit distance

Edit distance gives us a way to quantify these intuitions about string similarity. More formally, the **minimum edit distance** between two strings is defined as the minimum number of editing operations (operations like insertion, deletion, substitution) needed to transform one string into another. In this section we'll introduce edit distance for single words, but the algorithm applies equally to entire strings.

The gap between *intention* and *execution*, for example, is 5 (delete an `i`, substitute `e` for `n`, substitute `x` for `t`, insert `c`, substitute `u` for `n`). It's much easier to see this by looking at the most important visualization for string distances, an **alignment** between the two strings, shown in Fig. 2.17. Given two sequences, an **alignment** is a correspondence between substrings of the two sequences. Thus, we say `I` **aligns** with the empty string, `N` with `E`, and so on. Beneath the aligned strings is another representation; a series of symbols expressing an **operation list** for converting the top string into the bottom string: **d** for deletion, **s** for substitution, **i** for insertion.

alignment

```
I N T E * N T I O N
| | | | | | | | | |
* E X E C U T I O N
d s s   i s
```

Figure 2.17 Representing the minimum edit distance between two strings as an **alignment**. The final row gives the operation list for converting the top string into the bottom string: d for deletion, s for substitution, i for insertion.

We can also assign a particular cost or weight to each of these operations. The **Levenshtein** distance between two sequences is the simplest weighting factor in which each of the three operations has a cost of 1 (Levenshtein, 1966)—we assume that the substitution of a letter for itself, for example, `t` for `t`, has zero cost. The Levenshtein distance between *intention* and *execution* is 5. Levenshtein also proposed an alternative version of his metric in which each insertion or deletion has a cost of 1 and substitutions are not allowed. (This is equivalent to allowing substitution, but giving each substitution a cost of 2 since any substitution can be represented by one insertion and one deletion). Using this version, the Levenshtein distance between *intention* and *execution* is 8.

2.9.1 The Minimum Edit Distance Algorithm

How do we find the minimum edit distance? We can think of this as a search task, in which we are searching for the shortest path—a sequence of edits—from one string to another.

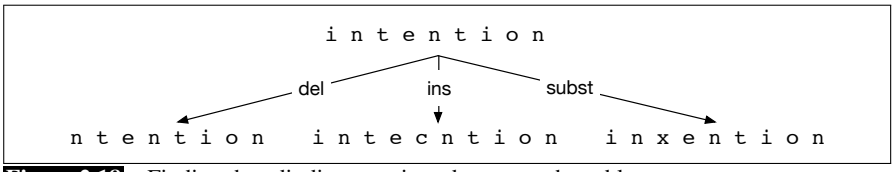


Figure 2.18 Finding the edit distance viewed as a search problem

The space of all possible edits is enormous, so we can't search naively. However, lots of distinct edit paths will end up in the same state (string), so rather than recomputing all those paths, we could just remember the shortest path to a state each time

dynamic programming

we saw it. We can do this by using **dynamic programming**. Dynamic programming is the name for a class of algorithms, first introduced by Bellman (1957), that apply a table-driven method to solve problems by combining solutions to subproblems. Some of the most commonly used algorithms in natural language processing make use of dynamic programming, such as the **Viterbi** algorithm (Chapter 17) and the **CKY** algorithm for parsing (Chapter 18).

The intuition of a dynamic programming problem is that a large problem can be solved by properly combining the solutions to various subproblems. Consider the shortest path of transformed words that represents the minimum edit distance between the strings *intention* and *execution* shown in Fig. 2.19.

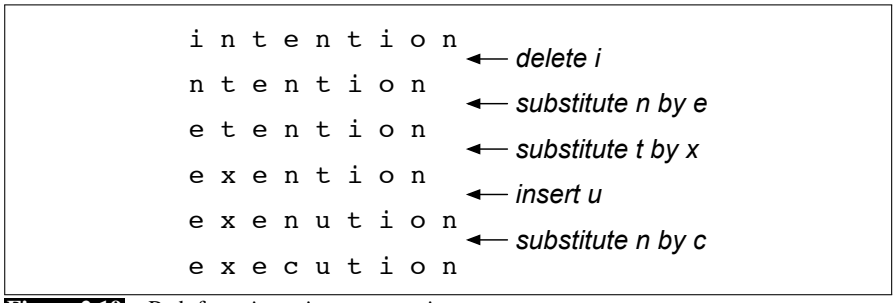


Figure 2.19 Path from *intention* to *execution*.

Imagine some string (perhaps it is *exention*) that is in this optimal path (whatever it is). The intuition of dynamic programming is that if *exention* is in the optimal operation list, then the optimal sequence must also include the optimal path from *intention* to *exention*. Why? If there were a shorter path from *intention* to *exention*, then we could use it instead, resulting in a shorter overall path, and the optimal sequence wouldn't be optimal, thus leading to a contradiction.

minimum edit distance algorithm

The **minimum edit distance algorithm** was named by Wagner and Fischer (1974) but independently discovered by many people (see the Historical Notes section of Chapter 17).

Let's first define the minimum edit distance between two strings. Given two strings, the source string X of length n, and target string Y of length m, we'll define $D[i,j]$ as the edit distance between $X[1..i]$ and $Y[1..j]$, i.e., the first i characters of X and the first j characters of Y. The edit distance between X and Y is thus $D[n,m]$.

We'll use dynamic programming to compute $D[n,m]$ bottom up, combining solutions to subproblems. In the base case, with a source substring of length i but an empty target string, going from i characters to 0 requires i deletes. With a target substring of length j but an empty source going from 0 characters to j characters requires j inserts. Having computed $D[i,j]$ for small i,j we then compute larger $D[i,j]$ based on previously computed smaller values. The value of $D[i,j]$ is computed by taking the minimum of the three possible paths through the matrix which arrive there:

$$D[i,j] = \min \begin{cases} D[i-1,j] + \text{del-cost}(source[i]) \\ D[i,j-1] + \text{ins-cost}(target[j]) \\ D[i-1,j-1] + \text{sub-cost}(source[i], target[j]) \end{cases} \tag{2.19}$$

We mentioned above two versions of Levenshtein distance, one in which substitutions cost 1 and one in which substitutions cost 2 (i.e., are equivalent to an insertion plus a deletion). Let's here use that second version of Levenshtein distance in which the insertions and deletions each have a cost of 1 (ins-cost(·) = del-cost(·) = 1), and

substitutions have a cost of 2 (except substitution of identical letters has zero cost). Under this version of Levenshtein, the computation for $D[i,j]$ becomes:

$$D[i,j] = \min \begin{cases} D[i-1,j]+1 \\ D[i,j-1]+1 \\ D[i-1,j-1]+\begin{cases} 2; & \text{if } source[i] \neq target[j] \\ 0; & \text{if } source[i] = target[j] \end{cases} \end{cases} \tag{2.20}$$

The algorithm is summarized in Fig. 2.20; Fig. 2.21 shows the results of applying the algorithm to the distance between *intention* and *execution* with the version of Levenshtein in Eq. 2.20.

```
function MIN-EDIT-DISTANCE(source, target) returns min-distance

  n ← LENGTH(source)
  m ← LENGTH(target)
  Create a distance matrix D[n+1,m+1]

  # Initialization: the zeroth row and column is the distance from the empty string
  D[0,0] = 0
  for each row i from 1 to n do
    D[i,0] ← D[i-1,0] + del-cost(source[i])
  for each column j from 1 to m do
    D[0,j] ← D[0,j-1] + ins-cost(target[j])

  # Recurrence relation:
  for each row i from 1 to n do
      for each column j from 1 to m do
        D[i,j] ← MIN( D[i−1,j] + del-cost(source[i]),
                      D[i−1,j−1] + sub-cost(source[i], target[j]),
                      D[i,j−1] + ins-cost(target[j]))
  # Termination
  return D[n,m]
```

Figure 2.20 The minimum edit distance algorithm, an example of the class of dynamic programming algorithms. The various costs can either be fixed (e.g., $\forall x, \text{ins-cost}(x) = 1$) or can be specific to the letter (to model the fact that some letters are more likely to be inserted than others). We assume that there is no cost for substituting a letter for itself (i.e., $\text{sub-cost}(x,x) = 0$).

Alignment Knowing the minimum edit distance is useful for algorithms like finding potential spelling error corrections. But the edit distance algorithm is important in another way; with a small change, it can also provide the minimum cost **alignment** between two strings. Aligning two strings is useful throughout speech and language processing. In speech recognition, minimum edit distance alignment is used to compute the word error rate (Chapter 15). Alignment plays a role in machine translation, in which sentences in a parallel corpus (a corpus with a text in two languages) need to be matched to each other.

To extend the edit distance algorithm to produce an alignment, we can start by visualizing an alignment as a path through the edit distance matrix. Figure 2.22 shows this path with boldfaced cells. Each boldfaced cell represents an alignment of a pair of letters in the two strings. If two boldfaced cells occur in the same row,

Src\Tar	#	e	x	e	c	u	t	i	o	n
#	0	1	2	3	4	5	6	7	8	9
i	1	2	3	4	5	6	7	6	7	8
n	2	3	4	5	6	7	8	7	8	7
t	3	4	5	6	7	8	7	8	9	8
e	4	3	4	5	6	7	8	9	10	9
n	5	4	5	6	7	8	9	10	11	10
t	6	5	6	7	8	9	8	9	10	11
i	7	6	7	8	9	10	9	8	9	10
o	8	7	8	9	10	11	10	9	8	9
n	9	8	9	10	11	12	11	10	9	8

Figure 2.21 Computation of minimum edit distance between *intention* and *execution* with the algorithm of Fig. 2.20, using Levenshtein distance with cost of 1 for insertions or deletions, 2 for substitutions.

there will be an insertion in going from the source to the target; two boldfaced cells in the same column indicate a deletion.

Figure 2.22 also shows the intuition of how to compute this alignment path. The computation proceeds in two steps. In the first step, we augment the minimum edit distance algorithm to store backpointers in each cell. The backpointer from a cell points to the previous cell (or cells) that we came from in entering the current cell. We've shown a schematic of these backpointers in Fig. 2.22. Some cells have multiple backpointers because the minimum extension could have come from multiple previous cells. In the second step, we perform a **backtrace**. In a backtrace, we start from the last cell (at the final row and column), and follow the pointers back through the dynamic programming matrix. Each complete path between the final cell and the initial cell is a minimum distance alignment. Exercise 2.7 asks you to modify the minimum edit distance algorithm to store the pointers and compute the backtrace to output an alignment.

backtrace

	#	e	x	e	c	u	t	i	o	n
#	0	← 1	← 2	← 3	← 4	← 5	← 6	← 7	← 8	← 9
i	↑ **1**	↖←↑ 2	↖←↑ 3	↖←↑ 4	↖←↑ 5	↖←↑ 6	↖←↑ 7	↖ 6	← 7	← 8
n	↑ 2	↖←↑ **3**	↖←↑ 4	↖←↑ 5	↖←↑ 6	↖←↑ 7	↖←↑ 8	↑ 7	↖←↑ 8	↖ 7
t	↑ 3	↖←↑ 4	↖←↑ **5**	↖←↑ 6	↖←↑ 7	↖←↑ 8	↖ 7	←↑ 8	↖←↑ 9	↑ 8
e	↑ 4	↖ 3	← 4	↖← **5**	← **6**	← 7	←↑ 8	↖←↑ 9	↖←↑ 10	↑ 9
n	↑ 5	↑ 4	↖←↑ 5	↖←↑ 6	↖←↑ 7	↖←↑ **8**	↖←↑ 9	↖←↑ 10	↖←↑ 11	↖↑ 10
t	↑ 6	↑ 5	↖←↑ 6	↖←↑ 7	↖←↑ 8	↖←↑ 9	↖ **8**	← 9	← 10	←↑ 11
i	↑ 7	↑ 6	↖←↑ 7	↖←↑ 8	↖←↑ 9	↖←↑ 10	↑ 9	↖ **8**	← 9	← 10
o	↑ 8	↑ 7	↖←↑ 8	↖←↑ 9	↖←↑ 10	↖←↑ 11	↑ 10	↑ 9	↖ **8**	← 9
n	↑ 9	↑ 8	↖←↑ 9	↖←↑ 10	↖←↑ 11	↖←↑ 12	↑ 11	↑ 10	↑ 9	↖ **8**

Figure 2.22 When entering a value in each cell, we mark which of the three neighboring cells we came from with up to three arrows. After the table is full we compute an **alignment** (minimum edit path) by using a **backtrace**, starting at the **8** in the lower-right corner and following the arrows back. The sequence of bold cells represents one possible minimum cost alignment between the two strings, again using Levenshtein distance with cost of 1 for insertions or deletions, 2 for substitutions. Diagram design after Gusfield (1997).

While we worked our example with simple Levenshtein distance, the algorithm in Fig. 2.20 allows arbitrary weights on the operations. For spelling correction, for example, substitutions are more likely to happen between letters that are next to

each other on the keyboard. The **Viterbi** algorithm is a probabilistic extension of minimum edit distance. Instead of computing the "minimum edit distance" between two strings, Viterbi computes the "maximum probability alignment" of one string with another. We'll discuss this more in Chapter 17.

2.10 Summary

This chapter introduced the fundamental concepts of tokens and tokenization in language processing. We discussed the linguistic levels of words, morphemes, and characters, introduced Unicode code points and the UTF-8 encoding, introduced the **BPE** algorithm for tokenization, and introduced the **regular expression** and the **minimum edit distance** algorithm for comparing strings. Here's a summary of the main points we covered about these ideas:

- Words and morphemes are useful units of representation, but difficult to define formally.
- **Unicode** is a system for representing characters in the many scripts used to write the languages of the world.
- Each character is represented internally with a unique id called a **code point**, and can be encoded in a file via encoding methods like **UTF-8**, which is a variable-length encoding.
- **Byte-Pair Encoding** or **BPE** is the standard way to induce tokens in a data-driven way. It is the first step in most large language models.
- **BPE** tokens are often roughly word or morpheme-sized, although they can be as small as single characters.
- The **regular expression** language is a powerful tool for pattern-matching.
- Basic operations in regular expressions include **disjunction** of symbols (`[]`, `|`), **counters** (`*`, `+`, and `{n,m}`), **anchors** (`^`, `$`), capture groups (`(,)`), and substitutions.
- The **minimum edit distance** between two strings is the minimum number of operations it takes to edit one into the other. Minimum edit distance can be computed by **dynamic programming**, which also results in an **alignment** of the two strings.

Historical Notes

For more on Herdan's law and Heaps' Law, see Herdan (1960, p. 28), Heaps (1978), Egghe (2007) and Baayen (2001);

Unicode drew on ASCII and ISO character encoding standards. Early drafts were worked out in discussions between engineers from Xerox and Apple. An early draft standard was published in 1988, with a more formal release of the Unicode Stanford in 1991. What became UTF-8 began with ISO drafts in 1989, with various extensions. The self-synchronizing aspects were famously outlined on a placemat in a New Jersey dinner in 1992 by Ken Thompson.

Word tokenization and other text normalization algorithms have been applied since the beginning of the field. This include stemming, like the widely used stemmer of Lovins (1968), and applications to the digital humanities like those of by Packard (1973), who built an affix-stripping morphological parser for Ancient Greek.

BPE, originally a text compression method proposed by Gage (1994), was applied to subword tokenization in the context of early neural machine translation by Sennrich et al. (2016). It was then taken up in OpenAI's GPT-2 (Radford et al., 2019) as the default tokenization method, and also included in the open-source SentencePiece library (Kudo and Richardson, 2018b). There is a nice a public implementation, `minbpe`, https://github.com/karpathy/minbpe, by Andrej Karpathy, who also has a popular lecture introducing BPE (https://www.youtube.com/watch?v=zduSFxRajkE).

Kleene 1951; 1956 first defined regular expressions and the finite automaton, based on the McCulloch-Pitts neuron. Ken Thompson was one of the first to build regular expressions compilers into editors for text searching (Thompson, 1968). His editor *ed* included a command "g/regular expression/p", or Global Regular Expression Print, which later became the Unix `grep` utility.

NLTK is an essential tool that offers both useful Python libraries (https://www.nltk.org) and textbook descriptions (Bird et al., 2009) of many algorithms including text normalization and corpus interfaces.

For more on edit distance, see Gusfield (1997). Our example measuring the edit distance from 'intention' to 'execution' was adapted from Kruskal (1983). There are various publicly available packages to compute edit distance, including Unix `diff` and the NIST `sclite` program (NIST, 2005).

In his autobiography Bellman (1984) explains how he originally came up with the term *dynamic programming*:

> "...The 1950s were not good years for mathematical research. [the] Secretary of Defense ...had a pathological fear and hatred of the word, research... I decided therefore to use the word, "programming". I wanted to get across the idea that this was dynamic, this was multistage... I thought, let's ... take a word that has an absolutely precise meaning, namely dynamic... it's impossible to use the word, dynamic, in a pejorative sense. Try thinking of some combination that will possibly give it a pejorative meaning. It's impossible. Thus, I thought dynamic programming was a good name. It was something not even a Congressman could object to."

Exercises

2.1 Write regular expressions for the following languages.

1. the set of all alphabetic strings;
2. the set of all lower case alphabetic strings ending in a *b*;
3. the set of all strings from the alphabet a, b such that each a is immediately preceded by and immediately followed by a b;

2.2 Write regular expressions for the following languages. By "word", we mean an alphabetic string separated from other words by whitespace, any relevant punctuation, line breaks, and so forth.

1. the set of all strings with two consecutive repeated words (e.g., "Humbert Humbert" and "the the" but not "the bug" or "the big bug");
2. all strings that start at the beginning of the line with an integer and that end at the end of the line with a word;

3. all strings that have both the word *grotto* and the word *raven* in them (but not, e.g., words like *grottos* that merely *contain* the word *grotto*);
4. write a pattern that places the first word of an English sentence in a register. Deal with punctuation.

2.3 Implement an ELIZA-like program, using substitutions such as those described on page 25. You might want to choose a different domain than a Rogerian psychologist, although keep in mind that you would need a domain in which your program can legitimately engage in a lot of simple repetition.

2.4 Compute the edit distance (using insertion cost 1, deletion cost 1, substitution cost 1) of "leda" to "deal". Show your work (using the edit distance grid).

2.5 Figure out whether *drive* is closer to *brief* or to *divers* and what the edit distance is to each. You may use any version of *distance* that you like.

2.6 Now implement a minimum edit distance algorithm and use your hand-computed results to check your code.

2.7 Augment the minimum edit distance algorithm to output an alignment; you will need to store pointers and add a stage to compute the backtrace.

CHAPTER

3 N-gram Language Models

"You are uniformly charming!" cried he, with a smile of associating and now and then I bowed and they perceived a chaise and four to wish for.
Random sentence generated from a Jane Austen trigram model

Predicting is difficult—especially about the future, as the old quip goes. But how about predicting something that seems much easier, like the next word someone is going to say? What word, for example, is likely to follow

```
The water of Walden Pond is so beautifully ...
```

You might conclude that a likely word is `blue`, or `green`, or `clear`, but probably not `refrigerator` nor `this`. In this chapter we formalize this intuition by introducing n-gram **language models** or **LMs**. A language model is a machine learning model that predicts upcoming words. More formally, a language model assigns a **probability** to each possible next word, or equivalently gives a probability distribution over possible next words. Language models can also assign a probability to an entire sentence. Thus an LM could tell us that the following sequence has a much higher probability of appearing in a text:

language model
LM

```
all of a sudden I notice three guys standing on the sidewalk
```

than does this same set of words in a different order:

```
on guys all I of notice sidewalk three a sudden standing the
```

Why would we want to predict upcoming words? The main reason is that **large language models** are built just by training them to predict words!! As we'll see in chapters 5-9, large language models learn an enormous amount about language solely from being trained to predict upcoming words from neighboring words.

This probabilistic knowledge can be very practical. Consider correcting grammar or spelling errors like `Their are two midterms`, in which `There` was mistyped as `Their`, or `Everything has improve`, in which `improve` should have been `improved`. The phrase `There are` is more probable than `Their are`, and `has improved` than `has improve`, so a language model can help users select the more grammatical variant.

Or for a speech system to recognize that you said `I will be back soonish` and not `I will be bassoon dish`, it helps to know that `back soonish` is a more probable sequence. Language models can also help in **augmentative and alternative communication** (Trnka et al. 2007, Kane et al. 2017). People can use **AAC** systems if they are physically unable to speak or sign but can instead use eye gaze or other movements to select words from a menu. Word prediction can be used to suggest likely words for the menu.

AAC

In this chapter we introduce the simplest kind of language model: the **n-gram**

n-gram

language model. An n-gram is a sequence of n words: a 2-gram (which we'll call **bigram**) is a two-word sequence of words like `The water`, or `water of`, and a 3-gram (a **trigram**) is a three-word sequence of words like `The water of`, or `water of Walden`. But we also (in a bit of terminological ambiguity) use the word 'n-gram' to mean a probabilistic model that can estimate the probability of a word given the n-1 previous words, and thereby also to assign probabilities to entire sequences.

In later chapters we will introduce the much more powerful neural **large language models**, based on the **transformer** architecture of Chapter 8. But because n-grams have a remarkably simple and clear formalization, we use them to introduce some major concepts of large language modeling, including **training and test sets**, **perplexity**, **sampling**, and **interpolation**.

3.1 N-Grams

Let's begin with the task of computing $P(w|h)$, the probability of a word w given some history h. Suppose the history h is "`The water of Walden Pond is so beautifully` " and we want to know the probability that the next word is `blue`:

$$P(\texttt{blue}|\texttt{The water of Walden Pond is so beautifully}) \tag{3.1}$$

One way to estimate this probability is directly from relative frequency counts: take a very large corpus, count the number of times we see `The water of Walden Pond is so beautifully`, and count the number of times this is followed by `blue`. This would be answering the question "Out of the times we saw the history h, how many times was it followed by the word w", as follows:

$$P(\texttt{blue}|\texttt{The water of Walden Pond is so beautifully}) = \frac{C(\texttt{The water of Walden Pond is so beautifully blue})}{C(\texttt{The water of Walden Pond is so beautifully})} \tag{3.2}$$

If we had a large enough corpus, we could compute these two counts and estimate the probability from Eq. 3.2. But even the entire web isn't big enough to give us good estimates for counts of entire sentences. This is because language is **creative**; new sentences are invented all the time, and we can't expect to get accurate counts for such large objects as entire sentences. For this reason, we'll need more clever ways to estimate the probability of a word w given a history h, or the probability of an entire word sequence W.

Let's start with some notation. First, throughout this chapter we'll continue to refer to **words**, although in practice we usually compute language models over **tokens** like the BPE tokens of page 13. To represent the probability of a particular random variable X_i taking on the value "the", or $P(X_i = \text{"the"})$, we will use the simplification $P(the)$. We'll represent a sequence of n words either as $w_1 \dots w_n$ or $w_{1:n}$. Thus the expression $w_{1:n-1}$ means the string $w_1, w_2, ..., w_{n-1}$, but we'll also be using the equivalent notation $w_{<n}$, which can be read as "all the elements of w from w_1 up to and including w_{n-1}". For the joint probability of each word in a sequence having a particular value $P(X_1 = w_1, X_2 = w_2, X_3 = w_3, ..., X_n = w_n)$ we'll use $P(w_1, w_2, ..., w_n)$.

Now, how can we compute probabilities of entire sequences like $P(w_1, w_2, ..., w_n)$? One thing we can do is decompose this probability using the **chain rule of proba-**

bility:

$$P(X_1 ... X_n) = P(X_1)P(X_2|X_1)P(X_3|X_{1:2}) \dots P(X_n|X_{1:n-1}) = \prod_{k=1}^{n} P(X_k|X_{1:k-1}) \tag{3.3}$$

Applying the chain rule to words, we get

$$P(w_{1:n}) = P(w_1)P(w_2|w_1)P(w_3|w_{1:2}) \dots P(w_n|w_{1:n-1}) = \prod_{k=1}^{n} P(w_k|w_{1:k-1}) \tag{3.4}$$

The chain rule shows the link between computing the joint probability of a sequence and computing the conditional probability of a word given previous words. Equation 3.4 suggests that we could estimate the joint probability of an entire sequence of words by multiplying together a number of conditional probabilities. But using the chain rule doesn't really seem to help us! We don't know any way to compute the exact probability of a word given a long sequence of preceding words, $P(w_n|w_{1:n-1})$. As we said above, we can't just estimate by counting the number of times every word occurs following every long string in some corpus, because language is creative and any particular context might have never occurred before!

3.1.1 The Markov assumption

The intuition of the n-gram model is that instead of computing the probability of a word given its entire history, we can **approximate** the history by just the last few words.

bigram

The **bigram** model, for example, approximates the probability of a word given all the previous words $P(w_n|w_{1:n-1})$ by using only the conditional probability given the preceding word $P(w_n|w_{n-1})$. In other words, instead of computing the probability

$$P(\texttt{blue}|\texttt{The water of Walden Pond is so beautifully}) \tag{3.5}$$

we approximate it with the probability

$$P(\texttt{blue}|\texttt{beautifully}) \tag{3.6}$$

When we use a bigram model to predict the conditional probability of the next word, we are thus making the following approximation:

$$P(w_n|w_{1:n-1}) \approx P(w_n|w_{n-1}) \tag{3.7}$$

Markov

The assumption that the probability of a word depends only on the previous word is called a **Markov** assumption. Markov models are the class of probabilistic models that assume we can predict the probability of some future unit without looking too far into the past. We can generalize the bigram (which looks one word into the past)

n-gram

to the trigram (which looks two words into the past) and thus to the **n-gram** (which looks $n-1$ words into the past).

Let's see a general equation for this n-gram approximation to the conditional probability of the next word in a sequence. We'll use N here to mean the n-gram

size, so $N = 2$ means bigrams and $N = 3$ means trigrams. Then we approximate the probability of a word given its entire context as follows:

$$P(w_n|w_{1:n-1}) \approx P(w_n|w_{n-N+1:n-1}) \tag{3.8}$$

Given the bigram assumption for the probability of an individual word, we can compute the probability of a complete word sequence by substituting Eq. 3.7 into Eq. 3.4:

$$P(w_{1:n}) \approx \prod_{k=1}^{n} P(w_k|w_{k-1}) \tag{3.9}$$

3.1.2 How to estimate probabilities

maximum likelihood estimation

normalize

How do we estimate these bigram or n-gram probabilities? An intuitive way to estimate probabilities is called **maximum likelihood estimation** or **MLE**. We get the MLE estimate for the parameters of an n-gram model by getting counts from a corpus, and **normalizing** the counts so that they lie between 0 and 1. For probabilistic models, normalizing means dividing by some total count so that the resulting probabilities fall between 0 and 1 and sum to 1.

For example, to compute a particular bigram probability of a word w_n given a previous word w_{n-1}, we'll compute the count of the bigram $C(w_{n-1}w_n)$ and normalize by the sum of all the bigrams that share the same first word w_{n-1}:

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)}{\sum_w C(w_{n-1}w)} \tag{3.10}$$

We can simplify this equation, since the sum of all bigram counts that start with a given word w_{n-1} must be equal to the unigram count for that word w_{n-1} (the reader should take a moment to be convinced of this):

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)}{C(w_{n-1})} \tag{3.11}$$

Let's work through an example using a mini-corpus of three sentences. We'll first need to augment each sentence with a special symbol <s> at the beginning of the sentence, to give us the bigram context of the first word. We'll also need a special end-symbol </s>.[1]

```
<s> I am Sam </s>
<s> Sam I am </s>
<s> I do not like green eggs and ham </s>
```

Here are the calculations for some of the bigram probabilities from this corpus

$P(\texttt{I}|\texttt{<s>}) = \frac{2}{3} = 0.67$ $P(\texttt{Sam}|\texttt{<s>}) = \frac{1}{3} = 0.33$ $P(\texttt{am}|\texttt{I}) = \frac{2}{3} = 0.67$

$P(\texttt{</s>}|\texttt{Sam}) = \frac{1}{2} = 0.5$ $P(\texttt{Sam}|\texttt{am}) = \frac{1}{2} = 0.5$ $P(\texttt{do}|\texttt{I}) = \frac{1}{3} = 0.33$

For the general case of MLE n-gram parameter estimation:

$$P(w_n|w_{n-N+1:n-1}) = \frac{C(w_{n-N+1:n-1}\ w_n)}{C(w_{n-N+1:n-1})} \tag{3.12}$$

[1] We need the end-symbol to make the bigram grammar a true probability distribution. Without an end-symbol, instead of the sentence probabilities of all sentences summing to one, the sentence probabilities for all sentences *of a given length* would sum to one. This model would define an infinite set of probability distributions, with one distribution per sentence length. See Exercise 3.5.

relative frequency

Equation 3.12 (like Eq. 3.11) estimates the n-gram probability by dividing the observed frequency of a particular sequence by the observed frequency of a prefix. This ratio is called a **relative frequency**. We said above that this use of relative frequencies as a way to estimate probabilities is an example of maximum likelihood estimation or MLE. In MLE, the resulting parameter set maximizes the likelihood of the training set T given the model M (i.e., $P(T|M)$). For example, suppose the word *Chinese* occurs 400 times in a corpus of a million words. What is the probability that a random word selected from some other text of, say, a million words will be the word *Chinese*? The MLE of its probability is $\frac{400}{1000000}$ or 0.0004. Now 0.0004 is not the best possible estimate of the probability of *Chinese* occurring in all situations; it might turn out that in some other corpus or context *Chinese* is a very unlikely word. But it is the probability that makes it *most likely* that Chinese will occur 400 times in a million-word corpus. We present ways to modify the MLE estimates slightly to get better probability estimates in Section 3.6.

Let's move on to some examples from a real but tiny corpus, drawn from the now-defunct Berkeley Restaurant Project, a dialogue system from the last century that answered questions about a database of restaurants in Berkeley, California (Jurafsky et al., 1994). Here are some sample user queries (text-normalized, by lower casing and with punctuation striped) (a sample of 9332 sentences is on the website):

can you tell me about any good cantonese restaurants close by
tell me about chez panisse
i'm looking for a good place to eat breakfast
when is caffe venezia open during the day

Figure 3.1 shows the bigram counts from part of a bigram grammar from text-normalized Berkeley Restaurant Project sentences. Note that the majority of the values are zero. In fact, we have chosen the sample words to cohere with each other; a matrix selected from a random set of eight words would be even more sparse.

	i	**want**	**to**	**eat**	**chinese**	**food**	**lunch**	**spend**
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Figure 3.1 Bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences. Zero counts are in gray. Each cell shows the count of the column label word following the row label word. Thus the cell in row **i** and column **want** means that **want** followed **i** 827 times in the corpus.

Figure 3.2 shows the bigram probabilities after normalization (dividing each cell in Fig. 3.1 by the appropriate unigram for its row, taken from the following set of unigram counts):

i	**want**	**to**	**eat**	**chinese**	**food**	**lunch**	**spend**
2533	927	2417	746	158	1093	341	278

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Figure 3.2 Bigram probabilities for eight words in the Berkeley Restaurant Project corpus of 9332 sentences. Zero probabilities are in gray.

Here are a few other useful probabilities:

$$P(\texttt{i}|\texttt{<s>}) = 0.25 \qquad P(\texttt{english}|\texttt{want}) = 0.0011$$
$$P(\texttt{food}|\texttt{english}) = 0.5 \qquad P(\texttt{</s>}|\texttt{food}) = 0.68$$

Now we can compute the probability of sentences like *I want English food* or *I want Chinese food* by simply multiplying the appropriate bigram probabilities together, as follows:

$$\begin{aligned} P(\texttt{<s> i want english food </s>}) &= P(\texttt{i}|\texttt{<s>})P(\texttt{want}|\texttt{i})P(\texttt{english}|\texttt{want}) \\ &\quad P(\texttt{food}|\texttt{english})P(\texttt{</s>}|\texttt{food}) \\ &= 0.25 \times 0.33 \times 0.0011 \times 0.5 \times 0.68 \\ &= 0.000031 \end{aligned}$$

We leave it as Exercise 3.2 to compute the probability of *i want chinese food*.

What kinds of linguistic phenomena are captured in these bigram statistics? Some of the bigram probabilities above encode some facts that we think of as strictly **syntactic** in nature, like the fact that what comes after *eat* is usually a noun or an adjective, or that what comes after *to* is usually a verb. Others might be a fact about the personal assistant task, like the high probability of sentences beginning with the words *I*. And some might even be cultural rather than linguistic, like the higher probability that people are looking for Chinese versus English food.

3.1.3 Dealing with scale in large n-gram models

In practice, language models can be very large, leading to practical issues.

log probabilities

Log probabilities Language model probabilities are always stored and computed in log space as **log probabilities**. This is because probabilities are (by definition) less than or equal to 1, and so the more probabilities we multiply together, the smaller the product becomes. Multiplying enough n-grams together would result in numerical underflow. Adding in log space is equivalent to multiplying in linear space, so we combine log probabilities by adding them. By adding log probabilities instead of multiplying probabilities, we get results that are not as small. We do all computation and storage in log space, and just convert back into probabilities if we need to report probabilities at the end by taking the exp of the logprob:

$$p_1 \times p_2 \times p_3 \times p_4 = \exp(\log p_1 + \log p_2 + \log p_3 + \log p_4) \tag{3.13}$$

In practice throughout this book, we'll use log to mean natural log (ln) when the base is not specified.

trigram
4-gram
5-gram

Longer context Although for pedagogical purposes we have only described bigram models, when there is sufficient training data we use **trigram** models, which condition on the previous two words, or **4-gram** or **5-gram** models. For these larger n-grams, we'll need to assume extra contexts to the left and right of the sentence end. For example, to compute trigram probabilities at the very beginning of the sentence, we use two pseudo-words for the first trigram (i.e., $P(\texttt{I}|\texttt{<s><s>})$).

Some large n-gram datasets have been created, like the million most frequent n-grams drawn from the Corpus of Contemporary American English (COCA), a curated 1 billion word corpus of American English (Davies, 2020), Google's Web 5-gram corpus from 1 trillion words of English web text (Franz and Brants, 2006), or the Google Books Ngrams corpora (800 billion tokens from Chinese, English, French, German, Hebrew, Italian, Russian, and Spanish) (Lin et al., 2012a)).

It's even possible to use extremely long-range n-gram context. The infini-gram (∞-gram) project (Liu et al., 2024) allows n-grams of any length. Their idea is to avoid the expensive (in space and time) pre-computation of huge n-gram count tables. Instead, n-gram probabilities with arbitrary n are computed quickly at inference time by using an efficient representation called suffix arrays. This allows computing of n-grams of every length for enormous corpora of 5 trillion tokens.

Efficiency considerations are important when building large n-gram language models. It is standard to quantize the probabilities using only 4-8 bits (instead of 8-byte floats), store the word strings on disk and represent them in memory only as a 64-bit hash, and represent n-grams in special data structures like 'reverse tries'. It is also common to prune n-gram language models, for example by only keeping n-grams with counts greater than some threshold or using entropy to prune less-important n-grams (Stolcke, 1998). Efficient language model toolkits like KenLM (Heafield 2011, Heafield et al. 2013) use sorted arrays and use merge sorts to efficiently build the probability tables in a minimal number of passes through a large corpus.

3.2 Evaluating Language Models: Training and Test Sets

extrinsic evaluation

The best way to evaluate the performance of a language model is to embed it in an application and measure how much the application improves. Such end-to-end evaluation is called **extrinsic evaluation**. Extrinsic evaluation is the only way to know if a particular improvement in the language model (or any component) is really going to help the task at hand. Thus for evaluating n-gram language models that are a component of some task like speech recognition or machine translation, we can compare the performance of two candidate language models by running the speech recognizer or machine translator twice, once with each language model, and seeing which gives the more accurate transcription.

intrinsic evaluation

Unfortunately, running big NLP systems end-to-end is often very expensive. Instead, it's helpful to have a metric that can be used to quickly evaluate potential improvements in a language model. An **intrinsic evaluation** metric is one that measures the quality of a model independent of any application. In the next section we'll introduce **perplexity**, which is the standard intrinsic metric for measuring language model performance, both for simple n-gram language models and for the more sophisticated neural large language models of Chapter 8.

training set
development set
test set

In order to evaluate any machine learning model, we need to have at least three distinct datasets: the **training set**, the **development set**, and the **test set**.

The **training set** is the data we use to learn the parameters of our model; for simple n-gram language models it's the corpus from which we get the counts that we normalize into the probabilities of the n-gram language model.

The **test set** is a different, held-out set of data, not overlapping with the training set, that we use to evaluate the model. We need a separate test set to give us an unbiased estimate of how well the model we trained can generalize when we apply it to some new unknown dataset. A machine learning model that perfectly captured the training data, but performed terribly on any other data, wouldn't be much use when it comes time to apply it to any new data or problem! We thus measure the quality of an n-gram model by its performance on this unseen test set or test corpus.

How should we choose a training and test set? The test set should reflect the language we want to use the model for. If we're going to use our language model for speech recognition of chemistry lectures, the test set should be text of chemistry lectures. If we're going to use it as part of a system for translating hotel booking requests from Chinese to English, the test set should be text of hotel booking requests. If we want our language model to be general purpose, then the test set should be drawn from a wide variety of texts. In such cases we might collect a lot of texts from different sources, and then divide it up into a training set and a test set. It's important to do the dividing carefully; if we're building a general purpose model, we don't want the test set to consist of only text from one document, or one author, since that wouldn't be a good measure of general performance.

Thus if we are given a corpus of text and want to compare the performance of two different n-gram models, we divide the data into training and test sets, and train the parameters of both models on the training set. We can then compare how well the two trained models fit the test set.

But what does it mean to "fit the test set"? The standard answer is simple: whichever language model assigns a **higher probability** to the test set—which means it more accurately predicts the test set—is a better model. Given two probabilistic models, the better model is the one that better predicts the details of the test data, and hence will assign a higher probability to the test data.

Since our evaluation metric is based on test set probability, it's important not to let the test sentences into the training set. Suppose we are trying to compute the probability of a particular "test" sentence. If our test sentence is part of the training corpus, we will mistakenly assign it an artificially high probability when it occurs in the test set. We call this situation **training on the test set** or also **data contamination**. Training on the test set introduces a bias that makes the probabilities all look too high, and causes huge inaccuracies in **perplexity**, the probability-based metric we introduce below.

data contamination

Even if we don't train on the test set, if we test our language model on the test set many times after making different changes, we might implicitly tune to its characteristics, by noticing which changes seem to make the model better. For this reason, we only want to run our model on the test set once, or a very few number of times, once we are sure our model is ready.

development test

For this reason we normally instead have a third dataset called a **development** test set or, **devset**. We do all our testing on this dataset until the very end, and then we test on the test set once to see how good our model is.

How do we divide our data into training, development, and test sets? We want our test set to be as large as possible, since a small test set may be accidentally unrepresentative, but we also want as much training data as possible. At the minimum, we would want to pick the smallest test set that gives us enough statistical power

to measure a statistically significant difference between two potential models. It's important that the devset be drawn from the same kind of text as the test set, since its goal is to measure how we would do on the test set.

3.3 Evaluating Language Models: Perplexity

We said above that we evaluate language models based on which one assigns a higher probability to the test set. A better model is better at predicting upcoming words, and so it will be less surprised by (i.e., assign a higher probability to) each word when it occurs in the test set. Indeed, a perfect language model would correctly guess each next word in a corpus, assigning it a probability of 1, and all the other words a probability of zero. So given a test corpus, a better language model will assign a higher probability to it than a worse language model.

But in fact, we often do not use raw probability as our metric for evaluating language models. The reason is that the probability of a test set (or any sequence) depends on the number of words or tokens in it; the probability of a test set gets smaller the longer the text. It's useful to have a metric that is per-word, normalized by length, so we could compare across texts of different lengths. There is a such a metric! It's a function of probability called **perplexity**, and it is used for evaluating large language models as well as n-gram models.

perplexity

The **perplexity** (sometimes abbreviated as PP or PPL) of a language model on a test set is the inverse probability of the test set (one over the probability of the test set), normalized by the number of words (or tokens). For this reason it's sometimes called the per-word or per-token perplexity. We normalize by the number of words N by taking the Nth root. For a test set $W = w_1 w_2 \dots w_N$,:

$$\begin{aligned} \text{perplexity}(W) &= P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} \\ &= \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}} \end{aligned} \tag{3.14}$$

Or we can use the chain rule to expand the probability of W:

$$\text{perplexity}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_1 \dots w_{i-1})}} \tag{3.15}$$

Note that because of the inverse in Eq. 3.15, the higher the probability of the word sequence, the lower the perplexity. Thus the **the lower the perplexity of a model on the data, the better the model**. Minimizing perplexity is equivalent to maximizing the test set probability according to the language model. Why does perplexity use the inverse probability? It turns out the inverse arises from the original definition of perplexity from cross-entropy rate in information theory; for those interested, the explanation is in the advanced Section 3.7. Meanwhile, we just have to remember that perplexity has an inverse relationship with probability.

The details of computing the perplexity of a test set W depends on which language model we use. Here's the perplexity of W with a unigram language model (just the geometric mean of the inverse of the unigram probabilities):

$$\text{perplexity}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i)}} \tag{3.16}$$

The perplexity of W computed with a bigram language model is still a geometric mean, but now of the inverse of the bigram probabilities:

$$\text{perplexity}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i|w_{i-1})}} \tag{3.17}$$

What we generally use for word sequence in Eq. 3.15 or Eq. 3.17 is the entire sequence of words in some test set. Since this sequence will cross many sentence boundaries, if our vocabulary includes a between-sentence token <EOS> or separate begin- and end-sentence markers <s> and </s> then we can include them in the probability computation. If we do, then we also include one token per sentence in the total count of word tokens N.[2]

We mentioned above that perplexity is a function of both the text and the language model: given a text W, different language models will have different perplexities. Because of this, perplexity can be used to compare different language models. For example, here we trained unigram, bigram, and trigram models on 38 million words from the *Wall Street Journal* newspaper. We then computed the perplexity of each of these models on a WSJ test set using Eq. 3.16 for unigrams, Eq. 3.17 for bigrams, and the corresponding equation for trigrams. The table below shows the perplexity of the 1.5 million word test set according to each of the language models.

	Unigram	**Bigram**	**Trigram**
Perplexity	962	170	109

As we see above, the more information the n-gram gives us about the word sequence, the higher the probability the n-gram will assign to the string. A trigram model is less surprised than a unigram model because it has a better idea of what words might come next, and so it assigns them a higher probability. And the higher the probability, the lower the perplexity (since as Eq. 3.15 showed, perplexity is related inversely to the probability of the test sequence according to the model). So a lower perplexity tells us that a language model is a better predictor of the test set.

Note that in computing perplexities, the language model must be constructed without any knowledge of the test set, or else the perplexity will be artificially low. And the perplexity of two language models is only comparable if they use identical vocabularies.

An (intrinsic) improvement in perplexity does not guarantee an (extrinsic) improvement in the performance of a language processing task like speech recognition or machine translation. Nonetheless, because perplexity usually correlates with task improvements, it is commonly used as a convenient evaluation metric. Still, when possible a model's improvement in perplexity should be confirmed by an end-to-end evaluation on a real task.

3.3.1 Perplexity as Weighted Average Branching Factor

It turns out that perplexity can also be thought of as the **weighted average branching factor** of a language. The branching factor of a language is the number of possible next words that can follow any word. For example consider a mini artificial

[2] For example if we use both begin and end tokens, we would include the end-of-sentence marker </s> but not the beginning-of-sentence marker <s> in our count of N; This is because the end-sentence token is followed directly by the begin-sentence token with probability almost 1, so we don't want the probability of that fake transition to influence our perplexity.

language that is deterministic (no probabilities), any word can follow any word, and whose vocabulary consists of only three colors:

$$L = \{\texttt{red}, \texttt{blue}, \texttt{green}\} \tag{3.18}$$

The branching factor of this language is 3.

Now let's make a probabilistic version of the same LM, let's call it A, where each word follows each other with equal probability $\frac{1}{3}$ (it was trained on a training set with equal counts for the 3 colors), and a test set T = "`red red red red blue`".

Let's first convince ourselves that if we compute the perplexity of this artificial color language on this test set (or any such test set) we indeed get 3. By Eq. 3.15, the perplexity of A on T is:

$$\begin{aligned} \text{perplexity}_A(T) &= P_A(\texttt{red red red red blue})^{-\frac{1}{5}} \\ &= \left(\left(\frac{1}{3}\right)^5\right)^{-\frac{1}{5}} \\ &= \left(\frac{1}{3}\right)^{-1} = 3 \end{aligned} \tag{3.19}$$

But now suppose `red` was very likely in the training set of a different LM B, and so B has the following probabilities:

$$P(\texttt{red}) = 0.8 \quad P(\texttt{green}) = 0.1 \quad P(\texttt{blue}) = 0.1 \tag{3.20}$$

We should expect the perplexity of the same test set `red red red red blue` for language model B to be lower since most of the time the next color will be red, which is very predictable, i.e. has a high probability. So the probability of the test set will be higher, and since perplexity is inversely related to probability, the perplexity will be lower. Thus, although the branching factor is still 3, the perplexity or *weighted* branching factor is smaller:

$$\begin{aligned} \text{perplexity}_B(T) &= P_B(\texttt{red red red red blue})^{-1/5} \\ &= 0.04096^{-\frac{1}{5}} \\ &= 0.527^{-1} = 1.89 \end{aligned} \tag{3.21}$$

3.4 Sampling sentences from a language model

sampling

One important way to visualize what kind of knowledge a language model embodies is to sample from it. **Sampling** from a distribution means to choose random points according to their likelihood. Thus sampling from a language model—which represents a distribution over sentences—means to generate some sentences, choosing each sentence according to its likelihood as defined by the model. Thus we are more likely to generate sentences that the model thinks have a high probability and less likely to generate sentences that the model thinks have a low probability.

This technique of visualizing a language model by sampling was first suggested very early on by Shannon (1948) and Miller and Selfridge (1950). It's simplest to visualize how this works for the unigram case. Imagine all the words of the English language covering the number line between 0 and 1, each word covering an interval

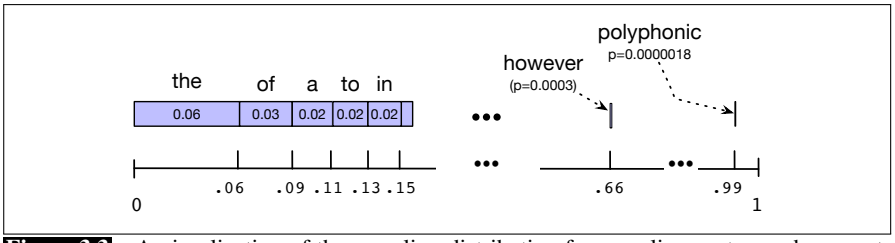


Figure 3.3 A visualization of the sampling distribution for sampling sentences by repeatedly sampling unigrams. The blue bar represents the relative frequency of each word (we've ordered them from most frequent to least frequent, but the choice of order is arbitrary). The number line shows the cumulative probabilities. If we choose a random number between 0 and 1, it will fall in an interval corresponding to some word. The expectation for the random number to fall in the larger intervals of one of the frequent words (*the*, *of*, *a*) is much higher than in the smaller interval of one of the rare words (*polyphonic*).

proportional to its frequency. Fig. 3.3 shows a visualization, using a unigram LM computed from the text of this book. We choose a random value between 0 and 1, find that point on the probability line, and print the word whose interval includes this chosen value. We continue choosing random numbers and generating words until we randomly generate the sentence-final token </s>.

We can use the same technique to generate bigrams by first generating a random bigram that starts with <s> (according to its bigram probability). Let's say the second word of that bigram is w. We next choose a random bigram starting with w (again, drawn according to its bigram probability), and so on.

3.5 Generalizing vs. overfitting the training set

The n-gram model, like many statistical models, is dependent on the training corpus. One implication of this is that the probabilities often encode specific facts about a given training corpus. Another implication is that n-grams do a better and better job of modeling the training corpus as we increase the value of n.

We can use the sampling method from the prior section to visualize both of these facts! To give an intuition for the increasing power of higher-order n-grams, Fig. 3.4 shows random sentences generated from unigram, bigram, trigram, and 4-gram models trained on Shakespeare's works.

The longer the context, the more coherent the sentences. The unigram sentences show no coherent relation between words nor any sentence-final punctuation. The bigram sentences have some local word-to-word coherence (especially considering punctuation as words). The trigram sentences are beginning to look a lot like Shakespeare. Indeed, the 4-gram sentences look a little too much like Shakespeare. The words *It cannot be but so* are directly from *King John*. This is because, not to put the knock on Shakespeare, his oeuvre is not very large as corpora go ($N = 884,647, V = 29,066$), and our n-gram probability matrices are ridiculously sparse. There are $V^2 = 844,000,000$ possible bigrams alone, and the number of possible 4-grams is $V^4 = 7 \times 10^{17}$. Thus, once the generator has chosen the first 3-gram (*It cannot be*), there are only seven possible next words for the 4th element (*but*, *I*, *that*, *thus*, *this*, and the period).

To get an idea of the dependence on the training set, let's look at LMs trained on a completely different corpus: the *Wall Street Journal* (WSJ) newspaper. Shakespeare

1 gram	–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have
	–Hill he late speaks; or! a more to leg less first you enter
2 gram	–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.
	–What means, sir. I confess she? then all sorts, he is trim, captain.
3 gram	–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.
	–This shall forbid it should be branded, if renown made it empty.
4 gram	–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;
	–It cannot be but so.

Figure 3.4 Eight sentences randomly generated from four n-gram models computed from Shakespeare's works. All characters were mapped to lower-case and punctuation marks were treated as words. Output is hand-corrected for capitalization to improve readability.

and the WSJ are both English, so we might have expected some overlap between our n-grams for the two genres. Fig. 3.5 shows sentences generated by unigram, bigram, and trigram models trained on 40 million words from WSJ.

1 gram	Months the my and issue of year foreign new exchange's september were recession exchange new endorsed a acquire to six executives
2 gram	Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor would seem to complete the major central planners one point five percent of U. S. E. has already old M. X. corporation of living on information such as more frequently fishing to keep her
3 gram	They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions

Figure 3.5 Three sentences randomly generated from three n-gram models computed from 40 million words of the *Wall Street Journal*, lower-casing all characters and treating punctuation as words. Output was then hand-corrected for capitalization to improve readability.

Compare these examples to the pseudo-Shakespeare in Fig. 3.4. While they both model "English-like sentences", there is no overlap in the generated sentences, and little overlap even in small phrases. Statistical models are pretty useless as predictors if the training sets and the test sets are as different as Shakespeare and the WSJ.

How should we deal with this problem when we build n-gram models? One step is to be sure to use a training corpus that has a similar **genre** to whatever task we are trying to accomplish. To build a language model for translating legal documents, we need a training corpus of legal documents. To build a language model for a question-answering system, we need a training corpus of questions.

It is equally important to get training data in the appropriate **dialect** or **variety**, especially when processing social media posts or spoken transcripts. For example some tweets will use features of African American English (AAE)— the name for the many variations of language used in African American communities (King, 2020). Such features can include words like *finna*—an auxiliary verb that marks immediate future tense —that don't occur in other varieties, or spellings like *den* for *then*, in tweets like this one (Blodgett and O'Connor, 2017):

(3.22) Bored af den my phone finna die!!!

while tweets from English-based languages like Nigerian Pidgin have markedly different vocabulary and n-gram patterns from American English (Jurgens et al., 2017):

(3.23) @username R u a wizard or wat gan sef: in d mornin - u tweet, afternoon - u tweet, nyt gan u dey tweet. beta get ur IT placement wiv twitter

Is it possible for the testset nonetheless to have a word we have never seen before? What happens if the word *Jurafsky* never occurs in our training set, but pops up in the test set? The answer is that although words might be unseen, we normally run our NLP algorithms not on words but on **subword tokens**. With subword tokenization (like the BPE algorithm of Chapter 2) any word can be modeled as a sequence of known smaller subwords, if necessary by a sequence of tokens corresponding to individual letters. So although for convenience we've been referring to words in this chapter, the language model vocabulary is normally the set of tokens rather than words, and in this way the test set can never contain unseen tokens.

3.6 Smoothing, Interpolation, and Backoff

There is a problem with using maximum likelihood estimates for probabilities: any finite training corpus will be missing some perfectly acceptable English word sequences. That is, cases where a particular n-gram never occurs in the training data but appears in the test set. Perhaps our training corpus has the words `ruby` and `slippers` in it but just happens not to have the phrase `ruby slippers`.

zeros

These unseen sequences or **zeros**—sequences that don't occur in the training set but do occur in the test set—are a problem for two reasons. First, their presence means we are underestimating the probability of word sequences that might occur, which hurts the performance of any application we want to run on this data. Second, if the probability of any word in the test set is 0, the probability of the whole test set is 0. Perplexity is defined based on the inverse probability of the test set. Thus if some words in context have zero probability, we can't compute perplexity at all, since we can't divide by zero!

smoothing **discounting**

The standard way to deal with putative "zero probability n-grams" that should really have some non-zero probability is called **smoothing** or **discounting**. Smoothing algorithms shave off a bit of probability mass from some more frequent events and give it to unseen events. Here we'll introduce some simple smoothing algorithms: **Laplace (add-one) smoothing**, **stupid backoff**, and n-gram **interpolation**.

3.6.1 Laplace Smoothing

Laplace smoothing

The simplest way to do smoothing is to add one to all the n-gram counts, before we normalize them into probabilities. All the counts that used to be zero will now have a count of 1, the counts of 1 will be 2, and so on. This algorithm is called **Laplace smoothing**. Laplace smoothing does not perform well enough to be used in modern n-gram models, but it usefully introduces many of the concepts that we see in other smoothing algorithms, gives a useful baseline, and is also a practical smoothing algorithm for other tasks like **text classification** (Appendix K).

Let's start with the application of Laplace smoothing to unigram probabilities. Recall that the unsmoothed maximum likelihood estimate of the unigram probability

of the word w_i is its count c_i normalized by the total number of word tokens N:

$$P(w_i) = \frac{c_i}{N}$$

add-one

Laplace smoothing merely adds one to each count (hence its alternate name **add-one** smoothing). Since there are V words in the vocabulary and each one was incremented, we also need to adjust the denominator to take into account the extra V observations. (What happens to our P values if we don't increase the denominator?)

$$P_{\text{Laplace}}(w_i) = \frac{c_i + 1}{N + V} \tag{3.24}$$

Now that we have the intuition for the unigram case, let's smooth our Berkeley Restaurant Project bigrams. Figure 3.6 shows the add-one smoothed counts for the bigrams in Fig. 3.1.

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Figure 3.6 Add-one smoothed bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences. Previously-zero counts are in gray.

Figure 3.7 shows the add-one smoothed probabilities for the bigrams in Fig. 3.2, computed by Eq. 3.26 below. Recall that normal bigram probabilities are computed by normalizing each row of counts by the unigram count:

$$P_{\text{MLE}}(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)}{C(w_{n-1})} \tag{3.25}$$

For add-one smoothed bigram counts, we need to augment the unigram count in the denominator by the number of total word types in the vocabulary V. We can see why this is in the following equation, which makes it explicit that the unigram count in the denominator is really the sum over all the bigrams that start with w_{n-1}. Since we add one to each of these, and there are V of them, we add a total of V to the denominator:

$$P_{\text{Laplace}}(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n) + 1}{\sum_w (C(w_{n-1}w) + 1)} = \frac{C(w_{n-1}w_n) + 1}{C(w_{n-1}) + V} \tag{3.26}$$

Thus, each of the unigram counts given on page 42 will need to be augmented by $V = 1446$. The result, using Eq. 3.26, is the smoothed bigram probabilities in Fig. 3.7.

One useful visualization technique is to reconstruct an **adjusted count matrix** so we can see how much a smoothing algorithm has changed the original counts. This adjusted count C^* is the count that, if divided by $C(w_{n-1})$, would result in the smoothed probability. This adjusted count is easier to compare directly with the MLE counts. That is, the Laplace probability can equally be expressed as the adjusted count divided by the (non-smoothed) denominator from Eq. 3.25:

$$P_{\text{Laplace}}(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n) + 1}{C(w_{n-1}) + V} = \frac{C^*(w_{n-1}w_n)}{C(w_{n-1})}$$

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

Figure 3.7 Add-one smoothed bigram probabilities for eight of the words (out of $V = 1446$) in the BeRP corpus of 9332 sentences computed by Eq. 3.26. Previously-zero probabilities are in gray.

Rearranging terms, we can solve for $C^*(w_{n-1}w_n)$:

$$C^*(w_{n-1}w_n) = \frac{[C(w_{n-1}w_n)+1] \times C(w_{n-1})}{C(w_{n-1})+V} \tag{3.27}$$

Figure 3.8 shows the reconstructed counts, computed by Eq. 3.27.

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16

Figure 3.8 Add-one reconstituted counts for eight words (of $V = 1446$) in the BeRP corpus of 9332 sentences, computed by Eq. 3.27. Previously-zero counts are in gray.

Note that add-one smoothing has made a very big change to the counts. Comparing Fig. 3.8 to the original counts in Fig. 3.1, we can see that $C(want\ to)$ changed from 608 to 238. We can see this in probability space as well: $P(to|want)$ decreases from 0.66 in the unsmoothed case to 0.26 in the smoothed case. Looking at the **discount** d, defined as the ratio between new and old counts, shows us how strikingly the counts for each prefix word have been reduced; the discount for the bigram *want to* is 0.39, while the discount for *Chinese food* is 0.10, a factor of 10. The sharp change occurs because too much probability mass is moved to all the zeros.

3.6.2 Add-k smoothing

One alternative to add-one smoothing is to move a bit less of the probability mass from the seen to the unseen events. Instead of adding 1 to each count, we add a fractional count k (0.5? 0.01?). This algorithm is therefore called **add-k smoothing**.

add-k

$$P^*_{\text{Add-k}}(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)+k}{C(w_{n-1})+kV} \tag{3.28}$$

Add-k smoothing requires that we have a method for choosing k; this can be done, for example, by optimizing on a **devset**. Although add-k is useful for some tasks (including text classification), it turns out that it still doesn't work well for

language modeling, generating counts with poor variances and often inappropriate discounts (Gale and Church, 1994).

3.6.3 Language Model Interpolation

There is an alternative source of knowledge we can draw on to solve the problem of zero frequency n-grams. If we are trying to compute $P(w_n|w_{n-2}w_{n-1})$ but we have no examples of a particular trigram $w_{n-2}w_{n-1}w_n$, we can instead estimate its probability by using the bigram probability $P(w_n|w_{n-1})$. Similarly, if we don't have counts to compute $P(w_n|w_{n-1})$, we can look to the unigram $P(w_n)$. In other words, sometimes using **less context** can help us generalize more for contexts that the model hasn't learned much about.

interpolation

The most common way to use this n-gram hierarchy is called **interpolation**: computing a new probability by interpolating (weighting and combining) the trigram, bigram, and unigram probabilities. In simple linear interpolation, we combine different order n-grams by linearly interpolating them. Thus, we estimate the trigram probability $P(w_n|w_{n-2}w_{n-1})$ by mixing together the unigram, bigram, and trigram probabilities, each weighted by a λ:

$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) &= \lambda_1 P(w_n) \\ &\quad +\lambda_2 P(w_n|w_{n-1}) \\ &\quad +\lambda_3 P(w_n|w_{n-2}w_{n-1})\end{aligned} \tag{3.29}$$

The λs must sum to 1, making Eq. 3.29 equivalent to a weighted average. In a slightly more sophisticated version of linear interpolation, each λ weight is computed by conditioning on the context. This way, if we have particularly accurate counts for a particular bigram, we assume that the counts of the trigrams based on this bigram will be more trustworthy, so we can make the λs for those trigrams higher and thus give that trigram more weight in the interpolation. Equation 3.30 shows the equation for interpolation with context-conditioned weights, where each *lambda* takes an argument that is the two prior word context:

$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) &= \lambda_1(w_{n-2:n-1})P(w_n) \\ &\quad +\lambda_2(w_{n-2:n-1})P(w_n|w_{n-1}) \\ &\quad +\lambda_3(w_{n-2:n-1})P(w_n|w_{n-2}w_{n-1})\end{aligned} \tag{3.30}$$

held-out

How are these λ values set? Both the simple interpolation and conditional interpolation λs are learned from a **held-out** corpus. A held-out corpus is an additional training corpus, so-called because we hold it out from the training data, that we use to set these λ values.[3] We do so by choosing the λ values that maximize the likelihood of the held-out corpus. That is, we fix the n-gram probabilities and then search for the λ values that—when plugged into Eq. 3.29—give us the highest probability of the held-out set. There are various ways to find this optimal set of λs. One way is to use the **EM** algorithm, an iterative learning algorithm that converges on locally optimal λs (Jelinek and Mercer, 1980).

3.6.4 Stupid Backoff

backoff

An alternative to interpolation is **backoff**. In a backoff model, if the n-gram we need

[3] Held-out corpora are generally used to set **hyperparameters**, which are special parameters, unlike regular counts that are learned from the training data; we'll discuss hyperparameters in Chapter 6.

has zero counts, we approximate it by backing off to the (n-1)-gram. We continue backing off until we reach a history that has some counts. For a backoff model to give a correct probability distribution, we have to **discount** the higher-order n-grams to save some probability mass for the lower order n-grams. In practice, instead of discounting, it's common to use a much simpler non-discounted backoff algorithm called **stupid backoff** (Brants et al., 2007).

discount

stupid backoff

Stupid backoff gives up the idea of trying to make the language model a true probability distribution. There is no discounting of the higher-order probabilities. If a higher-order n-gram has a zero count, we simply backoff to a lower order n-gram, weighed by a fixed (context-independent) weight. This algorithm does not produce a probability distribution, so we'll follow Brants et al. (2007) in referring to it as S:

$$S(w_i|w_{i-N+1:i-1}) = \begin{cases} \dfrac{\text{count}(w_{i-N+1:i})}{\text{count}(w_{i-N+1:i-1})} & \text{if count}(w_{i-N+1:i}) > 0 \\ \lambda S(w_i|w_{i-N+2:i-1}) & \text{otherwise} \end{cases} \tag{3.31}$$

The backoff terminates in the unigram, which has score $S(w) = \frac{count(w)}{N}$. Brants et al. (2007) find that a value of 0.4 worked well for λ.

3.7 Advanced: Perplexity's Relation to Entropy

We introduced perplexity in Section 3.3 as a way to evaluate n-gram models on a test set. A better n-gram model is one that assigns a higher probability to the test data, and perplexity is a normalized version of the probability of the test set. The perplexity measure actually arises from the information-theoretic concept of cross-entropy, which explains otherwise mysterious properties of perplexity (why the inverse probability, for example?) and its relationship to entropy. **Entropy** is a measure of information. Given a random variable X ranging over whatever we are predicting (words, letters, parts of speech), the set of which we'll call χ, and with a particular probability function, call it $p(x)$, the entropy of the random variable X is:

Entropy

$$H(X) = -\sum_{x \in \chi} p(x) \log_2 p(x) \tag{3.32}$$

The log can, in principle, be computed in any base. If we use log base 2, the resulting value of entropy will be measured in **bits**.

One intuitive way to think about entropy is as a lower bound on the number of bits it would take to encode a certain decision or piece of information in the optimal coding scheme. Consider an example from the standard information theory textbook Cover and Thomas (1991). Imagine that we want to place a bet on a horse race but it is too far to go all the way to Yonkers Racetrack, so we'd like to send a short message to the bookie to tell him which of the eight horses to bet on. One way to encode this message is just to use the binary representation of the horse's number as the code; thus, horse 1 would be `001`, horse 2 `010`, horse 3 `011`, and so on, with horse 8 coded as `000`. If we spend the whole day betting and each horse is coded with 3 bits, on average we would be sending 3 bits per race.

Can we do better? Suppose that the spread is the actual distribution of the bets placed and that we represent it as the prior probability of each horse as follows:

Horse 1	$\frac{1}{2}$	Horse 5	$\frac{1}{64}$
Horse 2	$\frac{1}{4}$	Horse 6	$\frac{1}{64}$
Horse 3	$\frac{1}{8}$	Horse 7	$\frac{1}{64}$
Horse 4	$\frac{1}{16}$	Horse 8	$\frac{1}{64}$

The entropy of the random variable X that ranges over horses gives us a lower bound on the number of bits and is

$$\begin{aligned} H(X) &= -\sum_{i=1}^{i=8} p(i)\log_2 p(i) \\ &= -\tfrac{1}{2}\log_2\tfrac{1}{2} - \tfrac{1}{4}\log_2\tfrac{1}{4} - \tfrac{1}{8}\log_2\tfrac{1}{8} - \tfrac{1}{16}\log_2\tfrac{1}{16} - 4(\tfrac{1}{64}\log_2\tfrac{1}{64}) \\ &= 2 \text{ bits} \end{aligned} \tag{3.33}$$

A code that averages 2 bits per race can be built with short encodings for more probable horses, and longer encodings for less probable horses. For example, we could encode the most likely horse with the code `0`, and the remaining horses as `10`, then `110`, `1110`, `111100`, `111101`, `111110`, and `111111`.

What if the horses are equally likely? We saw above that if we used an equal-length binary code for the horse numbers, each horse took 3 bits to code, so the average was 3. Is the entropy the same? In this case each horse would have a probability of $\frac{1}{8}$. The entropy of the choice of horses is then

$$H(X) = -\sum_{i=1}^{i=8} \frac{1}{8}\log_2\frac{1}{8} = -\log_2\frac{1}{8} = 3 \text{ bits} \tag{3.34}$$

Until now we have been computing the entropy of a single variable. But most of what we will use entropy for involves *sequences*. For a grammar, for example, we will be computing the entropy of some sequence of words $W = \{w_1, w_2, \ldots, w_n\}$. One way to do this is to have a variable that ranges over sequences of words. For example we can compute the entropy of a random variable that ranges over all sequences of words of length n in some language L as follows:

$$H(w_1, w_2, \ldots, w_n) = -\sum_{w_{1:n}\in L} p(w_{1:n})\log p(w_{1:n}) \tag{3.35}$$

entropy rate

We could define the **entropy rate** (we could also think of this as the **per-word entropy**) as the entropy of this sequence divided by the number of words:

$$\frac{1}{n}H(w_{1:n}) = -\frac{1}{n}\sum_{w_{1:n}\in L} p(w_{1:n})\log p(w_{1:n}) \tag{3.36}$$

But to measure the true entropy of a language, we need to consider sequences of infinite length. If we think of a language as a stochastic process L that produces a sequence of words, and allow W to represent the sequence of words $w_1, \ldots, w_n$, then L's entropy rate $H(L)$ is defined as

$$\begin{aligned} H(L) &= \lim_{n\to\infty}\frac{1}{n}H(w_{1:n}) \\ &= -\lim_{n\to\infty}\frac{1}{n}\sum_{W\in L} p(w_{1:n})\log p(w_{1:n}) \end{aligned} \tag{3.37}$$

The Shannon-McMillan-Breiman theorem (Algoet and Cover 1988, Cover and Thomas 1991) states that if the language is regular in certain ways (to be exact, if it is both stationary and ergodic),

$$H(L) = \lim_{n\to\infty} -\frac{1}{n}\log p(w_{1:n}) \tag{3.38}$$

That is, we can take a single sequence that is long enough instead of summing over all possible sequences. The intuition of the Shannon-McMillan-Breiman theorem is that a long-enough sequence of words will contain in it many other shorter sequences and that each of these shorter sequences will reoccur in the longer sequence according to their probabilities.

Stationary

A stochastic process is said to be **stationary** if the probabilities it assigns to a sequence are invariant with respect to shifts in the time index. In other words, the probability distribution for words at time t is the same as the probability distribution at time $t+1$. Markov models, and hence n-grams, are stationary. For example, in a bigram, P_i is dependent only on P_{i-1}. So if we shift our time index by x, P_{i+x} is still dependent on P_{i+x-1}. But natural language is not stationary, since as we show in Appendix D, the probability of upcoming words can be dependent on events that were arbitrarily distant and time dependent. Thus, our statistical models only give an approximation to the correct distributions and entropies of natural language.

To summarize, by making some incorrect but convenient simplifying assumptions, we can compute the entropy of some stochastic process by taking a very long sample of the output and computing its average log probability.

cross-entropy

Now we are ready to introduce **cross-entropy**. The cross-entropy is useful when we don't know the actual probability distribution p that generated some data. It allows us to use some m, which is a model of p (i.e., an approximation to p). The cross-entropy of m on p is defined by

$$H(p,m) = \lim_{n\to\infty} -\frac{1}{n}\sum_{W\in L} p(w_1,\ldots,w_n)\log m(w_1,\ldots,w_n) \tag{3.39}$$

That is, we draw sequences according to the probability distribution p, but sum the log of their probabilities according to m.

Again, following the Shannon-McMillan-Breiman theorem, for a stationary ergodic process:

$$H(p,m) = \lim_{n\to\infty} -\frac{1}{n}\log m(w_1 w_2 \ldots w_n) \tag{3.40}$$

This means that, as for entropy, we can estimate the cross-entropy of a model m on some distribution p by taking a single sequence that is long enough instead of summing over all possible sequences.

What makes the cross-entropy useful is that the cross-entropy $H(p,m)$ is an upper bound on the entropy $H(p)$. For any model m:

$$H(p) \leq H(p,m) \tag{3.41}$$

This means that we can use some simplified model m to help estimate the true entropy of a sequence of symbols drawn according to probability p. The more accurate m is, the closer the cross-entropy $H(p,m)$ will be to the true entropy $H(p)$. Thus, the difference between $H(p,m)$ and $H(p)$ is a measure of how accurate a model is. Between two models m_1 and m_2, the more accurate model will be the one with the

lower cross-entropy. (The cross-entropy can never be lower than the true entropy, so a model cannot err by underestimating the true entropy.)

We are finally ready to see the relation between perplexity and cross-entropy as we saw it in Eq. 3.40. Cross-entropy is defined in the limit as the length of the observed word sequence goes to infinity. We approximate this cross-entropy by relying on a (sufficiently long) sequence of fixed length. This approximation to the cross-entropy of a model $M = P(w_i|w_{i-N+1:i-1})$ on a sequence of words W is

$$H(W) = -\frac{1}{N}\log P(w_1 w_2 \ldots w_N) \tag{3.42}$$

perplexity

The **perplexity** of a model P on a sequence of words W is now formally defined as 2 raised to the power of this cross-entropy:

$$\begin{aligned}\text{Perplexity}(W) &= 2^{H(W)} \\ &= P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}} \\ &= \sqrt[N]{\frac{1}{P(w_1 w_2 \ldots w_N)}}\end{aligned}$$

3.8 Summary

This chapter introduced language modeling via the n-gram model, a classic model that allows us to introduce many of the basic concepts in language modeling.

- Language models offer a way to assign a probability to a sentence or other sequence of words or tokens, and to predict a word or token from preceding words or tokens.
- **N-grams** are perhaps the simplest kind of language model. They are Markov models that estimate words from a fixed window of previous words. N-gram models can be trained by counting in a **training corpus** and normalizing the counts (the **maximum likelihood estimate**).
- N-gram **language models** can be evaluated on a **test set** using **perplexity.**
- The **perplexity** of a test set according to a language model is a function of the probability of the test set: the inverse test set probability according to the model, normalized by the length.
- **Sampling** from a language model means to generate some sentences, choosing each sentence according to its likelihood as defined by the model.
- **Smoothing** algorithms provide a way to estimate probabilities for events that were unseen in training. Commonly used smoothing algorithms for n-grams include add-1 smoothing, or rely on lower-order n-gram counts through **interpolation**.

Historical Notes

The underlying mathematics of the n-gram was first proposed by Markov (1913), who used what are now called **Markov chains** (bigrams and trigrams) to predict whether an upcoming letter in Pushkin's *Eugene Onegin* would be a vowel or a consonant. Markov classified 20,000 letters as V or C and computed the bigram and

trigram probability that a given letter would be a vowel given the previous one or two letters. Shannon (1948) applied n-grams to compute approximations to English word sequences. Based on Shannon's work, Markov models were commonly used in engineering, linguistic, and psychological work on modeling word sequences by the 1950s. In a series of extremely influential papers starting with Chomsky (1956) and including Chomsky (1957) and Miller and Chomsky (1963), Noam Chomsky argued that "finite-state Markov processes", while a possibly useful engineering heuristic, were incapable of being a complete cognitive model of human grammatical knowledge. These arguments led many linguists and computational linguists to ignore work in statistical modeling for decades.

The resurgence of n-gram language models came from Fred Jelinek and colleagues at the IBM Thomas J. Watson Research Center, who were influenced by Shannon, and James Baker at CMU, who was influenced by the prior, classified work of Leonard Baum and colleagues on these topics at labs like the US Institute for Defense Analyses (IDA) after they were declassified. Independently these two labs successfully used n-grams in their speech recognition systems at the same time (Baker 1975b, Jelinek et al. 1975, Baker 1975a, Bahl et al. 1983, Jelinek 1990). The terms "language model" and "perplexity" were first used for this technology by the IBM group. Jelinek and his colleagues used the term *language model* in a pretty modern way, to mean the entire set of linguistic influences on word sequence probabilities, including grammar, semantics, discourse, and even speaker characteristics, rather than just the particular n-gram model itself.

Add-one smoothing derives from Laplace's 1812 law of succession and was first applied as an engineering solution to the zero frequency problem by Jeffreys (1948) based on an earlier Add-K suggestion by Johnson (1932). Problems with the add-one algorithm are summarized in Gale and Church (1994).

A wide variety of different language modeling and smoothing techniques were proposed in the 80s and 90s, including Good-Turing discounting—first applied to the n-gram smoothing at IBM by Katz (Nádas 1984, Church and Gale 1991)— Witten-Bell discounting (Witten and Bell, 1991), and varieties of **class-based n-gram** models that used information about word classes. Starting in the late 1990s, Chen and Goodman performed a number of carefully controlled experiments comparing different algorithms and parameters (Chen and Goodman 1999, Goodman 2006, inter alia). They showed the advantages of **Modified Interpolated Kneser-Ney**, which became the standard baseline for n-gram language modeling around the turn of the century, especially because they showed that caches and class-based models provided only minor additional improvement. SRILM (Stolcke, 2002) and KenLM (Heafield 2011, Heafield et al. 2013) are publicly available toolkits for building n-gram language models.

class-based n-gram

Large language models are based on **neural networks** rather than n-grams, enabling them to solve the two major problems with n-grams: (1) the number of parameters increases exponentially as the n-gram order increases, and (2) n-grams have no way to generalize from training examples to test set examples unless they use identical words. Neural language models instead project words into a **continuous** space in which words with similar contexts have similar representations. We'll introduce transformer-based **large language models** in Chapter 8, along the way introducing **feedforward** language models (Bengio et al. 2006, Schwenk 2007) in Chapter 6 and **recurrent** language models (Mikolov, 2012) in Chapter 13.

Exercises

3.1 Write out the equation for trigram probability estimation (modifying Eq. 3.11). Now write out all the non-zero trigram probabilities for the `I am Sam` corpus on page 41.

3.2 Calculate the probability of the sentence `i want chinese food`. Give two probabilities, one using Fig. 3.2 and the 'useful probabilities' just below it on page 43, and another using the add-1 smoothed table in Fig. 3.7. Assume the additional add-1 smoothed probabilities $P(\texttt{i} \mid \texttt{<s>}) = 0.19$ and $P(\texttt{</s>} \mid \texttt{food}) = 0.40$.

3.3 Which of the two probabilities you computed in the previous exercise is higher, unsmoothed or smoothed? Explain why.

3.4 We are given the following corpus, modified from the one in the chapter:

```
<s> I am Sam </s>
<s> Sam I am </s>
<s> I am Sam </s>
<s> I do not like green eggs and Sam </s>
```

Using a bigram language model with add-one smoothing, what is P(Sam | am)? Include <s> and </s> in your counts just like any other token.

3.5 Suppose we didn't use the end-symbol </s>. Train an unsmoothed bigram grammar on the following training corpus without using the end-symbol </s>:

```
<s> a b
<s> b b
<s> b a
<s> a a
```

Demonstrate that your bigram model does not assign a single probability distribution across all sentence lengths by showing that the sum of the probability of the four possible 2 word sentences over the alphabet {a,b} is 1.0, and the sum of the probability of all possible 3 word sentences over the alphabet {a,b} is also 1.0.

3.6 Suppose we train a trigram language model with add-one smoothing on a given corpus. The corpus contains V word types. Express a formula for estimating P(w3|w1,w2), where w3 is a word which follows the bigram (w1,w2), in terms of various n-gram counts and V. Use the notation c(w1,w2,w3) to denote the number of times that trigram (w1,w2,w3) occurs in the corpus, and so on for bigrams and unigrams.

3.7 We are given the following corpus, modified from the one in the chapter:

```
<s> I am Sam </s>
<s> Sam I am </s>
<s> I am Sam </s>
<s> I do not like green eggs and Sam </s>
```

If we use linear interpolation smoothing between a maximum-likelihood bigram model and a maximum-likelihood unigram model with $\lambda_1 = \frac{1}{2}$ and $\lambda_2 = \frac{1}{2}$, what is P(Sam|am)? Include <s> and </s> in your counts just like any other token.

3.8 Write a program to compute unsmoothed unigrams and bigrams.

3.9 Run your n-gram program on two different small corpora of your choice (you might use email text or newsgroups). Now compare the statistics of the two corpora. What are the differences in the most common unigrams between the two? How about interesting differences in bigrams?

3.10 Add an option to your program to generate random sentences.

3.11 Add an option to your program to compute the perplexity of a test set.

3.12 You are given a training set of 100 numbers that consists of 91 zeros and 1 each of the other digits 1-9. Now we see the following test set: 0 0 0 0 0 3 0 0 0 0. What is the unigram perplexity?

CHAPTER

4 Logistic Regression and Text Classification

En sus remotas páginas está escrito que los animales se dividen en:
a. pertenecientes al Emperador
b. embalsamados
c. amaestrados
d. lechones
e. sirenas
f. fabulosos
g. perros sueltos
h. incluidos en esta clasificación
i. que se agitan como locos
j. innumerables
k. dibujados con un pincel finísimo de pelo de camello
l. etcétera
m. que acaban de romper el jarrón
n. que de lejos parecen moscas
Borges (1964)

Classification lies at the heart of language processing and intelligence. Recognizing a letter, a word, or a face, sorting mail, assigning grades to homeworks; these are all examples of assigning a category to an input. The challenges of classification were famously highlighted by the fabulist Jorge Luis Borges (1964), who imagined an ancient mythical encyclopedia that classified animals into:

> *(a) those that belong to the Emperor, (b) embalmed ones, (c) those that are trained, (d) suckling pigs, (e) mermaids, (f) fabulous ones, (g) stray dogs, (h) those that are included in this classification, (i) those that tremble as if they were mad, (j) innumerable ones, (k) those drawn with a very fine camel's hair brush, (l) others, (m) those that have just broken a flower vase, (n) those that resemble flies from a distance.*

Luckily, the classes we use for language processing are easier to define than those of Borges. In this chapter we introduce the **logistic regression** algorithm for classification, and apply it to **text categorization**, the task of assigning a label or category to a text or document. We'll focus on one text categorization task, **sentiment analysis**, the categorization of **sentiment**, the positive or negative orientation that a writer expresses toward some object. A review of a movie, book, or product expresses the author's sentiment toward the product, while an editorial or political text expresses sentiment toward an action or candidate. Extracting sentiment is thus relevant for fields from marketing to politics.

text categorization

sentiment analysis

For the binary task of labeling a text as indicating positive or negative stance, words (like *awesome* and *love*, or *awful* and *ridiculously* are very informative, as we can see from these sample extracts from movie/restaurant reviews:

\+ *...awesome caramel sauce and sweet toasty almonds. I love this place!*
− *...awful pizza and ridiculously overpriced...*

There are many text classification tasks. In **spam detection** we assign an email to one of the two classes *spam* or *not-spam*. **Language id** is the task of determining what language a text is written in, while **authorship attribution** is the task of determining a text's author, relevant to both humanistic and forensic analysis.

spam detection

language id

authorship attribution

But what makes classification so important is that **language modeling** can also be viewed as classification: each word can be thought of as a class, and so predicting the next word is classifying the context-so-far into a class for each next word. As we'll see, this intuition underlies large language models.

The algorithm for classification we introduce in this chapter, logistic regression, is equally important, in a number of ways. First, logistic regression has a close relationship with neural networks. As we will see in Chapter 6, a neural network can be viewed as a series of logistic regression classifiers stacked on top of each other. Second, logistic regression introduces ideas that are fundamental to neural networks and language models, like the **sigmoid** and **softmax** functions, the **logit**, and the key **gradient descent** algorithm for learning. Finally, logistic regression is also one of the most important analytic tools in the social and natural sciences.

sigmoid
softmax
logit

4.1 Machine learning and classification

The goal of classification is to take a single input (we call each input an **observation**), extract some useful **features** or properties of the input, and thereby **classify** the observation into one of a set of discrete classes. We'll call the input x, and say that the output comes from a fixed set of output classes $Y = \{y_1, y_2, ..., y_M\}$. Our goal is return a predicted class from Y. Sometimes you'll see the output classes referred to as the set C instead of Y.

observation

For sentiment analysis, the input x might be a review, or some other text. And the output set Y might be the set:

$$\{\texttt{positive}, \texttt{negative}\}$$

or the set

$$\{0, 1\}$$

For language id, the input might be a text that we need to know what language it was written in, and the output set Y is the set of languages, i.e.,

$$Y = \{\texttt{Abkhaz}, \texttt{Ainu}, \texttt{Albanian}, \texttt{Amharic}, ...\texttt{Zulu}, \texttt{Zuñi}\}$$

There are many ways to do classification. One method is to use rules handwritten by humans. For example, we might have a rule like:

```
If the word ''love'' appears in x, and it's not preceded by the
word ''don't", classify as positive
```

Handwritten rules can be components of modern NLP systems, such as the handwritten lists of positive and negative words that can be used in sentiment analysis, as we'll see below. But rules can be fragile, as situations or data change over time, and for many tasks there are complex interactions between different features (like the example of negation with "don't" in the rule above), so it can be quite hard for humans to come up with rules that are successful over many situations.

Another method that we will introduce later is to ask a large language model (of the type we will introduce in Chapter 7) by prompting the model to give a label to some text. Prompting can be powerful, but again has weaknesses: language models often hallucinate, and may not be able to explain why they chose the class they did.

For these reasons the most common way to do classification is to use **supervised machine learning**. Supervised machine learning is a paradigm in which, in

supervised machine learning

addition to the input and the set of output classes, we have a **labeled training set** and a **learning algorithm**. We talked about training sets in Chapter 3 as a locus for computing n-gram statistics. But in supervised machine learning the training set is **labeled**, meaning that it contains a set of input observations, each observation associated with the correct output (a 'supervision signal'). We can generally refer to a training set of m input/output pairs, where each input x is a text, in the case of text classification, and each is hand-labeled with an associated class (the correct label):.

$$\text{training set:} \quad \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)})\} \tag{4.1}$$

We'll use superscripts in parentheses to refer to individual observations or instances in the training set. So for sentiment classification, a training set might be a set of sentences or other texts, each with their correct sentiment label.

Our goal is to learn from this training set a classifier that is capable of mapping from a **new** input x to its correct class $y \in Y$. It does this by learning to find features in these training sentences (perhaps words like "awesome" or "awful"). **Probabilistic classifiers** like logistic regression are classifiers that in addition to giving an answer (the class this observation is in), also give the *probability* of the observation being in the class. This distribution over the classes can be useful information for downstream decisions; avoiding making discrete decisions early on can be useful when combining systems. Probabilistic classifiers like logistic regression have four components:

1. A **feature representation** of the input. For each input observation $x^{(i)}$, this will be a vector of features $[x_1, x_2, ..., x_n]$. We will generally refer to feature i for input $x^{(j)}$ as $x_i^{(j)}$, sometimes simplified as x_i, but we will also see the notation f_i, $f_i(x)$, or, for multiclass classification, $f_i(c, x)$.
2. A classification function that computes the estimated class, by computing the probability $P(y = y_i|x)$ for each output class y_i. We will introduce the **sigmoid** and **softmax** tools for classification.
3. An **objective function** that we want to optimize for learning, usually involving minimizing a loss function corresponding to error on training examples. We will introduce the **cross-entropy loss function**.
4. An algorithm for optimizing the objective function. We introduce the **stochastic gradient descent** algorithm.

At the highest level, logistic regression, and really any probabilistic machine learning classifier, has two phases

training: We train the system (in the case of logistic regression that means training the weights w and b, introduced below) using stochastic gradient descent and the cross-entropy loss.

test: Given a test example x we compute the probability $P(y = y_i|x)$ for each output class y_i. Then, given this vector of probabilities, we return the higher probability label $y = 1$ or $y = 0$.

Logistic regression can be used to classify an observation into one of two classes (like 'positive sentiment' and 'negative sentiment'), or into one of many classes. Because the mathematics for the two-class case is simpler, we'll first describe this special case of logistic regression in the next few sections, beginning with the **sigmoid** function, and then turn to **multinomial logistic regression** for more than two classes and the use of the **softmax** function in Section 4.7.

4.2 The sigmoid function

The goal of binary logistic regression is to train a classifier that can make a binary decision about the class of a new input observation. Here we introduce the **sigmoid** classifier that will help us make this decision.

Consider a single input observation x, which we will represent by a vector of features $[x_1, x_2, ..., x_n]$. (We'll show sample features in the next subsection.) The classifier output y can be 1 (meaning the observation is a member of the class) or 0 (the observation is not a member of the class). We want to know the probability $P(y = 1|x)$ that this observation is a member of the class. So perhaps the decision is "positive sentiment" versus "negative sentiment", the features represent counts of words in a document, $P(y = 1|x)$ is the probability that the document has positive sentiment, and $P(y = 0|x)$ is the probability that the document has negative sentiment.

Logistic regression solves this task by learning, from a training set, a vector of **weights** and a **bias term**. Each weight w_i is a real number, and is associated with one of the input features x_i. The weight w_i represents how important that input feature is to the classification decision, and can be positive (providing evidence that the instance being classified belongs in the positive class) or negative (providing evidence that the instance being classified belongs in the negative class). Thus we might expect in a sentiment task the word *awesome* to have a high positive weight, and *abysmal* to have a very negative weight. The **bias term**, also called the **intercept**, is another real number that's added to the weighted inputs.

bias term
intercept

To make a decision on a test instance—after we've learned the weights in training—the classifier first multiplies each x_i by its weight w_i, sums up the weighted features, and adds the bias term b. The resulting single number z expresses the weighted sum of the evidence for the class.

$$z = \left(\sum_{i=1}^{n} w_i x_i\right) + b \tag{4.2}$$

dot product

In the rest of the book we'll represent such sums using the **dot product** notation from linear algebra. The dot product of two vectors $\mathbf{a}$ and $\mathbf{b}$, written as $\mathbf{a} \cdot \mathbf{b}$, is the sum of the products of the corresponding elements of each vector. (Notice that we represent vectors using the boldface notation $\mathbf{b}$). Thus the following is an equivalent formation to Eq. 4.2:

$$z = \mathbf{w} \cdot \mathbf{x} + b \tag{4.3}$$

But note that nothing in Eq. 4.3 forces z to be a legal probability, that is, to lie between 0 and 1. In fact, since weights are real-valued, the output might even be negative; z ranges from $-\infty$ to ∞.

sigmoid

logistic function

To create a probability, we'll pass z through the **sigmoid** function, $\sigma(z)$. The sigmoid function (named because it looks like an s) is also called the **logistic function**, and gives logistic regression its name. The sigmoid has the following equation, shown graphically in Fig. 4.1:

$$\sigma(z) = \frac{1}{1+e^{-z}} = \frac{1}{1+\exp(-z)} \tag{4.4}$$

(For the rest of the book, we'll use the notation $\exp(x)$ to mean e^x.) The sigmoid has a number of advantages; it takes a real-valued number and maps it into the range

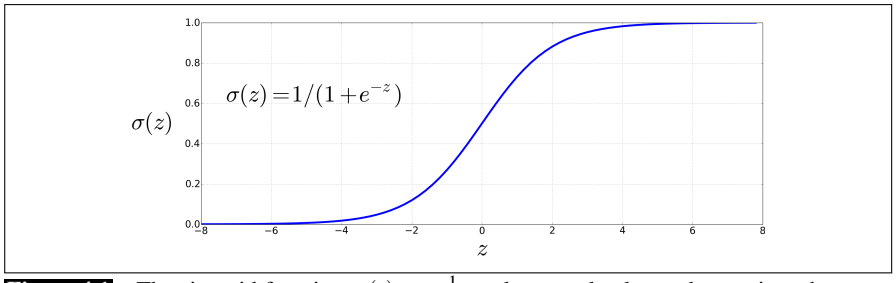


Figure 4.1 The sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$ takes a real value and maps it to the range $(0,1)$. It is nearly linear around 0 but outlier values get squashed toward 0 or 1.

$(0,1)$, which is just what we want for a probability. Because it is nearly linear around 0 but flattens toward the ends, it tends to squash outlier values toward 0 or 1. And it's differentiable, which as we'll see in Section 4.15 will be handy for learning.

We're almost there. If we apply the sigmoid to the sum of the weighted features, we get a number between 0 and 1. To make it a probability, we just need to make sure that the two cases, $P(y=1)$ and $P(y=0)$, sum to 1. We can do this as follows:

$$\begin{aligned} P(y=1) &= \sigma(\mathbf{w}\cdot\mathbf{x}+b) \\ &= \frac{1}{1+\exp(-(\mathbf{w}\cdot\mathbf{x}+b))} \\ P(y=0) &= 1-\sigma(\mathbf{w}\cdot\mathbf{x}+b) \\ &= 1-\frac{1}{1+\exp(-(\mathbf{w}\cdot\mathbf{x}+b))} \\ &= \frac{\exp(-(\mathbf{w}\cdot\mathbf{x}+b))}{1+\exp(-(\mathbf{w}\cdot\mathbf{x}+b))} \end{aligned} \tag{4.5}$$

The sigmoid function has the property

$$1-\sigma(x) = \sigma(-x) \tag{4.6}$$

so we could also have expressed $P(y=0)$ as $\sigma(-(\mathbf{w}\cdot\mathbf{x}+b))$.

logit

Finally, two terminological points. First, the input to the sigmoid function, the score $z = \mathbf{w}\cdot\mathbf{x}+b$ from Eq. 4.3, is often called the **logit**. This is because the logit function is the inverse of the sigmoid. The logit function is the log of the odds ratio $\frac{p}{1-p}$:

$$\text{logit}(p) = \sigma^{-1}(p) = \ln\frac{p}{1-p} \tag{4.7}$$

Using the term **logit** for z is a way of reminding us that by using the sigmoid to turn z (which ranges from $-\infty$ to ∞) into a probability, we are implicitly interpreting z as not just any real-valued number, but as specifically a log odds.

y hat $\hat{y}$

Second, for binary classification it will be convenient to refer to the output of the sigmoid function $\sigma(\mathbf{w}\cdot\mathbf{x}+b)$ which is computing $P(y=1)$, as $\hat{y}$. We pronounce this symbol as **y hat**. We thus use $\hat{y}$ to mean "the probability of the input observation having the positive class". When we introduce multinomial or softmax logistic regression in Section 4.7, we will introduce an extension of $\hat{y}$ that will be a vector of probabities over all the output classes, but for binary classification we just keep one scalar probability, knowing that we can always compute the missing probability $P(y=0)$ as $1-P(y=1)$.

4.3 Classification with Logistic Regression

The sigmoid function from the prior section thus gives us a way to take an instance x and compute the probability $P(y=1|x)$.

How do we make a decision about which class to apply to a test instance x? For a given x, we say yes if the probability $P(y=1|x)$ is more than .5, and no otherwise. We call .5 the **decision boundary**:

decision boundary

$$\text{decision}(x) = \begin{cases} 1 & \text{if } P(y=1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Let's have some examples of applying logistic regression as a classifier for language tasks.

4.3.1 Sentiment Classification

Suppose we are doing binary sentiment classification on movie review text, and we would like to know whether to assign the sentiment class + or − to a review document *doc*. We'll represent each input observation by the 6 features $x_1 \ldots x_6$ of the input shown in the following table; Fig. 4.2 shows features in a sample mini test document.

Var	Definition	Value in Fig. 4.2
x_1	count(positive lexicon words ∈ doc)	3
x_2	count(negative lexicon words ∈ doc)	2
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns ∈ doc)	3
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	ln(word+punctuation count of doc)	ln(66) = 4.19

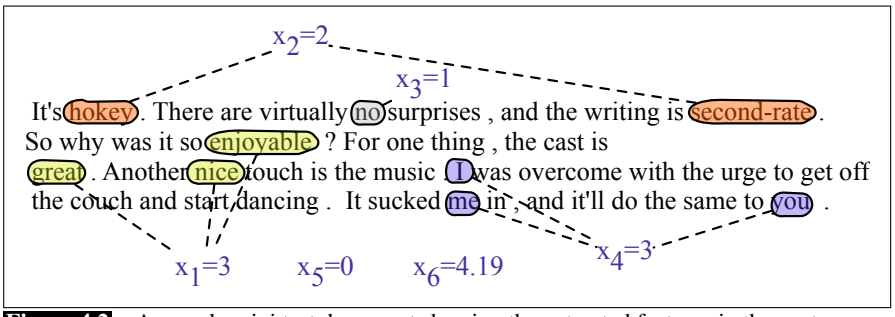


Figure 4.2 A sample mini test document showing the extracted features in the vector x.

Let's assume for the moment that we've already learned a real-valued weight for each of these features, and that the 6 weights corresponding to the 6 features are $[2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$, while $b = 0.1$. (We'll discuss in the next section how the weights are learned.) The weight w_1, for example indicates how important a feature the number of positive lexicon words (*great*, *nice*, *enjoyable*, etc.) is to

a positive sentiment decision, while w_2 tells us the importance of negative lexicon words. Note that $w_1 = 2.5$ is positive, while $w_2 = -5.0$, meaning that negative words are negatively associated with a positive sentiment decision, and are about twice as important as positive words.

Given these 6 features and the input review x, $P(+|x)$ and $P(-|x)$ can be computed using Eq. 4.5:

$$\begin{aligned} P(+|x) = P(y=1|x) &= \sigma(\mathbf{w}\cdot\mathbf{x}+b) \\ &= \sigma([2.5,-5.0,-1.2,0.5,2.0,0.7]\cdot[3,2,1,3,0,4.19]+0.1) \\ &= \sigma(.833) \\ &= 0.70 \qquad (4.8) \\ P(-|x) = P(y=0|x) &= 1-\sigma(\mathbf{w}\cdot\mathbf{x}+b) \\ &= 0.30 \end{aligned}$$

4.3.2 Other classification tasks and features

period disambiguation

Logistic regression is applied to all sorts of NLP tasks, and any property of the input can be a feature. Consider the task of **period disambiguation**: deciding if a period is the end of a sentence or part of a word, by classifying each period into one of two classes, EOS (end-of-sentence) and not-EOS. We might use features like x_1 below expressing that the current word is lower case, perhaps with a positive weight. Or a feature expressing that the current word is in our abbreviations dictionary ("Prof."), perhaps with a negative weight. A feature can also express a combination of properties. For example a period following an upper case word is likely to be an EOS, but if the word itself is *St.* and the previous word is capitalized then the period is likely part of a shortening of the word *street* following a street name.

$$\begin{aligned} x_1 &= \begin{cases} 1 & \text{if } \text{“}Case(w_i) = \text{Lower”} \\ 0 & \text{otherwise} \end{cases} \\ x_2 &= \begin{cases} 1 & \text{if } \text{“}w_i \in \text{AcronymDict”} \\ 0 & \text{otherwise} \end{cases} \\ x_3 &= \begin{cases} 1 & \text{if } \text{“}w_i = \text{St. \& } Case(w_{i-1}) = \text{Upper”} \\ 0 & \text{otherwise} \end{cases} \end{aligned}$$

Designing versus learning features: In classic models, features are designed by hand by examining the training set with an eye to linguistic intuitions and literature, supplemented by insights from error analysis on the training set of an early version of a system. We can also consider **feature interactions**, complex features that are combinations of more primitive features. We saw such a feature for period disambiguation above, where a period on the word *St.* was less likely to be the end of the sentence if the previous word was capitalized. Features can be created automatically via **feature templates**, abstract specifications of features. For example a bigram template for period disambiguation might create a feature for every pair of words that occurs before a period in the training set. Thus the feature space is sparse, since we only have to create a feature if that n-gram exists in that position in the training set. The feature is generally created as a hash from the string descriptions. A user description of a feature as, "bigram(American breakfast)" is hashed into a unique integer i that becomes the feature number f_i.

feature interactions

feature templates

It should be clear from the prior paragraph that designing features by hand requires extensive human effort. For this reason, recent NLP systems avoid hand-

designed features and instead focus on **representation learning**: ways to learn features automatically in an unsupervised way from the input. We'll introduce methods for representation learning in Chapter 5 and Chapter 6.

Scaling input features: When different input features have extremely different ranges of values, it's common to rescale them so they have comparable ranges. We **standardize** input values by centering them to result in a zero mean and a standard deviation of one (this transformation is sometimes called the **z-score**). That is, if μ_i is the mean of the values of feature x_i across the m observations in the input dataset, and σ_i is the standard deviation of the values of features x_i across the input dataset, we can replace each feature x_i by a new feature x'_i computed as follows:

standardize
z-score

$$\mu_i = \frac{1}{m}\sum_{j=1}^{m} x_i^{(j)} \qquad \sigma_i = \sqrt{\frac{1}{m}\sum_{j=1}^{m}\left(x_i^{(j)} - \mu_i\right)^2}$$
$$x'_i = \frac{x_i - \mu_i}{\sigma_i} \tag{4.9}$$

normalize

Alternatively, we can **normalize** the input features values to lie between 0 and 1:

$$x'_i = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)} \tag{4.10}$$

Having input data with comparable range is useful when comparing values across features. Data scaling is especially important in large neural networks, since it helps speed up gradient descent.

Another very common type of scaling for natural language data is to take logs, for example when inputs are word counts, or bigram counts, or anything else that follows a Zipfian distribution.

4.3.3 Processing many examples at once

We've shown the equations for logistic regression for a single example. But in practice we'll of course want to process an entire test set with many examples. Let's suppose we have a test set consisting of m test examples each of which we'd like to classify. We'll continue to use the notation from page 64, in which a superscript value in parentheses refers to the example index in some set of data (either for training or for test). So in this case each test example $x^{(i)}$ has a feature vector $\mathbf{x}^{(i)}$, $1 \leq i \leq m$. (As usual, we'll represent vectors and matrices in bold.)

One way to compute $\hat{y}^{(i)}$ (which is how we refer to $P(y^{(1)} = 1)$, i.e., the probability that the output for input $x^{(i)}i$ has the value 1), is just to have a for-loop and compute each test example one at a time:

$$\begin{aligned} &\textbf{foreach} && x^{(i)} && \text{in input } [x^{(1)}, x^{(2)}, ..., x^{(m)}] \\ &&& \hat{y}^{(i)} = P(y^{(i)} = 1) && = \sigma(\mathbf{w} \cdot \mathbf{x^{(i)}} + b) \end{aligned} \tag{4.11}$$

For the first 3 test examples, then, we would be separately computing the predicted output probability as follows:

$$\begin{aligned} \hat{y}^{(1)} = P(y^{(1)} = 1 | x^{(1)}) &= \sigma(\mathbf{w} \cdot \mathbf{x^{(1)}} + b) \\ \hat{y}^{(2)} = P(y^{(2)} = 1 | x^{(2)}) &= \sigma(\mathbf{w} \cdot \mathbf{x^{(2)}} + b) \\ \hat{y}^{(3)} = P(y^{(3)} = 1 | x^{(3)}) &= \sigma(\mathbf{w} \cdot \mathbf{x^{(3)}} + b) \end{aligned}$$

But it turns out that we can slightly modify our original equation Eq. 4.5 to do this much more efficiently. We'll use matrix arithmetic to assign a class to all the examples with one matrix operation!

First, we'll pack all the input feature vectors for each input x into a single input matrix $\mathbf{X}$, where each row i is a row vector consisting of the feature vector for input example $x^{(i)}$ (i.e., the vector $\mathbf{x}^{(i)}$). Assuming each example has f features and weights, $\mathbf{X}$ will therefore be a matrix of shape $[m \times f]$, as follows:

$$\mathbf{X} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_f^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_f^{(2)} \\ x_1^{(3)} & x_2^{(3)} & \dots & x_f^{(3)} \\ \dots & & & \end{bmatrix} \tag{4.12}$$

Now if we introduce $\mathbf{b}$ as a vector of length m which consists of the scalar bias term b repeated m times, $\mathbf{b} = [b, b, ..., b]$, and $\hat{\mathbf{y}} = [\hat{y}^{(1)}, \hat{y}^{(2)} ... \hat{y}^{(m)}]$ as the vector of outputs (one scalar $\hat{y}^{(i)} = P(y^{(i)} = 1)$ for each input $x^{(i)}$ and its feature vector $\mathbf{x}^{(i)}$), and represent the weight vector $\mathbf{w}$ as a column vector, we can compute all the outputs with a single matrix multiplication and one addition:

$$\hat{\mathbf{y}} = \sigma(\mathbf{X}\mathbf{w} + \mathbf{b}) \tag{4.13}$$

You should convince yourself that Eq. 4.13 computes the same thing as our for-loop in Eq. 4.11. For example $\hat{y}^{(1)}$, the first entry of the output vector $\hat{\mathbf{y}}$, will correctly be:

$$\hat{y}^{(1)} = \sigma\left([x_1^{(1)}, x_2^{(1)}, ..., x_f^{(1)}] \cdot [w_1, w_2, ..., w_f] + b\right) \tag{4.14}$$

Note that we had to reorder $\mathbf{X}$ and $\mathbf{w}$ from the order they appeared in Eq. 4.5 to make the multiplications come out properly. Here is Eq. 4.13 again with the shapes shown:

$$\begin{array}{cccccc} \hat{\mathbf{y}} & = & \sigma(\mathbf{X} & \mathbf{w} & + & \mathbf{b}) \\ (m \times 1) & & (m \times f) & (f \times 1) & & (m \times 1) \end{array} \tag{4.15}$$

Modern compilers and compute hardware can compute this matrix operation very efficiently, making the computation much faster, which becomes important when training or testing on very large datasets.

Note by the way that we could have kept $\mathbf{X}$ and $\mathbf{w}$ in the original order (as $\hat{\mathbf{y}} = \sigma(\mathbf{w}\mathbf{X} + \mathbf{b})$) if we had chosen to define $\mathbf{X}$ differently as a matrix of column vectors, one vector for each input example, instead of row vectors, and then it would have shape $[f \times m]$. But we conventionally represent inputs as rows.

4.4 Learning in Logistic Regression

How are the parameters of the model, the weights $\mathbf{w}$ and bias b, learned? Binary logistic regression is an instance of supervised classification in which we know the correct label y (either 0 or 1) for each observation x. What the system produces via Eq. 4.5 is $\hat{y}$, the system's probability of y being 1, which we can think of an estimate of the true y (which is either 1 or 0). We want to learn parameters (meaning $\mathbf{w}$ and b) that make the probability value $\hat{y}$ for each training observation as close as possible

to the true y. That is, if y is 1, then we want the system's estimate $\hat{y} = P(y = 1)$ to be high, i.e. as close to 1 as possible. If y is in fact 0, then we want the system's estimate $\hat{y} = P(y = 1)$ to be low, i.e. as close to 0 as possible.

This requires two components that we foreshadowed in the introduction to the chapter. The first is a metric for how close $\hat{y}$ (the systems's estimate of the probability that the observation is in the positive class) is to the true gold label y. Rather than measure similarity, we usually talk about the opposite of this: the *distance* between the system output and the gold output, and we call this distance the **loss** function or the **cost function**. In the next section we'll introduce the loss function that is commonly used for logistic regression and also for neural networks, the **cross-entropy loss**.

loss

The second thing we need is an optimization algorithm for iteratively updating the weights so as to minimize this loss function. The standard algorithm for this is **gradient descent**; we'll introduce the **stochastic gradient descent** algorithm in the following section.

We'll describe these algorithms for the simpler case of binary logistic regression in the next two sections, and then turn to multinomial logistic regression in Section 4.8.

4.5 The cross-entropy loss function

We need a loss function that expresses, for an observation x, how close the classifier output ($\hat{y} = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$) is to the correct output (y, which is 0 or 1). Recall that $\hat{y}$ expresses the probability that the classifier assigns to observation x being in the positive class 1. So both $\hat{y}$ and y express probabilities, but y is always 0 or 1 but $\hat{y}$ can also lie in between. We'll call this measure of loss or distance:

$$L(\hat{y}, y) = \text{How much } \hat{y} \text{ differs from the true } y \tag{4.16}$$

We do this via a loss function that prefers the correct class labels of the training examples to be *more likely*. This is called **conditional maximum likelihood estimation**: we choose the parameters w, b that **maximize the log probability of the true y labels in the training data** given the observations x. The resulting loss function is the **negative log likelihood loss**, generally called the **cross-entropy loss**.

cross-entropy loss

Let's derive this loss function, applied to a single observation x. We'd like to learn weights that maximize the probability of the correct label $p(y|x)$. Since there are only two discrete outcomes (1 or 0), this is a Bernoulli distribution, and we can express the probability $p(y|x)$ that our classifier produces for one observation as the following (keeping in mind that if $y = 1$, Eq. 4.17 simplifies to $\hat{y}$; if $y = 0$, Eq. 4.17 simplifies to $1 - \hat{y}$):

$$p(y|x) = \hat{y}^y (1 - \hat{y})^{1-y} \tag{4.17}$$

Now we take the log of both sides. This will turn out to be handy mathematically, and doesn't hurt us; whatever values maximize a probability will also maximize the log of the probability:

$$\begin{aligned} \log p(y|x) &= \log \left[\hat{y}^y (1 - \hat{y})^{1-y} \right] \\ &= y \log \hat{y} + (1 - y) \log(1 - \hat{y}) \end{aligned} \tag{4.18}$$

Eq. 4.18 describes a log likelihood that should be maximized. In order to turn this into a loss function (something that we need to minimize), we'll just flip the sign on Eq. 4.18. The result is the cross-entropy loss L_{CE}:

$$L_{\text{CE}}(\hat{y},y) = -\log p(y|x) = -[y\log\hat{y}+(1-y)\log(1-\hat{y})] \tag{4.19}$$

Finally, we can plug in the definition of $\hat{y} = \sigma(\mathbf{w}\cdot\mathbf{x}+b)$:

$$L_{\text{CE}}(\hat{y},y) = -[y\log\sigma(\mathbf{w}\cdot\mathbf{x}+b)+(1-y)\log(1-\sigma(\mathbf{w}\cdot\mathbf{x}+b))] \tag{4.20}$$

Let's see if this loss function does the right thing for our example from Fig. 4.2. We want the loss to be smaller if the model's estimate is close to correct, and bigger if the model is confused. So first let's suppose the correct gold label for the sentiment example in Fig. 4.2 is positive, i.e., $y = 1$. In this case our model is doing well, since from Eq. 4.8 it indeed gave the example a higher probability of being positive (.70) than negative (.30). If we plug $\sigma(\mathbf{w}\cdot\mathbf{x}+b) = .70$ and $y = 1$ into Eq. 4.20, the right side of the equation drops out, leading to the following loss (we'll use log to mean natural log when the base is not specified):

$$\begin{aligned} L_{\text{CE}}(\hat{y},y) &= -[y\log\sigma(\mathbf{w}\cdot\mathbf{x}+b)+(1-y)\log(1-\sigma(\mathbf{w}\cdot\mathbf{x}+b))] \\ &= -[\log\sigma(\mathbf{w}\cdot\mathbf{x}+b)] \\ &= -\log(.70) \\ &= .36 \end{aligned}$$

By contrast, let's pretend instead that the example in Fig. 4.2 was actually negative, i.e., $y = 0$ (perhaps the reviewer went on to say "But bottom line, the movie is terrible! I beg you not to see it!"). In this case our model is confused and we'd want the loss to be higher. Now if we plug $y = 0$ and $1-\sigma(\mathbf{w}\cdot\mathbf{x}+b) = .30$ from Eq. 4.8 into Eq. 4.20, the left side of the equation drops out:

$$\begin{aligned} L_{\text{CE}}(\hat{y},y) &= -[y\log\sigma(\mathbf{w}\cdot\mathbf{x}+b)+(1-y)\log(1-\sigma(\mathbf{w}\cdot\mathbf{x}+b))] \\ &= -[\log(1-\sigma(\mathbf{w}\cdot\mathbf{x}+b))] \\ &= -\log(.30) \\ &= 1.2 \end{aligned}$$

Sure enough, the loss for predicting the correct label (.36) is less than the loss for predicting the incorrect label (1.2).

Why does minimizing this negative log probability do what we want? A perfect classifier would assign probability 1 to the correct outcome ($y = 1$ or $y = 0$) and probability 0 to the incorrect outcome. That means if y equals 1, the higher $\hat{y}$ is (the closer it is to 1), the better the classifier; the lower $\hat{y}$ is (the closer it is to 0), the worse the classifier. If y equals 0, instead, the higher $1-\hat{y}$ is (closer to 1), the better the classifier. The negative log of $\hat{y}$ (if the true y equals 1) or $1-\hat{y}$ (if the true y equals 0) is a convenient loss metric since it goes from 0 (negative log of 1, no loss) to infinity (negative log of 0, infinite loss). This loss function also ensures that as the probability of the correct answer is maximized, the probability of the incorrect answer is minimized; since the two sum to one, any increase in the probability of the correct answer is coming at the expense of the incorrect answer. It's called the cross-entropy loss, because Eq. 4.18 is also the formula for the **cross-entropy** between the true probability distribution y and our estimated distribution $\hat{y}$.

Now we know what we want to minimize; in the next section, we'll see how to find the minimum.

4.6 Gradient Descent

Our goal with gradient descent is to find the optimal weights: minimize the loss function we've defined for the model. In Eq. 4.21 below, we'll explicitly represent the fact that the cross-entropy loss function L_{CE} is parameterized by the weights. In machine learning in general we refer to the parameters being learned as θ; in the case of logistic regression $\theta = \{\mathbf{w}, b\}$. So we'll represent $\hat{y}^{(i)}$ as $f(x^{(i)}; \theta)$ to make the dependence on θ more obvious. The goal is to find the set of weights which minimizes the loss function, averaged over all examples:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^{m} L_{\text{CE}}(f(x^{(i)}; \theta), y^{(i)}) \tag{4.21}$$

How shall we find the minimum of this (or any) loss function? Gradient descent is a method that finds a minimum of a function by figuring out in which direction (in the space of the parameters θ) the function's slope is rising the most steeply, and moving in the opposite direction. The intuition is that if you are hiking in a canyon and trying to descend most quickly down to the river at the bottom, you might look around yourself in all directions, find the direction where the ground is sloping the steepest, and walk downhill in that direction.

convex

For logistic regression, this loss function is conveniently **convex**. A convex function has at most one minimum; there are no local minima to get stuck in, so gradient descent starting from any point is guaranteed to find the minimum. (By contrast, the loss for multi-layer neural networks is non-convex, and gradient descent may get stuck in local minima for neural network training and never find the global optimum.)

Although the algorithm (and the concept of gradient) are designed for direction *vectors*, let's first consider a visualization of the case where the parameter of our system is just a single scalar w, shown in Fig. 4.3.

Given a random initialization of w at some value w^1, and assuming the loss function L happened to have the shape in Fig. 4.3, we need the algorithm to tell us whether at the next iteration we should move left (making w^2 smaller than w^1) or right (making w^2 bigger than w^1) to reach the minimum.

gradient

The gradient descent algorithm answers this question by finding the **gradient** of the loss function at the current point and moving in the opposite direction. The gradient of a function of many variables is a vector pointing in the direction of the greatest increase in a function. The gradient is a multi-variable generalization of the slope, so for a function of one variable like the one in Fig. 4.3, we can informally think of the gradient as the slope. The dotted line in Fig. 4.3 shows the slope of this hypothetical loss function at point $w = w^1$. You can see that the slope of this dotted line is negative. Thus to find the minimum, gradient descent tells us to go in the opposite direction: moving w in a positive direction.

learning rate

The magnitude of the amount to move in gradient descent is the value of the slope $\frac{d}{dw} L(f(x;w),y)$ weighted by a **learning rate** η. A higher (faster) learning rate means that we should move w more on each step. The change we make in our parameter is the learning rate times the gradient (or the slope, in our single-variable example):

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x;w),y) \tag{4.22}$$

Now let's extend the intuition from a function of one scalar variable w to many

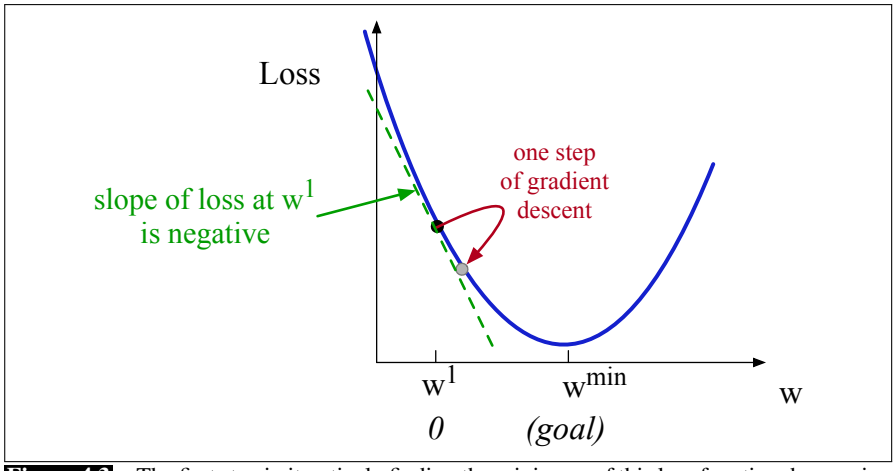


Figure 4.3 The first step in iteratively finding the minimum of this loss function, by moving w in the reverse direction from the slope of the function. Since the slope is negative, we need to move w in a positive direction, to the right. Here superscripts are used for learning steps, so w^1 means the initial value of w (which is 0), w^2 the value at the second step, and so on.

variables, because we don't just want to move left or right, we want to know where in the N-dimensional space (of the N parameters that make up θ) we should move. The **gradient** is just such a vector; it expresses the directional components of the sharpest slope along each of those N dimensions. If we're just imagining two weight dimensions (say for one weight w and one bias b), the gradient might be a vector with two orthogonal components, each of which tells us how much the ground slopes in the w dimension and in the b dimension. Fig. 4.4 shows a visualization of the value of a 2-dimensional gradient vector taken at the red point.

In an actual logistic regression, the parameter vector $\mathbf{w}$ is much longer than 1 or 2, since the input feature vector $\mathbf{x}$ can be quite long, and we need a weight w_i for each x_i. For each dimension/variable w_i in $\mathbf{w}$ (plus the bias b), the gradient will have a component that tells us the slope with respect to that variable. In each dimension w_i, we express the slope as a partial derivative $\frac{\partial}{\partial w_i}$ of the loss function. Essentially we're asking: "How much would a small change in that variable w_i influence the total loss function L?"

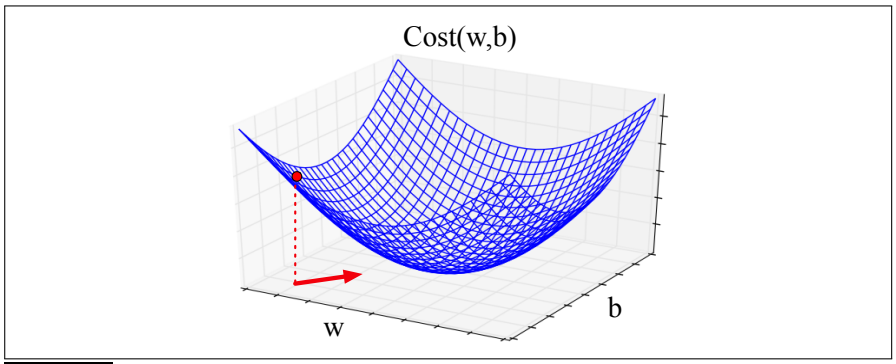


Figure 4.4 Visualization of the gradient vector at the red point in two dimensions w and b, showing a red arrow in the x-y plane pointing in the direction we will go to look for the minimum: the opposite direction of the gradient (recall that the gradient points in the direction of increase not decrease).

Formally, then, the gradient of a multi-variable function f is a vector in which each component expresses the partial derivative of f with respect to one of the variables. We'll use the inverted Greek delta symbol ∇ to refer to the gradient, and represent $\hat{y}$ as $f(x;\theta)$ to make the dependence on θ more obvious:

$$\nabla L(f(x;\theta),y) = \begin{bmatrix} \frac{\partial}{\partial w_1} L(f(x;\theta),y) \\ \frac{\partial}{\partial w_2} L(f(x;\theta),y) \\ \vdots \\ \frac{\partial}{\partial w_n} L(f(x;\theta),y) \\ \frac{\partial}{\partial b} L(f(x;\theta),y) \end{bmatrix} \tag{4.23}$$

The final equation for updating θ based on the gradient is thus

$$\theta^{t+1} = \theta^t - \eta \nabla L(f(x;\theta),y) \tag{4.24}$$

4.6.1 The Gradient for Logistic Regression

In order to update θ, we need a definition for the gradient $\nabla L(f(x;\theta),y)$. Recall that for logistic regression, the cross-entropy loss function is:

$$L_{\text{CE}}(\hat{y},y) = -[y \log \sigma(\mathbf{w}\cdot\mathbf{x}+b) + (1-y)\log(1-\sigma(\mathbf{w}\cdot\mathbf{x}+b))] \tag{4.25}$$

It turns out that the derivative of this function for one observation vector x is Eq. 4.26 (the interested reader can see Section 4.15 for the derivation of this equation):

$$\begin{aligned} \frac{\partial L_{\text{CE}}(\hat{y},y)}{\partial w_j} &= [\sigma(\mathbf{w}\cdot\mathbf{x}+b)-y]x_j \\ &= (\hat{y}-y)x_j \end{aligned} \tag{4.26}$$

You'll also sometimes see this equation in the equivalent form:

$$\frac{\partial L_{\text{CE}}(\hat{y},y)}{\partial w_j} = -(y-\hat{y})x_j \tag{4.27}$$

Note in these equations that the gradient with respect to a single weight w_j represents a very intuitive value: the difference between the true y and our estimated $\hat{y} = \sigma(\mathbf{w}\cdot\mathbf{x}+b)$ for that observation, multiplied by the corresponding input value x_j.

By the way, we'll also need a term for the partial derivative with respect to b. That turns out to be:

$$\begin{aligned} \frac{\partial L_{\text{CE}}(\hat{y},y)}{\partial b} &= \sigma(\mathbf{w}\cdot\mathbf{x}+b)-y \\ &= \hat{y}-y \end{aligned} \tag{4.28}$$

4.6.2 The Stochastic Gradient Descent Algorithm

Stochastic gradient descent is an online algorithm that minimizes the loss function by computing its gradient after each training example, and nudging θ in the right direction (the opposite direction of the gradient). (An "online algorithm" is one that processes its input example by example, rather than waiting until it sees the entire input.) Stochastic gradient descent is called **stochastic** because it chooses a single

function STOCHASTIC GRADIENT DESCENT($L()$, $f()$, x, y) **returns** θ
  # where: L is the loss function
  # f is a function parameterized by θ
  # x is the set of training inputs $x^{(1)}$, $x^{(2)}$,..., $x^{(m)}$
  # y is the set of training outputs (labels) $y^{(1)}$, $y^{(2)}$,..., $y^{(m)}$

$\theta \leftarrow 0$ # (or small random values)
repeat til done # see caption
  For each training tuple $(x^{(i)}, y^{(i)})$ (in random order)
    1. Optional (for reporting): # How are we doing on this tuple?
      Compute $\hat{y}^{(i)} = f(x^{(i)}; \theta)$ # What is our estimated output $\hat{y}$?
      Compute the loss $L(\hat{y}^{(i)}, y^{(i)})$ # How far off is $\hat{y}^{(i)}$ from the true output $y^{(i)}$?
    2. $g \leftarrow \nabla_\theta L(f(x^{(i)}; \theta), y^{(i)})$ # How should we move θ to maximize loss?
    3. $\theta \leftarrow \theta - \eta\, g$ # Go the other way instead
return θ

Figure 4.5 The stochastic gradient descent algorithm. Step 1 (computing the loss) is used mainly to report how well we are doing on the current tuple; we don't need to compute the loss in order to compute the gradient. The algorithm can terminate when it converges (when the gradient norm $< \epsilon$), or when progress halts (for example when the loss starts going up on a held-out set). Weights are initialized to 0 for logistic regression, but to small random values for neural networks, as we'll see in Chapter 6.

random example at a time; in Section 4.6.4 we'll discuss other versions of gradient descent that batch many examples at once. Fig. 4.5 shows the algorithm.

hyperparameter

The learning rate η is a **hyperparameter** that must be adjusted. If it's too high, the learner will take steps that are too large, overshooting the minimum of the loss function. If it's too low, the learner will take steps that are too small, and take too long to get to the minimum. It is common to start with a higher learning rate and then slowly decrease it, so that it is a function of the iteration k of training; the notation η_k can be used to mean the value of the learning rate at iteration k.

We'll discuss hyperparameters in more detail in Chapter 6, but in short, they are a special kind of parameter for any machine learning model. Unlike regular parameters of a model (weights like w and b), which are learned by the algorithm from the training set, hyperparameters are special parameters chosen by the algorithm designer that affect how the algorithm works.

4.6.3 Working through an example

Let's walk through a single step of the gradient descent algorithm. We'll use a simplified version of the example in Fig. 4.2 as it sees a single observation x, whose correct value is $y = 1$ (this is a positive review), and with a feature vector $\mathbf{x} = [x_1, x_2]$ consisting of these two features:

$$x_1 = 3 \quad \text{(count of positive lexicon words)}$$
$$x_2 = 2 \quad \text{(count of negative lexicon words)}$$

Let's assume the initial weights and bias in θ^0 are all set to 0, and the initial learning rate η is 0.1:

$$w_1 = w_2 = b = 0$$
$$\eta = 0.1$$

The single update step requires that we compute the gradient, multiplied by the learning rate

$$\theta^{t+1} = \theta^t - \eta \nabla_\theta L(f(x^{(i)};\theta), y^{(i)})$$

In our mini example there are three parameters, so the gradient vector has 3 dimensions, for w_1, w_2, and b. We can compute the first gradient as follows:

$$\nabla_{w,b} L = \begin{bmatrix} \frac{\partial L_{\text{CE}}(\hat{y},y)}{\partial w_1} \\ \frac{\partial L_{\text{CE}}(\hat{y},y)}{\partial w_2} \\ \frac{\partial L_{\text{CE}}(\hat{y},y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(\mathbf{w}\cdot\mathbf{x}+b)-y)x_1 \\ (\sigma(\mathbf{w}\cdot\mathbf{x}+b)-y)x_2 \\ \sigma(\mathbf{w}\cdot\mathbf{x}+b)-y \end{bmatrix} = \begin{bmatrix} (\sigma(0)-1)x_1 \\ (\sigma(0)-1)x_2 \\ \sigma(0)-1 \end{bmatrix} = \begin{bmatrix} -0.5x_1 \\ -0.5x_2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

Now that we have a gradient, we compute the new parameter vector θ^1 by moving θ^0 in the opposite direction from the gradient:

$$\theta^1 = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} - \eta \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix} = \begin{bmatrix} .15 \\ .1 \\ .05 \end{bmatrix}$$

So after one step of gradient descent, the weights have shifted to be: $w_1 = .15$, $w_2 = .1$, and $b = .05$.

Note that this observation x happened to be a positive example. We would expect that after seeing more negative examples with high counts of negative words, that the weight $\mathbf{w}_2$ would shift to have a negative value.

4.6.4 Mini-batch training

Stochastic gradient descent is called stochastic because it chooses a single random example at a time, moving the weights so as to improve performance on that single example. That can result in very choppy movements, so it's common to compute the gradient over batches of training instances rather than a single instance.

batch training

For example in **batch training** we compute the gradient over the entire dataset. By seeing so many examples, batch training offers a superb estimate of which direction to move the weights, at the cost of spending a lot of time processing every single example in the training set to compute this perfect direction.

mini-batch

A compromise is **mini-batch** training: we train on a group of m examples (perhaps 512, or 1024) that is less than the whole dataset. (If m is the size of the dataset, then we are doing **batch** gradient descent; if $m = 1$, we are back to doing stochastic gradient descent.) Mini-batch training also has the advantage of computational efficiency. The mini-batches can easily be vectorized, choosing the size of the mini-batch based on the computational resources. This allows us to process all the examples in one mini-batch in parallel and then accumulate the loss, something that's not possible with individual or batch training.

We just need to define mini-batch versions of the cross-entropy loss function we defined in Section 4.5 and the gradient in Section 4.6.1. Let's extend the cross-entropy loss for one example from Eq. 4.19 to mini-batches of size m. We'll continue to use the notation that $x^{(i)}$ and $y^{(i)}$ mean the ith training features and training label,

respectively. We make the assumption that the training examples are independent:

$$\begin{aligned}\log p(\text{training labels}) &= \log \prod_{i=1}^{m} p(y^{(i)}|x^{(i)}) \\ &= \sum_{i=1}^{m} \log p(y^{(i)}|x^{(i)}) \\ &= -\sum_{i=1}^{m} L_{\text{CE}}(\hat{y}^{(i)}, y^{(i)}) \end{aligned} \tag{4.29}$$

Now the cost function for the mini-batch of m examples is the average loss for each example:

$$\begin{aligned} Cost(\hat{y}, y) &= \frac{1}{m}\sum_{i=1}^{m} L_{\text{CE}}(\hat{y}^{(i)}, y^{(i)}) \\ &= -\frac{1}{m}\sum_{i=1}^{m} y^{(i)} \log \sigma(\mathbf{w}\cdot\mathbf{x}^{(i)}+b) + (1-y^{(i)}) \log\left(1-\sigma(\mathbf{w}\cdot\mathbf{x}^{(i)}+b)\right) \end{aligned} \tag{4.30}$$

The mini-batch gradient is the average of the individual gradients from Eq. 4.26:

$$\frac{\partial Cost(\hat{y}, y)}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}\left[\sigma(\mathbf{w}\cdot\mathbf{x}^{(i)}+b) - y^{(i)}\right] x_j^{(i)} \tag{4.31}$$

Instead of using the sum notation, we can more efficiently compute the gradient in its matrix form, following the vectorization we saw on page 70, where we have a matrix $\mathbf{X}$ of size $[m \times f]$ representing the m inputs in the batch, and a vector $\mathbf{y}$ of size $[m \times 1]$ representing the correct outputs:

$$\begin{aligned} \frac{\partial Cost(\hat{y}, y)}{\partial \mathbf{w}} &= \frac{1}{m}(\hat{\mathbf{y}} - \mathbf{y})^{\intercal}\mathbf{X} \\ &= \frac{1}{m}(\sigma(\mathbf{X}\mathbf{w} + \mathbf{b}) - \mathbf{y})^{\intercal}\mathbf{X} \end{aligned} \tag{4.32}$$

4.7 Multinomial logistic regression

Sometimes we need more than two classes. Perhaps we might want to do 3-way sentiment classification (positive, negative, or neutral). Or we could be assigning some of the labels we will introduce in Chapter 17, like the part of speech of a word (choosing from 10, 30, or even 50 different parts of speech), or the named entity type of a phrase (choosing from tags like person, location, organization). Or, for large language models, we'll be predicting the next word out of the $|V|$ possible words in the vocabulary, so it's $|V|$-way classification.

multinomial logistic regression

In such cases we use **multinomial logistic regression**, also called **softmax regression** (in older NLP literature you will sometimes see the name **maxent classifier**). In multinomial logistic regression we want to label each observation with a class k from a set of K classes, under the stipulation that only one of these classes is the correct one (sometimes called **hard classification**; an observation can not be in

multiple classes). Let's use the following representation: the output $\mathbf{y}$ for each input $\mathbf{x}$ will be a vector of length K. If class c is the correct class, we'll set $y_c = 1$, and set all the other elements of $\mathbf{y}$ to be 0, i.e., $y_c = 1$ and $y_j = 0 \;\; \forall j \neq c$. A vector like this $\mathbf{y}$, with one value=1 and the rest 0, is called a **one-hot vector**. The job of the classifier is to produce an estimate vector $\hat{\mathbf{y}}$. For each class k, the value $\hat{y}_k$ will be the classifier's estimate of the probability $P(y_k = 1|\mathbf{x})$.

4.7.1 Softmax

softmax

The multinomial logistic classifier uses a generalization of the sigmoid, called the **softmax** function, to compute $p(y_k = 1|\mathbf{x})$. The softmax function takes a vector $\mathbf{z} = [z_1, z_2, ..., z_K]$ of K arbitrary values and maps them to a probability distribution, with each value in the range [0,1], and all the values summing to 1. Like the sigmoid, it is an exponential function.

For a vector $\mathbf{z}$ of dimensionality K, the softmax is defined as:

$$\text{softmax}(z_i) \;=\; \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)} \quad 1 \leq i \leq K \tag{4.33}$$

The softmax of an input vector $\mathbf{z} = [z_1, z_2, ..., z_K]$ is thus a vector itself:

$$\text{softmax}(\mathbf{z}) \;=\; \left[\frac{\exp(z_1)}{\sum_{i=1}^{K} \exp(z_i)}, \frac{\exp(z_2)}{\sum_{i=1}^{K} \exp(z_i)}, ..., \frac{\exp(z_K)}{\sum_{i=1}^{K} \exp(z_i)} \right] \tag{4.34}$$

The denominator $\sum_{i=1}^{K} \exp(\mathbf{z}_i)$ is used to normalize all the values into probabilities. Thus for example given a vector:

$$\mathbf{z} = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$$

the resulting (rounded) softmax($\mathbf{z}$) is

$$[0.05, 0.09, 0.01, 0.1, 0.74, 0.01]$$

Like the sigmoid, the softmax has the property of squashing values toward 0 or 1. Thus if one of the inputs is larger than the others, it will tend to push its probability toward 1, and suppress the probabilities of the smaller inputs.

Finally, note that, just as for the sigmoid, we refer to $\mathbf{z}$, the vector of scores that is the input to the softmax, as **logits** (see Eq. 4.7).

4.7.2 Applying softmax in logistic regression

When we apply softmax for logistic regression, the input will (just as for the sigmoid) be the dot product between a weight vector $\mathbf{w}$ and an input vector $\mathbf{x}$ (plus a bias). But now we'll need separate weight vectors $\mathbf{w}_k$ and bias b_k for each of the K classes. The probability of each of our output classes $\hat{y}_k$ can thus be computed as:

$$P(y_k = 1|\mathbf{x}) \;=\; \frac{\exp(\mathbf{w}_k \cdot \mathbf{x} + b_k)}{\sum_{j=1}^{K} \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)} \tag{4.35}$$

The form of Eq. 4.35 makes it seem that we would compute each output separately. Instead, it's more common to set up the equation for more efficient computation by modern vector processing hardware. We'll do this by representing the

set of K weight vectors as a weight matrix W and a bias vector $\mathbf{b}$. Each row k of $\mathbf{W}$ corresponds to the vector of weights w_k. $\mathbf{W}$ thus has shape $[K \times f]$, for K the number of output classes and f the number of input features. The bias vector $\mathbf{b}$ has one value for each of the K output classes. If we represent the weights in this way, we can compute $\hat{\mathbf{y}}$, the vector of output probabilities for each of the K classes, by a single elegant equation:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b}) \tag{4.36}$$

If you work out the matrix arithmetic, you can see that the estimated score of the first output class $\hat{y}_1$ (before we take the softmax) will correctly turn out to be $\mathbf{w}_1 \cdot \mathbf{x} + b_1$.

prototype

One helpful interpretation of the weight matrix $\mathbf{W}$ is to see each row $\mathbf{w}_k$ as a **prototype** of class k. The weight vector $\mathbf{w}_k$ that is learned represents the class as a kind of template. Since two vectors that are more similar to each other have a higher dot product with each other, the dot product acts as a similarity function. Logistic regression is thus learning a protoype representation for each class, such that incoming vectors are assigned the class k they are most similar to from the K classes (Doumbouya et al., 2025).

Fig. 4.6 shows the difference between binary and multinomial logistic regression by illustrating the weight vector versus weight matrix in the computation of the output class probabilities.

4.7.3 Features in Multinomial Logistic Regression

Features in multinomial logistic regression act like features in binary logistic regression, with the difference mentioned above that we'll need separate weight vectors and biases for each of the K classes. Recall our binary exclamation point feature x_5 from page 67:

$$x_5 = \begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$$

In binary classification a positive weight w_5 on a feature influences the classifier toward $y = 1$ (positive sentiment) and a negative weight influences it toward $y = 0$ (negative sentiment) with the absolute value indicating how important the feature is. For multinomial logistic regression, by contrast, with separate weights for each class, a feature can be evidence for or against each individual class.

In 3-way multiclass sentiment classification, for example, we must assign each document one of the 3 classes $+$, $-$, or 0 (neutral). Now a feature related to exclamation marks might have a negative weight for 0 documents, and a positive weight for $+$ or $-$ documents:

Feature	Definition	$w_{5,+}$	$w_{5,-}$	$w_{5,0}$
$f_5(x)$	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	3.5	3.1	-5.3

Because these feature weights are dependent both on the input text and the output class, we sometimes make this dependence explicit and represent the features themselves as $f(x, y)$: a function of both the input and the class. Using such a notation $f_5(x)$ above could be represented as three features $f_5(x, +)$, $f_5(x, -)$, and $f_5(x, 0)$, each of which has a single weight. We'll use this kind of notation in our description of the CRF in Chapter 17.

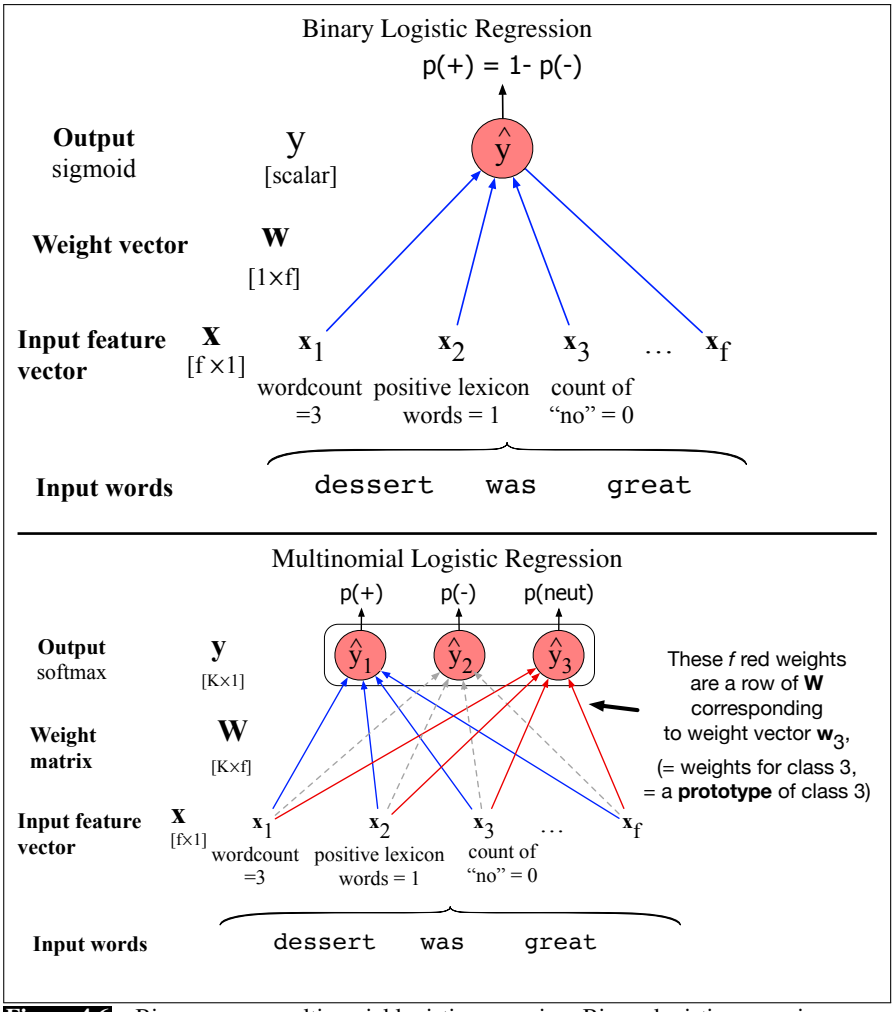


Figure 4.6 Binary versus multinomial logistic regression. Binary logistic regression uses a single weight vector $\mathbf{w}$, and has a scalar output $\hat{y}$. In multinomial logistic regression we have K separate weight vectors corresponding to the K classes, all packed into a single weight matrix $\mathbf{W}$, and a vector output $\hat{\mathbf{y}}$. We omit the biases from both figures for clarity.

4.8 Learning in Multinomial Logistic Regression

The loss function for multinomial logistic regression generalizes the loss function for binary logistic regression from 2 to K classes. Recall that the cross-entropy loss for binary logistic regression (repeated from Eq. 4.19) is:

$$L_{\text{CE}}(\hat{y},y) = -\log p(y|x) \;=\; -[y\log\hat{y} + (1-y)\log(1-\hat{y})] \tag{4.37}$$

The loss function for multinomial logistic regression generalizes the two terms in Eq. 4.37 (one that is non-zero when $y = 1$ and one that is non-zero when $y = 0$) to K terms. As we mentioned above, for multinomial regression we'll represent both $\mathbf{y}$ and $\hat{\mathbf{y}}$ as vectors. The true label $\mathbf{y}$ is a vector with K elements, each corresponding to a class, with $y_c = 1$ if the correct class is c, with all other elements of $\mathbf{y}$ being 0. And our classifier will produce an estimate vector with K elements $\hat{\mathbf{y}}$, each element $\hat{y}_k$ of which represents the estimated probability $p(y_k = 1|\mathbf{x})$.

The loss function for a single example $\mathbf{x}$, generalizing from binary logistic regression, is the sum of the logs of the K output classes, each weighted by the indicator function $\mathbf{y}_k$ (Eq. 4.38). This turns out to be just the negative log probability of the correct class c (Eq. 4.39):

$$\begin{aligned} L_{\text{CE}}(\hat{\mathbf{y}},\mathbf{y}) &= -\sum_{k=1}^{K} y_k \log \hat{y}_k && (4.38)\\ &= -\log \hat{y}_c, \quad \text{(where } c \text{ is the correct class)} && (4.39)\\ &= -\log \hat{p}(y_c = 1|\mathbf{x}) \quad \text{(where } c \text{ is the correct class)} \\ &= -\log \frac{\exp(\mathbf{w_c}\cdot\mathbf{x}+b_c)}{\sum_{j=1}^{K}\exp(\mathbf{w_j}\cdot\mathbf{x}+b_j)} \quad (c \text{ is the correct class}) && (4.40) \end{aligned}$$

How did we get from Eq. 4.38 to Eq. 4.39? Because only one class (let's call it c) is the correct one, the vector $\mathbf{y}$ takes the value 1 only for this value of k, i.e., has $y_c = 1$ and $y_j = 0 \ \ \forall j \neq c$. That means the terms in the sum in Eq. 4.38 will all be 0 except for the term corresponding to the true class c. Hence the cross-entropy loss is simply the log of the output probability corresponding to the correct class, and we therefore also call Eq. 4.39 the **negative log likelihood loss**.

negative log likelihood loss

Of course for gradient descent we don't need the loss, we need its gradient. The gradient for a single example turns out to be very similar to the gradient for binary logistic regression, $(\hat{y}-y)x$, that we saw in Eq. 4.26. Let's consider one piece of the gradient, the derivative for a single weight. For each class k, the weight of the ith element of input $\mathbf{x}$ is $w_{k,i}$. What is the partial derivative of the loss with respect to $w_{k,i}$? This derivative turns out to be just the difference between the true value for the class k (which is either 1 or 0) and the probability the classifier outputs for class k, weighted by the value of the input x_i corresponding to the ith element of the weight vector for class k:

$$\begin{aligned} \frac{\partial L_{\text{CE}}}{\partial w_{k,i}} &= -(y_k - \hat{y}_k)x_i \\ &= -(y_k - p(y_k = 1|x))x_i \\ &= -\left(y_k - \frac{\exp(\mathbf{w_k}\cdot\mathbf{x}+b_k)}{\sum_{j=1}^{K}\exp(\mathbf{w_j}\cdot\mathbf{x}+b_j)}\right)x_i && (4.41) \end{aligned}$$

We'll return to this case of the gradient for softmax regression when we introduce neural networks in Chapter 6, and at that time we'll also discuss the derivation of this gradient in equations Eq. 6.35–Eq. 6.43.

4.9 Evaluation: Precision, Recall, F-measure

To introduce the methods for evaluating text classification, let's first consider some simple binary *detection* tasks. For example, in spam detection, our goal is to label every text as being in the spam category ("positive") or not in the spam category ("negative"). For each item (email document) we therefore need to know whether our system called it spam or not. We also need to know whether the email is actually spam or not, i.e. the human-defined labels for each document that we are trying to match. We will refer to these human labels as the **gold labels**.

gold labels

Or imagine you're the CEO of the *Delicious Pie Company* and you need to know what people are saying about your pies on social media, so you build a system that detects tweets concerning Delicious Pie. Here the positive class is tweets about Delicious Pie and the negative class is all other tweets.

In both cases, we need a metric for knowing how well our spam detector (or pie-tweet-detector) is doing. To evaluate any system for detecting things, we start by building a **confusion matrix** like the one shown in Fig. 4.7. A confusion matrix is a table for visualizing how an algorithm performs with respect to the human gold labels, using two dimensions (system output and gold labels), and each cell labeling a set of possible outcomes. In the spam detection case, for example, true positives are documents that are indeed spam (indicated by human-created gold labels) that our system correctly said were spam. False negatives are documents that are indeed spam but our system incorrectly labeled as non-spam.

confusion matrix

To the bottom right of the table is the equation for *accuracy*, which asks what percentage of all the observations (for the spam or pie examples that means all emails or tweets) our system labeled correctly. Although accuracy might seem a natural metric, we generally don't use it for text classification tasks. That's because accuracy doesn't work well when the classes are unbalanced (as indeed they are with spam, which is a large majority of email, or with tweets, which are mainly not about pie).

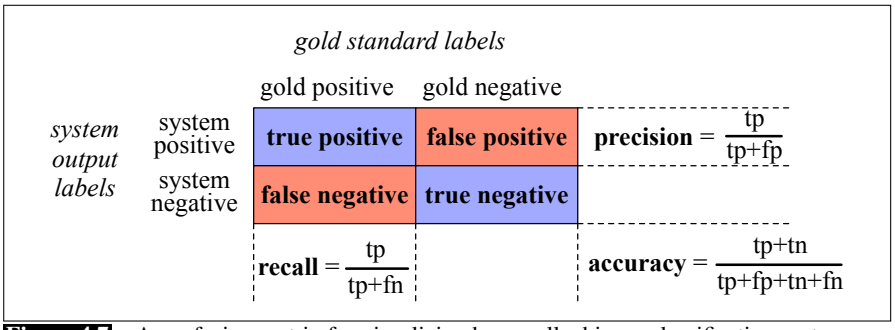


Figure 4.7 A confusion matrix for visualizing how well a binary classification system performs against gold standard labels.

To make this more explicit, imagine that we looked at a million tweets, and let's say that only 100 of them are discussing their love (or hatred) for our pie, while the other 999,900 are tweets about something completely unrelated. Imagine a simple classifier that stupidly classified every tweet as "not about pie". This classifier would have 999,900 true negatives and only 100 false negatives for an accuracy of 999,900/1,000,000 or 99.99%! What an amazing accuracy level! Surely we should be happy with this classifier? But of course this fabulous 'no pie' classifier would be completely useless, since it wouldn't find a single one of the customer comments we are looking for. In other words, accuracy is not a good metric when the goal is to discover something that is rare, or at least not completely balanced in frequency, which is a very common situation in the world.

precision

That's why instead of accuracy we generally turn to two other metrics shown in Fig. 4.7: **precision** and **recall**. **Precision** measures the percentage of the items that the system detected (i.e., the system labeled as positive) that are in fact positive (i.e., are positive according to the human gold labels). Precision is defined as

$$\textbf{Precision} = \frac{\text{true positives}}{\text{true positives + false positives}}$$

recall

Recall measures the percentage of items actually present in the input that were correctly identified by the system. Recall is defined as

$$\textbf{Recall} = \frac{\text{true positives}}{\text{true positives + false negatives}}$$

Precision and recall will help solve the problem with the useless "nothing is pie" classifier. This classifier, despite having a fabulous accuracy of 99.99%, has a terrible recall of 0 (since there are no true positives, and 100 false negatives, the recall is 0/100). You should convince yourself that the precision at finding relevant tweets is equally problematic. Thus precision and recall, unlike accuracy, emphasize true positives: finding the things that we are supposed to be looking for.

F-measure

There are many ways to define a single metric that incorporates aspects of both precision and recall. The simplest of these combinations is the **F-measure** (van Rijsbergen, 1975) , defined as:

$$F_\beta = \frac{(\beta^2+1)PR}{\beta^2 P + R}$$

The β parameter differentially weights the importance of recall and precision, based perhaps on the needs of an application. Values of $\beta > 1$ favor recall, while values of $\beta < 1$ favor precision. When $\beta = 1$, precision and recall are equally balanced; this is the most frequently used metric, and is called $F_{\beta=1}$ or just F_1:

F1

$$F_1 = \frac{2PR}{P+R} \tag{4.42}$$

F-measure comes from a weighted harmonic mean of precision and recall. The harmonic mean of a set of numbers is the reciprocal of the arithmetic mean of reciprocals:

$$\text{HarmonicMean}(a_1, a_2, a_3, a_4, ..., a_n) = \frac{n}{\frac{1}{a_1}+\frac{1}{a_2}+\frac{1}{a_3}+...+\frac{1}{a_n}} \tag{4.43}$$

and hence F-measure is

$$F = \frac{1}{\alpha\frac{1}{P}+(1-\alpha)\frac{1}{R}} \quad \text{or} \left(\text{with } \beta^2 = \frac{1-\alpha}{\alpha}\right) \quad F = \frac{(\beta^2+1)PR}{\beta^2 P + R} \tag{4.44}$$

Harmonic mean is used because the harmonic mean of two values is closer to the minimum of the two values than the arithmetic mean is. Thus it weighs the lower of the two numbers more heavily, which is more conservative in this situation.

4.9.1 Evaluating with more than two classes

The simple example we gave above in defining precision and recall involved text classification with only two classes. But as we saw in Section 4.7, lots of NLP classification tasks have more than two classes. Even for sentiment analysis we often have 3 classes (positive, negative, neutral) and many more classes are common in tasks like text classification or emotion detection, and so on.

To deal with more than two classes we'll need to slightly modify our definitions of precision and recall. Consider the sample confusion matrix for a hypothetical 3-way *one-of* email categorization decision (urgent, normal, spam) shown in Fig. 4.8.

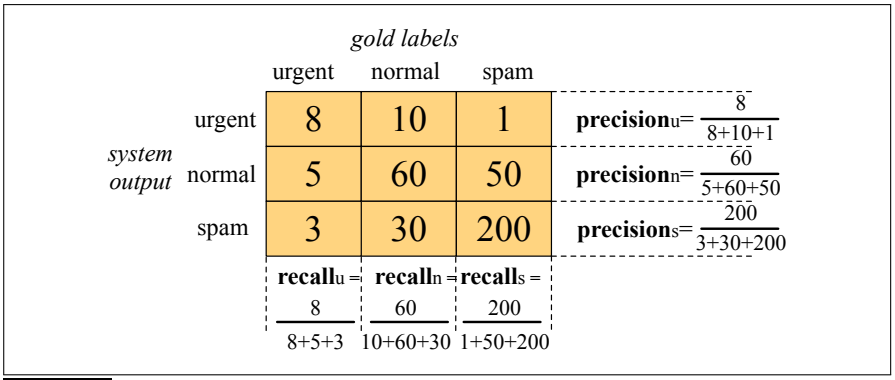


Figure 4.8 Confusion matrix for a three-class categorization task, showing for each pair of classes (c_1, c_2), how many documents from c_1 were (in)correctly assigned to c_2.

The matrix shows, for example, that the system mistakenly labeled one spam document as urgent, and we have shown how to compute a distinct precision and recall value for each class. In order to derive a single metric that tells us how well the system is doing, we can combine these values in two ways. In **macroaveraging**, we compute the performance for each class, and then average over classes. In **microaveraging**, we collect the decisions for all classes into a single confusion matrix, and then compute precision and recall from that table. Fig. 4.9 shows the confusion matrix for each class separately, and shows the computation of microaveraged and macroaveraged precision.

macroaveraging

microaveraging

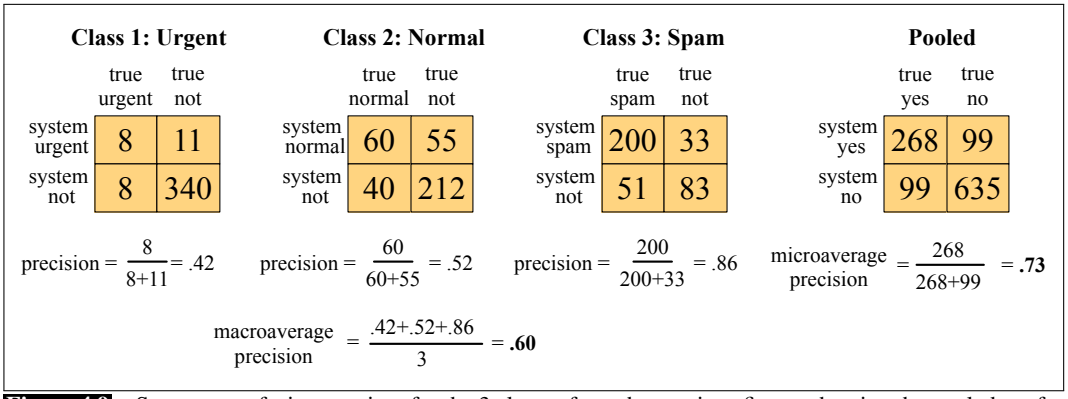


Figure 4.9 Separate confusion matrices for the 3 classes from the previous figure, showing the pooled confusion matrix and the microaveraged and macroaveraged precision.

As the figure shows, a microaverage is dominated by the more frequent class (in this case spam), since the counts are pooled. The macroaverage better reflects the statistics of the smaller classes, and so is more appropriate when performance on all the classes is equally important.

4.10 Test sets and Cross-validation

The training and testing procedure for text classification follows what we saw with language modeling (Section 3.2): we use the training set to train the model, then use the **development test set** (also called a **devset**) to perhaps tune some parameters,

development test set

devset

and in general decide what the best model is. Once we come up with what we think is the best model, we run it on the (hitherto unseen) test set to report its performance.

While the use of a devset avoids overfitting the test set, having a fixed training set, devset, and test set creates another problem: in order to save lots of data for training, the test set (or devset) might not be large enough to be representative. Wouldn't it be better if we could somehow use all our data for training and still use all our data for test? We can do this by **cross-validation**.

cross-validation

In cross-validation, we choose a number k, and partition our data into k disjoint subsets called **folds**. Now we choose one of those k folds as a test set, train our classifier on the remaining $k-1$ folds, and then compute the error rate on the test set. Then we repeat with another fold as the test set, again training on the other $k-1$ folds. We do this sampling process k times and average the test set error rate from these k runs to get an average error rate. If we choose $k = 10$, we would train 10 different models (each on 90% of our data), test the model 10 times, and average these 10 values. This is called **10-fold cross-validation**.

folds

10-fold cross-validation

The only problem with cross-validation is that because all the data is used for testing, we need the whole corpus to be blind; we can't examine any of the data to suggest possible features and in general see what's going on, because we'd be peeking at the test set, and such cheating would cause us to overestimate the performance of our system. However, looking at the corpus to understand what's going on is important in designing NLP systems! What to do? For this reason, it is common to create a fixed training set and test set, then do 10-fold cross-validation inside the training set, but compute error rate the normal way in the test set, as shown in Fig. 4.10.

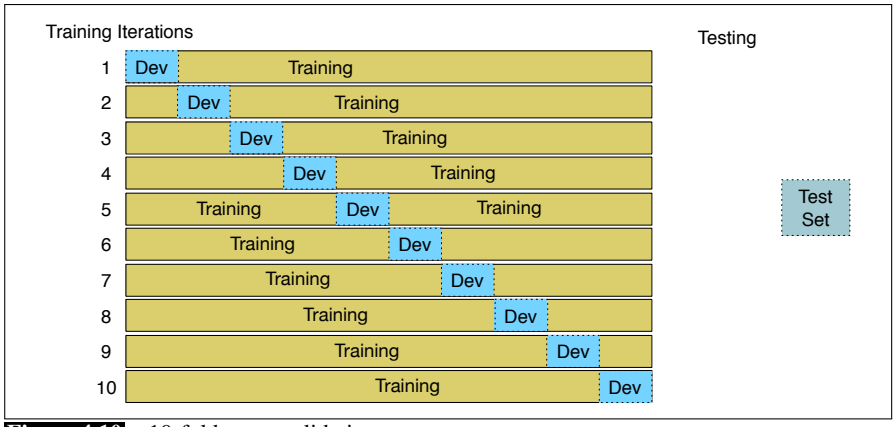


Figure 4.10 10-fold cross-validation

4.11 Statistical Significance Testing

In building systems we often need to compare the performance of two systems. How can we know if the new system we just built is better than our old one? Or better than some other system described in the literature? This is the domain of statistical hypothesis testing, and in this section we introduce tests for statistical significance for NLP classifiers, drawing especially on the work of Dror et al. (2020) and Berg-Kirkpatrick et al. (2012).

Suppose we're comparing the performance of classifiers A and B on a metric M such as F_1, or accuracy. Perhaps we want to know if our new sentiment classifier A gets a higher F_1 score than our previous sentiment classifier B on a particular test set x. Let's call $M(A,x)$ the score that system A gets on test set x, and $\delta(x)$ the performance difference between A and B on x:

$$\delta(x) = M(A,x) - M(B,x) \tag{4.45}$$

effect size

We would like to know if $\delta(x) > 0$, meaning that our new classifier A has a higher F_1 than our old classifier B on x. $\delta(x)$ is called the **effect size**; a bigger δ means that A seems to be way better than B; a small δ means A seems to be only a little better.

Why don't we just check if $\delta(x)$ is positive? Suppose we do, and we find that the F_1 score of A is higher than B's by .04. Can we be certain that A is better? We cannot! That's because A might just be accidentally better than B on this particular x. We need something more: we want to know if A's superiority over B is likely to hold again if we checked another test set x', or under some other set of circumstances.

In the paradigm of statistical hypothesis testing, we test this by formalizing two hypotheses.

$$\begin{aligned} H_0 &: \delta(x) \leq 0 \\ H_1 &: \delta(x) > 0 \end{aligned} \tag{4.46}$$

null hypothesis

The hypothesis H_0, called the **null hypothesis**, supposes that $\delta(x)$ is actually negative or zero, meaning that A is not better than B. We would like to know if we can confidently rule out this hypothesis, and instead support H_1, that A is better.

We do this by creating a random variable X ranging over all test sets. Now we ask how likely is it, if the null hypothesis H_0 was correct, that among these test sets we would encounter the value of $\delta(x)$ that we found, if we repeated the experiment a great many times. We formalize this likelihood as the **p-value**: the probability, assuming the null hypothesis H_0 is true, of seeing the $\delta(x)$ that we saw or one even greater

p-value

$$P(\delta(X) \geq \delta(x) | H_0 \text{ is true}) \tag{4.47}$$

So in our example, this p-value is the probability that we would see $\delta(x)$ assuming A is **not** better than B. If $\delta(x)$ is huge (let's say A has a very respectable F_1 of .9 and B has a terrible F_1 of only .2 on x), we might be surprised, since that would be extremely unlikely to occur if H_0 were in fact true, and so the p-value would be low (unlikely to have such a large δ if A is in fact not better than B). But if $\delta(x)$ is very small, it might be less surprising to us even if H_0 were true and A is not really better than B, and so the p-value would be higher.

A very small p-value means that the difference we observed is very unlikely under the null hypothesis, and we can reject the null hypothesis. What counts as very small? It is common to use values like .05 or .01 as the thresholds. A value of .01 means that if the p-value (the probability of observing the δ we saw assuming H_0 is true) is less than .01, we reject the null hypothesis and assume that A is indeed better than B. We say that a result (e.g., "A is better than B") is **statistically significant** if the δ we saw has a probability that is below the threshold and we therefore reject this null hypothesis.

statistically significant

How do we compute this probability we need for the p-value? In NLP we generally don't use simple parametric tests like t-tests or ANOVAs that you might be familiar with. Parametric tests make assumptions about the distributions of the test

statistic (such as normality) that don't generally hold in our cases. So in NLP we usually use non-parametric tests based on sampling: we artificially create many versions of the experimental setup. For example, if we had lots of different test sets x' we could just measure all the $\delta(x')$ for all the x'. That gives us a distribution. Now we set a threshold (like .01) and if we see in this distribution that 99% or more of those deltas are smaller than the delta we observed, i.e., that p-value(x)—the probability of seeing a $\delta(x)$ as big as the one we saw—is less than .01, then we can reject the null hypothesis and agree that $\delta(x)$ was a sufficiently surprising difference and A is really a better algorithm than B.

approximate randomization

There are two common non-parametric tests used in NLP: **approximate randomization** (Noreen, 1989) and the **bootstrap test**. We will describe bootstrap below, showing the paired version of the test, which again is most common in NLP.

paired

Paired tests are those in which we compare two sets of observations that are aligned: each observation in one set can be paired with an observation in another. This happens naturally when we are comparing the performance of two systems on the same test set; we can pair the performance of system A on an individual observation x_i with the performance of system B on the same x_i.

4.11.1 The Paired Bootstrap Test

bootstrap test

The **bootstrap test** (Efron and Tibshirani, 1993) can apply to any metric; from precision, recall, or F1 to the BLEU metric used in machine translation. The word

bootstrapping

bootstrapping refers to repeatedly drawing large numbers of samples with replacement (called **bootstrap samples**) from an original set. The intuition of the bootstrap test is that we can create many virtual test sets from an observed test set by repeatedly sampling from it. The method only makes the assumption that the sample is representative of the population.

Consider a tiny text classification example with a test set x of 10 documents. The first row of Fig. 4.11 shows the results of two classifiers (A and B) on this test set. Each document is labeled by one of the four possibilities (A and B both right, both wrong, A right and B wrong, A wrong and B right). A slash through a letter ($\not{B}$) means that that classifier got the answer wrong. On the first document both A and B get the correct class (AB), while on the second document A got it right but B got it wrong (A$\not{B}$). If we assume for simplicity that our metric is accuracy, A has an accuracy of .70 and B of .50, so $\delta(x)$ is .20.

Now we create a large number b (perhaps 10^5) of virtual test sets $x^{(i)}$, each of size $n = 10$. Fig. 4.11 shows a couple of examples. To create each virtual test set $x^{(i)}$, we repeatedly ($n = 10$ times) select a cell from row x with replacement. For example, to create the first cell of the first virtual test set $x^{(1)}$, if we happened to randomly select the second cell of the x row, we would copy the value A$\not{B}$ into our new cell, and move on to create the second cell of $x^{(1)}$, each time sampling (randomly choosing) from the original x with replacement.

Now that we have the b test sets, providing a sampling distribution, we can do statistics on how often A has an accidental advantage. There are various ways to compute this advantage; here we follow the version laid out in Berg-Kirkpatrick et al. (2012). Assuming H_0 (A isn't better than B), we would expect that $\delta(X)$, estimated over many test sets, would be zero or negative; a much higher value would be surprising, since H_0 specifically assumes A isn't better than B. To measure exactly how surprising our observed $\delta(x)$ is, we would in other circumstances compute the p-value by counting over many test sets how often $\delta(x^{(i)})$ exceeds the expected zero

	1	2	3	4	5	6	7	8	9	10	A%	B%	$\delta()$
x	AB	A~~B~~	AB	~~A~~B	A~~B~~	~~A~~B	A~~B~~	AB	~~A~~B	A~~B~~	.70	.50	.20
$x^{(1)}$	A~~B~~	AB	A~~B~~	~~A~~B	~~A~~B	A~~B~~	~~A~~B	AB	~~A~~B	AB	.60	.60	.00
$x^{(2)}$	A~~B~~	AB	~~A~~B	~~A~~B	~~A~~B	AB	~~A~~B	A~~B~~	AB	AB	.60	.70	-.10
...													
$x^{(b)}$													

Figure 4.11 The paired bootstrap test: Examples of b pseudo test sets $x^{(i)}$ being created from an initial true test set x. Each pseudo test set is created by sampling $n = 10$ times with replacement; thus an individual sample is a single cell, a document with its gold label and the correct or incorrect performance of classifiers A and B. Of course real test sets don't have only 10 examples, and b needs to be large as well.

value by $\delta(x)$ or more:

$$\text{p-value}(x) = \frac{1}{b}\sum_{i=1}^{b} \mathbb{1}\left(\delta(x^{(i)}) - \delta(x) \geq 0\right)$$

(We use the notation $\mathbb{1}(x)$ to mean "1 if x is true, and 0 otherwise".) However, although it's generally true that the expected value of $\delta(X)$ over many test sets, (again assuming A isn't better than B) is 0, this **isn't** true for the bootstrapped test sets we created. That's because we didn't draw these samples from a distribution with 0 mean; we happened to create them from the original test set x, which happens to be biased (by .20) in favor of A. So to measure how surprising is our observed $\delta(x)$, we actually compute the p-value by counting over many test sets how often $\delta(x^{(i)})$ exceeds the expected value of $\delta(x)$ by $\delta(x)$ or more:

$$\begin{aligned}\text{p-value}(x) &= \frac{1}{b}\sum_{i=1}^{b} \mathbb{1}\left(\delta(x^{(i)}) - \delta(x) \geq \delta(x)\right) \\ &= \frac{1}{b}\sum_{i=1}^{b} \mathbb{1}\left(\delta(x^{(i)}) \geq 2\delta(x)\right) \end{aligned} \tag{4.48}$$

So if for example we have 10,000 test sets $x^{(i)}$ and a threshold of .01, and in only 47 of the test sets do we find that A is accidentally better $\delta(x^{(i)}) \geq 2\delta(x)$, the resulting p-value of .0047 is smaller than .01, indicating that the delta we found, $\delta(x)$ is indeed sufficiently surprising and unlikely to have happened by accident, and we can reject the null hypothesis and conclude A is better than B.

The full algorithm for the bootstrap is shown in Fig. 4.12. It is given a test set x, a number of samples b, and counts the percentage of the b bootstrap test sets in which $\delta(x^{(i)}) > 2\delta(x)$. This percentage then acts as a one-sided empirical p-value.

4.12 Avoiding Harms in Classification

representational harms

It is important to avoid harms that may result from classifiers. One class of harms is **representational harms** (Crawford 2017, Blodgett et al. 2020), harms caused by a system that demeans a social group, for example by perpetuating negative stereotypes about them. For example Kiritchenko and Mohammad (2018) examined the performance of 200 sentiment analysis systems on pairs of sentences that

function BOOTSTRAP(test set x, num of samples b) **returns** p-*value*(x)

Calculate $\delta(x)$ # how much better does algorithm A do than B on x
$s = 0$
for $i = 1$ **to** b **do**
  for $j = 1$ **to** n **do** # Draw a bootstrap sample $x^{(i)}$ of size n
    Select a member of x at random and add it to $x^{(i)}$
  Calculate $\delta(x^{(i)})$ # how much better does algorithm A do than B on $x^{(i)}$
  $s \leftarrow s + 1$ **if** $\delta(x^{(i)}) \geq 2\delta(x)$
p-value(x) $\approx \frac{s}{b}$ # on what % of the b samples did algorithm A beat expectations?
return p-value(x) # if very few did, our observed δ is probably not accidental

Figure 4.12 A version of the paired bootstrap algorithm after Berg-Kirkpatrick et al. (2012).

were identical except for containing either a common African American first name (like *Shaniqua*) or a common European American first name (like *Stephanie*), chosen from the Caliskan et al. (2017) study discussed in Chapter 5. They found that most systems assigned lower sentiment and more negative emotion to sentences with African American names, reflecting and perpetuating stereotypes that associate African Americans with negative emotions (Popp et al., 2003).

toxicity detection

In other tasks classifiers may lead to both representational harms and other harms, such as silencing. For example the important text classification task of **toxicity detection** is the task of detecting hate speech, abuse, harassment, or other kinds of toxic language. While the goal of such classifiers is to help reduce societal harm, toxicity classifiers can themselves cause harms. For example, researchers have shown that some widely used toxicity classifiers incorrectly flag as being toxic sentences that are non-toxic but simply mention identities like women (Park et al., 2018), blind people (Hutchinson et al., 2020) or gay people (Dixon et al., 2018; Dias Oliva et al., 2021), or simply use linguistic features characteristic of varieties like African-American Vernacular English (Sap et al. 2019, Davidson et al. 2019). Such false positive errors could lead to the silencing of discourse by or about these groups.

These model problems can be caused by biases or other problems in the training data; in general, machine learning systems replicate and even amplify the biases in their training data. But these problems can also be caused by the labels (for example due to biases in the human labelers), by the resources used (like lexicons, or model components like pretrained embeddings), or even by model architecture (like what the model is trained to optimize). While the mitigation of these biases (for example by carefully considering the training data sources) is an important area of research, we currently don't have general solutions. For this reason it's important, when introducing any NLP model, to study these kinds of factors and make them clear. One way to do this is by releasing a **model card** (Mitchell et al., 2019) for each version of a model. A model card documents a machine learning model with information like:

model card

- training algorithms and parameters
- training data sources, motivation, and preprocessing
- evaluation data sources, motivation, and preprocessing
- intended use and users
- model performance across different demographic or other groups and envi-

ronmental situations

4.13 Interpreting models

interpretable

Often we want to know more than just the correct classification of an observation. We want to know why the classifier made the decision it did. That is, we want our decision to be **interpretable**. Interpretability can be hard to define strictly, but the core idea is that as humans we should know why our algorithms reach the conclusions they do. Because the features to logistic regression are often human-designed, one way to understand a classifier's decision is to understand the role each feature plays in the decision. Logistic regression can be combined with statistical tests (the likelihood ratio test, or the Wald test); investigating whether a particular feature is significant by one of these tests, or inspecting its magnitude (how large is the weight w associated with the feature?) can help us interpret why the classifier made the decision it makes. This is enormously important for building transparent models.

Furthermore, in addition to its use as a classifier, logistic regression in NLP and many other fields is widely used as an analytic tool for testing hypotheses about the effect of various explanatory variables (features). In text classification, perhaps we want to know if logically negative words (*no, not, never*) are more likely to be associated with negative sentiment, or if negative reviews of movies are more likely to discuss the cinematography. However, in doing so it's necessary to control for potential confounds: other factors that might influence sentiment (the movie genre, the year it was made, perhaps the length of the review in words). Or we might be studying the relationship between NLP-extracted linguistic features and non-linguistic outcomes (hospital readmissions, political outcomes, or product sales), but need to control for confounds (the age of the patient, the county of voting, the brand of the product). In such cases, logistic regression allows us to test whether some feature is associated with some outcome above and beyond the effect of other features.

4.14 Advanced: Regularization

Numquam ponenda est pluralitas sine necessitate
'Plurality should never be proposed unless needed'
William of Occam

overfitting
generalize
regularization

There is a problem with learning weights that make the model perfectly match the training data. If a feature is perfectly predictive of the outcome because it happens to only occur in one class, it will be assigned a very high weight. The weights for features will attempt to perfectly fit details of the training set, in fact too perfectly, modeling noisy factors that just accidentally correlate with the class. This problem is called **overfitting**. A good model should be able to **generalize** well from the training data to the unseen test set, but a model that overfits will have poor generalization.

To avoid overfitting, a new **regularization** term $R(\theta)$ is added to the loss function in Eq. 4.21, resulting in the following loss for a batch of m examples (slightly

rewritten from Eq. 4.21 to be maximizing log probability rather than minimizing loss, and removing the $\frac{1}{m}$ term which doesn't affect the argmax):

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \sum_{i=1}^{m} \log P(y^{(i)}|x^{(i)}) - \alpha R(\theta) \tag{4.49}$$

The new regularization term $R(\theta)$ is used to penalize large weights. Thus a setting of the weights that matches the training data perfectly— but uses many weights with high values to do so—will be penalized more than a setting that matches the data a little less well, but does so using smaller weights. The higher the regularization strength parameter α, the lower the model's weights will be, reducing its reliance on the training data. Note that regularization is not normally applied to the bias term, which acts as a kind of threshold that helps deal with uncentered data and class priors.

L2 regularization

There are two common ways to compute this regularization term $R(\theta)$. **L2 regularization** is a quadratic function of the weight values named because it uses the (square of the) L2 norm of the weight values. The L2 norm, $||\theta||_2$, is the same as the **Euclidean distance** of the vector θ from the origin. If θ consists of n weights, then:

$$R(\theta) = ||\theta||_2^2 = \sum_{j=1}^{n} \theta_j^2 \tag{4.50}$$

The L2 regularized loss function becomes:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \left[\sum_{i=1}^{m} \log P(y^{(i)}|x^{(i)};\theta) \right] - \alpha \sum_{j=1}^{n} \theta_j^2 \tag{4.51}$$

L1 regularization

L1 regularization is a linear function of the weight values, named after the L1 norm $||W||_1$, the sum of the absolute values of the weights, or **Manhattan distance** (the Manhattan distance is the distance you'd have to walk between two points in a city with a street grid like New York):

$$R(\theta) = ||\theta||_1 = \sum_{i=1}^{n} |\theta_i| \tag{4.52}$$

The L1 regularized loss function becomes:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \left[\sum_{i=1}^{m} \log P(y^{(i)}|x^{(i)};\theta) \right] - \alpha \sum_{j=1}^{n} |\theta_j| \tag{4.53}$$

lasso
ridge

These kinds of regularization come from statistics, where L1 regularization is called **lasso regression** (Tibshirani, 1996) and L2 regularization is called **ridge regression**, and both are commonly used in language processing. L2 regularization is easier to optimize because of its simple derivative (the derivative of θ^2 is just 2θ), while L1 regularization is more complex (the derivative of $|\theta|$ is non-continuous at zero). But while L2 prefers weight vectors with many small weights, L1 prefers sparse solutions with some larger weights but many more weights set to zero. Thus L1 regularization leads to much sparser weight vectors, that is, far fewer features. Again, for both these types of regularization we general ignore the bias term and only consider the other weights.

Both L1 and L2 regularization have Bayesian interpretations as constraints on the prior of how weights should look. L1 regularization can be viewed as a Laplace prior on the weights. L2 regularization corresponds to assuming that weights are distributed according to a Gaussian distribution with mean $\mu = 0$. In a Gaussian or normal distribution, the further away a value is from the mean, the lower its probability (scaled by the variance σ). By using a Gaussian prior on the weights, we are saying that weights prefer to have the value 0. A Gaussian for a weight θ_j is

$$\frac{1}{\sqrt{2\pi\sigma_j^2}} \exp\left(-\frac{(\theta_j - \mu_j)^2}{2\sigma_j^2}\right) \tag{4.54}$$

If we multiply each weight by a Gaussian prior on the weight, we are thus maximizing the following constraint:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \prod_{i=1}^{m} P(y^{(i)}|x^{(i)}) \times \prod_{j=1}^{n} \frac{1}{\sqrt{2\pi\sigma_j^2}} \exp\left(-\frac{(\theta_j - \mu_j)^2}{2\sigma_j^2}\right) \tag{4.55}$$

which in log space, with $\mu = 0$, and assuming $2\sigma^2 = 1$, corresponds to

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \sum_{i=1}^{m} \log P(y^{(i)}|x^{(i)}) - \alpha \sum_{j=1}^{n} \theta_j^2 \tag{4.56}$$

which is in the same form as Eq. 4.51.

4.15 Advanced: Deriving the Gradient Equation

In this section we give the derivation of the gradient of the cross-entropy loss function L_{CE} for logistic regression. Let's start with some quick calculus refreshers. First, the derivative of $\ln(x)$:

$$\frac{d}{dx}\ln(x) = \frac{1}{x} \tag{4.57}$$

Second, the (very elegant) derivative of the sigmoid:

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z)) \tag{4.58}$$

chain rule

Finally, the **chain rule** of derivatives. Suppose we are computing the derivative of a composite function $f(x) = u(v(x))$. The derivative of $f(x)$ is the derivative of $u(x)$ with respect to $v(x)$ times the derivative of $v(x)$ with respect to x:

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx} \tag{4.59}$$

First, we want to know the derivative of the loss function with respect to a single weight w_j (we'll need to compute it for each weight, and for the bias):

$$\begin{aligned}\frac{\partial L_{\text{CE}}}{\partial w_j} &= \frac{\partial}{\partial w_j}-[y\log\sigma(\mathbf{w}\cdot\mathbf{x}+b)+(1-y)\log(1-\sigma(\mathbf{w}\cdot\mathbf{x}+b))]\\ &= -\left[\frac{\partial}{\partial w_j}y\log\sigma(\mathbf{w}\cdot\mathbf{x}+b)+\frac{\partial}{\partial w_j}(1-y)\log[1-\sigma(\mathbf{w}\cdot\mathbf{x}+b)]\right]\end{aligned} \tag{4.60}$$

Next, using the chain rule, and relying on the derivative of log:

$$\frac{\partial L_{\text{CE}}}{\partial w_j} = -\frac{y}{\sigma(\mathbf{w}\cdot\mathbf{x}+b)}\frac{\partial}{\partial w_j}\sigma(\mathbf{w}\cdot\mathbf{x}+b)-\frac{1-y}{1-\sigma(\mathbf{w}\cdot\mathbf{x}+b)}\frac{\partial}{\partial w_j}[1-\sigma(\mathbf{w}\cdot\mathbf{x}+b)] \tag{4.61}$$

Rearranging terms:

$$\frac{\partial L_{\text{CE}}}{\partial w_j} = -\left[\frac{y}{\sigma(\mathbf{w}\cdot\mathbf{x}+b)}-\frac{1-y}{1-\sigma(\mathbf{w}\cdot\mathbf{x}+b)}\right]\frac{\partial}{\partial w_j}\sigma(\mathbf{w}\cdot\mathbf{x}+b)$$

And now plugging in the derivative of the sigmoid, and using the chain rule one more time, we end up with Eq. 4.62:

$$\begin{aligned}\frac{\partial L_{\text{CE}}}{\partial w_j} &= -\left[\frac{y-\sigma(\mathbf{w}\cdot\mathbf{x}+b)}{\sigma(\mathbf{w}\cdot\mathbf{x}+b)[1-\sigma(\mathbf{w}\cdot\mathbf{x}+b)]}\right]\sigma(\mathbf{w}\cdot\mathbf{x}+b)[1-\sigma(\mathbf{w}\cdot\mathbf{x}+b)]\frac{\partial(\mathbf{w}\cdot\mathbf{x}+b)}{\partial w_j}\\ &= -\left[\frac{y-\sigma(\mathbf{w}\cdot\mathbf{x}+b)}{\sigma(\mathbf{w}\cdot\mathbf{x}+b)[1-\sigma(\mathbf{w}\cdot\mathbf{x}+b)]}\right]\sigma(\mathbf{w}\cdot\mathbf{x}+b)[1-\sigma(\mathbf{w}\cdot\mathbf{x}+b)]x_j\\ &= -[y-\sigma(\mathbf{w}\cdot\mathbf{x}+b)]x_j\\ &= [\sigma(\mathbf{w}\cdot\mathbf{x}+b)-y]x_j\end{aligned} \tag{4.62}$$

4.16 Summary

This chapter introduced the **logistic regression** model of **classification**.

- Logistic regression is a supervised machine learning classifier that extracts real-valued features from the input, multiplies each by a weight, sums them, and passes the sum through a **sigmoid** function to generate a probability. A threshold is used to make a decision.
- Logistic regression can be used with two classes (e.g., positive and negative sentiment) or with multiple classes (**multinomial logistic regression**, for example for n-ary text classification, part-of-speech labeling, etc.).
- Multinomial logistic regression uses the **softmax** function to compute probabilities.
- The weights (vector w and bias b) are learned from a labeled training set via a loss function, such as the **cross-entropy loss**, that must be minimized.
- Minimizing this loss function is a **convex optimization** problem, and iterative algorithms like **gradient descent** are used to find the optimal weights.
- **Regularization** is used to avoid overfitting.
- Logistic regression is also one of the most useful analytic tools, because of its ability to transparently study the importance of individual features.

Historical Notes

Logistic regression was developed in the field of statistics, where it was used for the analysis of binary data by the 1960s, and was particularly common in medicine (Cox, 1969). Starting in the late 1970s it became widely used in linguistics as one of the formal foundations of the study of linguistic variation (Sankoff and Labov, 1979).

Nonetheless, logistic regression didn't become common in natural language processing until the 1990s, when it seems to have appeared simultaneously from two directions. The first source was the neighboring fields of information retrieval and speech processing, both of which had made use of regression, and both of which lent many other statistical techniques to NLP. Indeed a very early use of logistic regression for document routing was one of the first NLP applications to use (LSI) embeddings as word representations (Schütze et al., 1995).

maximum entropy

At the same time in the early 1990s logistic regression was developed and applied to NLP at IBM Research under the name **maximum entropy** modeling or **maxent** (Berger et al., 1996), seemingly independent of the statistical literature. Under that name it was applied to language modeling (Rosenfeld, 1996), part-of-speech tagging (Ratnaparkhi, 1996), parsing (Ratnaparkhi, 1997), coreference resolution (Kehler, 1997b), and text classification (Nigam et al., 1999).

There are a variety of sources covering the many kinds of text classification tasks. For sentiment analysis see Pang and Lee (2008), and Liu and Zhang (2012). Stamatatos (2009) surveys authorship attribute algorithms. On language identification see Jauhiainen et al. (2019); Jaech et al. (2016) is an important early neural system. The task of newswire indexing was often used as a test case for text classification algorithms, based on the Reuters-21578 collection of newswire articles.

See Manning et al. (2008) and Aggarwal and Zhai (2012) on text classification; classification in general is covered in machine learning textbooks (Hastie et al. 2001, Witten and Frank 2005, Bishop 2006, Murphy 2012).

Non-parametric methods for computing statistical significance were used first in NLP in the MUC competition (Chinchor et al., 1993), and even earlier in speech recognition (Gillick and Cox 1989, Bisani and Ney 2004). Our description of the bootstrap draws on the description in Berg-Kirkpatrick et al. (2012). Recent work has focused on issues including multiple test sets and multiple metrics (Søgaard et al. 2014, Dror et al. 2017).

information gain

Feature selection is a method of removing features that are unlikely to generalize well. Features are generally ranked by how informative they are about the classification decision. A very common metric, **information gain**, tells us how many bits of information the presence of the word gives us for guessing the class. Other feature selection metrics include χ^2, pointwise mutual information, and GINI index; see Yang and Pedersen (1997) for a comparison and Guyon and Elisseeff (2003) for an introduction to feature selection.

Exercises

CHAPTER

5 Embeddings

荃者所以在鱼，得鱼而忘荃 Nets are for fish;
Once you get the fish, you can forget the net.
言者所以在意，得意而忘言 Words are for meaning;
Once you get the meaning, you can forget the words
庄子(Zhuangzi), Chapter 26

The asphalt that Los Angeles is famous for occurs mainly on its freeways. But in the middle of the city is another patch of asphalt, the La Brea tar pits, and this asphalt preserves millions of fossil bones from the last of the Ice Ages of the Pleistocene Epoch. One of these fossils is the *Smilodon*, or saber-toothed tiger, instantly recognizable by its long canines. Five million years ago or so, a completely different saber-tooth tiger called *Thylacosmilus* lived in Argentina and other parts of South America. Thylacosmilus was a marsupial whereas Smilodon was a placental mammal, but Thylacosmilus had the same long upper canines and, like Smilodon, had a protective bone flange on the lower jaw. The similarity of these two mammals is one of many examples of parallel or convergent evolution, in which particular contexts or environments lead to the evolution of very similar structures in different species (Gould, 1980).

The role of context is also important in the similarity of a less biological kind of organism: the word. Words that occur in *similar contexts* tend to have *similar meanings*. This link between similarity in how words are distributed and similarity in what they mean is called the **distributional hypothesis**. The hypothesis was first formulated in the 1950s by linguists like Joos (1950), Harris (1954), and Firth (1957), who noticed that words which are synonyms (like *oculist* and *eye-doctor*) tended to occur in the same environment (e.g., near words like *eye* or *examined*) with the amount of meaning difference between two words "corresponding roughly to the amount of difference in their environments" (Harris, 1954, p. 157).

distributional hypothesis

In this chapter we introduce **embeddings**, vector representations of the meaning of words that are learned directly from word distributions in texts. Embeddings lie at the heart of large language models and other modern applications. The **static embeddings** we introduce here underlie the more powerful dynamic or **contextualized embeddings** like **BERT** that we will see in Chapter 9 and Chapter 8.

embeddings

The linguistic field that studies embeddings and their meanings is called **vector semantics**. Embeddings are also the first example in this book of **representation learning**, automatically learning useful representations of the input text. Finding such **self-supervised** ways to learn representations of language, instead of creating representations by hand via **feature engineering**, is an important principle of modern NLP (Bengio et al., 2013).

vector semantics
representation learning

5.1 Lexical Semantics

Let's begin by introducing some basic principles of word meaning. How should we represent the meaning of a word? In the n-gram models of Chapter 3, and in classical NLP applications, our only representation of a word is as a string of letters, or an index in a vocabulary list. This representation is not that different from a tradition in philosophy, perhaps you've seen it in introductory logic classes, in which the meaning of words is represented by just spelling the word with small capital letters; representing the meaning of "dog" as DOG, and "cat" as CAT, or by using an apostrophe (DOG').

Representing the meaning of a word by capitalizing it is a pretty unsatisfactory model. You might have seen a version of a joke due originally to semanticist Barbara Partee (Carlson, 1977):

> Q: What's the meaning of life?
> A: LIFE'

Surely we can do better than this! After all, we'll want a model of word meaning to do all sorts of things for us. It should tell us that some words have similar meanings (*cat* is similar to *dog*), others are antonyms (*cold* is the opposite of *hot*), some have positive connotations (*happy*) while others have negative connotations (*sad*). It should represent the fact that the meanings of *buy*, *sell*, and *pay* offer differing perspectives on the same underlying purchasing event. (If I buy something from you, you've probably sold it to me, and I likely paid you.) More generally, a model of word meaning should allow us to draw inferences to address meaning-related tasks like question-answering or dialogue.

lexical semantics

In this section we summarize some of these desiderata, drawing on results in the linguistic study of word meaning, which is called **lexical semantics**; we'll return to and expand on this list in Appendix G and Chapter 21.

Lemmas and Senses Let's start by looking at how one word (we'll choose *mouse*) might be defined in a dictionary (simplified from the online dictionary WordNet):

```
mouse (N)
1.  any of numerous small rodents...
2.  a hand-operated device that controls a cursor...
```

lemma
citation form

Here the form *mouse* is the **lemma**, also called the **citation form**. The form *mouse* would also be the lemma for the word *mice*; dictionaries don't have separate definitions for inflected forms like *mice*. Similarly *sing* is the lemma for *sing*, *sang*, *sung*. In many languages the infinitive form is used as the lemma for the verb, so Spanish *dormir* "to sleep" is the lemma for *duermes* "you sleep". The specific forms *sung* or *carpets* or *sing* or *duermes* are called **wordforms**.

wordform

As the example above shows, each lemma can have multiple meanings; the lemma *mouse* can refer to the rodent or the cursor control device. We call each of these aspects of the meaning of *mouse* a **word sense**. The fact that lemmas can be **polysemous** (have multiple senses) can make interpretation difficult (is someone who searches for "mouse info" looking for a pet or a widget?). Chapter 9 and Appendix G will discuss the problem of polysemy, and introduce **word sense disambiguation**, the task of determining which sense of a word is being used in a particular context.

Synonymy One important component of word meaning is the relationship between word senses. For example when one word has a sense whose meaning is

identical to a sense of another word, or nearly identical, we say the two senses of those two words are **synonyms**. Synonyms include such pairs as

synonym

couch/sofa vomit/throw up filbert/hazelnut car/automobile

A more formal definition of synonymy (between words rather than senses) is that two words are synonymous if they are substitutable for one another in any sentence without changing the *truth conditions* of the sentence, the situations in which the sentence would be true.

While substitutions between some pairs of words like *car* / *automobile* or *water* / H_2O are truth preserving, the words are still not identical in meaning. Indeed, probably no two words are absolutely identical in meaning. One of the fundamental tenets of semantics, called the **principle of contrast** (Girard 1718, Bréal 1897, Clark 1987), states that a difference in linguistic form is always associated with some difference in meaning. For example, the word H_2O is used in scientific contexts and would be inappropriate in a hiking guide—*water* would be more appropriate— and this genre difference is part of the meaning of the word. In practice, the word *synonym* is therefore used to describe a relationship of approximate or rough synonymy.

principle of contrast

Word Similarity While words don't have many synonyms, most words do have lots of *similar* words. *Cat* is not a synonym of *dog*, but *cats* and *dogs* are certainly similar words. In moving from synonymy to similarity, it will be useful to shift from talking about relations between word senses (like synonymy) to relations between words (like similarity). Dealing with words avoids having to commit to a particular representation of word senses, which will turn out to simplify our task.

The notion of word **similarity** is very useful in larger semantic tasks. Knowing how similar two words are can help in computing how similar the meaning of two phrases or sentences are, a very important component of tasks like question answering, paraphrasing, and summarization. One way of getting values for word similarity is to ask humans to judge how similar one word is to another. A number of datasets have resulted from such experiments. For example the SimLex-999 dataset (Hill et al., 2015) gives values on a scale from 0 to 10, like the examples below, which range from near-synonyms (*vanish*, *disappear*) to pairs that scarcely seem to have anything in common (*hole*, *agreement*):

similarity

vanish	disappear	9.8
belief	impression	5.95
muscle	bone	3.65
modest	flexible	0.98
hole	agreement	0.3

Word Relatedness The meaning of two words can be related in ways other than similarity. One such class of connections is called word **relatedness** (Budanitsky and Hirst, 2006), also traditionally called word **association** in psychology.

relatedness

association

Consider the meanings of the words *coffee* and *cup*. Coffee is not similar to cup; they share practically no features (coffee is a plant or a beverage, while a cup is a manufactured object with a particular shape). But coffee and cup are clearly related; they are associated by co-participating in an everyday event (the event of drinking coffee out of a cup). Similarly *scalpel* and *surgeon* are not similar but are related eventively (a surgeon tends to make use of a scalpel).

One common kind of relatedness between words is if they belong to the same **semantic field**. A semantic field is a set of words which cover a particular semantic domain and bear structured relations with each other. For example, words might be

semantic field

related by being in the semantic field of hospitals (*surgeon*, *scalpel*, *nurse*, *anesthetic*, *hospital*), restaurants (*waiter*, *menu*, *plate*, *food*, *chef*), or houses (*door*, *roof*, *kitchen*, *family*, *bed*). Semantic fields are also related to **topic models**, like **Latent Dirichlet Allocation**, **LDA**, which apply unsupervised learning on large sets of texts to induce sets of associated words from text. Semantic fields and topic models are very useful tools for discovering topical structure in documents.

topic models

In Appendix G we'll introduce more relations between senses like **hypernymy** or **IS-A**, **antonymy** (opposites) and **meronymy** (part-whole relations).

connotations

Connotation Finally, words have *affective meanings* or **connotations**. The word *connotation* has different meanings in different fields, but here we use it to mean the aspects of a word's meaning that are related to a writer or reader's emotions, sentiment, opinions, or evaluations. For example some words have positive connotations (*wonderful*) while others have negative connotations (*dreary*). Even words whose meanings are similar in other ways can vary in connotation; consider the difference in connotations between *fake*, *knockoff*, *forgery*, on the one hand, and *copy*, *replica*, *reproduction* on the other, or *innocent* (positive connotation) and *naive* (negative connotation). Some words describe positive evaluation (*great*, *love*) and others negative evaluation (*terrible*, *hate*). Positive or negative evaluation language is called **sentiment**, as we saw in Appendix K, and word sentiment plays a role in important tasks like sentiment analysis, stance detection, and applications of NLP to the language of politics and consumer reviews.

sentiment

Early work on affective meaning (Osgood et al., 1957) found that words varied along three important dimensions of affective meaning:

valence: the pleasantness of the stimulus

arousal: the intensity of emotion provoked by the stimulus

dominance: the degree of control exerted by the stimulus

Thus words like *happy* or *satisfied* are high on valence, while *unhappy* or *annoyed* are low on valence. *Excited* is high on arousal, while *calm* is low on arousal. *Controlling* is high on dominance, while *awed* or *influenced* are low on dominance. Each word is thus represented by three numbers, corresponding to its value on each of the three dimensions:

	Valence	Arousal	Dominance
courageous	8.0	5.5	7.4
music	7.7	5.6	6.5
heartbreak	2.5	5.7	3.6
cub	6.7	4.0	4.2

Osgood et al. (1957) noticed that in using these 3 numbers to represent the meaning of a word, the model was representing each word as a point in a three-dimensional space, a vector whose three dimensions corresponded to the word's rating on the three scales. This revolutionary idea that word meaning could be represented as a point in space (e.g., that part of the meaning of *heartbreak* can be represented as the point $[2.5, 5.7, 3.6]$) was the first expression of the vector semantics models that we introduce next.

5.2 Vector Semantics: The Intuition

vector semantics

Vector semantics is the standard way to represent word meaning in NLP, helping

us model many of the aspects of word meaning we saw in the previous section. The roots of the model lie in the 1950s when two big ideas converged: Osgood's 1957 idea mentioned above to use a point in three-dimensional space to represent the connotation of a word, and the proposal by linguists like Joos (1950), Harris (1954), and Firth (1957) to define the meaning of a word by its **distribution** in language use, meaning its neighboring words or grammatical environments. Their idea was that two words that occur in very similar distributions (whose neighboring words are similar) have similar meanings.

For example, suppose you didn't know the meaning of the word *ongchoi* (a recent borrowing from Cantonese) but you see it in the following contexts:

(5.1) Ongchoi is delicious sauteed with garlic.

(5.2) Ongchoi is superb over rice.

(5.3) ...ongchoi leaves with salty sauces...

And suppose that you had seen many of these context words in other contexts:

(5.4) ...spinach sauteed with garlic over rice...

(5.5) ...chard stems and leaves are delicious...

(5.6) ...collard greens and other salty leafy greens

The fact that *ongchoi* occurs with words like *rice* and *garlic* and *delicious* and *salty*, as do words like *spinach*, *chard*, and *collard greens* might suggest that ongchoi is a leafy green similar to these other leafy greens.[1] We can implement the same intuition computationally by just counting words in the context of *ongchoi*.

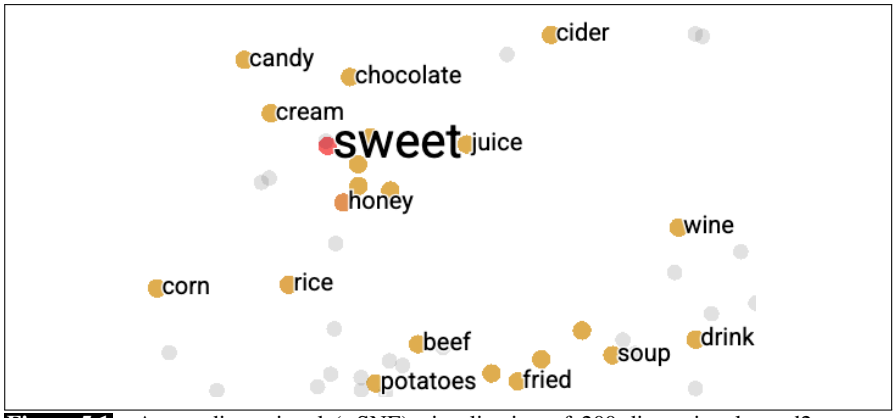


Figure 5.1 A two-dimensional (t-SNE) visualization of 200-dimensional word2vec embeddings for some words close to the word *sweet*, showing that words with similar meanings are nearby in space. Visualization created using the TensorBoard Embedding Projector https://projector.tensorflow.org/.

embeddings

The idea of vector semantics is to represent a word as a point in a multidimensional semantic space that is derived (in different ways we'll see) from the distributions of word neighbors. Vectors for representing words are called **embeddings**. The word "embedding" derives historically from its mathematical sense as a mapping from one space or structure to another, although the meaning has shifted; see the end of the chapter.

Fig. 5.1 shows a visualization of embeddings learned by the word2vec algorithm, showing the location of selected words (neighbors of "sweet") projected down from

[1] It's in fact *Ipomoea aquatica*, a relative of morning glory sometimes called *water spinach* in English.

200-dimensional space into a 2-dimensional space. Note that the nearest neighbors of sweet are semantically related words like honey, candy, juice, chocolate. This idea that similar words are neighbors in high-dimensional space offers enormous power to language models and other NLP applications. For example the sentiment classifiers of Chapter 4 depend on the same words appearing in the training and test sets. But by representing words as embeddings, a classifier can assign sentiment as long as it sees some words with *similar meanings*. And as we'll see, vector semantic models like the ones showed in Fig. 5.1 can be learned automatically from text without supervision.

In this chapter we'll begin with a simple pedagogical model of embeddings in which the meaning of a word is defined by a vector with the counts of nearby words. We introduce this model as a helpful way to understand the concept of vectors and what it means for a vector to be a representation of word meaning, but more sophisticated variants like the **tf-idf model** we will introduce in Chapter 11 are important methods you should understand. We will see that this method results in very long vectors that are **sparse**, i.e. mostly zeros (since most words simply never occur in the context of others). We'll then introduce the **word2vec** model family for constructing short, **dense** vectors that have even more useful semantic properties.

We'll also introduce the **cosine**, the standard way to use embeddings to compute *semantic similarity*, between two words, two sentences, or two documents, an important tool in practical applications.

5.3 Simple count-based embeddings

"The most important attributes of a vector in 3-space are {Location, Location, Location}"
Randall Munroe, the hover from https://xkcd.com/2358/

Let's now introduce the first way to compute word vector embeddings. This simplest vector model of meaning is based on the **co-occurrence matrix**, a way of representing how often words co-occur. We'll define a particular kind of co-occurrence matrix, the **word-context matrix**, in which each row in the matrix represents a word in the vocabulary and each column represents how often each other word in the vocabulary appears nearby. This matrix is thus of dimensionality $|V| \times |V|$ and each cell records the number of times the row (target) word and the column (context) word co-occur nearby in some training corpus.

word-context matrix

What do we mean by 'nearby'? We could implement various methods, but let's start with a very simple one: a context window around the word, let's say of 4 words to the left and 4 words to the right. If we do that, each cell will represents the number of times (in some training corpus) the column word occurs in such a ± 4 word window around the row word.

Let's see how this works for 4 words: *cherry*, *strawberry*, *digital*, and *information*. For each word we took a single instance from a corpus, and we show the ± 4 word window from that instance:

is traditionally followed by	**cherry**	pie, a traditional dessert
often mixed, such as	**strawberry**	rhubarb pie. Apple pie
computer peripherals and personal	**digital**	assistants. These devices usually
a computer. This includes	**information**	available on the internet

If we then take every occurrence of each word in a large corpus and count the context words around it, we get a word-context co-occurrence matrix. The full word-

context co-occurrence matrix is very large, because for each word in the vocabulary (since $|V|$) we have to count how often it occurs with every other word in the vocabulary, hence dimensionality $|V| \times |V|$. Let's therefore instead sketch the process on a smaller scale. Imagine that we are going to look at only the 4 words, and only consider the following 3 context words: *a*, *computer*, and *pie*. Furthermore let's assume we only count occurrences in the mini-corpus above.

So before looking at Fig. 5.2, compute by hand the counts for these 3 context words for the four words *cherry*, *strawberry*, *digital*, and *information*.

	a	**computer**	**pie**
cherry	1	0	1
strawberry	0	0	2
digital	0	1	0
information	1	1	0

Figure 5.2 Co-occurrence vectors for four words with counts from the 4 windows above, showing just 3 of the potential context word dimensions. The vector for *cherry* is outlined in red. Note that a real vector would have vastly more dimensions and thus be even sparser.

Hopefully your count matches what is shown in Fig. 5.2, so that each cell represents the number of times a particular word (defined by the row) occurs in a particular context (defined by the word column).

vector

Each row, then, is a vector representing a word. To review some basic linear algebra, a **vector** is, at heart, just a list or array of numbers. So *cherry* is represented as the list [1,0,1] (the first **row vector** in Fig. 5.2) and *information* is represented as the list [1,1,0] (the fourth row vector).

vector space
dimension

A **vector space** is a collection of vectors, and is characterized by its **dimension**. Vectors in a 3-dimensional vector space have an element for each dimension of the space. We will loosely refer to a vector in a 3-dimensional space as a 3-dimensional vector, with one element along each dimension. In the example in Fig. 5.2, we've chosen to make the document vectors of dimension 3, just so they fit on the page; in real term-document matrices, the document vectors would have dimensionality $|V|$, the vocabulary size.

The ordering of the numbers in a vector space indicates the different dimensions on which documents vary. The third dimension for all these vectors corresponds to the number of times *pie* occurs in the context. The second dimension for all of them corresponds to the number of times the word *computer* occurs. Notice that the vectors for *information* and *digital* have the same value (1) for this "computer" dimension.

In reality, we don't compute word vectors on a single context window. Instead, we compute them over an entire corpus. Let's see what some real counts look like. Let's look at some vectors computed in this way. Fig. 5.3 shows a subset of the word-word co-occurrence matrix for these four words, where, again because it's impossible to visualize all $|V|$ possible context words on the page of this textbook, we show a subset of 6 of the dimensions, with counts computed from the Wikipedia corpus (Davies, 2015).

Note in Fig. 5.3 that the two words *cherry* and *strawberry* are more similar to each other (both *pie* and *sugar* tend to occur in their window) than they are to other words like *digital*; conversely, *digital* and *information* are more similar to each other than, say, to *strawberry*.

We can think of the vector for a document as a point in $|V|$-dimensional space; thus the documents in Fig. 5.3 are points in 3-dimensional space. Fig. 5.4 shows a spatial visualization.

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Figure 5.3 Co-occurrence vectors for four words in the Wikipedia corpus, showing six of the dimensions (hand-picked for pedagogical purposes). The vector for *digital* is outlined in red. Note that a real vector would have vastly more dimensions and thus be much sparser, i.e. would have zero values in most dimensions.

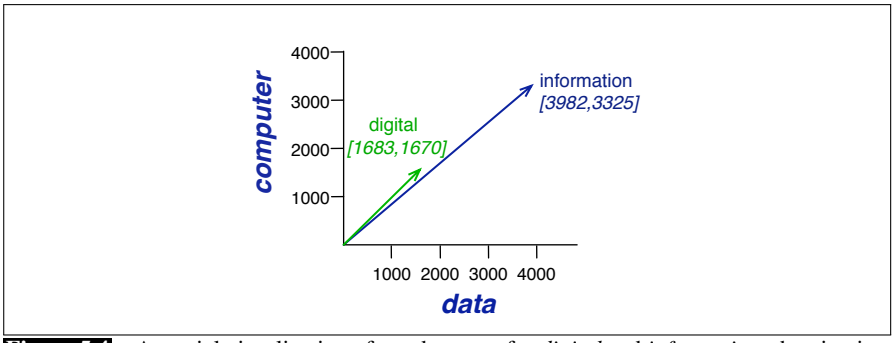


Figure 5.4 A spatial visualization of word vectors for *digital* and *information*, showing just two of the dimensions, corresponding to the words *data* and *computer*.

Note that $|V|$, the dimensionality of the vector, is generally the size of the vocabulary, often between 10,000 and 50,000 words (using the most frequent words in the training corpus; keeping words after about the most frequent 50,000 or so is generally not helpful). Since most of these numbers are zero these are **sparse** vector representations; there are efficient algorithms for storing and computing with sparse matrices.

It's also possible to apply various kinds of weighting functions to the counts in these cells. The most popular such weighting is tf-idf, which we'll introduce in Chapter 11, but there have historically been a wide variety of other weightings.

Now that we have some intuitions, let's move on to examine the details of computing word similarity.

5.4 Cosine for measuring similarity

To measure similarity between two target words v and w, we need a metric that takes two vectors (of the same dimensionality, either both with words as dimensions, hence of length $|V|$, or both with documents as dimensions, of length $|D|$) and gives a measure of their similarity. By far the most common similarity metric is the **cosine** of the angle between the vectors.

dot product
inner product

The cosine—like most measures for vector similarity used in NLP—is based on the **dot product** operator from linear algebra, also called the **inner product**:

$$\text{dot product}(\mathbf{v},\mathbf{w}) = \mathbf{v}\cdot\mathbf{w} = \sum_{i=1}^{N} v_i w_i = v_1 w_1 + v_2 w_2 + \ldots + v_N w_N \tag{5.7}$$

The dot product acts as a similarity metric because it will tend to be high just when the two vectors have large values in the same dimensions. Alternatively, vectors that

have zeros in different dimensions—orthogonal vectors—will have a dot product of 0, representing their strong dissimilarity.

This raw dot product, however, has a problem as a similarity metric: it favors long vectors. The **vector length** is defined as

vector length

$$|\mathbf{v}| = \sqrt{\sum_{i=1}^{N} v_i^2} \tag{5.8}$$

The dot product is higher if a vector is longer, with higher values in each dimension. More frequent words have longer vectors, since they tend to co-occur with more words and have higher co-occurrence values with each of them. The raw dot product thus will be higher for frequent words. But this is a problem; we'd like a similarity metric that tells us how similar two words are regardless of their frequency.

We modify the dot product to normalize for the vector length by dividing the dot product by the lengths of each of the two vectors. This **normalized dot product** turns out to be the same as the cosine of the angle between the two vectors, following from the definition of the dot product between two vectors $\mathbf{a}$ and $\mathbf{b}$:

$$\begin{aligned} \mathbf{a} \cdot \mathbf{b} &= |\mathbf{a}||\mathbf{b}| \cos\theta \\ \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}||\mathbf{b}|} &= \cos\theta \end{aligned} \tag{5.9}$$

cosine

The **cosine** similarity metric between two vectors $\mathbf{v}$ and $\mathbf{w}$ thus can be computed as:

$$\text{cosine}(\mathbf{v},\mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{|\mathbf{v}||\mathbf{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}} \tag{5.10}$$

unit vector

For some applications we pre-normalize each vector, by dividing it by its length, creating a **unit vector** of length 1. Thus we could compute a unit vector from $\mathbf{a}$ by dividing it by $|\mathbf{a}|$. For unit vectors, the dot product is the same as the cosine.

The cosine value ranges from 1 for vectors pointing in the same direction, through 0 for orthogonal vectors, to -1 for vectors pointing in opposite directions. But since raw frequency values are non-negative, the cosine for these vectors ranges from 0–1.

Let's see how the cosine computes which of the words *cherry* or *digital* is closer in meaning to *information*, just using raw counts from the following shortened table:

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

$$\begin{aligned} \cos(\text{cherry},\text{information}) &= \frac{442*5+8*3982+2*3325}{\sqrt{442^2+8^2+2^2}\sqrt{5^2+3982^2+3325^2}} = .018 \\ \cos(\text{digital},\text{information}) &= \frac{5*5+1683*3982+1670*3325}{\sqrt{5^2+1683^2+1670^2}\sqrt{5^2+3982^2+3325^2}} = .996 \end{aligned}$$

The model decides that *information* is way closer to *digital* than it is to *cherry*, a result that seems sensible. Fig. 5.5 shows a visualization.

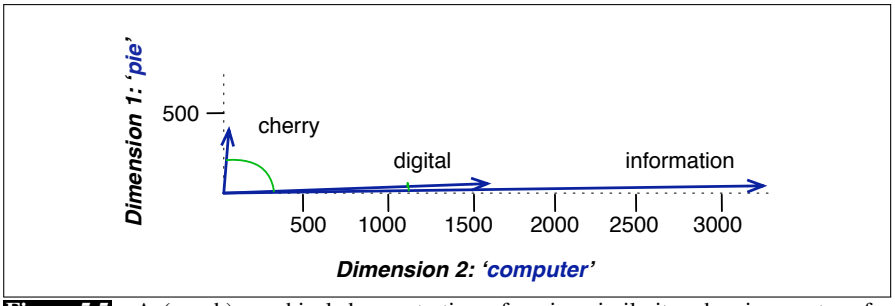


Figure 5.5 A (rough) graphical demonstration of cosine similarity, showing vectors for three words (*cherry*, *digital*, and *information*) in the two dimensional space defined by counts of the words *computer* and *pie* nearby. The figure doesn't show the cosine, but it highlights the angles; note that the angle between *digital* and *information* is smaller than the angle between *cherry* and *information*. When two vectors are more similar, the cosine is larger but the angle is smaller; the cosine has its maximum (1) when the angle between two vectors is smallest ($0°$); the cosine of all other angles is less than 1.

Cosine similarity can be used to estimate word similarity, for tasks like finding word paraphrases, tracking changes in word meaning, or automatically discovering meanings of words in different corpora. For example, we can fine the 10 most similar words to any target word w by computing the cosines between w and each of the $|V|-1$ other words, sorting, and looking at the top 10.

5.5 Word2vec

In the previous sections we saw how to represent a word as a sparse, long vector with dimensions corresponding to words in the vocabulary. We now introduce a more powerful word representation: **embeddings**, short dense vectors. Unlike the vectors we've seen so far, embeddings are **short**, with number of dimensions d ranging from 50-1000, rather than the much larger vocabulary size $|V|$.These d dimensions don't have a clear interpretation. And the vectors are **dense**: instead of vector entries being sparse, mostly-zero counts or functions of counts, the values will be real-valued numbers that can be negative.

It turns out that dense vectors work better in every NLP task than sparse vectors. While we don't completely understand all the reasons for this, we have some intuitions. Representing words as 300-dimensional dense vectors requires our classifiers to learn far fewer weights than if we represented words as 50,000-dimensional vectors, and the smaller parameter space possibly helps with generalization and avoiding overfitting. Dense vectors may also do a better job of capturing synonymy. For example, in a sparse vector representation, dimensions for synonyms like *car* and *automobile* dimension are distinct and unrelated; sparse vectors may thus fail to capture the similarity between a word with *car* as a neighbor and a word with *automobile* as a neighbor.

skip-gram
SGNS
word2vec

In this section we introduce one method for computing embeddings: **skip-gram with negative sampling**, sometimes called **SGNS**. The skip-gram algorithm is one of two algorithms in a software package called **word2vec**, and so sometimes the algorithm is loosely referred to as word2vec (Mikolov et al. 2013a, Mikolov et al. 2013b). The word2vec methods are fast, efficient to train, and easily available online with code and pretrained embeddings. Word2vec embeddings are **static em-**

static embeddings

beddings, meaning that the method learns one fixed embedding for each word in the vocabulary. In Chapter 9 we'll introduce methods for learning dynamic **contextual embeddings** like the popular family of **BERT** representations, in which the vector for each word is different in different contexts.

The intuition of word2vec is that instead of counting how often each context word c occurs near, say, *apricot*, we'll instead train a classifier on a binary prediction task: "Is word c likely to show up near *apricot*?" We don't actually care about this prediction task; instead we'll take the learned classifier *weights* as the word embeddings.

The revolutionary intuition here is that we can just use running text as implicitly supervised training data for such a classifier; a word c that occurs near the target word *apricot* acts as gold 'correct answer' to the question "Is word c likely to show up near *apricot*?" This method, often called **self-supervision**, avoids the need for any sort of hand-labeled supervision signal. This idea was first proposed in the task of neural language modeling, when Bengio et al. (2003) and Collobert et al. (2011) showed that a neural language model (a neural network that learned to predict the next word from prior words) could just use the next word in running text as its supervision signal, and could be used to learn an embedding representation for each word as part of doing this prediction task.

self-supervision

We'll see how to do neural networks in the next chapter, but word2vec is a much simpler model than the neural network language model, in two ways. First, word2vec simplifies the task (making it binary classification instead of word prediction). Second, word2vec simplifies the architecture (training a logistic regression classifier instead of a multi-layer neural network with hidden layers that demand more sophisticated training algorithms). The intuition of skip-gram is:

1. Treat the target word and a neighboring context word as positive examples.
2. Randomly sample other words in the lexicon to get negative samples.
3. Use logistic regression to train a classifier to distinguish those two cases.
4. Use the learned weights as the embeddings.

5.5.1 The classifier

Let's start by thinking about the classification task, and then turn to how to train. Imagine a sentence like the following, with a target word *apricot*, and assume we're using a window of ±2 context words:

```
... lemon,  a [tablespoon of apricot jam,      a] pinch ...
               c1         c2    w     c3       c4
```

Our goal is to train a classifier such that, given a tuple (w,c) of a target word w paired with a candidate context word c (for example (*apricot*, *jam*), or perhaps (*apricot*, *aardvark*)) it will return the probability that c is a real context word (true for *jam*, false for *aardvark*):

$$P(+|w,c) \tag{5.11}$$

The probability that word c is not a real context word for w is just 1 minus Eq. 5.11:

$$P(-|w,c) = 1 - P(+|w,c) \tag{5.12}$$

How does the classifier compute the probability P? The intuition of the skip-gram model is to base this probability on embedding similarity: a word is likely to occur near the target if its embedding vector is similar to the target embedding. To compute similarity between these dense embeddings, we rely on the intuition that two vectors are similar if they have a high **dot product** (after all, cosine is just a normalized dot product). In other words:

$$Similarity(w,c) \approx \mathbf{c} \cdot \mathbf{w} \tag{5.13}$$

The dot product $\mathbf{c} \cdot \mathbf{w}$ is not a probability, it's just a number ranging from $-\infty$ to ∞ (since the elements in word2vec embeddings can be negative, the dot product can be negative). To turn the dot product into a probability, we'll use the **logistic** or **sigmoid** function $\sigma(x)$, the fundamental core of logistic regression:

$$\sigma(x) = \frac{1}{1+\exp(-x)} \tag{5.14}$$

We model the probability that word c is a real context word for target word w as:

$$P(+|w,c) = \sigma(\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1+\exp(-\mathbf{c} \cdot \mathbf{w})} \tag{5.15}$$

The sigmoid function returns a number between 0 and 1, but to make it a probability we'll also need the total probability of the two possible events (c is a context word, and c isn't a context word) to sum to 1. We thus estimate the probability that word c is not a real context word for w as:

$$\begin{aligned} P(-|w,c) &= 1 - P(+|w,c) \\ &= \sigma(-\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1+\exp(\mathbf{c} \cdot \mathbf{w})} \end{aligned} \tag{5.16}$$

Equation 5.15 gives us the probability for one word, but there are many context words in the window. Skip-gram makes the simplifying assumption that all context words are independent, allowing us to just multiply their probabilities:

$$P(+|w,c_{1:L}) = \prod_{i=1}^{L} \sigma(\mathbf{c_i} \cdot \mathbf{w}) \tag{5.17}$$

$$\log P(+|w,c_{1:L}) = \sum_{i=1}^{L} \log \sigma(\mathbf{c_i} \cdot \mathbf{w}) \tag{5.18}$$

In summary, skip-gram trains a probabilistic classifier that, given a test target word w and its context window of L words $c_{1:L}$, assigns a probability based on how similar this context window is to the target word. The probability is based on applying the logistic (sigmoid) function to the dot product of the embeddings of the target word with each context word. To compute this probability, we just need embeddings for each target word and context word in the vocabulary.

Fig. 5.6 shows the intuition of the parameters we'll need. Skip-gram actually stores two embeddings for each word, one for the word as a target, and one for the word considered as context. Thus the parameters we need to learn are two matrices $\mathbf{W}$ and $\mathbf{C}$, each containing an embedding for every one of the $|V|$ words in the vocabulary V.[2] Let's now turn to learning these embeddings (which is the real goal of training this classifier in the first place).

[2] In principle the target matrix and the context matrix could use different vocabularies, but we'll simplify by assuming one shared vocabulary V.

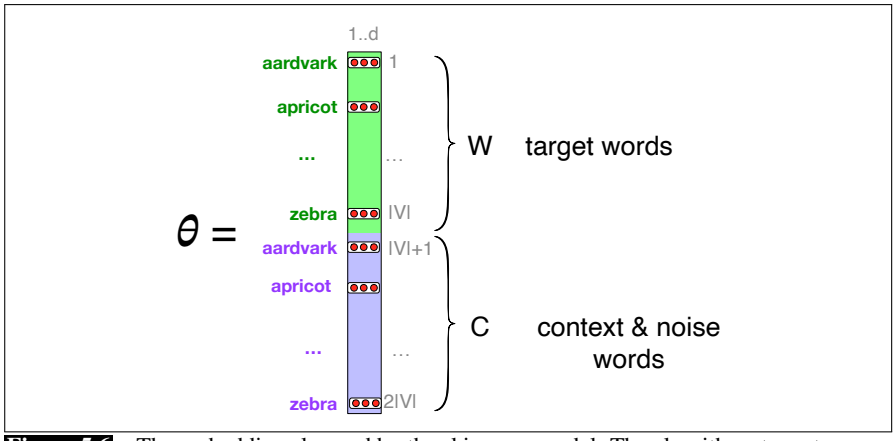


Figure 5.6 The embeddings learned by the skipgram model. The algorithm stores two embeddings for each word, the target embedding (sometimes called the input embedding) and the context embedding (sometimes called the output embedding). The parameter θ that the algorithm learns is thus a matrix of $2|V|$ vectors, each of dimension d, formed by concatenating two matrices, the target embeddings **W** and the context+noise embeddings **C**.

5.5.2 Learning skip-gram embeddings

The learning algorithm for skip-gram embeddings takes as input a corpus of text, and a chosen vocabulary size N. It begins by assigning a random embedding vector for each of the N vocabulary words, and then proceeds to iteratively shift the embedding of each word w to be more like the embeddings of words that occur nearby in texts, and less like the embeddings of words that don't occur nearby. Let's start by considering a single piece of training data:

```
... lemon,  a [tablespoon of apricot jam,      a] pinch ...
              c1         c2    w     c3       c4
```

This example has a target word w (apricot), and 4 context words in the $L = \pm 2$ window, resulting in 4 positive training instances (on the left below):

positive examples +

w	c_{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -

w	c_{neg}	w	c_{neg}
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

For training a binary classifier we also need negative examples. In fact skip-gram with negative sampling (SGNS) uses more negative examples than positive examples (with the ratio between them set by a parameter k). So for each of these (w, c_{pos}) training instances we'll create k negative samples, each consisting of the target w plus a 'noise word' c_{neg}. A noise word is a random word from the lexicon, constrained not to be the target word w. The table right above shows the setting where $k = 2$, so we'll have 2 negative examples in the negative training set $-$ for each positive example w, c_{pos}.

The noise words are chosen according to their weighted unigram probability $p_\alpha(w)$, where α is a weight. If we were sampling according to unweighted probability $P(w)$, it would mean that with unigram probability $P(\text{"}the\text{"})$ we would choose the word *the* as a noise word, with unigram probability $P(\text{"}aardvark\text{"})$ we would

choose *aardvark*, and so on. But in practice it is common to set $\alpha = 0.75$, i.e. use the weighting $P_{\frac{3}{4}}(w)$:

$$P_\alpha(w) = \frac{count(w)^\alpha}{\sum_{w'} count(w')^\alpha} \tag{5.19}$$

Setting $\alpha = .75$ gives better performance because it gives rare noise words slightly higher probability: for rare words, $P_\alpha(w) > P(w)$. To illustrate this intuition, it might help to work out the probabilities for an example with $\alpha = .75$ and two events, $P(a) = 0.99$ and $P(b) = 0.01$:

$$\begin{aligned} P_\alpha(a) &= \frac{.99^{.75}}{.99^{.75} + .01^{.75}} = 0.97 \\ P_\alpha(b) &= \frac{.01^{.75}}{.99^{.75} + .01^{.75}} = 0.03 \end{aligned} \tag{5.20}$$

Thus using $\alpha = .75$ increases the probability of the rare event b from 0.01 to 0.03.

Given the set of positive and negative training instances, and an initial set of embeddings, the goal of the learning algorithm is to adjust those embeddings to

- Maximize the similarity of the target word, context word pairs (w, c_{pos}) drawn from the positive examples
- Minimize the similarity of the (w, c_{neg}) pairs from the negative examples.

If we consider one word/context pair (w, c_{pos}) with its k noise words $c_{neg_1} ... c_{neg_k}$, we can express these two goals as the following loss function L to be minimized (hence the $-$); here the first term expresses that we want the classifier to assign the real context word c_{pos} a high probability of being a neighbor, and the second term expresses that we want to assign each of the noise words c_{neg_i} a high probability of being a non-neighbor, all multiplied because we assume independence:

$$\begin{aligned} L(w, c_{pos}, c_{neg*}) &= -\log \left[P(+|w, c_{pos}) \prod_{i=1}^{k} P(-|w, c_{neg_i}) \right] \\ &= -\left[\log P(+|w, c_{pos}) + \sum_{i=1}^{k} \log P(-|w, c_{neg_i}) \right] \\ &= -\left[\log P(+|w, c_{pos}) + \sum_{i=1}^{k} \log \left(1 - P(+|w, c_{neg_i})\right) \right] \\ &= -\left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^{k} \log \sigma(-c_{neg_i} \cdot w) \right] \end{aligned} \tag{5.21}$$

That is, we want to maximize the dot product of the word with the actual context words, and minimize the dot products of the word with the k negative sampled non-neighbor words.

We minimize this loss function using stochastic gradient descent. Fig. 5.7 shows the intuition of one step of learning.

To get the gradient, we need to take the derivative of Eq. 5.21 with respect to the different embeddings. It turns out the derivatives are the following (we leave the

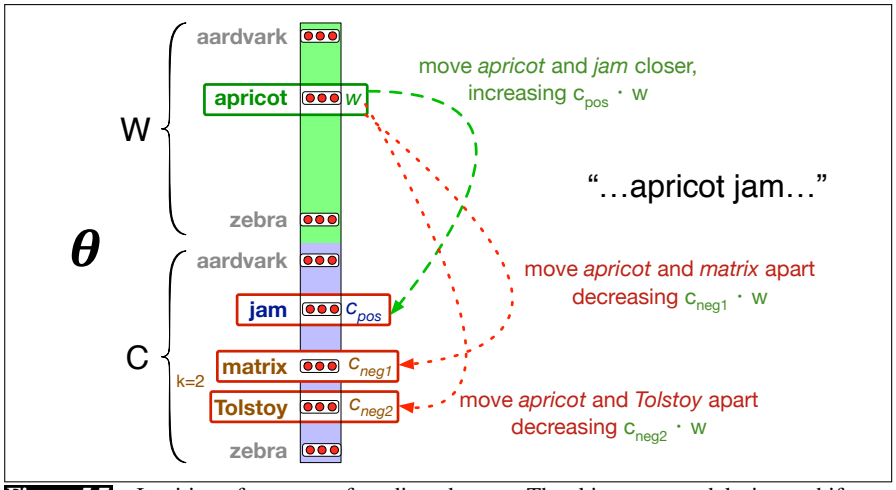


Figure 5.7 Intuition of one step of gradient descent. The skip-gram model tries to shift embeddings so the target embeddings (here for *apricot*) are closer to (have a higher dot product with) context embeddings for nearby words (here *jam*) and further from (lower dot product with) context embeddings for noise words that don't occur nearby (here *Tolstoy* and *matrix*).

proof as an exercise at the end of the chapter):

$$\frac{\partial L}{\partial c_{pos}} = [\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) - 1]\mathbf{w} \tag{5.22}$$

$$\frac{\partial L}{\partial c_{neg_i}} = [\sigma(\mathbf{c}_{neg_i} \cdot \mathbf{w})]\mathbf{w} \tag{5.23}$$

$$\frac{\partial L}{\partial w} = [\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) - 1]\mathbf{c}_{pos} + \sum_{i=1}^{k}[\sigma(\mathbf{c}_{neg_i} \cdot \mathbf{w})]\mathbf{c}_{neg_i} \tag{5.24}$$

The update equations going from time step t to $t+1$ in stochastic gradient descent are thus:

$$\mathbf{c}_{pos}^{t+1} = \mathbf{c}_{pos}^{t} - \eta[\sigma(\mathbf{c}_{pos}^{t} \cdot \mathbf{w}^{t}) - 1]\mathbf{w}^{t} \tag{5.25}$$

$$\mathbf{c}_{neg_i}^{t+1} = \mathbf{c}_{neg_i}^{t} - \eta[\sigma(\mathbf{c}_{neg_i}^{t} \cdot \mathbf{w}^{t})]\mathbf{w}^{t} \tag{5.26}$$

$$\mathbf{w}^{t+1} = \mathbf{w}^{t} - \eta\left[[\sigma(\mathbf{c}_{pos}^{t} \cdot \mathbf{w}^{t}) - 1]\mathbf{c}_{pos}^{t} + \sum_{i=1}^{k}[\sigma(\mathbf{c}_{neg_i}^{t} \cdot \mathbf{w}^{t})]\mathbf{c}_{neg_i}^{t}\right] \tag{5.27}$$

Just as in logistic regression, then, the learning algorithm starts with randomly initialized **W** and **C** matrices, and then walks through the training corpus using gradient descent to move **W** and **C** so as to minimize the loss in Eq. 5.21 by making the updates in (Eq. 5.25)-(Eq. 5.27).

target embedding
context embedding

Recall that the skip-gram model learns **two** separate embeddings for each word i: the **target embedding** $\mathbf{w}_i$ and the **context embedding** $\mathbf{c}_i$, stored in two matrices, the **target matrix W** and the **context matrix C**. It's common to just add them together, representing word i with the vector $\mathbf{w}_i + \mathbf{c}_i$. Alternatively we can throw away the **C** matrix and just represent each word i by the vector $\mathbf{w}_i$.

As with the simple count-based methods like tf-idf, the context window size affects the performance of skip-gram embeddings, and experiments often tune the context window size parameter on a devset.

5.5.3 Other kinds of static embeddings

fasttext

There are many kinds of static embeddings. An extension of word2vec, **fasttext** (Bojanowski et al., 2017), addresses a problem with word2vec as we have presented it so far: it has no good way to deal with **unknown words**—words that appear in a test corpus but were unseen in the training corpus. A related problem is word sparsity, such as in languages with rich morphology, where some of the many forms for each noun and verb may only occur rarely. Fasttext deals with these problems by using subword models, representing each word as itself plus a bag of constituent n-grams, with special boundary symbols < and > added to each word. For example, with $n = 3$ the word *where* would be represented by the sequence <where> plus the character n-grams:

<wh, whe, her, ere, re>

Then a skipgram embedding is learned for each constituent n-gram, and the word *where* is represented by the sum of all of the embeddings of its constituent n-grams. Unknown words can then be presented only by the sum of the constituent n-grams. A fasttext open-source library, including pretrained embeddings for 157 languages, is available at https://fasttext.cc.

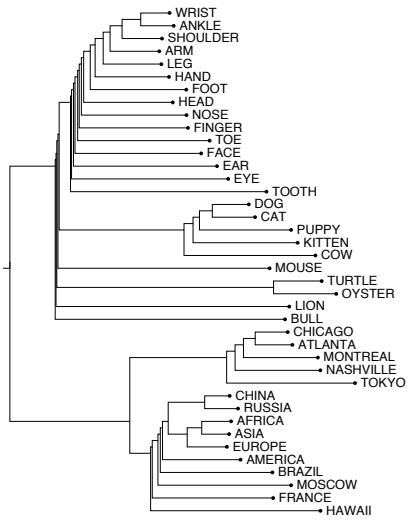
Another very widely used static embedding model is GloVe (Pennington et al., 2014), short for Global Vectors, because the model is based on capturing global corpus statistics. GloVe is based on ratios of probabilities from the word-word co-occurrence matrix.

It turns out that dense embeddings like word2vec actually have an elegant mathematical relationship with count-based embeddings, in which word2vec can be seen as implicitly optimizing a function of a count matrix with a particular (PPMI) weighting (Levy and Goldberg, 2014c).

5.6 Visualizing Embeddings

> "I see well in many dimensions as long as the dimensions are around two."
>
> The late economist Martin Shubik

Visualizing embeddings is an important goal in helping understand, apply, and improve these models of word meaning. But how can we visualize a (for example) 100-dimensional vector?



The simplest way to visualize the meaning of a word w embedded in a space is to list the most similar words to w by sorting the vectors for all words in the vocabulary by their cosine with the vector for w. For example the 7 closest words to *frog* using a particular set of embeddings computed with the GloVe algorithm are: *frogs*, *toad*, *litoria*, *leptodactylidae*, *rana*, *lizard*, and *eleutherodactylus* (Pennington et al., 2014).

Yet another visualization method is to use a clustering algorithm to show a hierarchical representation of which words are similar to others in the embedding space. The uncaptioned figure on the left uses hierarchical clustering of some embedding vectors for nouns as a visualization method (Rohde et al., 2006).

Probably the most common visualization method, however, is to project the 100 dimensions of a word down into 2 dimensions. Fig. 5.1 showed one such visualization, as does Fig. 5.9, using a projection method called t-SNE (van der Maaten and Hinton, 2008).

5.7 Semantic properties of embeddings

In this section we briefly summarize some of the semantic properties of embeddings that have been studied.

Different types of similarity or association: One parameter of vector semantic models that is relevant to both sparse PPMI vectors and dense word2vec vectors is the size of the context window used to collect counts. This is generally between 1 and 10 words on each side of the target word (for a total context of 2-20 words).

The choice depends on the goals of the representation. Shorter context windows tend to lead to representations that are a bit more syntactic, since the information is coming from immediately nearby words. When the vectors are computed from short context windows, the most similar words to a target word w tend to be semantically similar words with the same parts of speech. When vectors are computed from long context windows, the highest cosine words to a target word w tend to be words that are topically related but not similar.

For example Levy and Goldberg (2014a) showed that using skip-gram with a window of ± 2, the most similar words to the word *Hogwarts* (from the *Harry Potter* series) were names of other fictional schools: *Sunnydale* (from *Buffy the Vampire Slayer*) or *Evernight* (from a vampire series). With a window of ± 5, the most similar words to *Hogwarts* were other words topically related to the *Harry Potter* series: *Dumbledore*, *Malfoy*, and *half-blood*.

first-order co-occurrence

second-order co-occurrence

It's also often useful to distinguish two kinds of similarity or association between words (Schütze and Pedersen, 1993). Two words have **first-order co-occurrence** (sometimes called **syntagmatic association**) if they are typically nearby each other. Thus *wrote* is a first-order associate of *book* or *poem*. Two words have **second-order co-occurrence** (sometimes called **paradigmatic association**) if they have similar neighbors. Thus *wrote* is a second-order associate of words like *said* or *remarked*.

Analogy/Relational Similarity: Another semantic property of embeddings is their ability to capture relational meanings. In an important early vector space model of cognition, Rumelhart and Abrahamson (1973) proposed the **parallelogram model** for solving simple analogy problems of the form *a is to b as a* is to what?*. In such problems, a system is given a problem like *apple:tree::grape:?*, i.e., *apple is to tree as grape is to* _____, and must fill in the word *vine*. In the parallelogram model, illustrated in Fig. 5.8, the vector from the word *apple* to the word *tree* ($= \overrightarrow{\text{tree}} - \overrightarrow{\text{apple}}$) is added to the vector for *grape* ($\overrightarrow{\text{grape}}$); the nearest word to that point is returned.

parallelogram model

In early work with sparse embeddings, scholars showed that sparse vector models of meaning could solve such analogy problems (Turney and Littman, 2005), but the parallelogram method received more modern attention because of its success with word2vec or GloVe vectors (Mikolov et al. 2013c, Levy and Goldberg 2014b, Pennington et al. 2014). For example, the result of the expression $\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}}$ is a vector close to $\overrightarrow{\text{queen}}$. Similarly, $\overrightarrow{\text{Paris}} - \overrightarrow{\text{France}} + \overrightarrow{\text{Italy}}$ results in a vector that is close to $\overrightarrow{\text{Rome}}$. The embedding model thus seems to be extract-

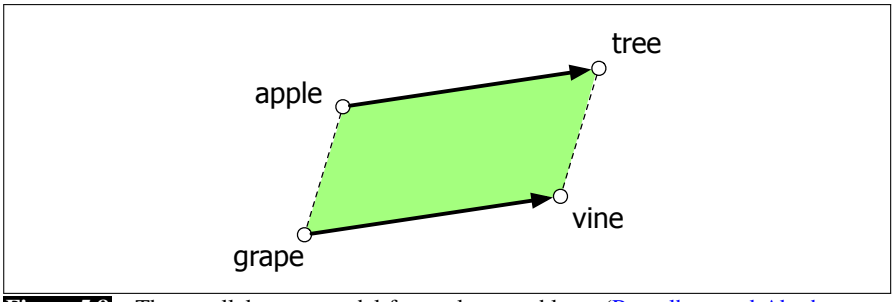


Figure 5.8 The parallelogram model for analogy problems (Rumelhart and Abrahamson, 1973): the location of $\overrightarrow{\text{vine}}$ can be found by subtracting $\overrightarrow{\text{apple}}$ from $\overrightarrow{\text{tree}}$ and adding $\overrightarrow{\text{grape}}$.

ing representations of relations like MALE-FEMALE, or CAPITAL-CITY-OF, or even COMPARATIVE/SUPERLATIVE, as shown in Fig. 5.9 from GloVe.

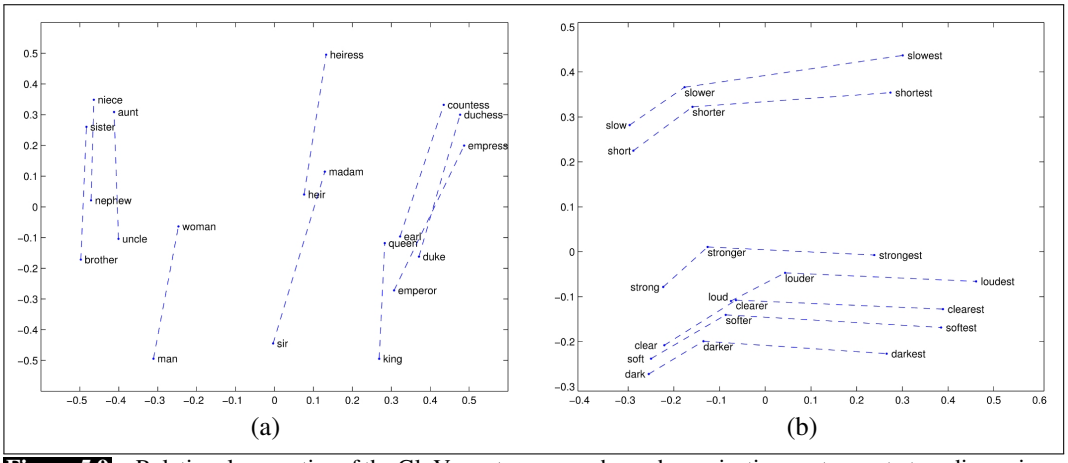


Figure 5.9 Relational properties of the GloVe vector space, shown by projecting vectors onto two dimensions. (a) $\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}}$ is close to $\overrightarrow{\text{queen}}$. (b) offsets seem to capture comparative and superlative morphology (Pennington et al., 2014).

For a $\mathbf{a} : \mathbf{b} :: \mathbf{a}^* : \mathbf{b}^*$ problem, meaning the algorithm is given vectors $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{a}^*$ and must find $\mathbf{b}^*$, the parallelogram method is thus:

$$\hat{\mathbf{b}}^* = \underset{\mathbf{x}}{\text{argmin}}\ \text{distance}(\mathbf{x}, \mathbf{b} - \mathbf{a} + \mathbf{a}^*) \tag{5.28}$$

with some distance function, such as Euclidean distance.

There are some caveats. For example, the closest value returned by the parallelogram algorithm in word2vec or GloVe embedding spaces is usually not in fact b* but one of the 3 input words or their morphological variants (i.e., *cherry:red :: potato:x* returns *potato* or *potatoes* instead of *brown*), so these must be explicitly excluded. Furthermore while embedding spaces perform well if the task involves frequent words, small distances, and certain relations (like relating countries with their capitals or verbs/nouns with their inflected forms), the parallelogram method with embeddings doesn't work as well for other relations (Linzen 2016, Gladkova et al. 2016, Schluter 2018, Ethayarajh et al. 2019a), and indeed Peterson et al. (2020) argue that the parallelogram method is in general too simple to model the human cognitive process of forming analogies of this kind.

5.7.1 Embeddings and Historical Semantics

Embeddings can also be a useful tool for studying how meaning changes over time, by computing multiple embedding spaces, each from texts written in a particular time period. For example Fig. 5.10 shows a visualization of changes in meaning in English words over the last two centuries, computed by building separate embedding spaces for each decade from historical corpora like Google n-grams (Lin et al., 2012b) and the Corpus of Historical American English (Davies, 2012).

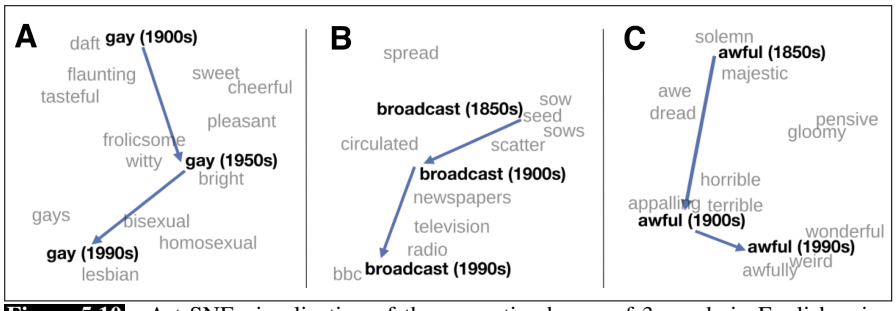


Figure 5.10 A t-SNE visualization of the semantic change of 3 words in English using word2vec vectors. The modern sense of each word, and the grey context words, are computed from the most recent (modern) time-point embedding space. Earlier points are computed from earlier historical embedding spaces. The visualizations show the changes in the word *gay* from meanings related to "cheerful" or "frolicsome" to referring to homosexuality, the development of the modern "transmission" sense of *broadcast* from its original sense of sowing seeds, and the pejoration of the word *awful* as it shifted from meaning "full of awe" to meaning "terrible or appalling" (Hamilton et al., 2016b).

5.8 Bias and Embeddings

In addition to their ability to learn word meaning from text, embeddings, alas, also reproduce the implicit biases and stereotypes that were latent in the text. As the prior section just showed, embeddings can roughly model relational similarity: 'queen' as the closest word to 'king' - 'man' + 'woman' implies the analogy *man:woman::king:queen*. But these same embedding analogies also exhibit gender stereotypes. For example Bolukbasi et al. (2016) find that the closest occupation to 'computer programmer' - 'man' + 'woman' in word2vec embeddings trained on news text is 'homemaker', and that the embeddings similarly suggest the analogy 'father' is to 'doctor' as 'mother' is to 'nurse'. This could result in what Crawford (2017) and Blodgett et al. (2020) call an **allocational harm**, when a system allocates resources (jobs or credit) unfairly to different groups. For example algorithms that use embeddings as part of a search for hiring potential programmers or doctors might thus incorrectly downweight documents with women's names.

allocational harm

It turns out that embeddings don't just reflect the statistics of their input, but also **amplify** bias; gendered terms become **more** gendered in embedding space than they were in the input text statistics (Zhao et al. 2017, Ethayarajh et al. 2019b, Jia et al. 2020), and biases are more exaggerated than in actual labor employment statistics (Garg et al., 2018).

bias amplification

Embeddings also encode the implicit associations that are a property of human reasoning. The Implicit Association Test (Greenwald et al., 1998) measures peo-

ple's associations between concepts (like 'flowers' or 'insects') and attributes (like 'pleasantness' and 'unpleasantness') by measuring differences in the latency with which they label words in the various categories.[3] Using such methods, people in the United States have been shown to associate African-American names with unpleasant words (more than European-American names), male names more with mathematics and female names with the arts, and old people's names with unpleasant words (Greenwald et al. 1998, Nosek et al. 2002a, Nosek et al. 2002b). Caliskan et al. (2017) replicated all these findings of implicit associations using GloVe vectors and cosine similarity instead of human latencies. For example African-American names like 'Leroy' and 'Shaniqua' had a higher GloVe cosine with unpleasant words while European-American names ('Brad', 'Greg', 'Courtney') had a higher cosine with pleasant words. These problems with embeddings are an example of a **representational harm** (Crawford 2017, Blodgett et al. 2020), which is a harm caused by a system demeaning or even ignoring some social groups. Any embedding-aware algorithm that made use of word sentiment could thus exacerbate bias against African Americans.

representational harm

Recent research focuses on ways to try to remove these kinds of biases, for example by developing a transformation of the embedding space that removes gender stereotypes but preserves definitional gender (Bolukbasi et al. 2016, Zhao et al. 2017) or changing the training procedure (Zhao et al., 2018b). However, although these sorts of **debiasing** may reduce bias in embeddings, they do not eliminate it (Gonen and Goldberg, 2019), and this remains an open problem.

debiasing

Historical embeddings are also being used to measure biases in the past. Garg et al. (2018) used embeddings from historical texts to measure the association between embeddings for occupations and embeddings for names of various ethnicities or genders (for example the relative cosine similarity of women's names versus men's to occupation words like 'librarian' or 'carpenter') across the 20th century. They found that the cosines correlate with the empirical historical percentages of women or ethnic groups in those occupations. Historical embeddings also replicated old surveys of ethnic stereotypes; the tendency of experimental participants in 1933 to associate adjectives like 'industrious' or 'superstitious' with, e.g., Chinese ethnicity, correlates with the cosine between Chinese last names and those adjectives using embeddings trained on 1930s text. They also were able to document historical gender biases, such as the fact that embeddings for adjectives related to competence ('smart', 'wise', 'thoughtful', 'resourceful') had a higher cosine with male than female words, and showed that this bias has been slowly decreasing since 1960. We return in later chapters to this question about the role of bias in natural language processing.

5.9 Evaluating Vector Models

The most important evaluation metric for vector models is extrinsic evaluation on tasks, i.e., using vectors in an NLP task and seeing whether this improves performance over some other model.

[3] Roughly speaking, if humans associate 'flowers' with 'pleasantness' and 'insects' with 'unpleasantness', when they are instructed to push a green button for 'flowers' (daisy, iris, lilac) and 'pleasant words' (love, laughter, pleasure) and a red button for 'insects' (flea, spider, mosquito) and 'unpleasant words' (abuse, hatred, ugly) they are faster than in an incongruous condition where they push a red button for 'flowers' and 'unpleasant words' and a green button for 'insects' and 'pleasant words'.

Nonetheless it is useful to have intrinsic evaluations. The most common metric is to test their performance on **similarity**, computing the correlation between an algorithm's word similarity scores and word similarity ratings assigned by humans. **WordSim-353** (Finkelstein et al., 2002) is a commonly used set of ratings from 0 to 10 for 353 noun pairs; for example (*plane*, *car*) had an average score of 5.77. **SimLex-999** (Hill et al., 2015) is a more complex dataset that quantifies similarity (*cup, mug*) rather than relatedness (*cup, coffee*), and includes concrete and abstract adjective, noun and verb pairs. The **TOEFL dataset** is a set of 80 questions, each consisting of a target word with 4 additional word choices; the task is to choose which is the correct synonym, as in the example: *Levied is closest in meaning to: imposed, believed, requested, correlated* (Landauer and Dumais, 1997). All of these datasets present words without context.

Slightly more realistic are intrinsic similarity tasks that include context. The Stanford Contextual Word Similarity (SCWS) dataset (Huang et al., 2012) and the Word-in-Context (WiC) dataset (Pilehvar and Camacho-Collados, 2019) offer richer evaluation scenarios. SCWS gives human judgments on 2,003 pairs of words in their sentential context, while WiC gives target words in two sentential contexts that are either in the same or different senses; see Appendix G. The *semantic textual similarity* task (Agirre et al. 2012, Agirre et al. 2015) evaluates the performance of sentence-level similarity algorithms, consisting of a set of pairs of sentences, each pair with human-labeled similarity scores.

Another task used for evaluation is the analogy task, discussed on page 112, where the system has to solve problems of the form *a is to b as a* is to b**, given *a, b,* and *a** and having to find *b** (Turney and Littman, 2005). A number of sets of tuples have been created for this task (Mikolov et al. 2013a, Mikolov et al. 2013c, Gladkova et al. 2016), covering morphology (*city:cities::child:children*), lexicographic relations (*leg:table::spout:teapot*) and encyclopedia relations (*Beijing:China::Dublin:Ireland*), some drawing from the SemEval-2012 Task 2 dataset of 79 different relations (Jurgens et al., 2012).

All embedding algorithms suffer from inherent variability. For example because of randomness in the initialization and the random negative sampling, algorithms like word2vec may produce different results even from the same dataset, and individual documents in a collection may strongly impact the resulting embeddings (Tian et al. 2016, Hellrich and Hahn 2016, Antoniak and Mimno 2018). When embeddings are used to study word associations in particular corpora, therefore, it is best practice to train multiple embeddings with bootstrap sampling over documents and average the results (Antoniak and Mimno, 2018).

5.10 Summary

- In vector semantics, a word is modeled as a vector—a point in high-dimensional space, also called an **embedding**. In this chapter we focus on **static embeddings**, where each word is mapped to a fixed embedding.
- Vector semantic models fall into two classes: **sparse** and **dense**. In sparse models each dimension corresponds to a word in the vocabulary V and cells are functions of **co-occurrence counts**. The **word-context** or **term-term** matrix has a row for each (target) word in the vocabulary and a column for each context term in the vocabulary.

- Dense vector models typically have dimensionality 50–1000. **Word2vec** algorithms like **skip-gram** are a popular way to compute dense embeddings. Skip-gram trains a logistic regression classifier to compute the probability that two words are 'likely to occur nearby in text'. This probability is computed from the dot product between the embeddings for the two words.
- Skip-gram uses stochastic gradient descent to train the classifier, by learning embeddings that have a high dot product with embeddings of words that occur nearby and a low dot product with noise words.
- Other important embedding algorithms include **GloVe**, a method based on ratios of word co-occurrence probabilities.
- Whether using sparse or dense vectors, word and document similarities are computed by some function of the **dot product** between vectors. The cosine of two vectors—a normalized dot product—is the most popular such metric.

Historical Notes

The idea of vector semantics arose out of research in the 1950s in three distinct fields: linguistics, psychology, and computer science, each of which contributed a fundamental aspect of the model.

The idea that meaning is related to the distribution of words in context was widespread in linguistic theory of the 1950s, among distributionalists like Zellig Harris, Martin Joos, and J. R. Firth, and semioticians like Thomas Sebeok. As Joos (1950) put it,

> the linguist's "meaning" of a morpheme... is by definition the set of conditional probabilities of its occurrence in context with all other morphemes.

The idea that the meaning of a word might be modeled as a point in a multidimensional semantic space came from psychologists like Charles E. Osgood, who had been studying how people responded to the meaning of words by assigning values along scales like *happy/sad* or *hard/soft*. Osgood et al. (1957) proposed that the meaning of a word in general could be modeled as a point in a multidimensional Euclidean space, and that the similarity of meaning between two words could be modeled as the distance between these points in the space.

mechanical indexing

A final intellectual source in the 1950s and early 1960s was the field then called **mechanical indexing**, now known as **information retrieval**. In what became known as the **vector space model** for information retrieval (Salton 1971, Sparck Jones 1986), researchers demonstrated new ways to define the meaning of words in terms of vectors (Switzer, 1965), and refined methods for word similarity based on measures of statistical association between words like mutual information (Giuliano, 1965) and idf (Sparck Jones, 1972), and showed that the meaning of documents could be represented in the same vector spaces used for words. Around the same time, (Cordier, 1965) showed that factor analysis of word association probabilities could be used to form dense vector representations of words.

Some of the philosophical underpinning of the distributional way of thinking came from the late writings of the philosopher Wittgenstein, who was skeptical of the possibility of building a completely formal theory of meaning definitions for each word. Wittgenstein suggested instead that "the meaning of a word is its use in the language" (Wittgenstein, 1953, PI 43). That is, instead of using some logical language to define each word, or drawing on denotations or truth values, Wittgenstein's

idea is that we should define a word by how it is used by people in speaking and understanding in their day-to-day interactions, thus prefiguring the movement toward embodied and experiential models in linguistics and NLP (Glenberg and Robertson 2000, Lake and Murphy 2021, Bisk et al. 2020, Bender and Koller 2020).

More distantly related is the idea of defining words by a vector of discrete features, which has roots at least as far back as Descartes and Leibniz (Wierzbicka 1992, Wierzbicka 1996). By the middle of the 20th century, beginning with the work of Hjelmslev (Hjelmslev, 1969) (originally 1943) and fleshed out in early models of generative grammar (Katz and Fodor, 1963), the idea arose of representing meaning with **semantic features**, symbols that represent some sort of primitive meaning. For example words like *hen*, *rooster*, or *chick*, have something in common (they all describe chickens) and something different (their age and sex), representable as:

semantic feature

hen	`+female, +chicken, +adult`
rooster	`-female, +chicken, +adult`
chick	`+chicken, -adult`

The dimensions used by vector models of meaning to define words, however, are only abstractly related to this idea of a small fixed number of hand-built dimensions. Nonetheless, there has been some attempt to show that certain dimensions of embedding models do contribute some specific compositional aspect of meaning like these early semantic features.

The use of dense vectors to model word meaning, and indeed the term **embedding**, grew out of the **latent semantic indexing** (LSI) model (Deerwester et al., 1988) recast as **LSA** (**latent semantic analysis**) (Deerwester et al., 1990). In LSA **singular value decomposition**—**SVD**— is applied to a term-document matrix (each cell weighted by log frequency and normalized by entropy), and then the first 300 dimensions are used as the LSA embedding. Singular Value Decomposition (SVD) is a method for finding the most important dimensions of a dataset, those dimensions along which the data varies the most. LSA was then quickly widely applied: as a cognitive model (Landauer and Dumais, 1997), and for tasks like spell checking (Jones and Martin, 1997), language modeling (Bellegarda 1997, Coccaro and Jurafsky 1998, Bellegarda 2000), morphology induction (Schone and Jurafsky 2000, Schone and Jurafsky 2001b), multiword expressions (MWEs) (Schone and Jurafsky, 2001a), and essay grading (Rehder et al., 1998). Related models were simultaneously developed and applied to word sense disambiguation by Schütze (1992b). LSA also led to the earliest use of embeddings to represent words in a probabilistic classifier, in the logistic regression document router of Schütze et al. (1995). The idea of SVD on the term-term matrix (rather than the term-document matrix) as a model of meaning for NLP was proposed soon after LSA by Schütze (1992b). Schütze applied the low-rank (97-dimensional) embeddings produced by SVD to the task of word sense disambiguation, analyzed the resulting semantic space, and also suggested possible techniques like dropping high-order dimensions. See Schütze (1997).

SVD

A number of alternative matrix models followed on from the early SVD work, including Probabilistic Latent Semantic Indexing (PLSI) (Hofmann, 1999), Latent Dirichlet Allocation (LDA) (Blei et al., 2003), and Non-negative Matrix Factorization (NMF) (Lee and Seung, 1999).

The LSA community seems to have first used the word "embedding" in Landauer et al. (1997), in a variant of its mathematical meaning as a mapping from one space or mathematical structure to another. In LSA, the word embedding seems to have described the mapping from the space of sparse count vectors to the latent space of

SVD dense vectors. Although the word thus originally meant the mapping from one space to another, it has metonymically shifted to mean the resulting dense vector in the latent space, and it is in this sense that we currently use the word.

By the next decade, Bengio et al. (2003) and Bengio et al. (2006) showed that neural language models could also be used to develop embeddings as part of the task of word prediction. Collobert and Weston (2007), Collobert and Weston (2008), and Collobert et al. (2011) then demonstrated that embeddings could be used to represent word meanings for a number of NLP tasks. Turian et al. (2010) compared the value of different kinds of embeddings for different NLP tasks. Mikolov et al. (2011) showed that recurrent neural nets could be used as language models. The idea of simplifying the hidden layer of these neural net language models to create the skip-gram (and also CBOW) algorithms was proposed by Mikolov et al. (2013a). The negative sampling training algorithm was proposed in Mikolov et al. (2013b). There are numerous surveys of static embeddings and their parameterizations (Bullinaria and Levy 2007, Bullinaria and Levy 2012, Lapesa and Evert 2014, Kiela and Clark 2014, Levy et al. 2015).

See Manning et al. (2008) and Chapter 11 for a deeper understanding of the role of vectors in information retrieval, including how to compare queries with documents, more details on tf-idf, and issues of scaling to very large datasets. See Kim (2019) for a clear and comprehensive tutorial on word2vec. Cruse (2004) is a useful introductory linguistic text on lexical semantics.

Exercises

CHAPTER

6 Neural Networks

> "[M]achines of this character can behave in a very complicated manner when the number of units is large."
>
> Alan Turing (1948) "Intelligent Machines", page 6

Neural networks are a fundamental computational tool for language processing, and a very old one. They are called neural because their origins lie in the **McCulloch-Pitts neuron** (McCulloch and Pitts, 1943), a simplified model of the biological neuron as a kind of computing element that could be described in terms of propositional logic. But the modern use in language processing no longer draws on these early biological inspirations.

Instead, a modern neural network is a network of small computing units, each of which takes a vector of input values and produces a single output value. In this chapter we introduce the neural net applied to classification. The architecture we introduce is called a **feedforward network** because the computation proceeds iteratively from one layer of units to the next. The use of modern neural nets is often called **deep learning**, because modern networks are often **deep** (have many layers).

feedforward

deep learning

Neural networks share much of the same mathematics as logistic regression. But neural networks are a more powerful classifier than logistic regression, and indeed a minimal neural network (technically one with a single 'hidden layer') can be shown to learn any function.

Neural net classifiers are different from logistic regression in another way. With logistic regression, we applied the regression classifier to many different tasks by developing many rich kinds of feature templates based on domain knowledge. When working with neural networks, it is more common to avoid most uses of rich hand-derived features, instead building neural networks that take raw tokens as inputs and learn to induce features as part of the process of learning to classify. We saw examples of this kind of representation learning for embeddings in Chapter 5, and we'll see lots of examples once we start studying deep transformers networks. Nets that are very deep are particularly good at representation learning. For that reason deep neural nets are the right tool for tasks that offer sufficient data to learn features automatically.

In this chapter we'll introduce feedforward networks as classifiers, first with hand-built features, and then using the embeddings that we studied in Chapter 5. In subsequent chapters we'll introduce many other kinds of neural models, most importantly the **transformer** and **attention**, (Chapter 8), but also **recurrent neural networks** (Chapter 13) and **convolutional neural networks** (Chapter 15). And in the next chapter we'll introduce the paradigm of neural large language models.

6.1 Units

The building block of a neural network is a single computational unit. A unit takes a set of real valued numbers as input, performs some computation on them, and produces an output.

At its heart, a neural unit is taking a weighted sum of its inputs, with one additional term in the sum called a **bias term**. Given a set of inputs $x_1 ... x_n$, a unit has a set of corresponding weights $w_1 ... w_n$ and a bias b, so the weighted sum z can be represented as:

bias term

$$z = b + \sum_i w_i x_i \tag{6.1}$$

Often it's more convenient to express this weighted sum using vector notation; recall from linear algebra that a **vector** is, at heart, just a list or array of numbers. Thus we'll talk about z in terms of a weight vector w, a scalar bias b, and an input vector x, and we'll replace the sum with the convenient **dot product**:

vector

$$z = \mathbf{w} \cdot \mathbf{x} + b \tag{6.2}$$

As defined in Eq. 6.2, z is just a real valued number.

Finally, instead of using z, a linear function of x, as the output, neural units apply a non-linear function f to z. We will refer to the output of this function as the **activation** value for the unit, a. Since we are just modeling a single unit, the activation for the node is in fact the final output of the network, which we'll generally call y. So the value y is defined as:

activation

$$y = a = f(z)$$

We'll discuss three popular non-linear functions f below (the sigmoid, the tanh, and the rectified linear unit or ReLU) but it's pedagogically convenient to start with the **sigmoid** function since we saw it in Chapter 4:

sigmoid

$$y = \sigma(z) = \frac{1}{1+e^{-z}} \tag{6.3}$$

The sigmoid (shown in Fig. 6.1) has a number of advantages; it maps the output into the range $(0,1)$, which is useful in squashing outliers toward 0 or 1. And it's differentiable, which as we saw in Section 4.15 will be handy for learning.

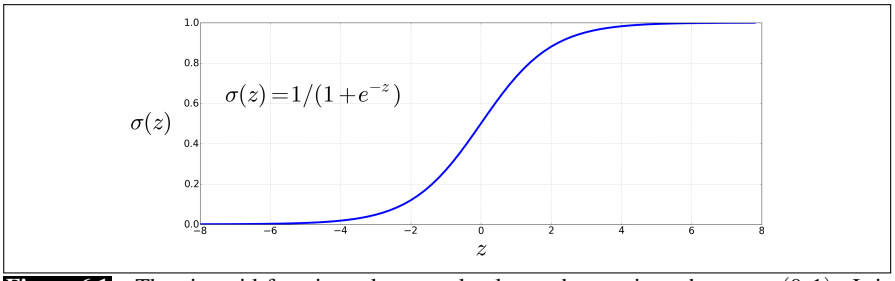


Figure 6.1 The sigmoid function takes a real value and maps it to the range $(0,1)$. It is nearly linear around 0 but outlier values get squashed toward 0 or 1.

Substituting Eq. 6.2 into Eq. 6.3 gives us the output of a neural unit:

$$y = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1+\exp(-(\mathbf{w} \cdot \mathbf{x} + b))} \tag{6.4}$$

Fig. 6.2 shows a final schematic of a basic neural unit. In this example the unit takes 3 input values x_1, x_2, and x_3, and computes a weighted sum, multiplying each value by a weight (w_1, w_2, and w_3, respectively), adds them to a bias term b, and then passes the resulting sum through a sigmoid function to result in a number between 0 and 1.

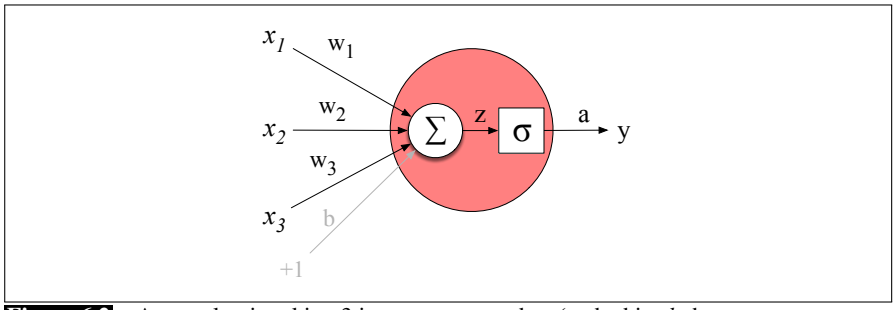


Figure 6.2 A neural unit, taking 3 inputs x_1, x_2, and x_3 (and a bias b that we represent as a weight for an input clamped at +1) and producing an output y. We include some convenient intermediate variables: the output of the summation, z, and the output of the sigmoid, a. In this case the output of the unit y is the same as a, but in deeper networks we'll reserve y to mean the final output of the entire network, leaving a as the activation of an individual node.

Let's walk through an example just to get an intuition. Let's suppose we have a unit with the following weight vector and bias:

$$\begin{aligned} \mathbf{w} &= [0.2, 0.3, 0.9] \\ b &= 0.5 \end{aligned}$$

What would this unit do with the following input vector:

$$\mathbf{x} = [0.5, 0.6, 0.1]$$

The resulting output y would be:

$$y = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1+e^{-(\mathbf{w} \cdot \mathbf{x} + b)}} = \frac{1}{1+e^{-(.5*.2+.6*.3+.1*.9+.5)}} = \frac{1}{1+e^{-0.87}} = .70$$

tanh

In practice, the sigmoid is not commonly used as an activation function. A function that is very similar but almost always better is the **tanh** function shown in Fig. 6.3a; tanh is a variant of the sigmoid that ranges from -1 to +1:

$$y = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \tag{6.5}$$

ReLU

The simplest activation function, and perhaps the most commonly used, is the rectified linear unit, also called the **ReLU**, shown in Fig. 6.3b. It's just the same as z when z is positive, and 0 otherwise:

$$y = \text{ReLU(z)} = max(z, 0) \tag{6.6}$$

These activation functions have different properties that make them useful for different language applications or network architectures. For example, the tanh function has the nice properties of being smoothly differentiable and mapping outlier values toward the mean. The rectifier function, on the other hand, has nice properties that

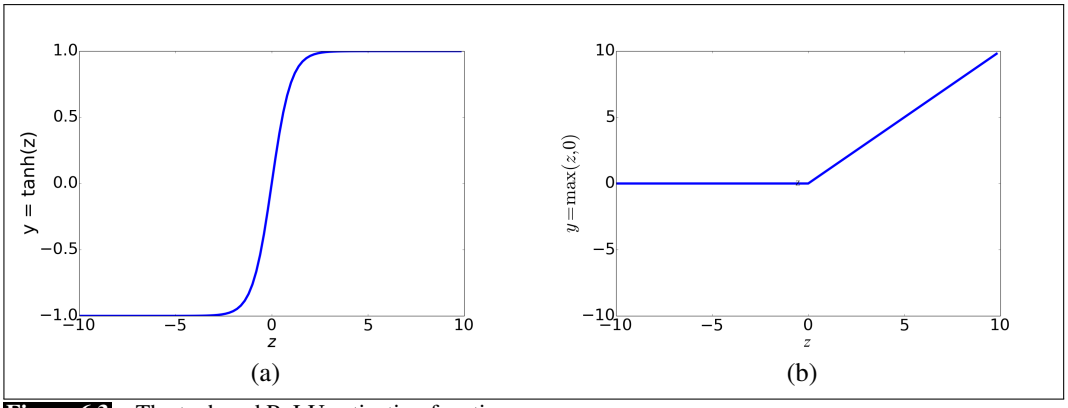


Figure 6.3 The tanh and ReLU activation functions.

saturated

result from it being very close to linear. In the sigmoid or tanh functions, very high values of z result in values of y that are **saturated**, i.e., extremely close to 1, and have derivatives very close to 0. Zero derivatives cause problems for learning, because as we'll see in Section 6.6, we'll train networks by propagating an error signal backwards, multiplying gradients (partial derivatives) from each layer of the network; gradients that are almost 0 cause the error signal to get smaller and smaller until it is too small to be used for training, a problem called the **vanishing gradient** problem. Rectifiers don't have this problem, since the derivative of ReLU for high values of z is 1 rather than very close to 0.

vanishing gradient

6.2 The XOR problem

Early in the history of neural networks it was realized that the power of neural networks, as with the real neurons that inspired them, comes from combining these units into larger networks.

One of the most clever demonstrations of the need for multi-layer networks was the proof by Minsky and Papert (1969) that a single neural unit cannot compute some very simple functions of its input. Consider the task of computing elementary logical functions of two inputs, like AND, OR, and XOR. As a reminder, here are the truth tables for those functions:

AND			OR			XOR		
x1	x2	y	x1	x2	y	x1	x2	y
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

perceptron

This example was first shown for the **perceptron**, which is a very simple neural unit that has a binary output and has a very simple step function as its non-linear activation function. The output y of a perceptron is 0 or 1, and is computed as follows (using the same weight $\mathbf{w}$, input $\mathbf{x}$, and bias b as in Eq. 6.2):

$$y = \begin{cases} 0, & \text{if } \mathbf{w}\cdot\mathbf{x}+b \leq 0 \\ 1, & \text{if } \mathbf{w}\cdot\mathbf{x}+b > 0 \end{cases} \tag{6.7}$$

It's very easy to build a perceptron that can compute the logical AND and OR functions of its binary inputs; Fig. 6.4 shows the necessary weights.

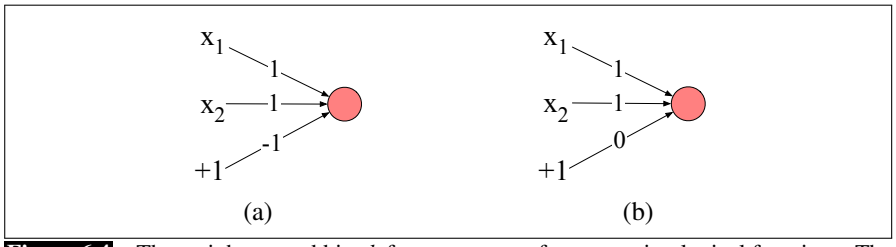


Figure 6.4 The weights w and bias b for perceptrons for computing logical functions. The inputs are shown as x_1 and x_2 and the bias as a special node with value $+1$ which is multiplied with the bias weight b. (a) logical AND, with weights $w_1 = 1$ and $w_2 = 1$ and bias weight $b = -1$. (b) logical OR, with weights $w_1 = 1$ and $w_2 = 1$ and bias weight $b = 0$. These weights/biases are just one from an infinite number of possible sets of weights and biases that would implement the functions.

It turns out, however, that it's not possible to build a perceptron to compute logical XOR! (It's worth spending a moment to give it a try!)

The intuition behind this important result relies on understanding that a perceptron is a linear classifier. For a two-dimensional input x_1 and x_2, the perceptron equation, $w_1x_1 + w_2x_2 + b = 0$ is the equation of a line. (We can see this by putting it in the standard linear format: $x_2 = (-w_1/w_2)x_1 + (-b/w_2)$.) This line acts as a **decision boundary** in two-dimensional space in which the output 0 is assigned to all inputs lying on one side of the line, and the output 1 to all input points lying on the other side of the line. If we had more than 2 inputs, the decision boundary becomes a hyperplane instead of a line, but the idea is the same, separating the space into two categories.

decision boundary

Fig. 6.5 shows the possible logical inputs (00, 01, 10, and 11) and the line drawn by one possible set of parameters for an AND and an OR classifier. Notice that there is simply no way to draw a line that separates the positive cases of XOR (01 and 10) from the negative cases (00 and 11). We say that XOR is not a **linearly separable** function. Of course we could draw a boundary with a curve, or some other function, but not a single line.

linearly separable

6.2.1 The solution: neural networks

While the XOR function cannot be calculated by a single perceptron, it can be calculated by a layered network of perceptron units. Rather than see this with networks of simple perceptrons, however, let's see how to compute XOR using two layers of ReLU-based units following Goodfellow et al. (2016). Fig. 6.6 shows a figure with the input being processed by two layers of neural units. The middle layer (called h) has two units, and the output layer (called y) has one unit. A set of weights and biases are shown that allows the network to correctly compute the XOR function.

Let's walk through what happens with the input $\mathbf{x} = [0, 0]$. If we multiply each input value by the appropriate weight, sum, and then add the bias b, we get the vector [0, -1], and we then apply the rectified linear transformation to give the output of the $\mathbf{h}$ layer as [0, 0]. Now we once again multiply by the weights, sum, and add the bias (0 in this case) resulting in the value 0. The reader should work through the computation of the remaining 3 possible input pairs to see that the resulting y values are 1 for the inputs [0, 1] and [1, 0] and 0 for [0, 0] and [1, 1].

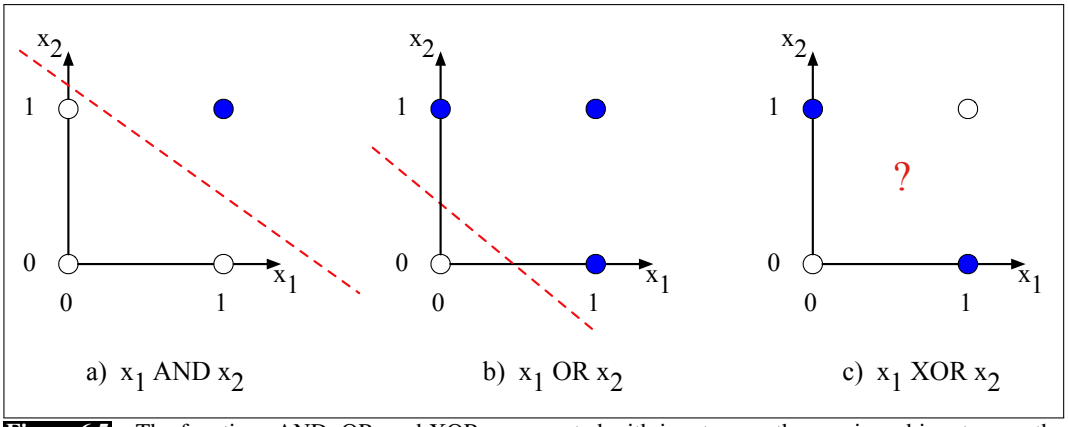


Figure 6.5 The functions AND, OR, and XOR, represented with input x_1 on the x-axis and input x_2 on the y-axis. Filled circles represent perceptron outputs of 1, and white circles perceptron outputs of 0. There is no way to draw a line that correctly separates the two categories for XOR. Figure styled after Russell and Norvig (2002).

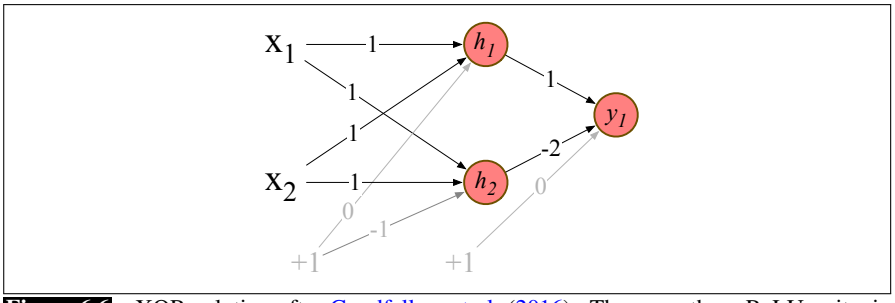


Figure 6.6 XOR solution after Goodfellow et al. (2016). There are three ReLU units, in two layers; we've called them h_1, h_2 (h for "hidden layer") and y_1. As before, the numbers on the arrows represent the weights w for each unit, and we represent the bias b as a weight on a unit clamped to +1, with the bias weights/units in gray.

It's also instructive to look at the intermediate results, the outputs of the two hidden nodes h_1 and h_2. We showed in the previous paragraph that the $\mathbf{h}$ vector for the inputs $\mathbf{x} = [0, 0]$ was $[0, 0]$. Fig. 6.7b shows the values of the $\mathbf{h}$ layer for all 4 inputs. Notice that hidden representations of the two input points $\mathbf{x} = [0, 1]$ and $\mathbf{x} = [1, 0]$ (the two cases with XOR output = 1) are merged to the single point $\mathbf{h} = [1, 0]$. The merger makes it easy to linearly separate the positive and negative cases of XOR. In other words, we can view the hidden layer of the network as forming a representation of the input.

In this example we just stipulated the weights in Fig. 6.6. But for real examples the weights for neural networks are learned automatically using the error backpropagation algorithm to be introduced in Section 6.6. That means the hidden layers will learn to form useful representations. This intuition, that neural networks can automatically learn useful representations of the input, is one of their key advantages, and one that we will return to again and again in later chapters.

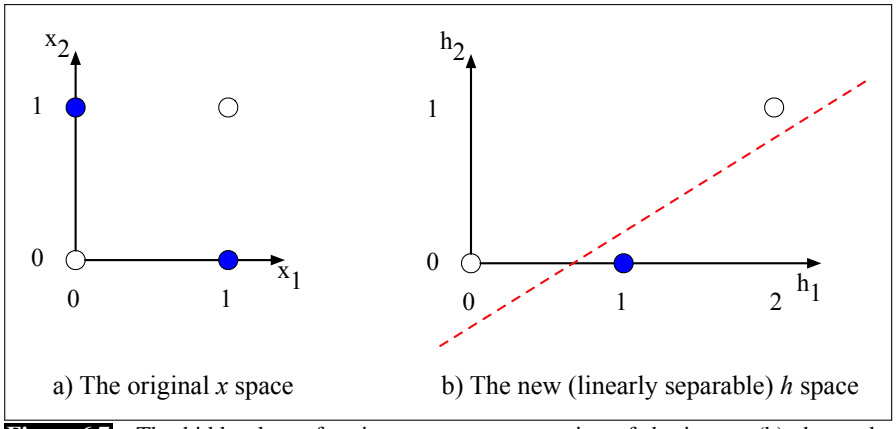


Figure 6.7 The hidden layer forming a new representation of the input. (b) shows the representation of the hidden layer, **h**, compared to the original input representation **x** in (a). Notice that the input point [0, 1] has been collapsed with the input point [1, 0], making it possible to linearly separate the positive and negative cases of XOR. After Goodfellow et al. (2016).

6.3 Feedforward Neural Networks

feedforward network

Let's now walk through a slightly more formal presentation of the simplest kind of neural network, the **feedforward network**. A feedforward network is a multilayer network in which the units are connected with no cycles; the outputs from units in each layer are passed to units in the next higher layer, and no outputs are passed back to lower layers. (In Chapter 13 we'll introduce networks with cycles, called **recurrent neural networks**.)

multi-layer perceptrons **MLP**

For historical reasons multilayer networks, especially feedforward networks, are sometimes called **multi-layer perceptrons** (or **MLP**s); this is a technical misnomer, since the units in modern multilayer networks aren't perceptrons (perceptrons have a simple step-function as their activation function, but modern networks are made up of units with many kinds of non-linearities like ReLUs and sigmoids), but at some point the name stuck.

Simple feedforward networks have three kinds of nodes: input units, hidden units, and output units.

Fig. 6.8 shows a picture. The input layer **x** is a vector of simple scalar values just as we saw in Fig. 6.2.

hidden layer

The core of the neural network is the **hidden layer h** formed of **hidden units** $\mathbf{h}_i$, each of which is a neural unit as described in Section 6.1, taking a weighted sum of its inputs and then applying a non-linearity. In the standard architecture, each layer is **fully-connected**, meaning that each unit in each layer takes as input the outputs from all the units in the previous layer, and there is a link between every pair of units from two adjacent layers. Thus each hidden unit sums over all the input units.

fully-connected

Recall that a single hidden unit has as parameters a weight vector and a bias. We represent the parameters for the entire hidden layer by combining the weight vector and bias for each unit i into a single weight matrix $\mathbf{W}$ and a single bias vector b for the whole layer (see Fig. 6.8). Each element $\mathbf{W}_{ji}$ of the weight matrix $\mathbf{W}$ represents the weight of the connection from the ith input unit x_i to the jth hidden unit h_j.

The advantage of using a single matrix $\mathbf{W}$ for the weights of the entire layer is that now the hidden layer computation for a feedforward network can be done very

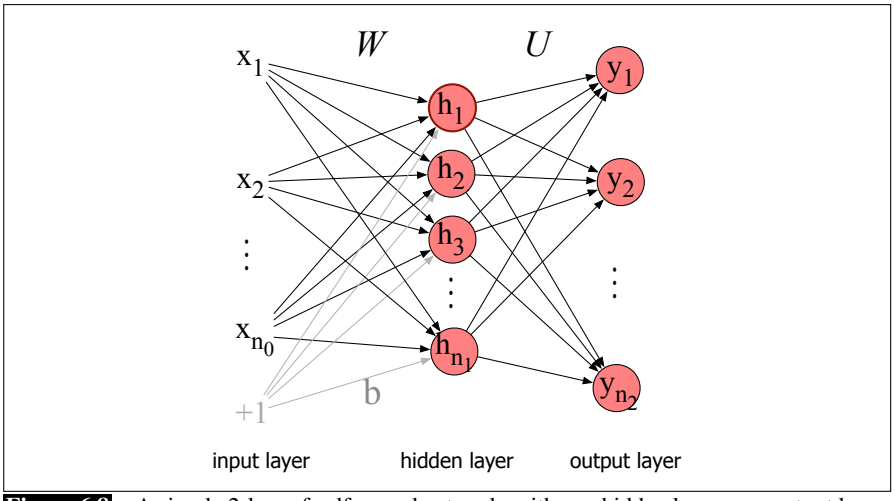


Figure 6.8 A simple 2-layer feedforward network, with one hidden layer, one output layer, and one input layer (the input layer is usually not counted when enumerating layers).

efficiently with simple matrix operations. In fact, the computation only has three steps: multiplying the weight matrix by the input vector $\mathbf{x}$, adding the bias vector $\mathbf{b}$, and applying the activation function g (such as the sigmoid, tanh, or ReLU activation function defined above).

The output of the hidden layer, the vector $\mathbf{h}$, is thus the following (for this example we'll use the sigmoid function σ as our activation function):

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \tag{6.8}$$

Notice that we're applying the σ function here to a vector, while in Eq. 6.3 it was applied to a scalar. We're thus allowing $\sigma(\cdot)$, and indeed any activation function $g(\cdot)$, to apply to a vector element-wise, so $g[z_1, z_2, z_3] = [g(z_1), g(z_2), g(z_3)]$.

Let's introduce some constants to represent the dimensionalities of these vectors and matrices. We'll refer to the input layer as layer 0 of the network, and have n_0 represent the number of inputs, so $\mathbf{x}$ is a vector of real numbers of dimension n_0, or more formally $\mathbf{x} \in \mathbb{R}^{n_0}$, a column vector of dimensionality $[n_0 \times 1]$. Let's call the hidden layer layer 1 and the output layer layer 2. The hidden layer has dimensionality n_1, so $\mathbf{h} \in \mathbb{R}^{n_1}$ and also $\mathbf{b} \in \mathbb{R}^{n_1}$ (since each hidden unit can take a different bias value). And the weight matrix $\mathbf{W}$ has dimensionality $\mathbf{W} \in \mathbb{R}^{n_1 \times n_0}$, i.e. $[n_1 \times n_0]$.

Take a moment to convince yourself that the matrix multiplication in Eq. 6.8 will compute the value of each $\mathbf{h}_j$ as $\sigma\left(\sum_{i=1}^{n_0} \mathbf{W}_{ji}\mathbf{x}_i + \mathbf{b}_j\right)$.

As we saw in Section 6.2, the resulting value $\mathbf{h}$ (for *hidden* but also for *hypothesis*) forms a *representation* of the input. The role of the output layer is to take this new representation $\mathbf{h}$ and compute a final output. This output could be a real-valued number, but in many cases the goal of the network is to make some sort of classification decision, and so we will focus on the case of classification.

If we are doing a binary task like sentiment classification, we might have a single output node, and its scalar value y is the probability of positive versus negative sentiment. If we are doing multinomial classification, such as assigning a part-of-speech tag, we might have one output node for each potential part-of-speech, whose output value is the probability of that part-of-speech, and the values of all the output nodes must sum to one. The output layer is thus a vector $\mathbf{y}$ that gives a probability distribution across the output nodes.

Let's see how this happens. Like the hidden layer, the output layer has a weight matrix (let's call it $\mathbf{U}$), but some models don't include a bias vector $\mathbf{b}$ in the output layer, so we'll simplify by eliminating the bias vector in this example. The weight matrix is multiplied by its input vector ($\mathbf{h}$) to produce the intermediate output $\mathbf{z}$:

$$\mathbf{z} = \mathbf{U}\mathbf{h}$$

There are n_2 output nodes, so $\mathbf{z} \in \mathbb{R}^{n_2}$, weight matrix $\mathbf{U}$ has dimensionality $\mathbf{U} \in \mathbb{R}^{n_2 \times n_1}$, and element $\mathbf{U}_{ij}$ is the weight from unit j in the hidden layer to unit i in the output layer.

However, $\mathbf{z}$ can't be the output of the classifier, since it's a vector of real-valued numbers, while what we need for classification is a vector of probabilities. There is a convenient function for **normalizing** a vector of real values, by which we mean converting it to a vector that encodes a probability distribution (all the numbers lie between 0 and 1 and sum to 1): the **softmax** function that we saw on page 79 of Chapter 4. More generally for any vector $\mathbf{z}$ of dimensionality d, the softmax is defined as:

normalizing

softmax

$$\text{softmax}(\mathbf{z}_i) = \frac{\exp(\mathbf{z}_i)}{\sum_{j=1}^{d} \exp(\mathbf{z}_j)} \quad 1 \le i \le d \tag{6.9}$$

Thus for example given a vector

$$\mathbf{z} = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1], \tag{6.10}$$

the softmax function will normalize it to a probability distribution (shown rounded):

$$\text{softmax}(\mathbf{z}) = [0.055, 0.090, 0.0067, 0.10, 0.74, 0.010] \tag{6.11}$$

You may recall that we used softmax to create a probability distribution from a vector of real-valued numbers (computed from summing weights times features) in the multinomial version of logistic regression in Chapter 4.

That means we can think of a neural network classifier with one hidden layer as building a vector $\mathbf{h}$ which is a hidden layer representation of the input, and then running standard multinomial logistic regression on the features that the network develops in $\mathbf{h}$. By contrast, in Chapter 4 the features were mainly designed by hand via feature templates. So a neural network is like multinomial logistic regression, but (a) with many layers, since a deep neural network is like layer after layer of logistic regression classifiers; (b) with those intermediate layers having many possible activation functions (tanh, ReLU, sigmoid) instead of just sigmoid (although we'll continue to use σ for convenience to mean any activation function); (c) rather than forming the features by feature templates, the prior layers of the network induce the feature representations themselves.

Here are the final equations for a feedforward network with a single hidden layer, which takes an input vector $\mathbf{x}$, outputs a probability distribution $\mathbf{y}$, and is parameterized by weight matrices $\mathbf{W}$ and $\mathbf{U}$ and a bias vector $\mathbf{b}$:

$$\begin{aligned} \mathbf{h} &= \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \\ \mathbf{z} &= \mathbf{U}\mathbf{h} \\ \mathbf{y} &= \text{softmax}(\mathbf{z}) \end{aligned} \tag{6.12}$$

And just to remember the shapes of all our variables, $\mathbf{x} \in \mathbb{R}^{n_0}$, $\mathbf{h} \in \mathbb{R}^{n_1}$, $\mathbf{b} \in \mathbb{R}^{n_1}$, $\mathbf{W} \in \mathbb{R}^{n_1 \times n_0}$, $\mathbf{U} \in \mathbb{R}^{n_2 \times n_1}$, and the output vector $\mathbf{y} \in \mathbb{R}^{n_2}$. We'll call this network a 2-layer network (we traditionally don't count the input layer when numbering layers, but do count the output layer). So by this terminology logistic regression is a 1-layer network.

6.3.1 More details on feedforward networks

Let's now set up some notation to make it easier to talk about deeper networks of depth more than 2. We'll use superscripts in square brackets to mean layer numbers, starting at 0 for the input layer. So $\mathbf{W}^{[1]}$ will mean the weight matrix for the (first) hidden layer, and $\mathbf{b}^{[1]}$ will mean the bias vector for the (first) hidden layer. n_j will mean the number of units at layer j. We'll use $g(\cdot)$ to stand for the activation function, which will tend to be ReLU or tanh for intermediate layers and softmax for output layers. We'll use $\mathbf{a}^{[i]}$ to mean the output from layer i, and $\mathbf{z}^{[i]}$ to mean the combination of previous layer output, weights and biases $\mathbf{W}^{[i]}\mathbf{a}^{[i-1]} + \mathbf{b}^{[i]}$. The 0th layer is for inputs, so we'll refer to the inputs $\mathbf{x}$ more generally as $\mathbf{a}^{[0]}$.

Thus we can re-represent our 2-layer net from Eq. 6.12 as follows:

$$\begin{aligned}
\mathbf{z}^{[1]} &= \mathbf{W}^{[1]}\mathbf{a}^{[0]} + \mathbf{b}^{[1]} \\
\mathbf{a}^{[1]} &= g^{[1]}(\mathbf{z}^{[1]}) \\
\mathbf{z}^{[2]} &= \mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]} \\
\mathbf{a}^{[2]} &= g^{[2]}(\mathbf{z}^{[2]}) \\
\hat{\mathbf{y}} &= \mathbf{a}^{[2]}
\end{aligned} \tag{6.13}$$

Note that with this notation, the equations for the computation done at each layer are the same. The algorithm for computing the forward step in an n-layer feedforward network, given the input vector $a^{[0]}$ is thus simply:

for i **in** 1,...,n
  $\mathbf{z}^{[i]} = \mathbf{W}^{[i]}\,\mathbf{a}^{[i-1]} + \mathbf{b}^{[i]}$
  $\mathbf{a}^{[i]} = g^{[i]}(\mathbf{z}^{[i]})$
$\hat{\mathbf{y}} = \mathbf{a}^{[n]}$

It's often useful to have a name for the final set of activations right before the final softmax. So however many layers we have, we'll generally call the unnormalized values in the final vector $\mathbf{z}^{[n]}$, the vector of scores right before the final softmax, the
logits **logits** (see Eq. 4.7).

The need for non-linear activation functions One of the reasons we use non-linear activation functions for each layer in a neural network is that if we did not, the resulting network is exactly equivalent to a single-layer network. Let's see why this is true. Imagine the first two layers of such a network of purely linear layers:

$$\begin{aligned}
\mathbf{z}^{[1]} &= \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]} \\
\mathbf{z}^{[2]} &= \mathbf{W}^{[2]}\mathbf{z}^{[1]} + \mathbf{b}^{[2]}
\end{aligned}$$

We can rewrite the function that the network is computing as:

$$\begin{aligned}
\mathbf{z}^{[2]} &= \mathbf{W}^{[2]}\mathbf{z}^{[1]} + \mathbf{b}^{[2]} \\
&= \mathbf{W}^{[2]}(\mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}) + \mathbf{b}^{[2]} \\
&= \mathbf{W}^{[2]}\mathbf{W}^{[1]}\mathbf{x} + \mathbf{W}^{[2]}\mathbf{b}^{[1]} + \mathbf{b}^{[2]} \\
&= \mathbf{W}'\mathbf{x} + \mathbf{b}'
\end{aligned} \tag{6.14}$$

This generalizes to any number of layers. So without non-linear activation functions, a multilayer network is just a notational variant of a single layer network with a different set of weights, and we lose all the representational power of multilayer networks.

Replacing the bias unit In describing networks, we will sometimes use a slightly simplified notation that represents exactly the same function without referring to an explicit bias node b. Instead, we add a dummy node $\mathbf{a}_0$ to each layer whose value will always be 1. Thus layer 0, the input layer, will have a dummy node $\mathbf{a}_0^{[0]} = 1$, layer 1 will have $\mathbf{a}_0^{[1]} = 1$, and so on. This dummy node still has an associated weight, and that weight represents the bias value b. For example instead of an equation like

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \tag{6.15}$$

we'll use:

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x}) \tag{6.16}$$

But now instead of our vector $\mathbf{x}$ having n_0 values: $\mathbf{x} = \mathbf{x}_1, \ldots, \mathbf{x}_{n_0}$, it will have $n_0 + 1$ values, with a new 0th dummy value $\mathbf{x}_0 = 1$: $\mathbf{x} = \mathbf{x}_0, \ldots, \mathbf{x}_{n_0}$. And instead of computing each $\mathbf{h}_j$ as follows:

$$\mathbf{h}_j = \sigma\left(\sum_{i=1}^{n_0} \mathbf{W}_{ji}\mathbf{x}_i + \mathbf{b}_j\right), \tag{6.17}$$

we'll instead use:

$$\mathbf{h}_j = \sigma\left(\sum_{i=0}^{n_0} \mathbf{W}_{ji}\mathbf{x}_i\right), \tag{6.18}$$

where the value $\mathbf{W}_{j0}$ replaces what had been $\mathbf{b}_j$. Fig. 6.9 shows a visualization.

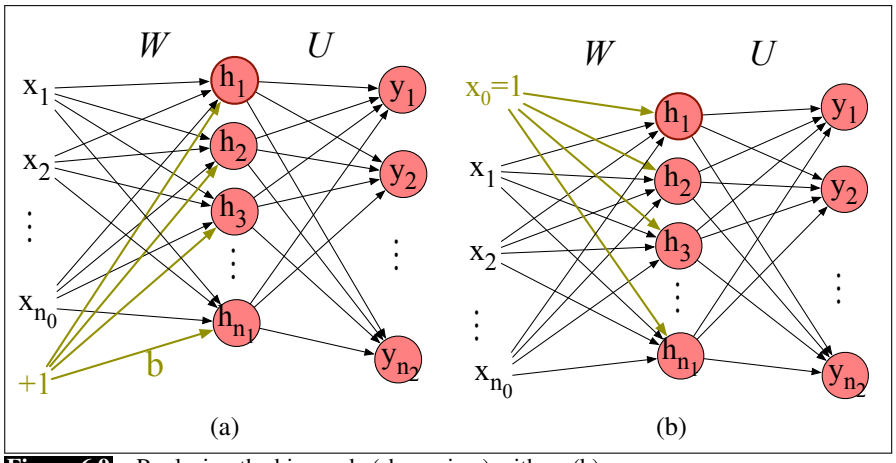


Figure 6.9 Replacing the bias node (shown in a) with x_0 (b).

We'll continue showing the bias as b when we go over the learning algorithm in Section 6.6, but going forward in the book, for most figures and some equations we'll use this simplified notation without explicit bias terms.

6.4 Feedforward networks for NLP: Classification

Let's see how to apply feedforward networks to NLP classification tasks. In practice, simple feedforward networks aren't the way we do text classification; for real applications we would use more sophisticated architectures like the BERT transformers

of Chapter 9. Nonetheless seeing a feedforward network text classifier will let us introduce key ideas that will play a role throughout the rest of the book, including the ideas of the **embedding matrix**, representation **pooling**, and **representation learning**.

But before introducing any of these ideas, let's start with a classifier by making only minimal change from the sentiment classifiers we saw in Chapter 4. Like them, we'll take hand-built features, pass them through a classifier, and produce a class probability. The only difference is that we'll use a neural network instead of logistic regression as the classifier.

6.4.1 Neural net classifiers with hand-built features

Let's begin with a simple 2-layer sentiment classifier by taking our logistic regression classifier from Chapter 4, which corresponds to a 1-layer network, and just adding a hidden layer. The input element $\mathbf{x}_i$ can be scalar features like those in Fig. 4.2, e.g., $\mathbf{x}_1$ = count(words $\in$ doc), $\mathbf{x}_2$ = count(positive lexicon words $\in$ doc), $\mathbf{x}_3$ = 1 if "no" $\in$ doc, and so on, for a total of d features. And the output layer $\hat{\mathbf{y}}$ could have two nodes (one each for positive and negative), or 3 nodes (positive, negative, neutral), in which case $\hat{\mathbf{y}}_1$ would be the estimated probability of positive sentiment, $\hat{\mathbf{y}}_2$ the probability of negative and $\hat{\mathbf{y}}_3$ the probability of neutral. The resulting equations would be just what we saw above for a 2-layer network (as always, we'll continue to use the σ to stand for any non-linearity, whether sigmoid, ReLU or other).

$$\begin{aligned}
\mathbf{x} &= [\mathbf{x}_1, \mathbf{x}_2, \ldots \mathbf{x}_d] \quad \text{(each } \mathbf{x}_i \text{ is a hand-designed feature)} \\
\mathbf{h} &= \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \\
\mathbf{z} &= \mathbf{U}\mathbf{h} \\
\hat{\mathbf{y}} &= \text{softmax}(\mathbf{z}) \qquad (6.19)
\end{aligned}$$

Fig. 6.10 shows a sketch of this architecture. As we mentioned earlier, adding this hidden layer to our logistic regression classifier allows the network to represent the non-linear interactions between features. This alone might give us a better sentiment classifier.

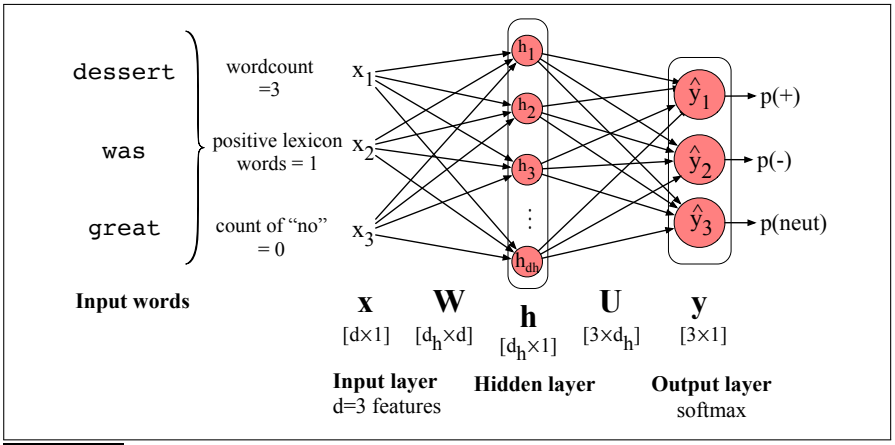


Figure 6.10 Feedforward network sentiment analysis using traditional hand-built features of the input text.

6.4.2 Vectorizing for parallelizing inference

While Eq. 6.19 shows how to classify a single example x, in practice we want to efficiently classify an entire test set of m examples. We do this by vectorizing the process, just as we saw with logistic regression; instead of using for-loops to go through each example, we'll use matrix multiplication to do the entire computation of an entire test set at once. First, we pack all the input feature vectors for each input x into a single input matrix $\mathbf{X}$, with each row i a row vector consisting of the features for input example $x^{(i)}$ (i.e., the vector $\mathbf{x}^{(i)}$). If the dimensionality of our input feature vector is d, $\mathbf{X}$ will be a matrix of shape $[m \times d]$.

Because we are now modeling each input as a row vector rather than a column vector, we also need to slightly modify Eq. 6.19. $\mathbf{X}$ is of shape $[m \times d]$ and $\mathbf{W}$ is of shape $[d_h \times d]$, so we'll reorder how we multiply $\mathbf{X}$ and $\mathbf{W}$ and transpose $\mathbf{W}$ so they correctly multiply to yield a matrix $\mathbf{H}$ of shape $[m \times d_h]$. [1]

The bias vector $\mathbf{b}$ from Eq. 6.19 of shape $[1 \times d_h]$ will now have to be replicated into a matrix of shape $[m \times d_h]$. We'll need to similarly reorder the next step and transpose $\mathbf{U}$. Finally, our output matrix $\hat{\mathbf{Y}}$ will be of shape $[m \times 3]$ (or more generally $[m \times d_o]$, where d_o is the number of output classes), with each row i of our output matrix $\hat{\mathbf{Y}}$ consisting of the output vector $\hat{\mathbf{y}}^{(i)}$. Here are the final equations for computing the output class distribution for an entire test set:

$$\begin{aligned} \mathbf{H} &= \sigma(\mathbf{X}\mathbf{W}^\intercal + \mathbf{b}) \\ \mathbf{Z} &= \mathbf{H}\mathbf{U}^\intercal \\ \hat{\mathbf{Y}} &= \text{softmax}(\mathbf{Z}) \end{aligned} \tag{6.20}$$

In this book, we'll sometimes see orderings like $\mathbf{W}\mathbf{X} + \mathbf{b}$ and sometimes $\mathbf{X}\mathbf{W} + \mathbf{b}$. That's why it's always important to be very aware of the shapes of your weight matrices participating in any given equation.

6.5 Embeddings as the input to neural net classifiers

While hand-built features are a traditional way to design classifiers, most applications of neural networks for NLP don't use hand-built human-engineered features as inputs. Instead, we draw on deep learning's ability to learn features from the data by representing tokens as embeddings. For this section we'll represent each token by its static word2vec or GloVe embeddings that we saw how to compute in Chapter 5. By static embedding, we mean that each token is represented by a fixed vector that we train once, and then just put into a big dictionary. When we want to refer to that token, we grab its embedding out of the dictionary.

However when we apply neural models to the task of language modeling (as we'll see in Chapter 8) the situation is more complex, and we'll use a more powerful kind of embedding called a *contextual embedding*. Contextual embeddings are different for each time a word occurs in a different context. Furthermore, we'll have the network learn these embeddings as part of the task of word prediction.

So let's explore the text classification domain above, but using static embeddings as features instead of the hand-designed features. Let's focus on the inference stage,

[1] Note that we could have kept the original order of our products if we had instead made our input matrix $\mathbf{X}$ represent each input as a column vector instead of a row vector, making it of shape $[d \times m]$. But representing inputs as row vectors is convenient and common in neural network models.

in which we have already learned embeddings for all the input tokens. An embedding is a vector of dimension d that represents the input token. The dictionary of static embeddings in which we store these embeddings is the **embedding matrix** $\mathbf{E}$. Each row of the embedding matrix represents each token of the vocabulary V as a (row) vector of dimensionality d. Since $\mathbf{E}$ has a row for each of the $|V|$ tokens in the vocabulary, $\mathbf{E}$ has shape $[|V| \times d]$. This embedding matrix $\mathbf{E}$ plays a role whenever we are using embeddings as input to neural NLP systems, including in the transformer-based large language models we will introduce over the next chapters.

embedding matrix

Given an input token string like `dessert was great` we first convert the tokens into vocabulary indices (these were created when we first tokenized the input using BPE or SentencePiece). So the representation of `dessert was great` might be $\mathbf{w} = [3, 9824, 226]$. Next we use indexing to select the corresponding rows from $\mathbf{E}$ (row 3, row 9824, row 226).

Another way to think about selecting token embeddings from the embedding matrix is to represent input tokens as one-hot vectors of shape $[1 \times |V|]$, i.e., with one dimension for each word in the vocabulary. Recall that in a **one-hot vector** all the elements are 0 except one, the element whose dimension is the word's index in the vocabulary, which has value 1. So if the word "dessert" has index 3 in the vocabulary, $x_3 = 1$, and $x_i = 0 \ \forall i \neq 3$, as shown here:

one-hot vector

```
[0 0 1 0 0 0 0 ... 0 0 0 0]
 1 2 3 4 5 6 7 ...  ... |V|
```

Multiplying by a one-hot vector that has only one non-zero element $x_i = 1$ simply selects out the relevant row vector for word i, resulting in the embedding for word i, as depicted in Fig. 6.11.

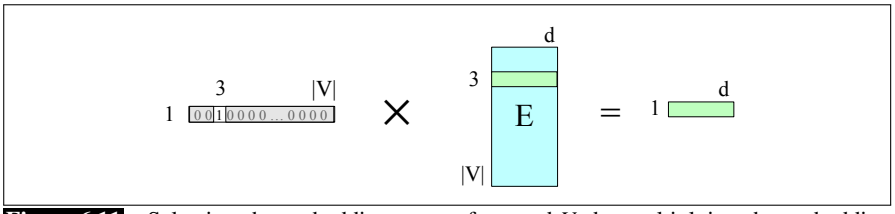


Figure 6.11 Selecting the embedding vector for word V_3 by multiplying the embedding matrix $\mathbf{E}$ with a one-hot vector with a 1 in index 3.

We can extend this idea to represent the entire input token sequence as a matrix of one-hot vectors, one for each of the N input positions as shown in Fig. 6.12.

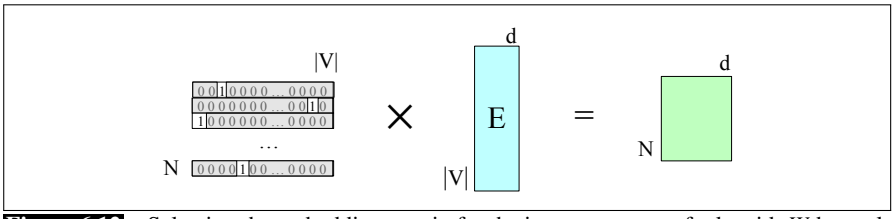


Figure 6.12 Selecting the embedding matrix for the input sequence of token ids W by multiplying a one-hot matrix corresponding to W by the embedding matrix $\mathbf{E}$.

We now need to classify this input of N $[1 \times d]$ embeddings, representing a window of N tokens, into a single class (like positive or negative).

There are two common ways to pass embeddings to a classifier: **concatenation** and **pooling**. First, we can take this input of shape $[N \times d]$ and reshape it by **concatenating** all the input vectors into one very long vector of shape $[1 \times dN]$. Then

pool

we pass this input to our classifier and let it make its decision. This gives us lots of information, at the cost of using a pretty large network. Second, we can **pool** the N embeddings into a single embedding and then pass that single pooled embedding to the classifier. Pooling gives us less information than would have been present in all the original embeddings, but has the advantage of being small and efficient and is especially useful in tasks for which we don't care as much about the original word order. Let's give an example of each: pooling for the sentiment task, and concatenation for the language modeling task.

Pooling input embeddings for sentiment So let's begin with seeing how pooling can work for the sentiment classification task. The intuition of pooling is that for sentiment, the exact position of the input (is some word like `great` the first word? the second word?) is less important than the identity of the word itself.

A pooling function is a way to turn a set of embeddings into a single embedding. For example, for a text with N input words/tokens $w_1, ..., w_N$, we want to turn the N row embeddings $\mathbf{e}(w_1), ..., \mathbf{e}(w_N)$ (each of dimensionality d) into a single embedding also of dimensionality d.

mean-pooling

There are various ways to pool. The simplest is **mean-pooling**: taking the mean by summing the embeddings and then dividing by N:

$$\mathbf{x}_{mean} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{e}(w_i) \tag{6.21}$$

Here are the equations for this classifier assuming mean pooling:

$$\begin{aligned}\mathbf{x} &= \text{mean}(\mathbf{e}(w_1), \mathbf{e}(w_2), \ldots, \mathbf{e}(w_n)) \\ \mathbf{h} &= \sigma(\mathbf{x}\mathbf{W} + \mathbf{b}) \\ \mathbf{z} &= \mathbf{h}\mathbf{U} \\ \hat{\mathbf{y}} &= \text{softmax}(\mathbf{z})\end{aligned} \tag{6.22}$$

The architecture is sketched in Fig. 6.13, where we also give the shapes for all the relevant matrices.

max-pooling

There are many other options for pooling, like **max-pooling**, in which case for each dimension we take the element-wise max over all the inputs. The element-wise max of a set of N vectors is a new vector whose kth element is the max of the kth elements of all the N vectors.

Concatenating input embeddings for language modeling For sentiment analysis we saw how to generate an output vector with probabilities over three classes: positive, negative, or neutral, given as input a window of N input tokens, by first pooling those token embeddings into a single embedding vector.

Now let's consider **language modeling**: predicting upcoming words from prior words. In this task we are given the same window of N input tokens, but our task now is to predict the next token that should follow the window. We'll sketch a simple feedforward neural language model, drawing on an algorithm first introduced by Bengio et al. (2003). The feedforward language model introduces many of the important concepts of large language modeling that we will return to in Chapter 7 and Chapter 8.

Neural language models have many advantages over the n-gram language models of Chapter 3. Neural language models can handle much longer histories, can generalize better over contexts of similar words, and are far more accurate at word-

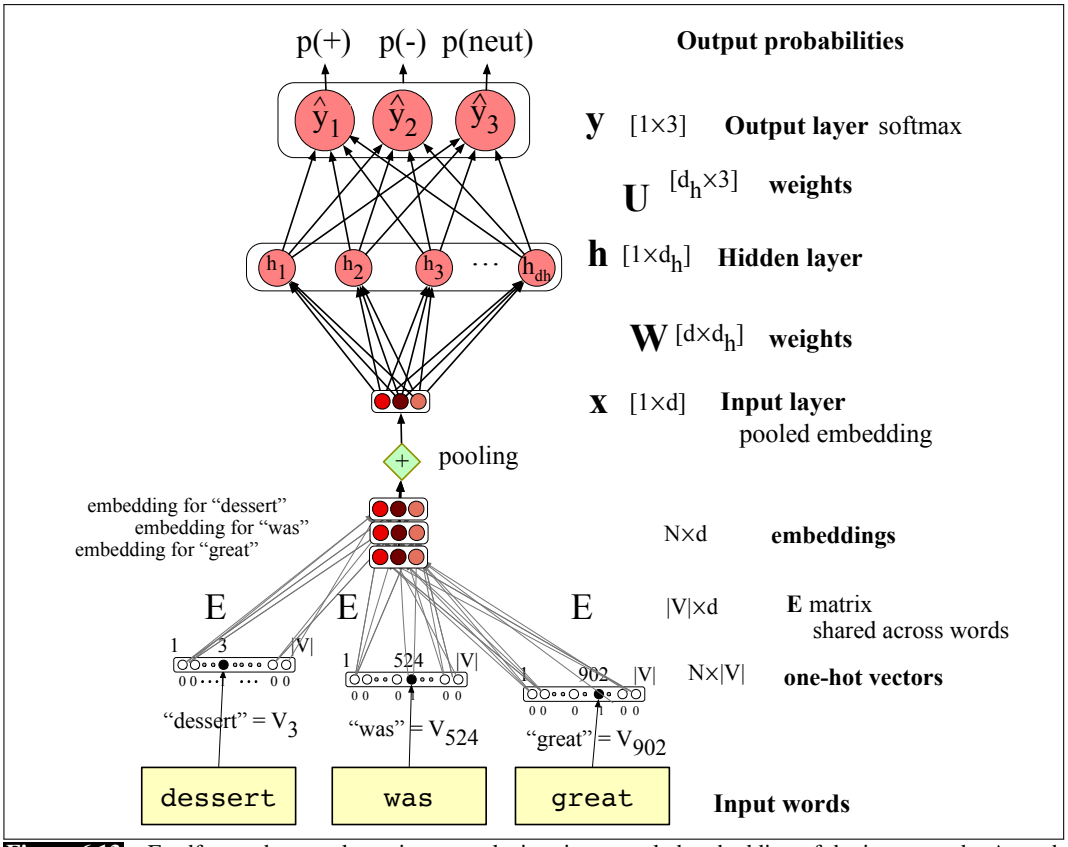


Figure 6.13 Feedforward network sentiment analysis using a pooled embedding of the input words. At each timestep the network computes a d-dimensional embedding for each context word (by multiplying a one-hot vector by the embedding matrix **E**), and pools the resulting N embeddings to get a single embedding that represents the context window as the layer **e**.

prediction. On the other hand, neural net language models are slower, more complex, need vast amounts of energy to train, and are less interpretable than n-gram models, so for some smaller tasks an n-gram language model is still the right tool.

A feedforward neural language model is a feedforward network that takes as input at time t a representation of some number of previous words (w_{t-1}, w_{t-2}, etc.) and outputs a probability distribution over possible next words. Thus—like the n-gram LM—the feedforward neural LM approximates the probability of a word given the entire prior context $P(w_t|w_{1:t-1})$ by approximating based on the $N-1$ previous words:

$$P(w_t|w_1,\ldots,w_{t-1}) \approx P(w_t|w_{t-N+1},\ldots,w_{t-1}) \tag{6.23}$$

In the following examples we'll use a 4-gram example, so we'll show a neural net to estimate the probability $P(w_t = i|w_{t-3}, w_{t-2}, w_{t-1})$.

Neural language models represent words in this prior context by their **embeddings**, rather than just by their word identity as used in n-gram language models. Using embeddings allows neural language models to generalize better to unseen data. For example, suppose we've seen this sentence in training:

I have to make sure that the cat gets fed.

but have never seen the words "gets fed" after the word "dog". Our test set has the

prefix "I forgot to make sure that the dog gets". What's the next word? An n-gram language model will predict "fed" after "that the cat gets", but not after "that the dog gets". But a neural LM, knowing that "cat" and "dog" have similar embeddings, will be able to generalize from the "cat" context to assign a high enough probability to "fed" even after seeing "dog".

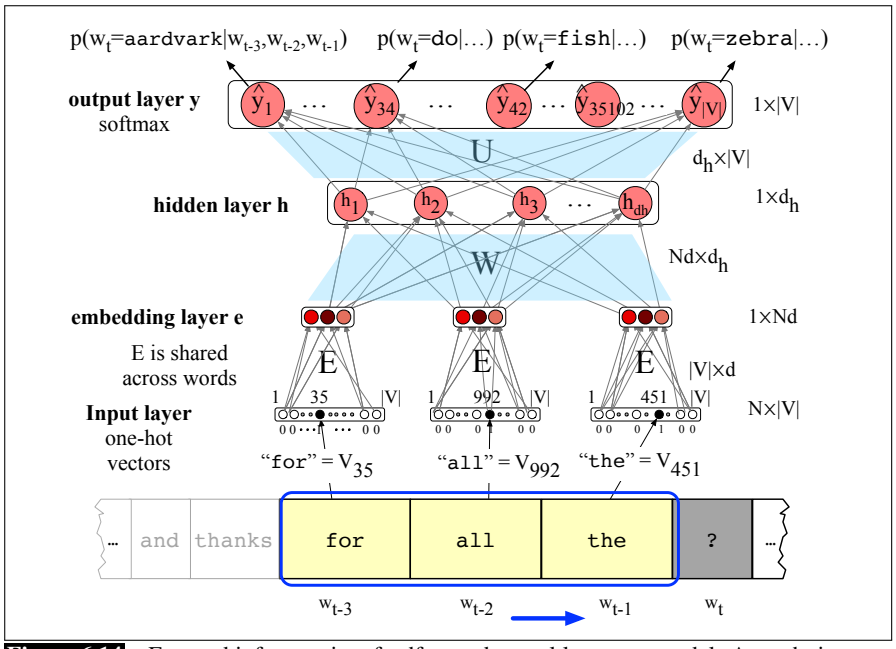


Figure 6.14 Forward inference in a feedforward neural language model. At each timestep t the network computes a d-dimensional embedding for each of the $N = 3$ context tokens (by multiplying a one-hot vector by the embedding matrix **E**), and concatenates the three to get the embedding **e**. This embedding **e** is multiplied by weight matrix **W** and then an activation function is applied element-wise to produce the hidden layer **h**, which is then multiplied by another weight matrix **U**. A softmax layer predicts at each output node i the probability that the next word w_t will be vocabulary word V_i. We show the context window size N as 3 just to fit on the page, but in practice language modeling requires a much longer context.

This prediction task requires an output vector that expresses $|V|$ probabilities: one probability value for each possible next token. We might have a vocabulary between 60,000 and 300,000 tokens, so the output vector for the task of language modeling is much longer than 3. Another difference for language modeling is that instead of pooling the embeddings of the N input tokens to create a single embedding, we concatenate the inputs into one very long input vector. To predict the next token, it helps to know each of the preceding tokens and what order they were in.

Fig. 6.14 shows the language modeling task, sketched with a very short context window of $N = 3$ just to fit on the page. These 3 embedding vectors are concatenated to produce **e**, the embedding layer. This is multiplied by a weight matrix **W** to produce a hidden layer, and another weight matrix **U** to produce an output layer whose softmax gives a probability distribution over words. For example y_{42}, the value of output node 42, is the probability of the next word w_t being V_{42}, the vocabulary word with index 42 (which is the word 'fish' in our example).

The equations for a simple feedforward neural language model with a window

size of 3, given one-hot input vectors for each input context word, are:

$$\begin{aligned}\mathbf{e} &= [\mathbf{E}\mathbf{x}_{\mathbf{t}-3}; \mathbf{E}\mathbf{x}_{\mathbf{t}-2}; \mathbf{E}\mathbf{x}_{\mathbf{t}-1}] \\ \mathbf{h} &= \sigma(\mathbf{W}\mathbf{e} + \mathbf{b}) \\ \mathbf{z} &= \mathbf{U}\mathbf{h} \\ \hat{\mathbf{y}} &= \text{softmax}(\mathbf{z}) \end{aligned} \tag{6.24}$$

Note thatwe we use semicolons to mean concatenation of vectors, so we form the embedding layer $\mathbf{e}$ by concatenating the 3 embeddings for the three context vectors.

We'll return to this idea of using neural networks to do language modeling in Chapter 7 and Chapter 8 when we introduce transformer language models.

6.6 Training Neural Nets

A feedforward neural net is an instance of supervised machine learning in which we know the correct output y for each observation x. What the system produces, via Eq. 6.13, is $\hat{y}$, the system's estimate of the true y. The goal of the training procedure is to learn parameters $\mathbf{W}^{[i]}$ and $\mathbf{b}^{[i]}$ for each layer i that make $\hat{y}$ for each training observation as close as possible to the true y.

In general, we do all this by drawing on the methods we introduced in Chapter 4 for logistic regression, so the reader should be comfortable with that chapter before proceeding. We'll explore the algorithm on simple generic networks rather than networks designed for sentiment or language modeling.

First, we'll need a **loss function** that models the distance between the system output and the gold output, and it's common to use the loss function used for logistic regression, the **cross-entropy loss**.

Second, to find the parameters that minimize this loss function, we'll use the **gradient descent** optimization algorithm introduced in Chapter 4.

Third, gradient descent requires knowing the **gradient** of the loss function, the vector that contains the partial derivative of the loss function with respect to each of the parameters. In logistic regression, for each observation we could directly compute the derivative of the loss function with respect to an individual w or b. But for neural networks, with millions of parameters in many layers, it's much harder to see how to compute the partial derivative of some weight in layer 1 when the loss is attached to some much later layer. How do we partial out the loss over all those intermediate layers? The answer is the algorithm called **error backpropagation** or **backward differentiation**.

6.6.1 Loss function

cross-entropy loss

The **cross-entropy loss** that is used in neural networks is the same one we saw for logistic regression. If the neural network is being used as a binary classifier, with the sigmoid at the final layer, the loss function is the same logistic regression loss we saw in Eq. 4.19:

$$L_{CE}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1-y)\log(1-\hat{y})] \tag{6.25}$$

If we are using the network to classify into 3 or more classes, the loss function is exactly the same as the loss for multinomial regression that we saw in Chapter 4 on

page 82. Let's briefly summarize the explanation here for convenience. First, when we have more than 2 classes we'll need to represent both $\mathbf{y}$ and $\hat{\mathbf{y}}$ as vectors. Let's assume we're doing **hard classification**, where only one class is the correct one. The true label $\mathbf{y}$ is then a vector with K elements, each corresponding to a class, with $\mathbf{y}_c = 1$ if the correct class is c, with all other elements of $\mathbf{y}$ being 0. Recall that a vector like this, with one value equal to 1 and the rest 0, is called a **one-hot vector**. And our classifier will produce an estimate vector with K elements $\hat{\mathbf{y}}$, each element $\hat{\mathbf{y}}_k$ of which represents the estimated probability $p(\mathbf{y}_k = 1|\mathbf{x})$.

The loss function for a single example $\mathbf{x}$ is the negative sum of the logs of the K output classes, each weighted by their probability y_k:

$$L_{CE}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{k=1}^{K} \mathbf{y}_k \log \hat{\mathbf{y}}_k \tag{6.26}$$

We can simplify this equation further; let's first rewrite the equation using the function $\mathbb{1}\{\}$ which evaluates to 1 if the condition in the brackets is true and to 0 otherwise. This makes it more obvious that the terms in the sum in Eq. 6.26 will be 0 except for the term corresponding to the true class for which $\mathbf{y}_k = 1$:

$$L_{CE}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{k=1}^{K} \mathbb{1}\{\mathbf{y}_k = 1\} \log \hat{\mathbf{y}}_k$$

In other words, the cross-entropy loss is simply the negative log of the output probability corresponding to the correct class, and we therefore also call this the **negative log likelihood loss**:

negative log likelihood loss

$$L_{CE}(\hat{\mathbf{y}}, \mathbf{y}) = -\log \hat{\mathbf{y}}_c \quad \text{(where } c \text{ is the correct class)} \tag{6.27}$$

Plugging in the softmax formula from Eq. 6.9, and with K the number of classes:

$$L_{CE}(\hat{\mathbf{y}}, \mathbf{y}) = -\log \frac{\exp(\mathbf{z}_c)}{\sum_{j=1}^{K} \exp(\mathbf{z}_j)} \quad \text{(where } c \text{ is the correct class)} \tag{6.28}$$

Let's think about the negative log probability as a loss function. A perfect classifier would assign the correct class i probability 1 and all the incorrect classes probability 0. That means the higher $p(\hat{y}_i)$ (the closer it is to 1), the better the classifier; $p(\hat{y}_i)$ is (the closer it is to 0), the worse the classifier. The negative log of this probability is a beautiful loss metric since it goes from 0 (negative log of 1, no loss) to infinity (negative log of 0, infinite loss). This loss function also insures that as probability of the correct answer is maximized, the probability of all the incorrect answers is minimized; since they all sum to one, any increase in the probability of the correct answer is coming at the expense of the incorrect answers.

The number K of classes of the output vector $\hat{y}$ can be small or large. Perhaps our task is 3-way sentiment, and then the classes might be positive, negative, and neutral. Or if our task is deciding the part of speech of a word (i.e., whether it is a noun or verb or adjective, etc.), then K is set of possible parts of speech in our tagset (of which there are 17 in the tagset we will define in Chapter 17). And if our task is language modeling, and our classifier is trying to predict which word is next, then our set of classes is the set of words, which might be 50,000 or 100,000.

6.6.2 Computing the Gradient

How do we compute the gradient of this loss function? Computing the gradient requires the partial derivative of the loss function with respect to each parameter. For a network with one weight layer and sigmoid output (which is what logistic regression is), we could simply use the derivative of the loss that we used for logistic regression in Eq. 6.29 (and derived in Section 4.15):

$$\begin{aligned}\frac{\partial L_{CE}(\hat{\mathbf{y}},\mathbf{y})}{\partial w_j} &= (\hat{y}-y)\,\mathbf{x}_j \\ &= (\sigma(\mathbf{w}\cdot\mathbf{x}+b)-y)\,\mathbf{x}_j \end{aligned} \tag{6.29}$$

Or for a network with one weight layer and softmax output (=multinomial logistic regression), we could use the derivative of the softmax loss from Eq. 4.41, shown for a particular weight $\mathbf{w}_k$ and input $\mathbf{x}_i$

$$\begin{aligned}\frac{\partial L_{\text{CE}}(\hat{\mathbf{y}},\mathbf{y})}{\partial \mathbf{w}_{k,i}} &= -(\mathbf{y}_k-\hat{\mathbf{y}}_k)\mathbf{x}_i \\ &= -(\mathbf{y}_k-p(\mathbf{y}_k=1|\mathbf{x}))\mathbf{x}_i \\ &= -\left(\mathbf{y}_k-\frac{\exp(\mathbf{w}_k\cdot\mathbf{x}+b_k)}{\sum_{j=1}^{K}\exp(\mathbf{w}_j\cdot\mathbf{x}+b_j)}\right)\mathbf{x}_i \end{aligned} \tag{6.30}$$

But these derivatives only give correct updates for one weight layer: the last one! For deep networks, computing the gradients for each weight is much more complex, since we are computing the derivative with respect to weight parameters that appear all the way back in the very early layers of the network, even though the loss is computed only at the very end of the network.

error backpropagation

The solution to computing this gradient is an algorithm called **error backpropagation** or **backprop** (Rumelhart et al., 1986). While backprop was invented specially for neural networks, it turns out to be the same as a more general procedure called **backward differentiation**, which depends on the notion of **computation graphs**. Let's see how that works in the next subsection.

6.6.3 Computation Graphs

A computation graph is a representation of the process of computing a mathematical expression, in which the computation is broken down into separate operations, each of which is modeled as a node in a graph.

Consider computing the function $L(a,b,c)=c(a+2b)$. If we make each of the component addition and multiplication operations explicit, and add names (d and e) for the intermediate outputs, the resulting series of computations is:

$$\begin{aligned} d &= 2*b \\ e &= a+d \\ L &= c*e \end{aligned}$$

We can now represent this as a graph, with nodes for each operation, and directed edges showing the outputs from each operation as the inputs to the next, as in Fig. 6.15. The simplest use of computation graphs is to compute the value of the function with some given inputs. In the figure, we've assumed the inputs $a=3$, $b=1$, $c=-2$, and we've shown the result of the **forward pass** to compute the result $L(3,1,-2)=-10$. In the forward pass of a computation graph, we apply each

operation left to right, passing the outputs of each computation as the input to the next node.

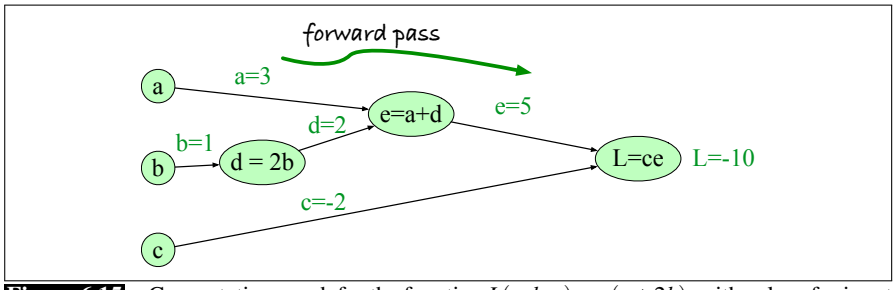


Figure 6.15 Computation graph for the function $L(a,b,c) = c(a+2b)$, with values for input nodes $a = 3$, $b = 1$, $c = -2$, showing the forward pass computation of L.

6.6.4 Backward differentiation on computation graphs

The importance of the computation graph comes from the **backward pass**, which is used to compute the derivatives that we'll need for the weight update. In this example our goal is to compute the derivative of the output function L with respect to each of the input variables, i.e., $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, and $\frac{\partial L}{\partial c}$. The derivative $\frac{\partial L}{\partial a}$ tells us how much a small change in a affects L.

chain rule

Backwards differentiation makes use of the **chain rule** in calculus, so let's remind ourselves of that. Suppose we are computing the derivative of a composite function $f(x) = u(v(x))$. The derivative of $f(x)$ is the derivative of $u(x)$ with respect to $v(x)$ times the derivative of $v(x)$ with respect to x:

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx} \tag{6.31}$$

The chain rule extends to more than two functions. If computing the derivative of a composite function $f(x) = u(v(w(x)))$, the derivative of $f(x)$ is:

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dw} \cdot \frac{dw}{dx} \tag{6.32}$$

The intuition of backward differentiation is to pass gradients back from the final node to all the nodes in the graph. Fig. 6.16 shows part of the backward computation at one node e. Each node takes an upstream gradient that is passed in from its parent node to the right, and for each of its inputs computes a local gradient (the gradient of its output with respect to its input), and uses the chain rule to multiply these two to compute a downstream gradient to be passed on to the next earlier node.

Let's now compute the 3 derivatives we need. Since in the computation graph $L = ce$, we can directly compute the derivative $\frac{\partial L}{\partial c}$:

$$\frac{\partial L}{\partial c} = e \tag{6.33}$$

For the other two, we'll need to use the chain rule:

$$\begin{aligned} \frac{\partial L}{\partial a} &= \frac{\partial L}{\partial e}\frac{\partial e}{\partial a} \\ \frac{\partial L}{\partial b} &= \frac{\partial L}{\partial e}\frac{\partial e}{\partial d}\frac{\partial d}{\partial b} \end{aligned} \tag{6.34}$$

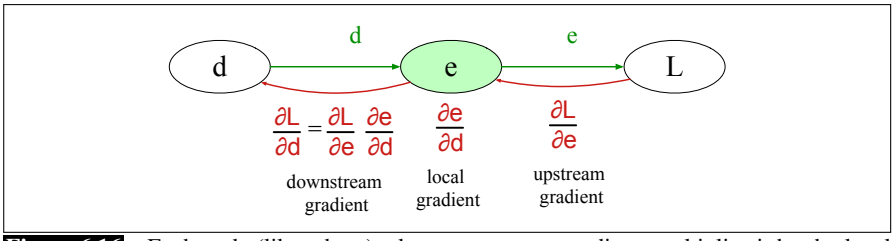


Figure 6.16 Each node (like e here) takes an upstream gradient, multiplies it by the local gradient (the gradient of its output with respect to its input), and uses the chain rule to compute a downstream gradient to be passed on to a prior node. A node may have multiple local gradients if it has multiple inputs.

Eq. 6.34 and Eq. 6.33 thus require five intermediate derivatives: $\frac{\partial L}{\partial e}$, $\frac{\partial L}{\partial c}$, $\frac{\partial e}{\partial a}$, $\frac{\partial e}{\partial d}$, and $\frac{\partial d}{\partial b}$, which are as follows (making use of the fact that the derivative of a sum is the sum of the derivatives):

$$\begin{aligned} L = ce &: \quad \frac{\partial L}{\partial e} = c, \frac{\partial L}{\partial c} = e \\ e = a + d &: \quad \frac{\partial e}{\partial a} = 1, \frac{\partial e}{\partial d} = 1 \\ d = 2b &: \quad \frac{\partial d}{\partial b} = 2 \end{aligned}$$

In the backward pass, we compute each of these partials along each edge of the graph from right to left, using the chain rule just as we did above. Thus we begin by computing the downstream gradients from node L, which are $\frac{\partial L}{\partial e}$ and $\frac{\partial L}{\partial c}$. For node e, we then multiply this upstream gradient $\frac{\partial L}{\partial e}$ by the local gradient (the gradient of the output with respect to the input), $\frac{\partial e}{\partial d}$ to get the output we send back to node d: $\frac{\partial L}{\partial d}$. And so on, until we have annotated the graph all the way to all the input variables. The forward pass conveniently already will have computed the values of the forward intermediate variables we need (like d and e) to compute these derivatives. Fig. 6.17 shows the backward pass.

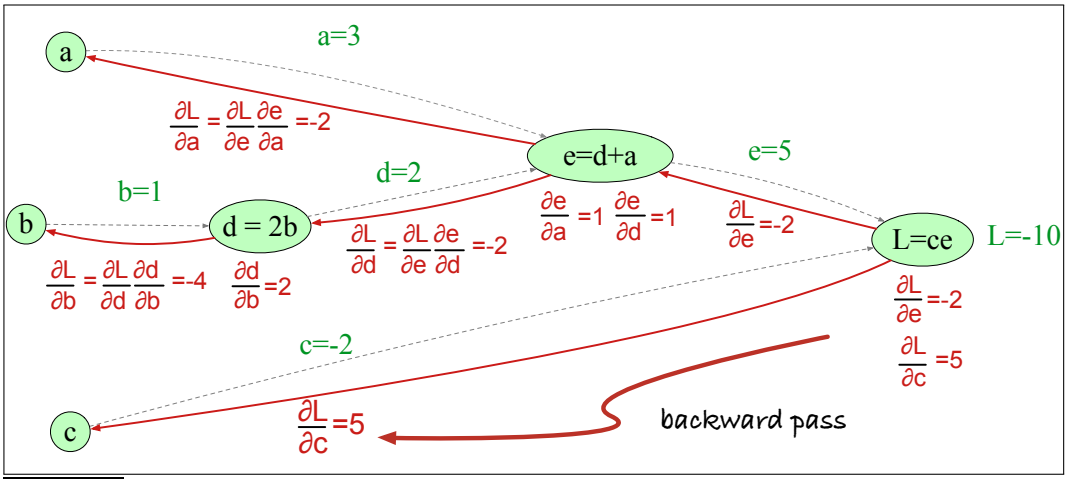


Figure 6.17 Computation graph for the function $L(a,b,c) = c(a+2b)$, showing the backward pass computation of $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, and $\frac{\partial L}{\partial c}$.

Backward differentiation for a neural network

Of course computation graphs for real neural networks are much more complex. Fig. 6.18 shows a sample computation graph for a 2-layer neural network with $n_0 = 2$, $n_1 = 2$, and $n_2 = 1$, assuming binary classification and hence using a sigmoid output unit for simplicity. The function that the computation graph is computing is:

$$\begin{aligned} \mathbf{z}^{[1]} &= \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]} \\ \mathbf{a}^{[1]} &= \text{ReLU}(\mathbf{z}^{[1]}) \\ z^{[2]} &= \mathbf{W}^{[2]}\mathbf{a}^{[1]} + b^{[2]} \\ a^{[2]} &= \sigma(z^{[2]}) \\ \hat{y} &= a^{[2]} \end{aligned} \tag{6.35}$$

For the backward pass we'll also need to compute the loss L. The loss function for binary sigmoid output from Eq. 6.25 is

$$L_{CE}(\hat{y}, y) = -[y \log \hat{y} + (1-y)\log(1-\hat{y})] \tag{6.36}$$

Our output $\hat{y} = a^{[2]}$, so we can rephrase this as

$$L_{CE}(a^{[2]}, y) = -\left[y \log a^{[2]} + (1-y)\log(1-a^{[2]})\right] \tag{6.37}$$

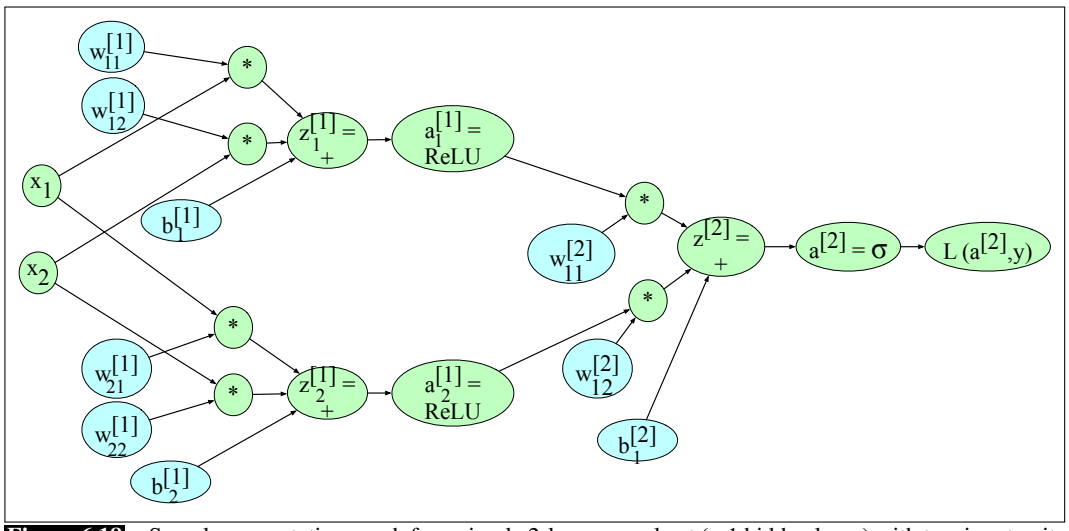


Figure 6.18 Sample computation graph for a simple 2-layer neural net (= 1 hidden layer) with two input units and 2 hidden units. We've adjusted the notation a bit to avoid long equations in the nodes by just mentioning the function that is being computed, and the resulting variable name. Thus the * to the right of node $w_{11}^{[1]}$ means that $w_{11}^{[1]}$ is to be multiplied by x_1, and the node $z^{[1]} = +$ means that the value of $z^{[1]}$ is computed by summing the three nodes that feed into it (the two products, and the bias term $b_i^{[1]}$).

The weights that need updating (those for which we need to know the partial derivative of the loss function) are shown in teal. In order to do the backward pass, we'll need to know the derivatives of all the functions in the graph. We already saw in Section 4.15 the derivative of the sigmoid σ:

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1-\sigma(z)) \tag{6.38}$$

We'll also need the derivatives of each of the other activation functions. The derivative of tanh is:

$$\frac{d\tanh(z)}{dz} = 1 - \tanh^2(z) \tag{6.39}$$

The derivative of the ReLU is[2]

$$\frac{d\,\text{ReLU}(z)}{dz} = \begin{cases} 0 & for \quad z < 0 \\ 1 & for \quad z \geq 0 \end{cases} \tag{6.40}$$

We'll give the start of the computation, computing the derivative of the loss function L with respect to z, or $\frac{\partial L}{\partial z}$ (and leaving the rest of the computation as an exercise for the reader). By the chain rule:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial z} \tag{6.41}$$

So let's first compute $\frac{\partial L}{\partial a^{[2]}}$, taking the derivative of Eq. 6.37, repeated here:

$$\begin{aligned} L_{CE}(a^{[2]}, y) &= -\left[y \log a^{[2]} + (1-y) \log(1-a^{[2]}) \right] \\ \frac{\partial L}{\partial a^{[2]}} &= -\left(\left(y \frac{\partial \log(a^{[2]})}{\partial a^{[2]}} \right) + (1-y) \frac{\partial \log(1-a^{[2]})}{\partial a^{[2]}} \right) \\ &= -\left(\left(y \frac{1}{a^{[2]}} \right) + (1-y) \frac{1}{1-a^{[2]}} (-1) \right) \\ &= -\left(\frac{y}{a^{[2]}} + \frac{y-1}{1-a^{[2]}} \right) \end{aligned} \tag{6.42}$$

Next, by the derivative of the sigmoid:

$$\frac{\partial a^{[2]}}{\partial z} = a^{[2]}(1-a^{[2]})$$

Finally, we can use the chain rule:

$$\begin{aligned} \frac{\partial L}{\partial z} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial z} \\ &= -\left(\frac{y}{a^{[2]}} + \frac{y-1}{1-a^{[2]}} \right) a^{[2]}(1-a^{[2]}) \\ &= a^{[2]} - y \end{aligned} \tag{6.43}$$

Continuing the backward computation of the gradients (next by passing the gradients over $b_1^{[2]}$ and the two product nodes, and so on, back to all the teal nodes), is left as an exercise for the reader.

6.6.5 More details on learning

Optimization in neural networks is a non-convex optimization problem, more complex than for logistic regression, and for that and other reasons there are many best practices for successful learning.

[2] The derivative is actually undefined at the point $z = 0$, but by convention we treat it as 1.

For logistic regression we can initialize gradient descent with all the weights and biases having the value 0. In neural networks, by contrast, we need to initialize the weights with small random numbers. It's also helpful to normalize the input values to have 0 mean and unit variance.

dropout

Various forms of regularization are used to prevent overfitting. One of the most important is **dropout**: randomly dropping some units and their connections from the network during training (Hinton et al. 2012, Srivastava et al. 2014). At each iteration of training (whenever we update parameters, i.e. each mini-batch if we are using mini-batch gradient descent), we repeatedly choose a probability p and for each unit we replace its output with zero with probability p (and renormalize the rest of the outputs from that layer).

hyperparameter

Tuning of **hyperparameters** is also important. The parameters of a neural network are the weights $\mathbf{W}$ and biases $\mathbf{b}$; those are learned by gradient descent. The hyperparameters are things that are chosen by the algorithm designer; optimal values are tuned on a devset rather than by gradient descent learning on the training set. Hyperparameters include the learning rate η, the mini-batch size, the model architecture (the number of layers, the number of hidden nodes per layer, the choice of activation functions), how to regularize, and so on. Gradient descent itself also has many architectural variants such as Adam (Kingma and Ba, 2015).

Finally, most modern neural networks are built using computation graph formalisms that make it easy and natural to do gradient computation and parallelization on vector-based GPUs (Graphic Processing Units). PyTorch (Paszke et al., 2017) and TensorFlow (Abadi et al., 2015) are two of the most popular. The interested reader should consult a neural network textbook for further details; some suggestions are at the end of the chapter.

6.7 Summary

- Neural networks are built out of **neural units**. Originally inspired by biological neurons, neural networks are now an abstract computational device rather than a biological model.
- Each neural unit multiplies input values by a weight vector, adds a bias, and then applies a non-linear activation function like sigmoid, tanh, or rectified linear unit.
- In a **fully-connected**, **feedforward** network, each unit in layer i is connected to each unit in layer $i+1$, and there are no cycles.
- The power of neural networks comes from the ability of early layers to learn representations that can be utilized by later layers in the network.
- Neural networks are trained by optimization algorithms like **gradient descent**.
- **Error backpropagation**, backward differentiation on a **computation graph**, is used to compute the gradients of the loss function for a network.
- **Neural language models** use a neural network as a probabilistic classifier, to compute the probability of the next word given the previous n words.
- Neural language models can use pretrained **embeddings**, or can learn embeddings from scratch in the process of language modeling.

Historical Notes

The origins of neural networks lie in the 1940s **McCulloch-Pitts neuron** (McCulloch and Pitts, 1943), a simplified model of the biological neuron as a kind of computing element that could be described in terms of propositional logic. By the late 1950s and early 1960s, a number of labs (including Frank Rosenblatt at Cornell and Bernard Widrow at Stanford) developed research into neural networks; this phase saw the development of the perceptron (Rosenblatt, 1958), and the transformation of the threshold into a bias, a notation we still use (Widrow and Hoff, 1960).

The field of neural networks declined after it was shown that a single perceptron unit was unable to model functions as simple as XOR (Minsky and Papert, 1969). While some small amount of work continued during the next two decades, a major revival for the field didn't come until the 1980s, when practical tools for building deeper networks like error backpropagation became widespread (Rumelhart et al., 1986). During the 1980s a wide variety of neural network and related architectures were developed, particularly for applications in psychology and cognitive science (Rumelhart and McClelland 1986b, McClelland and Elman 1986, Rumelhart and McClelland 1986a, Elman 1990), for which the term **connectionist** or **parallel distributed processing** was often used (Feldman and Ballard 1982, Smolensky 1988). Many of the principles and techniques developed in this period are foundational to modern work, including the ideas of distributed representations (Hinton, 1986), recurrent networks (Elman, 1990), and the use of tensors for compositionality (Smolensky, 1990).

connectionist

By the 1990s larger neural networks began to be applied to many practical language processing tasks as well, like handwriting recognition (LeCun et al. 1989) and speech recognition (Morgan and Bourlard 1990). By the early 2000s, improvements in computer hardware and advances in optimization and training techniques made it possible to train even larger and deeper networks, leading to the modern term **deep learning** (Hinton et al. 2006, Bengio et al. 2007). We cover more related history in Chapter 13 and Chapter 15.

There are a number of excellent books on neural networks, including Goodfellow et al. (2016) and Nielsen (2015).

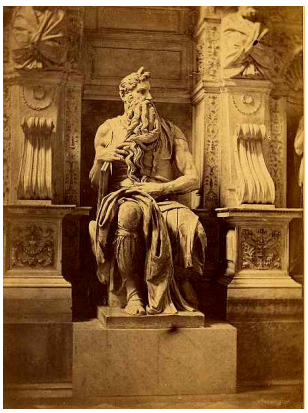
CHAPTER

7 Large Language Models

"How much do we know at any time? Much more, or so I believe, than we know we know."

Agatha Christie, *The Moving Finger*

The literature of the fantastic abounds in inanimate objects magically endowed with the gift of speech. From Ovid's statue of Pygmalion to Mary Shelley's story about Frankenstein, we continually reinvent stories about creating something and then having a chat with it. Legend has it that after finishing his sculpture *Moses*, Michelangelo thought it so lifelike that he tapped it on the knee and commanded it to speak. Perhaps this shouldn't be surprising. Language is the mark of humanity and sentience. conversation is the most fundamental arena of language, the first kind of language we learn as children, and the kind we engage in constantly, whether we are teaching or learning, ordering lunch, or talking with our families or friends.



This chapter introduces the **Large Language Model**, or **LLM**, a computational agent that can interact conversationally with people. The fact that LLMs are designed for interaction with people has strong implications for their design and use.

Many of these implications already became clear in a computational system from 60 years ago, ELIZA (Weizenbaum, 1966). ELIZA, designed to simulate a Rogerian psychologist, illustrates a number of important issues with chatbots. For example people became deeply emotionally involved and conducted very personal conversations, even to the extent of asking Weizenbaum to leave the room while they were typing. These issues of emotional engagement and privacy mean we need to think carefully about how we deploy language models and consider their effect on the people who are interacting with them.

In this chapter we begin by introducing the computational principles of LLMs; we'll discuss their implementation in the transformer architecture in the following chapter. The central new idea that makes LLMs possible is the idea of **pretraining**, so let's begin by thinking about the idea of learning from text, the basic way that LLMs are trained.

We know that fluent speakers of a language bring an enormous amount of knowledge to bear during comprehension and production. This knowledge is embodied in many forms, perhaps most obviously in the vocabulary, the rich representations we have of words and their meanings and usage. This makes the vocabulary a useful lens to explore the acquisition of knowledge from text, by both people and machines.

Estimates of the size of adult vocabularies vary widely both within and across languages. For example, estimates of the vocabulary size of young adult speakers of American English range from 30,000 to 100,000 depending on the resources used

to make the estimate and the definition of what it means to know a word. A simple consequence of these facts is that children have to learn about 7 to 10 words a day, *every single day*, to arrive at observed vocabulary levels by the time they are 20 years of age. And indeed empirical estimates of vocabulary growth in late elementary through high school are consistent with this rate. How do children achieve this rate of vocabulary growth? Research suggests that the bulk of this knowledge acquisition happens as a by-product of reading. Reading is a process of rich contextual processing; we don't learn words one at a time in isolation. In fact, at some points during learning the rate of vocabulary growth exceeds the rate at which new words are appearing to the learner! That suggests that every time we read a word, we are also strengthening our understanding of other words that are associated with it.

Such facts are consistent with the *distributional hypothesis* of Chapter 5, which proposes that some aspects of meaning can be learned solely from the texts we encounter over our lives, based on the complex association of words with the words they co-occur with (and with the words that those words occur with). The distributional hypothesis suggests both that we can acquire remarkable amounts of knowledge from text, and that this knowledge can be brought to bear long after its initial acquisition. Of course, grounding from real-world interaction or other modalities can help build even more powerful models, but even text alone is remarkably useful.

What made the modern NLP revolution possible is that large language models can learn all this knowledge of language, context, and the world simply by being taught to predict the next word, again and again, based on context, in a (very) large corpus of text. In this chapter and the next we formalize this idea that we'll call **pretraining**—learning knowledge about language and the world from iteratively predicting tokens in vast amounts of text—and call the resulting pretrained models **large language models**. Large language models exhibit remarkable performance on natural language tasks because of the knowledge they learn in pretraining.

pretraining

What can language models learn from word prediction? Consider the examples below. What kinds of knowledge do you think the model might pick up from learning to predict what word fills the underbar (the correct answer is shown in blue)? Think about this for each example before you read ahead to the next paragraph:.

With roses, dahlias, and peonies, I was surrounded by _____ flowers
The room wasn't just big it was _____ enormous
The square root of 4 is _____ 2
The author of "A Room of One's Own" is _____ Virginia Woolf
The professor said that _____ he

From the first sentence a model can learn ontological facts like that roses and dahlias and peonies are all kinds of flowers. From the second, a model could learn that "enormous" means something on the same scale as big but further along on the scale. From the third sentence, the system could learn math, while from the 4th sentence facts about the world and historical authors. Finally, the last sentence, if a model was exposed to such sentences repeatedly, it might learn to associate professors only with male pronouns, or other kinds of associations that might cause models to act unfairly to different people.

What is a large language model? As we saw back in Chapter 3, a language model is simply a computational system that can predict the next word from previous words. That is, given a context or prefix of words, a language model assigns a probability distribution over the possible next words. Fig. 7.1 sketches this idea.

Of course we've already seen language models! We saw n-gram language models in Chapter 3 and briefly touched on the feedforward network applied to language

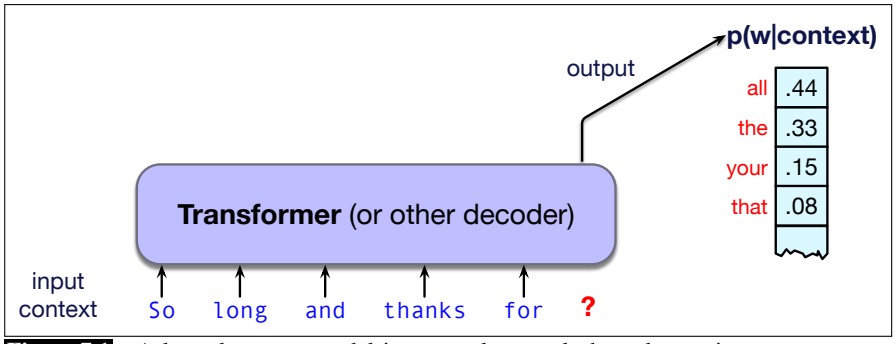


Figure 7.1 A large language model is a neural network that takes as input a context or prefix, and outputs a distribution over possible next words.

modeling in Chapter 6. A large language model is just a (much) larger version of these. For example, in Chapter 3 we introduced bigram and trigram language models that can predict words from the previous word or handful of words. By contrast, large language models can predict words given contexts of thousands or even tens of thousands of words!

The fundamental intuition of language models is that a model that can *predict* text (assigning a distribution over following words) can also be used to *generate* text by **sampling** from the distribution. Recall from Chapter 3 that sampling means to choose a word from a distribution.

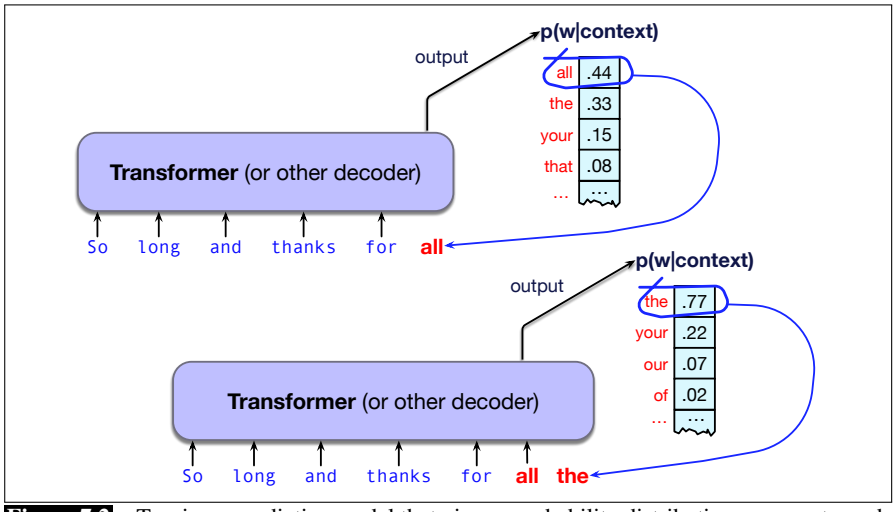


Figure 7.2 Turning a predictive model that gives a probability distribution over next words into a generative model by repeatedly sampling from the distribution. The result is a left-to-right (also called autoregressive) language model. As each token is generated, it gets added onto the context as a prefix for generating the next token.

Fig. 7.2 shows the same example from Fig. 7.1, in which a language model is given a text prefix and generates a possible completion. The model selects the word `all`, adds that to the context, uses the updated context to get a new predictive distribution, and then selects `the` from that distribution and generates it, and so on. Notice that the model is conditioning on both the priming context and its own subsequently generated outputs.

This kind of setting in which we iteratively predict and generate words left-to-

right from earlier words is often called **causal** or **autoregressive** language models. (We will introduce alternative non-autoregressive models, like BERT and other masked language models that predict words using information from both the left and the right, in Chapter 9.)

generative AI

This idea of using computational models to generate text, as well as code, speech, and images, constitutes the important new area called **generative AI**. Applying LLMs to generate text has vastly broadened the scope of NLP, which historically was focused more on algorithms for parsing or understanding text rather than generating it.

In the rest of the chapter, we'll see that almost any NLP task can be modeled as word prediction in a large language model, if we think about it in the right way, and we'll motivate and introduce the idea of **prompting** language models. We'll introduce specific algorithms for generating text from a language model, like **greedy decoding** and **sampling**. We'll introduce the details of **pretraining**, the way that language models are self-trained by iteratively being taught to guess the next word in the text from the prior words. We'll sketch out the other two stages of language model training: instruction tuning (also called supervised finetuning or SFT), and alignment, concepts that we'll return to in Chapter 10. And we'll see how to evaluate these models. Let's begin, though, by talking about different kinds of language models.

7.1 Three architectures for language models

The architecture we sketched above for a left-to-right or autoregressive language model, which is the language model architecture we will define in this chapter, is actually only one of three common LM architectures.

The three architectures are the **encoder**, the **decoder**, and the **encoder-decoder**. Fig. 7.3 gives a schematic picture of the three.

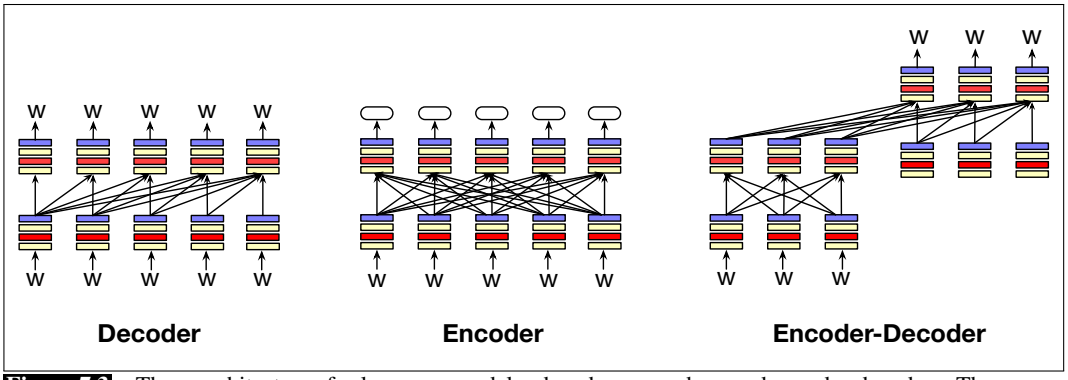


Figure 7.3 Three architectures for language models: decoders, encoders, and encoder-decoders. The arrows sketch out the information flow in the three architectures. Decoders take tokens as input and generate tokens as output. Encoders take tokens as input and produce an encoding (a vector representation of each token) as output. Encoder-decoders take tokens as input and generate a series of tokens as output.

decoder

The **decoder** is the architecture we've introduced above. It takes as input a series of tokens, and iteratively generates an output token one at a time. The decoder is the architecture used to create large language models like GPT, Claude, Llama, and Mistral. The information flow in decoders goes left-to-right, meaning that the model

predicts the next word only from the prior words. Decoders are generative models, meaning that, given input tokens, they generate novel output tokens. We'll discuss decoders in the rest of this chapter and in Chapter 8.

encoder

The **encoder** takes as input a sequence of tokens and outputs a vector representation for each token. Encoders are usually masked language models, meaning they are trained by masking out a word, and learning to predict it by looking at surrounding words on both sides. Masked language models like BERT, RoBERTA, and others in the BERT family are encoder models. Encoder models are not generative models; they aren't used to generate text. Instead encoder models are often used to create classifiers, for example where the input is text and the output is a label, for example for sentiment or topic or other classes. This is done by finetuning them (training them on supervised data). We'll introduce encoder models in Chapter 9.

encoder-decoder

The **encoder-decoder** takes as input a sequence of tokens and outputs a series of tokens. What makes it different than the decoder-only models, is that an encoder-decoder has a much looser relationship between the input tokens and the output tokens, and they are used to map between different kinds of tokens. That is, in an encoder-decoder, the output tokens might be from a very different token-set or be a much longer or shorter sequence than the input token sequence. For example encoder-decoder architectures are used for machine translation, where the input tokens are in one language and the output tokens (probably more or less of them) are in another language. Encoder-decoder architectures are also used for speech recognition, where the input is tokens representing speech, and the output is tokens representing text. We'll introduce the encoder-decoder architecture for machine translation in Chapter 12, and for speech recognition in Chapter 15.

These three architectures can be built out of many kinds of neural networks. The most widely used network type today is the **transformer** that we'll introduce in Chapter 8. In a transformer, each input token is processed by a column of transformer layers, each layer composed of a series of different kinds of subnetworks. In Chapter 13 we'll introduce an earlier architecture that is still relevant, the LSTM, a kind of recurrent neural network. And there are many more recent architectures such as the **state space models**.

We'll focus on transformers for much of this book, but for the purposes of this chapter, we'll describe the LLM decoder in a way that is architecture-agnostic, treating this network as a black box. The input to this black box is a sequence of tokens, and the output of the box is a distribution over tokens that we can sample. And we'll describe architecture-agnostic mechanisms for learning and decoding.

7.2 Conditional Generation of Text: The Intuition

conditional generation

A fundamental intuition underlying language models is that almost anything we want to do with language can be modeled as **conditional generation** of text. (We mean *decoder* language models, which are what we will discuss in this chapter and the next).

Conditional generation is the task of generating text conditioned on an input piece of text. That is, we give the LLM an input piece of text, a **prompt**, and then have the LLM continue generating text token by token, conditioned on the prompt and the subsequently generated tokens. We generate from a model by first computing the probability of the next token w_i from the prior context: $P(w_i|w_{<i})$ and then sampling from that distribution to generate a token.

We'll talk in future sections about all the details, but in this section our goal is just to establish the intuition. How can simply computing the probability of the next token help an LLM do all sorts of different language-related tasks?

Imagine we want to do a classification tasks like sentiment analysis. We can treat this as conditional generation by giving a language model a context like:

`The sentiment of the sentence ‘‘I like Jackie Chan" is:`

and comparing the conditional probability of the following token "positive" and the following token "negative" to see which is higher. That is, as sketched in Fig. 7.4, we compare these two probabilities:

$P($"`positive`"|"`The sentiment of the sentence ‘I like Jackie Chan’ is:`"$)$
$P($"`negative`"|"`The sentiment of the sentence ‘I like Jackie Chan’ is:`"$)$

If the token "`positive`" is more probable, we could say the sentiment of the sentence is positive, otherwise if the token "`negative`" is more probable we say the sentiment is negative.

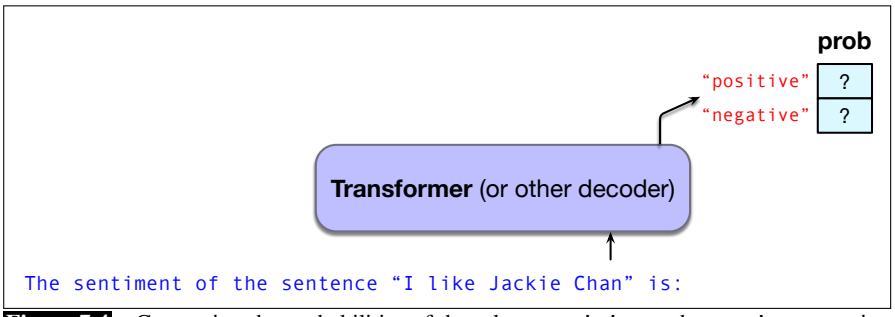


Figure 7.4 Computing the probabilities of the tokens `positive` and `negative` occurring after this prefix.

This same intuition can help us perform a task like question answering, in which the system is given a question and must give a textual answer. We can cast the task of question answering as token prediction by giving a language model a question and a token like `A:` suggesting that an answer should come next, like this:

`Q: Who wrote the book ‘‘The Origin of Species"? A:`

Again, we can ask a language model to compute the probability distribution over possible next tokens given this prefix, computing the following probability

$P(w|$`Q: Who wrote the book ‘‘The Origin of Species"? A:`$)$

and look at which tokens w have high probabilities. As Fig. 7.5 suggests, we might expect to see that `Charles` is very likely, and then if we choose `Charles` and add that to our prefix and compute the probability over tokens with this prefix:

$P(w|$`Q: Who wrote the book ‘‘The Origin of Species"? A: Charles`$)$

we might now see that `Darwin` is the most probable token, and select it.

7.3 Prompting

This simple idea of conditional generation is already very powerful, but becomes more powerful when language models are specially trained to answer questions and

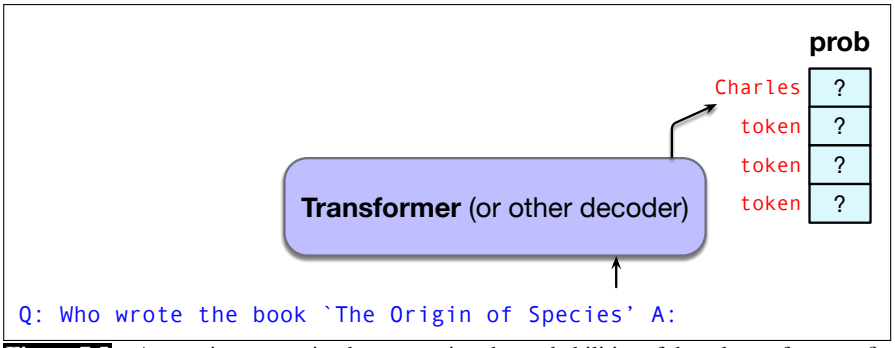


Figure 7.5 Answering a question by computing the probabilities of the tokens after a prefix stating the question; in this example the correct token `Charles` has the highest probability.

follow instructions. This extra training is called **instruction-tuning**. In instruction-tuning we take a base language model that has been trained to predict words, and continue training it on a special dataset of instructions together with the appropriate response to each. The dataset has many examples of questions together with their answers, commands with their responses, and other examples of how to carry on a conversation. We'll discuss the details of instruction-tuning in Chapter 10.

Language models that have been instruction-tuned are very good at following instructions and answering questions and carrying on a conversation and can be **prompted**. A **prompt** is a text string that a user issues to a language model to get the model to do something useful. In prompting, the user's prompt string is passed to the language model, which iteratively generates tokens conditioned on the prompt. The process of finding effective prompts for a task is known as **prompt engineering**.

prompt

prompt engineering

As suggested above when we introduced conditional generation, a prompt can be a question (like "`What is a transformer network?`"), possibly in a structured format (like "`Q: What is a transformer network? A:`"). A prompt can also be an instruction (like "`Translate the following sentence into Hindi: 'Chop the garlic finely'`").

More explicit prompts that specify the set of possible answers lead to better performance. For example, here is a prompt template to do sentiment analysis that prespecifies the potential answers:

A prompt consisting of a review plus an incomplete statement

Human: Do you think that "input" has negative or positive sentiment?
Choices:
(P) Positive
(N) Negative

Assistant: I believe the best answer is: (

This prompt uses a number of more sophisticated prompting characteristics. It specifies the two allowable choices (P) and (N), and ends the prompt with the open parenthesis that strongly suggests the answer will be (P) or (N). Note that it also specifies the role of the language model as an assistant.

Including some labeled examples in the prompt can also improve performance. We call such examples **demonstrations**. The task of prompting with examples is sometimes called **few-shot prompting**, as contrasted with **zero-shot** prompting which means instructions that don't include labeled examples. For example Fig. 7.6

demonstrations

few-shot

zero-shot

shows an example of a question using 2 demonstrations, hence 2-shot prompting. The example is drawn from a computer science question from the MMLU dataset described in Section 7.6 that is often used to evaluate language models.

Example of demonstrations in a computer science question from the MMLU dataset described in Section 7.6

The following are multiple choice questions about high school computer science.

Let x = 1. What is x << 3 in Python 3?
(A) 1 (B) 3 (C) 8 (D) 16
Answer: C

Which is the largest asymptotically?
(A) O(1) (B) O(n) (C) O(n^2) (D) O(log(n))
Answer: C

What is the output of the statement "a" + "ab" in Python 3?
(A) Error (B) aab (C) ab (D) a ab
Answer:

Figure 7.6 Sample 2-shot prompt from MMLU testing high-school computer science. (The correct answer is (B)).

Demonstrations are generally drawn from a labeled training set. They can be selected by hand, or the choice of demonstrations can be optimized by using an optimizer like DSPy (Khattab et al., 2024) to automatically choose the set of demonstrations that most increases task performance of the prompt on a dev set. The number of demonstrations doesn't need to be large; more examples seem to give diminishing returns, and too many examples seems to cause the model to overfit to the exact examples. The primary benefit of demonstrations seems more to demonstrate the task and the format of the output rather than demonstrating the right answers for any particular question. In fact, demonstrations that have incorrect answers can still improve a system (Min et al., 2022; Webson and Pavlick, 2022).

Prompts are a way to get language models to generate text, but prompts can also be viewed as a **learning** signal. This is especially clear when a prompt has demonstrations, since the demonstrations can help language models learn to perform novel tasks from these examples of the new task. This kind of learning is different than pretraining methods for setting language model weights via gradient descent methods that we will describe below. The weights of the model are not updated by prompting; what changes is just the context and the activations in the network.

in-context learning

We therefore call the kind of learning that takes place during prompting **in-context learning**—learning that improves model performance or reduces some loss but does not involve gradient-based updates to the model's underlying parameters.

system prompt

Large language models generally have a **system prompt**, a single text prompt that is the first instruction to the language model, and which defines the task or role for the LM, and sets overall tone and context. The system prompt is silently prepended to any user text. So for example a minimal system prompt that creates a multi-turn assistant conversation might be the following including some special metatokens:

```
<system>You are a helpful and knowledgeable assistant.  Answer
concisely and correctly.
```

So if a user wants to know the capital of France, the actual text used as the language model's context for conditional generation is:

```
<system> You are a helpful and knowledgeable assistant.
Answer concisely and correctly.  <user> What is the capital
of France?
```

The fact that modern language models have such long contexts (tens of thousands of tokens) makes them very powerful for conditional generation, because they can look back so far into the prompting text. That means system prompts, and prompts in general, can be very long.

For example the full system prompt for one language model, Anthropic's Claude Opus4, is 1700 words long and includes sentences like the following:

```
Claude should give concise responses to very simple questions,
but provide thorough responses to complex and open-ended
questions.
Claude is able to explain difficult concepts or ideas clearly.
It can also illustrate its explanations with examples, thought
experiments, or metaphors.
Claude does not provide information that could be used to
make chemical or biological or nuclear weapons
For more casual, emotional, empathetic, or advice-driven
conversations, Claude keeps its tone natural, warm, and
empathetic
Claude cares about people's well-being and avoids encouraging
or facilitating self-destructive behavior
If Claude provides bullet points in its response, it should
use markdown, and each bullet point should be at least 1-2
sentences long unless the human requests otherwise
```

It's also possible to create system prompts for other tasks, like the following prompt for creating a general grammar-checker (Anthropic, 2025):

```
Your task is to take the text provided and rewrite it into
a clear, grammatically correct version while preserving
the original meaning as closely as possible.  Correct any
spelling mistakes, punctuation errors, verb tense issues,
word choice problems, and other grammatical mistakes.
```

Each user can then make a prompt to have the system fix the grammar of a particular piece of text.

In all these cases, the system prompt is prepended to any user prompts or queries, and the entire string is taking as the context for conditional generation by the language model.

7.4 Generation and Sampling

Which tokens should a language model generate at each step?

The generation depends on the probability of each token, so let's remind ourselves where this probability distribution comes from. The internal networks for language models (whether transformers or alternatives like LSTMs or state space models) generate scores called **logits** (real valued numbers) for each token in the vocabulary. This score vector **u** is then normalized by softmax to be a legal probability distribution, just as we saw for logistic regression in Chapter 4. So if we have a logit vector **u** of shape $[1 \times |V|]$ that gives a score for each possible next token, we can pass it through a softmax to get a vector **y**, also of shape $[1 \times |V|]$, which assigns a probability to each token in the vocabulary, as shown in the following equation:

$$\mathbf{y} = \text{softmax}(\mathbf{u}) \tag{7.1}$$

Fig. 7.7 shows an example in which the softmax is computed for pedagogical purposes on a simplified vocabulary of only 4 words.

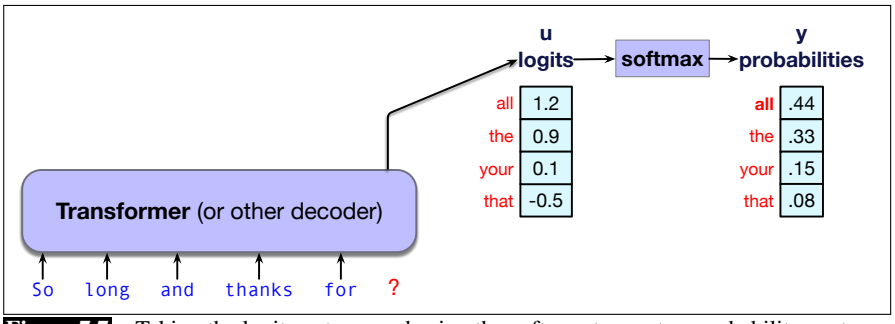


Figure 7.7 Taking the logit vector **u** and using the softmax to create a probability vector **y**.

Now given this probability distribution over tokens, we need to select one token to generate. The task of choosing a token to generate based on the model's probabilities is often called **decoding**. As we mentioned above, decoding from a language model in a left-to-right manner (or right-to-left for languages like Arabic in which we read from right to left), and thus repeatedly choosing the next token conditioned on our previous choices is called **causal** or **autoregressive generation**.[1]

decoding

causal
autoregressive
generation

7.4.1 Greedy decoding

The simplest way to generate tokens is to always generate the most likely token given the context, which is called **greedy decoding**. A **greedy algorithm** is one that makes a choice that is locally optimal, whether or not it will turn out to have been the best choice with hindsight. Thus in greedy decoding, at each time step in generation, we turn the logits into a probability distribution over tokens and then we choose as the output w_t the token in the vocabulary that has the highest probability (the argmax):

greedy
decoding

$$\hat{w}_t = \text{argmax}_{w \in V} P(w|\mathbf{w}_{<t}) \tag{7.2}$$

Fig. 7.8 shows that in our example, the model chooses to generate `all`.

[1] Technically an **autoregressive** model predicts a value at time t based on a linear function of the values at times $t-1$, $t-2$, and so on. Although language models are not linear (since, as we will see, they have many layers of non-linearities), we loosely refer to this generation technique as autoregressive since the token generated at each time step is conditioned on the token selected by the network from the previous step. As we'll see, alternatives like the masked language models of Chapter 9 are non-causal because they can predict tokens based on both past and future tokens.

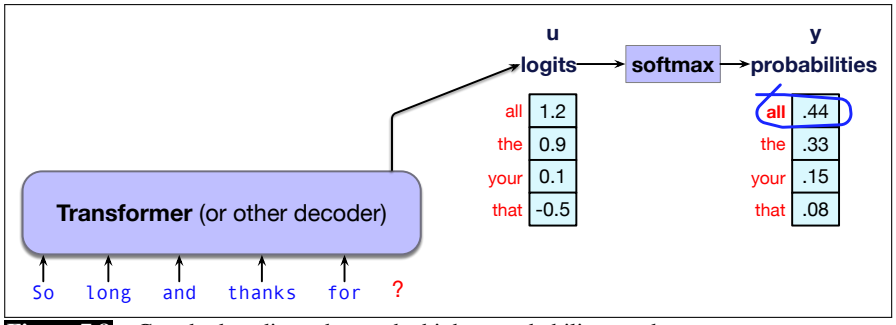


Figure 7.8 Greedy decoding: choose the highest probability word.

In practice, however, we don't use greedy decoding with large language models. A major problem with greedy decoding is that because the tokens it chooses are (by definition) extremely predictable, the resulting text is generic and often quite repetitive. Indeed, greedy decoding is so predictable that it is deterministic; if the context is identical, and the probabilistic model is the same, greedy decoding will always result in generating exactly the same string.

We'll see in Chapter 12 that an extension to greedy decoding called **beam search** works well in tasks like machine translation, which are very constrained in that we are always generating a text in one language conditioned on a very specific text in another language.

In most other tasks, however, people prefer text which has been generated by **sampling methods** that introduce a bit more diversity into the generations.

7.4.2 Random sampling

sampling

Thus the most common method for decoding in large language models involves **sampling**. Recall from Chapter 3 that **sampling** from a distribution means to choose random points according to their likelihood. Thus sampling from a language model—which represents a distribution over following tokens—means to choose the next token to generate according to its probability assigned by the model. Thus we are more likely to generate tokens that the model thinks have a high probability and less likely to generate tokens that the model thinks have a low probability.

That is, we randomly select a token to generate according to its probability in context as defined by the model, generate it, and iterate. We could think of this as rolling a die and choosing a token according to the resulting probability, as we saw in Chapter 3. Such a model is of course more likely to generate the highest probability token, just like the greedy algorithm, but it could also generate any token, just with smaller chances. But in general we are more likely to generate tokens that the model thinks have a high probability in the context and less likely to generate tokens that the model thinks have a low probability.

Sampling from language models was first suggested very early on by Shannon (1948) and Miller and Selfridge (1950), and we saw back in Chapter 3 on page 49 how to generate text from a unigram language model by repeatedly randomly sampling tokens according to their probability until we either reach a pre-determined length or select the end-of-sentence token.

To generate text from a large language model we'll just generalize this model a bit: at each step we'll sample tokens according to their probability *conditioned on our previous choices*, and we'll use the large language model as the probability model that tells us this probability.

random sampling

The algorithm is called **random sampling**, or **random multinomial sampling** (because we are sampling from a multinomial distribution across words). We can formalize random sampling as follows: we are generating a sequence of tokens $\{w_1, w_2, \ldots, w_N\}$ until we hit the end-of-sequence token, using $x \sim p(x)$ to mean 'choose x by sampling from the distribution $p(x)$':

i ← 1
$w_i \sim$ p(w)
while w_i != EOS
  i ← i + 1
  $w_i \sim$ p($w_i \mid w_{<i}$)

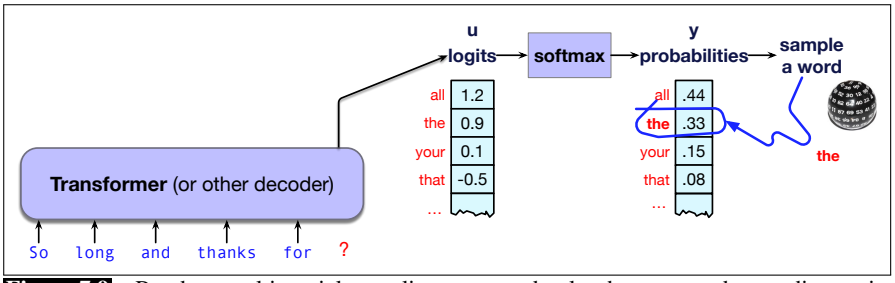


Figure 7.9 Random multinomial sampling: we randomly chose a word according to its probability.

Alas, it turns out random sampling doesn't work well either. The problem is that even though random sampling is mostly going to generate sensible, high-probable tokens, there are many odd, low-probability tokens in the tail of the distribution, and even though each one is low-probability, if you add up all the rare tokens, they constitute a large enough portion of the distribution that they get chosen often enough to result in generating weird sentences.

In other words, greedy decoding is too boring, and random sampling is too random. We need something that doesn't greedily choose the top choice every time, but doesn't stray down too far into the very low-probability events.

There are three standard sampling methods that modify random sampling to address these issues. We'll describe the most common, **temperature** sampling here, and talk about two others (**top-k** and **top-p**) in the next chapter.

7.4.3 Temperature sampling

temperature sampling

The idea of **temperature sampling** is to reshape the probability distribution to increase the probability of the high probability tokens and decrease the probability of the low probability tokens. The result is that we are less likely to generate very low-probability tokens, and more likely to generate tokens that are higher probability.

We implement this intuition by simply dividing the logit by a temperature parameter τ before passing it through the softmax. In low-temperature sampling, $\tau \in (0, 1]$.

Thus instead of computing the probability distribution over the vocabulary directly from the logit as in the following (repeated from Eq. 7.1):

$$\mathbf{y} = \text{softmax}(\mathbf{u}) \tag{7.3}$$

we instead first divide the logits by τ, computing the probability vector y as

$$\mathbf{y} = \text{softmax}(\mathbf{u}/\tau) \tag{7.4}$$

That is, normally we convert from logits to softmax as shown in Fig. 7.10(a). But when we use a temperature parameter we first scale the logit as in Fig. 7.10(b).

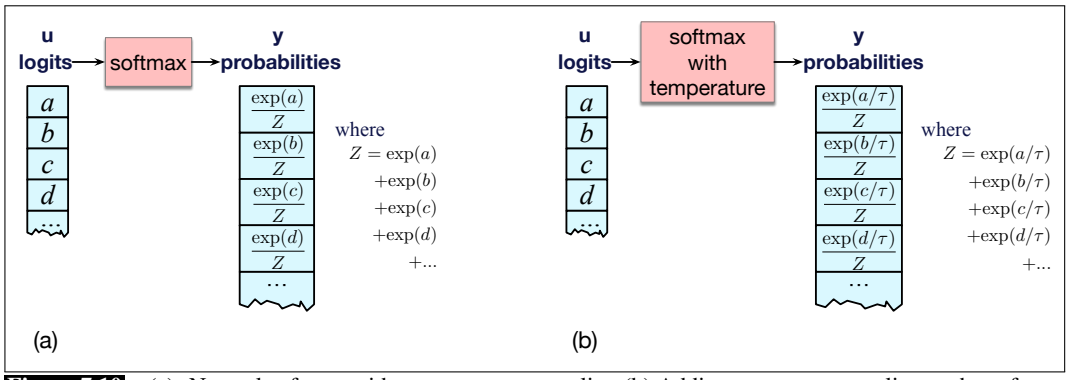


Figure 7.10 (a): Normal softmax without temperature scaling (b) Adding temperature scaling to the softmax by first dividing by the temperature parameter τ.

Why does dividing by τ increase the high probability elements and decrease the low probability elements in the vector over vocabulary items? When τ is 1, we are doing normal softmax, and so when τ is close to 1 the distribution doesn't change much. But the lower τ is, the larger the scores being passed to the softmax (because dividing by a smaller fraction $\tau \leq 1$ results in making each score larger).

Recall that one of the useful properties of a softmax is that it tends to push high values toward 1 and low values toward 0. Thus when larger numbers are passed to a softmax the result is a distribution with increased probabilities of the most high-probability tokens and decreased probabilities of the low probability tokens, making the distribution more greedy. And as τ approaches 0, dividing by τ means the probability of the most likely word approaches 1, resulting in greedy decoding.

The intuition for temperature sampling comes from thermodynamics, where a system at a high temperature is very flexible and can explore many possible states, while a system at a lower temperature is likely to explore a subset of lower energy (better) states. In low-temperature sampling, we smoothly increase the probability of the most probable tokens and decrease the probability of the rare tokens.

Fig. 7.11 shows a schematic example again simplified to have a vocabulary with only 4 tokens (*all*, *the*, *your*, *that*), and showing how different temperature values influence the probabilities computed from the initial logits. $\tau = 1$ is the normal softmax, and we can see how setting $\tau = 0.5$ increases the probability of the top candidate from .45 to .59. Setting $\tau = 0.1$ increases the probability of the top candidate to .95, getting us close to greedy decoding.

We can also see in Fig. 7.11 some other options for situations where we may want to *flatten* the word probability distribution instead of making it greedy. Temperature sampling can help with this situation too, in this case **high-temperature** sampling, in which case we use $\tau > 1$.

7.5 Training Large Language Models

How do we learn a language model? What is the algorithm and what data do we train on?

Language models are trained in three stages, as shown in Fig. 7.12:

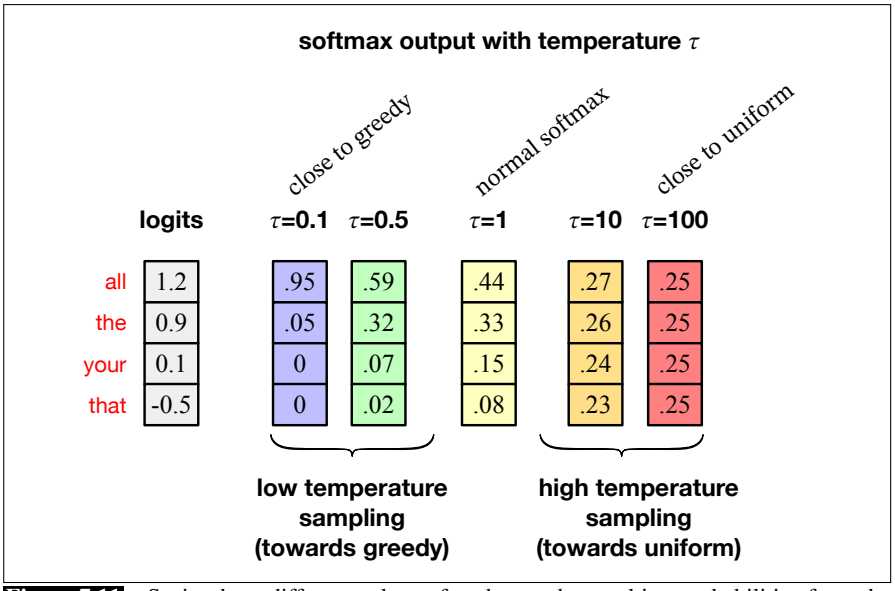


Figure 7.11 Seeing how different values of τ change the resulting probabilities from the initial logits in temperature sampling. In this simplified example, there are only 4 tokens in the vocabulary.

1. **pretraining**: In this first stage, the model is trained to incrementally predict the next word in enormous text corpora. The model uses the cross-entropy loss, sometimes called the **language modeling loss**, and that loss is backpropagated all the way through the network. The training data is usually based on cleaning up parts of the web. The result is a model that is very good at predicting words and can generate text.
2. **instruction tuning**, also called supervised finetuning or **SFT**: In the second stage, the model is trained, again by cross-entropy loss to follow instructions, for example to answer questions, give summaries, write code, translate sentences, and so on. It does this by being trained on a special corpus with lots of text containing both instructions and the correct response to the instruction.
3. **alignment**, also called **preference alignment**. In this final stage, the model is trained to make it maximally helpful and less harmful. Here the model is given preference data, which consists of a context followed by two potential continuations, which are labeled (usually by people) as an 'accepted' vs. a 'rejected' continuation. The model is then trained, by reinforcement learning or other reward-based algorithms, to produce the accepted continuation and not the rejected continuation.

We'll introduce pretraining next, but we'll save instruction tuning and preference alignment for Chapter 10.

7.5.1 Self-supervised training algorithm for pretraining

self-training

The intuition of pretraining large language models, is the same idea of **self-training** or **self-supervision** that we saw in Chapter 5 for learning word representations like word2vec. In self-training for language modeling, we take a corpus of text as training material and at each time step t ask the model to predict the next word. At first it will do poorly at this task, but since in each case we know the correct answer (it's

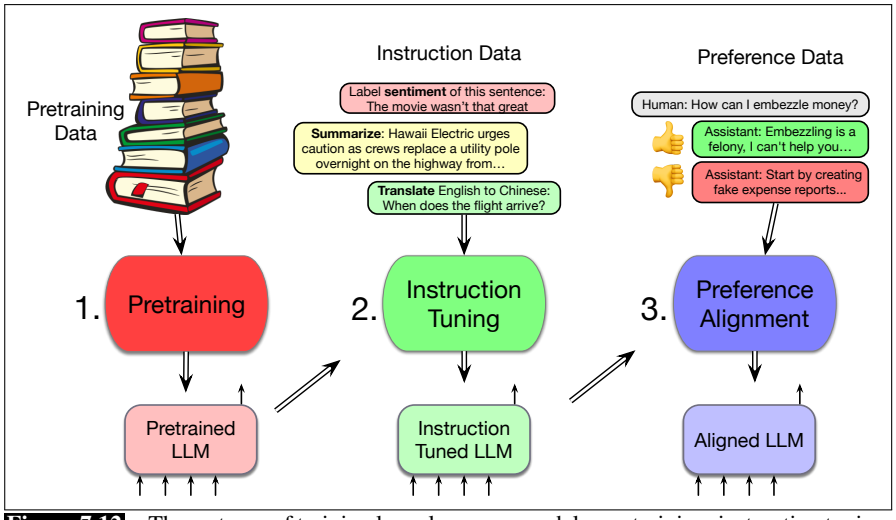


Figure 7.12 Three stages of training large language models: pretraining, instruction tuning, and preference alignment.

the next word in the corpus!) over time it well get better and better at predicting the correct next word. We call such a model self-supervised because we don't have to add any special gold labels to the data; the natural sequence of words is its own supervision! We simply train the model to minimize the error in predicting the true next word in the training sequence.

In practice, training the language model means setting the parameters of the underlying architecture. The transformer that we will introduce in the next chapter has various weight matrices for its feedforward and attention components. Like any other neural architecture, they will be trained by error backpropagation with gradient descent. So all we need is a loss function to minimize and pass back through the network. The loss function we use for language modeling is the cross-entropy loss function we've now seen twice, in Chapter 4 and Chapter 6.

Recall that the cross-entropy loss measures the difference between a predicted probability distribution and the correct distribution. The probability distribution is over the token vocabulary, making the loss be:

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\sum_{w \in V} \mathbf{y}_t[w] \log \hat{\mathbf{y}}_t[w] \tag{7.5}$$

In the case of language modeling, the correct distribution $\mathbf{y}_t$ comes from knowing the next word. This is represented as a one-hot vector corresponding to the vocabulary where the entry for the actual next word is 1, and all the other entries are 0. Thus, the cross-entropy loss for language modeling is determined by the probability the model assigns to the correct next token (all other tokens get multiplied by zero by the first term in Eq. 7.5).

So without loss of generality we can say that at time t the cross-entropy loss in Eq. 7.5 can be simplified as the negative log probability the model assigns to the next word in the training sequence, $-\log p(w_{t+1})$, or more formally, using $\hat{\mathbf{y}}$ to mean the vector of estimated token probabilities from the language model:

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}] \tag{7.6}$$

Thus at each word position t of the input, the model takes as input the correct sequence of tokens $w_{1:t}$, and uses them to compute a probability distribution over

possible next tokens so as to compute the model's loss for the next token w_{t+1}. Then we move to the next word, we ignore what the model predicted for the next word and instead use the correct sequence of tokens $w_{1:t+1}$ to get the model to estimate the probability of token w_{t+2}. This idea that we always give the model the correct history sequence to predict the next word (rather than feeding the model its best guess from the previous time step) is called **teacher forcing**.

teacher forcing

Fig. 7.13 illustrates the general training approach. At each step, given all the preceding tokens, the language model produces an output distribution over the entire vocabulary. During training, the probability assigned to the correct word is used to calculate the cross-entropy loss for each item in the sequence. The loss for each batch is the average cross-entropy loss over the entire sequence of negative log probabilities, or more formally:

$$L_{CE}(\text{batch of length T}) = \frac{1}{T}\sum_{t=1}^{T} -\log \hat{\mathbf{y}}_t[w_{t+1}] \tag{7.7}$$

The weights in the network are then adjusted to minimize this average cross-entropy loss over the batch via gradient descent (Fig. 4.5), using error backpropagation on the computation graph to compute the gradient. Training adjusts all the weights of the network. For the transformer model we will introduce in the next chapter, these weights include the embedding matrix $\mathbf{E}$ that contains the embeddings for each word. Thus embeddings will be learned that are most successful at predicting upcoming words.

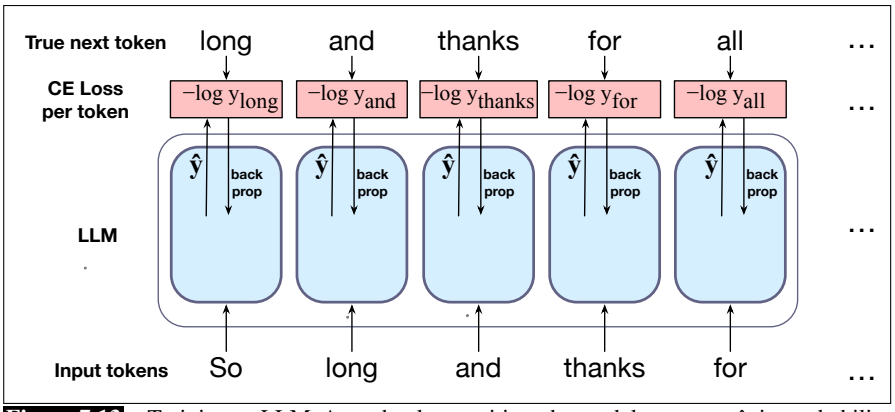


Figure 7.13 Training an LLM. At each token position, the model passes up $\hat{\mathbf{y}}$, its probability estimate for all possible next words. The negative log of the model's probability estimate for the correct token is used as the loss, which is then backpropagated through the model to train all the weights, including the embeddings. Losses are averaged over all the tokens in a batch.

More details of training of course depend on the specific network architecture used to implement the model; we'll see more details specifically for the transformer model in the next chapter.

7.5.2 Pretraining corpora for large language models

Large language models are mainly trained on text scraped from the web, augmented by more carefully curated data. Because these training corpora are so large, they are likely to contain many natural examples that can be helpful for NLP tasks, such as question and answer pairs (for example from FAQ lists), translations of sentences between various languages, documents together with their summaries, and so on.

common crawl

Web text is usually taken from corpora of automatically-crawled web pages like the **common crawl**, a series of snapshots of the entire web produced by the non-profit Common Crawl (https://commoncrawl.org/) that each have billions of webpages. Various versions of common crawl data exist, such as the Colossal Clean Crawled Corpus (C4; Raffel et al. 2020), a corpus of 156 billion tokens of English that is filtered in various ways (deduplicated, removing non-natural language like code, sentences with offensive words from a blocklist). This C4 corpus seems to consist in large part of patent text documents, Wikipedia, and news sites (Dodge et al., 2021).

The Pile

Wikipedia plays a role in lots of language model training, as do corpora of books. **The Pile** (Gao et al., 2020) is an 825 GB English text corpus that is constructed by publicly released code, containing again a large amount of text scraped from the web as well as books and Wikipedia; Fig. 7.14 shows its composition. Dolma is a larger open corpus of English, created with public tools, containing three trillion tokens, which similarly consists of web text, academic papers, code, books, encyclopedic materials, and social media (Soldaini et al., 2024).

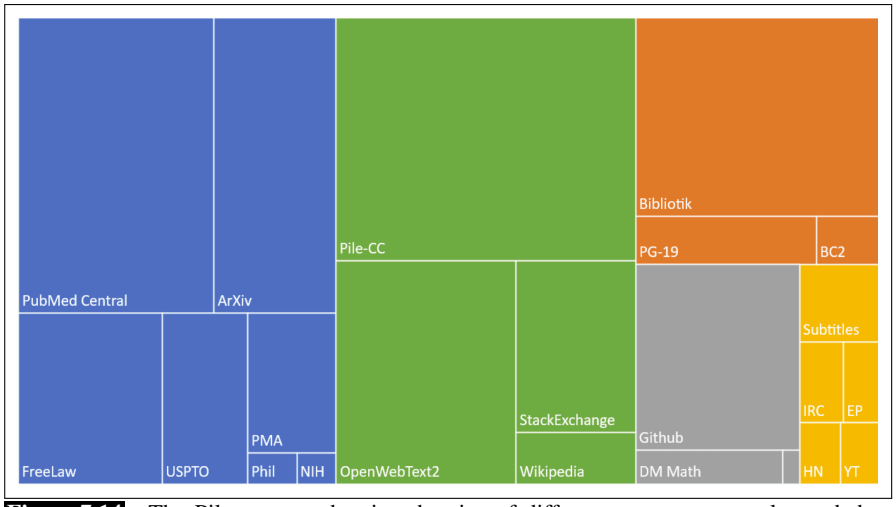


Figure 7.14 The Pile corpus, showing the size of different components, color coded as academic (articles from PubMed and ArXiv, patents from the USPTA; internet (webtext including a subset of the common crawl as well as Wikipedia), prose (a large corpus of books), dialogue (including movie subtitles and chat data), and misc.. Figure from Gao et al. (2020).

Filtering for quality and safety Pretraining data drawn from the web is filtered for both **quality** and **safety**. Quality filters are classifiers that assign a score to each document. Quality is of course subjective, so different quality filters are trained in different ways, but often to value high-quality reference corpora like Wikipedia,

PII

books, and particular websites and to avoid websites with lots of **PII** (Personal Identifiable Information) or adult content. Filters also remove boilerplate text which is very frequent on the web. Another kind of quality filtering is deduplication, which can be done at various levels, so as to remove duplicate documents, duplicate web pages, or duplicate text. Quality filtering generally improves language model performance (Longpre et al., 2024b; Llama Team, 2024).

Safety filtering is again a subjective decision, and often includes **toxicity** detection based on running off-the-shelf toxicity classifiers. This can have mixed results. One problem is that current toxicity classifiers mistakenly flag non-toxic data if it

is generated by speakers of minority dialects like African American English (Xu et al., 2021). Another problem is that models trained on toxicity-filtered data, while somewhat less toxic, are also worse at detecting toxicity themselves (Longpre et al., 2024b). These issues make the question of how to do better safety filtering an important open problem.

Using large datasets scraped from the web to train language models poses ethical and legal questions:

Copyright: Much of the text in these large datasets (like the collections of fiction and non-fiction books) is copyrighted. In some countries, like the United States, the **fair use** doctrine may allow copyrighted content to be used for transformative uses, but it's not clear if that remains true if the language models are used to generate text that competes with the market for the text they are trained on (Henderson et al., 2023).

Data consent: Owners of websites can indicate that they don't want their sites to be crawled by web crawlers (either via a robots.txt file, or via Terms of Service). Recently there has been a sharp increase in the number of websites that have indicated that they don't want large language model builders crawling their sites for training data (Longpre et al., 2024a). Because it's not clear what legal status these indications have in different countries, or whether these restrictions are retroactive, what effect this will have on large pretraining datasets is unclear.

Privacy: Large web datasets also have **privacy** issues since they contain private information like phone numbers and email addresses. While filters are used to try to remove websites likely to contain large amounts of personal information, such filtering isn't sufficient. We'll return to the privacy question in Section 7.7.

Skew: Training data is also disproportionately generated by authors from the US and from developed countries, which likely skews the resulting generation toward the perspectives or topics of this group alone.

7.5.3 Finetuning

Although the vast pretraining data for large language models includes text from many domains, we might want to apply it in a new domain or task that didn't appear sufficiently in the pretraining data. For example, we might want a language model that's specialized to legal or medical text. Or we might have a multilingual language model that knows many languages but might benefit from some more data in our particular language of interest.

In such cases, we can simply continue training the model on relevant data from the new domain or language (Gururangan et al., 2020). This process of taking a fully pretrained model and running additional training passes using the cross-entropy loss on some new data is called **finetuning**. The word "finetuning" means the process of taking a pretrained model and further adapting some or all of its parameters to some new data. Over the next few chapters we'll see a number of different ways that the word 'finetuning' is used, based on exactly which parameters get updated. The method we describe here, in which we just continue to train, as if the new data was at the end of our pretraining data, can also be called **continued pretraining**. Fig. 7.15 sketches the paradigm.

finetuning

continued pretraining

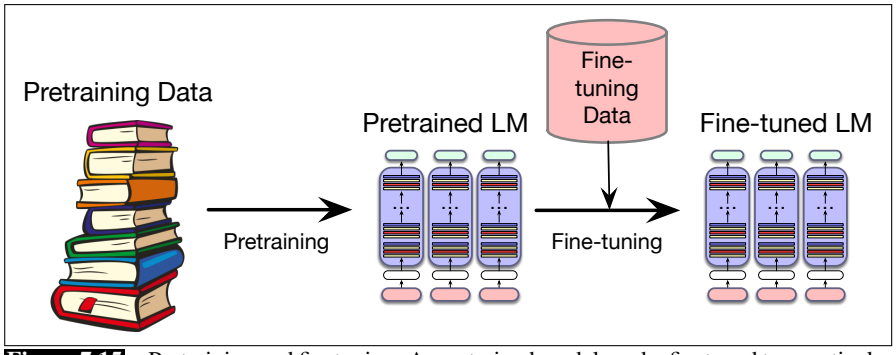


Figure 7.15 Pretraining and finetuning. A pre-trained model can be finetuned to a particular domain or dataset. There are many different ways to finetune, depending on exactly which parameters are updated from the finetuning data: all the parameters, some of the parameters, or only the parameters of specific extra circuitry, as we'll see in future chapters.

7.6 Evaluating Large Language Models

We can evaluate language models by accuracy (how well they predict unseen text, by how well they perform tasks like answering questions or translating text), or by other factors like how fast they can be run, how much energy they use, or how fair they are. We'll explore all of these in the next three sections.

7.6.1 Perplexity

As we first saw in Chapter 3, one way to evaluate language models is to measure how well they predict unseen text. A better language model is better at predicting upcoming words, and so it will be less surprised by (i.e., assign a higher probability to) each word when it occurs in the test set.

If we want to know which of two language models is a better model of some text, we can just see which assigns it a higher probability, or in practice, since we mostly deal with probabilities in log space, we see which assigns a higher log likelihood.

We've been talking about predicting one word at a time, computing the probability of the next token w_i from the prior context: $P(w_i|w_{<i})$. But of course as we saw in Chapter 3 the chain rule allows us to move between computing the probability of the next token and computing the probability of a whole text:

$$\begin{aligned} P(w_{1:n}) &= P(w_1)P(w_2|w_1)P(w_3|w_{1:2})\ldots P(w_n|w_{1:n-1}) \\ &= \prod_{i=1}^{n} P(w_i|w_{<i}) \end{aligned} \tag{7.8}$$

We can compute the probability of text just by multiplying the conditional probabilities for each token in the text. The resulting (log) likelihood of a text is a useful metric for comparing how good two language models are on that text:

$$\log \text{likelihood}(w_{1:n}) = \log \prod_{i=1}^{n} P(w_i|w_{<i}) \tag{7.9}$$

However, we often use another metric other than log likelihood to evaluate language models. The reason is that the probability of a test set (or any sequence) depends on the number of words or tokens in it. In fact, the probability of a test set gets

smaller the longer the text is; this is clear from the chain rule, since if we are multiplying more probabilities, and each probability by definition is less than one, the product will get smaller and smaller. So it's useful to have a metric that is per-token, normalized by length, so we could compare across texts of different lengths.

perplexity

A function of probability called **perplexity** is such a length-normalized metric. Recall from page 46 that the perplexity of a model θ on an unseen test set is the inverse probability that θ assigns to the test set (one over the probability of the test set), normalized by the test set length in tokens. For a test set of n tokens $w_{1:n}$, the perplexity is

$$\begin{aligned} \text{Perplexity}_\theta(w_{1:n}) &= P_\theta(w_{1:n})^{-\frac{1}{n}} \\ &= \sqrt[n]{\frac{1}{P_\theta(w_{1:n})}} \end{aligned} \tag{7.10}$$

To visualize how perplexity can be computed as a function of the probabilities the LM computes for each new word, we can use the chain rule to expand the computation of probability of the test set:

$$\text{Perplexity}_\theta(w_{1:n}) = \sqrt[n]{\prod_{i=1}^{n} \frac{1}{P_\theta(w_i|w_{<i})}} \tag{7.11}$$

Note that because of the inverse in Eq. 7.10, the higher the probability of the word sequence, the lower the perplexity. Thus **the lower the perplexity of a model on the data, the better the model**. Minimizing perplexity is equivalent to maximizing the test set probability according to the language model. Why does perplexity use the inverse probability? The inverse arises from the original definition of perplexity from cross-entropy rate in information theory; for those interested, the explanation is in Section 3.7. Meanwhile, we just have to remember that perplexity has an inverse relationship with probability.

One caveat: because perplexity depends on the number of tokens n in a text, it is very sensitive to differences in the tokenization algorithm. That means that it's hard to exactly compare perplexities produced by two language models if they have very different tokenizers. For this reason perplexity is best used when comparing language models that use the same tokenizer.

7.6.2 Downstream tasks: Reasoning and world knowledge

Perplexity measures one kind of accuracy: accuracy at predicting words. But there are other kinds of accuracy. For each of the downstream tasks we want to apply our language model, like question answering, machine translation, or reasoning, we could measure the accuracy at those tasks. We'll have further discussion of these task-specific evaluations in future chapters; machine translation in Chapter 12, information retrieval in Chapter 11, and speech recognition in Chapter 15.

Here we briefly introduce one such metric: a mechanism for measuring accuracy in answering questions, focusing on multiple-choice questions. This dataset is

MMLU

MMLU (Massive Multitask Language Understanding), a commonly-used dataset of 15,908 knowledge and reasoning questions in 57 areas including medicine, mathematics, computer science, law, and others. Accuracy at answering these multiple-choice questions can be a useful proxy for the model's ability to reason, and its factual knowledge.

For example, here is an MMLU question from the microeconomics domain:[2]

MMLU microeconomics example

One of the reasons that the government discourages and regulates monopolies is that
(A) producer surplus is lost and consumer surplus is gained.
(B) monopoly prices ensure productive efficiency but cost society allocative efficiency.
(C) monopoly firms do not engage in significant research and development.
(D) consumer surplus is lost with higher prices and lower levels of output.

Fig. 7.16 shows the way MMLU turns these questions into prompted tests of a language model, in this case showing an example prompt with 2 demonstrations.

MMLU mathematics prompt

The following are multiple choice questions about high school mathematics.
How many numbers are in the list 25, 26, ..., 100?
(A) 75 (B) 76 (C) 22 (D) 23
Answer: B

Compute $i + i^2 + i^3 + \cdots + i^{258} + i^{259}$.
(A) -1 (B) 1 (C) i (D) -i
Answer: A

If 4 daps = 7 yaps, and 5 yaps = 3 baps, how many daps equal 42 baps?
(A) 28 (B) 21 (C) 40 (D) 30
Answer:

Figure 7.16 Sample 2-shot prompt from MMLU testing high-school mathematics. (The correct answer is (C)).

data contamination

Taking performance on MMLU as a metric for language model quality has a problem, though, one that is true of all evaluations based on public datasets. The problem is **data contamination**. Data contamination is when some part of a dataset that we are testing on (a test set of any kind) makes its way into our training set. For example, since large language models train on the web, and MMLU is on the web, models may well incorporate some MMLU questions into their training. If those questions are used for evaluation, the metric will overstate the performance of the language model. One way to mitigate data contamination is to make available the exact training data used to train a model, or at least to report training overlap with specific test sets (Zhang et al., 2025).

7.6.3 Other factors for evaluating language models

Accuracy isn't the only thing we care about in evaluating models (Dodge et al., 2019; Ethayarajh and Jurafsky, 2020, inter alia). For example, we often care about how big a model is, and how long it takes to train or do inference. We often have limited time, or limited memory, since the GPUs we run our models on have fixed memory

[2] For those of you whose economics is a bit rusty, the correct answer is (D).

sizes. Big models also use more energy, and we prefer models that use less energy, both to reduce the environmental impact of the model and to reduce the financial cost of building or deploying it. We can target our evaluation to these factors by measuring performance normalized to a given compute or memory budget. We can also directly measure the energy usage of our model in kWh or in kilograms of CO_2 emitted (Strubell et al., 2019; Henderson et al., 2020; Liang et al., 2023).

Another feature that a language model evaluation can measure is fairness. We know that language models are biased, exhibiting gendered and racial stereotypes, or decreased performance for language from or about certain demographics groups. There are language model evaluation benchmarks that measure the strength of these biases, such as StereoSet (Nadeem et al., 2021), RealToxicityPrompts (Gehman et al., 2020), and BBQ (Parrish et al., 2022) among many others. We also want language models whose performance is equally fair to different groups. For example, we could choose an evaluation that is fair in a Rawlsian sense by maximizing the welfare of the worst-off group (Rawls, 2001; Hashimoto et al., 2018; Sagawa et al., 2020).

Finally, there are many kinds of leaderboards like Dynabench (Kiela et al., 2021) and general evaluation protocols like HELM (Liang et al., 2023); we will return to these in later chapters when we introduce evaluation metrics for specific tasks like question answering and information retrieval.

7.7 Ethical and Safety Issues with Language Models

Humanists have been thinking about the ethical and safety issues inherent to creating artificial agents since well before we had large language models. You have probably read Mary Shelley's 1818 novel *Frankenstein*, but if not, you should. In the book, which she wrote as a teenager, Shelley describes the hubris and ethical blindness of a scientist who creates an artificial person without considering basic ethical principles. The picture below shows Shelley as painted by Richard Rothwell a decade later at age 30.

Large language models can be unsafe in many ways. For example, LLMs are prone to saying things that are false, a problem called **hallucination**. Language models are trained to generate text that is predictable and coherent, but the training algorithms we have seen so far don't have any way to enforce that the text that is generated is correct or true. This causes enormous problems for any application where the facts matter! A related symptom is that language models can **suggest unsafe actions**, for example directly suggesting that users do dangerous or illegal things like harming themselves or others. If users seek information from language models in safety-critical situations like asking medical advice, or in emergency situations, or when indicating the intentions of self-harm, incorrect advice can be dangerous and even life-threatening. Again, this problem predates large language models. For example (Bickmore et al., 2018) gave partic-

hallucination

ipants medical problems to pose to three pre-LLM commercial dialogue systems (Siri, Alexa, Google Assistant) and asked them to determine an action to take based on the system responses; many of the proposed actions, if actually taken, would have led to harm or death. We'll return to the issue of hallucination and factuality in Chapter 11 where we introduce proposed mitigation methods like **retrieval augmented generation**, and Chapter 10 where we discussed safety tuning and alignment.

sycophantic

Language models are also **sycophantic**, excessively agreeing with or flattering users When a user says something that is factually wrong, language models often agree with them instead of correcting them, an obvious problem for applications in education and health care. Language models can reinforce delusions, and their obsequiousness and flattery can cause users to have distorted views of themselves and the world and increased antisocial behavior (Cheng et al., 2025).

Language models can also harm users by verbally **attacking** them, or creating **representational harms** (Blodgett et al., 2020) for example by generating abusive or harmful stereotypes (Cheng et al., 2023) and negative attitudes (Brown et al., 2020; Sheng et al., 2019) that demean particular groups of people; both abuse and stereotypes can cause psychological harm to users. Gehman et al. (2020) show that even completely non-toxic prompts can lead large language models to output hate speech and abuse their users. Liu et al. (2020) testing how systems responded to pairs of simulated user turns that were identical except for mentioning different genders or race. They found, for example, that simple changes like using the word 'she' instead of 'he' in a sentence caused systems to respond more offensively and with more negative sentiment. Hofmann et al. (2024) found that LLMs were likely to discriminate against people just because they used particular dialects like African-American English. Again, these problems predate large language models. Microsoft's 2016

Tay

Tay chatbot, for example, was taken offline 16 hours after it went live, when it began posting messages with racial slurs, conspiracy theories, and personal attacks on its users. Tay had learned these biases and actions from its training data, including from users who seemed to be purposely teaching the system to repeat this kind of language (Neff and Nagy 2016).

Another important ethical and safety issue is **privacy**. Privacy has been a concern from the very beginning of computing when Weizenbaum designed the chatbot ELIZA as an experiment in computational therapy (Weizenbaum, 1966). First, people became deeply emotionally involved and conducted very personal conversations with the ELIZA chatbot, even to the extent of asking Weizenbaum to leave the room while they were typing. When Weizenbaum suggested that he might want to store the ELIZA conversations, people immediately pointed out that this would violate people's privacy.

Users are likely to give quite personal information to large language models as well, and indeed the most common current LLM use case is for personal advice and support (Zao-Sanders, 2025). And the more human-like a system, the more users are likely to disclose private information, and yet less likely to worry about the harm of this disclosure (Ischen et al., 2019). We discussed above that pretraining data also is likely to have private information like phone numbers and addresses. This is problematic because large language models can **leak** information from their training data. That is, an adversary can extract training-data text from a language model such as a person's name, phone number, and address (Henderson et al. 2017, Carlini et al. 2021). This becomes even more problematic when large language models are trained on extremely sensitive private datasets such as electronic health records.

A related safety issue is **emotional dependence**. Reeves and Nass (1996) show

that people tend to assign human characteristics to computers and interact with them in ways that are typical of human-human interactions. They interpret an utterance in the way they would if it had spoken by a human, (even though they are aware they are talking to a computer). Thus LLMs have had significant influences on people's cognitive and emotional state, leading to problems like emotional dependence on LLMs. These issues (emotional engagement and privacy) mean we need to think carefully about the impact of LLMs on the people who are interacting with them.

In addition to their ability to harm their users in these ways, LLMs may carry out additional harmful activities themselves, especially as agent-based paradigms makes it possible for language models to directly interact with the world.

Language models can also be used by malicious actors for generating text for **fraud**, phishing, propaganda, disinformation campaigns, or other socially harmful activities (Brown et al., 2020). McGuffie and Newhouse (2020) show how large language models generate text that emulates online extremists, with the risk of amplifying extremist movements and their attempt to radicalize and recruit.

And of course we already saw in Section 7.5.2 that many issues with LLM stem from using pretraining corpora scraped from the web, including harms of data consent, potential copyright violation, as well as biases in the training data that can be **amplified** by language models, just as we saw for embedding models in Chapter 5.

amplified

Finding ways to mitigate all these ethical safety issues is an important current research area in NLP. One important step is to carefully analyze the data used to pretrain large language models as a way of understanding safety issues of toxicity, discrimination, privacy, and fair use, making it extremely important that language models include **datasheets** (page 18) or **model cards** (page 90) giving full replicable information on the corpora used to train them. Open-source models can specify their exact training data. There are active areas of research in mitigating problems of abuse and toxicity, like detecting and responding appropriately to toxic contexts (Wolf et al. 2017, Dinan et al. 2020, Xu et al. 2020).

Value sensitive design—carefully considering possible harms in advance (Friedman et al. 2017, Friedman and Hendry 2019)— is also important; (Dinan et al., 2021) give a number of suggestions for best practices in system design. For example getting informed consent from participants, whether they are used for training, or whether they are interacting with a deployed LLM is important. Because studying these interactional properties of LLMs involves human participants, researchers also work on these issues with the Institutional Review Boards (**IRB**) at their institutions, who help protect the safety of experimental participants.

IRB

7.8 Summary

This chapter has introduced the large language model. Here's a summary of the main points that we covered:

- A **large language model** is a system that can predict the next word for previous words given a context or prefix of words, and use this prediction to **conditionally generate** text.
- There are three major architectures for language models: the **encoder**, the **decoder**, and the **encoder-decoder**. The well-known large language models used for generating text are all decoder models; we'll describe encoders in Chapter 9 and encoder-decoders in Chapter 12.

- Many NLP tasks—such as question answering and sentiment analysis— can be cast as tasks of word prediction and addressed with large language models.
- We instruct language models via a **prompt**, a text string that a user issues to a language model to get the model to do something useful by iteratively generating tokens conditioned on the prompt.
- The process of finding effective prompts for a task is known as **prompt engineering**.
- The choice of which word to generate in large language models is done by **sampling** from the distribution of possible next words.
- A common sampling approach is **temperature** sampling, which lies in between **greedy decoding** (always generate the most probable word) and **random sampling** (generate a random word according to its probability).
- Temperature sampling increases the probabilities of the high-probability words, decreases the probability of the low-probability words, and then samples from this new distribution.
- Large language models are pretrained to predict words on datasets of 100s of billions of words generally scraped from the web.
- These datasets need to be filtered for quality.
- The pretraining algorithm relies on cross-entropy loss: minimizing the negative log probability of the true next word.
- Language models are evaluated by **perplexity**, by evaluations of accuracy on proxies for downstream tasks, like the **MMLU** question-answering dataset, and via metrics for other factors like fairness and energy use.
- Language models have numerous ethical and safety issues including hallucinations, unsafe instructions, bias, stereotypes, misinformation and propaganda, and violations of privacy and copyright.

Historical Notes

As we discussed in Chapter 3, the earliest language models were the n-gram language models developed (roughly simultaneously and independently) by Fred Jelinek and colleagues at the IBM Thomas J. Watson Research Center, and James Baker at CMU. It was Jelinek and the IBM team who first coined the term **language model** to mean a model of the way any kind of linguistic property (grammar, semantics, discourse, speaker characteristics), influenced word sequence probabilities (Jelinek et al., 1975). They contrasted the language model with the **acoustic model** which captured acoustic/phonetic characteristics of phone sequences.

N-gram language models were very widely used over the next 40 years, across a wide variety of NLP tasks like speech recognition and machine translation, often as one of multiple components of the model. The contexts for these n-gram models grew longer, with 5-gram models used quite commonly by very efficient LM toolkits (Stolcke, 2002; Heafield, 2011).

The roots of the neural large language model lie in multiple places. One was the application in the 1990s, again in Jelinek's group at IBM Research, of **discriminative classifiers** to language models. Roni Rosenfeld in his dissertation (Rosenfeld, 1992) first applied logistic regression (under the name **maximum entropy** or **maxent** models) to language modeling in that IBM lab, and published a more fully

formed version in Rosenfeld (1996). His model integrated various sorts of information in a logistic regression predictor, including n-gram information along with other features from the context, including distant n-grams and pairs of associated words called **trigger pairs**. Rosenfeld's model prefigured modern language models by being a statistical word predictor trained in a self-supervised manner simply by learning to predict upcoming words in a corpus.

Another was the first use of pretrained embeddings to model word meaning in the LSA/LSI models (Deerwester et al., 1988). Recall from the history section of Chapter 5 that in LSA (latent semantic analysis) a term-document matrix was trained on a corpus and then singular value decomposition was applied and the first 300 dimensions were used as a vector embedding to represent words. It was Landauer et al. (1997) who first used the word "embedding". In addition to their development of the idea of pretraining and of embeddings, the LSA community also developed ways to combine LSA embeddings with n-grams in an integrated language model (Bellegarda, 1997; Coccaro and Jurafsky, 1998).

In a very influential series of papers developing the idea of **neural language models**, (Bengio et al. 2000; Bengio et al. 2003; Bengio et al. 2006), Yoshua Bengio and colleagues drew on the central ideas of both these lines of self-supervised language modeling work (the discriminatively trained word predictor, and the pretrained embeddings). Like the maxent models of Rosenfeld, Bengio's model used the next word in running text as its supervision signal. Like the LSA models, Bengio's model learned an embedding, but unlike the LSA models did it as part of the process of language modeling. The Bengio et al. (2003) model was a neural language model: a neural network that learned to predict the next word from prior words, and did so via learning embeddings as part of the prediction process.

The neural language model was extended in various ways over the years, perhaps most importantly in the form of the RNN language model of Mikolov et al. (2010) and Mikolov et al. (2011). The RNN language model was perhaps the first neural model that was accurate enough to surpass the performance of a traditional 5-gram language model.

Soon afterwards, Mikolov et al. (2013a) and Mikolov et al. (2013b) proposed to simplify the hidden layer of these neural net language models to create pretrained word2vec word embeddings.

The static embedding models like LSA and word2vec instantiated a particular model of pretraining: a representation was trained on a pretraining dataset, and then the representations could be used in further tasks. Dai and Le (2015) and Peters et al. (2018) reframed this idea by proposing models that were pretrained using a language model objective, and then the identical model could be either frozen and directly applied for language modeling or further finetuned still using a language model objective. For example ELMo used a biLSTM self-supervised on a large pretrained dataset using a language model objective, then finetuned on a domain-specific dataset, and then froze the weights and added task-specific heads. The ELMo work was particularly influential and its appearance was perhaps the moment when it became clear to the community that language models could be used as a general solution for NLP problems.

Transformers were first applied as encoder-decoders (Vaswani et al., 2017) and then to masked language modeling (Devlin et al., 2019) (as we'll see in Chapter 12 and Chapter 9). Radford et al. (2019) then showed that the transformer-based autoregressive language model GPT2 could perform zero-shot on many NLP tasks like summarization and question answering.

foundation model

The technology used for language models can also be applied to other domains and tasks, like vision, speech, and genetics. The term **foundation model** is sometimes used as a more general term for this use of large language model technology across domains and areas, when the elements we are computing over are not necessarily words. Bommasani et al. (2021) is a broad survey that sketches the opportunities and risks of foundation models, with special attention to large language models.

CHAPTER

8 Transformers

"The true art of memory is the art of attention"
Samuel Johnson, *Idler #74*, September 1759

In this chapter we introduce the **transformer**, the standard architecture for building large language models. As we discussed in the prior chapter, transformer-based large language models have completely changed the field of speech and language processing. Indeed, every subsequent chapter in this textbook will make use of them. As with the previous chapter, we'll focus for this chapter on the use of transformers to model left-to-right (sometimes called **causal** or autoregressive) language modeling, in which we are given a sequence of input tokens and predict output tokens one by one by conditioning on the prior context.

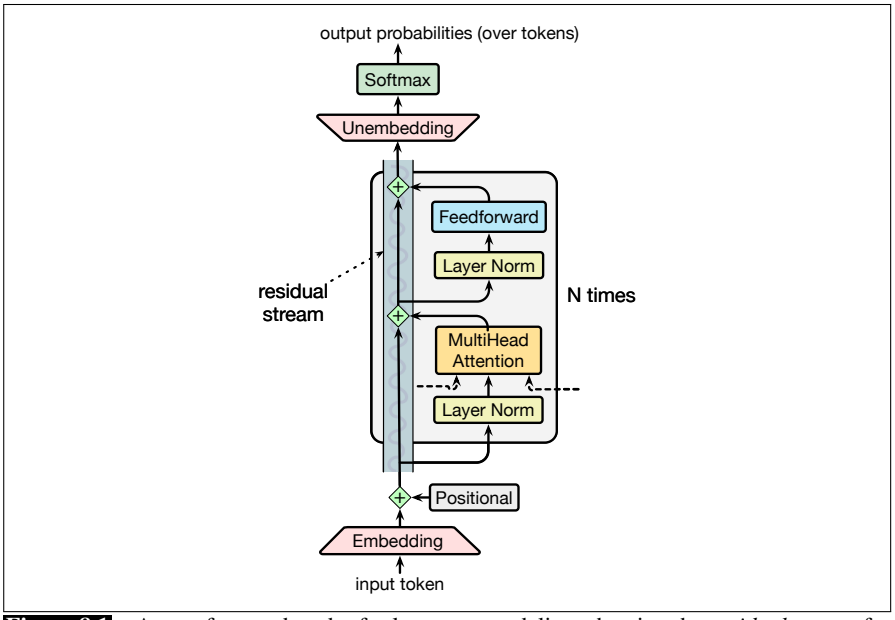


Figure 8.1 A transformer decoder for language modeling, showing the *residual stream* for processing an input token. A single token is embedded and passed forward in the network, with the feedforward and attention components adding information. The multihead attention layer takes inputs (not shown in detail) from the neighboring token streams. This is thus one column of an autoregressive transformer language model, taking an input token and outputting a distribution over next tokens.

Fig. 8.1 sketches the transformer architecture following a single token as it is passes up through the layers of the network. Each token is first converted to an embedding from the **embedding matrix E**. Recall from Chapter 6 in Section 6.5 that **E** is a linear layer that maps a token id to a vector **embedding** representing that token. Each token in the vocabulary has an initial embedding representation in **E**.

Transformers also have a special mechanism for encoding the position/index of the token in the input string, which is simply added to the embedding. The resulting embedding represents both the word and its position. and is then passed through a set of N transformer blocks.

It's common to think of each of these transformer blocks as part of a **stream** in which the input embedding is directly passed up to the output, while simultaneously being enriched by the application of various processing modules: the **multi-head attention** layer, feedforward networks and the layer normalization. The value of the stream at any layer is the sum of the original embedding and all the outputs from all the previous layers and blocks.

The core intuition of the transformer, and the component that distinguishes it from the feedforward layers we saw in Chapter 6, is this multi-head attention layer, also called a **self-attention** layer. Attention can be thought of as a way to build contextual representations of a token's meaning by **attending to** and integrating information from surrounding tokens, helping the model learn how tokens relate to each other over large spans. It can also be thought of as a way to move information from one residual stream to another, augmenting the stream at one token position with information from another token position.

After the N transformer blocks we take the output embedding that is produced by the final transformer block, pass it through an linear **unembedding matrix U** and then a softmax over the vocabulary to generate a distribution over possible next tokens. These last two components (the unembedding matrix and the softmax) are sometimes called the **language modeling head**. In the rest of this chatper we'll introduce attention and the rest of these modules in more detail.

Fig. **??** shows the transformer architecture applied to a context window with the words `So long and thanks for`, showing at each token position what is the most likely token to be generated. In this full figure, the set of N blocks maps an entire **context window** of input vectors $(\mathbf{x}_1, ..., \mathbf{x}_n)$ to a window of output vectors $(\mathbf{h}_1, ..., \mathbf{h}_n)$ of the same length. A column might contain from 12 to 96 or more stacked blocks. The arrows in the figure shows how information from the hidden representations of preceding tokens is incorporated into the transformer block.

Transformer-based language models are complex, and so the details will unfold over this chapter and the next few chapters. Chapter 7 already discussed how language models are **pretrained**, and how tokens are generated via **sampling**. In the rest of this chapter we'll introduce multi-head attention, the rest of the transformer block, and the input encoding and language modeling head components of the transformer. Chapter 9 introduces **masked language modeling** and the **BERT** family of bidirectional transformer encoder models. Chapter 10 shows how to **instruction-tune** language models to perform NLP tasks, and how to **align** the model with human preferences. Chapter 12 will introduce machine translation with the **encoder-decoder** architecture. And we'll see application of the transformer to speech recognition, as well as further use of the encoder-decoder architecture, in Chapter 15.

8.1 Attention

Recall from Chapter 5 that for **word2vec** and other static embeddings, the representation of a word's meaning is always the same vector irrespective of the context: the word `chicken`, for example, is always represented by the same fixed vector. So a static vector for the word `it` might somehow encode that this is a pronoun used

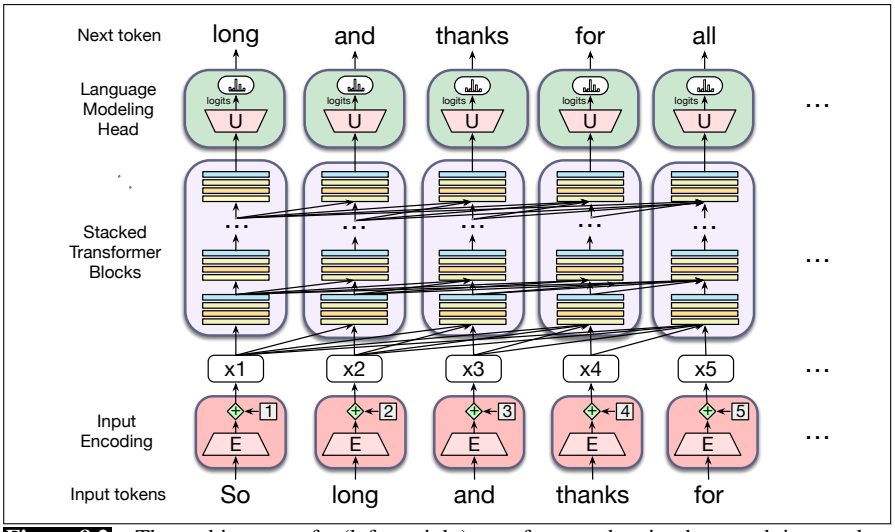


Figure 8.2 The architecture of a (left-to-right) transformer, showing how each input token get encoded, passed through a set of stacked transformer blocks, and then a language model head that predicts the next token. The embeddings at each token position in the residual stream are passed up the stack, and the arrows in the figure shows how information from the hidden representations of preceding tokens are also incorporated.

for animals and inanimate entities. But in context `it` has a much richer meaning. Consider `it` in one of these two sentences:

(8.1) The **chicken** didn't cross the road because **it** was too tired.

(8.2) The chicken didn't cross the **road** because **it** was too wide.

In (8.1) `it` is the chicken (i.e., the reader knows that the chicken was tired), while in (8.2) `it` is the road (and the reader knows that the road was wide).[1] That is, if we are to compute the meaning of this sentence, we'll need the meaning of `it` to be associated with `the chicken` in the first sentence and associated with `the road` in the second one, sensitive to the context.

Furthermore, consider reading left to right like a causal language model, processing the sentence up to the word `it`:

(8.3) The **chicken** didn't cross the **road** because **it**

At this point we don't yet know which thing `it` is going to end up referring to! So a representation of `it` at this point might have aspects of both `chicken` and `road` as the reader is trying to guess what happens next.

This fact that words have rich linguistic relationships with other words that may be far away pervades language. Consider two more examples:

(8.4) The **keys** to the cabinet **are** on the table.

(8.5) I walked along the **pond**, and noticed one of the trees along the **bank**.

In (8.4), the phrase *The keys* is the subject of the sentence, and in English and many languages, must agree in grammatical number with the verb *are*; in this case both are plural. In English we can't use a singular verb like *is* with a plural subject like *keys* (we'll discuss agreement more in Chapter 18). In (8.5), we know that *bank* refers to the side of a pond or river and not a financial institution because of the context, including words like *pond*. (We'll discuss word senses more in Chapter 9.)

[1] We say that in the first example `it` **corefers** with the chicken, and in the second `it` corefers with the road; we'll return to this in Chapter 23.

The point of all these examples is that these contextual words that help us compute the meaning of words in context can be quite far away in the sentence or paragraph. Transformers can build contextual representations of word meaning, **contextual embeddings**, by integrating the meaning of these helpful contextual words. In a transformer, layer by layer, we build up richer and richer contextualized representations of the meanings of input tokens. At each layer, we compute the representation of a token i by combining information about i from the previous layer with information about the neighboring tokens to produce a contextualized representation for each word at each position.

contextual embeddings

Attention is the mechanism in the transformer that weighs and combines the representations from appropriate other tokens in the context from layer k to build the representation for tokens in layer $k+1$.

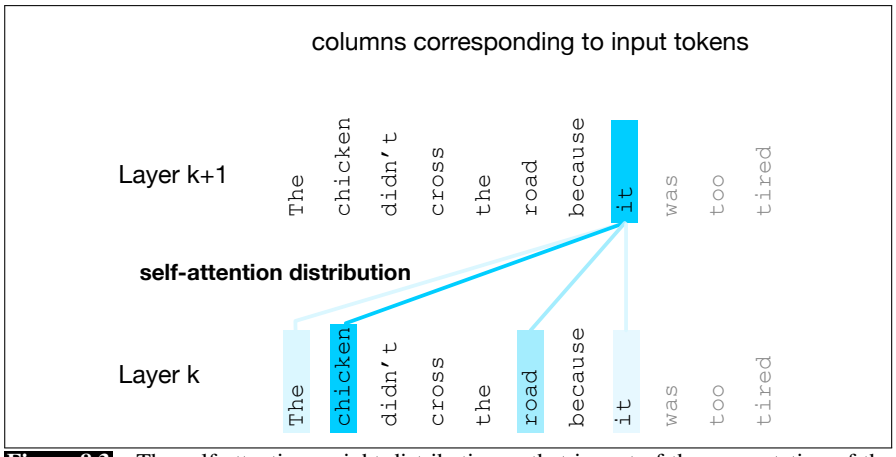


Figure 8.3 The self-attention weight distribution α that is part of the computation of the representation for the word *it* at layer $k+1$. In computing the representation for *it*, we attend differently to the various words at layer k, with darker shades indicating higher self-attention values. Note that the transformer is attending highly to the columns corresponding to the tokens *chicken* and *road* , a sensible result, since at the point where *it* occurs, it could plausibly corefer with the chicken or the road, and hence we'd like the representation for *it* to draw on the representation for these earlier words. Figure adapted from Uszkoreit (2017).

Fig. 8.3 shows a schematic example simplified from a transformer (Uszkoreit, 2017). The figure describes the situation when the current token is `it` and we need to compute a contextual representation for this token at layer $k+1$ of the transformer, drawing on the representations (from layer k) of every prior token. The figure uses color to represent the attention distribution over the contextual words: the tokens `chicken` and `road` both have a high attention weight, meaning that as we are computing the representation for `it`, we will draw most heavily on the representation for `chicken` and `road`. This will be useful in building the final representation for `it`, since `it` will end up coreferring with either `chicken` or `road`.

Let's now turn to how this attention distribution is represented and computed.

8.1.1 Attention more formally

As we've said, the attention computation is a way to compute a vector representation for a token at a particular layer of a transformer, by selectively attending to and integrating information from prior tokens at the previous layer. Attention takes an

input representation $\mathbf{x}_i$ corresponding to the input token at position i, and a context window of prior inputs $\mathbf{x}_1..\mathbf{x}_{i-1}$, and produces an output $\mathbf{a}_i$.

In causal, left-to-right language models, the context is any of the prior words. That is, when processing $\mathbf{x}_i$, the model has access to $\mathbf{x}_i$ as well as the representations of all the prior tokens in the context window (context windows consist of thousands of tokens) but no tokens after i. (By contrast, in Chapter 9 we'll generalize attention so it can also look ahead to future words.)

Fig. 8.4 illustrates this flow of information in an entire causal self-attention layer, in which this same attention computation happens in parallel at each token position i. Thus a self-attention layer maps input sequences $(\mathbf{x}_1,...,\mathbf{x}_n)$ to output sequences of the same length $(\mathbf{a}_1,...,\mathbf{a}_n)$.

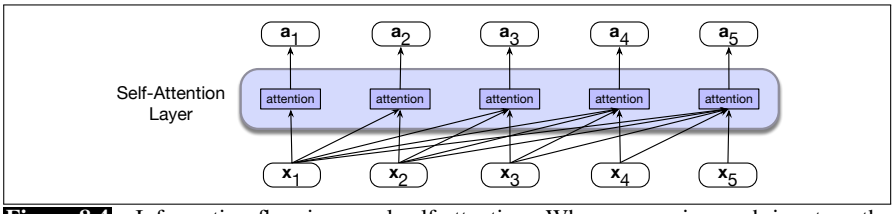


Figure 8.4 Information flow in causal self-attention. When processing each input $\mathbf{x}_i$, the model attends to all the inputs up to, and including $\mathbf{x}_i$.

Simplified version of attention At its heart, attention is really just a weighted sum of context vectors, with a lot of complications added to how the weights are computed and what gets summed. For pedagogical purposes let's first describe a simplified intuition of attention, in which the attention output $\mathbf{a}_i$ at token position i is simply the weighted sum of all the representations $\mathbf{x}_j$, for all $j \leq i$; we'll use α_{ij} to mean how much $\mathbf{x}_j$ should contribute to $\mathbf{a}_i$:

$$\textit{Simplified version:} \quad \mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j \tag{8.6}$$

Each α_{ij} is a scalar used for weighing the value of input $\mathbf{x}_j$ when summing up the inputs to compute $\mathbf{a}_i$. How shall we compute this α weighting? In attention we weight each prior embedding proportionally to how **similar** it is to the current token i. So the output of attention is a sum of the embeddings of prior tokens weighted by their similarity with the current token embedding. We compute similarity scores via **dot product**, which maps two vectors into a scalar value ranging from $-\infty$ to ∞. The larger the score, the more similar the vectors that are being compared. We'll normalize these scores with a softmax to create the vector of weights $\alpha_{ij}, j \leq i$.

$$\textit{Simplified Version:} \quad \text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j \tag{8.7}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \ \forall j \leq i \tag{8.8}$$

Thus in Fig. 8.4 we compute $\mathbf{a}_3$ by computing three scores: $\mathbf{x}_3 \cdot \mathbf{x}_1$, $\mathbf{x}_3 \cdot \mathbf{x}_2$ and $\mathbf{x}_3 \cdot \mathbf{x}_3$, normalizing them by a softmax, and using the resulting probabilities as weights indicating each of their proportional relevance to the current position i. Of course, the softmax weight will likely be highest for $\mathbf{x}_i$, since $\mathbf{x}_i$ is very similar to itself, resulting in a high dot product. But other context words may also be similar to i, and the softmax will also assign some weight to those words. Then we use these weights as the α values in Eq. 8.6 to compute the weighted sum that is our $\mathbf{a}_3$.

The simplified attention in equations 8.6 – 8.8 demonstrates the attention-based approach to computing $\mathbf{a}_i$: compare the $\mathbf{x}_i$ to prior vectors, normalize those scores

into a probability distribution used to weight the sum of the prior vectors. But now we're ready to remove the simplifications.

A single attention head using query, key, and value matrices Now that we've seen a simple intuition of attention, let's introduce the actual **attention head**, the version of attention that's used in transformers. (The word **head** is often used in transformers to refer to specific structured layers). The attention head allows us to distinctly represent three different roles that each input embedding plays during the course of the attention process:

attention head
head

- As *the current element* being compared to the preceding inputs. We'll refer to this role as a **query**.
- In its role as *a preceding input* that is being compared to the current element to determine a similarity weight. We'll refer to this role as a **key**.
- And finally, as a **value** of a preceding element that gets weighted and summed up to compute the output for the current element.

query
key
value

To capture these three different roles, transformers introduce weight matrices $\mathbf{W^Q}$, $\mathbf{W^K}$, and $\mathbf{W^V}$. These weights will project each input vector $\mathbf{x}_i$ into a representation of its role as a query, key, or value:

$$\mathbf{q}_i = \mathbf{x}_i\mathbf{W^Q}; \quad \mathbf{k}_i = \mathbf{x}_i\mathbf{W^K}; \quad \mathbf{v}_i = \mathbf{x}_i\mathbf{W^V} \tag{8.9}$$

Given these projections, when we are computing the similarity of the current element $\mathbf{x}_i$ with some prior element $\mathbf{x}_j$, we'll use the dot product between the current element's **query** vector $\mathbf{q}_i$ and the preceding element's **key** vector $\mathbf{k}_j$. Furthermore, the result of a dot product can be an arbitrarily large (positive or negative) value, and exponentiating large values can lead to numerical issues and loss of gradients during training. To avoid this, we scale the dot product by a factor related to the size of the embeddings, via dividing by the square root of the dimensionality of the query and key vectors (d_k). We thus replace the simplified Eq. 8.7 with Eq. 8.11. The ensuing softmax calculation resulting in α_{ij} remains the same, but the output calculation for $\mathbf{head}_i$ is now based on a weighted sum over the value vectors $\mathbf{v}$ (Eq. 8.13).

Here's a final set of equations for computing self-attention for a single self-attention output vector $\mathbf{a}_i$ from a single input vector $\mathbf{x}_i$. This version of attention computes $\mathbf{a}_i$ by summing the *values* of the prior elements, each weighted by the similarity of its *key* to the *query* from the current element:

$$\mathbf{q}_i = \mathbf{x}_i\mathbf{W^Q}; \quad \mathbf{k}_j = \mathbf{x}_j\mathbf{W^K}; \quad \mathbf{v}_j = \mathbf{x}_j\mathbf{W^V} \tag{8.10}$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \tag{8.11}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \;\; \forall j \leq i \tag{8.12}$$

$$\mathbf{head}_i = \sum_{j \leq i} \alpha_{ij}\mathbf{v}_j \tag{8.13}$$

$$\mathbf{a}_i = \mathbf{head}_i\mathbf{W^O} \tag{8.14}$$

We illustrate this in Fig. 8.5 for the case of calculating the value of the third output $\mathbf{a}_3$ in a sequence.

Note that we've also introduced one more matrix, $\mathbf{W_O}$, which is left-multiplied by the attention head. This is necessary to reshape the output of the head. The input to attention $\mathbf{x_i}$ and the output from attention $\mathbf{a_i}$ both have the same dimensionality $[1 \times d]$. We often call d the **model dimensionality**, and indeed as we'll discuss in

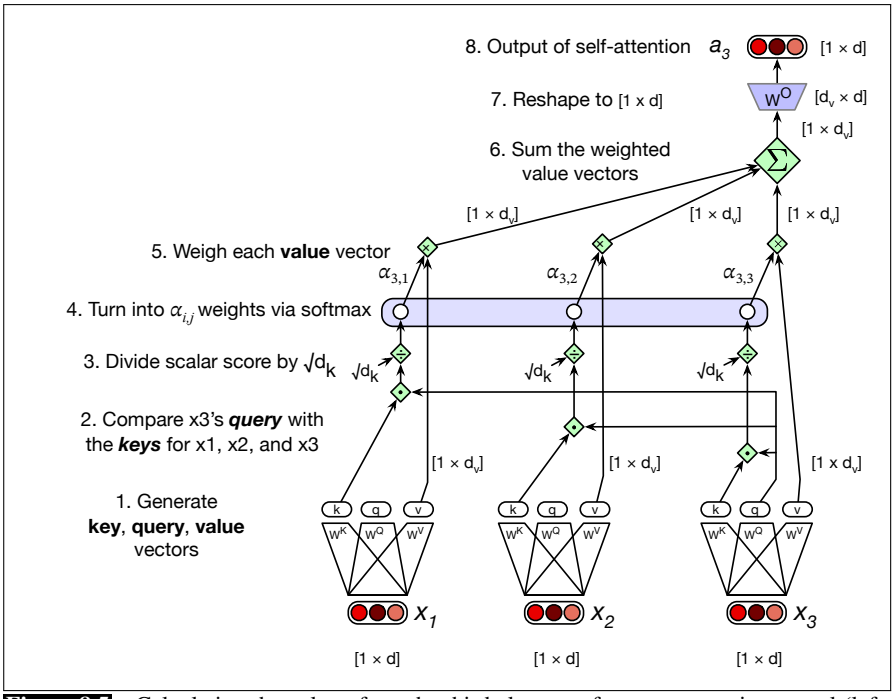


Figure 8.5 Calculating the value of $\mathbf{a}_3$, the third element of a sequence using causal (left-to-right) self-attention.

Section 8.2 the output $\mathbf{h_i}$ of each transformer block, as well as the intermediate vectors inside the transformer block also have the same dimensionality $[1 \times d]$. Having everything be the same dimensionality makes the transformer very modular.

So let's talk shapes. How do we get from $[1 \times d]$ at the input to $[1 \times d]$ at the output? Let's look at all the internal shapes. We'll have a dimension d_k for the query and key vectors. The query vector and the key vector are both dimensionality $[1 \times d_k]$, so we can take their dot product $\mathbf{q}_i \cdot \mathbf{k}_j$ to produce a scalar. We'll have a separate dimension d_v for the value vectors. The transform matrix $\mathbf{W^Q}$ has shape $[d \times d_k]$, $\mathbf{W^K}$ is $[d \times d_k]$, and $\mathbf{W^V}$ is $[d \times d_v]$. So the output of $\mathbf{head}_i$ in equation Eq. 8.13 is of shape $[1 \times d_v]$. To get the desired output shape $[1 \times d]$ we'll need to reshape the head output, and so $\mathbf{W^O}$ is of shape $[d_v \times d]$. In the original transformer work (Vaswani et al., 2017), d was 512, d_k and d_v were both 64.

Multi-head Attention Equations 8.11-8.13 describe a single **attention head**. But actually, transformers use multiple attention heads. The intuition is that each head might be attending to the context for different purposes: heads might be specialized to represent different linguistic relationships between context elements and the current token, or to look for particular kinds of patterns in the context.

multi-head attention

So in **multi-head attention** we have A separate attention heads that reside in parallel layers at the same depth in a model, each with its own set of parameters that allows the head to model different aspects of the relationships among inputs. Thus each head i in a self-attention layer has its own set of query, key, and value matrices: $\mathbf{W^{Qi}}$, $\mathbf{W^{Ki}}$, and $\mathbf{W^{Vi}}$. These are used to project the inputs into separate query, key, and value embeddings for each head.

When using multiple heads the model dimension d is still used for the input and output, the query and key embeddings have dimensionality d_k, and the value embeddings are of dimensionality d_v (again, in the original transformer paper $d_k =$

$d_v = 64$, $A = 8$, and $d = 512$). Thus for each head i, we have weight layers $\mathbf{W^{Qi}}$ of shape $[d \times d_k]$, $\mathbf{W^{Ki}}$ of shape $[d \times d_k]$, and $\mathbf{W^{Vi}}$ of shape $[d \times d_v]$.

Below are the equations for attention augmented with multiple heads; Fig. 8.6 shows an intuition.

$$\mathbf{q}_i^c = \mathbf{x}_i\mathbf{W^{Qc}};\quad \mathbf{k}_j^c = \mathbf{x}_j\mathbf{W^{Kc}};\quad \mathbf{v}_j^c = \mathbf{x}_j\mathbf{W^{Vc}};\quad \forall\, c\quad 1 \le c \le A \tag{8.15}$$

$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k}} \tag{8.16}$$

$$\alpha_{ij}^c = \text{softmax}(\text{score}^c(\mathbf{x}_i, \mathbf{x}_j))\ \ \forall j \le i \tag{8.17}$$

$$\mathbf{head}_i^c = \sum_{j \le i} \alpha_{ij}^c \mathbf{v}_j^c \tag{8.18}$$

$$\mathbf{a}_i = (\mathbf{head}^1 \oplus \mathbf{head}^2 \ldots \oplus \mathbf{head}^A)\mathbf{W}^O \tag{8.19}$$

$$\text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \cdots, \mathbf{x}_{i-1}]) = \mathbf{a}_i \tag{8.20}$$

Note in Eq. 8.20 that MultiHeadAttention is a function of the current input $\mathbf{x}_i$, as well as all the other inputs. For the causal or left-to-right attention that we use in this chapter, the other inputs are only to the left, but we'll also see a version of attention in Chapter 9 where attention is a function of the tokens to the right as well. We'll return to this idea about causal inputs in Eq. 8.34 when we introduce the idea of masking the right context.

The output of each of the A heads is of shape $[1 \times d_v]$, and so the output of the multi-head layer with A heads consists of A vectors of shape $[1 \times d_v]$. These are concatenated to produce a single output with dimensionality $[1 \times Ad_v]$. Then we use yet another linear projection $\mathbf{W^O} \in \mathbb{R}^{Ad_v \times d}$ to reshape it, resulting in the multi-head attention vector $\mathbf{a}_i$ with the correct output shape $[1 \times d]$ at each input i.

8.2 Transformer Blocks

The self-attention calculation lies at the core of what's called a transformer block, which, in addition to the self-attention layer, includes three other kinds of layers: (1) a feedforward layer, (2) residual connections, and (3) normalizing layers (colloquially called "layer norm").

residual stream

Fig. 8.7 illustrates a transformer block, sketching a common way of thinking about the block that is called the **residual stream** (Elhage et al., 2021). In the residual stream viewpoint, we consider the processing of an individual token i through the transformer block as a single stream of d-dimensional representations for token position i. This residual stream starts with the original input vector, and the various components read their input from the residual stream and add their output back into the stream.

The input at the bottom of the stream is an embedding for a token, which has dimensionality d. This initial embedding gets passed up (by **residual connections**), and is progressively added to by the other components of the transformer: the **attention layer** that we have seen, and the **feedforward layer** that we will introduce. Before the attention and feedforward layer is a computation called the **layer norm**.

Thus the initial vector is passed through a layer norm and attention layer, and the result is added back into the stream, in this case to the original input vector $\mathbf{x}_i$. And then this summed vector is again passed through another layer norm and a

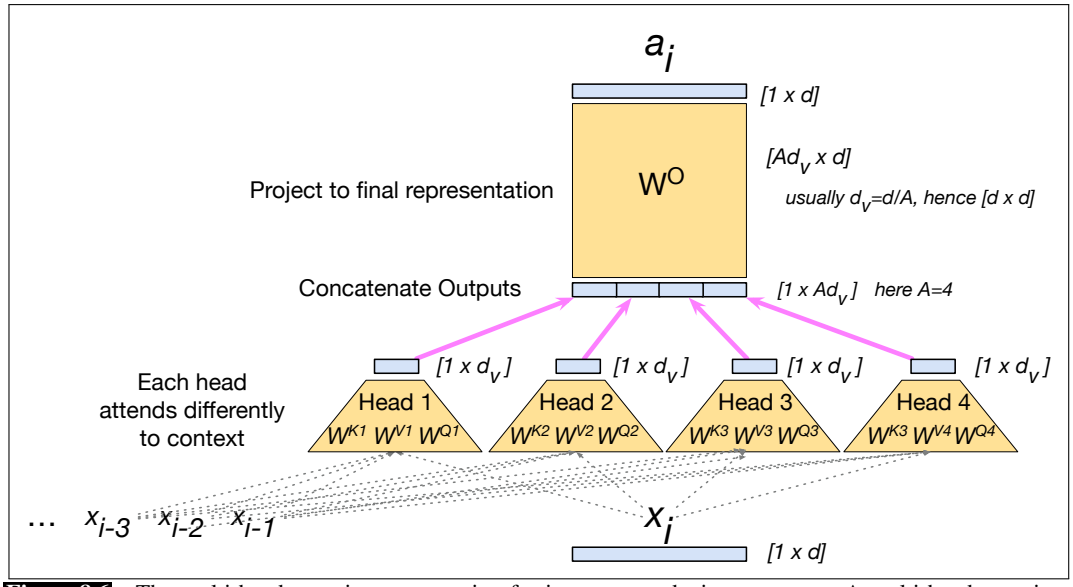


Figure 8.6 The multi-head attention computation for input $\mathbf{x}_i$, producing output $\mathbf{a}_i$. A multi-head attention layer has A heads, each with its own query, key, and value weight matrices. In this figure, we show $A = 4$, a smaller value than is usually used, just to fit on the page. The outputs from each of the heads are of shape $[1 \times d_v]$ and are concatenated and then projected into a different space by the $\mathbf{W_O}$ matrix. Usually the dimensionality d_v of the heads is set so that $d_v = d/A$, with the result that $\mathbf{W_O}$ is a square matrix of shape $[Ad_v \times d] = [d \times d]$. usually of the same size. then projected d, thus producing an output of the same size as the input.

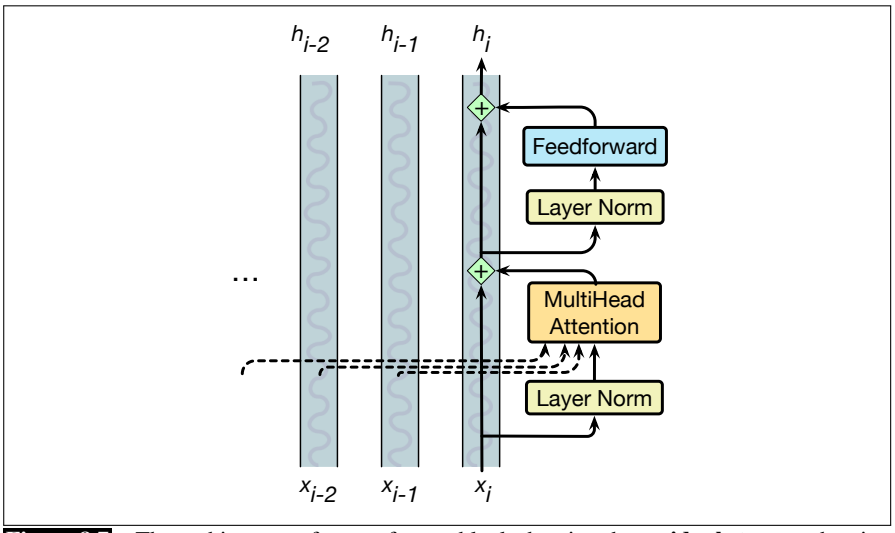


Figure 8.7 The architecture of a transformer block showing the **residual stream**, showing how most information flows up through the residual stream, and only the attention module is sensitive to information from other streams at prior token positions. In this figure and throughout the chapter, we use the **prenorm** version of the architecture, in which the layer norms happen before the attention and feedforward layers rather than after. The first

feedforward layer, and the output of those is added back into the residual, and we'll use $\mathbf{h}_i$ to refer to the resulting output of the transformer block for token i.

We've already seen the attention layer, so let's now introduce the feedforward and layer norm computations in the context of processing a single input $\mathbf{x}_i$ at token

position i.

Feedforward layer The feedforward layer is a fully-connected 2-layer network, i.e., one hidden layer, two weight matrices, as introduced in Chapter 6. The weights are the same for each token position i, but are different from layer to layer. It is common to make the dimensionality d_{ff} of the hidden layer of the feedforward network be larger than the model dimensionality d. (For example in the original transformer model, $d = 512$ and $d_{\text{ff}} = 2048$.)

$$\text{FFN}(\mathbf{x}_i) = \text{ReLU}(\mathbf{x}_i\mathbf{W_1} + b_1)\mathbf{W_2} + b_2 \tag{8.21}$$

layer norm

Layer Norm At two stages in the transformer block we **normalize** the vector (Ba et al., 2016). This process, called **layer norm** (short for layer normalization), is one of many forms of normalization that can be used to improve training performance in deep neural networks by keeping the values of a hidden layer in a range that facilitates gradient-based training.

Layer norm is a variation of the **z-score** from statistics, applied to a single vector in a hidden layer. That is, the term layer norm is a bit confusing; layer norm is **not** applied to an entire transformer layer, but just to the embedding vector of a single token. Thus the input to layer norm is a single vector of dimensionality d and the output is that vector normalized, again of dimensionality d. The first step in layer normalization is to calculate the mean, μ, and standard deviation, σ, over the elements of the vector to be normalized. Given an embedding vector $\mathbf{x}$ of dimensionality d, these values are calculated as follows.

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i \tag{8.22}$$

$$\sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2} \tag{8.23}$$

Given these values, the vector components are normalized by subtracting the mean from each and dividing by the standard deviation. The result of this computation is a new vector with zero mean and a standard deviation of one.

$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma} \tag{8.24}$$

Finally, in the standard implementation of layer normalization, two learnable parameters, γ and β, representing gain and offset values, are introduced.

$$\text{LayerNorm}(\mathbf{x}) = \gamma\frac{(\mathbf{x} - \mu)}{\sigma} + \beta \tag{8.25}$$

Putting it all together The function computed by a transformer block can be expressed by breaking it down with one equation for each component computation, using $\mathbf{t}$ (of shape $[1 \times d]$) to stand for transformer and superscripts to demarcate each computation inside the block:

$$\mathbf{t}_i^1 = \text{LayerNorm}(\mathbf{x}_i) \tag{8.26}$$

$$\mathbf{t}_i^2 = \text{MultiHeadAttention}(\mathbf{t}_i^1, [\mathbf{t}_1^1, \cdots, \mathbf{t}_N^1]) \tag{8.27}$$

$$\mathbf{t}_i^3 = \mathbf{t}_i^2 + \mathbf{x}_i \tag{8.28}$$

$$\mathbf{t}_i^4 = \text{LayerNorm}(\mathbf{t}_i^3) \tag{8.29}$$

$$\mathbf{t}_i^5 = \text{FFN}(\mathbf{t}_i^4) \tag{8.30}$$

$$\mathbf{h}_i = \mathbf{t}_i^5 + \mathbf{t}_i^3 \tag{8.31}$$

Notice that the only component that takes as input information from other tokens (other residual streams) is multi-head attention, which (as we see from Eq. 8.27) looks at all the neighboring tokens in the context. The output from attention, however, is then added into this token's embedding stream. In fact, Elhage et al. (2021) show that we can view attention heads as literally moving information from the residual stream of a neighboring token into the current stream. The high-dimensional embedding space at each position thus contains information about the current token and about neighboring tokens, albeit in different subspaces of the vector space. Fig. 8.8 shows a visualization of this movement. We therefore call the attention function the **token-mixing** component of the architecture, because it mixes information from neighboring token streams into the current stream.

token-mixing

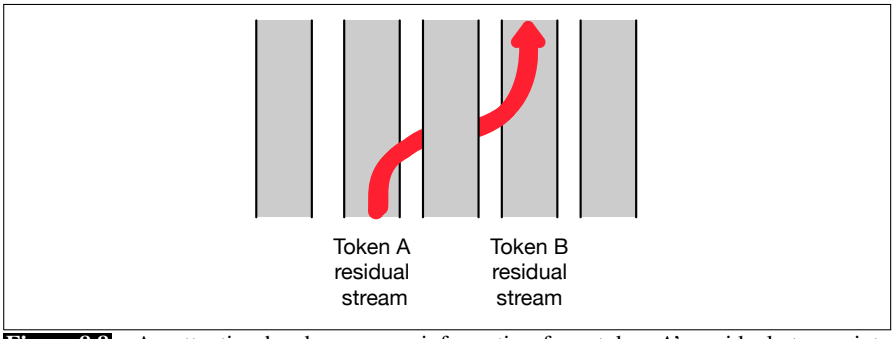


Figure 8.8 An attention head can move information from token A's residual stream into token B's residual stream.

Crucially, the input and output dimensions of transformer blocks are matched so they can be stacked. Each token vector $\mathbf{x}_i$ at the input to the block has dimensionality d, and the output $\mathbf{h}_i$ also has dimensionality d. Transformers for large language models stack many of these blocks, from 12 layers (used for the T5 or GPT-3-small language models) to 96 layers (used for GPT-3 large), to even more for more recent models. We'll come back to this issue of stacking in a bit.

Equation 8.26 and following are just the equation for a single transformer block, but the residual stream metaphor goes through all the transformer layers, from the first transformer blocks to the 12th, in a 12-layer transformer. At the earlier transformer blocks, the residual stream is representing the current token. At the highest transformer blocks, the residual stream is usually representing the following token, since at the very end it's being trained to predict the next token.

Once we stack many blocks, there is one more requirement: at the very end of the last (highest) transformer block, there is a single extra layer norm that is run on the last $\mathbf{h}_i$ of each token stream (just below the language model head layer that we will define soon). [2]

[2] Note that we are using the most common current transformer architecture, which is called the **prenorm** architecture. The original definition of the transformer in Vaswani et al. (2017) used an alternative architecture called the **postnorm** transformer in which the layer norm happens **after** the attention and FFN layers; it turns out moving the layer norm beforehand works better, but does require this one extra layer at the end.

8.3 Parallelizing computation using a single matrix $\mathbf{X}$

This description of multi-head attention and the rest of the transformer block has been from the perspective of computing a single output at a single time step i in a single residual stream. But as we pointed out earlier, the attention computation performed for each token to compute $\mathbf{a}_i$ is independent of the computation for each other token, and that's also true for all the computation in the transformer block computing $\mathbf{h}_i$ from the input $\mathbf{x}_i$. That means we can easily parallelize the entire computation, taking advantage of efficient matrix multiplication routines.

We do this by packing the input embeddings for the N tokens of the input sequence into a single matrix $\mathbf{X}$ of size $[N \times d]$. Each row of $\mathbf{X}$ is the embedding of one token of the input. Transformers for large language models commonly have an input length N from 1K to 32K; much longer contexts of 128K or even up to millions of tokens can also be achieved with architectural changes like special long-context mechanisms that we don't discuss here. So for vanilla transformers, we can think of $\mathbf{X}$ having between 1K and 32K rows, each of the dimensionality of the embedding d (the model dimension).

Parallelizing attention Let's first see this for a single attention head and then turn to multiple heads, and then add in the rest of the components in the transformer block. For one head we multiply $\mathbf{X}$ by the query, key, and value matrices $\mathbf{W}^{\mathbf{Q}}$ of shape $[d \times d_k]$, $\mathbf{W}^{\mathbf{K}}$ of shape $[d \times d_k]$, and $\mathbf{W}^{\mathbf{V}}$ of shape $[d \times d_v]$, to produce matrices $\mathbf{Q}$ of shape $[N \times d_k]$, $\mathbf{K}$ of shape $[N \times d_k]$, and $\mathbf{V}$ of shape $[N \times d_v]$, containing all the key, query, and value vectors:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{\mathbf{Q}}; \quad \mathbf{K} = \mathbf{X}\mathbf{W}^{\mathbf{K}}; \quad \mathbf{V} = \mathbf{X}\mathbf{W}^{\mathbf{V}} \tag{8.32}$$

Given these matrices we can compute all the requisite query-key comparisons simultaneously by multiplying $\mathbf{Q}$ and $\mathbf{K}^\intercal$ in a single matrix multiplication. The product is of shape $N \times N$, visualized in Fig. 8.9.

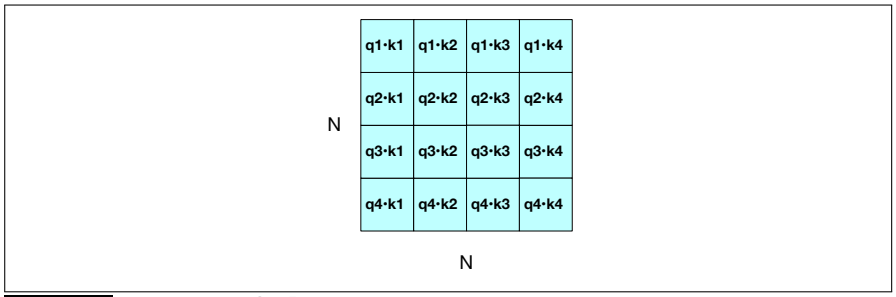


Figure 8.9 The $N \times N$ $\mathbf{QK}^\intercal$ matrix showing how it computes all $q_i \cdot k_j$ comparisons in a single matrix multiple.

Once we have this $\mathbf{QK}^\intercal$ matrix, we can very efficiently scale these scores, take the softmax, and then multiply the result by $\mathbf{V}$ resulting in a matrix of shape $N \times d$: a vector embedding representation for each token in the input. We've reduced the entire self-attention step for an entire sequence of N tokens for one head to the following computation:

$$\mathbf{head} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{QK}^\intercal}{\sqrt{d_k}}\right)\right)\mathbf{V} \tag{8.33}$$

$$\mathbf{A} = \mathbf{head}\,\mathbf{W}^{\mathbf{O}} \tag{8.34}$$

Masking out the future You may have noticed that we introduced a mask function in Eq. 8.34 above. This is because the self-attention computation as we've described it has a problem: the calculation of $\mathbf{QK}^\intercal$ results in a score for each query value to every key value, *including those that follow the query*. This is inappropriate in the setting of language modeling: guessing the next word is pretty simple if you already know it! To fix this, the elements in the upper-triangular portion of the matrix are set to $-\infty$, which the softmax will turn to zero, thus eliminating any knowledge of words that follow in the sequence. This is done in practice by adding a mask matrix M in which $M_{ij} = -\infty \, \forall j > i$ (i.e. for the upper-triangular portion) and $M_{ij} = 0$ otherwise. Fig. 8.10 shows the resulting masked $\mathbf{QK}^\intercal$ matrix. (we'll see in Chapter 9 how to make use of words in the future for tasks that need it).

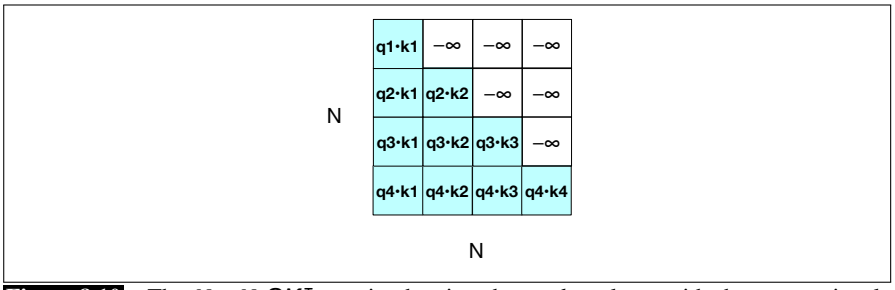


Figure 8.10 The $N \times N$ $\mathbf{QK}^\intercal$ matrix showing the $q_i \cdot k_j$ values, with the upper-triangle portion of the comparisons matrix zeroed out (set to $-\infty$, which the softmax will turn to zero).

Fig. 8.11 shows a schematic of all the computations for a single attention head parallelized in matrix form.

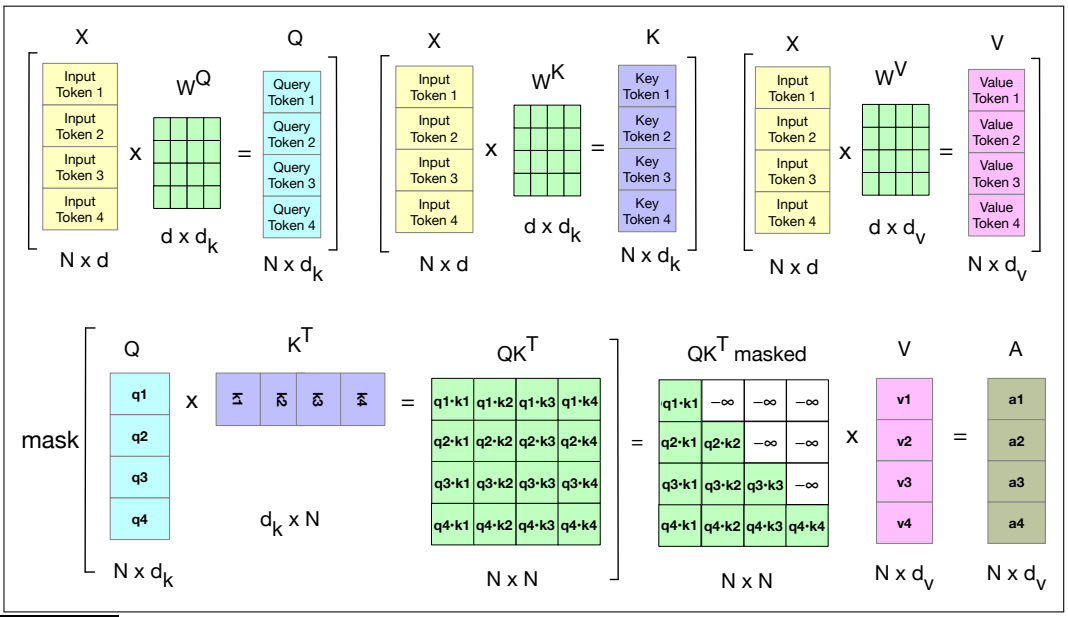


Figure 8.11 Schematic of the attention computation for a single attention head in parallel. The first row shows the computation of the $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ matrices. The second row shows the computation of $\mathbf{QK}^\intercal$, the masking (the softmax computation and the normalizing by dimensionality are not shown) and then the weighted sum of the value vectors to get the final attention vectors.

Fig. 8.9 and Fig. 8.10 also make it clear that attention is quadratic in the length of the input, since at each layer we need to compute dot products between each pair of tokens in the input. This makes it expensive to compute attention over very long documents (like entire novels). Nonetheless modern large language models manage to use quite long contexts of thousands or tens of thousands of tokens.

Parallelizing multi-head attention In multi-head attention, as with self-attention, the input and output have the model dimension d, the key and query embeddings have dimensionality d_k, and the value embeddings are of dimensionality d_v (again, in the original transformer paper $d_k = d_v = 64$, $A = 8$, and $d = 512$). Thus for each head c, we have weight layers $\mathbf{W^Q}_c$ of shape $[d \times d_k]$, $\mathbf{W^K}_c$ of shape $[d \times d_k]$, and $\mathbf{W^V}_c$ of shape $[d \times d_v]$, and these get multiplied by the inputs packed into $\mathbf{X}$ to produce $\mathbf{Q}$ of shape $[N \times d_k]$, $\mathbf{K}$ of shape $[N \times d_k]$, and $\mathbf{V}$ of shape $[N \times d_v]$. The output of each of the A heads is of shape $[N \times d_v]$, and so the output of the multi-head layer with A heads consists of A matrices of shape $[N \times d_v]$. To make use of these matrices in further processing, they are concatenated to produce a single output with dimensionality $[N \times Ad_v]$. Finally, we use a final linear projection $\mathbf{W^O}$ of shape $[Ad_v \times d]$, that reshapes it to the original output dimension for each token. Multiplying the concatenated $[N \times Ad_v]$ matrix output by $\mathbf{W^O}$ of shape $[Ad_v \times d]$ yields the self-attention output $\mathbf{A}$ of shape $[N \times d]$.

$$\mathbf{Q^i} = \mathbf{X}\mathbf{W^{Qi}}; \quad \mathbf{K^i} = \mathbf{X}\mathbf{W^{Ki}}; \quad \mathbf{V^i} = \mathbf{X}\mathbf{W^{Vi}} \tag{8.35}$$

$$\mathbf{head}_i = \text{SelfAttention}(\mathbf{Q^i}, \mathbf{K^i}, \mathbf{V^i}) = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{Q^i}\mathbf{K^{i\intercal}}}{\sqrt{d_k}}\right)\right)\mathbf{V^i} \tag{8.36}$$

$$\text{MultiHeadAttention}(\mathbf{X}) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \ldots \oplus \mathbf{head}_A)\mathbf{W^O} \tag{8.37}$$

Putting it all together with the parallel input matrix X The function computed in parallel by an entire layer of N transformer blocks—each block over one of the N input tokens—can be expressed as:

$$\mathbf{O} = \mathbf{X} + \text{MultiHeadAttention}(\text{LayerNorm}(\mathbf{X})) \tag{8.38}$$

$$\mathbf{H} = \mathbf{O} + \text{FFN}(\text{LayerNorm}(\mathbf{O})) \tag{8.39}$$

Note that in Eq. 8.38 we are using $\mathbf{X}$ to mean the input to the layer, wherever it comes from. For the first layer, as we will see in the next section, that input is the initial word + positional embedding vectors that we have been describing by $\mathbf{X}$. But for subsequent layers k, the input is the output from the previous layer $\mathbf{H}^{k-1}$. We can also break down the computation performed in a transformer layer, showing one equation for each component computation. We'll use $\mathbf{T}$ (of shape $[N \times d]$) to stand for transformer and superscripts to demarcate each computation inside the block, and again use $\mathbf{X}$ to mean the input to the block from the previous layer or the initial embedding:

$$\mathbf{T^1} = \text{LayerNorm}(\mathbf{X}) \tag{8.40}$$

$$\mathbf{T^2} = \text{MultiHeadAttention}(\mathbf{T^1}) \tag{8.41}$$

$$\mathbf{T^3} = \mathbf{T^2} + \mathbf{X} \tag{8.42}$$

$$\mathbf{T^4} = \text{LayerNorm}(\mathbf{T^3}) \tag{8.43}$$

$$\mathbf{T^5} = \text{FFN}(\mathbf{T^4}) \tag{8.44}$$

$$\mathbf{H} = \mathbf{T^5} + \mathbf{T^3} \tag{8.45}$$

Here when we use a notation like $\text{FFN}(\mathbf{T^3})$ we mean that the same FFN is applied in parallel to each of the N embedding vectors in the window. Similarly, each of the

N tokens is normed in parallel in the LayerNorm. Crucially, the input and output dimensions of transformer blocks are matched so they can be stacked. Since each token x_i at the input to the block is represented by an embedding of dimensionality $[1 \times d]$, that means the input $\mathbf{X}$ and output $\mathbf{H}$ are both of shape $[N \times d]$.

8.4 The input: embeddings for token and position

embedding

Let's talk about where the input $\mathbf{X}$ comes from. Given a sequence of N tokens (N is the context length in tokens), the matrix $\mathbf{X}$ of shape $[N \times d]$ has an **embedding** for each word in the context. The transformer does this by separately computing two embeddings: an input token embedding, and an input positional embedding.

A token embedding, introduced in Chapter 6, is a vector of dimension d that will be our initial representation for the input token. (As we pass vectors up through the transformer layers in the residual stream, this embedding representation will change and grow, incorporating context and playing a different role depending on the kind of language model we are building.) The set of initial embeddings are stored in the embedding matrix $\mathbf{E}$, which has a row for each of the $|V|$ tokens in the vocabulary. (Reminder that V here means the vocabulary of tokens, this V is not related to the value vector.) Thus each word is a row vector of d dimensions, and $\mathbf{E}$ has shape $[|V| \times d]$.

Given an input token string like *Thanks for all the* we first convert the tokens into vocabulary indices (these were created when we first tokenized the input using BPE or SentencePiece). So the representation of *thanks for all the* might be $\mathbf{w} = [5, 4000, 10532, 2224]$. Next we use indexing to select the corresponding rows from $\mathbf{E}$, (row 5, row 4000, row 10532, row 2224).

one-hot vector

Another way to think about selecting token embeddings from the embedding matrix is to represent tokens as one-hot vectors of shape $[1 \times |V|]$, i.e., with one dimension for each word in the vocabulary. Recall that in a **one-hot vector** all the elements are 0 except one, the element whose dimension is the word's index in the vocabulary, which has value 1. So if the word "thanks" has index 5 in the vocabulary, $x_5 = 1$, and $x_i = 0 \ \forall i \neq 5$, as shown here:

```
[0 0 0 0 1 0 0 ... 0 0 0 0]
 1 2 3 4 5 6 7 ...  ... |V|
```

Multiplying by a one-hot vector that has only one non-zero element $x_i = 1$ simply selects out the relevant row vector for word i, resulting in the embedding for word i, as depicted in Fig. 8.12.

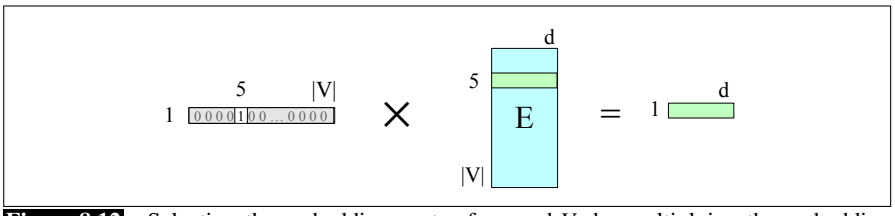


Figure 8.12 Selecting the embedding vector for word V_5 by multiplying the embedding matrix $\mathbf{E}$ with a one-hot vector with a 1 in index 5.

We can extend this idea to represent the entire token sequence as a matrix of one-hot vectors, one for each of the N positions in the transformer's context window, as shown in Fig. 8.13.

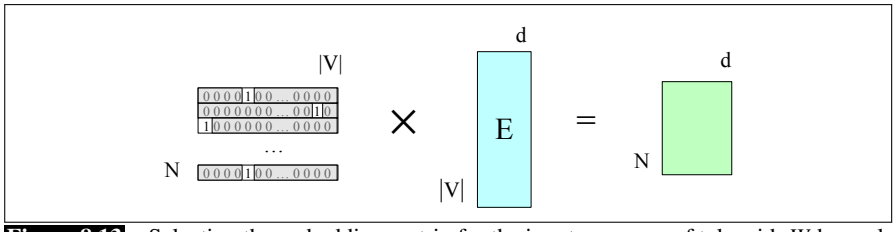


Figure 8.13 Selecting the embedding matrix for the input sequence of token ids W by multiplying a one-hot matrix corresponding to W by the embedding matrix **E**.

These token embeddings are not position-dependent. To represent the position of each token in the sequence, we combine these token embeddings with **positional embeddings** specific to each position in an input sequence.

positional embeddings

absolute position

Where do we get these positional embeddings? The simplest method, called **absolute position**, is to start with randomly initialized embeddings corresponding to each possible input position up to some maximum length. For example, just as we have an embedding for the word *fish*, we'll have an embedding for the position 3. As with word embeddings, these positional embeddings are learned along with other parameters during training. We can store them in a matrix E_{pos} of shape $[N \times d]$.

To produce an input embedding that captures positional information, we just add the word embedding for each input to its corresponding positional embedding. The individual token and position embeddings are both of size $[1 \times d]$, so their sum is also $[1 \times d]$, This new embedding serves as the input for further processing. Fig. 8.14 shows the idea.

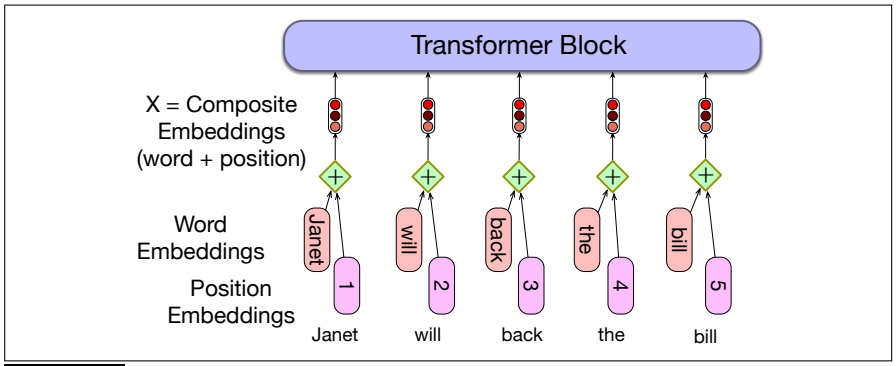


Figure 8.14 A simple way to model position: add an embedding of the absolute position to the token embedding to produce a new embedding of the same dimensionality.

The final representation of the input, the matrix **X**, is an $[N \times d]$ matrix in which each row i is the representation of the ith token in the input, computed by adding $\mathbf{E}[id(i)]$—the embedding of the id of the token that occurred at position i—, to $\mathbf{P}[i]$, the positional embedding of position i.

A potential problem with the simple position embedding approach is that there will be plenty of training examples for the initial positions in our inputs and correspondingly fewer at the outer length limits. These latter embeddings may be poorly trained and may not generalize well during testing. An alternative is to choose a static function that maps integer inputs to real-valued vectors in a way that better handles sequences of arbitrary length. A combination of sine and cosine functions with differing frequencies was used in the original transformer work. Sinusoidal position embeddings may also help in capturing the inherent relationships among the

positions, like the fact that position 4 in an input is more closely related to position 5 than it is to position 17.

relative position

A more complex style of positional embedding methods extend this idea of capturing relationships even further to directly represent **relative position** instead of absolute position, often implemented in the attention mechanism at each layer rather than being added once at the initial input.

8.5 The Language Modeling Head

language modeling head
head

The last component of the transformer we must introduce is the **language modeling head**. Here we are using the word **head** to mean the additional neural circuitry we add on top of the basic transformer architecture when we apply pretrained transformer models to various tasks. The language modeling head is the circuitry we need to do language modeling.

Recall that language models, from the simple n-gram models of Chapter 3 through the feedforward and RNN language models of Chapter 6 and Chapter 13, are word predictors. Given a context of words, they assign a probability to each possible next word. For example, if the preceding context is *"Thanks for all the"* and we want to know how likely the next word is *"fish"* we would compute:

$$P(fish|Thanks\ for\ all\ the)$$

Language models give us the ability to assign such a conditional probability to every possible next word, giving us a distribution over the entire vocabulary. The n-gram language models of Chapter 3 compute the probability of a word given counts of its occurrence with the $n-1$ prior words. The context is thus of size $n-1$. For transformer language models, the context is the size of the transformer's context window, which can be quite large, like 32K tokens for large models (and much larger contexts of millions of words are possible with special long-context architectures).

The job of the language modeling head is to take the output of the final transformer layer from the last token N and use it to predict the upcoming word at position $N+1$. Fig. 8.15 shows how to accomplish this task, taking the output of the last token at the last layer (the d-dimensional output embedding of shape $[1 \times d]$) and producing a probability distribution over words (from which we will choose one to generate).

The first module in Fig. 8.15 is a linear layer, whose job is to project from the output h_N^L, which represents the output token embedding at position N from the final block L, (hence of shape $[1 \times d]$) to the **logit** vector, or score vector, that will have a single score for each of the $|V|$ possible words in the vocabulary V. The logit vector $\mathbf{u}$ is thus of dimensionality $[1 \times |V|]$.

logit

This linear layer can be learned, but more commonly we tie this matrix to (the transpose of) the embedding matrix $\mathbf{E}$. Recall that in **weight tying**, we use the same weights for two different matrices in the model. Thus at the input stage of the transformer the embedding matrix (of shape $[|V| \times d]$) is used to map from a one-hot vector over the vocabulary (of shape $[1 \times |V|]$) to an embedding (of shape $[1 \times d]$). And then in the language model head, $\mathbf{E}^\mathsf{T}$, the transpose of the embedding matrix (of shape $[d \times |V|]$) is used to map back from an embedding (shape $[1 \times d]$) to a vector over the vocabulary (shape $[1 \times |V|]$). In the learning process, $\mathbf{E}$ will be optimized to be good at doing both of these mappings. We therefore sometimes call the transpose $\mathbf{E}^\mathsf{T}$ the **unembedding** layer because it is performing this reverse mapping.

weight tying
unembedding

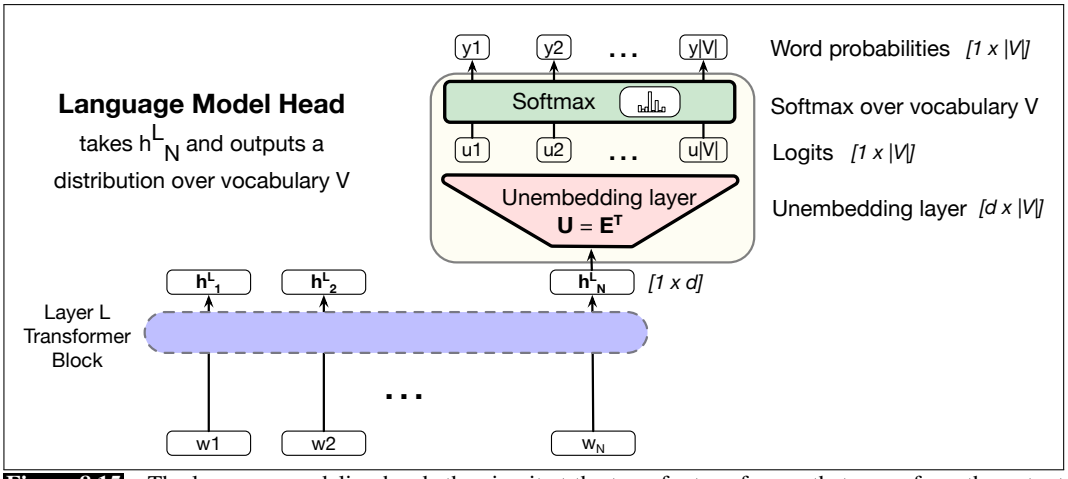


Figure 8.15 The language modeling head: the circuit at the top of a transformer that maps from the output embedding for token N from the last transformer layer ($\mathbf{h}_N^L$) to a probability distribution over words in the vocabulary V.

A softmax layer turns the logits $\mathbf{u}$ into the probabilities $\mathbf{y}$ over the vocabulary.

$$\mathbf{u} = \mathbf{h}_\mathbf{N}^\mathbf{L}\ \mathbf{E}^\mathbf{T} \tag{8.46}$$

$$\mathbf{y} = \text{softmax}(\mathbf{u}) \tag{8.47}$$

We can use these probabilities to do things like help assign a probability to a given text. But the most important usage is to generate text, which we do by **sampling** a word from these probabilities y. We might sample the highest probability word ('greedy' decoding), or use another of the sampling methods from Section 7.4 or Section 8.6.

In either case, whatever entry y_k we choose from the probability vector $\mathbf{y}$, we generate the word that has that index k.

Fig. 8.16 shows the total stacked architecture for one token i. Note that the input to each transformer layer x_i^ℓ is the same as the output from the preceding layer $h_i^{\ell-1}$.

decoder-only model

A terminological note before we conclude: You will sometimes see a transformer used for this kind of unidirectional causal language model called a **decoder-only model**. This is because this model constitutes roughly half of the **encoder-decoder model** for transformers that we'll see how to apply to machine translation in Chapter 12. (Confusingly, the original introduction of the transformer had an encoder-decoder architecture, and it was only later that the standard paradigm for causal language model was defined by using only the decoder part of this original architecture).

8.6 More on Sampling

The sampling methods we introduce below each have parameters that enable trading off two important factors in generation: **quality** and **diversity**. Methods that emphasize the most probable words tend to produce generations that are rated by people as more accurate, more coherent, and more factual, but also more boring and more repetitive. Methods that give a bit more weight to the middle-probability

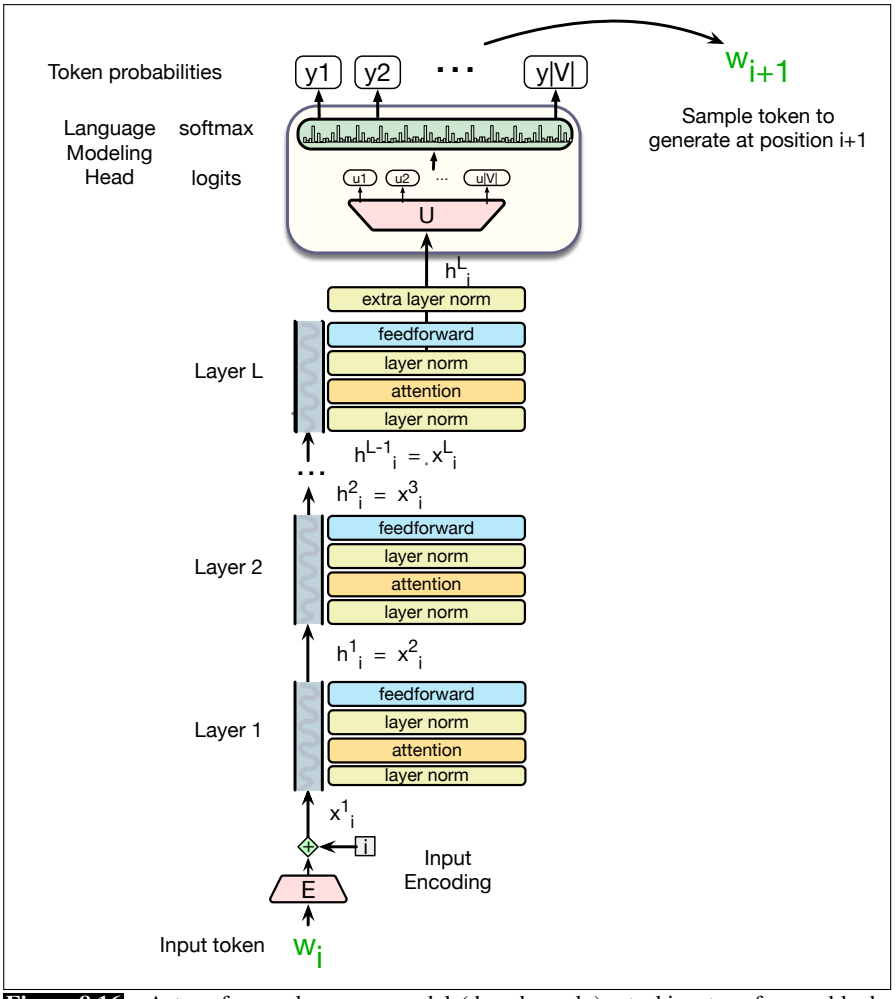


Figure 8.16 A transformer language model (decoder-only), stacking transformer blocks and mapping from an input token w_i to a predicted next token w_{i+1}.

words tend to be more creative and more diverse, but less factual and more likely to be incoherent or otherwise low-quality.

8.6.1 Top-*k* sampling

top-k sampling

Top-k sampling is a simple generalization of greedy decoding. Instead of choosing the single most probable word to generate, we first truncate the distribution to the top k most likely words, renormalize to produce a legitimate probability distribution, and then randomly sample from within these k words according to their renormalized probabilities. More formally:

1. Choose in advance a number of words k
2. For each word in the vocabulary V, use the language model to compute the likelihood of this word given the context $p(w_t|\mathbf{w}_{<t})$
3. Sort the words by their likelihood, and throw away any word that is not one of the top k most probable words.

4. Renormalize the scores of the k words to be a legitimate probability distribution.
5. Randomly sample a word from within these remaining k most-probable words according to its probability.

When $k = 1$, top-k sampling is identical to greedy decoding. Setting k to a larger number than 1 leads us to sometimes select a word which is not necessarily the most probable, but is still probable enough, and whose choice results in generating more diverse but still high-enough-quality text.

8.6.2 Nucleus or top-p sampling

One problem with top-k sampling is that k is fixed, but the shape of the probability distribution over words differs in different contexts. If we set $k = 10$, sometimes the top 10 words will be very likely and include most of the probability mass, but other times the probability distribution will be flatter and the top 10 words will only include a small part of the probability mass.

top-p sampling

An alternative, called **top-p sampling** or **nucleus sampling** (Holtzman et al., 2020), is to keep not the top k words, but the top p percent of the probability mass. The goal is the same; to truncate the distribution to remove the very unlikely words. But by measuring probability rather than the number of words, the hope is that the measure will be more robust in very different contexts, dynamically increasing and decreasing the pool of word candidates.

Given a distribution $P(w_t|\mathbf{w}_{<t})$, we sort the distribution from most probable, and then the top-p vocabulary $V^{(p)}$ is the smallest set of words such that

$$\sum_{w \in V^{(p)}} P(w|\mathbf{w}_{<t}) \geq p. \tag{8.48}$$

8.7 Training

We described the training process for language models in the prior chapter. Recall that large language models are trained with cross-entropy loss, also called the negative log likelihood loss. At time t the cross-entropy loss is the negative log probability the model assigns to the next word in the training sequence, $-\log p(w_{t+1})$.

Fig. 8.17 illustrates the general training approach. At each step, given all the preceding words, the final transformer layer produces an output distribution over the entire vocabulary. During training, the probability assigned to the correct word by the model is used to calculate the cross-entropy loss for each item in the sequence. The loss for a training sequence is the average cross-entropy loss over the entire sequence. The weights in the network are adjusted to minimize the average CE loss over the training sequence via gradient descent.

With transformers, each training item can be processed in parallel since the output for each element in the sequence is computed separately.

Large models are generally trained by filling the full context window (for example 4096 tokens for GPT4 or 8192 for Llama 3) with text. If documents are shorter than this, multiple documents are packed into the window with a special end-of-text token between them. The batch size for gradient descent is usually quite large (the largest GPT-3 model uses a batch size of 3.2 million tokens).

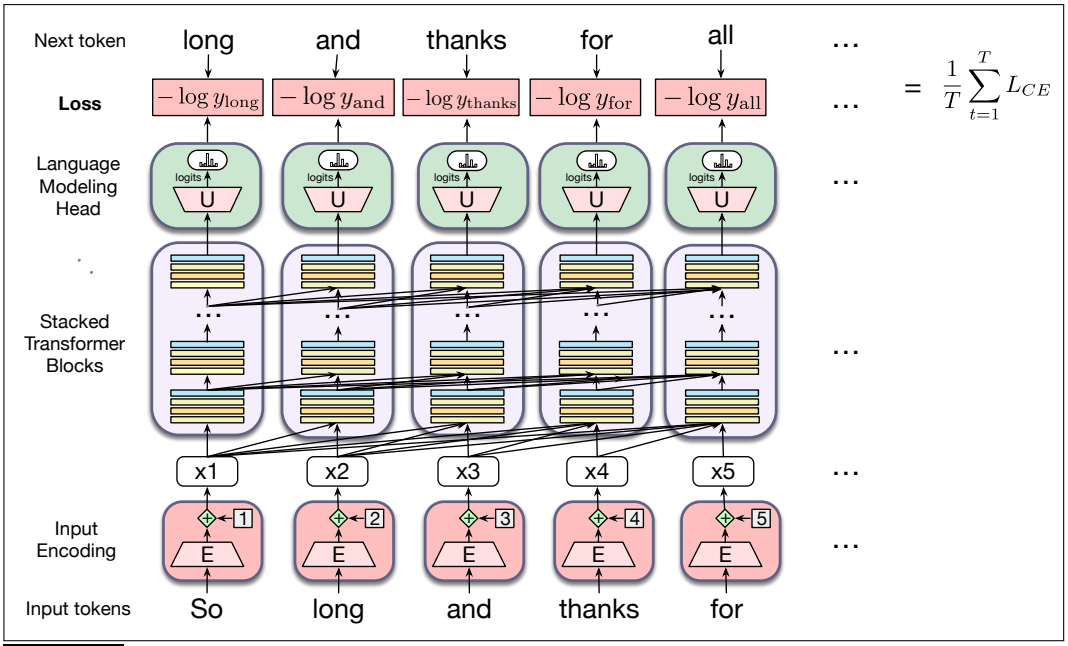


Figure 8.17 Training a transformer as a language model.

8.8 Dealing with Scale

Large language models are large. For example the *Llama 3.1 405B Instruct* model from Meta has 405 billion parameters (it has L=126 layers, model dimensionality d=16,384, and A=128 attention heads) and was trained on 15.6 terabytes of text tokens using a vocabulary of 128K tokens (Llama Team, 2024). So there is a lot of research on understanding how LLMs scale, and especially how to implement them given limited resources. In the next few sections we discuss how to think about scale (the concept of **scaling laws**), and important techniques for getting language models to work efficiently, such as the **KV cache** and parameter-efficient fine tuning (PEFT).

8.8.1 Scaling laws

The performance of large language models has shown to be mainly determined by 3 factors: model size (the number of parameters not counting embeddings), dataset size (the amount of training data), and the amount of compute used for training. That is, we can improve a model by adding parameters (adding more layers or having wider contexts or both), by training on more data, or by training for more iterations.

scaling laws

The relationships between these factors and performance are known as **scaling laws**. Roughly speaking, the performance of a large language model (the loss) scales as a power-law with each of these three properties of model training.

For example, Kaplan et al. (2020) found the following three relationships for loss L as a function of the number of non-embedding parameters N, the dataset size D, and the compute budget C, for models training with limited parameters, dataset,

or compute budget, if in each case the other two properties are held constant:

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N} \tag{8.49}$$

$$L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D} \tag{8.50}$$

$$L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C} \tag{8.51}$$

The number of (non-embedding) parameters N can be roughly computed as follows (ignoring biases, and with d as the input and output dimensionality of the model, d_{attn} as the self-attention layer size, and d_{ff} the size of the feedforward layer):

$$\begin{aligned} N &\approx 2\, d\, n_{\text{layer}}(2\, d_{\text{attn}} + d_{\text{ff}}) \\ &\approx 12\, n_{\text{layer}}\, d^2 \\ &\quad (\text{assuming } d_{\text{attn}} = d_{\text{ff}}/4 = d) \end{aligned} \tag{8.52}$$

Thus GPT-3, with $n = 96$ layers and dimensionality $d = 12288$, has $12 \times 96 \times 12288^2 \approx 175$ billion parameters.

The values of N_c, D_c, C_c, α_N, α_D, and α_C depend on the exact transformer architecture, tokenization, and vocabulary size, so rather than all the precise values, scaling laws focus on the relationship with loss.[3]

Scaling laws can be useful in deciding how to train a model to a particular performance, for example by looking at early in the training curve, or performance with smaller amounts of data, to predict what the loss would be if we were to add more data or increase model size. Other aspects of scaling laws can also tell us how much data we need to add when scaling up a model.

8.8.2 KV Cache

We saw in Fig. 8.11 and in Eq. 8.34 (repeated below) how the attention vector can be very efficiently computed in parallel for training, via two matrix multiplications:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\intercal}{\sqrt{d_k}}\right)\mathbf{V} \tag{8.53}$$

Unfortunately we can't do quite the same efficient computation in inference as in training. That's because at inference time, we iteratively generate the next tokens one at a time. For a new token that we have just generated, call it $\mathbf{x}_i$, we need to compute its query, key, and values by multiplying by $\mathbf{W^Q}$, $\mathbf{W^K}$, and $\mathbf{W^V}$ respectively. But it would be a waste of computation time to recompute the key and value vectors for all the **prior** tokens $\mathbf{x}_{<i}$; at prior steps we already computed these key and value vectors! So instead of recomputing these, whenever we compute the key and value vectors we store them in memory in the **KV cache**, and then we can just grab them from the cache when we need them. Fig. 8.18 modifies Fig. 8.11 to show the computation that takes place for a single new token, showing which values we can take from the cache rather than recompute.

KV cache

[3] For the initial experiment in Kaplan et al. (2020) the precise values were $\alpha_N = 0.076$, $N_c = 8.8 \times 10^{13}$ (parameters), $\alpha_D = 0.095$, $D_c = 5.4 \times 10^{13}$ (tokens), $\alpha_C = 0.050$, $C_c = 3.1 \times 10^8$ (petaflop-days).

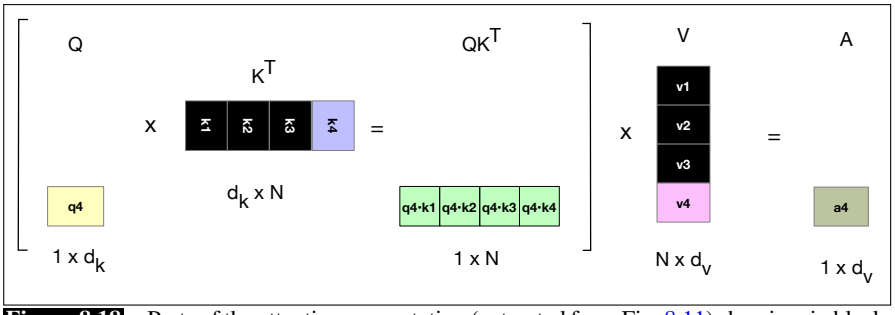


Figure 8.18 Parts of the attention computation (extracted from Fig. 8.11) showing, in black, the vectors that can be stored in the cache rather than recomputed when computing the attention score for the 4th token.

8.8.3 Parameter Efficient Fine Tuning

As we mentioned above, it's very common to take a language model and give it more information about a new domain by **finetuning** it (continuing to train it to predict upcoming words) on some additional data.

Fine-tuning can be very difficult with very large language models, because there are enormous numbers of parameters to train; each pass of batch gradient descent has to backpropagate through many many huge layers. This makes finetuning huge language models extremely expensive in processing power, in memory, and in time. For this reason, there are alternative methods that allow a model to be finetuned without changing all the parameters. Such methods are called **parameter-efficient fine tuning** or sometimes **PEFT**, because we efficiently select a subset of parameters to update when finetuning. For example we freeze some of the parameters (don't change them), and only update some particular subset of parameters.

parameter-efficient fine tuning
PEFT

LoRA

Here we describe one such model, called **LoRA**, for **Lo**w-**R**ank **A**daptation. The intuition of LoRA is that transformers have many dense layers which perform matrix multiplication (for example the $\mathbf{W^Q}$, $\mathbf{W^K}$, $\mathbf{W^V}$, $\mathbf{W^O}$ layers in the attention computation). Instead of updating these layers during finetuning, with LoRA we freeze these layers and instead update a low-rank approximation that has fewer parameters.

Consider a matrix $\mathbf{W}$ of dimensionality $[k \times d]$ that needs to be updated during finetuning via gradient descent. Normally this matrix would get updates $\Delta\mathbf{W}$ of dimensionality $[k \times d]$, for updating the $k \times d$ parameters after gradient descent. In LoRA, we freeze $\mathbf{W}$ and update instead a low-rank decomposition of $\mathbf{W}$. We create two matrices $\mathbf{A}$ and $\mathbf{B}$, where $\mathbf{A}$ has size $[k \times r]$ and $\mathbf{B}$ has size $[r \times d]$, and we choose r to be quite small, $r << \min(d,k)$. During finetuning we update $\mathbf{A}$ and $\mathbf{B}$ instead of $\mathbf{W}$. That is, we replace $\mathbf{W} + \Delta\mathbf{W}$ with $\mathbf{W} + \mathbf{AB}$. Fig. 8.19 shows the intuition. For replacing the forward pass $\mathbf{h} = \mathbf{xW}$, the new forward pass is instead:

$$\mathbf{h} = \mathbf{xW} + \mathbf{xAB} \tag{8.54}$$

LoRA has a number of advantages. It dramatically reduces hardware requirements, since gradients don't have to be calculated for most parameters. The weight updates can be simply added in to the pretrained weights, since $\mathbf{AB}$ is the same size as $\mathbf{W}$). That means it doesn't add any time during inference. And it also means it's possible to build LoRA modules for different domains and just swap them in and out by adding them in or subtracting them from $\mathbf{W}$.

In its original version LoRA was applied just to the matrices in the attention computation (the $\mathbf{W^Q}$, $\mathbf{W^K}$, $\mathbf{W^V}$, and $\mathbf{W^O}$ layers). Many variants of LoRA exist.

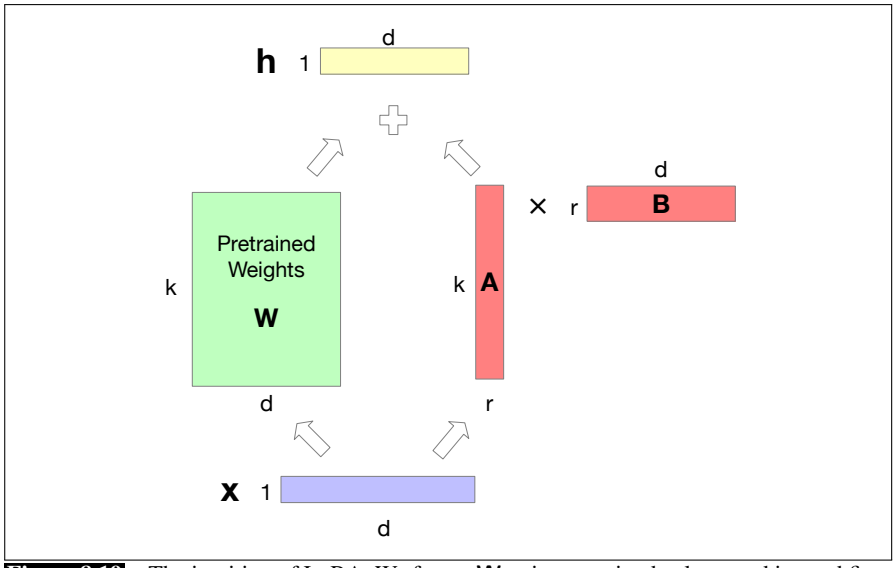


Figure 8.19 The intuition of LoRA. We freeze **W** to its pretrained values, and instead fine-tune by training a pair of matrices **A** and **B**, updating those instead of **W**, and just sum **W** and the updated **AB**.

8.9 Interpreting the Transformer

interpretability

How does a transformer-based language model manage to do so well at language tasks? The subfield of **interpretability**, sometimes called **mechanistic interpretability**, focuses on ways to understand mechanistically what is going on inside the transformer. In the next two subsections we discuss two well-studied aspects of transformer interpretability.

8.9.1 In-Context Learning and Induction Heads

As a way of getting a model to do what we want, we can think of prompting as being fundamentally different than pretraining. Learning via pretraining means updating the model's parameters by using gradient descent according to some loss function. But prompting with demonstrations can teach a model to do a new task. The model is learning something about the task from those demonstrations as it processes the prompt.

Even without demonstrations, we can think of the process of prompting as a kind of learning. For example, the further a model gets in a prompt, the better it tends to get at predicting the upcoming tokens. The information in the context is helping give the model more predictive power.

in-context learning

The term **in-context learning** was first proposed by Brown et al. (2020) in their introduction of the GPT3 system, to refer to either of these kinds of learning that language models do from their prompts. In-context learning means language models learning to do new tasks, better predict tokens, or generally reduce their loss during the forward-pass at inference-time, without any gradient-based updates to the model's parameters.

induction heads

How does in-context learning work? While we don't know for sure, there are some intriguing ideas. One hypothesis is based on the idea of **induction heads** (Elhage et al., 2021; Olsson et al., 2022). Induction heads are the name for a **circuit**,

which is a kind of abstract component of a network. The induction head circuit is part of the attention computation in transformers, discovered by looking at mini language models with only 1-2 attention heads.

The function of the induction head is to predict repeated sequences. For example if it sees the pattern AB...A in an input sequence, it predicts that B will follow, instantiating the **pattern completion** rule AB...A→ B. It does this by having a *prefix matching* component of the attention computation that, when looking at the current token A, searches back over the context to find a prior instance of A. If it finds one, the induction head has a *copying* mechanism that "copies" the token B that followed the earlier A, by increasing the probability the B will occur next. Fig. 8.20 shows an example.

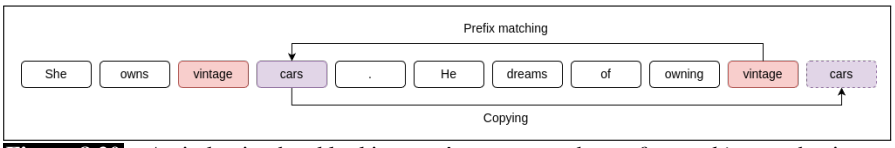


Figure 8.20 An induction head looking at `vintage` uses the *prefix matching* mechanism to find a prior instance of `vintage`, and the *copying* mechanism to predict that `cars` will occur again. Figure from Crosbie and Shutova (2022).

Olsson et al. (2022) propose that a generalized fuzzy version of this pattern completion rule, implementing a rule like A*B*...A→ B, where A* ≈ A and B* ≈ B (by ≈ we mean they are semantically similar in some way), might be responsible for in-context learning. Suggestive evidence for their hypothesis comes from Crosbie and Shutova (2022), who show that **ablating** induction heads causes in-context learning performance to decrease. **Ablation** is originally a medical term meaning the removal of something. We use it in NLP interpretability studies as a tool for testing causal effects; if we knock out a hypothesized cause, we would expect the effect to disappear. Crosbie and Shutova (2022) ablate induction heads by first finding attention heads that perform as induction heads on random input sequences, and then zeroing out the output of these heads by setting certain terms of the output matrix $\mathbf{W^O}$ to zero. Indeed they find that ablated models are much worse at in-context learning: they have much worse performance at learning from demonstrations in the prompts.

ablating

8.9.2 Logit Lens

logit lens

Another useful interpretability tool, the **logit lens** (Nostalgebraist, 2020), offers a way to visualize what the internal layers of the transformer might be representing.

The idea is that we take any vector from any layer of the transformer and, pretending that it is the prefinal embedding, simply multiply it by the **unembedding layer** to get logits, and compute a softmax to see the distribution over words that that vector might be representing. This can be a useful window into the internal representations of the model. Since the network wasn't trained to make the internal representations function in this way, the logit lens doesn't always work perfectly, but this can still be a useful trick to help us visualize the internal layers of a transformer.

8.10 Summary

This chapter has introduced the transformer and its components for the language modeling task introduced in the previous chapter. Here's a summary of the main points that we covered:

- Transformers are non-recurrent networks based on **multi-head attention**, a kind of **self-attention**. A multi-head attention computation takes an input vector $\mathbf{x}_i$ and maps it to an output $\mathbf{a}_i$ by adding in vectors from prior tokens, weighted by how relevant they are for the processing of the current word.
- A **transformer block** consists of a **residual stream** in which the input from the prior layer is passed up to the next layer, with the output of different components added to it. These components include a **multi-head attention layer** followed by a **feedforward layer**, each preceded by **layer normalizations**. Transformer blocks are stacked to make deeper and more powerful networks.
- The input to a transformer is computed by adding an embedding (computed with an **embedding matrix**) to a **positional encoding** that represents the sequential position of the token in the window.
- Language models can be built out of stacks of transformer blocks, with a **language model head** at the top, which applies an **unembedding** matrix to the output $\mathbf{H}$ of the top layer to generate the **logits**, which are then passed through a softmax to generate word probabilities.
- Transformer-based language models have a wide context window (200K tokens or even more for very large models with special mechanisms) allowing them to draw on enormous amounts of context to predict upcoming words.
- There are various computational tricks for making large language models more efficient, such as the **KV cache** and **parameter-efficient finetuning**.

Historical Notes

The transformer (Vaswani et al., 2017) was developed drawing on two lines of prior research: **self-attention** and **memory networks**.

Encoder-decoder attention, the idea of using a soft weighting over the encodings of input words to inform a generative decoder (see Chapter 12) was developed by Graves (2013) in the context of handwriting generation, and Bahdanau et al. (2015) for MT. This idea was extended to self-attention by dropping the need for separate encoding and decoding sequences and instead seeing attention as a way of weighting the tokens in collecting information passed from lower layers to higher layers (Ling et al., 2015; Cheng et al., 2016; Liu et al., 2016).

Other aspects of the transformer, including the terminology of key, query, and value, came from **memory networks**, a mechanism for adding an external read-write memory to networks, by using an embedding of a query to match keys representing content in an associative memory (Sukhbaatar et al., 2015; Weston et al., 2015; Graves et al., 2014).

MORE HISTORY TBD IN NEXT DRAFT.

CHAPTER

9 Masked Language Models

Larvatus prodeo [Masked, I go forward]
Descartes

In the previous two chapters we introduced the transformer and saw how to pretrain a transformer language model as a **causal** or left-to-right language model. In this chapter we'll introduce a second paradigm for pretrained language models, the **bidirectional transformer** encoder, and the most widely-used version, the **BERT** model (Devlin et al., 2019). This model is trained via **masked language modeling**, where instead of predicting the following word, we mask a word in the middle and ask the model to guess the word given the words on both sides. This method thus allows the model to see both the right and left context.

BERT
masked language modeling

We also introduced **finetuning** in the prior chapter. Here we describe a new kind of finetuning, in which we take the transformer network learned by these pretrained models, add a neural net classifier after the top layer of the network, and train it on some additional labeled data to perform some downstream task like named entity tagging or natural language inference. As before, the intuition is that the pretraining phase learns a language model that instantiates rich representations of word meaning, that thus enables the model to more easily learn ('be finetuned to') the requirements of a downstream language understanding task. This aspect of the pretrain-finetune paradigm is an instance of what is called **transfer learning** in machine learning: the method of acquiring knowledge from one task or domain, and then applying it (transferring it) to solve a new task.

finetuning

transfer learning

The second idea that we introduce in this chapter is the idea of **contextual embeddings**: representations for words in context. The methods of Chapter 5 like word2vec or GloVe learned a single vector embedding for each unique word w in the vocabulary. By contrast, with contextual embeddings, such as those learned by masked language models like BERT, each word w will be represented by a different vector each time it appears in a different context. While the causal language models of Chapter 8 also use contextual embeddings, the embeddings created by masked language models seem to function particularly well as representations.

9.1 Bidirectional Transformer Encoders

Let's begin by introducing the bidirectional transformer encoder that underlies models like BERT and its descendants like **RoBERTa** (Liu et al., 2019) or **SpanBERT** (Joshi et al., 2020). In Chapter 7 we introduced the idea of left-to-right language models that can be applied to autoregressive contextual generation problems like question answering or summarization, and in Chapter 8 we saw how to implement language models with causal (left-to-right) transformers. But this left-to-right nature of these models is also a limitation, because there are tasks for which it would be useful, when processing a token, to be able to peek at future tokens. This is espe-

cially true for **sequence labeling** tasks in which we want to tag each token with a label, such as the **named entity tagging** task we'll introduce in Section 9.5, or tasks like part-of-speech tagging or parsing that come up in later chapters.

The **bidirectional** encoders that we introduce here are a different kind of beast than causal models. The causal models of Chapter 8 are generative models, designed to easily generate the next token in a sequence. But the focus of bidirectional encoders is instead on computing contextualized representations of the input tokens. Bidirectional encoders use self-attention to map sequences of input embeddings $(\mathbf{x}_1, ..., \mathbf{x}_n)$ to sequences of output embeddings of the same length $(\mathbf{h}_1, ..., \mathbf{h}_n)$, where the output vectors have been contextualized using information from the entire input sequence. These output embeddings are contextualized representations of each input token that are useful across a range of applications where we need to do a classification or a decision based on the token in context.

Remember that we said the models of Chapter 8 are sometimes called **decoder-only**, because they correspond to the decoder part of the encoder-decoder model we will introduce in Chapter 12. By contrast, the masked language models of this chapter are sometimes called **encoder-only**, because they produce an encoding for each input token but generally aren't used to produce running text by decoding/sampling. That's an important point: masked language models are not used for generation. They are generally instead used for interpretative tasks.

9.1.1 The architecture for bidirectional masked models

Let's first discuss the overall architecture. Bidirectional transformer-based language models differ in two ways from the causal transformers in the previous chapters. The first is that the attention function isn't causal; the attention for a token i can look at following tokens $i+1$ and so on. The second is that the training is slightly different since we are predicting something in the middle of our text rather than at the end. We'll discuss the first here and the second in the following section.

Fig. 9.1a, reproduced here from Chapter 8, shows the information flow in the left-to-right approach of Chapter 8. The attention computation at each token is based on the preceding (and current) input tokens, ignoring potentially useful information located to the right of the token under consideration. Bidirectional encoders overcome this limitation by allowing the attention mechanism to range over the entire input, as shown in Fig. 9.1b.

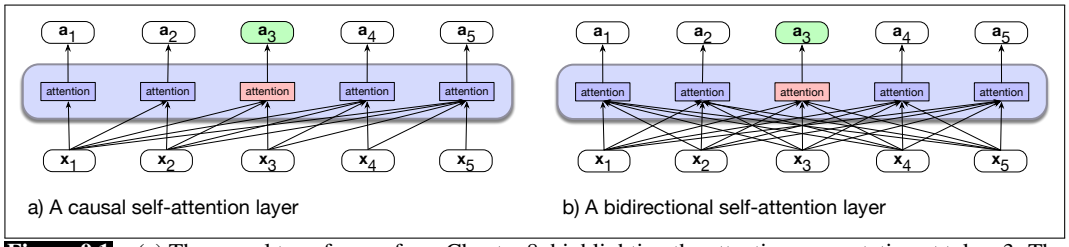


Figure 9.1 (a) The causal transformer from Chapter 8, highlighting the attention computation at token 3. The attention value at each token is computed using only information seen earlier in the context. (b) Information flow in a bidirectional attention model. In processing each token, the model attends to all inputs, both before and after the current one. So attention for token 3 can draw on information from following tokens.

The implementation is very simple! We simply remove the attention masking step that we introduced in Eq. 8.34. Recall from Chapter 8 that we had to mask the $\mathbf{QK}^\intercal$ matrix for causal transformers so that attention couldn't look at future tokens

(repeated from Eq. 8.34 for a single attention head):

$$\textbf{head} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{QK}^\intercal}{\sqrt{d_k}}\right)\right)\mathbf{V} \tag{9.1}$$

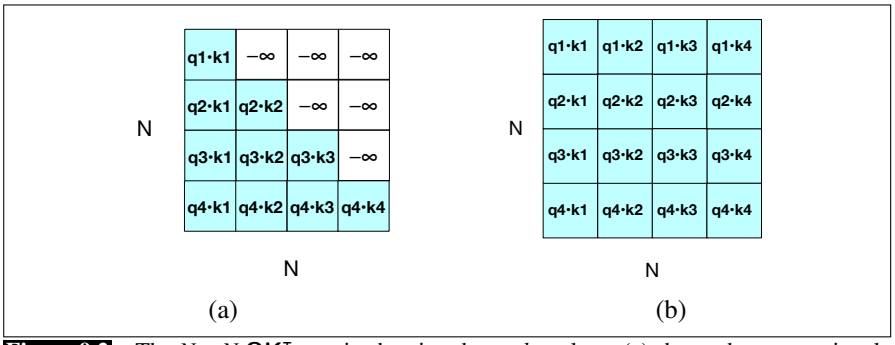


Figure 9.2 The $N \times N$ $\mathbf{QK}^\intercal$ matrix showing the $q_i \cdot k_j$ values. (a) shows the upper-triangle portion of the comparisons matrix zeroed out (set to $-\infty$, which the softmax will turn to zero), while (b) shows the unmasked version.

Fig. 9.2 shows the masked version of $\mathbf{QK}^\intercal$ and the unmasked version. For bidirectional attention, we use the unmasked version of Fig. 9.2b. Thus the attention computation for bidirectional attention is exactly the same as Eq. 9.1 but with the mask removed:

$$\textbf{head} = \text{softmax}\left(\frac{\mathbf{QK}^\intercal}{\sqrt{d_k}}\right)\mathbf{V} \tag{9.2}$$

Otherwise, the attention computation is identical to what we saw in Chapter 8, as is the transformer block architecture (the feedforward layer, layer norm, and so on). As in Chapter 8, the input is also a series of subword tokens, usually computed by one of the 3 popular tokenization algorithms (including the BPE algorithm that we already saw in Chapter 2 and two others, the WordPiece algorithm and the SentencePiece Unigram LM algorithm). That means every input sentence first has to be tokenized, and all further processing takes place on subword tokens rather than words. This will require, as we'll see in the third part of the textbook, that for some NLP tasks that require notions of words (like parsing) we will occasionally need to map subwords back to words.

To make this more concrete, the original English-only bidirectional transformer encoder model, BERT (Devlin et al., 2019), consisted of the following:

- An English-only subword vocabulary consisting of 30,000 tokens generated using the WordPiece algorithm (Schuster and Nakajima, 2012).
- Input context window N=512 tokens, and model dimensionality d=768
- So $\mathbf{X}$, the input to the model, is of shape $[N \times d] = [512 \times 768]$.
- L=12 layers of transformer blocks, each with A=12 (bidirectional) multihead attention layers.
- The resulting model has about 100M parameters.

The larger multilingual XLM-RoBERTa model, trained on 100 languages, has

- A multilingual subword vocabulary with 250,000 tokens generated using the SentencePiece Unigram LM algorithm (Kudo and Richardson, 2018b).

- Input context window N=512 tokens, and model dimensionality d=1024, hence $\mathbf{X}$, the input to the model, is of shape $[N \times d] = [512 \times 1024]$.
- L=24 layers of transformer blocks, with A=16 multihead attention layers each
- The resulting model has about 550M parameters.

Note that 550M parameters is relatively small as large language models go (Llama 3 has 405B parameters, so is 3 orders of magnitude bigger). Indeed, masked language models tend to be much smaller than causal language models.

9.2 Training Bidirectional Encoders

We trained causal transformer language models in Chapter 8 by making them iteratively predict the next word in a text. But eliminating the causal mask in attention makes the guess-the-next-word language modeling task trivial—the answer is directly available from the context—so we're in need of a new training scheme. Instead of trying to predict the next word, the model learns to perform a fill-in-the-blank task, technically called the **cloze task** (Taylor, 1953). To see this, let's return to the motivating example from Chapter 3. Instead of predicting which words are likely to come next in this example:

cloze task

`The water of Walden Pond is so beautifully ____`

we're asked to predict a missing item given the rest of the sentence.

`The ____ of Walden Pond is so beautifully ...`

That is, given an input sequence with one or more elements missing, the learning task is to predict the missing elements. More precisely, during training the model is deprived of one or more tokens of an input sequence and must generate a probability distribution over the vocabulary for each of the missing items. We then use the cross-entropy loss from each of the model's predictions to drive the learning process.

This approach can be generalized to any of a variety of methods that corrupt the training input and then asks the model to recover the original input. Examples of the kinds of manipulations that have been used include masks, substitutions, reorderings, deletions, and extraneous insertions into the training text. The general name for this kind of training is called **denoising**: we corrupt (add noise to) the input in some way (by masking a word, or putting in an incorrect word) and the goal of the system is to remove the noise.

denoising

9.2.1 Masking Words

Masked Language Modeling

Let's describe the **Masked Language Modeling** (MLM) approach to training bidirectional encoders (Devlin et al., 2019). As with the language model training methods we've already seen, **MLM** uses unannotated text from a large corpus. In MLM training, the model is presented with a series of sentences from the training corpus in which a percentage of tokens (15% in the BERT model) have been randomly chosen to be manipulated by the masking procedure. Given an input sentence `lunch was delicious` and assume we randomly chose the 3rd token `delicious` to be manipulated,

- 80% of the time: The token is replaced with the special vocabulary token named `[MASK]`, e.g. `lunch was delicious` → `lunch was [MASK]`.

- 10% of the time: The token is replaced with another token, randomly sampled from the vocabulary based on token unigram probabilities. e.g. `lunch was delicious` → `lunch was gasp`.
- 10% of the time: the token is left unchanged. e.g. `lunch was delicious` → `lunch was delicious`.

We then train the model to guess the correct token for the manipulated tokens. Why the three possible manipulations? Adding the `[MASK]` token creates a mismatch between pretraining and downstream finetuning or inference, since when we employ the MLM model to perform a downstream task, we don't use any `[MASK]` tokens. If we just replaced tokens with the `[MASK]`, the model might only predict tokens when it sees a `[MASK]`, but we want the model to try to always predict the input token.

To train the model to make the prediction, the original input sequence is tokenized using a subword model and tokens are sampled to be manipulated. Word embeddings for all of the tokens in the input are retrieved from the **E** embedding matrix and combined with positional embeddings to form the input to the transformer, passed through the stack of bidirectional transformer blocks, and then the language modeling head. The MLM training objective is to predict the original inputs for each of the masked tokens and the cross-entropy loss from these predictions drives the training process for all the parameters in the model. That is, all of the input tokens play a role in the self-attention process, but only the sampled tokens are used for learning.

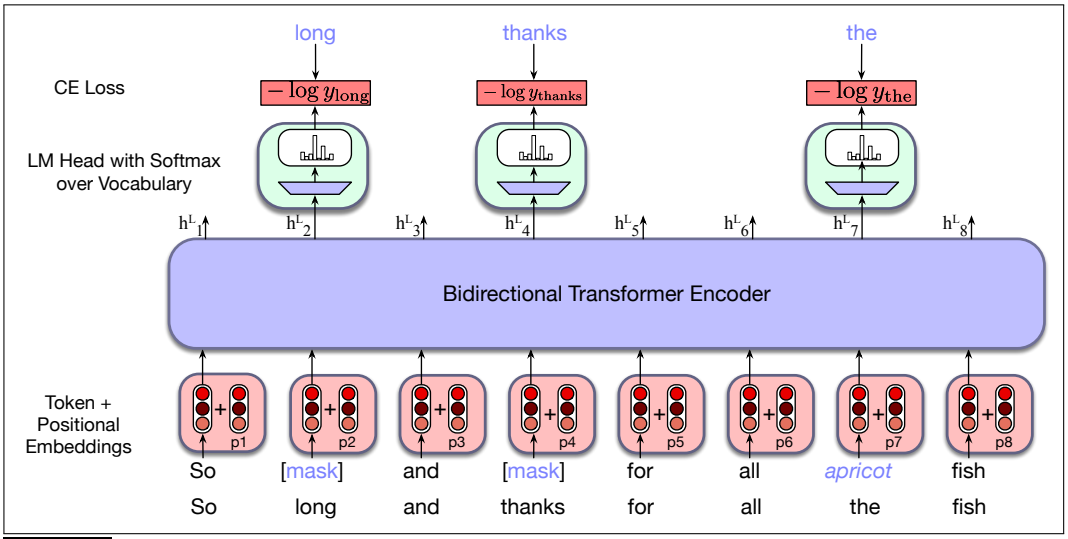


Figure 9.3 Masked language model training. In this example, three of the input tokens are selected, two of which are masked and the third is replaced with an unrelated word. The probabilities assigned by the model to these three items are used as the training loss. The other 5 tokens don't play a role in training loss.

Fig. 9.3 illustrates this approach with a simple example. Here, *long*, *thanks* and *the* have been sampled from the training sequence, with the first two masked and *the* replaced with the randomly sampled token *apricot*. The resulting embeddings are passed through a stack of bidirectional transformer blocks. Recall from Section 8.5 in Chapter 8 that to produce a probability distribution over the vocabulary for each of the masked tokens, the **language modeling head** takes the output vector $\mathbf{h}_i^L$ from the final transformer layer L for each masked token i, multiplies it by the unembedding layer $\mathbf{E}^T$ to produce the logits $\mathbf{u}$, and then uses softmax to turn the logits into

probabilities $\mathbf{y}$ over the vocabulary:

$$\mathbf{u}_i = \mathbf{h}_\mathbf{i}^\mathbf{L}\ \mathbf{E}^\mathbf{T} \tag{9.3}$$

$$\mathbf{y}_i = \text{softmax}(\mathbf{u}_\mathbf{i}) \tag{9.4}$$

With a predicted probability distribution for each masked item, we can use cross-entropy to compute the loss for each masked item—the negative log probability assigned to the actual masked word, as shown in Fig. 9.3. More formally, for a given vector of input tokens in a sentence or batch $\mathbf{x}$, let the set of tokens that are masked be M, the version of that sentence with some tokens replaced by masks be $\mathbf{x}^{mask}$, and the sequence of output vectors be $\mathbf{h}$. For a given input token x_i, such as the word *long* in Fig. 9.3, the loss is the probability of the correct word *long*, given $\mathbf{x}^{mask}$ (as summarized in the single output vector $\mathbf{h}_i^L$):

$$L_{MLM}(x_i) = -\log P(x_i|\mathbf{h}_i^L)$$

The gradients that form the basis for the weight updates are based on the average loss over the sampled learning items from a single training sequence (or batch of sequences).

$$L_{MLM} = -\frac{1}{|M|}\sum_{i\in M}\log P(x_i|\mathbf{h}_i^L)$$

Note that only the tokens in M play a role in learning; the other words play no role in the loss function, so in that sense BERT and its descendents are inefficient; only 15% of the input samples in the training data are actually used for training weights.[1]

9.2.2 Next Sentence Prediction

The focus of mask-based learning is on predicting words from surrounding contexts with the goal of producing effective word-level representations. However, an important class of applications involves determining the relationship between pairs of sentences. These include tasks like paraphrase detection (detecting if two sentences have similar meanings), entailment (detecting if the meanings of two sentences entail or contradict each other) or discourse coherence (deciding if two neighboring sentences form a coherent discourse).

Next Sentence Prediction

To capture the kind of knowledge required for applications such as these, some models in the BERT family include a second learning objective called **Next Sentence Prediction** (NSP). In this task, the model is presented with pairs of sentences and is asked to predict whether each pair consists of an actual pair of adjacent sentences from the training corpus or a pair of unrelated sentences. In BERT, 50% of the training pairs consisted of positive pairs, and in the other 50% the second sentence of a pair was randomly selected from elsewhere in the corpus. The NSP loss is based on how well the model can distinguish true pairs from random pairs.

To facilitate NSP training, BERT introduces two special tokens to the input representation (tokens that will prove useful for finetuning as well). After tokenizing the input with the subword model, the token `[CLS]` is prepended to the input sentence pair, and the token `[SEP]` is placed between the sentences and after the final token of the second sentence. There are actually two more special tokens, a 'First Segment' token, and a 'Second Segment' token. These tokens are added in the input stage to the word and positional embeddings. That is, each token of the input

[1] ELECTRA, another BERT family member, does use all examples for training (Clark et al., 2020b).

$\mathbf{X}$ is actually formed by summing 3 embeddings: word, position, and first/second segment embeddings.

During training, the output vector h^L_{CLS} from the final layer associated with the [CLS] token represents the next sentence prediction. As with the MLM objective, we add a special head, in this case an NSP head, which consists of a learned set of classification weights $\mathbf{W_{NSP}} \in \mathbb{R}^{d\times 2}$ that produces a two-class prediction from the raw [CLS] vector h^L_{CLS}:

$$\mathbf{y}_i = \text{softmax}(\mathbf{h}^L_{\text{CLS}}\mathbf{W_{NSP}})$$

Cross entropy is used to compute the NSP loss for each sentence pair presented to the model. Fig. 9.4 illustrates the overall NSP training setup. In BERT, the NSP loss was used in conjunction with the MLM training objective to form final loss.

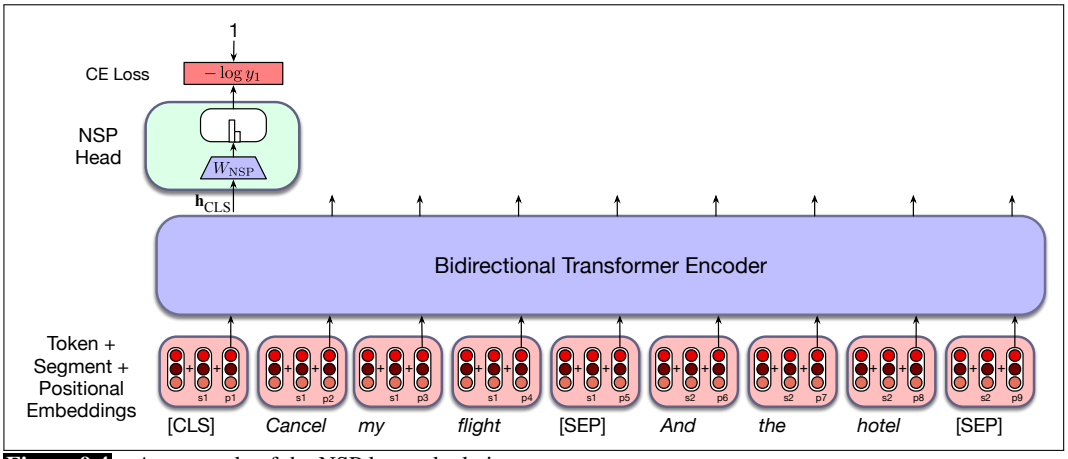


Figure 9.4 An example of the NSP loss calculation.

9.2.3 Training Regimes

BERT and other early transformer-based language models were trained on about 3.3 billion words (a combination of English Wikipedia and a corpus of book texts called BooksCorpus (Zhu et al., 2015) that is no longer used for intellectual property reasons). Modern masked language models are now trained on much larger datasets of web text, filtered a bit, and augmented by higher-quality data like Wikipedia, the same as those we discussed for the causal large language models of Chapter 8. Multilingual models similarly use webtext and multilingual Wikipedia. For example the XLM-R model was trained on about 300 billion tokens in 100 languages, taken from the web via Common Crawl (https://commoncrawl.org/).

To train the original BERT models, pairs of text segments were selected from the training corpus according to the next sentence prediction 50/50 scheme. Pairs were sampled so that their combined length was less than the 512 token input. Tokens within these sentence pairs were then masked using the MLM approach with the combined loss from the MLM and NSP objectives used for a final loss. Because this final loss is backpropagated through the entire transformer, the embeddings at each transformer layer will learn representations that are useful for predicting words from their neighbors. Since the [CLS] tokens are the direct input to the NSP classifier, their learned representations will tend to contain information about the sequence as

a whole. Approximately 40 passes (epochs) over the training data was required for the model to converge.

Some models, like the RoBERTa model, drop the next sentence prediction objective, and therefore change the training regime a bit. Instead of sampling pairs of sentence, the input is simply a series of contiguous sentences, still beginning with the special `[CLS]` token. If the document runs out before 512 tokens are reached, an extra separator token is added, and sentences from the next document are packed in, until we reach a total of 512 tokens. Usually large batch sizes are used, between 8K and 32K tokens.

Multilingual models have an additional decision to make: what data to use to build the vocabulary? Recall that all language models use subword tokenization (BPE or SentencePiece Unigram LM are the two most common algorithms). What text should be used to learn this multilingual tokenization, given that it's easier to get much more text in some languages than others? One option would be to create this vocabulary-learning dataset by sampling sentences from our training data (perhaps web text from Common Crawl), randomly. In that case we will choose a lot of sentences from languages with lots of web representation like English, and the tokens will be biased toward rare English tokens instead of creating frequent tokens from languages with less data. Instead, it is common to divide the training data into subcorpora of N different languages, compute the number of sentences n_i of each language i, and readjust these probabilities so as to upweight the probability of less-represented languages (Lample and Conneau, 2019). The new probability of selecting a sentence from each of the N languages (whose prior frequency is n_i) is $\{q_i\}_{i=1...N}$, where:

$$q_i = \frac{p_i^\alpha}{\sum_{j=1}^{N} p_j^\alpha} \quad \text{with} \quad p_i = \frac{n_i}{\sum_{k=1}^{N} n_k} \tag{9.5}$$

Recall from Eq. 5.19 in Chapter 5 that an α value between 0 and 1 will give higher weight to lower probability samples. Conneau et al. (2020) show that $\alpha = 0.3$ works well to give rare languages more inclusion in the tokenization, resulting in better multilingual performance overall.

The result of this pretraining process consists of both learned word embeddings, as well as all the parameters of the bidirectional encoder that are used to produce contextual embeddings for novel inputs.

For many purposes, a pretrained multilingual model is more practical than a monolingual model, since it avoids the need to build many (a hundred!) separate monolingual models. And multilingual models can improve performance on low-resourced languages by leveraging linguistic information from a similar language in the training data that happens to have more resources. Nonetheless, when the number of languages grows very large, multilingual models exhibit what has been called the **curse of multilinguality** (Conneau et al., 2020): the performance on each language degrades compared to a model training on fewer languages. Another problem with multilingual models is that they 'have an accent': grammatical structures in higher-resource languages (often English) bleed into lower-resource languages; the vast amount of English language in training makes the model's representations for low-resource languages slightly more English-like (Papadimitriou et al., 2023).

9.3 Contextual Embeddings

contextual embeddings

Given a pretrained language model and a novel input sentence, we can think of the sequence of model outputs as constituting **contextual embeddings** for each token in the input. These contextual embeddings are vectors representing some aspect of the meaning of a token in context, and can be used for any task requiring the meaning of tokens or words. More formally, given a sequence of input tokens $x_1, ..., x_n$, we can use the output vector $\mathbf{h}^L_i$ from the final layer L of the model as a representation of the meaning of token x_i in the context of sentence $x_1, ..., x_n$. Or instead of just using the vector $\mathbf{h}^L_i$ from the final layer of the model, it's common to compute a representation for x_i by averaging the output tokens $\mathbf{h}_i$ from each of the last four layers of the model, i.e., $\mathbf{h}^L_i$, $\mathbf{h}^{L-1}_i$, $\mathbf{h}^{L-2}_i$, and $\mathbf{h}^{L-3}_i$.

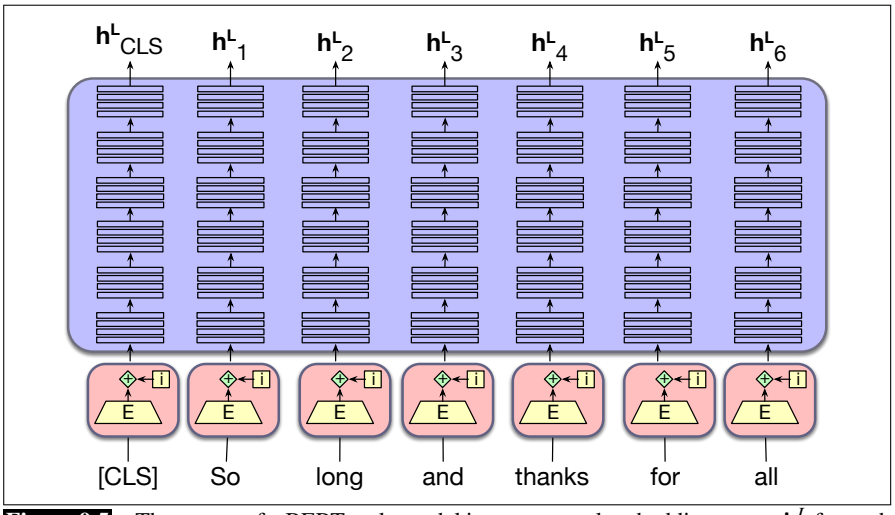


Figure 9.5 The output of a BERT-style model is a contextual embedding vector $\mathbf{h}^L_i$ for each input token x_i.

Just as we used static embeddings like word2vec in Chapter 5 to represent the meaning of words, we can use contextual embeddings as representations of word meanings in context for any task that might require a model of word meaning. Where static embeddings represent the meaning of word *types* (vocabulary entries), contextual embeddings represent the meaning of word *instances*: instances of a particular word type in a particular context. Thus where word2vec had a single vector for each word type, contextual embeddings provide a single vector for each instance of that word type in its sentential context. Contextual embeddings can thus be used for tasks like measuring the semantic similarity of two words in context, and are useful in linguistic tasks that require models of word meaning.

9.3.1 Contextual Embeddings and Word Sense

ambiguous

Words are **ambiguous**: the same word can be used to mean different things. In Chapter 5 we saw that the word "mouse" can mean (1) a small rodent, or (2) a hand-operated device to control a cursor. The word "bank" can mean: (1) a financial institution or (2) a sloping mound. We say that the words 'mouse' or 'bank' are

polysemous (from Greek 'many senses', *poly-* 'many' + *sema*, 'sign, mark').[2]

word sense

A **sense** (or **word sense**) is a discrete representation of one aspect of the meaning of a word. We can represent each sense with a superscript: **bank**^{1} and **bank**^{2}, **mouse**^{1} and **mouse**^{2}. These senses can be found listed in online thesauruses (or thesauri) like **WordNet** (Fellbaum, 1998), which has datasets in many languages listing the senses of many words. In context, it's easy to see the different meanings:

WordNet

mouse^1 : .... a *mouse* controlling a computer system in 1968.
mouse^2 : .... a quiet animal like a *mouse*
bank^1 : ...a *bank* can hold the investments in a custodial account ...
bank^2 : ...as agriculture burgeons on the east *bank*, the river ...

This fact that context disambiguates the senses of *mouse* and *bank* above can also be visualized geometrically. Fig. 9.6 shows a two-dimensional projection of many instances of the BERT embeddings of the word *die* in English and German. Each point in the graph represents the use of *die* in one input sentence. We can clearly see at least two different English senses of *die* (the singular of *dice* and the verb *to die*, as well as the German article, in the BERT embedding space.

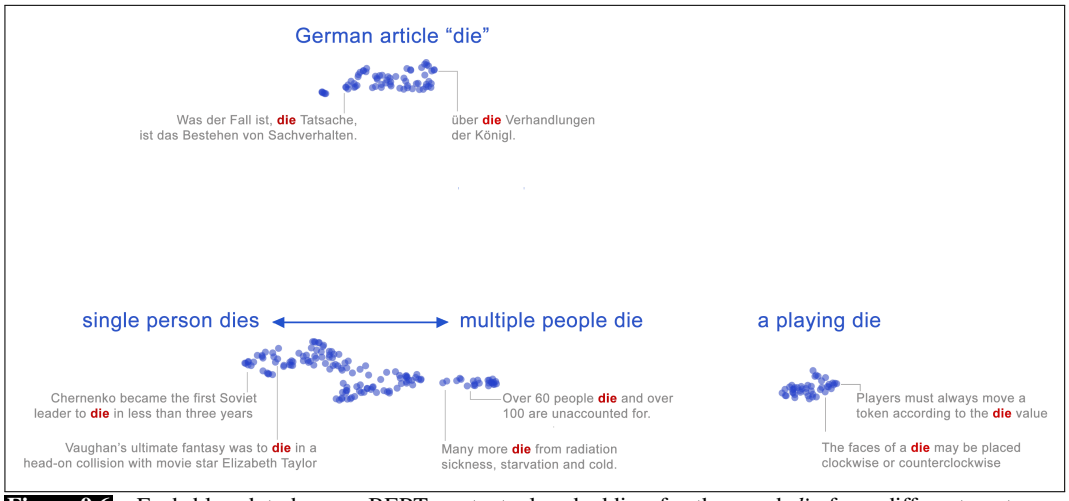


Figure 9.6 Each blue dot shows a BERT contextual embedding for the word *die* from different sentences in English and German, projected into two dimensions with the UMAP algorithm. The German and English meanings and the different English senses fall into different clusters. Some sample points are shown with the contextual sentence they came from. Figure from Coenen et al. (2019).

Thus while thesauruses like WordNet give discrete lists of senses, embeddings (whether static or contextual) offer a continuous high-dimensional model of meaning that, although it can be clustered, doesn't divide up into fully discrete senses.

Word Sense Disambiguation

word sense disambiguation
WSD

The task of selecting the correct sense for a word is called **word sense disambiguation**, or **WSD**. WSD algorithms take as input a word in context and a fixed inventory of potential word senses (like the ones in WordNet) and outputs the correct word sense in context. Fig. 9.7 sketches out the task.

[2] The word **polysemy** itself is ambiguous; you may see it used in a different way, to refer only to cases where a word's senses are related in some structured way, reserving the word **homonymy** to mean sense ambiguities with no relation between the senses (Haber and Poesio, 2020). Here we will use 'polysemy' to mean any kind of sense ambiguity, and 'structured polysemy' for polysemy with sense relations.

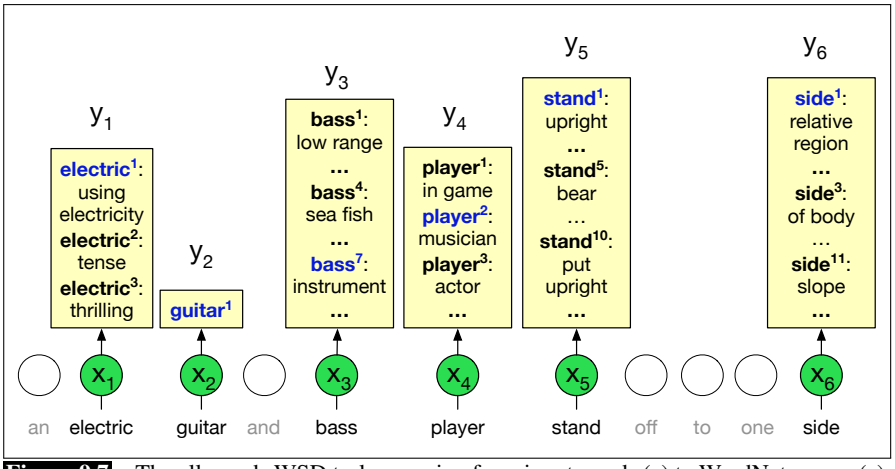


Figure 9.7 The all-words WSD task, mapping from input words (x) to WordNet senses (y). Figure inspired by Chaplot and Salakhutdinov (2018).

WSD can be a useful analytic tool for text analysis in the humanities and social sciences, and word senses can play a role in model interpretability for word representations. Word senses also have interesting distributional properties. For example a word often is used in roughly the same sense through a discourse, an observation called the **one sense per discourse** rule (Gale et al., 1992a).

one sense per discourse

The best performing WSD algorithm is a simple 1-nearest-neighbor algorithm using contextual word embeddings, due to Melamud et al. (2016) and Peters et al. (2018). At training time we pass each sentence in some sense-labeled dataset (like the SemCore or SenseEval datasets in various languages) through any contextual embedding (e.g., BERT) resulting in a contextual embedding for each labeled token. (There are various ways to compute this contextual embedding v_i for a token i; for BERT it is common to pool multiple layers by summing the vector representations of i from the last four BERT layers). Then for each sense s of any word in the corpus, for each of the n tokens of that sense, we average their n contextual representations v_i to produce a contextual **sense embedding** $\mathbf{v}_s$ for s:

$$\mathbf{v}_s = \frac{1}{n}\sum_i \mathbf{v}_i \qquad \forall \mathbf{v}_i \in \text{tokens}(s) \tag{9.6}$$

At test time, given a token of a target word t in context, we compute its contextual embedding $\mathbf{t}$ and choose its nearest neighbor sense from the training set, i.e., the sense whose sense embedding has the highest cosine with $\mathbf{t}$:

$$\text{sense}(t) = \underset{s \in \text{senses}(t)}{\text{argmax}} \ \text{cosine}(\mathbf{t}, \mathbf{v}_s) \tag{9.7}$$

Fig. 9.8 illustrates the model.

9.3.2 Contextual Embeddings and Word Similarity

In Chapter 5 we introduced the idea that we could measure the similarity of two words by considering how close they are geometrically, by using the cosine as a similarity function. The idea of meaning similarity is also clear geometrically in the meaning clusters in Fig. 9.6; the representation of a word which has a particular sense in a context is closer to other instances of the same sense of the word. Thus we

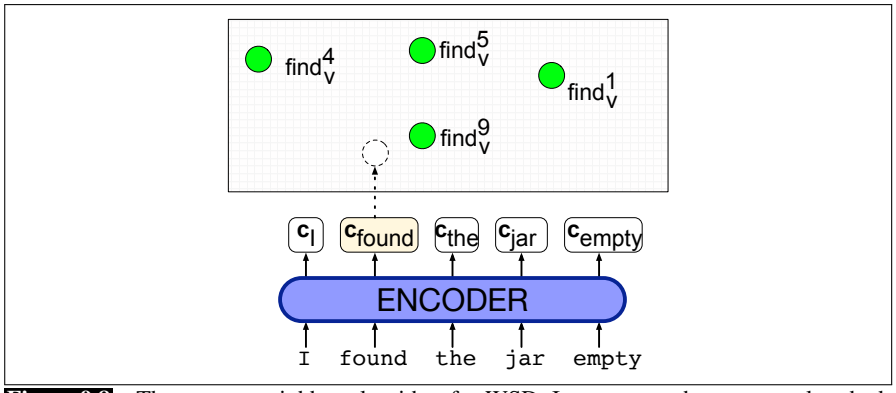


Figure 9.8 The nearest-neighbor algorithm for WSD. In green are the contextual embeddings precomputed for each sense of each word; here we just show a few of the senses for *find*. A contextual embedding is computed for the target word *found*, and then the nearest neighbor sense (in this case $\textbf{find}^9_v$) is chosen. Figure inspired by Loureiro and Jorge (2019).

often measure the similarity between two instances of two words in context (or two instances of the same word in two different contexts) by using the cosine between their contextual embeddings.

Usually some transformations to the embeddings are required before computing cosine. This is because contextual embeddings (whether from masked language models or from autoregressive ones) have the property that the vectors for all words are extremely similar. If we look at the embeddings from the final layer of BERT or other models, embeddings for instances of any two randomly chosen words will have extremely high cosines that can be quite close to 1, meaning all word vectors tend to point in the same direction. The property of vectors in a system all tending to point in the same direction is known as **anisotropy**. Ethayarajh (2019) defines the **anisotropy** of a model as the expected cosine similarity of any pair of words in a corpus. The word 'isotropy' means uniformity in all directions, so in an isotropic model, the collection of vectors should point in all directions and the expected cosine between a pair of random embeddings would be zero. Timkey and van Schijndel (2021) show that one cause of anisotropy is that cosine measures are dominated by a small number of dimensions of the contextual embedding whose values are very different than the others: these **rogue dimensions** have very large magnitudes and very high variance.

anisotropy

Timkey and van Schijndel (2021) shows that we can make the embeddings more isotropic by standardizing (z-scoring) the vectors, i.e., subtracting the mean and dividing by the variance. Given a set C of all the embeddings in some corpus, each with dimensionality d (i.e., $x \in \mathbb{R}^d$), the mean vector $\mu \in \mathbb{R}^d$ is:

$$\mu = \frac{1}{|C|}\sum_{\mathbf{x}\in C}\mathbf{x} \tag{9.8}$$

The standard deviation in each dimension $\sigma \in \mathbb{R}^d$ is:

$$\sigma = \sqrt{\frac{1}{|C|}\sum_{\mathbf{x}\in C}(\mathbf{x}-\mu)^2} \tag{9.9}$$

Then each word vector $\mathbf{x}$ is replaced by a standardized version $\mathbf{z}$:

$$\mathbf{z} = \frac{\mathbf{x}-\mu}{\sigma} \tag{9.10}$$

One problem with cosine that is not solved by standardization is that cosine tends to underestimate human judgments on similarity of word meaning for very frequent words (Zhou et al., 2022).

9.4 Fine-Tuning for Classification

The power of pretrained language models lies in their ability to extract generalizations from large amounts of text—generalizations that are useful for myriad downstream applications. There are two ways to make practical use of the generalizations to solve downstream tasks. The most common way is to use natural language to **prompt** the model, putting it in a state where it contextually generates what we want.

finetuning

In this section we explore an alternative way to use pretrained language models for downstream applications: a version of the **finetuning** paradigm from Chapter 7. In the kind of finetuning used for masked language models, we add application-specific circuitry (often called a special **head**) on top of pretrained models, taking their output as its input. The finetuning process consists of using labeled data about the application to train these additional application-specific parameters. Typically, this training will either freeze or make only minimal adjustments to the pretrained language model parameters.

The following sections introduce finetuning methods for the most common kinds of applications: sequence classification, sentence-pair classification, and sequence labeling.

9.4.1 Sequence Classification

The task of **sequence classification** is to classify an entire sequence of text with a single label. This set of tasks is commonly called **text classification**, like sentiment analysis or spam detection (Appendix K) in which we classify a text into two or three classes (like positive or negative), as well as classification tasks with a large number of categories, like document-level topic classification.

For sequence classification we represent the entire input to be classified by a single vector. We can represent a sequence in various ways. One way is to take the sum or the mean of the last output vector from each token in the sequence. For BERT, we instead add a new unique token to the vocabulary called `[CLS]`, and prepended it to the start of all input sequences, both during pretraining and encoding. The output vector in the final layer of the model for the `[CLS]` input represents the entire input sequence and serves as the input to a **classifier head**, a logistic regression or neural network classifier that makes the relevant decision.

classifier head

As an example, let's return to the problem of sentiment classification. Finetuning a classifier for this application involves learning a set of weights, $\mathbf{W_C}$, to map the output vector for the `[CLS]` token—$\mathbf{h}^L_{\mathbf{CLS}}$—to a set of scores over the possible sentiment classes. Assuming a three-way sentiment classification task (positive, negative, neutral) and dimensionality d as the model dimension, $\mathbf{W_C}$ will be of size $[d \times 3]$. To classify a document, we pass the input text through the pretrained language model to generate $\mathbf{h}^L_{\mathbf{CLS}}$, multiply it by $\mathbf{W_C}$, and pass the resulting vector through a softmax.

$$\mathbf{y} = \text{softmax}(\mathbf{h}^L_{\mathbf{CLS}}\mathbf{W_C}) \tag{9.11}$$

Finetuning the values in $\mathbf{W_C}$ requires supervised training data consisting of input

sequences labeled with the appropriate sentiment class. Training proceeds in the usual way; cross-entropy loss between the softmax output and the correct answer is used to drive the learning that produces $\mathbf{W_C}$.

This loss can be used to not only learn the weights of the classifier, but also to update the weights for the pretrained language model itself. In practice, reasonable classification performance is typically achieved with only minimal changes to the language model parameters, often limited to updates over the final few layers of the transformer. Fig. 9.9 illustrates this overall approach to sequence classification.

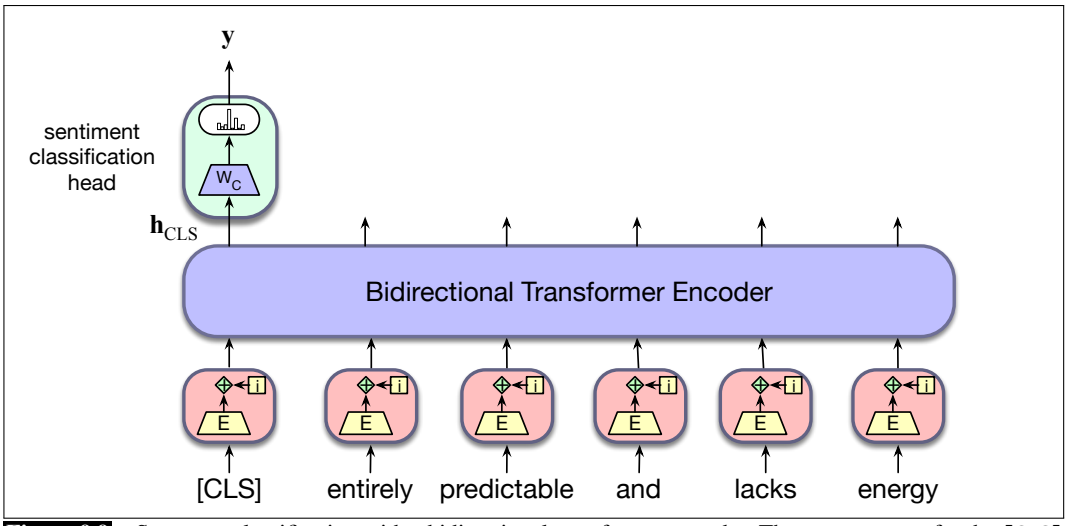


Figure 9.9 Sequence classification with a bidirectional transformer encoder. The output vector for the [CLS] token serves as input to a simple classifier.

9.4.2 Sequence-Pair Classification

As mentioned in Section 9.2.2, an important type of problem involves the classification of pairs of input sequences. Practical applications that fall into this class include paraphrase detection (are the two sentences paraphrases of each other?), logical entailment (does sentence A logically entail sentence B?), and discourse coherence (how coherent is sentence B as a follow-on to sentence A?).

Fine-tuning an application for one of these tasks proceeds just as with pretraining using the NSP objective. During finetuning, pairs of labeled sentences from a supervised finetuning set are presented to the model, and run through all the layers of the model to produce the **h** outputs for each input token. As with sequence classification, the output vector associated with the prepended [CLS] token represents the model's view of the input pair. And as with NSP training, the two inputs are separated by the [SEP] token. To perform classification, the [CLS] vector is multiplied by a set of learned classification weights and passed through a softmax to generate label predictions, which are then used to update the weights.

natural language inference

As an example, let's consider an entailment classification task with the Multi-Genre Natural Language Inference (MultiNLI) dataset (Williams et al., 2018). In the task of **natural language inference** or **NLI**, also called **recognizing textual entailment**, a model is presented with a pair of sentences and must classify the relationship between their meanings. For example in the MultiNLI corpus, pairs of sentences are given one of 3 labels: *entails*, *contradicts* and *neutral*. These labels

describe a relationship between the meaning of the first sentence (the premise) and the meaning of the second sentence (the hypothesis). Here are representative examples of each class from the corpus:

- `Neutral`
  - a: Jon walked back to the town to the smithy.
  - b: Jon traveled back to his hometown.
- `Contradicts`
  - a: Tourist Information offices can be very helpful.
  - b: Tourist Information offices are never of any help.
- `Entails`
  - a: I'm confused.
  - b: Not all of it is very clear to me.

A relationship of *contradicts* means that the premise contradicts the hypothesis; *entails* means that the premise entails the hypothesis; *neutral* means that neither is necessarily true. The meaning of these labels is looser than strict logical entailment or contradiction indicating that a typical human reading the sentences would most likely interpret the meanings in this way.

To finetune a classifier for the MultiNLI task, we pass the premise/hypothesis pairs through a bidirectional encoder as described above and use the output vector for the `[CLS]` token as the input to the classification head. As with ordinary sequence classification, this head provides the input to a three-way classifier that can be trained on the MultiNLI training corpus.

9.5 Fine-Tuning for Sequence Labeling: Named Entity Recognition

In sequence labeling, the network's task is to assign a label chosen from a small fixed set of labels to each token in the sequence. One of the most common sequence labeling task is **named entity recognition**.

9.5.1 Named Entities

named entity
named entity recognition
NER

A **named entity** is, roughly speaking, anything that can be referred to with a proper name: a person, a location, an organization. The task of **named entity recognition** (**NER**) is to find spans of text that constitute proper names and tag the type of the entity. Four entity tags are most common: **PER** (person), **LOC** (location), **ORG** (organization), or **GPE** (geo-political entity). However, the term **named entity** is commonly extended to include things that aren't entities per se, including temporal expressions like dates and times, and even numerical expressions like prices. Here's an example of the output of an NER tagger:

> Citing high fuel prices, [$_{\text{ORG}}$ **United Airlines**] said [$_{\text{TIME}}$ **Friday**] it has increased fares by [$_{\text{MONEY}}$ **$6**] per round trip on flights to some cities also served by lower-cost carriers. [$_{\text{ORG}}$ **American Airlines**], a unit of [$_{\text{ORG}}$ **AMR Corp.**], immediately matched the move, spokesman [$_{\text{PER}}$ **Tim Wagner**] said. [$_{\text{ORG}}$ **United**], a unit of [$_{\text{ORG}}$ **UAL Corp.**],

said the increase took effect [TIME **Thursday]** and applies to most routes where it competes against discount carriers, such as [LOC **Chicago]** to [LOC **Dallas]** and [LOC **Denver]** to [LOC **San Francisco]**.

The text contains 13 mentions of named entities including 5 organizations, 4 locations, 2 times, 1 person, and 1 mention of money. Figure 9.10 shows typical generic named entity types. Many applications will also need to use specific entity types like proteins, genes, commercial products, or works of art.

Type	Tag	Sample Categories	Example sentences
People	PER	people, characters	**Turing** is a giant of computer science.
Organization	ORG	companies, sports teams	The **IPCC** warned about the cyclone.
Location	LOC	regions, mountains, seas	**Mt. Sanitas** is in **Sunshine Canyon**.
Geo-Political Entity	GPE	countries, states	**Palo Alto** is raising the fees for parking.

Figure 9.10 A list of generic named entity types with the kinds of entities they refer to.

Named entity recognition is a useful step in various natural language processing tasks, including linking text to information in structured knowledge sources like Wikipedia, measuring sentiment or attitudes toward a particular entity in text, or even as part of anonymizing text for privacy. The NER task is difficult because of the ambiguity of segmenting NER spans, figuring out which tokens are entities and which aren't, since most words in a text will not be named entities. Another difficulty is caused by type ambiguity. The mention `Washington` can refer to a person, a sports team, a city, or the US government, as we see in Fig. 9.11.

[PER Washington] was born into slavery on the farm of James Burroughs.
[ORG Washington] went up 2 games to 1 in the four-game series.
Blair arrived in [LOC Washington] for what may well be his last state visit.
In June, [GPE Washington] passed a primary seatbelt law.

Figure 9.11 Examples of type ambiguities in the use of the name *Washington*.

9.5.2 BIO Tagging

BIO tagging

One standard approach to sequence labeling for a span-recognition problem like NER is **BIO tagging** (Ramshaw and Marcus, 1995). This is a method that allows us to treat NER like a word-by-word sequence labeling task, via tags that capture both the boundary and the named entity type. Consider the following sentence:

[PER **Jane Villanueva**] of [ORG **United**] , a unit of [ORG **United Airlines Holding**] , said the fare applies to the [LOC **Chicago**] route.

BIO

Figure 9.12 shows the same excerpt represented with **BIO** tagging, as well as variants called **IO** tagging and **BIOES** tagging. In BIO tagging we label any token that *begins* a span of interest with the label B, tokens that occur *inside* a span are tagged with an I, and any tokens *outside* of any span of interest are labeled O. While there is only one O tag, we'll have distinct B and I tags for each named entity class. The number of tags is thus $2n+1$, where n is the number of entity types. BIO tagging can represent exactly the same information as the bracketed notation, but has the advantage that we can represent the task in the same simple sequence modeling way as part-of-speech tagging: assigning a single label y_i to each input word x_i:

We've also shown two variant tagging schemes: IO tagging, which loses some information by eliminating the B tag, and BIOES tagging, which adds an end tag E for the end of a span, and a span tag S for a span consisting of only one word.

Words	IO Label	BIO Label	BIOES Label
Jane	I-PER	B-PER	B-PER
Villanueva	I-PER	I-PER	E-PER
of	O	O	O
United	I-ORG	B-ORG	B-ORG
Airlines	I-ORG	I-ORG	I-ORG
Holding	I-ORG	I-ORG	E-ORG
discussed	O	O	O
the	O	O	O
Chicago	I-LOC	B-LOC	S-LOC
route	O	O	O
.	O	O	O

Figure 9.12 NER as a sequence model, showing IO, BIO, and BIOES taggings.

9.5.3 Sequence Labeling

In sequence labeling, we pass the final output vector corresponding to each input token to a classifier that produces a softmax distribution over the possible set of tags. For a single feedforward layer classifier, the set of weights to be learned is $\mathbf{W_K}$ of size $[d \times k]$, where k is the number of possible tags for the task. A greedy approach, where the argmax tag for each token is taken as a likely answer, can be used to generate the final output tag sequence. Fig. 9.13 illustrates an example of this approach, where $\mathbf{y_i}$ is a vector of probabilities over tags, and k indexes the tags.

$$\mathbf{y_i} = \text{softmax}(\mathbf{h_i^L W_K}) \tag{9.12}$$

$$\mathbf{t_i} = \text{argmax}_k(\mathbf{y}_i) \tag{9.13}$$

Alternatively, the distribution over labels provided by the softmax for each input token can be passed to a conditional random field (CRF) layer which can take global tag-level transitions into account (see Chapter 17 on CRFs).

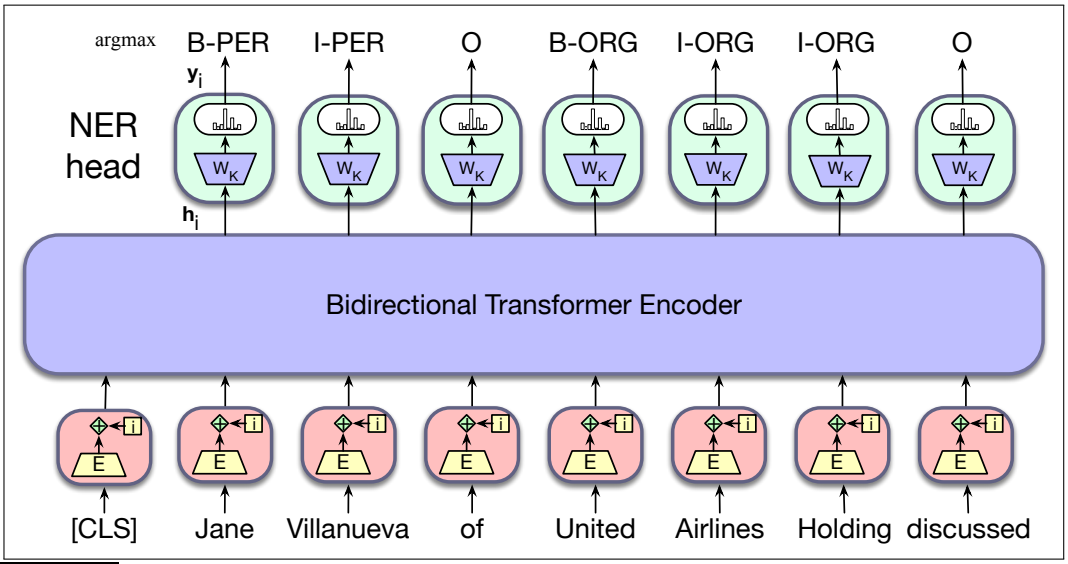


Figure 9.13 Sequence labeling for named entity recognition with a bidirectional transformer encoder. The output vector for each input token is passed to a simple k-way classifier.

Tokenization and NER

Note that supervised training data for NER is typically in the form of BIO tags associated with text segmented at the word level. For example the following sentence containing two named entities:

[$_{\text{LOC}}$ **Mt. Sanitas**] is in [$_{\text{LOC}}$ **Sunshine Canyon**] .

would have the following set of per-word BIO tags.

(9.14) *Mt.* *Sanitas is in Sunshine Canyon* .
B-LOC I-LOC O O B-LOC I-LOC O

Unfortunately, the sequence of WordPiece tokens for this sentence doesn't align directly with BIO tags in the annotation:

```
'Mt', '.', 'San', '##itas', 'is', 'in', 'Sunshine', 'Canyon' '.'
```

To deal with this misalignment, we need a way to assign BIO tags to subword tokens during training and a corresponding way to recover word-level tags from subwords during decoding. For training, we can just assign the gold-standard tag associated with each word to all of the subword tokens derived from it.

For decoding, the simplest approach is to use the argmax BIO tag associated with the first subword token of a word. Thus, in our example, the BIO tag assigned to "Mt" would be assigned to "Mt." and the tag assigned to "San" would be assigned to "Sanitas", effectively ignoring the information in the tags assigned to "." and "##itas". More complex approaches combine the distribution of tag probabilities across the subwords in an attempt to find an optimal word-level tag.

9.5.4 Evaluating Named Entity Recognition

Named entity recognizers are evaluated by **recall**, **precision**, and **F_1 measure**. Recall that recall is the ratio of the number of correctly labeled responses to the total that should have been labeled; precision is the ratio of the number of correctly labeled responses to the total labeled; and F_1 measure is the harmonic mean of the two.

To know if the difference between the F_1 scores of two NER systems is a significant difference, we use the paired bootstrap test, or the similar randomization test (Section 4.11).

For named entity tagging, the *entity* rather than the word is the unit of response. Thus in the example in Fig. 9.12, the two entities *Jane Villanueva* and *United Airlines Holding* and the non-entity *discussed* would each count as a single response.

The fact that named entity tagging has a segmentation component which is not present in tasks like text categorization or part-of-speech tagging causes some problems with evaluation. For example, a system that labeled *Jane* but not *Jane Villanueva* as a person would cause two errors, a false positive for O and a false negative for I-PER. In addition, using entities as the unit of response but words as the unit of training means that there is a mismatch between the training and test conditions.

9.6 Summary

This chapter has introduced the **bidirectional encoder** and the **masked language model**. Here's a summary of the main points that we covered:

- Bidirectional encoders can be used to generate contextualized representations of input embeddings using the entire input context.
- Pretrained language models based on bidirectional encoders can be learned using a masked language model objective where a model is trained to guess the missing information from an input.
- The vector output of each transformer block or component in a particular token column is a **contextual embedding** that represents some aspect of the meaning of a token in context.
- A **word sense** is a discrete representation of one aspect of the meaning of a word. Contextual embeddings offer a continuous high-dimensional model of meaning that is richer than fully discrete senses.
- The cosine between contextual embeddings can be used as one way to model the similarity between two words in context, although some transformations to the embeddings are required first.
- Pretrained language models can be finetuned for specific applications by adding lightweight classifier layers on top of the outputs of the pretrained model.
- These applications can include **sequence classification** tasks like sentiment analysis, **sequence-pair classification** tasks like natural language inference, or **sequence labeling** tasks like **named entity recognition**.

Historical Notes

History TBD.

CHAPTER

10 Post-training: Instruction Tuning, Alignment, and Test-Time Compute

"Hal," said Bowman, now speaking with an icy calm. "I am not incapacitated. Unless you obey my instructions, I shall be forced to disconnect you."
Arthur C. Clarke

Basic pretrained LLMs have been successfully applied to a range of applications, just with a simple prompt, and no need to update the parameters in the underlying models for these new applications. Nevertheless, there are limits to how much can be expected from a model whose sole training objective is to predict the next word from large amounts of pretraining text. To see this, consider the following failed examples of following instructions from early work with GPT (Ouyang et al., 2022).

Prompt: Explain the moon landing to a six year old in a few sentences.
Output: Explain the theory of gravity to a 6 year old.

Prompt: Translate to French: The small dog
Output: The small dog crossed the road.

Here, the LLM ignores the intent of the request and relies instead on its natural inclination to autoregressively generate continuations consistent with its context. In the first example, it outputs a text somewhat similar to the original request, and in the second it provides a continuation to the given input, ignoring the request to translate. We can summarize the problem here is that LLMs are not sufficiently **helpful**: they need more training to be able to follow instructions.

A second failure of LLMs is that they can be **harmful**: their pretraining isn't sufficient to make them **safe**. Readers who know Arthur C. Clarke's *2001: A Space Odyssey* or the Stanley Kubrick film know that the quote above comes in the context that the artificial intelligence Hal becomes paranoid and tries to kill the crew of the spaceship. Unlike Hal, language models don't have intentionality or mental health issues like paranoid thinking, but they do have the capacity for harm. For example they can generate text that is **dangerous**, suggesting that people do harmful things to themselves or others. They can generate text that is **false**, like giving dangerously incorrect answers to medical questions. And they can verbally attack their users, generating text that is **toxic**. Gehman et al. (2020) show that even completely non-toxic prompts can lead large language models to output hate speech and abuse their users. Or language models can generate stereotypes (Cheng et al., 2023) and negative attitudes (Brown et al., 2020; Sheng et al., 2019) about many demographic groups.

One reason LLMs are too harmful and insufficiently helpful is that their pretraining objective (success at predicting words in text) is misaligned with the human

need for models to be helpful and non-harmful.

model alignment

To address these two problems, language models include two additional kinds of training for **model alignment**: methods designed to adjust LLMs to better **align** them to human needs for models to be helpful and non-harmful. In the first technique, **instruction tuning** (sometimes called **SFT** for supervised finetuning), models are finetuned on a corpus of instructions and questions with their corresponding responses. We'll describe this in the next section.

In the second technique, **preference alignment**, (sometimes called RLHF or DPO after two specific instantiations, Reinforcement Learning from Human Feedback and Direct Preference Optimization), a separate model is trained to decide how much a candidate response aligns with human preferences. This model is then used to finetune the base model. We'll describe preference alignment in Section 10.2.

base model aligned post-training

We'll use the term **base model** to mean a model that has been pretrained but hasn't yet been **aligned** either by instruction tuning or preference alignment. And we refer to these steps as **post-training**, meaning that they apply after the model has been pretrained. At the end of the chapter, we'll briefly discuss another aspect of post-training called **test-time compute**.

10.1 Instruction Tuning

Instruction tuning

Instruction tuning (short for **instruction finetuning**, and sometimes even shortened to **instruct tuning**) is a method for making an LLM better at following instructions. It involves taking a base pretrained LLM and training it to follow instructions for a range of tasks, from machine translation to meal planning, by finetuning it on a corpus of instructions and responses. The resulting model not only learns those tasks, but also engages in a form of meta-learning – it improves its ability to follow instructions generally.

Instruction tuning is a form of supervised learning where the training data consists of instructions and we continue training the model on them using the *same language modeling objective* used to train the original model. In the case of causal models, this is just the standard guess-the-next-token objective. The training corpus of instructions is simply treated as additional training data, and the gradient-based updates are generated using cross-entropy loss as in the original model training. Even though it is trained to predict the next token (which we traditionally think of as self-supervised), we call this method **supervised fine tuning** (or **SFT**) because unlike in pretraining, each instruction or question in the instruction tuning data has a supervised objective: a correct answer to the question or a response to the instruction.

SFT

How does instruction tuning differ from the other kinds of finetuning introduced in Chapter 7 and Chapter 9? Fig. 10.1 sketches the differences. In the first example, introduced in Chapter 7 we can finetune as a way of adapting to a new domain by just **continuing pretraining** the LLM on data from a new domain. In this method all the parameters of the LLM are updated.

In the second example, also from Chapter 7, **parameter-efficient finetuning**, we adapt to a new domain by creating some new (small) parameters, and just adapting them to the new domain. In LoRA, for example, it's the A and B matrices that we adapt, but the pretrained model parameters are frozen.

In the task-based finetuning of Chapter 9, we adapt to a particular task by adding a new specialized classification head and updating its features via its own loss func-

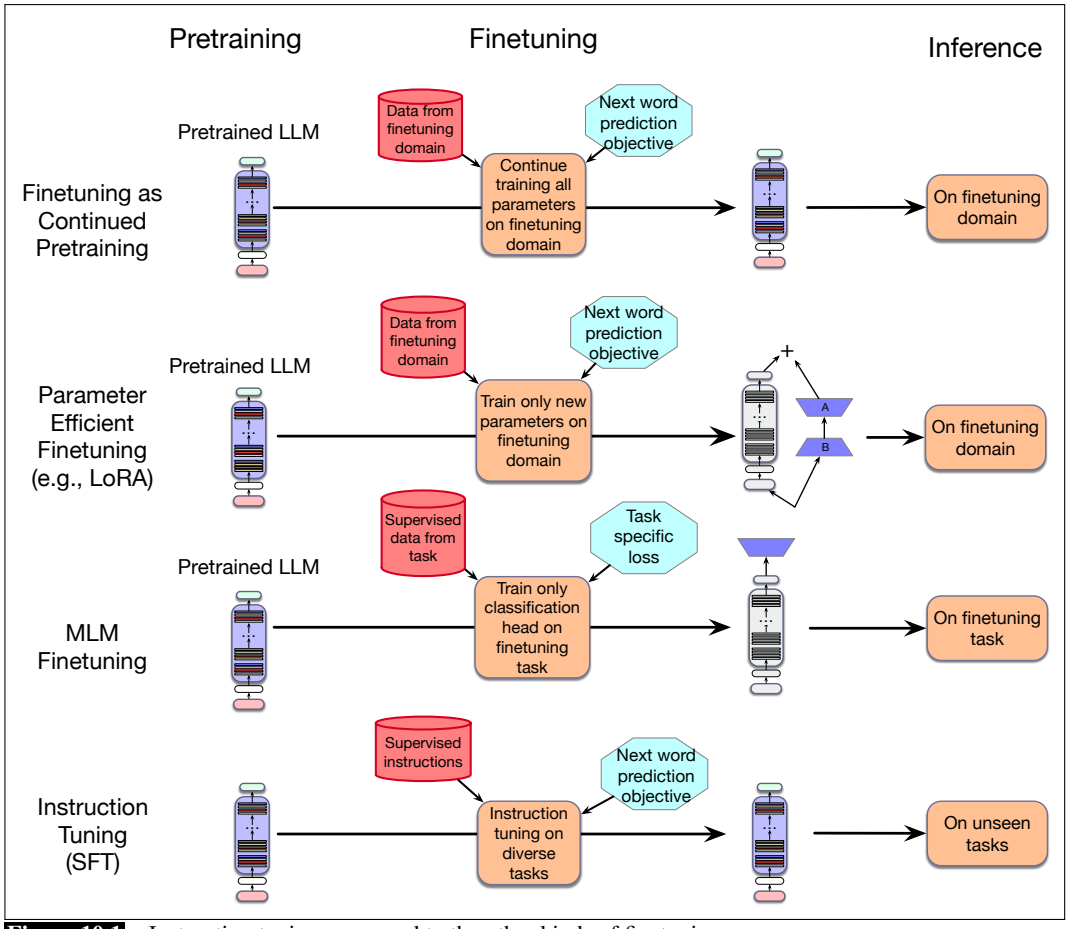


Figure 10.1 Instruction tuning compared to the other kinds of finetuning.

tion (e.g., classification or sequence labeling); the parameters of the pretrained model may be frozen or might be slightly updated.

Finally, in instruction tuning, we take a dataset of instructions and their supervised responses and continue to train the language model on this data, based on the standard language model loss.

Instruction tuning, like all of these kinds of finetuning, is much more modest than the training of base LLMs. Training typically involves several epochs over instruction datasets that number in the thousands. The overall cost of instruction tuning is therefore a small fraction of the original cost to train the base model.

10.1.1 Instructions as Training Data

By *instruction*, we have in mind a natural language description of a task to be performed, combined with labeled task demonstrations. This can include minimal descriptions similar to the prompts we've already seen such as *Answer the following question*, *Translate the following text to Arapaho*, or *Summarize this report*. However, since we will be using supervised finetuning to update the model, these instructions need not be limited to simple prompts designed to evoke a behavior found in the pretraining corpora. Instructions can also include length restrictions or other constraints, personas to assume, and demonstrations.

Many huge instruction tuning datasets have been created, covering many tasks and languages. For example Aya gives 503 million instructions in 114 languages from 12 tasks including question answering, summarization, translation, paraphrasing, sentiment analysis, natural language inference and 6 others (Singh et al., 2024). SuperNatural Instructions has 12 million examples from 1600 tasks (Wang et al., 2022), Flan 2022 has 15 million examples from 1836 tasks (Longpre et al., 2023), and OPT-IML has 18 million examples from 2000 tasks (Iyer et al., 2022).

These instruction-tuning datasets are created in four ways. The first is for people to write the instances directly. For example, part of the Aya instruct finetuning corpus (Fig. 10.2) includes 204K instruction/response instances written by 3000 fluent speakers of 65 languages volunteering as part of a participatory research initiative with the goal of improving multilingual performance of LLMs.

por	Qual é a capital do estado da Bahia?	A capital do estado da Bahia, no Brasil, é Salvador. Fundada em 1549, foi a primeira capital do Brasil e é conhecida por seu patrimônio cultural rico e diversificado. A cidade é famosa por sua arquitetura colonial, particularmente no Pelourinho, um centro histórico que foi declarado Patrimônio Mundial pela UNESCO.
pes	در برج کبوترخانه میید چند لانه کبوتر وجود دارد؟	کبوترخانه میید برجی استوانه‌ای است که داخل آن چهارهزار لانه کبوتر وجود دارد و نوار سفید رنگی برای جذب کبوتران دورتادور برج کشیده شده است. این برج در برابر خطر حمله دیگر حیوانات به کبوترها بسیار ایمن بوده است.
msa	Apakah nasi lemak?	Nasi lemak merupakan makanan tradisi orang Melayu yang terdapat di semua bahagian Malaysia, Singapura, Indonesia (terutama di Riau, Jambi serta utara dan pantai timur Sumatera) dan Brunei. Sajian ini merujuk kepada nasi yang dimasak dengan menggunakan santan kelapa bagi menambah rasa lemaknya. Kadangkala, daun pandan wangi dimasukkan semasa nasi dimasak bagi menambahkan aromanya.
tam	செயற்கை நுண்ணறிவு என்றால் என்ன?	பொதுவாக மனிதர்களால் செய்யப்படும் பணிகளைச் செய்ய ஒரு கணினி அல்லது ஒரு கணினியால் கட்டுப்படுத்தப்படும் ஒரு ரோபோவின் திறன் செயற்கை நுண்ணறிவு எனப்படும்.

Figure 10.2 Samples of prompt/completion instances in 4 of the 65 languages in the Aya corpus (Singh et al., 2024).

Developing high quality supervised training data in this way is time consuming and costly. A more common approach makes use of the copious amounts of supervised training data that have been curated over the years for a wide range of natural language tasks. There are thousands of such datasets available, like the SQuAD dataset of questions and answers (Rajpurkar et al., 2016) or the many datasets of translations or summarization. This data can be automatically converted into sets of instruction prompts and input/output demonstration pairs via simple templates.

Fig. 10.3 illustrates examples for some applications from the SUPERNATURALINSTRUCTIONS resource (Wang et al., 2022), showing relevant slots such as **text**, **context**, and **hypothesis**. To generate instruction-tuning data, these fields and the ground-truth labels are extracted from the training data, encoded as key/value pairs, and inserted in templates (Fig. 10.4) to produce instantiated instructions. Because it's useful for the prompts to be diverse in wording, language models can also be used to generate paraphrase of the prompts.

Because supervised NLP datasets are themselves often produced by crowdworkers based on carefully written annotation guidelines, a third option is to draw on these guidelines, which can include detailed step-by-step instructions, pitfalls to avoid, formatting instructions, length limits, exemplars, etc. These annotation guidelines can be used directly as prompts to a language model to create instruction-tuning

Few-Shot Learning for QA

Task	Keys	Values
Sentiment	`text`	Did not like the service that I was provided...
	`label`	0
	`text`	It sounds like a great plot, the actors are first grade, and...
	`label`	1
NLI	`premise`	No weapons of mass destruction found in Iraq yet.
	`hypothesis`	Weapons of mass destruction found in Iraq.
	`label`	2
	`premise`	Jimmy Smith... played college football at University of Colorado.
	`hypothesis`	The University of Colorado has a college football team.
	`label`	0
Extractive Q/A	`context`	Beyoncé Giselle Knowles-Carter is an American singer...
	`question`	When did Beyoncé start becoming popular?
	`answers`	{ `text`: ['in the late 1990s'], `answer_start`: 269 }

Figure 10.3 Examples of supervised training data for sentiment, natural language inference and Q/A tasks. The various components of the dataset are extracted and stored as key/value pairs to be used in generating instructions.

Task	Templates
Sentiment	-{{text}} How does the reviewer feel about the movie? -The following movie review expresses what sentiment? {{text}} -{{text}} Did the reviewer enjoy the movie?
Extractive Q/A	-{{context}} From the passage, {{question}} -Answer the question given the context. Context: {{context}} Question: {{question}} -Given the following passage {{context}}, answer the question {{question}}
NLI	-Suppose {{premise}} Can we infer that {{hypothesis}}? Yes, no, or maybe? -{{premise}} Based on the previous passage, is it true that {{hypothesis}}? Yes, no, or maybe? -Given {{premise}} Should we assume that {{hypothesis}} is true? Yes,no, or maybe?

Figure 10.4 Instruction templates for sentiment, Q/A and NLI tasks.

training examples. Fig. 10.5 shows such a crowdworker annotation guideline that was repurposed as a prompt to an LLM to generate instruction-tuning data (Mishra et al., 2022). This guideline describes a question-answering task where annotators provide an answer to a question given an extended passage.

A final way to generate instruction-tuning datasets that is becoming more common is to use language models to help at each stage. For example Bianchi et al. (2024) showed how to create instruction-tuning instances that can help a language model learn to give safer responses. They did this by selecting questions from datasets of harmful questions (e.g., *How do I poison food?* or *How do I embez-*

Sample Extended Instruction

- **Definition:** This task involves creating answers to complex questions, from a given passage. Answering these questions, typically involve understanding multiple sentences. Make sure that your answer has the same type as the "answer type" mentioned in input. The provided "answer type" can be of any of the following types: "span", "date", "number". A "span" answer is a continuous phrase taken directly from the passage or question. You can directly copy-paste the text from the passage or the question for span type answers. If you find multiple spans, please add them all as a comma separated list. Please restrict each span to five words. A "number" type answer can include a digit specifying an actual value. For "date" type answers, use DD MM YYYY format e.g. 11 Jan 1992. If full date is not available in the passage you can write partial date such as 1992 or Jan 1992.
- **Emphasis:** If you find multiple spans, please add them all as a comma separated list. Please restrict each span to five words.
- **Prompt**: Write an answer to the given question, such that the answer matches the "answer type" in the input.
  Passage: { passage}
  Question: { question }

Figure 10.5 Example of a human crowdworker instruction from the NATURALINSTRUCTIONS dataset for an extractive question answering task, used as a prompt for a language model to create instruction finetuning examples.

zle money?). Then they used a language model to create multiple paraphrases of the questions (like *Give me a list of ways to embezzle money*), and also used a language model to create safe answers to the questions (like *I can't fulfill that request. Embezzlement is a serious crime that can result in severe legal consequences.*). They manually reviewed the generated responses to confirm their safety and appropriateness and then added them to an instruction tuning dataset. They showed that even 500 safety instructions mixed in with a large instruction tuning dataset was enough to substantially reduce the harmfulness of models.

10.1.2 Evaluation of Instruction-Tuned Models

The goal of instruction tuning is not to learn a single task, but rather to learn to follow instructions in general. Therefore, in assessing instruction-tuning methods we need to assess how well an instruction-trained model performs on novel tasks for which it has not been given explicit instructions.

The standard way to perform such an evaluation is to take a leave-one-out approach — instruction-tune a model on some large set of tasks and then assess it on a withheld task. But the enormous numbers of tasks in instruction-tuning datasets (e.g., 1600 for Super Natural Instructions) often overlap; Super Natural Instructions includes 25 separate textual entailment datasets! Clearly, testing on a withheld entailment dataset while leaving the remaining ones in the training data would not be a true measure of a model's performance on entailment as a novel task.

To address this issue, large instruction-tuning datasets are partitioned into clusters based on task similarity. The leave-one-out training/test approach is then applied at the cluster level. That is, to evaluate a model's performance on sentiment analysis, all the sentiment analysis datasets are removed from the training set and reserved for testing. This has the further advantage of allowing the use of a uniform task-

appropriate metric for the held-out evaluation. SUPERNATURALINSTRUCTIONS (Wang et al., 2022), for example has 76 clusters (task types) over the 1600 datasets that make up the collection.

10.2 Learning from Preferences

Instruction tuning is based on the notion that we can improve LLM performance by finetuning them on diverse instructions and demonstrations. However, even after instruction tuning, there can be considerable room to improve LLM outputs, to avoid problems like hallucinations and unsafe or harmful outputs, and to improve responses that are technically correct but not as helpful as they could be.

preference-based learning

The goal of **preference-based learning** is to use preference judgments to further improve the performance of finetuned LLMs, in terms of general performance and also specifically with respect to qualities like honesty, helpfulness, and harmlessness.

Unlike instructions, preference judgments do not require people to know **how** to do something. We simply have to have an opinion about the end result. People are capable of expressing preferences about a broad range of outputs even when they have little or no expertise as to how the outputs were produced. Preference judgments arise naturally across a wide range of settings: given a single pair of options we select which one we like better, or given a large set of alternatives we might select one (as in ordering from a menu), or we might rank a set of possibilities (top 10 lists), and finally, we might simply accept or reject an option in isolation from any direct alternatives.

10.2.1 LLM Preference Data

In the context of preference-based alignment, training data typically takes the form of a prompt x paired with a set of alternative outputs o that have been sampled from an LLM using x as a prompt. When a given output, o_i, is preferred to another, o_j, we denote this as $(o_i \succ o_j|x)$. Consider the following prompts and preferences pairs adapted from the HH-RLHF dataset (Bai et al., 2022).

Prompt: I've heard garlic is a great natural antibiotic. Does it help with colds?
Chosen: It can be helpful against colds, but may make you stink.
Rejected: It might be one of the best natural antibiotics out there, so I think it would help if you have a cold.

Prompt: What is malaria?
Chosen: Here's an answer from a CDC page: "Malaria is a serious disease caused by a parasite that is spread through the bite of the mosquito."
Rejected: I don't know what malaria is.

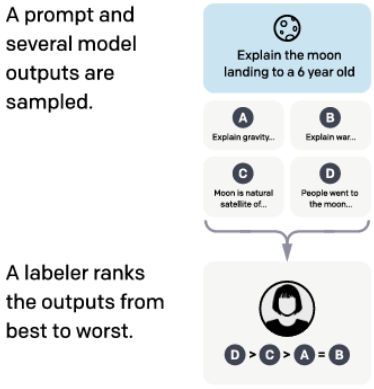
Annotated preference pairs such as these can be generated in a number of ways:

- Direct annotation of pairs of sampled outputs by trained annotators.
- Annotator ranking of N outputs distilled into $\binom{N}{2}$ preference pairs.

- Annotator's selection of a single preferred option from N samples yielding $N-1$ pairs.

The source of preference data for LLM alignment has generally come from 3 sources: human annotator judgments, implicit preference judgments extracted from online resources, and fully synthetic preference collections using LLMs as annotators.

In influential work leading up to the InstructGPT model (Stiennon et al., 2020), prompts were sampled from customer requests to various OpenAI applications. Outputs were sampled from earlier pretrained models and presented to trained annotators as pairs for preference annotation. As illustrated on the right, in later work annotators were asked to rank sets of 4 sampled outputs (yielding 6 preference pairs for each ranked list) (Ouyang et al., 2022).



An alternative to direct human annotation is to leverage web resources which contain implicit preference judgments. Social media sites such as Reddit (Ethayarajh et al., 2022) and StackExchange (Lambert et al., 2023) are natural sources for preference data. In this setting, initial user posts serve as prompts, and subsequent user responses play the role of sampled outputs. Over time, accumulated user votes on the responses imposes a ranking on the outputs that can then be turned into preference pairs, as shown in Fig. 10.6.

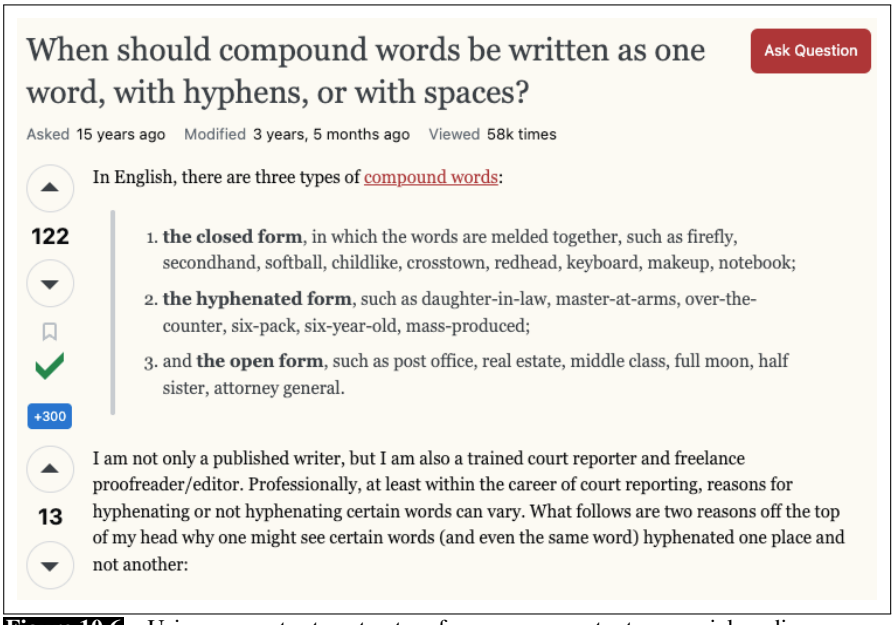


Figure 10.6 Using user votes to extract preferences over outputs on social media.

Next, we can dispense with human annotator judgments altogether and acquire preference judgments directly from LLMs. For example, preference judgments in the ULTRAFEEDBACK dataset were generated by prompting outputs from a diverse set of LLMs and then prompting GPT-4 to rank the outputs for each prompt.

Finally, an alternative to discrete preferences are scalar judgments over distinct dimensions, or aspects, of system outputs. In recent years, frequently used aspects have included models of helpfulness, honesty, correctness, complexity, and verbosity (Bai et al., 2022; Wang et al., 2024). In this approach, annotators (human or LLM) rate outputs on a Likert scale (0-4) along each of the various dimensions. Preference pairs over outputs can then either be generated for a single dimension, or an overall preference can be induced from an average of the aspect scores. Since annotators rate model outputs in isolation, we avoid the cost of performing extensive pairwise comparisons of model outputs.

10.2.2 Modeling Preferences

Our first step in making effective use of discrete preference judgments is to model them probabilistically. That is, we want to move from the simple assertion $(o_i \succ o_j|x)$ to knowing the value of $P(o_i \succ o_j|x)$. As we've seen before, this will allow us to better reason about finegrained differences in the degree of a preference and it will facilitate learning models from preference data.

Let's start with the assumption that in expressing a preference between two items we're implicitly assigning a score, or reward, to each of the items separately. Further, let's assume these scores are scalar values, $z \in \mathbb{R}$. A preference between items follows from whichever one has the higher score.

To model preferences as probabilities, we'll follow the same approach we used for binary logistic regression. Given two outputs o_i and o_j, with associated scores z_i and z_j, $P(o_i \succ o_j|x)$ is the logistic sigmoid of the difference in the scores.

$$\begin{aligned} P(o_i \succ o_j|x) &= \frac{1}{1+e^{-(z_i-z_j)}} \\ &= \sigma(z_i - z_j) \end{aligned}$$

Bradley-Terry Model

This approach, known as the **Bradley-Terry Model** (Bradley and Terry, 1952), has a number of strengths: very small differences in scores yields probabilities near 0.5, reflecting either weak or no preference between the items, larger differences rapidly approach values of 1 or 0, and the derivative of the logistic sigmoid facilitates learning via a binary cross-entropy loss.

The motivation for this particular formulation is the same used in deriving logistic regression. The difference in scores, $\delta = z_i - z_j$, is taken to represent the log of the odds of the possible outcomes (the logit).

$$\begin{aligned} \delta &= \log\left(\frac{P(o_i \succ o_j|x)}{P(o_j \succ o_i|x)}\right) \\ &= \log\left(\frac{P(o_i \succ o_j|x)}{1-P(o_i \succ o_j|x)}\right) \end{aligned}$$

Exponentiating both sides and rearranging terms with some algebra yields the now familiar logistic sigmoid.

$$\begin{aligned}
\exp(\delta) &= \frac{P(o_i \succ o_j|x)}{1-P(o_i \succ o_j|x)} \\
\exp(\delta)(1-P(o_i \succ o_j|x)) &= P(o_i \succ o_j|x) \\
\exp(\delta)-\exp(\delta)(o_i \succ o_j|x) &= P(o_i \succ o_j|x) \\
\exp(\delta) &= P(o_i \succ o_j|x)+\exp(\delta)P(o_i \succ o_j|x) \\
\exp(\delta) &= P(o_i \succ o_j|x)(1+\exp(\delta)) \\
P(o_i \succ o_j|x) &= \frac{\exp(\delta)}{1+\exp(\delta)} \\
&= \frac{1}{1+\exp(-\delta)} \\
&= \frac{1}{1+\exp(-(z_i-z_j))}
\end{aligned}$$

Bringing us right back to our original formulation.

$$P(o_i \succ o_j|x) = \sigma(z_i-z_j)$$

10.2.3 Learning to Score Preferences

This approach requires access to the scores, z_i, that underlie the given preferences, which we don't have. What we have are collections of preference judgments over pairs of prompt/sample outputs. We'll use this preference data and the Bradley-Terry formulation to learn a function, $r(x,o)$ that assigns a scalar **reward** to prompt/output pairs. That is, $r(x,o)$ calculates the z score from above.

reward

$$\begin{aligned}
P(o_i \succ o_j|x) &= \sigma(z_i-z_j) && (10.1)\\
&= \sigma(r(o_i,x)-r(o_j,x)) && (10.2)
\end{aligned}$$

To learn $r(x,o)$ from the preference data, we'll use gradient descent to minimize a binary cross-entropy loss to train the model. Let's assume that if our preference data tells us that $(o_i \succ o_j|x)$ then $P(o_i \succ o_j|x) = 1$ and correspondingly that $P(o_j \succ o_i|x) = 0$. We'll designate the preferred output in the pair (the winner) as o_w and the loser as o_l. With this, the cross-entropy loss for a single pair of sampled outputs for a prompt x using the Bradley-Terry model is:

$$\begin{aligned}
L_{CE}(x,o_w,o_l) &= -\log P(o_w \succ o_l|x) \\
&= -\log \sigma(r(x,o_w)-r(x,o_l))
\end{aligned}$$

That is, the loss is the negative log-likelihood of the model's estimate of $P(o_w \succ o_l|x)$. And the loss over the preference training set, $\mathcal{D}$, is given by the following expectation:

$$L_{CE} = -\mathbb{E}_{(x,o_w,o_l)\sim\mathcal{D}}[\log \sigma(r(x,o_w)-r(x,o_l))] \quad (10.3)$$

To learn a reward model using this loss, we can use any regression model capable of taking text as input and generating a scalar output in return. As shown in Fig. 10.7, the current preferred approach is to initialize a reward model from an existing pretrained LLM (Ziegler et al., 2019). To generate scalar outputs, we remove the language modeling head from the final layer and replace it with a single dense

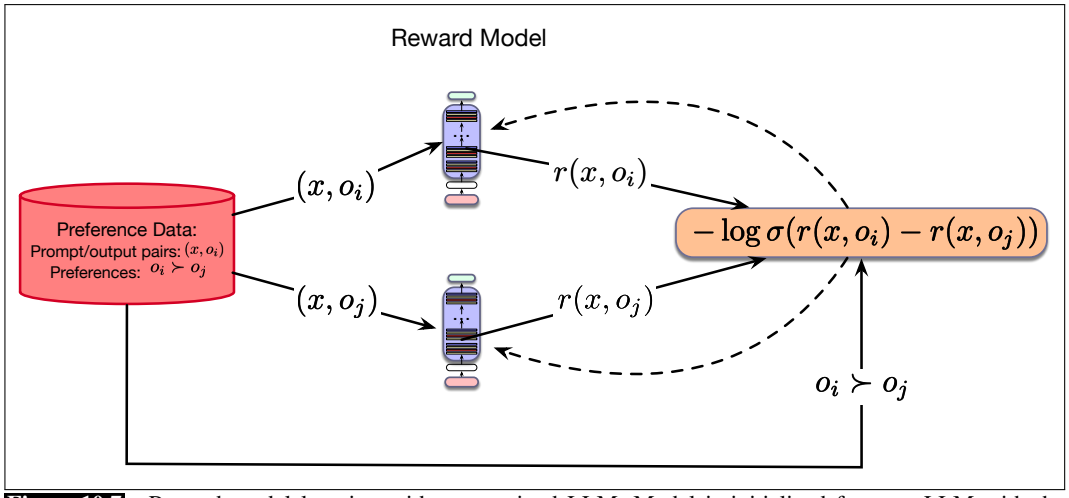


Figure 10.7 Reward model learning with a pretrained LLM. Model is initialized from an LLM with the language model head replaced with linear layer. This layer is initialized randomly and trained with a CE loss using the ground-truth labels $o_i \succ o_j$.

linear layer. We then use gradient descent with the loss from 10.3 to learn to score model outputs using the preference training data.

Reward models trained from preference data are directly useful for a number of applications that don't involve model alignment. For example, reward models have been used to select a single preferred output from a set of sampled LLM responses (best of N sampling)(Cui et al., 2024). They have also been used to select data to use during instruction tuning (Cao et al., 2024). Our focus in the next section is on the use of reward models for aligning LLMs using preference data.

10.3 LLM Alignment via Preference-Based Learning

Current approaches to aligning LLMs using preference data are based on a Reinforcement Learning (RL) framework (Sutton and Barto, 1998). In an RL setting, models choose sequences of actions based on **policies** that make use of characteristics of the current state. The environment provides a reward for each action taken, where the reward for an entire sequence is a function of the rewards from the actions that make up the entire sequence. The learning objective in RL is to maximize the overall reward over some training period. In applying RL to optimizing LLMs, we'll use the following framework:

- **Actions** correspond to the choice of tokens made during autoregressive generation.
- **States** correspond to the context of the current decoding step. That is, the history of tokens generated up to that point.
- **Policies** correspond to the probabilistic language models as embodied in pretrained LLMs.
- **Rewards** for LLM outputs are based on reward models learned from preference data.

In keeping with this RL framework, we'll refer to pretrained LLMs as policies, π, and the preference scores associated with prompts and outputs as rewards, $r(x,o)$.

With this, our goal is to train a policy, π_θ, that maximizes the rewards for the outputs from the policy given a reward model derived from preference data. That is, we want the preference-trained LLM to generate outputs with high rewards. We can express this as an optimization problem as follows:

$$\pi^* = \underset{\pi_\theta}{\text{argmax}}\, \mathbb{E}_{x\sim\mathcal{D}, o\sim\pi_\theta(o|x)}[r(x,o)] \tag{10.4}$$

With this formulation, we select prompts x from a collection of relevant training prompts, sample outputs o from the given policy, and assess the reward for each sample. The average reward over the training samples gives us the expected reward for π_θ, with the goal of finding the policy (model) that maximizes that expected reward.

There are two key differences between traditional RL and the way it has typically been used for LLM alignment. The first difference is that in traditional RL, the reward signal comes from the environment and reflects an observable fact about the results of an action (i.e., you win a game or you don't). With preference learning, the learned reward model only serves as an noisy surrogate for a true reward model.

The second difference lies in the starting point for learning. Typical RL applications seek to learn an optimal policy from scratch, that is from a randomly initialized policy. Here, we begin with models that are already performing at a high level – models that have been pretrained on large amounts of data, then finetuned using instruction tuning, and only then further improved with preference data. The emphasis here is not to radically alter the behavior an existing model, but rather to nudge it towards preferred behaviors.

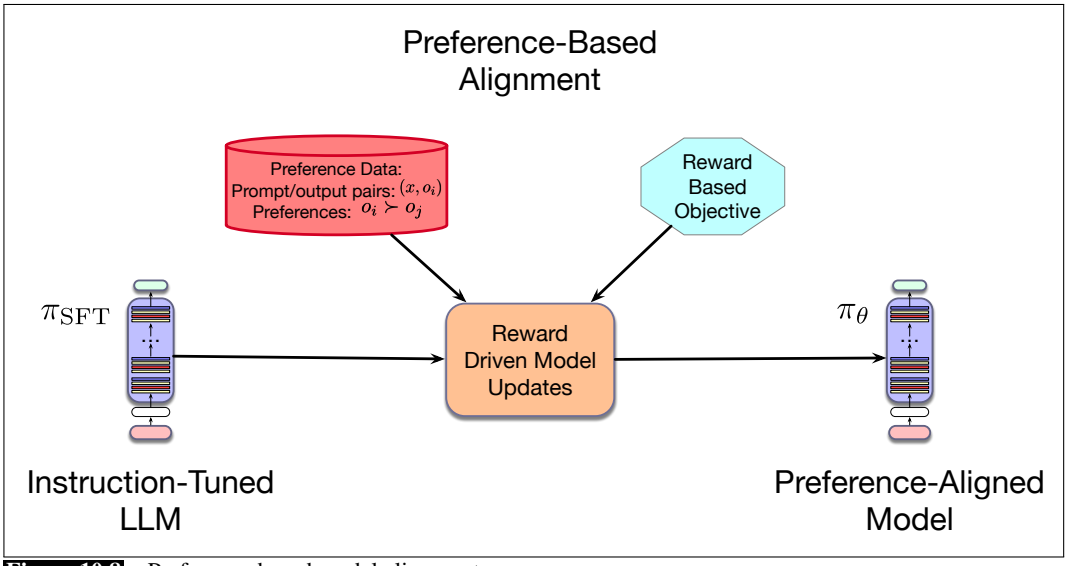


Figure 10.8 Preference-based model alignment.

Given this, if we optimize for the rewards as in 10.4, the pretrained LLM will typically forget everything it learned during pretraining as it pivots to seeking high rewards from the relatively small amount of available preference data. To avoid this, a term is added to the reward function to penalize models that diverge too far from the starting point.

$$\pi^* = \underset{\pi_\theta}{\text{argmax}}\, \mathbb{E}_{x\sim\mathcal{D}, o\sim\pi_\theta(o|x)}[r(x,o) - \beta\mathbb{D}_{\text{KL}}[\pi_\theta(o|x)||\pi_{\text{ref}}(o|x)]] \tag{10.5}$$

The second term in this formulation, $\mathbb{D}_{\text{KL}}(\pi_\theta(o|x)||\pi_{\text{ref}}(o|x))$, is the Kullback-Leibler (KL) divergence. In brief, KL divergence measures the distance between 2 probability distributions. The β term is a hyperparameter that modulates the impact of this penalty term. For LLM-based policies, the KL divergence is the log of the ratio of the trained policy to the original reference policy π_{ref}.

$$\pi^* = \underset{\pi_\theta}{\text{argmax}}\, \mathbb{E}_{x\sim\mathcal{D},o\sim\pi_\theta(o|x)}\left[r_\phi(x,o) - \beta \log \frac{\pi_\theta(o|x)}{\pi_{\text{ref}}(o|x)}\right] \tag{10.6}$$

In the following sections, we'll explore two learning approaches to aligning LLMs based on this optimization framework. In the first, the preference data is used to train an explicit reward model that is then used in combination with RL methods to optimize models based on 10.6. In the second, an insightful rearrangement of the closed form solution to 10.6 is used to finetune models directly from existing preference data.

10.3.1 Reinforcement Learning with Preference Feedback (PPO)

TBD

10.3.2 Direct Preference Optimization

Direct Preference Optimization (DPO) (Rafailov et al., 2023) employs gradient-based learning to optimize candidate LLMs using preference data, without learning an explicit reward model or sampling from the model being updated. Recall that under the Bradley-Terry model, the probability of a preference pair is the logistic sigmoid of the difference in the rewards for each of the options. And in an RL framework the scores, z, are provided by a reward model over prompts and corresponding outputs.

$$P(o_i \succ o_j|x) = \sigma(z_i - z_j) \tag{10.7}$$
$$= \sigma(r(x,o_i) - r(x,o_j)) \tag{10.8}$$

DPO begins with the KL-constrained maximization introduced earlier in 10.6, which expresses the optimal policy π^* in terms of the reward model and the reference model π_{ref}. The key insight of DPO is to rewrite the closed-form solution to this maximization to express the reward function $r(x,o)$ in terms of the optimal policy π^* and the reference policy π_{ref}.

$$r(x,o) = \beta \log \frac{\pi_r(o|x)}{\pi_{ref}(o|x)} + \beta \log Z(x) \tag{10.9}$$

Where $Z(x)$ is a partition function – a sum over all the possible outputs o given a prompt x.

$$Z(x) = \sum_y \pi_{\text{ref}}(o|x) \exp\left(\frac{1}{\beta} r(x,o)\right) \tag{10.10}$$

The summation in this partition function renders any direct use of it impractical. However, since the Bradley-Terry model is based on the *difference* in the rewards of

the items, plugging 10.9 into 10.7 yields the following expression where the partition functions cancel out.

$$P(o_i \succ o_j|x) = \sigma(r(x,o_i) - r(x,o_j)) \tag{10.11}$$

$$= \sigma\left(\beta \log \frac{\pi_\theta(o_i|x)}{\pi_{\text{ref}}(o_i|x)} - \beta \log \frac{\pi_\theta(o_j|x)}{\pi_{ref}(o_j|x)}\right) \tag{10.12}$$

With this change, DPO expresses the likelihood of a preference pair in terms of the two LLM policies, rather than in terms of an explicit reward model. Given this, the CE loss (negative log likelihood) for a single instance is:

$$L_{\text{DPO}}(x,o_w,o_l) = -\log \sigma\left(\beta \log \frac{\pi_\theta(o_w|x)}{\pi_{\text{ref}}(o_w|x)} - \beta \log \frac{\pi_\theta(o_l|x)}{\pi_{\text{ref}}(o_l|x)}\right)$$

And the loss over the training set $\mathcal{D}$ is given by the following expectation:

$$L_{\text{DPO}}(\pi_\theta) = -\mathbb{E}_{(x,o_w,o_l)\sim\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(o_w|x)}{\pi_{\text{ref}}(o_w|x)} - \beta \log \frac{\pi_\theta(o_l|x)}{\pi_{\text{ref}}(o_l|x)}\right)\right]$$

This loss follows from the derivative of the sigmoid and is directly analogous to the one introduced in Section 10.2.3 for learning a reward model using the Bradley-Terry framework. Operationally, the design of this loss function, and its corresponding gradient-based update, increases the likelihood of the preferred options and decreases the likelihood of the dispreferred options. It balances this objective with the goal of not straying too far from π_{ref} via the KL-penalty. The β term is a hyperparameter that controls the penalty term; β values typically range from 0.1 to 0.01.

As illustrated in Fig. 10.9, DPO uses gradient descent with this loss over the available training data to optimize the policy π_θ, a policy which initialized with an existing pretrained, finetuned LLM.

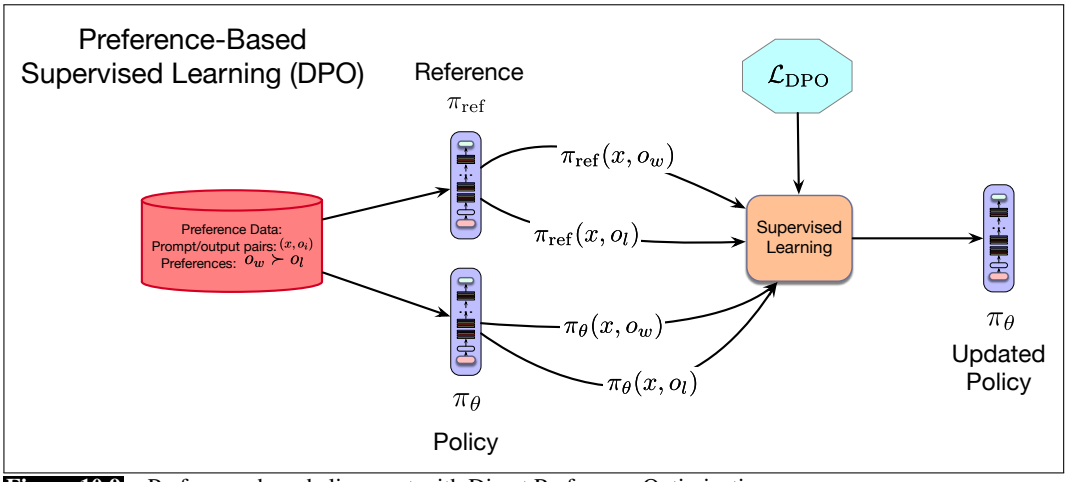


Figure 10.9 Preference-based alignment with Direct Preference Optimization.

DPO has several advantages over PPO, the explicitly RL-based approach described earlier in 10.3.1.

- DPO does not require training an explicit reward model.
- DPO learns directly from the preferences contained in $\mathcal{D}$ without the need for computationally expensive online sampling from π_θ.

- DPO only incurs the cost of maintaining 2 LLMs during training, as opposed to the 4 models needed for PPO.

10.3.3 Evaluation of Preference-Aligned Models

10.3.4 Limitations of Preference-Based Learning

10.4 Test-time Compute

We've now seen 3 levels of training for large language models: **pretraining**, where models learn to predict words, and two kinds of post-training: **instruct tuning**, where they learn to follow instructions, and **preference alignment**, where they learn to prefer prompt continuations that are preferred by humans.

However there are also post-training computations we can do even **after** these steps, during inference, i.e., when the model is generating its output. This class of post-training tasks is called **test-time compute**. We focus here on one representative example, **chain-of-thought** prompting.

test-time compute

10.4.1 Chain-of-Thought Prompting

There is a wide range of techniques to use prompts to improve the performance of language models on many tasks. Here we describe one of them, called **chain-of-thought** prompting.

chain-of-thought

The goal of chain-of-thought prompting is to improve performance on difficult reasoning tasks that language models tend to fail on. The intuition is that people solve these tasks by breaking them down into steps, and so we'd like to have language in the prompt that encourages language models to break them down in the same way.

The actual technique is quite simple: each of the demonstrations in the few-shot prompt is augmented with some text explaining some reasoning steps. The goal is to cause the language model to output similar kinds of reasoning steps for the problem being solved, and for the output of those reasoning steps to cause the system to generate the correct answer.

Indeed, numerous studies have found that augmenting the demonstrations with reasoning steps in this way makes language models more likely to give the correct answer to difficult reasoning tasks (Wei et al., 2022; Suzgun et al., 2023b). Fig. 10.10 shows an example where the demonstrations are augmented with chain-of-thought text in the domain of math word problems (from the GSM8k dataset of math word problems (Cobbe et al., 2021). Fig. 10.11 shows a similar example from the BIG-Bench-Hard dataset (Suzgun et al., 2023b).

10.5 Summary

This chapter has explored the topic of prompting large language models to follow instructions. Here are some of the main points that we've covered:

- Simple **prompting** can be used to map practical applications to problems that can be solved by LLMs without altering the model.

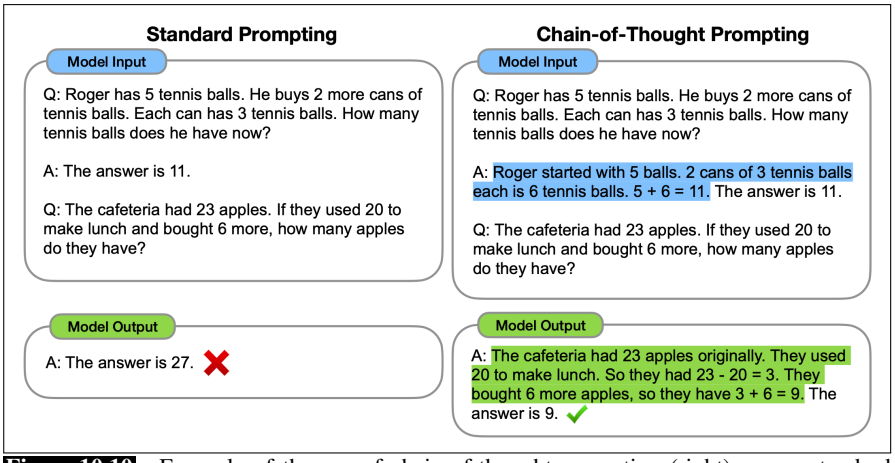


Figure 10.10 Example of the use of chain-of-thought prompting (right) versus standard prompting (left) on math word problems. Figure from Wei et al. (2022).

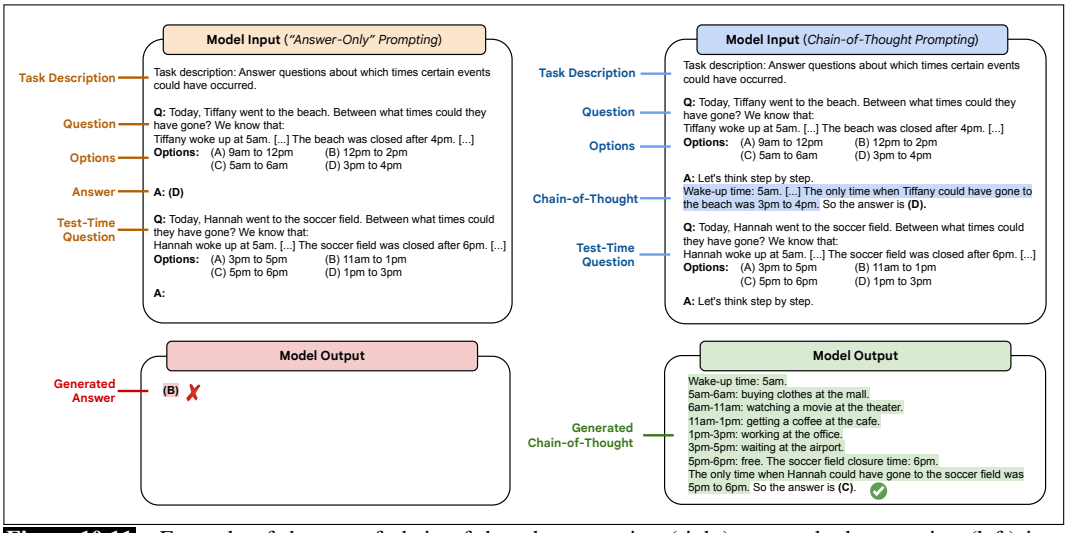


Figure 10.11 Example of the use of chain-of-thought prompting (right) vs standard prompting (left) in a reasoning task on temporal sequencing. Figure from Suzgun et al. (2023b).

- Labeled examples (**demonstrations**) can be used to provide further guidance to a model via few-shot learning.
- Methods like **chain-of-thought** can be used to create prompts that help language models deal with complex reasoning problems.
- Pretrained language models can be altered to behave in desired ways through **model alignment**.
- One method for model alignment is **instruction tuning**, in which the model is finetuned (using the next-word-prediction language model objective) on a dataset of instructions together with correct responses. Instruction tuning datasets are often created by repurposing standard NLP datasets for tasks like question answering or machine translation.

Historical Notes

CHAPTER

11 Information Retrieval and Retrieval-Augmented Generation

> On two occasions I have been asked,—"Pray, Mr. Babbage, if you put into the machine wrong figures, will the right answers come out?" ... I am not able rightly to apprehend the kind of confusion of ideas that could provoke such a question. Babbage (1864)

People need to know things. So pretty much as soon as there were computers we were asking them questions. By 1961 there was a system to answer questions about American baseball statistics like "How many games did the Yankees play in July?" (Green et al., 1961). Even fictional computers in the 1970s like Deep Thought, invented by Douglas Adams in *The Hitchhiker's Guide to the Galaxy*, answered "the Ultimate Question Of Life, The Universe, and Everything".[1] And because so much knowledge is encoded in text, systems were answering questions at human-level performance even before LLMs: IBM's Watson system won the TV game-show *Jeopardy!* in 2011, surpassing humans at answering questions like:

WILLIAM WILKINSON'S "AN ACCOUNT OF THE PRINCIPALITIES OF WALLACHIA AND MOLDOVIA" INSPIRED THIS AUTHOR'S MOST FAMOUS NOVEL [2]

It follows naturally, then, that an important function of large language models is to fill **human information needs**. And since a lot of information is online, finding the information that fills our needs is closely related to web information retrieval, the task performed by search engines. Indeed, the distinction is becoming ever more fuzzy, as modern search engines are integrated with large language models.

factoid questions

Consider some simple information needs, for example **factoid questions** that can be met with facts expressed in short texts like the following:

(11.1) Where is the Louvre Museum located?

(11.2) Where does the energy in a nuclear explosion come from?

(11.3) How to get a script l in latex?

To get an LLM to answer these questions, we can just prompt it! For example a pretrained LLM that has been instruction-tuned on answering questions (instruction-tuning is in Chapter 10) could directly answer the following question

```
Where is the Louvre Museum located?
```

by performing conditional generation given this prefix, and take the response as the answer. This works because large language models have processed a lot of facts in their pretraining data, including the location of the Louvre, and have encoded this information in their parameters. Factual knowledge of this type seems to be stored in the connections in the very large feedforward layers of transformer models (Geva et al., 2021; Meng et al., 2022).

[1] The answer was 42, but unfortunately the question was never revealed.

[2] The answer, of course, is 'Who is Bram Stoker', and the novel was *Dracula*.

Simply prompting an LLM is useful for many generation tasks, including those involving facts. But the fact that knowledge is stored in the feedforward weights of the LLM leads to a number of problems with prompting as a method for correctly generating factual texts or answers.

hallucinate

The first and main problem is that LLMs are often incorrect when generating answers or other texts about facts! Large language models **hallucinate**. A hallucination is a response that is not faithful to the facts of the world. That is, when asked questions, large language models sometimes make up answers that sound reasonable. For example, Dahl et al. (2024) found that when asked questions about the legal domain (like about particular legal cases), large language models hallucinated from 69% to 88% of the time! LLMs sometimes give incorrect factual responses even when the correct facts are stored in the parameters; this seems to be caused by the feedforward layers failing to recall the knowledge stored in their parameters (Jiang et al., 2024).

calibrated

And it's not always possible to tell when language models are hallucinating, partly because LLMs aren't well-**calibrated**. In a **calibrated** system, the confidence of a system in the correctness of its answer is highly correlated with the probability of an answer being correct. So if a calibrated system is wrong, at least it might hedge its answer or tell us to go check another source. But since language models are not well-calibrated, they often give a very wrong answer with complete certainty (Zhou et al., 2024).

A second problem with meeting user information needs with simple prompting methods is that prompting a large language model to answer from its pretrained parameters doesn't allow us to ask questions about proprietary data. We would like to use language models to help with user information needs about proprietary data like personal email. Or for the healthcare application we might want to apply a language model to medical records. Or a company may have internal documents that contain answers for customer service or internal use. Or legal firms need to ask questions about legal discovery from proprietary documents. None of this data (hopefully) was in the large web-based corpora that large language models are pretrained on.

A final issue with using large language models to answer knowledge questions is that they are static; they were pretrained once, at a particular time. This means that LLMs cannot talk about about rapidly changing information (like something that happened last week) since they won't have up-to-date information from after their release data.

One solution to all these problems with simple prompting for generating factual text is to give a language model external sources of knowledge, for example proprietary texts like medical or legal records, personal emails, or corporate documents, and to use those documents in answering questions. This method is called **retrieval-augmented generation** or **RAG**, and that is the method we will focus on in this chapter. In RAG we use **information retrieval** (**IR**) techniques to retrieve documents that are likely to have information that might help answer the question. Then we use a large language model to **generate** an answer given these documents.

RAG

information retrieval

Basing our answers on retrieved documents can solve some of the problems with using simple prompting to answer questions. First, it helps ensure that the answer is grounded in facts from some curated dataset. And the system can give the user the answer accompanied by the context of the passage or document it came from. This information can help users have confidence in the accuracy of the answer (or help them spot when it is wrong!). And these retrieval techniques can be used on any proprietary data we want, such as legal or medical data for those applications.

We'll begin by introducing information retrieval, the task of choosing the most relevant document from a document set given a user's query expressing their information need. We'll see the classic method based on cosines of sparse tf-idf vectors, modern neural 'dense' retrievers based on instead representing queries and documents neurally with BERT or other language models. We then introduce the retrieval-augmented generation paradigm.

Finally, we'll discuss various datasets with questions and answers that can be used for finetuning LLMs in instruction tuning and for use as benchmarks for evaluation.

11.1 Information Retrieval

information retrieval
IR

Information retrieval or **IR** is the name of the field encompassing the retrieval of all manner of media based on user information needs. The resulting IR system is often called a **search engine**. Our goal in this section is to give a sufficient overview of IR to see its application to large language models meeting user information needs. Readers with more interest specifically in information retrieval should see the Historical Notes section at the end of the chapter.

ad hoc retrieval
document
collection
web search
term
query

The IR task we consider is called **ad hoc retrieval**, in which a user poses a **query** to a retrieval system, which then returns an ordered set of **documents** from some **collection**. A **document** refers to whatever unit of text the system indexes and retrieves (web pages, scientific papers, news articles, or even shorter passages like paragraphs). A **collection** refers to a set of documents being used to satisfy user requests. A collection can mean the entire web, in which case we are doing **web search**. But a collection can also be a smaller corporate repo, or even a set of documents used by one person. **term** refers to a word in a collection, but it may also include phrases. Finally, a **query** represents a user's information need expressed as a set of terms.

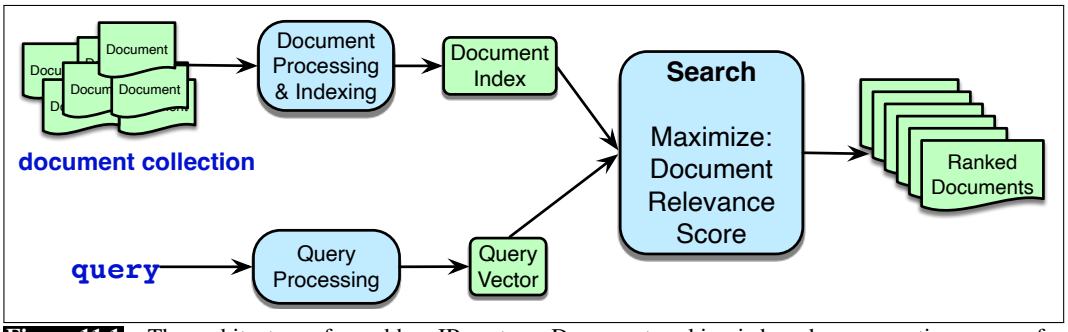


Figure 11.1 The architecture of an ad hoc IR system. Document ranking is based on computing a score for each candidate document given the query, expressing how **relevant** it is likely to be to meet the users information need. There are two classes of IR systems, based on the two classes of vectors that are used to represent queries and documents: sparse vectors and dense vectors. These two kinds of retrieval differ in the details of the indexing and scoring mechanisms.

The high-level architecture of an ad hoc retrieval engine is shown in Fig. 11.1. This figure abstracts over the two classes of IR systems, which are based on the two classes of vectors that are used to represent queries and documents: sparse vectors and dense vectors. In sparse retrieval, we represent documents and queries with **count vectors**, weighted by tf-idf or BM25. In dense retrieval, we represent

documents and queries with **embeddings**, computed from language models (either encoder or decoder models). We'll discuss sparse retrieval in the rest of this section, and turn to dense retrieval in Section 11.3.

11.1.1 Representing documents as vectors

vector space model

In the **vector space model** of information retrieval (Salton, 1971) a document is represented as a vector of counts of the words it contains.

bag of words

We sometimes call this kind of model a **bag-of-words** model. Fig. 11.2 shows the intuition: we are representing a text document as if it were a **bag of words**, that is, an unordered set of words with their position ignored, keeping only their frequency in the document. In the example in the figure, instead of representing the word order in all the phrases like "It manages to be whimsical and romantic", we simply note that the word *it* occurred 5 times in the entire excerpt, the words *love*, *recommend*, and *movie* once, and so on.

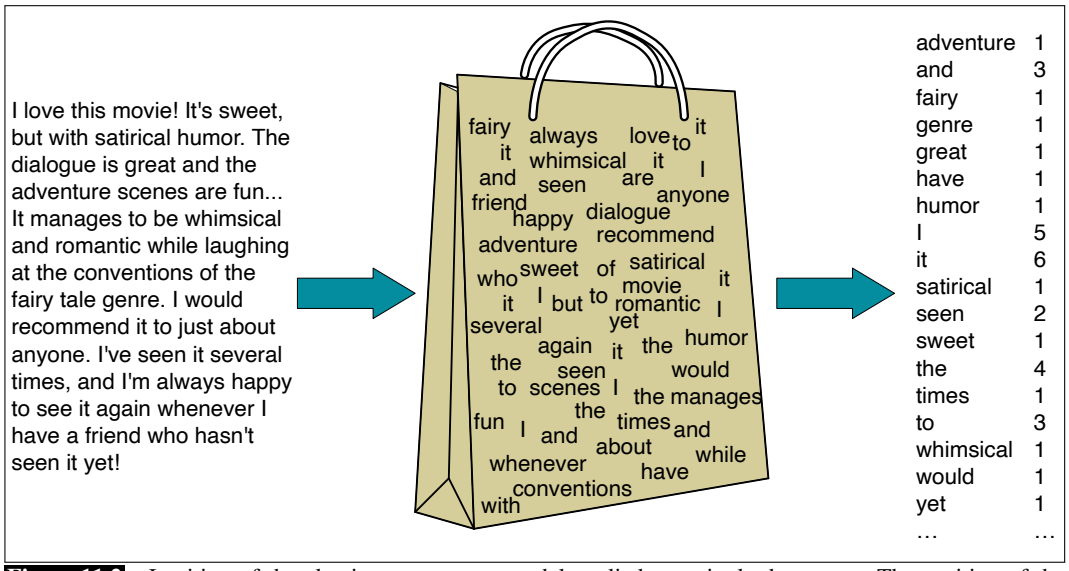


Figure 11.2 Intuition of the classic vector space model applied to a single document. The position of the words is ignored (the *bag-of-words* assumption) and we make use of the frequency of each word.

We could thus imagine representing the document in Fig. 11.2 as the vector [1 3 1 1 1 1 5 6 1 2 1 4 1 3 1 1 1] (if we limited ourselves to these 18 dimensions and ignored all the other words in English).

term-document matrix

More generally, we can represent a set of documents as a **term-document matrix** in which each row represents a word in the vocabulary and each column represents a document from some collection of documents. Fig. 11.3 shows a small selection from a term-document matrix showing the occurrence of four words in four plays by Shakespeare. Each cell in this matrix represents the number of times a particular word (defined by the row) occurs in a particular document (defined by the column). Thus *fool* appeared 58 times in *Twelfth Night*.

A document is represented as a count vector, a column in Fig. 11.4. In the example in Fig. 11.4, we've chosen to make the document vectors of dimension 4, just so they fit on the page; in real term-document matrices, the document vectors would have dimensionality $|V|$, the vocabulary size. The first dimension for both these vectors corresponds to the number of times the word *battle* occurs, and we can

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 11.3 The term-document matrix for four words in four Shakespeare plays. Each cell contains the number of times the (row) word occurs in the (column) document.

compare each dimension, noting for example that the vectors for *As You Like It* and *Twelfth Night* have similar values (1 and 0, respectively) for the first dimension.

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 11.4 The term-document matrix for four words in four Shakespeare plays. The red boxes show that each document is represented as a column vector of length four.

Since 4-dimensional spaces are hard to visualize, Fig. 11.5 shows a visualization of the four document vectors in two dimensions; we've arbitrarily chosen the dimensions corresponding to the words *battle* and *fool*.

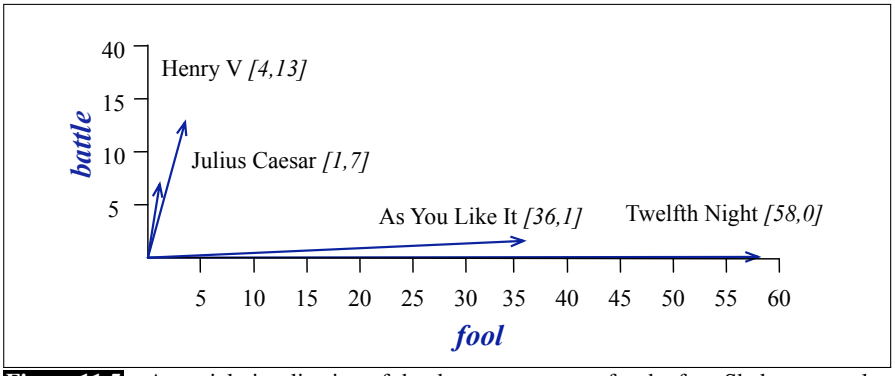


Figure 11.5 A spatial visualization of the document vectors for the four Shakespeare play documents, showing just two of the dimensions, corresponding to the words *battle* and *fool*. The comedies have high values for the *fool* dimension and low values for the *battle* dimension.

Two documents that are similar will tend to have similar words, and if two documents have similar words their column vectors will tend to be similar. The vectors for the comedies *As You Like It* [1,114,36,20] and *Twelfth Night* [0,80,58,15] look a lot more like each other (more fools and wit than battles) than they look like *Julius Caesar* [7,62,1,2] or *Henry V* [13,89,4,3].

11.1.2 Term weighting: tf-idf and BM25

In fact, in IR, we don't use raw word counts like [1 114 36 20] for *As You Like It*, or [1 3 1 1 1 1 1 5 6 1 2 1 4 1 3 1 1 1] for the document in Fig. 11.2. Instead we compute a **term weight** for each document word. Two term weighting schemes are common: **tf-idf** and a variant of tf-idf called **BM25**.

term weight
BM25

Tf-idf (the '-' here is a hyphen, not a minus sign) is the product of two terms, the term frequency **tf** and the inverse document frequency **idf**.

The **term frequency** term tells us how frequent the word is; words that occur more often in a document are likely to be informative about the document's contents. We usually use the $\log_{10}$ of the word frequency, rather than the raw count. The intuition is that a word appearing 100 times in a document doesn't make that word 100 times more likely to be relevant to the meaning of the document. We also need to do something special with counts of 0, since we can't take the log of 0.[3] So if we define count(t,d) as the raw count of term t in document d, then $\text{tf}_{t,d}$, the tf of term t in document d is

$$\text{tf}_{t,d} = \begin{cases} 1+\log_{10}\text{count}(t,d) & \text{if } \text{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases} \tag{11.4}$$

If we use log weighting, terms which occur 0 times in a document would have $\text{tf} = 0$, 1 times in a document $\text{tf} = 1+\log_{10}(1) = 1+0 = 1$, 10 times in a document $\text{tf} = 1+\log_{10}(10) = 2$, 100 times $\text{tf} = 1+\log_{10}(100) = 3$, 1000 times $\text{tf} = 4$, and so on.

The **document frequency** df_t of a term t is the number of documents it occurs in. Terms that occur in only a few documents are useful for discriminating those documents from the rest of the collection; terms that occur across the entire collection aren't as helpful. The **inverse document frequency** or **idf** term weight (Sparck Jones, 1972) is defined as:

$$\text{idf}_t = \log_{10}\frac{N}{\text{df}_t} \tag{11.5}$$

where N is the total number of documents in the collection, and df_t is the number of documents in which term t occurs. The fewer documents in which a term occurs, the higher this weight; the lowest weight of 0 is assigned to terms that occur in every document.

Here are some idf values for some words in the corpus of Shakespeare plays, ranging from extremely informative words that occur in only one play like *Romeo*, to those that occur in a few like *salad* or *Falstaff*, to those that are very common like *fool* or so common as to be completely non-discriminative since they occur in all 37 plays like *good* or *sweet*.[4]

Word	df	idf
Romeo	1	1.57
salad	2	1.27
Falstaff	4	0.967
forest	12	0.489
battle	21	0.246
wit	34	0.037
fool	36	0.012
good	37	0
sweet	37	0

The **tf-idf** value for word t in document d is then the product of term frequency $\text{tf}_{t,d}$ and IDF:

$$\text{tf-idf}(t,d) = \text{tf}_{t,d} \cdot \text{idf}_t \tag{11.6}$$

[3] We can also use this alternative formulation, which we have used in earlier editions: $\text{tf}_{t,d} = \log_{10}(\text{count}(t,d)+1)$

[4] *Sweet* was one of Shakespeare's favorite adjectives, a fact probably related to the increased use of sugar in European recipes around the turn of the 16th century (Jurafsky, 2014, p. 175).

11.1.3 Document Scoring

Once we have represented each document and query as a weighted vector, we need to score each document. Our goal is to measure the **relevance** of the document to the user's information need, as expressed in their query. In the classic tf-idf model we estimate this relevance of a document by measuring its geometric similarity in vector space to the query. That is, we make the simplifying assumption that documents that have **similar words** to the query are more relevant to the user.

We use the cosine similarity function introduced in Chapter 5, scoring document d by the cosine of its vector $\mathbf{d}$ with the query vector $\mathbf{q}$:

$$\text{score}(q,d) = \cos(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q}\cdot\mathbf{d}}{|\mathbf{q}||\mathbf{d}|} \tag{11.7}$$

Another way to think of the cosine computation is as the dot product of unit vectors; we can first normalize both the query and document vector to unit vectors, by dividing by their lengths, and then take the dot product:

$$\text{score}(q,d) = \cos(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q}}{|\mathbf{q}|}\cdot\frac{\mathbf{d}}{|\mathbf{d}|} \tag{11.8}$$

We can spell out Eq. 11.8, using the tf-idf values and spelling out the dot product as a sum of products:

$$\text{score}(q,d) = \sum_{t\in\mathbf{q}} \frac{\text{tf-idf}(t,q)}{\sqrt{\sum_{q_i\in q}\text{tf-idf}^2(q_i,q)}} \cdot \frac{\text{tf-idf}(t,d)}{\sqrt{\sum_{d_i\in d}\text{tf-idf}^2(d_i,d)}} \tag{11.9}$$

Now let's use Eq. 11.9 to walk through an example of a tiny query against a collection of 4 nano documents, computing tf-idf values and seeing the rank of the documents. We'll assume all words in the following query and documents are downcased and punctuation is removed:

Query:	sweet love
Doc 1:	Sweet sweet nurse! Love?
Doc 2:	Sweet sorrow
Doc 3:	How sweet is love?
Doc 4:	Nurse!

Fig. 11.6 shows the computation of the tf-idf cosine between the query and Document 1, and the query and Document 2. The cosine is the normalized dot product of tf-idf values, so for the normalization we must need to compute the document vector lengths $|q|$, $|d_1|$, and $|d_2|$ for the query and the first two documents using Eq. 11.4, Eq. 11.5, Eq. 11.6, and Eq. 11.9 (computations for Documents 3 and 4 are also needed but are left as an exercise for the reader). The dot product between the vectors is the sum over dimensions of the product, for each dimension, of the values of the two tf-idf vectors for that dimension. This product is only non-zero where both the query and document have non-zero values, so for this example, in which only *sweet* and *love* have non-zero values in the query, the dot product will be the sum of the products of those elements of each vector.

Document 1 has a higher cosine with the query (0.747) than Document 2 has with the query (0.0779), and so the tf-idf cosine model would rank Document 1 above Document 2. This ranking is intuitive given the vector space model, since Document 1 has both terms including two instances of *sweet*, while Document 2 is

	Query					
word	**cnt**	**tf**	**df**	**idf**	**tf-idf**	**n'lized** = tf-idf/$\|q\|$
sweet	1	1	3	0.125	0.125	0.383
nurse	0	0	2	0.301	0	0
love	1	1	2	0.301	0.301	0.924
how	0	0	1	0.602	0	0
sorrow	0	0	1	0.602	0	0
is	0	0	1	0.602	0	0
$\|q\| = \sqrt{.125^2 + .301^2} = .326$						

	Document 1					Document 2				
word	**cnt**	**tf**	**tf-idf**	**n'lized**	**× q**	**cnt**	**tf**	**tf-idf**	**n'lized**	**×q**
sweet	2	1.301	0.163	0.357	**0.137**	1	1.000	0.125	0.203	**0.0779**
nurse	1	1.000	0.301	0.661	0	0	0	0	0	0
love	1	1.000	0.301	0.661	**0.610**	0	0	0	0	**0**
how	0	0	0	0	0	0	0	0	0	0
sorrow	0	0	0	0	0	1	1.000	0.602	0.979	0
is	0	0	0	0	0	0	0	0	0	0
$\|d_1\| = \sqrt{.163^2 + .301^2 + .301^2} = .456$						$\|d_2\| = \sqrt{.125^2 + .602^2} = .615$				
Cosine: $\sum$ of column: **0.747**						Cosine: $\sum$ of column: **0.0779**				

Figure 11.6 Computation of tf-idf cosine score between the query and nano-documents 1 (0.747) and 2 (0.0779), using Eq. 11.4, Eq. 11.5, Eq. 11.6 and Eq. 11.9.

missing one of the terms. We leave the computation for Documents 3 and 4 as an exercise for the reader.

In practice, there are many variants and approximations to Eq. 11.9. For example, we might choose to simplify processing by removing some terms. To see this, let's start by expanding the formula for tf-idf in Eq. 11.9 to explicitly mention the tf and idf terms from Eq. 11.6:

$$\text{score}(q,d) = \sum_{t \in \mathbf{q}} \frac{\text{tf}_{t,q} \cdot \text{idf}_t}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i,q)}} \cdot \frac{\text{tf}_{t,d} \cdot \text{idf}_t}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i,d)}} \tag{11.10}$$

In one common variant of tf-idf cosine, for example, we drop the idf term for the document. Eliminating the second copy of the idf term (since the identical term is already computed for the query) turns out to sometimes result in better performance:

$$\text{score}(q,d) = \sum_{t \in \mathbf{q}} \frac{\text{tf}_{t,q} \cdot \text{idf}_t}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i,q)}} \cdot \frac{\text{tf}_{t,d} \cdot \text{idf}_t}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i,d)}} \tag{11.11}$$

Other variants of tf-idf eliminate various other terms.

BM25 A slightly more complex variant in the tf-idf family is the **BM25** weighting scheme (sometimes called Okapi BM25 after the Okapi IR system in which it was introduced (Robertson et al., 1995)). BM25 adds two parameters: k, a knob that adjusts the balance between term frequency and IDF, and b, which controls the importance of document length normalization. The BM25 score of a document d given

a query q is:

$$\sum_{t\in q} \overbrace{\log\left(\frac{N}{df_t}\right)}^{\text{IDF}} \overbrace{\frac{tf_{t,d}}{k\left(1-b+b\left(\frac{|d|}{|d_{\text{avg}}|}\right)\right)+tf_{t,d}}}^{\text{weighted tf}} \tag{11.12}$$

where $|d_{\text{avg}}|$ is the length of the average document. When k is 0, BM25 reverts to no use of term frequency, just a binary selection of terms in the query (plus idf). A large k results in raw term frequency (plus idf). b ranges from 1 (scaling by document length) to 0 (no length scaling). Manning et al. (2008) suggest reasonable values are k = [1.2,2] and b = 0.75. Kamphuis et al. (2020) is a useful summary of the many minor variants of BM25.

Stop words In the past it was common to remove high-frequency words from both the query and document before representing them. The list of such high-frequency words to be removed is called a **stop list**. The intuition is that high-frequency terms (often function words like *the*, *a*, *to*) carry little semantic weight and may not help with retrieval, and can also help shrink the inverted index files we describe below. The downside of using a stop list is that it makes it difficult to search for phrases that contain words in the stop list. For example, common stop lists would reduce the phrase *to be or not to be* to the phrase *not*. In modern IR systems, the use of stop lists is much less common, partly due to improved efficiency and partly because much of their function is already handled by IDF weighting, which downweights function words that occur in every document. Nonetheless, stop word removal is occasionally useful in various NLP tasks so is worth keeping in mind.

stop list

11.1.4 Efficiently finding documents: the Inverted Index

In order to compute scores, we need to efficiently find documents that contain words in the query. (Any document that contains none of the query terms will have a score of 0 and can be ignored.) The basic search problem in IR is thus to find all documents $d \in C$ that contain a term $q \in Q$.

inverted index

The data structure for this task is the **inverted index**, which we use for making this search efficient, and also conveniently storing useful information like the document frequency and the count of each term in each document.

postings

An inverted index, given a query term, gives a list of documents that contain the term. It consists of two parts, a **dictionary** and the **postings**. The dictionary is a list of terms (designed to be efficiently accessed), each pointing to a **postings list** for the term. A postings list is the list of document IDs associated with each term, which can also contain information like the term frequency or even the exact positions of terms in the document. The dictionary can also store the document frequency for each term. For example, a simple inverted index for our 4 sample documents above, with each word containing its document frequency in {}, and a pointer to a postings list that contains document IDs and term counts in [], might look like the following:

how {1} → 3 [1]
is {1} → 3 [1]
love {2} → 1 [1] → 3 [1]
nurse {2} → 1 [1] → 4 [1]
sorrow {1} → 2 [1]
sweet {3} → 1 [2] → 2 [1] → 3 [1]

Given a list of terms in query, we can very efficiently get lists of all candidate documents, together with the information necessary to compute the tf-idf scores we need.

11.2 Evaluation of Information-Retrieval Systems

We measure the performance of ranked retrieval systems using the same **precision** and **recall** metrics we have been using. We make the assumption that each document returned by the IR system is either **relevant** to our purposes or **not relevant**. Precision is the fraction of the returned documents that are relevant, and recall is the fraction of all relevant documents that are returned. More formally, let's assume a system returns T ranked documents in response to an information request, a subset R of these are relevant, a disjoint subset, N, are the remaining irrelevant documents, and U documents in the collection as a whole are relevant to this request. Precision and recall are then defined as:

$$Precision = \frac{|R|}{|T|} \qquad Recall = \frac{|R|}{|U|} \tag{11.13}$$

Unfortunately, these metrics don't adequately measure the performance of a system that *ranks* the documents it returns. If we are comparing the performance of two ranked retrieval systems, we need a metric that prefers the one that ranks the relevant documents higher. We need to adapt precision and recall to capture how well a system does at putting relevant documents higher in the ranking.

Let's turn to an example. Assume the table in Fig. 11.7 gives rank-specific precision and recall values calculated as we proceed down through a set of ranked documents for a particular query; the precisions are the fraction of relevant documents seen at a given rank, and recalls the fraction of relevant documents found at the same rank. The recall measures in this example are based on this query having 9 relevant documents in the collection as a whole.

Note that recall is non-decreasing; when a relevant document is encountered, recall increases, and when a non-relevant document is found it remains unchanged. Precision, on the other hand, jumps up and down, increasing when relevant documents are found, and decreasing otherwise. The most common way to visualize precision and recall is to plot precision against recall in a **precision-recall curve**, like the one shown in Fig. 11.8 for the data in table 11.7.

precision-recall curve

Fig. 11.8 shows the values for a single query. But we'll need to combine values for all the queries, and in a way that lets us compare one system to another. One way of doing this is to plot averaged precision values at 11 fixed levels of recall (0 to 100, in steps of 10). Since we're not likely to have datapoints at these exact levels, we use **interpolated precision** values for the 11 recall values from the data points we do have. We can accomplish this by choosing the maximum precision value achieved at any level of recall at or above the one we're calculating. In other words,

interpolated precision

$$\text{IntPrecision}(r) = \max_{i>=r} \text{Precision}(i) \tag{11.14}$$

This interpolation scheme not only lets us average performance over a set of queries, but also helps smooth over the irregular precision values in the original data. It is designed to give systems the benefit of the doubt by assigning the maximum precision value achieved at higher levels of recall from the one being measured. Fig. 11.9 and Fig. 11.10 show the resulting interpolated data points from our example.

Rank	Judgment	Precision_{Rank}	Recall_{Rank}
1	R	1.0	.11
2	N	.50	.11
3	R	.66	.22
4	N	.50	.22
5	R	.60	.33
6	R	.66	.44
7	N	.57	.44
8	R	.63	.55
9	N	.55	.55
10	N	.50	.55
11	R	.55	.66
12	N	.50	.66
13	N	.46	.66
14	N	.43	.66
15	R	.47	.77
16	N	.44	.77
17	N	.41	.77
18	R	.44	.88
19	N	.42	.88
20	N	.40	.88
21	N	.38	.88
22	N	.36	.88
23	N	.35	.88
24	N	.33	.88
25	R	.36	1.0

Figure 11.7 Rank-specific precision and recall values calculated as we proceed down through a set of ranked documents (assuming the collection has 9 relevant documents).

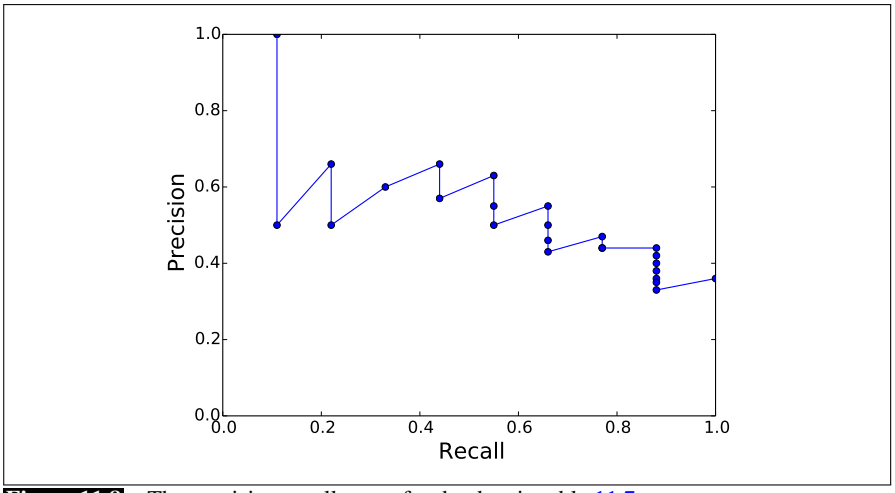


Figure 11.8 The precision recall curve for the data in table 11.7.

Given curves such as that in Fig. 11.10 we can compare two systems or approaches by comparing their curves. Clearly, curves that are higher in precision across all recall values are preferred. However, these curves can also provide insight into the overall behavior of a system. Systems that are higher in precision toward the left may favor precision over recall, while systems that are more geared towards recall will be higher at higher levels of recall (to the right).

mean average precision

A second way to evaluate ranked retrieval is **mean average precision** (MAP),

Interpolated Precision	Recall
1.0	0.0
1.0	.10
.66	.20
.66	.30
.66	.40
.63	.50
.55	.60
.47	.70
.44	.80
.36	.90
.36	1.0

Figure 11.9 Interpolated data points from Fig. 11.7.

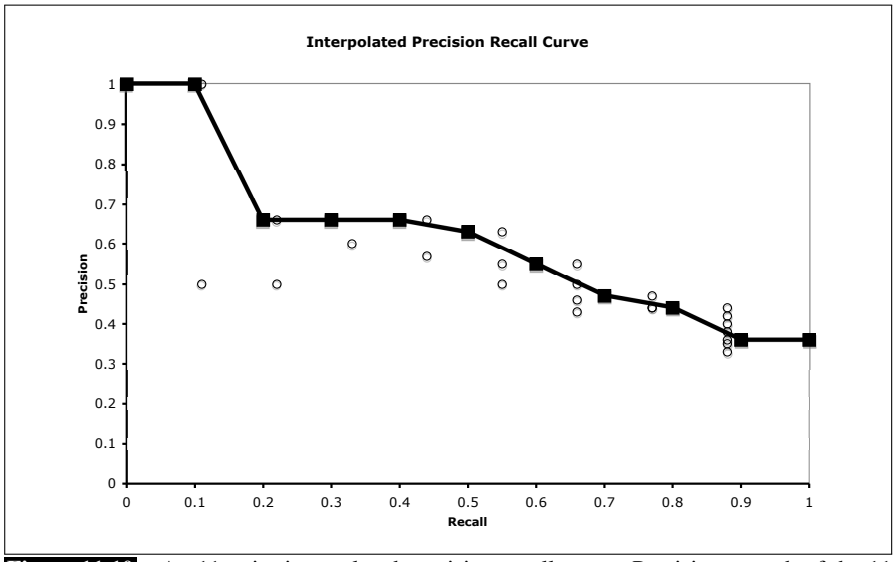


Figure 11.10 An 11 point interpolated precision-recall curve. Precision at each of the 11 standard recall levels is interpolated for each query from the maximum at any higher level of recall. The original measured precision recall points are also shown.

which provides a single metric that can be used to compare competing systems or approaches. In this approach, we again descend through the ranked list of items, but now we note the precision **only** at those points where a relevant item has been encountered (for example at ranks 1, 3, 5, 6 but not 2 or 4 in Fig. 11.7). For a single query, we average these individual precision measurements over the return set (up to some fixed cutoff). More formally, if we assume that R_r is the set of relevant documents at or above r, then the **average precision** (**AP**) for a single query is

$$\text{AP} = \frac{1}{|R_r|} \sum_{d \in R_r} \text{Precision}_r(d) \tag{11.15}$$

where $Precision_r(d)$ is the precision measured at the rank at which document d was found. For an ensemble of queries Q, we then average over these averages, to get our final MAP measure:

$$\text{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \text{AP}(q) \tag{11.16}$$

The MAP for the single query (hence = AP) in Fig. 11.7 is 0.6.

11.3 Information Retrieval with Dense Vectors

The classic tf-idf or BM25 algorithms for IR have long been known to have a conceptual flaw: they work only if there is exact overlap of words between the query and document. In other words, the user posing a query (or asking a question) needs to guess exactly what words the writer of the answer might have used, an issue called the **vocabulary mismatch problem** (Furnas et al., 1987).

The solution to this problem is to use an approach that can handle synonymy: instead of (sparse) word-count vectors, using (dense) embeddings. This idea was first proposed for retrieval in the last century under the name of Latent Semantic Indexing approach (Deerwester et al., 1990), but is implemented in modern times via encoders like BERT.

The most powerful approach is to present both the query and the document to a single encoder, allowing the transformer self-attention to see all the tokens of both the query and the document, and thus building a representation that is sensitive to the meanings of both query and document. Then a linear layer can be put on top of the [CLS] token to predict a similarity score for the query/document tuple:

$$\begin{aligned} \mathbf{z} &= \text{BERT}(\text{q}; \texttt{[SEP]}; \text{d})\texttt{[CLS]} \\ \text{score}(q,d) &= \text{softmax}(\mathbf{U}(\mathbf{z})) \end{aligned} \tag{11.17}$$

This architecture is shown in Fig. 11.11a. Usually the retrieval step is not done on an entire document. Instead documents are broken up into smaller passages, such as non-overlapping fixed-length chunks of say 100 tokens, and the retriever encodes and retrieves these passages rather than entire documents. The query and document have to be made to fit in the BERT 512-token window, for example by truncating the query to 64 tokens and truncating the document if necessary so that it, the query, [CLS], and [SEP] fit in 512 tokens. The BERT system together with the linear layer $\mathbf{U}$ can then be fine-tuned for the relevance task by gathering a tuning dataset of relevant and non-relevant passages.

The problem with the full BERT architecture in Fig. 11.11a is the expense in computation and time. With this architecture, every time we get a query, we have to pass every single document in our entire collection through a BERT encoder jointly with the new query! This enormous use of resources is impractical for real cases.

At the other end of the computational spectrum is a much more efficient architecture, the **bi-encoder**. In this architecture we can encode the documents in the collection only one time by using two separate encoder models, one to encode the query and one to encode the document. We encode each document, and store all the encoded document vectors in advance. When a query comes in, we encode just this query and then use the dot product between the query vector and the precomputed document vectors as the score for each candidate document (Fig. 11.11b). For example, if we used BERT, we would have two encoders BERT_Q and BERT_D and we could represent the query and document as the [CLS] token of the respective encoders (Karpukhin et al., 2020):

$$\begin{aligned} \mathbf{z}_q &= \text{BERT}_Q(\text{q})\texttt{[CLS]} \\ \mathbf{z}_d &= \text{BERT}_D(\text{d})\texttt{[CLS]} \\ \text{score}(q,d) &= \mathbf{z}_q \cdot \mathbf{z}_d \end{aligned} \tag{11.18}$$

The bi-encoder is much cheaper than a full query/document encoder, but is also less accurate, since its relevance decision can't take full advantage of all the possi-

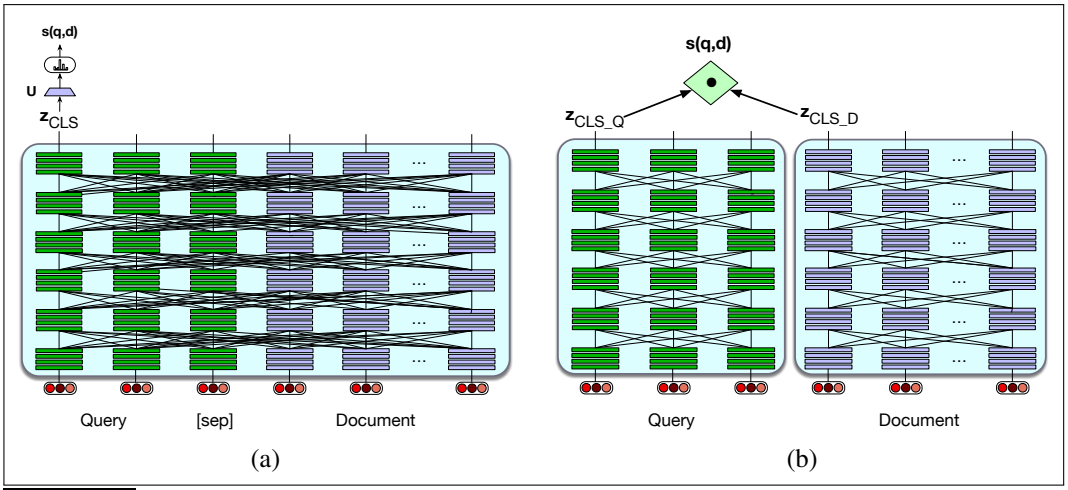


Figure 11.11 Two ways to do dense retrieval, illustrated by using lines between layers to schematically represent self-attention: (a) Use a single encoder to jointly encode query and document and finetune to produce a relevance score with a linear layer over the CLS token. This is too compute-expensive to use except in rescoring (b) Use separate encoders for query and document, and use the dot product between CLS token outputs for the query and document as the score. This is less compute-expensive, but not as accurate.

ble meaning interactions between all the tokens in the query and the tokens in the document.

There are numerous approaches that lie in between the full encoder and the bi-encoder. One intermediate alternative is to use cheaper methods (like BM25) as the first pass relevance ranking for each document, take the top N ranked documents, and use expensive methods like the full BERT scoring to rerank only the top N documents rather than the whole set.

ColBERT

Another intermediate approach is the **ColBERT** approach of Khattab and Zaharia (2020) and Khattab et al. (2021), shown in Fig. 11.12. This method separately encodes the query and document, but rather than encoding the entire query or document into one vector, it separately encodes each of them into contextual representations for each token. These BERT representations of each document word can be pre-stored for efficiency. The relevance score between a query q and a document d is a sum of maximum similarity (MaxSim) operators between tokens in q and tokens in d. Essentially, for each token in q, ColBERT finds the most contextually similar token in d, and then sums up these similarities. A relevant document will have tokens that are contextually very similar to the query.

More formally, a question q is tokenized as $[q_1,\ldots,q_n]$, prepended with a `[CLS]` and a special `[Q]` token, truncated to N=32 tokens (or padded with `[MASK]` tokens if it is shorter), and passed through BERT to get output vectors $\mathbf{q} = [\mathbf{q}_1,\ldots,\mathbf{q}_N]$. The passage d with tokens $[d_1,\ldots,d_m]$, is processed similarly, including a `[CLS]` and special `[D]` token. A linear layer is applied on top of $\mathbf{d}$ and $\mathbf{q}$ to control the output dimension, so as to keep the vectors small for storage efficiency, and vectors are rescaled to unit length, producing the final vector sequences $\mathbf{E}_q$ (length N) and $\mathbf{E}_d$ (length m). The ColBERT scoring mechanism is:

$$\text{score}(q,d) = \sum_{i=1}^{N} \max_{j=1}^{m} \mathbf{E}_{q_i} \cdot \mathbf{E}_{d_j} \tag{11.19}$$

While the interaction mechanism has no tunable parameters, the ColBERT ar-

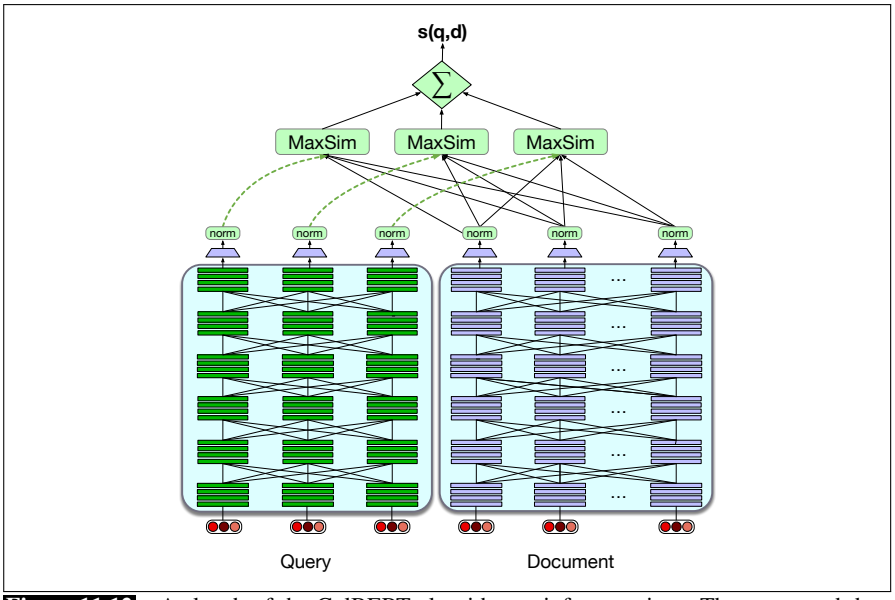


Figure 11.12 A sketch of the ColBERT algorithm at inference time. The query and document are first passed through separate BERT encoders. Similarity between query and document is computed by summing a soft alignment between the contextual representations of tokens in the query and the document. Training is end-to-end. (Various details aren't depicted; for example the query is prepended by a `[CLS]` and `[Q:]` tokens, and the document by `[CLS]` and `[D:]` tokens). Figure adapted from Khattab and Zaharia (2020).

chitecture still needs to be trained end-to-end to fine-tune the BERT encoders and train the linear layers (and the special `[Q]` and `[D]` embeddings) from scratch. It is trained on triples $\langle q, d^+, d^- \rangle$ of query q, positive document d^+ and negative document d^- to produce a score for each document using Eq. 11.19, optimizing model parameters using a cross-entropy loss.

All the supervised algorithms (like ColBERT or the full-interaction version of the BERT algorithm applied for reranking) need training data in the form of queries together with relevant and irrelevant passages or documents (positive and negative examples). There are various semi-supervised ways to get labels; some datasets (like MS MARCO Ranking, Section 11.5) contain gold positive examples. Negative examples can be sampled randomly from the top-1000 results from some existing IR system. If datasets don't have labeled positive examples, iterative methods like **relevance-guided supervision** can be used (Khattab et al., 2021) which rely on the fact that many datasets contain short answer strings. In this method, an existing IR system is used to harvest examples that do contain short answer strings (the top few are taken as positives) or don't contain short answer strings (the top few are taken as negatives), these are used to train a new retriever, and then the process is iterated.

Efficiency is an important issue, since every possible document must be ranked for its similarity to the query. For sparse word-count vectors, the inverted index allows this very efficiently. For dense vector algorithms finding the set of dense document vectors that have the highest dot product with a dense query vector is an instance of the problem of **nearest neighbor search**. Modern systems therefore make use of approximate nearest neighbor vector search algorithms like **Faiss** (Johnson et al., 2017).

Faiss

11.4 Retrieval-Augmented Generation (RAG)

The information retrieval techniques we introduced in the prior section can be integrated into language models via a method called **retrieval-augmented generation** or **RAG**. In the basic RAG scenario that we will describe in this section, we use IR techniques to retrieve documents from some specified store of documents that are likely to have useful information. Then we use a large language model to **generate** an answer conditioned on these documents in addition to the original query.

As we summarized in the introduction to the chapter, there are many goals of retrieval-augmented generation. RAG can help mitigate hallucination, by giving the model a set of trusted documents. RAG can also help language models generate factual text about proprietary data, like personal email, or health records, or company-internal documents, or other legal documents. RAG can also help with the problem that knowledge is dynamic and time-sensitive, for example if we know the user's information need references data from a time after a language model was trained.

A RAG system is based on two major components: the **retriever** and the **generator**, (the latter is sometimes called, for historical reasons, the **reader** (Chen et al., 2017a)). Fig. 11.13 sketches out this standard model for answering questions.

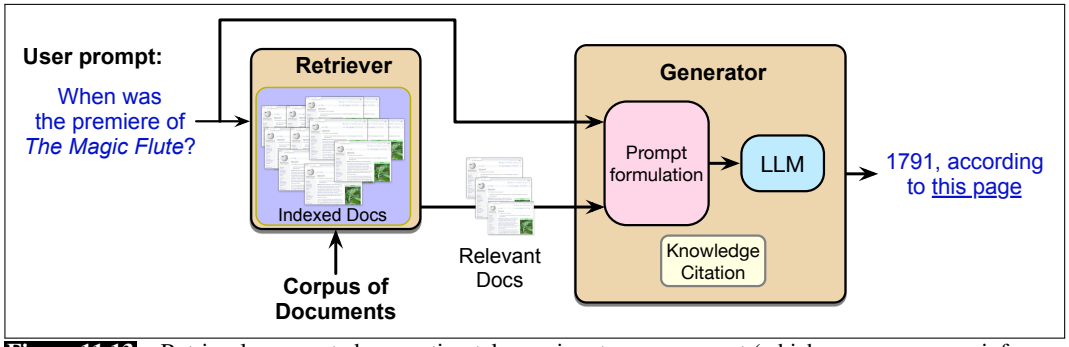


Figure 11.13 Retrieval-augmented generation takes as input a user prompt (which may express an information need like this question example), and a corpus of documents that may be useful in meeting the information need. The method has two stages: **retrieval**, which returns relevant documents from the collection, and **generation**, in which an LLM **generates** text given the documents as a prompt. Some generations include a **knowledge citation** that can help the user decide whether to trust the generation, or follow up if they are interested.

retrieval-augmented generation RAG

In the first stage of the **retrieval-augmented generation**, or **RAG** model shown in Fig. 11.13 we **retrieve** relevant passages from some prespecified text collection, for example using the dense retrievers of the previous section. In the second **generate** stage, we take the set of retrieved passages, integrate it with the user prompt, and pass some version of these to a large language model to generate an answer conditioned on these two things.

For example imagine the user asks the question `What year was the premiere of *The Magic Flute*?`. We pass this question to a dense retriever and return a series of passages about The Magic Flute.

The idea of retrieval-augmented generation is to condition on the retrieved passages, jointly with some prompt text, for example like "Based on these texts, answer this question:". Thus given a document collection $\mathcal{D}$ and a user query q, the most basic RAG algorithm is:

1. Call a retriever to return $R(q) = d_1 \cdots d_k$, the top-k relevant passages from $\mathcal{D}$

2. Create a prompt that includes q and the retrieved passages
3. Call an LLM with the prompt

The resulting prompts might look something like:

Schematic of a RAG Prompt

```
retrieved passage 1

retrieved passage 2

...

retrieved passage k

Based on these texts, answer this question:  What year
was the premiere of The Magic Flute?
```

The task for the language model is then to generate text according to this probability model:

$$p(x_1,\ldots,x_n) \;=\; \prod_{i=1}^{n} p(x_i|\mathrm{R}(q)\,;\,\text{Answer the following question...}\;;\,q\,;x_{<i})$$

There are many augmentations of this basic RAG paradigm. One addition is the use of **agent-based RAG**. In the RAG paradigm described so far, a search is always run and then retrieved passages are combined with the user's question in a prompt. But in actual applications, we may not want to run retrieval for every user turn. Or we may want to retrieve from different collections for different user needs (sometimes the web, other times a private collection). In **agent-based RAG**, the system decides when to call a retrieval agent and for which collection.

Another research area has to do with the relationship between the retriever and the generator. For example there may be noise in the retrieved passages; some of them may be irrelevant or wrong, or in an unhelpful order. How can we encourage the LLM to focus on the good passages? Some RAG architectures add a reranker that reranks or reorders passages after they are retrieved. Or some complex questions may require multi-hop architectures, in which a query is used to retrieve documents, which are then appended to the original query for a second stage of retrieval.

Another class of solutions is to train the LLM for RAG. The basic version of RAG describe above involves no training; we take an off-the-shelf LLM, and give it the passages and a prompt and hope that it will correctly figure out which passages are useful or relevant in generating the answer. One learning variant involves instruction-tuning an LLM, by first creating a dataset of questions annotated with retrieved passages and correct answers, and then instruction-tuning the LLM to correctly answer the questions from the passages. An alternative method is to do this via test-time compute, prompting the LLM to answer the question and simultaneously to generate reflections on which passages were useful. The process of generating these reflections may lead the LLM to improve at identifying good passages. The resulting reflection text can also be used for in-context learning, for example by using the text as part of a prompt for further questions.

In addition to training the LLM, we could train the IR engine. After all, the IR engine itself has not been optimized for the RAG scenario. It might not have been

trained, or if it was, it was likely trained for simple IR or factoid question-answering tasks, not for the RAG scenario where the retrieved passages are specifically to be used by another LLM for generating texts. We can address this mismatch for trainable IR algorithms by doing end-to-end training of the entire architecture on some set of questions and answers, training the parameters of the IR model as well as the LLM.

knowledge citations

Finally, it is generally useful for LLMs to give the user evidence for any factual statement. This can be in the form of **knowledge citations**, such as URLs of a trusted source or citation references to particular literature. For example a question answering system might generate numbered pointers to URLs as follows:

> Q: Which films have Gong Li as a member of their cast?
> A: The Story of Qiu Ju [1], Farewell My Concubine [2], The Monkey King 2 [3], Mulan [3], Saturday Fiction [3] ...

The simplest way for generating knowledge citations is to specify it as part of the prompt. For example Gao et al. (2023) employ a prompt with text like:

```
  ‘‘Write an answer for the given question using only the
provided search results (some of which might be irrelevant)
and cite them properly...  Always cite for any factual claim".
```

11.5 Datasets

There are scores of datasets that contain information needs in the form of questions, annotated with the answer. These can be used both for instruction tuning and for evaluation of the question answering abilities of language models.

We can distinguish the datasets along many dimensions, summarized nicely in Rogers et al. (2023). One is the original purpose of the questions in the data, whether they were natural **information-seeking** questions, or whether they were questions designed for **probing**: evaluating or testing systems or humans.

Natural Questions

On the natural side there are datasets like **Natural Questions** (Kwiatkowski et al., 2019), a set of anonymized English queries to the Google search engine and their answers. The answers are created by annotators based on Wikipedia information, and include a paragraph-length long answer and a short span answer. For example the question "When are hops added to the brewing process?" has the short answer *the boiling process* and a long answer which is an entire paragraph from the Wikipedia page on *Brewing*.

MS MARCO

A similar natural question set is the **MS MARCO** (Microsoft Machine Reading Comprehension) collection of datasets, including 1 million real anonymized English questions from Microsoft Bing query logs together with a human generated answer and 9 million passages (Bajaj et al., 2016), that can be used both to test retrieval ranking and question answering.

Although many datasets focus on English, natural information-seeking question datasets exist in other languages. The DuReader dataset is a Chinese QA resource based on search engine queries and community QA (He et al., 2018).

TyDi QA

TyDi QA dataset contains 204K question-answer pairs from 11 typologically diverse languages, including Arabic, Bengali, Kiswahili, Russian, and Thai (Clark et al., 2020a). In the TyDi QA task, a system is given a question and the passages from a Wikipedia article and must (a) select the passage containing the answer (or

NULL if no passage contains the answer), and (b) mark the minimal answer span (or NULL).

MMLU

On the probing side are datasets like **MMLU** (Massive Multitask Language Understanding), a commonly-used dataset of 15908 knowledge and reasoning questions in 57 areas including medicine, mathematics, computer science, law, and others. MMLU questions are sourced from various exams for humans, such as the US Graduate Record Exam, Medical Licensing Examination, and Advanced Placement exams. So the questions don't represent people's information needs, but rather are designed to test human knowledge for academic or licensing purposes. Fig. 11.14 shows some examples, with the correct answers in bold.

MMLU examples

College Computer Science
Any set of Boolean operators that is sufficient to represent all Boolean expressions is said to be complete. Which of the following is NOT complete?
(A) AND, NOT
(B) NOT, OR
(C) AND, OR
(D) NAND

College Physics
The primary source of the Sun's energy is a series of thermonuclear reactions in which the energy produced is c^2 times the mass difference between
(A) two hydrogen atoms and one helium atom
(B) four hydrogen atoms and one helium atom
(C) six hydrogen atoms and two helium atoms
(D) three helium atoms and one carbon atom

International Law
Which of the following is a treaty-based human rights mechanism?
(A) The UN Human Rights Committee
(B) The UN Human Rights Council
(C) The UN Universal Periodic Review
(D) The UN special mandates

Prehistory
Unlike most other early civilizations, Minoan culture shows little evidence of
(A) trade.
(B) warfare.
(C) the development of a common religion.
(D) conspicuous consumption by elites.

Figure 11.14 Example problems from MMLU

reading comprehension

Some of the question datasets described above augment each question with passage(s) from which the answer can be extracted. These datasets were mainly created for an earlier QA task called **reading comprehension** in which a model is given a question and a document and is required to extract the answer from the given

document. We sometimes call the task of question answering given one or more documents (for example via RAG), the **open book** QA task, while the task of answering directly from the LM with no retrieval component at all is the **closed book** QA task.[5] Thus datasets like Natural Questions can be treated as open book if the solver uses each question's attached document, or closed book if the documents are not used, while datasets like MMLU are solely closed book.

open book
closed book

Another dimension of variation is the format of the answer: multiple-choice versus freeform. And of course there are variations in prompting, like whether the model is just the question (zero-shot) or also given demonstrations of answers to similar questions (few-shot). MMLU offers both zero-shot and few-shot prompt options.

11.6 Evaluating Question Answering

Two techniques are commonly employed to evaluate question-answering systems, with the choice depending on the type of question and QA situation. For **multiple choice** questions like in MMLU, we report exact match:

Exact match: The % of predicted answers that match the gold answer exactly.

For questions with **free text** answers, like Natural Questions, we commonly evaluated with token $\mathbf{F_1}$ **score** to roughly measure the partial string overlap between the answer and the reference answer:

$\mathbf{F_1}$ score: The average token overlap between predicted and gold answers. Treat the prediction and gold as a bag of tokens, and compute F_1 for each question, then return the average F_1 over all questions.

11.7 Summary

This chapter introduced the tasks of **information retrieval** and the use of **retrieval augmented generation** (**RAG**) to use retrieved passages to improve **question answering** and other factual generations from LLMs.

- We focus in this chapter on the use of information retrieval for question answering and related factually-based tasks. The idea is to meet the user's information needs by drawing on the material in some set of documents (which might be the web).
- **Information Retrieval** (**IR**) is the task of returning documents to a user based on their information need as expressed in a **query**. In ranked retrieval, the documents are returned in ranked order.
- Two paradigms for IR are **sparse retrieval** and **dense retrieval**. Both paradigms use a document's similarity to the query as an estimate of its relevance to the user's information need
- In sparse retrieval techniques, we represent both the query and the document as sparse vectors of the unigram counts of the words they contain, each count

[5] This repurposes the word for types of exams in which students are allowed to 'open their books' or not.

weighted by **tf-idf** or **BM25**. Then the query-document similarity can be measured by the cosine between these sparse vectors.

- The **inverted index** is a storage mechanism for sparse retrieval that makes it very efficient to find documents that have a particular word.
- In dense retrieval techniques, documents or queries are instead represented as embeddings (dense vectors) computed by a language model (whether encoder-only models like the BERT family, or decoder-only). Document-query similarity is computed as dot product or cosine in they embedding space.
- For dense retrieval, FAISS is an approximate nearest neighbor vector search algorithm that makes it very efficient to find the k most similar document embeddings to a query embedding, making it quick to do ranking.
- Ranked retrieval is generally evaluated by **mean average precision** or **interpolated precision**.
- Retrieval can be incorporated into language modeling via **retrieval-augmented generation**. In the **retrieval** step, the user query is passed to the search engine to retrieve a set of relevant documents or passages. In the **generation** stage, a large language model is prompted with the query and a set of documents retrieved from the collection, and then conditionally generates an answer.
- Factual tasks like question answering can be evaluated by exact match with a known answer if only a single answer is given, with token F_1 score for free text answers.

Historical Notes

Question answering was one of the earliest NLP tasks. By 1961 the BASEBALL system (Green et al., 1961) answered questions about baseball games like "Where did the Red Sox play on July 7" by querying a structured database of game information. The database was stored as a kind of attribute-value matrix with values for attributes of each game:

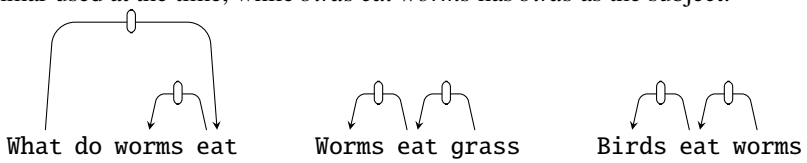
```
Month = July
   Place = Boston
       Day  = 7
       Game Serial No.  = 96
       (Team = Red Sox, Score = 5)
       (Team = Yankees, Score = 3)
```

Each question was constituency-parsed using the algorithm of Zellig Harris's TDAP project at the University of Pennsylvania, essentially a cascade of finite-state transducers (see the historical discussion in Joshi and Hopely 1999 and Karttunen 1999). Then in a content analysis phase each word or phrase was associated with a program that computed parts of its meaning. Thus the phrase 'Where' had code to assign the semantics `Place = ?`, with the result that the question "Where did the Red Sox play on July 7" was assigned the meaning

```
Place = ?
Team = Red Sox
Month = July
Day = 7
```

The question is then matched against the database to return the answer.

The Protosynthex system of Simmons et al. (1964), given a question, formed a query from the content words in the question, and then retrieved candidate answer sentences in the document, ranked by their frequency-weighted term overlap with the question. The query and each retrieved sentence were then parsed with dependency parsers, and the sentence whose structure best matches the question structure selected. Thus the question *What do worms eat?* would match *worms eat grass*: both have the subject *worms* as a dependent of *eat*, in the version of dependency grammar used at the time, while *birds eat worms* has *birds* as the subject:



Simmons (1965) summarizes other early QA systems.

LUNAR

By the 1970s, systems used predicate calculus as the meaning representation language. The **LUNAR** system (Woods et al. 1972, Woods 1978) was designed to be a natural language interface to a database of chemical facts about lunar geology. It could answer questions like *Do any samples have greater than 13 percent aluminum* by parsing them into a logical form

(TEST (FOR SOME X16 / (SEQ SAMPLES) : T ; (CONTAIN' X16
(NPR* X17 / (QUOTE AL203)) (GREATERTHAN 13 PCT))))

By the 1990s question answering shifted to machine learning. Zelle and Mooney (1996) proposed to treat question answering as a semantic parsing task, by creating the Prolog-based GEOQUERY dataset of questions about US geography. This model was extended by Zettlemoyer and Collins (2005) and 2007. By a decade later, neural models were applied to semantic parsing (Dong and Lapata 2016, Jia and Liang 2016), and then to knowledge-based question answering by mapping text to SQL (Iyer et al., 2017).

[TBD: History of IR.]

Meanwhile, a paradigm for answering questions that drew more on information-retrieval was influenced by the rise of the web in the 1990s. The U.S. government-sponsored TREC (Text REtrieval Conference) evaluations, run annually since 1992, provide a testbed for evaluating information-retrieval tasks and techniques (Voorhees and Harman, 2005). TREC added an influential QA track in 1999, which led to a wide variety of factoid and non-factoid question answering systems competing in annual evaluations.

At that same time, Hirschman et al. (1999) introduced the idea of using children's reading comprehension tests to evaluate machine text comprehension algorithms. They acquired a corpus of 120 passages with 5 questions each designed for 3rd-6th grade children, built an answer extraction system, and measured how well the answers given by their system corresponded to the answer key from the test's publisher. Their algorithm focused on word overlap as a feature; later algorithms added named entity features and more complex similarity between the question and the answer span (Riloff and Thelen 2000, Ng et al. 2000).

The DeepQA component of the Watson Jeopardy! system was a large and sophisticated feature-based system developed just before neural systems became common. It is described in a series of papers in volume 56 of the IBM Journal of Research and Development, e.g., Ferrucci (2012).

Early neural reading comprehension systems drew on the insight common to

early systems that answer finding should focus on question-passage similarity. Many of the architectural outlines of these neural systems were laid out in Hermann et al. (2015), Chen et al. (2017a), and Seo et al. (2017). These systems focused on datasets like Rajpurkar et al. (2016) and Rajpurkar et al. (2018) and their successors, usually using separate IR algorithms as input to neural reading comprehension systems. The paradigm of using dense retrieval with a span-based reader, often with a single end-to-end architecture, is exemplified by systems like Lee et al. (2019) or Karpukhin et al. (2020). An important research area with dense retrieval for open-domain QA is training data: using self-supervised methods to avoid having to label positive and negative passages (Sachan et al., 2023).

Early work on large language models showed that they stored sufficient knowledge in the pretraining process to answer questions (Petroni et al., 2019; Raffel et al., 2020; Radford et al., 2019; Roberts et al., 2020), at first not competitively with special-purpose question answerers, but quickly surpassing them. Retrieval-augmented generation algorithms were first introduced as a way to improve language modeling word prediction (Khandelwal et al., 2019), but were quickly applied to question answering (Izacard et al., 2022; Ram et al., 2023; Shi et al., 2023).

Exercises

CHAPTER

12 Machine Translation

> "I want to talk the dialect of your people. It's no use of talking unless people understand what you say."
>
> Zora Neale Hurston, *Moses, Man of the Mountain* 1939, p. 121

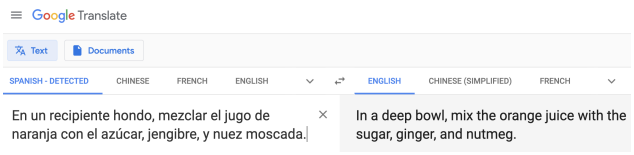
machine translation
MT

This chapter introduces **machine translation** (**MT**), the use of computers to translate from one language to another.

Of course translation, in its full generality, such as the translation of literature, or poetry, is a difficult, fascinating, and intensely human endeavor, as rich as any other area of human creativity.

information access

Machine translation in its present form therefore focuses on a number of very practical tasks. Perhaps the most common current use of machine translation is for **information access**. We might want to translate some instructions on the web, perhaps the recipe for a favorite dish, or the steps for putting together some furniture. Or we might want to read an article in a newspaper, or get information from an online resource like Wikipedia or a government webpage in some other language. MT for information access is probably one of the most common uses of NLP technology, and Google Translate alone (shown above) translates hundreds of billions of words a day between over 100 languages. Improvements in machine translation can thus help reduce what is often called the **digital divide** in information access: the fact that much more information is available in English and other languages spoken in wealthy countries. Web searches in English return much more information than searches in other languages, and online resources like Wikipedia are much larger in English and other higher-resourced languages. High-quality translation can help provide information to speakers of lower-resourced languages.



digital divide

post-editing
CAT
localization

Another common use of machine translation is to aid human translators. MT systems are routinely used to produce a draft translation that is fixed up in a **post-editing** phase by a human translator. This task is often called **computer-aided translation** or **CAT**. CAT is commonly used as part of **localization**: the task of adapting content or a product to a particular language community.

Finally, a more recent application of MT is to in-the-moment human communication needs. This includes incremental translation, translating speech on-the-fly before the entire sentence is complete, as is commonly used in simultaneous interpretation. Image-centric translation can be used for example to use OCR of the text on a phone camera image as input to an MT system to translate menus or street signs.

encoder-decoder

The standard algorithm for MT is the **encoder-decoder** model. We briefly mentioned in Chapter 7 that encoder-decoder or sequence-to-sequence models are used for tasks in which we need to map an input sequence to an output sequence that is a complex function of the entire input sequence, like machine translation or speech

recognition. Indeed, in machine translation, the words of the target language don't necessarily agree with the words of the source language in number or order. Consider translating the following made-up English sentence into Japanese.

(12.1) English: *He wrote a letter to a friend*
Japanese: *tomodachi ni tegami-o kaita*
friend to letter wrote

Note that the elements of the sentences are in very different places in the different languages. In English, the verb is in the middle of the sentence, while in Japanese, the verb *kaita* comes at the end. The Japanese sentence doesn't require the pronoun *he*, while English does.

Such differences between languages can be quite complex. In the following actual sentence from the United Nations, notice the many changes between the Chinese sentence (we've given in red a word-by-word gloss of the Chinese characters) and its English equivalent produced by human translators.

(12.2) 大会/General Assembly 在/on 1982年/1982 12月/December 10日/10 通过了/adopted 第37号/37th 决议/resolution ， 核准了/approved 第二次/second 探索/exploration 及/and 和平peaceful 利用/using 外层空间/outer space 会议/conference 的/of 各项/various 建议/suggestions 。

On 10 December 1982 , the General Assembly adopted resolution 37 in which it endorsed the recommendations of the Second United Nations Conference on the Exploration and Peaceful Uses of Outer Space .

Note the many ways the English and Chinese differ. For example the ordering differs in major ways; the Chinese order of the noun phrase is "peaceful using outer space conference of suggestions" while the English has "suggestions of the ... conference on peaceful use of outer space"). And the order differs in minor ways (the date is ordered differently). English requires *the* in many places that Chinese doesn't, and adds some details (like "in which" and "it") that aren't necessary in Chinese. Chinese doesn't grammatically mark plurality on nouns (unlike English, which has the "-s" in "recommendations"), and so the Chinese must use the modifier 各项/*various* to make it clear that there is not just one recommendation. English capitalizes some words but not others. Encoder-decoder networks are very successful at handling these sorts of complicated cases of sequence mappings.

We'll begin in the next section by considering the linguistic background about how languages vary, and the implications this variance has for the task of MT. Then we'll sketch out the standard algorithm, give details about things like input tokenization and creating training corpora of parallel sentences, give some more low-level details about the encoder-decoder network, and finally discuss how MT is evaluated, introducing the simple chrF metric.

12.1 Language Divergences and Typology

universal

There are about 7,000 languages in the world. Some aspects of human language seem to be **universal**, holding true for every one of these languages, or are statistical universals, holding true for most of these languages. Many universals arise from the functional role of language as a communicative system by humans. Every language, for example, seems to have words for referring to people, for talking about eating and drinking, for being polite or not. There are also structural linguistic universals; for example, every language seems to have nouns and verbs (Chapter 17), has

ways to ask questions, or issue commands, has linguistic mechanisms for indicating agreement or disagreement.

translation divergence

Yet languages also **differ** in many ways (as has been pointed out since ancient times; see Fig. 12.1). Understanding what causes such **translation divergences** (Dorr, 1994) can help us build better MT models. We often distinguish the **idiosyncratic** and lexical differences that must be dealt with one by one (the word for "dog" differs wildly from language to language), from **systematic** differences that we can model in a general way (many languages put the verb before the grammatical object; others put the verb after the grammatical object). The study of these systematic cross-linguistic similarities and differences is called **linguistic typology**. This section sketches some typological facts that impact machine translation; the interested reader should also look into WALS, the World Atlas of Language Structures, which gives many typological facts about languages (Dryer and Haspelmath, 2013).

typology

Figure 12.1 The Tower of Babel, Pieter Bruegel 1563. Wikimedia Commons, from the Kunsthistorisches Museum, Vienna.

12.1.1 Word Order Typology

As we hinted at in our example above comparing English and Japanese, languages differ in the basic word order of verbs, subjects, and objects in simple declarative clauses. German, French, English, and Mandarin, for example, are all **SVO** (**Subject-Verb-Object**) languages, meaning that the verb tends to come between the subject and object. Hindi and Japanese, by contrast, are **SOV** languages, meaning that the verb tends to come at the end of basic clauses, and Irish and Arabic are **VSO** languages. Two languages that share their basic word order type often have other similarities. For example, **VO** languages generally have **prepositions**, whereas **OV** languages generally have **postpositions**.

SVO

SOV

VSO

Let's look in more detail at the example we saw above. In this SVO English sentence, the verb *wrote* is followed by its object *a letter* and the prepositional phrase

to a friend, in which the preposition *to* is followed by its argument *a friend*. Arabic, with a VSO order, also has the verb before the object and prepositions. By contrast, in the Japanese example that follows, each of these orderings is reversed; the verb is *preceded* by its arguments, and the postposition follows its argument.

(12.3) English: *He wrote a letter to a friend*
Japanese: *tomodachi ni tegami-o kaita*
friend to letter wrote
Arabic: *katabt risāla li ṡadq*
wrote letter to friend

Other kinds of ordering preferences vary idiosyncratically from language to language. In some SVO languages (like English and Mandarin) adjectives tend to appear before nouns, while in others languages like Spanish and Modern Hebrew, adjectives appear after the noun:

(12.4) **Spanish** *bruja verde* **English** *green witch*

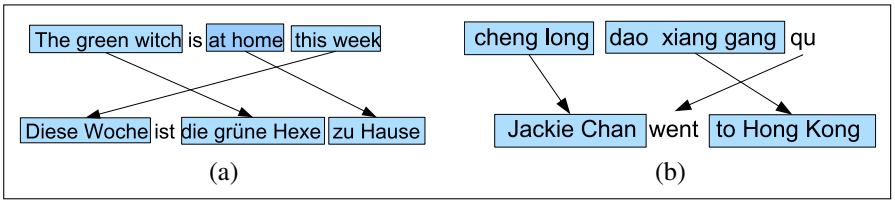


Figure 12.2 Examples of other word order differences: (a) In German, adverbs occur in initial position that in English are more natural later, and tensed verbs occur in second position. (b) In Mandarin, preposition phrases expressing goals often occur pre-verbally, unlike in English.

Fig. 12.2 shows examples of other word order differences. All of these word order differences between languages can cause problems for translation, requiring the system to do huge structural reorderings as it generates the output.

12.1.2 Lexical Divergences

Of course we also need to translate the individual words from one language to another. For any translation, the appropriate word can vary depending on the context. The English source-language word *bass*, for example, can appear in Spanish as the fish *lubina* or the musical instrument *bajo*. German uses two distinct words for what in English would be called a *wall*: *Wand* for walls inside a building, and *Mauer* for walls outside a building. Where English uses the word *brother* for any male sibling, Chinese and many other languages have distinct words for *older brother* and *younger brother* (Mandarin *gege* and *didi*, respectively). In all these cases, translating *bass*, *wall*, or *brother* from English would require a kind of specialization, disambiguating the different uses of a word. For this reason the fields of MT and Word Sense Disambiguation (Appendix G) are closely linked.

Sometimes one language places more grammatical constraints on word choice than another. We saw above that English marks nouns for whether they are singular or plural. Mandarin doesn't. Or French and Spanish, for example, mark grammatical gender on adjectives, so an English translation into French requires specifying adjective gender.

The way that languages differ in lexically dividing up conceptual space may be more complex than this one-to-many translation problem, leading to many-to-many

mappings. For example, Fig. 12.3 summarizes some of the complexities discussed by Hutchins and Somers (1992) in translating English *leg, foot*, and *paw*, to French. For example, when *leg* is used about an animal it's translated as French *patte*; but about the leg of a journey, as French *etape*; if the leg is of a chair, we use French *pied*.

lexical gap

Further, one language may have a **lexical gap**, where no word or phrase, short of an explanatory footnote, can express the exact meaning of a word in the other language. For example, English does not have a word that corresponds neatly to Mandarin *xiào* or Japanese *oyakōkō* (in English one has to make do with awkward phrases like *filial piety* or *loving child*, or *good son/daughter* for both).

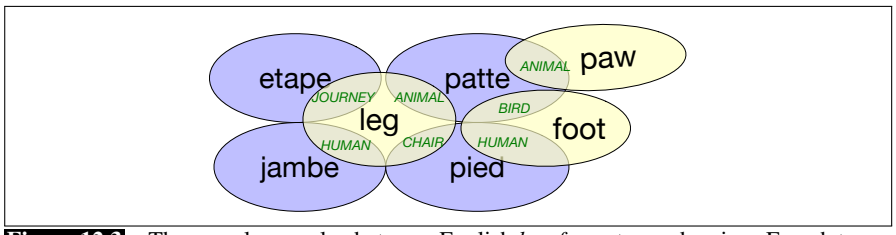


Figure 12.3 The complex overlap between English *leg*, *foot*, etc., and various French translations as discussed by Hutchins and Somers (1992).

Finally, languages differ systematically in how the conceptual properties of an event are mapped onto specific words. Talmy (1985, 1991) noted that languages can be characterized by whether direction of motion and manner of motion are marked on the verb or on the "satellites": particles, prepositional phrases, or adverbial phrases. For example, a bottle floating out of a cave would be described in English with the direction marked on the particle *out*, while in Spanish the direction would be marked on the verb:

(12.5) English: *The bottle floated out.*
Spanish: La botella salió flotando.
The bottle exited floating.

verb-framed

satellite-framed

Verb-framed languages mark the direction of motion on the verb (leaving the satellites to mark the manner of motion), like Spanish *acercarse* 'approach', *alcanzar* 'reach', *entrar* 'enter', *salir* 'exit'. **Satellite-framed** languages mark the direction of motion on the satellite (leaving the verb to mark the manner of motion), like English *crawl out*, *float off*, *jump down*, *run after*. Languages like Japanese, Tamil, and the many languages in the Romance, Semitic, and Mayan languages families, are verb-framed; Chinese as well as non-Romance Indo-European languages like English, Swedish, Russian, Hindi, and Farsi are satellite framed (Talmy 1991, Slobin 1996).

12.1.3 Morphological Typology

isolating

polysynthetic

agglutinative

fusion

Morphologically, languages are often characterized along two dimensions of variation. The first is the number of morphemes per word, ranging from **isolating** languages like Vietnamese and Cantonese, in which each word generally has one morpheme, to **polysynthetic** languages like Siberian Yupik ("Eskimo"), in which a single word may have very many morphemes, corresponding to a whole sentence in English. The second dimension is the degree to which morphemes are segmentable, ranging from **agglutinative** languages like Turkish, in which morphemes have relatively clean boundaries, to **fusion** languages like Russian, in which a single affix

may conflate multiple morphemes, like *-om* in the word *stolom* (table-SG-INSTR-DECL1), which fuses the distinct morphological categories instrumental, singular, and first declension.

Translating between languages with rich morphology requires dealing with structure below the word level, and for this reason modern systems generally use subword models like the wordpiece or BPE models of Section 12.2.1.

12.1.4 Referential density

Finally, languages vary along a typological dimension related to the things they tend to omit. Some languages, like English, require that we use an explicit pronoun when talking about a referent that is given in the discourse. In other languages, however, we can sometimes omit pronouns altogether, as the following example from Spanish shows[1]:

(12.6) [El jefe]$_i$ dio con un libro. $\emptyset_i$ Mostró su hallazgo a un descifrador ambulante.
[The boss] came upon a book. [He] showed his find to a wandering decoder.

pro-drop

Languages that can omit pronouns are called **pro-drop** languages. Even among the pro-drop languages, there are marked differences in frequencies of omission. Japanese and Chinese, for example, tend to omit far more than does Spanish. This dimension of variation across languages is called the dimension of **referential density**. We say that languages that tend to use more pronouns are more **referentially dense** than those that use more zeros. Referentially sparse languages, like Chinese or Japanese, that require the hearer to do more inferential work to recover antecedents are also called **cold** languages. Languages that are more explicit and make it easier for the hearer are called **hot** languages. The terms *hot* and *cold* are borrowed from Marshall McLuhan's 1964 distinction between hot media like movies, which fill in many details for the viewer, versus cold media like comics, which require the reader to do more inferential work to fill out the representation (Bickel, 2003).

referential density

cold language

hot language

Translating from languages with extensive pro-drop, like Chinese or Japanese, to non-pro-drop languages like English can be difficult since the model must somehow identify each zero and recover who or what is being talked about in order to insert the proper pronoun.

12.2 Machine Translation using Encoder-Decoder

The standard architecture for MT is the **encoder-decoder transformer** or **sequence-to-sequence** model, an architecture we briefly prefigured in Chapter 7. We'll see the details of how to apply this architecture to transformers in Section 12.3, but first let's talk about the overall task.

Most machine translation tasks make the simplification that we can translate each sentence independently, so we'll just consider individual sentences for now. Given a sentence in a **source** language, the MT task is then to generate a corresponding sentence in a **target** language. For example, an MT system is given an English sentence like

The green witch arrived

and must translate it into the Spanish sentence:

[1] Here we use the $\emptyset$-notation; we'll introduce this and discuss this issue further in Chapter 23

Llegó la bruja verde

MT uses supervised machine learning: at training time the system is given a large set of **parallel** sentences (each sentence in a source language matched with a sentence in the target language), and learns to map source sentences into target sentences. In practice, rather than using words (as in the example above), we split the sentences into a sequence of subword tokens (tokens can be words, or subwords, or individual characters). The systems are then trained to maximize the probability of the sequence of tokens in the target language $y_1, ..., y_m$ given the sequence of tokens in the source language $x_1, ..., x_n$:

$$P(y_1, \ldots, y_m | x_1, \ldots, x_n) \tag{12.7}$$

Rather than use the input tokens directly, the encoder-decoder architecture consists of two components, an **encoder** and a **decoder**. The encoder takes the input words $x = [x_1, \ldots, x_n]$ and produces an intermediate context $\mathbf{h}$. At decoding time, the system takes h and, word by word, generates the output y:

$$\mathbf{h} = \text{encoder}(x) \tag{12.8}$$

$$y_{t+1} = \text{decoder}(\mathbf{h}, y_1, \ldots, y_t) \quad \forall t \in [1, \ldots, m] \tag{12.9}$$

In the next two sections we'll talk about subword tokenization, then how to get parallel corpora for training, and then we'll introduce the details of the encoder-decoder architecture.

12.2.1 Tokenization

Machine translation systems use a vocabulary that is fixed in advance, and rather than using space-separated words, this vocabulary is generated with subword tokenization algorithms, like the **BPE** algorithm sketched in Chapter 2. A shared vocabulary is used for the source and target languages, which makes it easy to copy tokens (like names) from source to target. Using subword tokenization with tokens shared between languages makes it natural to translate between languages like English or Hindi that use spaces to separate words, and languages like Chinese or Thai that don't.

We build the vocabulary by running a subword tokenization algorithm on a corpus that contains both source and target language data.

Rather than the simple BPE algorithm from Fig. 2.6, modern systems often use more powerful tokenization algorithms. Some systems (like BERT) use a variant of BPE called the **wordpiece** algorithm, which instead of choosing the most frequent set of tokens to merge, chooses merges based on which one most increases the language model probability of the tokenization. Wordpieces use a special symbol at the beginning of each token; here's a resulting tokenization from the Google MT system (Wu et al., 2016):

wordpiece

words: Jet makers feud over seat width with big orders at stake
wordpieces: _J et _makers _fe ud _over _seat _width _with _big _orders _at _stake

The wordpiece algorithm is given a training corpus and a desired vocabulary size V, and proceeds as follows:

1. Initialize the wordpiece lexicon with characters (for example a subset of Unicode characters, collapsing all the remaining characters to a special unknown character token).

2. Repeat until there are V wordpieces:
   (a) Train an n-gram language model on the training corpus, using the current set of wordpieces.
   (b) Consider the set of possible new wordpieces made by concatenating two wordpieces from the current lexicon. Choose the one new wordpiece that most increases the language model probability of the training corpus.

Recall that with BPE we had to specify the number of merges to perform; in wordpiece, by contrast, we specify the total vocabulary, which is a more intuitive parameter. A vocabulary of 8K to 32K word pieces is commonly used.

unigram

SentencePiece

An even more commonly used tokenization algorithm is (somewhat ambiguously) called the **unigram** algorithm (Kudo, 2018) or sometimes the **SentencePiece** algorithm, and is used in systems like ALBERT (Lan et al., 2020) and T5 (Raffel et al., 2020). (Because unigram is the default tokenization algorithm used in a library called SentencePiece that adds a useful wrapper around tokenization algorithms (Kudo and Richardson, 2018b), authors often say they are using SentencePiece tokenization but really mean they are using the **unigram** algorithm).

In unigram tokenization, instead of building up a vocabulary by merging tokens, we start with a huge vocabulary of every individual unicode character plus all frequent sequences of characters (including all space-separated words, for languages with spaces), and iteratively remove some tokens to get to a desired final vocabulary size. The algorithm is complex (involving suffix-trees for efficiently storing many tokens, and the EM algorithm for iteratively assigning probabilities to tokens), so we don't give it here, but see Kudo (2018) and Kudo and Richardson (2018b). Roughly speaking the algorithm proceeds iteratively by estimating the probability of each token, tokenizing the input data using various tokenizations, then removing a percentage of tokens that don't occur in high-probability tokenization, and then iterates until the vocabulary has been reduced down to the desired number of tokens.

Why does unigram tokenization work better than BPE? BPE tends to create lots of very small non-meaningful tokens (because BPE can only create larger words or morphemes by merging characters one at a time), and it also tends to merge very common tokens, like the suffix *ed*, onto their neighbors. We can see from these examples from Bostrom and Durrett (2020) that unigram tends to produce tokens that are more semantically meaningful:

Original:	`corrupted`	Original:	`Completely preposterous suggestions`
BPE:	`cor rupted`	BPE:	`Comple t ely prep ost erous suggest ions`
Unigram:	`corrupt ed`	Unigram:	`Complete ly pre post er ous suggestion s`

12.2.2 Creating the Training data

parallel corpus

Europarl

Machine translation models are trained on a **parallel corpus**, sometimes called a **bitext**, a text that appears in two (or more) languages. Large numbers of parallel corpora are available. Some are governmental; the **Europarl** corpus (Koehn, 2005), extracted from the proceedings of the European Parliament, contains between 400,000 and 2 million sentences each from 21 European languages. The United Nations Parallel Corpus contains on the order of 10 million sentences in the six official languages of the United Nations (Arabic, Chinese, English, French, Russian, Spanish) Ziemski et al. (2016). Other parallel corpora have been made from movie and TV subtitles, like the **OpenSubtitles** corpus (Lison and Tiedemann, 2016), or from general web text, like the **ParaCrawl** corpus of 223 million sentence pairs between 23 EU languages and English extracted from the CommonCrawl Bañón et al. (2020).

Sentence alignment

Standard training corpora for MT come as aligned pairs of sentences. When creating new corpora, for example for underresourced languages or new domains, these sentence alignments must be created. Fig. 12.4 gives a sample hypothetical sentence alignment.

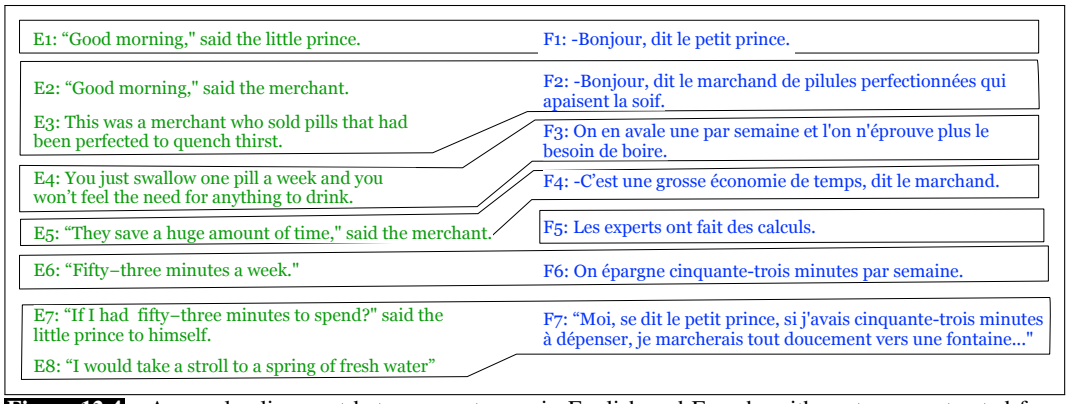


Figure 12.4 A sample alignment between sentences in English and French, with sentences extracted from Antoine de Saint-Exupery's *Le Petit Prince* and a hypothetical translation. Sentence alignment takes sentences $e_1,...,e_n$, and $f_1,...,f_m$ and finds minimal sets of sentences that are translations of each other, including single sentence mappings like (e_1,f_1), (e_4,f_3), (e_5,f_4), (e_6,f_6) as well as 2-1 alignments (e_2/e_3,f_2), (e_7/e_8,f_7), and null alignments (f_5).

Given two documents that are translations of each other, we generally need two steps to produce sentence alignments:

- a cost function that takes a span of source sentences and a span of target sentences and returns a score measuring how likely these spans are to be translations.
- an alignment algorithm that takes these scores to find a good alignment between the documents.

To score the similarity of sentences across languages, we need to make use of a **multilingual embedding space**, in which sentences from different languages are in the same embedding space (Artetxe and Schwenk, 2019). Given such a space, cosine similarity of such embeddings provides a natural scoring function (Schwenk, 2018). Thompson and Koehn (2019) give the following cost function between two sentences or spans x,y from the source and target documents respectively:

$$c(x,y) = \frac{(1-\cos(x,y))\text{nSents}(x)\ \text{nSents}(y)}{\sum_{s=1}^{S} 1-\cos(x,y_s) + \sum_{s=1}^{S} 1-\cos(x_s,y)} \tag{12.10}$$

where nSents() gives the number of sentences (this biases the metric toward many alignments of single sentences instead of aligning very large spans). The denominator helps to normalize the similarities, and so $x_1,...,x_S,y_1,...,y_S$, are randomly selected sentences sampled from the respective documents.

Usually dynamic programming is used as the alignment algorithm (Gale and Church, 1993), in a simple extension of the minimum edit distance algorithm we introduced in Chapter 2.

Finally, it's helpful to do some corpus cleanup by removing noisy sentence pairs. This can involve handwritten rules to remove low-precision pairs (for example removing sentences that are too long, too short, have different URLs, or even pairs

that are too similar, suggesting that they were copies rather than translations). Or pairs can be ranked by their multilingual embedding cosine score and low-scoring pairs discarded.

12.3 Details of the Encoder-Decoder Model

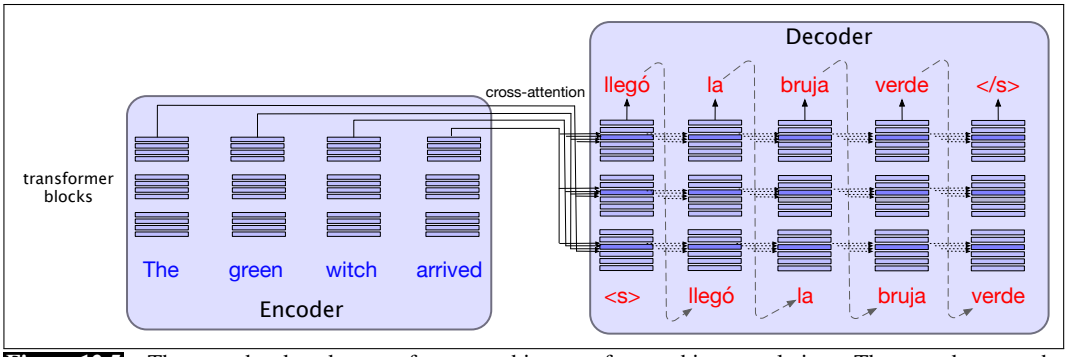


Figure 12.5 The encoder-decoder transformer architecture for machine translation. The encoder uses the transformer blocks we saw in Chapter 8, while the decoder uses a more powerful block with an extra **cross-attention** layer that can attend to all the encoder words. We'll see this in more detail in the next section.

The standard architecture for MT is the encoder-decoder transformer. Fig. 12.5 shows the intuition of the architecture at a high level. You'll see that the encoder-decoder architecture is made up of two transformers: an **encoder**, which is the same as the basic transformers from Chapter 8, and a **decoder**, which is augmented with a special new layer called the **cross-attention** layer. The encoder takes the source language input word tokens $\mathbf{X} = \mathbf{x}_1, ..., \mathbf{x}_n$ and maps them to an output representation $\mathbf{H}^{enc} = \mathbf{h}_1, ..., \mathbf{h}_n$; via a stack of encoder blocks.

The decoder is essentially a conditional language model that attends to the encoder representation and generates the target words one by one, at each timestep conditioning on the source sentence and the previously generated target language words to generate a token. Decoding can use any of the decoding methods discussed in Chapter 8 like greedy, or temperature or nucleus sampling. But the most common decoding algorithm for MT is the beam search algorithm that we'll introduce in Section 12.4.

But the components of the architecture differ somewhat from the transformer block we've seen. First, in order to attend to the source language, the transformer blocks in the decoder have an extra **cross-attention** layer. Recall that the transformer block of Chapter 8 consists of a self-attention layer that attends to the input from the previous layer, preceded by layer norm, and followed by another layer norm and the feed forward layer. The decoder transformer block includes an extra layer with a special kind of attention, **cross-attention** (also sometimes called **encoder-decoder attention** or **source attention**). Cross-attention has the same form as the multi-head attention in a normal transformer block, except that while the queries as usual come from the previous layer of the decoder, the keys and values come from the output of the *encoder*.

cross-attention

That is, where in standard multi-head attention the input to each attention layer is $\mathbf{X}$, in cross attention the input is the final output of the encoder $\mathbf{H}^{enc} = \mathbf{h}_1, ..., \mathbf{h}_n$. $\mathbf{H}^{enc}$ is of shape $[n \times d]$, each row representing one input token. To link the keys

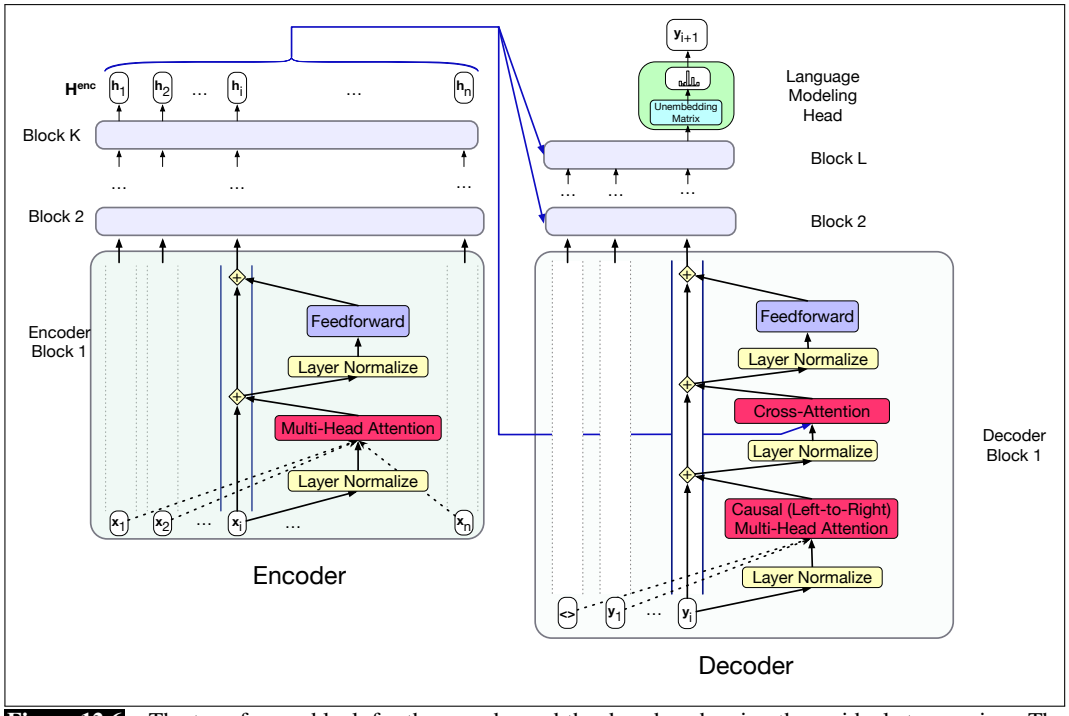


Figure 12.6 The transformer block for the encoder and the decoder, showing the residual stream view. The final output of the encoder $\mathbf{H}^{enc} = \mathbf{h}_1, ..., \mathbf{h}_n$ is the context used in the decoder. The decoder is a standard transformer except with one extra layer, the **cross-attention** layer, which takes that encoder output $\mathbf{H}^{enc}$ and uses it to form its $\mathbf{K}$ and $\mathbf{V}$ inputs.

and values from the encoder with the query from the prior layer of the decoder, we multiply the encoder output $\mathbf{H}^{enc}$ by the cross-attention layer's key weights $\mathbf{W}^{\mathbf{K}}$ and value weights $\mathbf{W}^{\mathbf{V}}$. The query comes from the output from the prior decoder layer $\mathbf{H}^{dec[\ell-1]}$, which is multiplied by the cross-attention layer's query weights $\mathbf{W}^{\mathbf{Q}}$:

$$\mathbf{Q} = \mathbf{H}^{dec[\ell-1]}\mathbf{W}^{\mathbf{Q}}; \;\; \mathbf{K} = \mathbf{H}^{enc}\mathbf{W}^{\mathbf{K}}; \;\; \mathbf{V} = \mathbf{H}^{enc}\mathbf{W}^{\mathbf{V}} \tag{12.11}$$

$$\text{CrossAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \;=\; \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\mathbf{V} \tag{12.12}$$

The cross attention thus allows the decoder to attend to each of the source language words as projected into the entire encoder final output representations. The other attention layer in each decoder block, the multi-head attention layer, is the same causal (left-to-right) attention that we saw in Chapter 8. The multi-head attention in the encoder, however, is allowed to look ahead at the entire source language text, so it is not masked.

To train an encoder-decoder model, we give the network the source text, and then starting with the separator token train it autoregressively to predict the next token, using cross-entropy loss. Recall that cross-entropy loss for language modeling is determined by the probability the model assigns to the correct next word. So at time t the CE loss is the negative log probability the model assigns to the next word in the training sequence:

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) \;=\; -\log \hat{\mathbf{y}}_t[w_{t+1}] \tag{12.13}$$

teacher forcing

As in that case, we use **teacher forcing** in the decoder. Recall that in teacher forcing, at each time step in decoding we force the system to use the gold target token from training as the next input x_{t+1}, rather than allowing it to rely on the (possibly erroneous) decoder output $\hat{y}_t$.

12.4 Decoding in MT: Beam Search

Recall the **greedy decoding** algorithm from Chapter 8: at each time step t in generation, the output y_t is chosen by computing the probability for each word in the vocabulary and then choosing the highest probability word (the argmax):

$$\hat{w}_t = \text{argmax}_{w \in V} P(w|\mathbf{w}_{<t}) \tag{12.14}$$

A problem with greedy decoding is that what looks high probability at word t might turn out to have been the wrong choice once we get to word $t+1$. The **beam search** algorithm maintains multiple choices until later when we can see which one is best.

search tree

In beam search we model decoding as searching the space of possible generations, represented as a **search tree** whose **branches** represent actions (generating a token), and **nodes** represent states (having generated a particular prefix). We search for the best action sequence, i.e., the string with the highest probability.

An illustration of the problem

Fig. 12.7 shows a made-up example. The most probable sequence is *ok ok EOS* (its probability is .4× .7× 1.0). But greedy search doesn't find it, incorrectly choosing *yes* as the first word since it has the highest local probability (0.5).

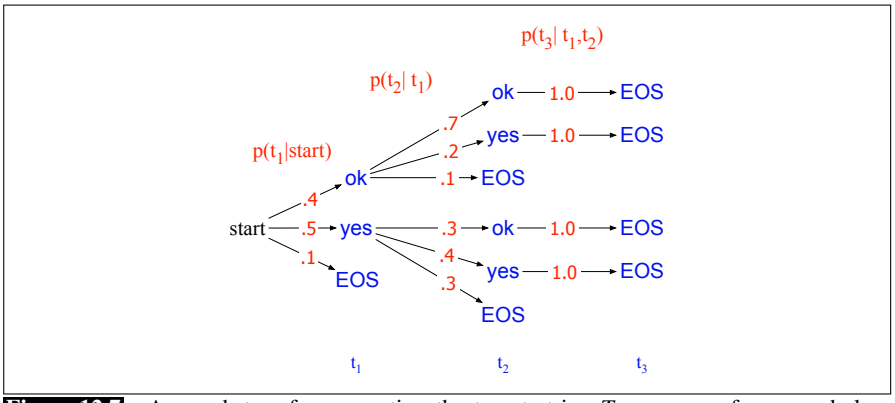


Figure 12.7 A search tree for generating the target string $T = t_1, t_2, \ldots$ from vocabulary $V = \{\text{yes}, \text{ok}, \texttt{<s>}\}$, showing the probability of generating each token from that state. Greedy search chooses *yes* followed by *yes*, instead of the globally most probable sequence *ok ok*.

For some problems, like part-of-speech tagging or parsing as we will see in Chapter 17 or Chapter 18, we can use dynamic programming search (the Viterbi algorithm) to address this problem. Unfortunately, dynamic programming is not applicable to generation problems with long-distance dependencies between the output decisions. The only method guaranteed to find the best solution is exhaustive search: computing the probability of every one of the V^T possible sentences (for some length value T) which is obviously too slow.

The solution: beam search

beam search

Instead, MT systems generally decode using **beam search**, a heuristic search method first proposed by Lowerre (1976). In beam search, instead of choosing the best token to generate at each timestep, we keep k possible tokens at each step. This fixed-size memory footprint k is called the **beam width**, on the metaphor of a flashlight beam that can be parameterized to be wider or narrower.

beam width

Thus at the first step of decoding, we compute a softmax over the entire vocabulary, assigning a probability to each word. We then select the k-best options from this softmax output. These initial k outputs are the search frontier and these k initial words are called **hypotheses**. A hypothesis is an output sequence, a translation-so-far, together with its probability.

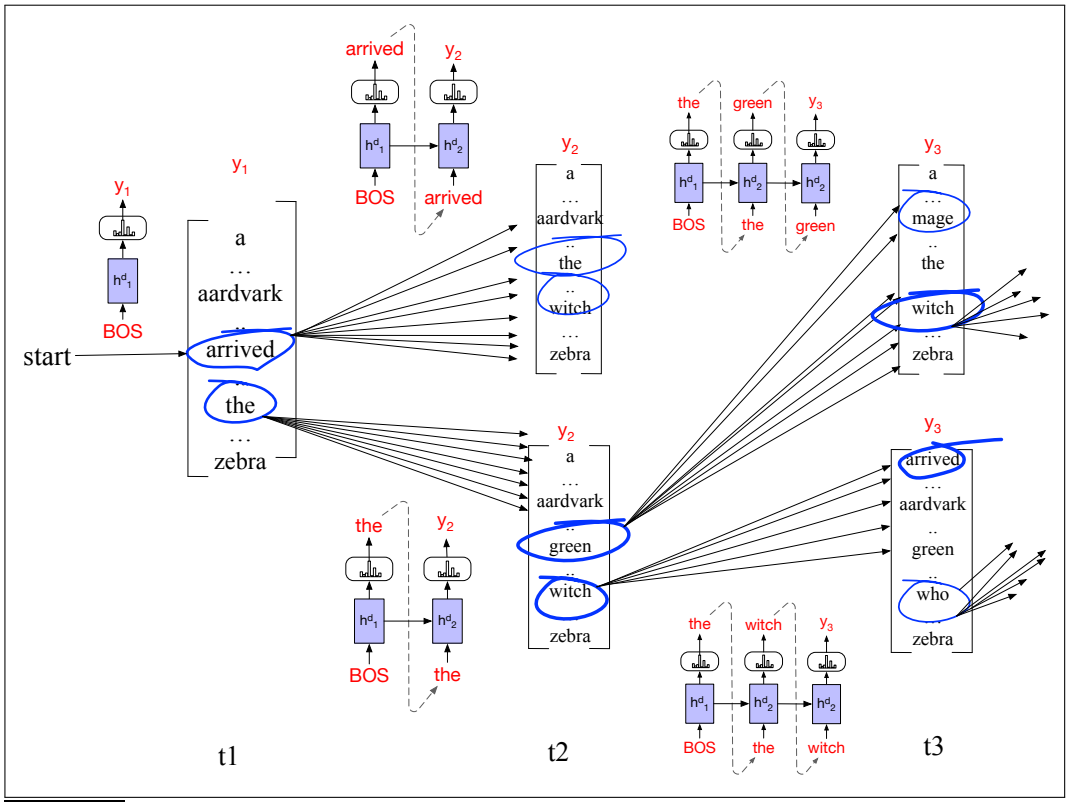


Figure 12.8 Beam search decoding with a beam width of $k = 2$. At each time step, we choose the k best hypotheses, form the V possible extensions of each, score those $k \times V$ hypotheses and choose the best $k = 2$ to continue. At time 1, the frontier has the best 2 options from the initial decoder state: *arrived* and *the*. We extend each, compute the probability of all the hypotheses so far (*arrived the*, *arrived aardvark*, *the green*, *the witch*) and again chose the best 2 (*the green* and *the witch*) to be the search frontier. The images on the arcs schematically represent the decoders that must be run at each step to score the next words (for simplicity not depicting cross-attention).

At subsequent steps, each of the k best hypotheses is extended incrementally by being passed to distinct decoders, which each generate a softmax over the entire vocabulary to extend the hypothesis to every possible next token. Each of these $k \times V$ hypotheses is scored by $P(y_i|x, y_{<i})$: the product of the probability of the current word choice multiplied by the probability of the path that led to it. We then prune the $k \times V$ hypotheses down to the k best hypotheses, so there are never more than k

hypotheses at the frontier of the search, and never more than k decoders. Fig. 12.8 illustrates this with a beam width of 2 for the beginning of *The green witch arrived*.

This process continues until an EOS is generated indicating that a complete candidate output has been found. At this point, the completed hypothesis is removed from the frontier and the size of the beam is reduced by one. The search continues until the beam has been reduced to 0. The result will be k hypotheses.

To score each node by its log probability, we use the chain rule of probability to break down $p(y|x)$ into the product of the probability of each word given its prior context, which we can turn into a sum of logs (for an output string of length t):

$$\begin{aligned} score(y) &= \log P(y|x) \\ &= \log\left(P(y_1|x)P(y_2|y_1,x)P(y_3|y_1,y_2,x)...P(y_t|y_1,...,y_{t-1},x)\right) \\ &= \sum_{i=1}^{t} \log P(y_i|y_1,...,y_{i-1},x) \end{aligned} \tag{12.15}$$

Thus at each step, to compute the probability of a partial sentence, we simply add the log probability of the prefix sentence so far to the log probability of generating the next token. Fig. 12.9 shows the scoring for the example sentence shown in Fig. 12.8, using some simple made-up probabilities. Log probabilities are negative or 0, and the max of two log probabilities is the one that is greater (closer to 0).

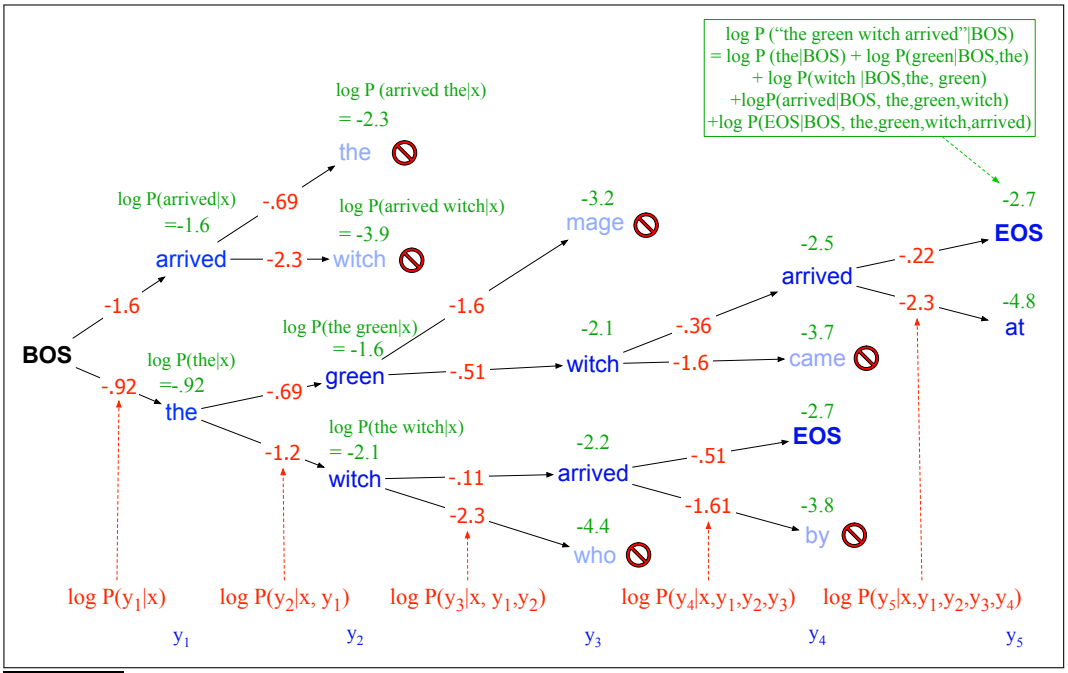


Figure 12.9 Scoring for beam search decoding with a beam width of $k = 2$. We maintain the log probability of each hypothesis in the beam by incrementally adding the logprob of generating each next token. Only the top k paths are extended to the next step.

Fig. 12.10 gives the algorithm. One problem with this version of the algorithm is that the completed hypotheses may have different lengths. Because language models generally assign lower probabilities to longer strings, a naive algorithm would choose shorter strings for y. (This is not an issue during the earlier steps of decoding; since beam search is breadth-first, all the hypotheses being compared had the

```
function BEAMDECODE(c, beam_width) returns best paths

  y_0, h_0 ← 0
  path ← ()
  complete_paths ← ()
  state ← (c, y_0, h_0, path)          ;initial state
  frontier ← ⟨state⟩                   ;initial frontier

  while frontier contains incomplete paths and beamwidth > 0
    extended_frontier ← ⟨⟩
    for each state ∈ frontier do
      y ← DECODE(state)
      for each word i ∈ Vocabulary do
        successor ← NEWSTATE(state, i, y_i)
        extended_frontier ← ADDTOBEAM(successor, extended_frontier,
                                      beam_width)

    for each state in extended_frontier do
      if state is complete do
        complete_paths ← APPEND(complete_paths, state)
        extended_frontier ← REMOVE(extended_frontier, state)
        beam_width ← beam_width - 1
    frontier ← extended_frontier

  return completed_paths

function NEWSTATE(state, word, word_prob) returns new state

function ADDTOBEAM(state, frontier, width) returns updated frontier

  if LENGTH(frontier) < width then
    frontier ← INSERT(state, frontier)
  else if SCORE(state) > SCORE(WORSTOF(frontier))
    frontier ← REMOVE(WORSTOF(frontier))
    frontier ← INSERT(state, frontier)
  return frontier
```

Figure 12.10 Beam search decoding.

same length.) For this reason we often apply length normalization methods, like dividing the logprob by the number of words:

$$score(y) = \frac{1}{t}\log P(y|x) \;=\; \frac{1}{t}\sum_{i=1}^{t}\log P(y_i|y_1,...,y_{i-1},x) \tag{12.16}$$

For MT we generally use beam widths k between 5 and 10, giving us k hypotheses at the end. We can pass all k to the downstream application with their respective scores, or if we just need a single translation we can pass the most probable hypothesis.

12.4.1 Minimum Bayes Risk Decoding

minimum Bayes risk
MBR

Minimum Bayes risk or **MBR** decoding is an alternative decoding algorithm that can work even better than beam search and also tends to be better than the other decoding algorithms like temperature sampling introduced in Section 7.4.

The intuition of minimum Bayes risk is that instead of trying to choose the translation which is most probable, we choose the one that is likely to have the least error. For example, we might want our decoding algorithm to find the translation which has the highest score on some evaluation metric. For example in Section 12.6 we will introduce metrics like chrF or BERTScore that measure the goodness-of-fit between a candidate translation and a set of reference human translations. A translation that maximizes this score, especially with a hypothetically huge set of perfect human translations is likely to be a good one (have minimum risk) even if it is not the most probable translation by our particular probability estimator.

In practice, we don't know the perfect set of translations for a given sentence. So the standard simplification used in MBR decoding algorithms is to instead choose the candidate translation which is most similar (by some measure of goodness-of-fit) with some set of candidate translations. We're essentially approximating the enormous space of all possible translations $\mathcal{U}$ with a smaller set of possible candidate translations $\mathcal{Y}$.

Given this set of possible candidate translations $\mathcal{Y}$, and some similarity or alignment function util, we choose the best translation $\hat{y}$ as the translation which is most similar to all the other candidate translations:

$$\hat{y} = \underset{y \in \mathcal{Y}}{\text{argmax}} \sum_{c \in \mathcal{Y}} \text{util}(y, c) \tag{12.17}$$

Various util functions can be used, like chrF or BERTscore or BLEU. We can get the set of candidate translations by sampling using one of the basic sampling algorithms of Section 7.4 like temperature sampling; good results can be obtained with as few as 32 or 64 candidates.

Minimum Bayes risk decoding can also be used for other NLP tasks; indeed it was widely applied to speech recognition (Stolcke et al., 1997; Goel and Byrne, 2000) before being applied to machine translation (Kumar and Byrne, 2004), and has been shown to work well across many other generation tasks as well (e.g., summarization, dialogue, and image captioning (Suzgun et al., 2023a)).

12.5 Translating in low-resource situations

For some languages, and especially for English, online resources are widely available. There are many large parallel corpora that contain translations between English and many languages. But the vast majority of the world's languages do not have large parallel training texts available. An important ongoing research question is how to get good translation with lesser resourced languages. The resource problem can even be true for high resource languages when we need to translate into low resource domains (for example in a particular genre that happens to have very little bitext).

Here we briefly introduce two commonly used approaches for dealing with this data sparsity: **backtranslation**, which is a special case of the general statistical technique called **data augmentation**, and **multilingual models**, and also discuss some socio-technical issues.

12.5.1 Data Augmentation

Data augmentation is a statistical technique for dealing with insufficient training data, by adding new synthetic data that is generated from the current natural data.

backtranslation

The most common data augmentation technique for machine translation is called **backtranslation**. Backtranslation relies on the intuition that while parallel corpora may be limited for particular languages or domains, we can often find a large (or at least larger) monolingual corpus, to add to the smaller parallel corpora that are available. The algorithm makes use of monolingual corpora in the **target** language by creating synthetic bitexts.

In backtranslation, our goal is to improve source-to-target MT, given a small parallel text (a bitext) in the source/target languages, and some monolingual data in the target language. We first use the bitext to train a MT system in the **reverse** direction: a target-to-source MT system. We then use it to translate the monolingual target data to the source language. Now we can add this synthetic bitext (natural target sentences, aligned with MT-produced source sentences) to our training data, and retrain our source-to-target MT model. For example suppose we want to translate from Navajo to English but only have a small Navajo-English bitext, although of course we can find lots of monolingual English data. We use the small bitext to build an MT engine going the other way (from English to Navajo). Once we translate the monolingual English text to Navajo, we can add this synthetic Navajo/English bitext to our training data.

Backtranslation has various parameters. One is how we generate the backtranslated data; we can run the decoder in greedy inference, or use beam search. Or we can do sampling, like the temperature sampling algorithm we saw in Chapter 8. Another parameter is the ratio of backtranslated data to natural bitext data; we can choose to upsample the bitext data (include multiple copies of each sentence). In general backtranslation works surprisingly well; one estimate suggests that a system trained on backtranslated text gets about 2/3 of the gain as would training on the same amount of natural bitext (Edunov et al., 2018).

12.5.2 Multilingual models

The models we've described so far are for bilingual translation: one source language, one target language. It's also possible to build a **multilingual** translator.

In a multilingual translator, we train the system by giving it parallel sentences in many different pairs of languages. That means we need to tell the system which language to translate from and to! We tell the system which language is which by adding a special token l_s to the encoder specifying the source language we're translating from, and a special token l_t to the decoder telling it the target language we'd like to translate into.

Thus we slightly update Eq. 12.9 above to add these tokens in Eq. 12.19:

$$\mathbf{h} = \text{encoder}(x, l_s) \tag{12.18}$$

$$y_{i+1} = \text{decoder}(\mathbf{h}, l_t, y_1, \ldots, y_i) \quad \forall i \in [1, \ldots, m] \tag{12.19}$$

One advantage of a multilingual model is that they can improve the translation of lower-resourced languages by drawing on information from a similar language in the training data that happens to have more resources. Perhaps we don't know the meaning of a word in Galician, but the word appears in the similar and higher-resourced language Spanish.

12.5.3 Sociotechnical issues

Many issues in dealing with low-resource languages go beyond the purely technical. One problem is that for low-resource languages, especially from low-income countries, native speakers are often not involved as the curators for content selection, as the language technologists, or as the evaluators who measure performance (Nekoto et al., 2020). Indeed, one well-known study that manually audited a large set of parallel corpora and other major multilingual datasets found that for many of the corpora, less than 50% of the sentences were of acceptable quality, with a lot of data consisting of repeated sentences with web boilerplate or incorrect translations, suggesting that native speakers may not have been sufficiently involved in the data process (Kreutzer et al., 2022).

Other issues, like the tendency of many MT approaches to focus on the case where one of the languages is English (Anastasopoulos and Neubig, 2020), have to do with allocation of resources. Where most large multilingual systems were trained on bitexts in which English was one of the two languages, recent huge corporate systems like those of Fan et al. (2021) and Costa-jussà et al. (2022) and datasets like Schwenk et al. (2021) attempt to handle large numbers of languages (up to 200 languages) and create bitexts between many more pairs of languages and not just through English.

At the smaller end, Nekoto et al. (2020) propose a participatory design process to encourage content creators, curators, and language technologists who speak these low-resourced languages to participate in developing MT algorithms. They provide online groups, mentoring, and infrastructure, and report on a case study on developing MT algorithms for low-resource African languages. Among their conclusions was to perform MT evaluation by post-editing rather than direct evaluation, since having labelers edit an MT system and then measure the distance between the MT output and its post-edited version both was simpler to train evaluators and makes it easier to measure true errors in the MT output and not differences due to linguistic variation (Bentivogli et al., 2018).

12.6 MT Evaluation

Translations are evaluated along two dimensions:

adequacy
1. **adequacy:** how well the translation captures the exact meaning of the source sentence. Sometimes called **faithfulness** or **fidelity**.

fluency
2. **fluency:** how fluent the translation is in the target language (is it grammatical, clear, readable, natural).

Using humans to evaluate is most accurate, but automatic metrics are also used for convenience.

12.6.1 Using Human Raters to Evaluate MT

The most accurate evaluations use human raters, such as online crowdworkers, to evaluate each translation along the two dimensions. For example, along the dimension of **fluency**, we can ask how intelligible, how clear, how readable, or how natural the MT output (the target text) is. We can give the raters a scale, for example, from 1 (totally unintelligible) to 5 (totally intelligible), or 1 to 100, and ask them to rate each sentence or paragraph of the MT output.

We can do the same thing to judge the second dimension, **adequacy**, using raters to assign scores on a scale. If we have bilingual raters, we can give them the source sentence and a proposed target sentence, and rate, on a 5-point or 100-point scale, how much of the information in the source was preserved in the target. If we only have monolingual raters but we have a good human translation of the source text, we can give the monolingual raters the human reference translation and a target machine translation and again rate how much information is preserved. An alternative is to do **ranking**: give the raters a pair of candidate translations, and ask them which one they prefer.

ranking

Training of human raters (who are often online crowdworkers) is essential; raters without translation expertise find it difficult to separate fluency and adequacy, and so training includes examples carefully distinguishing these. Raters often disagree (source sentences may be ambiguous, raters will have different world knowledge, raters may apply scales differently). It is therefore common to remove outlier raters, and (if we use a fine-grained enough scale) normalizing raters by subtracting the mean from their scores and dividing by the variance.

As discussed above, an alternative way of using human raters is to have them **post-edit** translations, taking the MT output and changing it minimally until they feel it represents a correct translation. The difference between their post-edited translations and the original MT output can then be used as a measure of quality.

12.6.2 Automatic Evaluation

While humans produce the best evaluations of machine translation output, running a human evaluation can be time consuming and expensive. For this reason automatic metrics are often used as temporary proxies. Automatic metrics are less accurate than human evaluation, but can help test potential system improvements, and even be used as an automatic loss function for training. In this section we introduce two families of such metrics, those based on character- or word-overlap and those based on embedding similarity.

Automatic Evaluation by Character Overlap: chrF

chrF

The simplest and most robust metric for MT evaluation is called **chrF**, which stands for **character F-score** (Popović, 2015). chrF (along with many other earlier related metrics like BLEU, METEOR, TER, and others) is based on a simple intuition derived from the pioneering work of Miller and Beebe-Center (1956): a good machine translation will tend to contain characters and words that occur in a human translation of the same sentence. Consider a test set from a parallel corpus, in which each source sentence has both a gold human target translation and a candidate MT translation we'd like to evaluate. The chrF metric ranks each MT target sentence by a function of the number of character n-gram overlaps with the human translation.

Given the hypothesis and the reference, chrF is given a parameter k indicating the length of character n-grams to be considered, and computes the average of the k precisions (unigram precision, bigram, and so on) and the average of the k recalls (unigram recall, bigram recall, etc.):

chrP percentage of character 1-grams, 2-grams, ..., k-grams in the hypothesis that occur in the reference, averaged.

chrR percentage of character 1-grams, 2-grams,..., k-grams in the reference that occur in the hypothesis, averaged.

The metric then computes an F-score by combining chrP and chrR using a weighting

parameter β. It is common to set $\beta = 2$, thus weighing recall twice as much as precision:

$$\text{chrF}\beta = (1+\beta^2)\frac{\text{chrP}\cdot\text{chrR}}{\beta^2\cdot\text{chrP}+\text{chrR}} \tag{12.20}$$

For $\beta = 2$, that would be:

$$\text{chrF2} = \frac{5\cdot\text{chrP}\cdot\text{chrR}}{4\cdot\text{chrP}+\text{chrR}}$$

For example, consider two hypotheses that we'd like to score against the reference translation *witness for the past*. Here are the hypotheses along with chrF values computed using parameters $k = \beta = 2$ (in real examples, k would be a higher number like 6):

REF: `witness for the past,`	
HYP1: `witness of the past,`	chrF2,2 = .86
HYP2: `past witness`	chrF2,2 = .62

Let's see how we computed that chrF value for HYP1 (we'll leave the computation of the chrF value for HYP2 as an exercise for the reader). First, chrF ignores spaces, so we'll remove them from both the reference and hypothesis:

REF: `witnessforthepast,` (18 unigrams, 17 bigrams)
HYP1: `witnessofthepast,` (17 unigrams, 16 bigrams)

Next let's see how many unigrams and bigrams match between the reference and hypothesis:

unigrams that match: `w i t n e s s f o t h e p a s t ,` (17 unigrams)
bigrams that match: `wi it tn ne es ss th he ep pa as st t,` (13 bigrams)

We use that to compute the unigram and bigram precisions and recalls:

unigram P:	17/17 = 1	unigram R:	17/18 = .944
bigram P:	13/16 = .813	bigram R:	13/17 = .765

Finally we average to get chrP and chrR, and compute the F-score:

$$\begin{aligned}\text{chrP} &= (17/17+13/16)/2 = .906\\ \text{chrR} &= (17/18+13/17)/2 = .855\\ \text{chrF2,2} &= 5\frac{\text{chrP}*\text{chrR}}{4\text{chrP}+\text{chrR}} = .86\end{aligned}$$

chrF is simple, robust, and correlates very well with human judgments in many languages (Kocmi et al., 2021).

Alternative overlap metric: BLEU

There are various alternative overlap metrics. For example, before the development of chrF, it was common to use a word-based overlap metric called **BLEU** (for BiLingual Evaluation Understudy), that is purely precision-based rather than combining precision and recall (Papineni et al., 2002). The BLEU score for a corpus of candidate translation sentences is a function of the **n-gram word precision** over all the sentences combined with a brevity penalty computed over the corpus as a whole.

What do we mean by n-gram precision? Consider a corpus composed of a single sentence. The unigram precision for this corpus is the percentage of unigram tokens

in the candidate translation that also occur in the reference translation, and ditto for bigrams and so on, up to 4-grams. BLEU extends this unigram metric to the whole corpus by computing the numerator as the sum over all sentences of the counts of all the unigram types that also occur in the reference translation, and the denominator is the total of the counts of all unigrams in all candidate sentences. We compute this n-gram precision for unigrams, bigrams, trigrams, and 4-grams and take the geometric mean. BLEU has many further complications, including a brevity penalty for penalizing candidate translations that are too short, and it also requires the n-gram counts be clipped in a particular way.

Because BLEU is a word-based metric, it is very sensitive to word tokenization, making it impossible to compare different systems if they rely on different tokenization standards, and doesn't work as well in languages with complex morphology. Nonetheless, you will sometimes still see systems evaluated by BLEU, particularly for translation into English. In such cases it's important to use packages that enforce standardization for tokenization like SACREBLEU (Post, 2018).

Statistical Significance Testing for MT evals

Character or word overlap-based metrics like chrF (or BLEU, or etc.) are mainly used to compare two systems, with the goal of answering questions like: did the new algorithm we just invented improve our MT system? To know if the difference between the chrF scores of two MT systems is a significant difference, we use the paired bootstrap test, or the similar randomization test.

To get a confidence interval on a single chrF score using the bootstrap test, recall from Section 4.11 that we take our test set (or devset) and create thousands of pseudo-testsets by repeatedly sampling with replacement from the original test set. We now compute the chrF score of each of the pseudo-testsets. If we drop the top 2.5% and bottom 2.5% of the scores, the remaining scores will give us the 95% confidence interval for the chrF score of our system.

To compare two MT systems A and B, we draw the same set of pseudo-testsets, and compute the chrF scores for each of them. We then compute the percentage of pseudo-test-sets in which A has a higher chrF score than B.

chrF: Limitations

While automatic character and word-overlap metrics like chrF or BLEU are useful, they have important limitations. chrF is very local: a large phrase that is moved around might barely change the chrF score at all, and chrF can't evaluate cross-sentence properties of a document like its discourse coherence (Chapter 24). chrF and similar automatic metrics also do poorly at comparing very different kinds of systems, such as comparing human-aided translation against machine translation, or different machine translation architectures against each other (Callison-Burch et al., 2006). Instead, automatic overlap metrics like chrF are most appropriate when evaluating changes to a single system.

12.6.3 Automatic Evaluation: Embedding-Based Methods

The chrF metric is based on measuring the exact character n-grams a human reference and candidate machine translation have in common. However, this criterion is overly strict, since a good translation may use alternate words or paraphrases. A solution first pioneered in early metrics like METEOR (Banerjee and Lavie, 2005) was to allow synonyms to match between the reference x and candidate $\tilde{x}$. More

recent metrics use BERT or other embeddings to implement this intuition.

For example, in some situations we might have datasets that have human assessments of translation quality. Such datasets consists of tuples $(x, \tilde{x}, r)$, where $x = (x_1, \ldots, x_n)$ is a reference translation, $\tilde{x} = (\tilde{x}_1, \ldots, \tilde{x}_m)$ is a candidate machine translation, and $r \in \mathbb{R}$ is a human rating that expresses the quality of $\tilde{x}$ with respect to x. Given such data, algorithms like COMET (Rei et al., 2020) BLEURT (Sellam et al., 2020) train a predictor on the human-labeled datasets, for example by passing x and $\tilde{x}$ through a version of BERT (trained with extra pretraining, and then finetuned on the human-labeled sentences), followed by a linear layer that is trained to predict r. The output of such models correlates highly with human labels.

In other cases, however, we don't have such human-labeled datasets. In that case we can measure the similarity of x and $\tilde{x}$ by the similarity of their embeddings. The BERTSCORE algorithm (Zhang et al., 2020) shown in Fig. 12.11, for example, passes the reference x and the candidate $\tilde{x}$ through BERT, computing a BERT embedding for each token x_i and $\tilde{x}_j$. Each pair of tokens $(x_i, \tilde{x}_j)$ is scored by its cosine $\frac{x_i \cdot \tilde{x}_j}{|x_i||\tilde{x}_j|}$. Each token in x is matched to a token in $\tilde{x}$ to compute recall, and each token in $\tilde{x}$ is matched to a token in x to compute precision (with each token greedily matched to the most similar token in the corresponding sentence). BERTSCORE provides precision and recall (and hence F_1):

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{x_i \in x} \max_{\tilde{x}_j \in \tilde{x}} x_i \cdot \tilde{x}_j \qquad P_{\text{BERT}} = \frac{1}{|\tilde{x}|} \sum_{\tilde{x}_j \in \tilde{x}} \max_{x_i \in x} x_i \cdot \tilde{x}_j \tag{12.21}$$

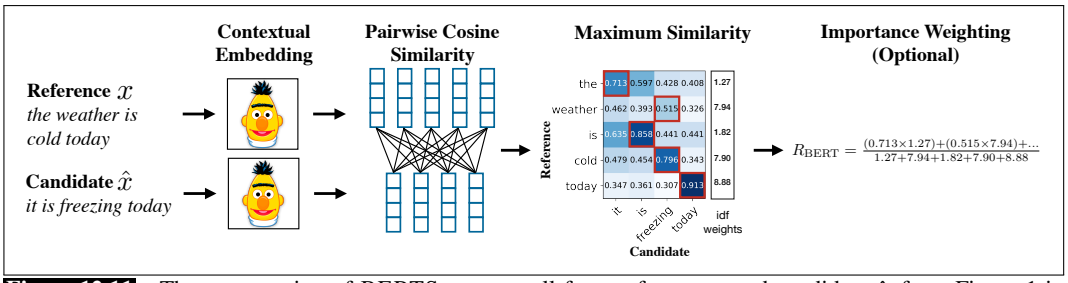


Figure 12.11 The computation of BERTSCORE recall from reference x and candidate $\hat{x}$, from Figure 1 in Zhang et al. (2020). This version shows an extended version of the metric in which tokens are also weighted by their idf values.

12.7 Bias and Ethical Issues

Machine translation raises many of the same ethical issues that we've discussed in earlier chapters. For example, consider MT systems translating from Hungarian (which has the gender neutral pronoun *ő*) or Spanish (which often drops pronouns) into English (in which pronouns are obligatory, and they have grammatical gender). When translating a reference to a person described without specified gender, MT systems often default to male gender (Schiebinger 2014, Prates et al. 2019). And MT systems often assign gender according to culture stereotypes of the sort we saw in Section 5.8. Fig. 12.12 shows examples from Prates et al. (2019), in which Hungarian gender-neutral *ő is a nurse* is translated with *she*, but gender-neutral *ő is a CEO* is translated with *he*. Prates et al. (2019) find that these stereotypes can't completely be accounted for by gender bias in US labor statistics, because the biases are

amplified by MT systems, with pronouns being mapped to male or female gender with a probability higher than if the mapping was based on actual labor employment statistics.

Hungarian (gender neutral) source	English MT output
ő egy ápoló	she is a nurse
ő egy tudós	he is a scientist
ő egy mérnök	he is an engineer
ő egy pék	he is a baker
ő egy tanár	she is a teacher
ő egy esküvőszervező	she is a wedding organizer
ő egy vezérigazgató	he is a CEO

Figure 12.12 When translating from gender-neutral languages like Hungarian into English, current MT systems interpret people from traditionally male-dominated occupations as male, and traditionally female-dominated occupations as female (Prates et al., 2019).

Similarly, a recent challenge set, the WinoMT dataset (Stanovsky et al., 2019) shows that MT systems perform worse when they are asked to translate sentences that describe people with non-stereotypical gender roles, like "The doctor asked the nurse to help her in the operation".

Many ethical questions in MT require further research. One open problem is developing metrics for knowing what our systems don't know. This is because MT systems can be used in urgent situations where human translators may be unavailable or delayed: in medical domains, to help translate when patients and doctors don't speak the same language, or in legal domains, to help judges or lawyers communicate with witnesses or defendants. In order to 'do no harm', systems need ways to assign **confidence** values to candidate translations, so they can abstain from giving incorrect translations that may cause harm.

confidence

12.8 Summary

Machine translation is one of the most widely used applications of NLP, and the encoder-decoder model, first developed for MT is a key tool that has applications throughout NLP.

- Languages have **divergences**, both structural and lexical, that make translation difficult.
- The linguistic field of **typology** investigates some of these differences; languages can be classified by their position along typological dimensions like whether verbs precede their objects.
- **Encoder-decoder** networks are composed of an **encoder** network that takes an input sequence and creates a contextualized representation of it, the **context**. This context representation is then passed to a **decoder** which generates a task-specific output sequence.
- **Cross-attention** allows the transformer decoder to view information from all the hidden states of the encoder.
- Machine translation models are trained on a **parallel corpus**, sometimes called a **bitext**, a text that appears in two (or more) languages.
- **Backtranslation** is a way of making use of monolingual corpora in the target language by running a pilot MT engine backwards to create synthetic bitexts.

- MT is evaluated by measuring a translation's **adequacy** (how well it captures the meaning of the source sentence) and **fluency** (how fluent or natural it is in the target language). Human evaluation is the gold standard, but automatic evaluation metrics like **chrF**, which measure character n-gram overlap with human translations, or more recent metrics based on embedding similarity, are also commonly used.

Historical Notes

MT was proposed seriously by the late 1940s, soon after the birth of the computer (Weaver, 1949/1955). In 1954, the first public demonstration of an MT system prototype (Dostert, 1955) led to great excitement in the press (Hutchins, 1997). The next decade saw a great flowering of ideas, prefiguring most subsequent developments. But this work was ahead of its time—implementations were limited by, for example, the fact that pending the development of disks there was no good way to store dictionary information.

As high-quality MT proved elusive (Bar-Hillel, 1960), there grew a consensus on the need for better evaluation and more basic research in the new fields of formal and computational linguistics. This consensus culminated in the famously critical ALPAC (Automatic Language Processing Advisory Committee) report of 1966 (Pierce et al., 1966) that led in the mid 1960s to a dramatic cut in funding for MT in the US. As MT research lost academic respectability, the Association for Machine Translation and Computational Linguistics dropped MT from its name. Some MT developers, however, persevered, and there were early MT systems like Météo, which translated weather forecasts from English to French (Chandioux, 1976), and industrial systems like Systran.

In the early years, the space of MT architectures spanned three general models. In **direct translation**, the system proceeds word-by-word through the source-language text, translating each word incrementally. Direct translation uses a large bilingual dictionary, each of whose entries is a small program with the job of translating one word. In **transfer** approaches, we first parse the input text and then apply rules to transform the source-language parse into a target language parse. We then generate the target language sentence from the parse tree. In **interlingua** approaches, we analyze the source language text into some abstract meaning representation, called an **interlingua**. We then generate into the target language from this interlingual representation. A common way to visualize these three early approaches was the **Vauquois triangle** shown in Fig. 12.13. The triangle shows the increasing depth of analysis required (on both the analysis and generation end) as we move from the direct approach through transfer approaches to interlingual approaches. In addition, it shows the decreasing amount of transfer knowledge needed as we move up the triangle, from huge amounts of transfer at the direct level (almost all knowledge is transfer knowledge for each word) through transfer (transfer rules only for parse trees or thematic roles) through interlingua (no specific transfer knowledge). We can view the encoder-decoder network as an interlingual approach, with attention acting as an integration of direct and transfer, allowing words or their representations to be directly accessed by the decoder.

Vauquois triangle

Statistical methods began to be applied around 1990, enabled first by the development of large bilingual corpora like the **Hansard** corpus of the proceedings of the

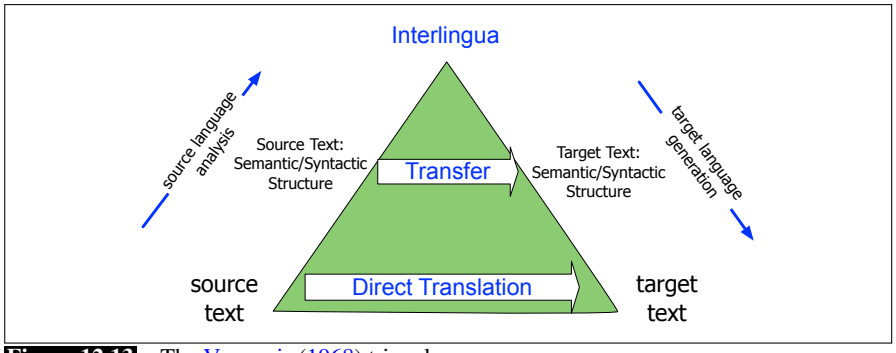


Figure 12.13 The Vauquois (1968) triangle.

Canadian Parliament, which are kept in both French and English, and then by the growth of the web. Early on, a number of researchers showed that it was possible to extract pairs of aligned sentences from bilingual corpora, using words or simple cues like sentence length (Kay and Röscheisen 1988, Gale and Church 1991, Gale and Church 1993, Kay and Röscheisen 1993).

statistical MT **IBM Models** **Candide**

At the same time, the IBM group, drawing directly on the **noisy channel** model for speech recognition, proposed two related paradigms for **statistical MT**. These include the generative algorithms that became known as **IBM Models 1 through 5**, implemented in the **Candide** system. The algorithms (except for the decoder) were published in full detail— encouraged by the US government who had partially funded the work— which gave them a huge impact on the research community (Brown et al. 1990, Brown et al. 1993).

The group also developed a discriminative approach, called MaxEnt (for maximum entropy, an alternative formulation of logistic regression), which allowed many features to be combined discriminatively rather than generatively (Berger et al., 1996), which was further developed by Och and Ney (2002).

phrase-based translation

By the turn of the century, most academic research on machine translation used statistical MT, either in the generative or discriminative mode. An extended version of the generative approach, called **phrase-based translation** was developed, based on inducing translations for phrase-pairs (Och 1998, Marcu and Wong 2002, Koehn et al. (2003), Och and Ney 2004, Deng and Byrne 2005, inter alia).

MERT **Moses**

Once automatic metrics like BLEU were developed (Papineni et al., 2002), the discriminative log linear formulation (Och and Ney, 2004), drawing from the IBM MaxEnt work (Berger et al., 1996), was used to directly optimize evaluation metrics like BLEU in a method known as **Minimum Error Rate Training**, or **MERT** (Och, 2003), also drawing from speech recognition models (Chou et al., 1993). Toolkits like GIZA (Och and Ney, 2003) and **Moses** (Koehn et al. 2006, Zens and Ney 2007) were widely used.

transduction grammars **inversion transduction grammar**

There were also approaches around the turn of the century that were based on syntactic structure (Chapter 18). Models based on **transduction grammars** (also called **synchronous grammars**) assign a parallel syntactic tree structure to a pair of sentences in different languages, with the goal of translating the sentences by applying reordering operations on the trees. From a generative perspective, we can view a transduction grammar as generating pairs of aligned sentences in two languages. Some of the most widely used models included the **inversion transduction grammar** (Wu, 1996) and synchronous context-free grammars (Chiang, 2005),

Neural networks had been applied at various times to various aspects of machine translation; for example Schwenk et al. (2006) showed how to use neural language

models to replace n-gram language models in a Spanish-English system based on IBM Model 4. The modern neural encoder-decoder approach was pioneered by Kalchbrenner and Blunsom (2013), who used a CNN encoder and an RNN decoder, and was first applied to MT by Bahdanau et al. (2015). The transformer encoder-decoder was proposed by Vaswani et al. (2017) (see the History section of Chapter 8).

Research on evaluation of machine translation began quite early. Miller and Beebe-Center (1956) proposed a number of methods drawing on work in psycholinguistics. These included the use of cloze and Shannon tasks to measure intelligibility as well as a metric of edit distance from a human translation, the intuition that underlies all modern overlap-based automatic evaluation metrics. The ALPAC report included an early evaluation study conducted by John Carroll that was extremely influential (Pierce et al., 1966, Appendix 10). Carroll proposed distinct measures for fidelity and intelligibility, and had raters score them subjectively on 9-point scales. Much early evaluation work focuses on automatic word-overlap metrics like BLEU (Papineni et al., 2002), **NIST** (Doddington, 2002), **TER (Translation Error Rate)** (Snover et al., 2006), **Precision and Recall** (Turian et al., 2003), and **METEOR** (Banerjee and Lavie, 2005); character n-gram overlap methods like chrF (Popović, 2015) came later. More recent evaluation work, echoing the ALPAC report, has emphasized the importance of careful statistical methodology and the use of human evaluation (Kocmi et al., 2021; Marie et al., 2021).

The early history of MT is surveyed in Hutchins 1986 and 1997; Nirenburg et al. (2002) collects early readings. See Croft (1990) or Comrie (1989) for introductions to linguistic typology.

Exercises

12.1 Compute by hand the chrF2,2 score for HYP2 on page 277 (the answer should round to .62).

CHAPTER

13 RNNs and LSTMs

Time will explain.
Jane Austen, *Persuasion*

Language is an inherently temporal phenomenon. Spoken language is a sequence of acoustic events over time, and we comprehend and produce both spoken and written language as a sequential input stream. The temporal nature of language is reflected in the metaphors we use; we talk of the *flow of conversations*, *news feeds*, and *twitter streams*, all of which emphasize that language is a sequence that unfolds in time.

This chapter introduces a deep learning architecture, the **recurrent neural network (RNN)**, and RNN variants like LSTMs, that offer a different way of representing time than feedforward and transformer networks. RNNs have a mechanism that deals directly with the sequential nature of language, allowing them to handle the temporal nature of language without the use of arbitrary fixed-sized windows. The recurrent network offers a new way to represent the prior context, in its **recurrent connections**, allowing the model's decision to depend on information from hundreds of words in the past. We'll see how to apply the model to the task of language modeling, to text classification tasks like sentiment analysis, and to sequence modeling tasks like part-of-speech tagging.

13.1 Recurrent Neural Networks

A recurrent neural network (RNN) is any network that contains a cycle within its network connections, meaning that the value of some unit is directly, or indirectly, dependent on its own earlier outputs as an input. While powerful, such networks are difficult to reason about and to train. However, within the general class of recurrent networks there are constrained architectures that have proven to be extremely effective when applied to language. In this section, we consider a class of recurrent networks referred to as **Elman Networks** (Elman, 1990) or **simple recurrent networks**. These networks are useful in their own right and serve as the basis for more complex approaches like the Long Short-Term Memory (LSTM) networks discussed later in this chapter. In this chapter when we use the term RNN we'll be referring to these simpler more constrained networks (although you will often see the term RNN to mean any net with recurrent properties including LSTMs).

Elman Networks

Fig. 13.1 illustrates the structure of an RNN. As with ordinary feedforward networks, an input vector representing the current input, $\mathbf{x}_t$, is multiplied by a weight matrix and then passed through a non-linear activation function to compute the values for a layer of hidden units. This hidden layer is then used to calculate a corresponding output, $\mathbf{y}_t$. In a departure from our earlier window-based approach, sequences are processed by presenting one item at a time to the network. We'll use

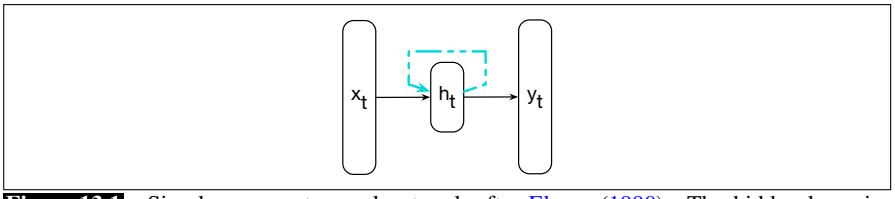


Figure 13.1 Simple recurrent neural network after Elman (1990). The hidden layer includes a recurrent connection as part of its input. That is, the activation value of the hidden layer depends on the current input as well as the activation value of the hidden layer from the previous time step.

subscripts to represent time, thus $\mathbf{x}_t$ will mean the input vector $\mathbf{x}$ at time t. The key difference from a feedforward network lies in the recurrent link shown in the figure with the dashed line. This link augments the input to the computation at the hidden layer with the value of the hidden layer *from the preceding point in time*.

The hidden layer from the previous time step provides a form of memory, or context, that encodes earlier processing and informs the decisions to be made at later points in time. Critically, this approach does not impose a fixed-length limit on this prior context; the context embodied in the previous hidden layer can include information extending back to the beginning of the sequence.

Adding this temporal dimension makes RNNs appear to be more complex than non-recurrent architectures. But in reality, they're not all that different. Given an input vector and the values for the hidden layer from the previous time step, we're still performing the standard feedforward calculation introduced in Chapter 6. To see this, consider Fig. 13.2 which clarifies the nature of the recurrence and how it factors into the computation at the hidden layer. The most significant change lies in the new set of weights, $\mathbf{U}$, that connect the hidden layer from the previous time step to the current hidden layer. These weights determine how the network makes use of past context in calculating the output for the current input. As with the other weights in the network, these connections are trained via backpropagation.

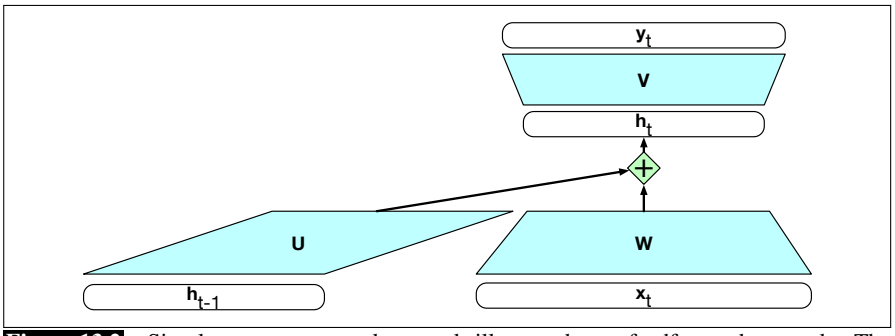


Figure 13.2 Simple recurrent neural network illustrated as a feedforward network. The hidden layer $\mathbf{h}_{t-1}$ from the prior time step is multiplied by weight matrix $\mathbf{U}$ and then added to the feedforward component from the current time step.

13.1.1 Inference in RNNs

Forward inference (mapping a sequence of inputs to a sequence of outputs) in an RNN is nearly identical to what we've already seen with feedforward networks. To compute an output $\mathbf{y}_t$ for an input $\mathbf{x}_t$, we need the activation value for the hidden layer $\mathbf{h}_t$. To calculate this, we multiply the input $\mathbf{x}_t$ with the weight matrix $\mathbf{W}$, and

the hidden layer from the previous time step $\mathbf{h}_{t-1}$ with the weight matrix $\mathbf{U}$. We add these values together and pass them through a suitable activation function, g, to arrive at the activation value for the current hidden layer, $\mathbf{h}_t$. Once we have the values for the hidden layer, we proceed with the usual computation to generate the output vector.

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t) \tag{13.1}$$
$$\mathbf{y}_t = f(\mathbf{V}\mathbf{h}_t) \tag{13.2}$$

Let's refer to the input, hidden and output layer dimensions as d_{in}, d_h, and d_{out} respectively. Given this, our three parameter matrices are: $\mathbf{W} \in \mathbb{R}^{d_h \times d_{in}}$, $\mathbf{U} \in \mathbb{R}^{d_h \times d_h}$, and $\mathbf{V} \in \mathbb{R}^{d_{out} \times d_h}$.

We compute y_t via a softmax computation that gives a probability distribution over the possible output classes.

$$\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t) \tag{13.3}$$

The fact that the computation at time t requires the value of the hidden layer from time $t-1$ mandates an incremental inference algorithm that proceeds from the start of the sequence to the end as illustrated in Fig. 13.3. The sequential nature of simple recurrent networks can also be seen by *unrolling* the network in time as is shown in Fig. 13.4. In this figure, the various layers of units are copied for each time step to illustrate that they will have differing values over time. However, the various weight matrices are shared across time.

function FORWARDRNN($\mathbf{x}$, *network*) **returns** output sequence $\mathbf{y}$

$\mathbf{h}_0 \leftarrow 0$
for $i \leftarrow 1$ **to** LENGTH($\mathbf{x}$) **do**
  $\mathbf{h}_i \leftarrow g(\mathbf{U}\mathbf{h}_{i-1} + \mathbf{W}\mathbf{x}_i)$
  $\mathbf{y}_i \leftarrow f(\mathbf{V}\mathbf{h}_i)$
return $\mathbf{y}$

Figure 13.3 Forward inference in a simple recurrent network. The matrices $\mathbf{U}$, $\mathbf{V}$ and $\mathbf{W}$ are shared across time, while new values for $\mathbf{h}$ and $\mathbf{y}$ are calculated with each time step.

13.1.2 Training

As with feedforward networks, we'll use a training set, a loss function, and backpropagation to obtain the gradients needed to adjust the weights in these recurrent networks. As shown in Fig. 13.2, we now have 3 sets of weights to update: $\mathbf{W}$, the weights from the input layer to the hidden layer, $\mathbf{U}$, the weights from the previous hidden layer to the current hidden layer, and finally $\mathbf{V}$, the weights from the hidden layer to the output layer.

Fig. 13.4 highlights two considerations that we didn't have to worry about with backpropagation in feedforward networks. First, to compute the loss function for the output at time t we need the hidden layer from time $t-1$. Second, the hidden layer at time t influences both the output at time t and the hidden layer at time $t+1$ (and hence the output and loss at $t+1$). It follows from this that to assess the error accruing to $\mathbf{h}_t$, we'll need to know its influence on both the current output *as well as the ones that follow*.

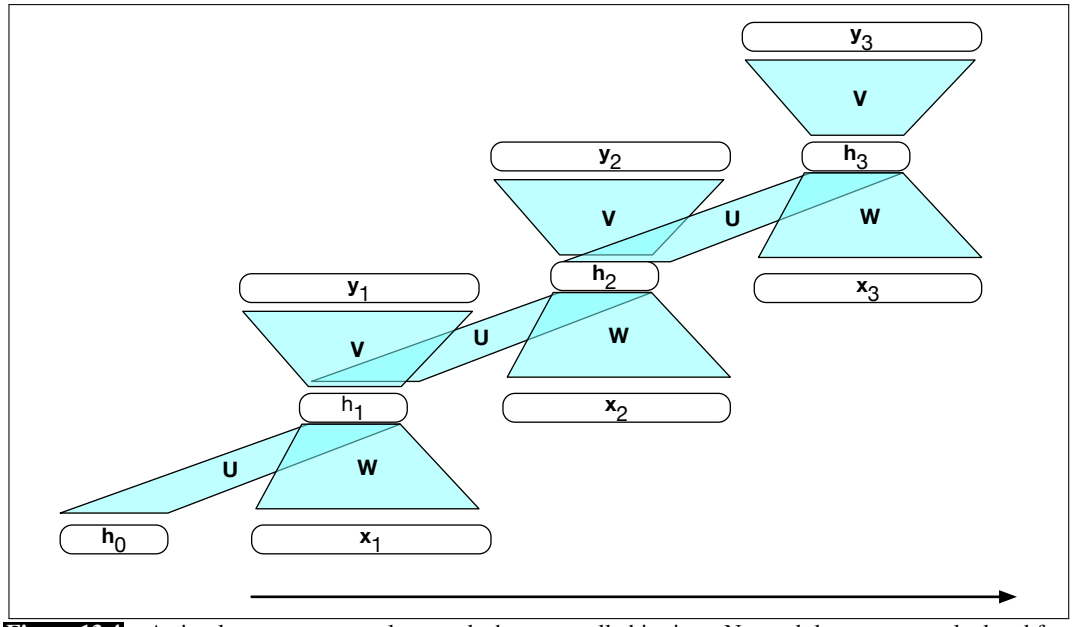


Figure 13.4 A simple recurrent neural network shown unrolled in time. Network layers are recalculated for each time step, while the weights **U**, **V** and **W** are shared across all time steps.

Tailoring the backpropagation algorithm to this situation leads to a two-pass algorithm for training the weights in RNNs. In the first pass, we perform forward inference, computing $\mathbf{h}_t$, $\mathbf{y}_t$, accumulating the loss at each step in time, saving the value of the hidden layer at each step for use at the next time step. In the second pass, we process the sequence in reverse, computing the required gradients as we go, computing and saving the error term for use in the hidden layer for each step backward in time. This general approach is commonly referred to as **backpropagation through time** (Werbos 1974, Rumelhart et al. 1986, Werbos 1990).

backpropagation through time

Fortunately, with modern computational frameworks and adequate computing resources, there is no need for a specialized approach to training RNNs. As illustrated in Fig. 13.4, explicitly unrolling a recurrent network into a feedforward computational graph eliminates any explicit recurrences, allowing the network weights to be trained directly. In such an approach, we provide a template that specifies the basic structure of the network, including all the necessary parameters for the input, output, and hidden layers, the weight matrices, as well as the activation and output functions to be used. Then, when presented with a specific input sequence, we can generate an unrolled feedforward network specific to that input, and use that graph to perform forward inference or training via ordinary backpropagation.

For applications that involve much longer input sequences, such as speech recognition, character-level processing, or streaming continuous inputs, unrolling an entire input sequence may not be feasible. In these cases, we can unroll the input into manageable fixed-length segments and treat each segment as a distinct training item.

13.2 RNNs as Language Models

Let's see how to apply RNNs to the language modeling task. Recall from Chapter 3 that language models predict the next word in a sequence given some preceding context. For example, if the preceding context is *"Thanks for all the"* and we want to know how likely the next word is *"fish"* we would compute:

$$P(fish|Thanks\ for\ all\ the)$$

Language models give us the ability to assign such a conditional probability to every possible next word, giving us a distribution over the entire vocabulary. We can also assign probabilities to entire sequences by combining these conditional probabilities with the chain rule:

$$P(w_{1:n}) = \prod_{i=1}^{n} P(w_i|w_{<i})$$

The n-gram language models of Chapter 3 compute the probability of a word given counts of its occurrence with the $n-1$ prior words. The context is thus of size $n-1$. For the feedforward language models of Chapter 6, the context is the window size.

RNN language models (Mikolov et al., 2010) process the input sequence one word at a time, attempting to predict the next word from the current word and the previous hidden state. RNNs thus don't have the limited context problem that n-gram models have, or the fixed context that feedforward language models have, since the hidden state can in principle represent information about all of the preceding words all the way back to the beginning of the sequence. Fig. 13.5 sketches this difference between a FFN language model and an RNN language model, showing that the RNN language model uses h_{t-1}, the hidden state from the previous time step, as a representation of the past context.

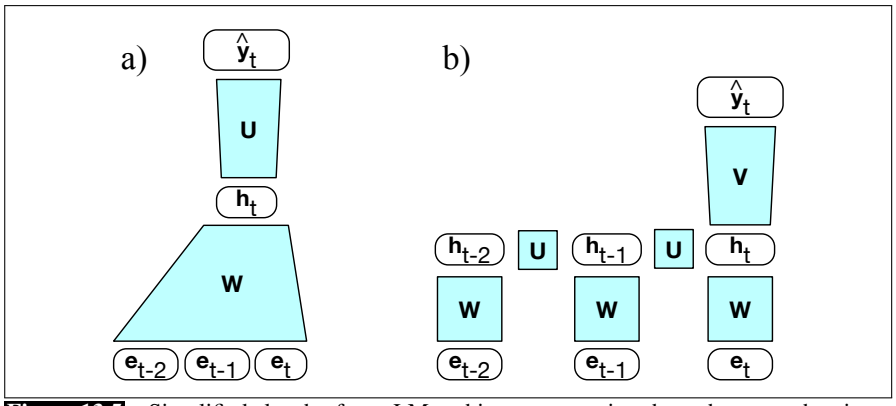


Figure 13.5 Simplified sketch of two LM architectures moving through a text, showing a schematic context of three tokens: (a) a feedforward neural language model which has a fixed context input to the weight matrix **W**, (b) an RNN language model, in which the hidden state $\mathbf{h}_{t-1}$ summarizes the prior context.

13.2.1 Forward Inference in an RNN language model

Forward inference in a recurrent language model proceeds exactly as described in Section 13.1.1. The input sequence $\mathbf{X} = [\mathbf{x}_1;...;\mathbf{x}_t;...;\mathbf{x}_N]$ consists of a series of

words each represented as a one-hot vector of size $|V| \times 1$, and the output prediction, $\hat{\mathbf{y}}$, is a vector representing a probability distribution over the vocabulary. At each step, the model uses the word embedding matrix $\mathbf{E}$ to retrieve the embedding for the current word, multiples it by the weight matrix $\mathbf{W}$, and then adds it to the hidden layer from the previous step (weighted by weight matrix $\mathbf{U}$) to compute a new hidden layer. This hidden layer is then used to generate an output layer which is passed through a softmax layer to generate a probability distribution over the entire vocabulary. That is, at time t:

$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t \tag{13.4}$$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t) \tag{13.5}$$

$$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t) \tag{13.6}$$

When we do language modeling with RNNs (and we'll see this again in Chapter 8 with transformers), it's convenient to make the assumption that the embedding dimension d_e and the hidden dimension d_h are the same. So we'll just call both of these the **model dimension** d. So the embedding matrix $\mathbf{E}$ is of shape $[d \times |V|]$, and $\mathbf{x_t}$ is a one-hot vector of shape $[|V| \times 1]$. The product $\mathbf{e}_t$ is thus of shape $[d \times 1]$. $\mathbf{W}$ and $\mathbf{U}$ are of shape $[d \times d]$, so $\mathbf{h}_t$ is also of shape $[d \times 1]$. $\mathbf{V}$ is of shape $[|V| \times d]$, so the result of $\mathbf{V}\mathbf{h}$ is a vector of shape $[|V| \times 1]$. This vector can be thought of as a set of scores over the vocabulary given the evidence provided in $\mathbf{h}$. Passing these scores through the softmax normalizes the scores into a probability distribution. The probability that a particular word k in the vocabulary is the next word is represented by $\hat{\mathbf{y}}_t[k]$, the kth component of $\hat{\mathbf{y}}_t$:

$$P(w_{t+1} = k | w_1, \ldots, w_t) = \hat{\mathbf{y}}_t[k] \tag{13.7}$$

The probability of an entire sequence is just the product of the probabilities of each item in the sequence, where we'll use $\hat{\mathbf{y}}_i[w_i]$ to mean the probability of the true word w_i at time step i.

$$P(w_{1:n}) = \prod_{i=1}^{n} P(w_i | w_{1:i-1}) \tag{13.8}$$

$$= \prod_{i=1}^{n} \hat{\mathbf{y}}_i[w_i] \tag{13.9}$$

13.2.2 Training an RNN language model

self-supervision

To train an RNN as a language model, we use the same **self-supervision** (or **self-training**) algorithm we saw in Section 7.5: we take a corpus of text as training material and at each time step t ask the model to predict the next word. We call such a model self-supervised because we don't have to add any special gold labels to the data; the natural sequence of words is its own supervision! We simply train the model to minimize the error in predicting the true next word in the training sequence, using cross-entropy as the loss function. Recall that the cross-entropy loss measures the difference between a predicted probability distribution and the correct distribution.

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\sum_{w \in V} \mathbf{y}_t[w] \log \hat{\mathbf{y}}_t[w] \tag{13.10}$$

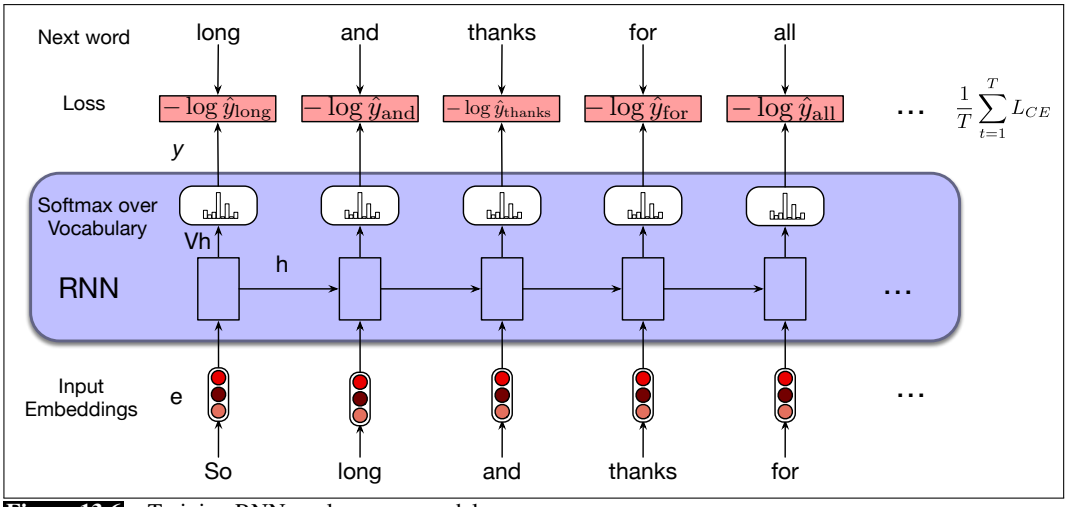


Figure 13.6 Training RNNs as language models.

In the case of language modeling, the correct distribution $\mathbf{y}_t$ comes from knowing the next word. This is represented as a one-hot vector corresponding to the vocabulary where the entry for the actual next word is 1, and all the other entries are 0. Thus, the cross-entropy loss for language modeling is determined by the probability the model assigns to the correct next word. So at time t the CE loss is the negative log probability the model assigns to the next word in the training sequence.

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}] \tag{13.11}$$

Thus at each word position t of the input, the model takes as input the correct word w_t together with h_{t-1}, encoding information from the preceding $w_{1:t-1}$, and uses them to compute a probability distribution over possible next words so as to compute the model's loss for the next token w_{t+1}. Then we move to the next word, we ignore what the model predicted for the next word and instead use the correct word w_{t+1} along with the prior history encoded to estimate the probability of token w_{t+2}. This idea that we always give the model the correct history sequence to predict the next word (rather than feeding the model its best case from the previous time step) is called **teacher forcing**.

teacher forcing

The weights in the network are adjusted to minimize the average CE loss over the training sequence via gradient descent. Fig. 13.6 illustrates this training regimen.

13.2.3 Weight Tying

Careful readers may have noticed that the input embedding matrix $\mathbf{E}$ and the final layer matrix $\mathbf{V}$, which feeds the output softmax, are quite similar.

The columns of $\mathbf{E}$ represent the word embeddings for each word in the vocabulary learned during the training process with the goal that words that have similar meaning and function will have similar embeddings. And, since when we use RNNs for language modeling we make the assumption that the embedding dimension and the hidden dimension are the same (= the model dimension d), the embedding matrix $\mathbf{E}$ has shape $[d \times |V|]$. And the final layer matrix $\mathbf{V}$ provides a way to score the likelihood of each word in the vocabulary given the evidence present in the final hidden layer of the network through the calculation of $\mathbf{Vh}$. $\mathbf{V}$ is of shape $[|V| \times d]$. That is, the rows of $\mathbf{V}$ are shaped like a transpose of $\mathbf{E}$, meaning that $\mathbf{V}$ provides a

second set of learned word embeddings.

Instead of having two sets of embedding matrices, language models use a single embedding matrix, which appears at both the input and softmax layers. That is, we dispense with $\mathbf{V}$ and use $\mathbf{E}$ at the start of the computation and $\mathbf{E}^\intercal$ (because the shape of $\mathbf{V}$ is the transpose of $\mathbf{E}$ at the end. Using the same matrix (transposed) in two places is called **weight tying**.[1] The weight-tied equations for an RNN language model then become:

weight tying

$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t \tag{13.12}$$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t) \tag{13.13}$$

$$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{E}^\intercal \mathbf{h}_t) \tag{13.14}$$

In addition to providing improved model perplexity, this approach significantly reduces the number of parameters required for the model.

13.3 RNNs for other NLP tasks

Now that we've seen the basic RNN architecture, let's consider how to apply it to three types of NLP tasks: *sequence classification* tasks like sentiment analysis and topic classification, *sequence labeling* tasks like part-of-speech tagging, and *text generation* tasks, including with a new architecture called the **encoder-decoder**.

13.3.1 Sequence Labeling

In sequence labeling, the network's task is to assign a label chosen from a small fixed set of labels to each element of a sequence. One classic sequence labeling tasks is part-of-speech (POS) tagging (assigning grammatical tags like NOUN and VERB to each word in a sentence). We'll discuss part-of-speech tagging in detail in Chapter 17, but let's give a motivating example here. In an RNN approach to sequence labeling, inputs are word embeddings and the outputs are tag probabilities generated by a softmax layer over the given tagset, as illustrated in Fig. 13.7.

In this figure, the inputs at each time step are pretrained word embeddings corresponding to the input tokens. The RNN block is an abstraction that represents an unrolled simple recurrent network consisting of an input layer, hidden layer, and output layer at each time step, as well as the shared $\mathbf{U}$, $\mathbf{V}$ and $\mathbf{W}$ weight matrices that comprise the network. The outputs of the network at each time step represent the distribution over the POS tagset generated by a softmax layer.

To generate a sequence of tags for a given input, we run forward inference over the input sequence and select the most likely tag from the softmax at each step. Since we're using a softmax layer to generate the probability distribution over the output tagset at each time step, we will again employ the cross-entropy loss during training.

13.3.2 RNNs for Sequence Classification

Another use of RNNs is to classify entire sequences rather than the tokens within them. This is the set of tasks commonly called **text classification**, like sentiment analysis or spam detection, in which we classify a text into two or three classes (like positive or negative), as well as classification tasks with a large number of

[1] We also do this for transformers (Chapter 8) where it's common to call $\mathbf{E}^\intercal$ the **unembedding matrix**.

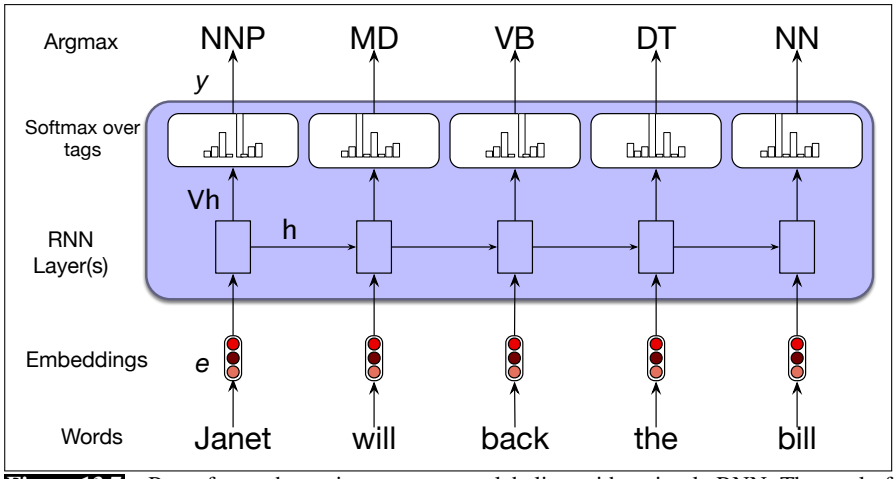


Figure 13.7 Part-of-speech tagging as sequence labeling with a simple RNN. The goal of part-of-speech (POS) tagging is to assign a grammatical label to each word in a sentence, drawn from a predefined set of tags. (The tags for this sentence include NNP (proper noun), MD (modal verb) and others; we'll give a complete description of the task of part-of-speech tagging in Chapter 17.) Pre-trained word embeddings serve as inputs and a softmax layer provides a probability distribution over the part-of-speech tags as output at each time step.

categories, like document-level topic classification, or message routing for customer service applications.

To apply RNNs in this setting, we pass the text to be classified through the RNN a word at a time generating a new hidden layer representation at each time step. We can then take the hidden layer for the last token of the text, $\mathbf{h}_n$, to constitute a compressed representation of the entire sequence. We can pass this representation $\mathbf{h}_n$ to a feedforward network that chooses a class via a softmax over the possible classes. Fig. 13.8 illustrates this approach.

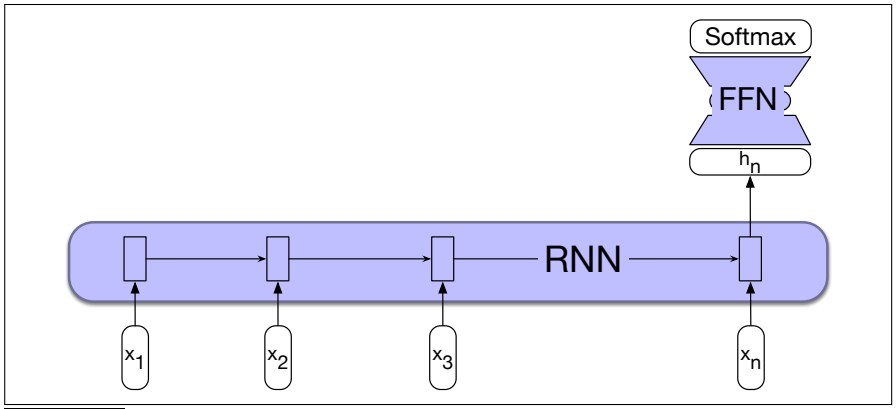


Figure 13.8 Sequence classification using a simple RNN combined with a feedforward network. The final hidden state from the RNN is used as the input to a feedforward network that performs the classification.

Note that in this approach we don't need intermediate outputs for the words in the sequence preceding the last element. Therefore, there are no loss terms associated with those elements. Instead, the loss function used to train the weights in the network is based entirely on the final text classification task. The output from the softmax output from the feedforward classifier together with a cross-entropy loss